



Segurança para Desenvolvedores Web

USANDO JAVASCRIPT, HTML E CSS

Segurança para Desenvolvedores Web

USANDO JAVASCRIPT, HTML E CSS

John Paul Mueller

O'REILLY®
Novatec
São Paulo | 2019

Authorized Portuguese translation of the English edition of Security for Web Developers, ISBN 9781491928646 © 2015 John Mueller. This translation is published and sold by permission of O'Reilly Media, Inc., which owns or controls all rights to publish and sell the same.

Tradução em português autorizada da edição em inglês da obra Security for Web Developers, ISBN 9781491928646 © 2015 John Mueller. Esta tradução é publicada e vendida com a permissão da O'Reilly Media, Inc., detentora de todos os direitos para publicação e venda desta obra.

© Novatec Editora Ltda. 2016.

Todos os direitos reservados e protegidos pela Lei 9.610 de 19/02/1998. É proibida a reprodução desta obra, mesmo parcial, por qualquer processo, sem prévia autorização, por escrito, do autor e da Editora.

Editor: Rubens Prates

Tradução: Lúcia Kinoshita

Revisão gramatical: Smirna Cavaleiro

Assistente editorial: Priscila A. Yoshimatsu

Editoração eletrônica: Carolina Kuwabata

ISBN: 978-85-7522-767-1

Histórico de edições impressas:

Março/2016 Primeira edição

Novatec Editora Ltda.

Rua Luís Antônio dos Santos 110

02460-000 – São Paulo, SP – Brasil

Tel.: +55 11 2959-6529

Email: novatec@novatec.com.br

Site: www.novatec.com.br

Twitter: twitter.com/novateceditora

Facebook: facebook.com/novatec

LinkedIn: linkedin.com/in/novatec

Este livro é dedicado a todos os profissionais médicos que me ajudaram a recuperar minha saúde – que ouviram todas as minhas queixas e encontraram maneiras de tratá-las. Sim, eu precisei seguir suas recomendações, mas foram eles que as ofereceram a mim. Boa saúde é uma dádiva excepcional.

Sumário

[Prefácio](#)

[Parte I ■ Desenvolvendo um plano de segurança](#)

[Capítulo 1 ■ Definindo o ambiente da aplicação](#)

[Especificando as ameaças às aplicações web](#)

[Entendendo a Software Security Assurance \(SSA\)](#)

[Considerando o OSSAP](#)

[Definindo os requisitos de SSA](#)

[Classificando dados e recursos](#)

[Fazendo a análise necessária](#)

[Mergulhando em questões específicas de linguagem](#)

[Definindo os principais problemas de HTML](#)

[Definindo os principais problemas de CSS](#)

[Definindo os principais problemas de JavaScript](#)

[Considerando o essencial para defesa nos endpoints](#)

[Evitando falhas de segurança](#)

[Detectando falhas de segurança](#)

[Corrigindo um software com problemas](#)

[Lidando com armazenamento em nuvem](#)

[Usando códigos e recursos externos](#)

[Definindo o uso de bibliotecas](#)

[Definindo o uso de APIs](#)

[Definindo o uso de microsserviços](#)

[Acessando dados externos](#)

[Permitindo acesso a outros](#)

[Capítulo 2 ■ Acolhendo as necessidades e expectativas dos usuários](#)

[Desenvolvendo uma visão de usuário à aplicação](#)

Considerando os problemas de BYOD (Bring Your Own Device)

Entendendo a segurança de aplicações baseadas em web

Considerando problemas de aplicações nativas

Usando navegadores customizados

Verificando problemas de compatibilidade de código

Tratando atualizações quase contínuas de dispositivos

Planejando alternativas a senhas

Trabalhando com frases-senha

Usando soluções biométricas

Contando com cartões-chave

Contando com chaves USB

Implementando uma estratégia de token

Focando nas expectativas dos usuários

Deixando a aplicação fácil de usar

Deixando a aplicação rápida

Criando um ambiente confiável

Mantendo a segurança em perspectiva

Capítulo 3 ■ Obtendo assistência de terceiros

Identificando soluções de terceiros para segurança

Considerando soluções para segurança em nuvem

Entendendo os repositórios de dados

Lidando com problemas de compartilhamento de arquivos

Considerando o armazenamento em nuvem

Escolhendo um tipo de produto

Trabalhando com bibliotecas

Acessando APIs

Considerando os microserviços

Parte II ■ Aplicando práticas de programação bem-sucedidas

Capítulo 4 ■ Desenvolvendo interfaces bem-sucedidas

Acessando a interface de usuário

Criando uma interface clara

Deixando as interfaces flexíveis

- [Oferecendo auxílio ao usuário](#)
- [Definindo os problemas de acessibilidade](#)
- [Oferecendo opções controladas](#)
- [Escolhendo um nível de solução para interface de usuário](#)
- [Implementando controles HTML padrão](#)
- [Trabalhando com controles CSS](#)
- [Criando controles com JavaScript](#)
- [Validando a entrada](#)
 - [Permitindo apenas entradas específicas](#)
 - [Procurando dados de entrada furtivos](#)
 - [Solicitando novas entradas](#)
 - [Usando validação tanto do lado cliente quanto do lado servidor](#)
- [Esperando o inesperado](#)

[Capítulo 5 ■ Implementando um código confiável](#)

- [Diferenciando confiabilidade e segurança](#)
 - [Definindo os papéis da confiabilidade e da segurança](#)
 - [Evitando brechas de segurança em códigos confiáveis](#)
 - [Focando nas funcionalidades da aplicação](#)
- [Desenvolvendo protocolos de equipe](#)
- [Criando um ciclo de feedback de lições aprendidas](#)
- [Considerando os problemas de soluções prontas](#)
 - [Lidando com bibliotecas externas](#)
 - [Lidando com APIs externas](#)
 - [Trabalhando com frameworks](#)
 - [Acionando os microsserviços](#)

[Capítulo 6 ■ Incorporando bibliotecas](#)

- [Considerando os usos de bibliotecas](#)
 - [Melhorando o CSS com bibliotecas](#)
 - [Interagindo com HTML usando bibliotecas](#)
 - [Estendendo o JavaScript com bibliotecas](#)
- [Diferenciando bibliotecas armazenadas internamente de bibliotecas armazenadas externamente](#)
- [Definindo as ameaças de segurança causadas por bibliotecas](#)
 - [Habilitando o modo estrito](#)

- [Desenvolvendo uma CSP \(Content Security Policy\)](#)
- [Incorporando bibliotecas de forma segura](#)
- [Fazendo uma pesquisa completa sobre a biblioteca](#)
- [Definindo exatamente quais são os usos da biblioteca](#)
- [Mantendo o tamanho da biblioteca reduzido e o conteúdo focado](#)
- [Executando os testes necessários](#)
- [Diferenciando bibliotecas de frameworks](#)

[Capítulo 7 ■ Usando APIs com cuidado](#)

- [Diferenciando APIs de bibliotecas](#)
- [Considerando as diferenças em popularidade](#)
- [Definindo as diferenças quanto ao uso](#)
- [Estendendo JavaScript usando APIs](#)
- [Localizando as APIs apropriadas](#)
- [Criando um exemplo simples](#)
- [Definindo as ameaças de segurança causadas por APIs](#)
- [Arruinando seu bom nome com o MailPoet](#)
- [Desenvolvendo uma imagem do snapping](#)
- [Perdendo seu dispositivo com o Find My iPhone](#)
- [Deixando suas informações mais importantes vazarem com o Heartbleed](#)
- [Sofrendo com o Shellshock](#)
- [Acessando APIs de forma segura a partir de JavaScript](#)
- [Verificando a segurança da API](#)
- [Testando entradas e saídas](#)
- [Mantendo os dados localizados e seguros](#)
- [Programando de forma defensiva](#)

[Capítulo 8 ■ Considerando o uso de microserviços](#)

- [Definindo microserviços](#)
- [Especificando as características dos microserviços](#)
- [Diferenciando microserviços de bibliotecas](#)
- [Diferenciando microserviços de APIs](#)
- [Considerando as políticas dos microserviços](#)
- [Fazendo chamadas a microserviços usando JavaScript](#)
- [Entendendo o papel do REST na comunicação](#)
- [Transmitindo dados usando JSON](#)

[Definindo as ameaças de segurança causadas por microsserviços](#)

[Falta de consistência](#)

[Considerando o papel da máquina virtual](#)

[Usando JSON para transferências de dados](#)

[Definindo a TLS](#)

[Criando paths alternativos para microsserviços](#)

[Parte III ■ Criando estratégias de testes úteis e eficientes](#)

[Capítulo 9 ■ Pensando como um hacker](#)

[Definindo a necessidade de scans de segurança web](#)

[Criando um sistema de testes](#)

[Obtendo o treinamento necessário](#)

[Criando o ambiente correto](#)

[Usando máquinas virtuais](#)

[Obtendo as ferramentas](#)

[Configurando o sistema](#)

[Restaurando o sistema](#)

[Definindo as fontes mais comuns de brechas de segurança](#)

[Evitando ataques de injeção de SQL](#)

[Entendendo o Cross-Site Scripting](#)

[Enfrentando problemas de negação de serviço \(denial-of-service\)](#)

[Aproveitando-se da localização previsível de recursos](#)

[Evitando a divulgação de informações que não deveriam ser reveladas](#)

[Testando em um ambiente BYOD](#)

[Configurando uma zona de acesso remoto](#)

[Verificando hacks entre aplicações](#)

[Lidando com equipamentos e softwares realmente antigos](#)

[Contando com testes de usuário](#)

[Deixando o usuário correr solto](#)

[Desenvolvendo passos reproduzíveis](#)

[Dando voz ao usuário](#)

[Usando pentesters externos](#)

[Considerando a empresa de testes de invasão](#)

[Gerenciando o projeto](#)

[Incluindo o essencial](#)

[Obtendo o relatório](#)

[Capítulo 10 ■ Criando uma zona de segurança para APIs](#)

[Entendendo o conceito de zona de segurança de API](#)

[Definindo a necessidade de uma zona de segurança de API](#)

[Garantindo que sua API funcione](#)

[Possibilitando um desenvolvimento rápido](#)

[Garantindo a melhor integração possível](#)

[Verificando se a API se comporta quando houver carga](#)

[Mantendo a API segura contra hackers](#)

[Desenvolvendo com uma API em sandbox](#)

[Usando uma solução de prateleira](#)

[Usando sandboxes de outros fornecedores](#)

[Considerando ambientes virtuais](#)

[Definindo o ambiente virtual](#)

[Diferenciando ambientes virtuais de sandboxing](#)

[Implementando a virtualização](#)

[Contando com a virtualização de aplicações](#)

[Capítulo 11 ■ Verificando se há brechas de segurança em bibliotecas e APIs](#)

[Criando um plano de testes](#)

[Considerando metas e objetivos](#)

[Testando bibliotecas internas](#)

[Testando APIs internas](#)

[Testando bibliotecas externas](#)

[Testando APIs externas](#)

[Estendendo os testes aos microserviços](#)

[Testando bibliotecas e APIs individualmente](#)

[Criando um test harness para bibliotecas](#)

[Criando scripts de teste para APIs](#)

[Estendendo as estratégias de teste aos microserviços](#)

[Desenvolvendo estratégias de resposta](#)

[Realizando testes de integração](#)

[Testando problemas específicos de linguagem](#)

[Projetando testes para problemas de HTML](#)

[Projetando testes para problemas de CSS](#)

[Projetando testes para problemas de JavaScript](#)

[Capítulo 12 ■ Usando empresas terceirizadas de testes](#)

[Localizando serviços de testes oferecidos por terceiros](#)

[Definindo os motivos para contratar um terceiro](#)

[Considerando a variedade de possíveis serviços de testes](#)

[Garantindo a legitimidade do terceiro](#)

[Entrevistando o terceiro](#)

[Realizando testes em uma instalação para testes](#)

[Criando um plano de testes](#)

[Especificando as metas da empresa terceirizada nos testes](#)

[Gerando um plano de testes por escrito](#)

[Listando as saídas dos testes e os requisitos de relatório](#)

[Considerando os requisitos de testes](#)

[Implementando um plano de testes](#)

[Determinando a participação da empresa nos testes](#)

[Iniciando o processo de testes](#)

[Fazendo a monitoração necessária nos testes](#)

[Tratando problemas inesperados nos testes](#)

[Usando os relatórios resultantes](#)

[Discutindo o relatório gerado com o terceiro](#)

[Apresentando o relatório à empresa](#)

[Atuando sobre as recomendações dos testes](#)

[Parte IV ■ Implementando um ciclo de manutenção](#)

[Capítulo 13 ■ Definindo claramente os ciclos de upgrade](#)

[Desenvolvendo um plano detalhado de ciclo de upgrade](#)

[Procurando upgrades](#)

[Determinando os requisitos do upgrade](#)

[Definindo a criticidade do upgrade](#)

[Verificando upgrades para ver se há problemas](#)

[Criando cenários de testes](#)

[Implementando as mudanças](#)

[Criando um cronograma para teste de upgrade](#)

Executando os pré-testes necessários
Executando os testes de integração necessários
Passando um upgrade para o ambiente de produção

Capítulo 14 ■ Considerando as opções de update

Diferenciando entre upgrades e updates
Determinando quando o update deve ser feito
Trabalhando com updates de biblioteca
Trabalhando com updates de API e microserviços
Aceitando updates automáticos
Fazendo update em pacotes de idiomas
Criando uma lista de idiomas aceitos
Contando com especialistas de confiança em idiomas
Conferindo se os prompts específicos de um idioma funcionam na aplicação
Garantindo que os dados apareçam no formato correto
Definindo os requisitos especiais para testes de suporte a idiomas
Efetuando updates de emergência
Evitando as emergências sempre que for possível
Criando uma equipe para respostas rápidas
Executando testes simplificados
Criando um cronograma permanente de updates
Criando um cronograma para testes de update

Capítulo 15 ■ Considerando a necessidade de relatórios

Usando relatórios para fazer alterações
Evitando relatórios inúteis
Controlando o instante e a frequência de relatórios de upgrades e updates
Usando relatórios gerados automaticamente
Usando relatórios personalizados
Criando relatórios consistentes
Usando relatórios para executar tarefas específicas da aplicação
Criando relatórios internos
Determinando quais fontes de dados serão usadas
Especificando usos para os relatórios
Contando com relatórios gerados externamente

- [Obtendo relatórios completos de terceiros](#)
- [Desenvolvendo relatórios a partir de dados brutos](#)
- [Mantendo os dados internos seguros](#)
- [Possibilitando feedback de usuários](#)
- [Obtendo feedback de usuários](#)
- [Determinando a usabilidade do feedback de usuário](#)

[Parte V ■ Localizando recursos de segurança](#)

[Capítulo 16 ■ Monitorando as ameaças de segurança atuais](#)

- [Desenvolvendo fontes para obter informações sobre ameaças de segurança](#)
- [Lendo artigos relacionados à segurança escritos por especialistas](#)
- [Verificando sites de segurança](#)
- [Obtendo informações de consultores](#)
- [Evitando sobrecarga de informações](#)
- [Criando um plano para upgrades baseado em ameaças](#)
- [Previendo situações que não exigem nenhuma ação](#)
- [Decidir-se entre um upgrade e um update](#)
- [Definindo um plano de upgrade](#)
- [Criando um plano para updates baseado em ameaças](#)
- [Verificar se os updates resolvem as ameaças](#)
- [Determinando se a ameaça é uma emergência](#)
- [Definindo um plano de update](#)
- [Solicitando updates de terceiros](#)

[Capítulo 17 ■ Proporcionando o treinamento necessário](#)

- [Criando um plano de treinamento interno para segurança](#)
- [Definindo os treinamentos necessários](#)
- [Definindo metas razoáveis](#)
- [Usando instrutores internos](#)
- [Monitorando os resultados](#)
- [Obtendo treinamento de terceiros para os desenvolvedores](#)
- [Especificando os requisitos de treinamento](#)
- [Contratando um instrutor terceirizado para sua empresa](#)
- [Usando cursos online](#)

[Contando com centros de treinamento](#)

[Usando faculdades e universidades locais](#)

[Garantindo que os usuários estejam cientes da segurança](#)

[Criando treinamentos específicos para segurança](#)

[Combinando treinamento com orientações por escrito](#)

[Criando e usando lembretes alternativos para segurança](#)

[Verificando se os treinamentos são eficientes](#)

[Sobre o autor](#)

[Colofão](#)

Prefácio

Ransomware¹, vírus, ataques DDoS (Distributed Denial-of-Service, ou Negação de serviço distribuído), ataques man-in-the-middle, brechas de segurança e problemas semelhantes trazem à mente aquilo que qualquer pessoa envolvida com gerenciamento de aplicações mais detesta: pesadelos. Chega-se ao ponto em que as pessoas que façam qualquer trabalho que afete a segurança de uma aplicação ou de seus dados associados passam a ter medo de tudo – elas se tornam absurdamente conservadoras. Você não quer realmente ter a responsabilidade de garantir a segurança de uma aplicação – isso simplesmente faz parte do pacote.

Para aumentar seu fardo, resultados desastrosos de qualquer tipo de erro poderiam assombrar você pelo resto de sua vida. De modo diferente da maioria dos erros, é provável que você não possa varrer esse para baixo do tapete, pois ele aparecerá na imprensa especializada, onde todos poderão ler. Mesmo que seu nome não se torne sinônimo de falha de segurança, um problema de segurança sempre apresenta ramificações: questões legais, perda de emprego e assim por diante. Então, como você pode lidar com esse problema?

Enterrar sua cabeça na areia não parece ser uma solução – pelo menos, não por muito tempo. *Segurança para desenvolvedores web* não tem a intenção de apresentar todas as ameaças que existem por aí nem de resolver todos os problemas de segurança que você encontrará. Em vez disso, o livro apresenta diretrizes e ferramentas necessárias para resolver qualquer problema de segurança por conta própria – para que você possa ver uma luz no fim do túnel que não tenha a ver com um trem. O livro realmente diz respeito a ser capaz de ter uma boa noite de sono, sabendo que você tem o que é necessário para fazer o trabalho da maneira certa.

Sobre este livro

Segurança para desenvolvedores web oferece a você os recursos necessários para trabalhar com os problemas de segurança de aplicações web. Sim, você verá também algumas informações sobre plataformas, pois os navegadores executam em plataformas específicas. Além disso, você poderá ver alguns desses problemas de segurança quando trabalhar com aplicações desktop, simplesmente porque a área de segurança ocupa os dois domínios de aplicações. Entretanto, este livro tem como foco a segurança de aplicações web, independentemente de onde essas aplicações são executadas. Você pode encontrar informações sobre tudo, desde os mais novos smartphones até um desktop mais antigo e tudo que estiver entre eles. O livro separa as informações nas partes a seguir; cada uma dá um passo adiante no caminho para a criação de um plano melhor de segurança para sua empresa:

Parte I

Nada funciona bem sem planejamento. No entanto, alguns dos piores desastres no mercado de informática ocorreram por causa de um plano ruim, não pela falta de um. Essa parte do livro ajudará você a criar um bom plano de segurança para sua empresa – um plano que considere todos os dispositivos mais recentes e as necessidades dos usuários como parte do quadro geral. Essa parte também discute a necessidade de suporte de terceiros, pois vamos encarar este fato: o ambiente complexo da segurança realmente dificulta criar um ambiente seguro por conta própria. Os materiais ajudarão você a encontrar o tipo certo de ajuda de terceiros e a garantir que você realmente obtenha o que precisa deles.

Parte II

Criar aplicações hoje em dia significa contar com códigos de terceiros que se encontram em bibliotecas, APIs e microsserviços. Essa parte do livro ajuda você a considerar as questões ligadas à programação. Você não verá muitos bits e bytes, mas encontrará dicas úteis para incorporar esses elementos com sucesso em sua aplicação. Essa parte ajuda você a gerenciar suas aplicações, em vez de deixar que elas o gerenciem.

Parte III

Existem várias maneiras de testar aplicações e diversos meios de fazê-lo.

Por exemplo, você pode criar suas próprias suítes de teste ou pode contar com uma suíte criada por outra pessoa. Um terceiro pode realizar os testes para você. Talvez você queira saber qual a melhor maneira de combinar diferentes estratégias para garantir que a aplicação toda será incluída. Essa parte do livro responde a todas as suas perguntas sobre estratégias modernas de testes e detalha o que você pode fazer para que seus esforços sejam mais eficientes.

Parte IV

Em algum momento, sua aplicação estará em ambiente de produção, executando suavemente. Algumas aplicações continuam a executar assim durante anos, sem receber a manutenção apropriada. Infelizmente, um desenvolvimento moderno de aplicações implica realizar atualizações de forma regular, pois os hackers constantemente criam novas estratégias para acessar seu sistema. Para aumentar a confusão, é preciso fazer todas aquelas atualizações em bibliotecas de terceiros, APIs e microserviços que você usa. Essa parte do livro apresenta um mapa para o labirinto de atualizações e faz com que seja possível manter tudo funcionando em perfeita ordem, sem que você perca a cabeça antes.

Parte V

As ameaças à segurança evoluem todos os dias; isso quer dizer que você precisa de alguns meios para se manter atualizado. Um método consiste em monitorar as ameaças à segurança. É claro que, se monitorar todas as ameaças, você não fará mais nada. Essa parte do livro descreve técnicas que podem ser usadas para evitar a sobrecarga de informações. A segunda técnica é obter treinamento adicional. De fato, toda a sua empresa precisa de algum tipo de treinamento para se manter a par dos problemas atuais de segurança e das técnicas para lidar com eles. Aqui também são discutidos os requisitos de treinamento que qualquer empresa pode usar – mesmo que seu negócio seja de uma só pessoa ou seja uma startup recente.

O que você deve saber

Os leitores deste livro podem ter qualquer cargo, por exemplo, web designer,

desenvolvedor de frontend, designer de UI, designer de UX, designer de interação, diretor de arte, estrategista de conteúdo, DevOps, gerente de produto, especialista em SEO, cientista de dados, engenheiro de software ou analista de sistemas. O que todos têm em comum é a necessidade de criar aplicações web de algum tipo com as quais os usuários possam interagir de forma segura de modo significativo. Todos vocês são profissionais que já criaram aplicações web antes. O que talvez vocês realmente precisem é aperfeiçoar suas habilidades de segurança, considerando o novo ambiente de grandes invasões a aplicações por meio de métodos não tradicionais, por exemplo, contaminando APIs e bibliotecas de terceiros.

Segurança para desenvolvedores web oferece um tratamento para segurança do início ao fim, mas o livro não guiará você pela mão. A obra parte do pressuposto que você quer as informações mais recentes sobre como vencer as ameaças à segurança em vários níveis, incluindo um texto sobre as categorias exatas em que essas ameaças se enquadram e como os hackers as usam para passar por cima de suas medidas de segurança.

O livro inclui alguns exemplos de programação relacionados à segurança. Para usar esses exemplos, você precisa ter um bom conhecimento de técnicas de programação com CSS3, HTML5 e JavaScript. No entanto, se não tiver essas habilidades, você poderá ignorar os exemplos de programação e, apesar disso, ainda terá uma quantidade considerável de informações do livro. Os exemplos de programação oferecem detalhes nos quais somente os programadores estarão realmente interessados.

Afora as habilidades de programação, é mais importante que você já tenha algum nível de treinamento em segurança. Por exemplo, se você não tem a mínima ideia do que seja um ataque man-in-the-middle, leia um livro mais básico antes. Este livro obviamente não supõe que você seja um especialista que saiba tudo sobre ataques man-in-the-middle, mas parte do princípio que você já se deparou com o termo antes.

Considerações sobre o ambiente de desenvolvimento

Tudo de que você precisa para usar os exemplos de programação deste livro são um editor de texto e um navegador. O editor de texto deve gerar texto

puro, sem qualquer tipo de formatação. Também deve permitir que você salve os arquivos usando as extensões corretas para o arquivo de exemplo (*.html*, *.css* e *.js*). Os vários revisores do livro, os leitores beta e eu testamos os exemplos usando os navegadores mais populares nas plataformas Linux, Mac e Windows. Na verdade, os exemplos foram testados até mesmo no navegador Edge para Windows 10.

Ícones usados neste livro

Os ícones dão ênfase de vários tipos. Este livro usa um mínimo de ícones, mas você deve conhecer cada um deles:



Uma nota dá ênfase a um conteúdo importante que se distancia um pouco do assunto ou que, talvez, tenha uma natureza que causaria ruptura no fluxo normal do texto do capítulo. Você deve ler as notas porque normalmente elas fazem referência a informações adicionais necessárias para realizar bem as tarefas de segurança. As notas também facilitam encontrar um conteúdo importante que você se lembra que está em determinado local, mas teria dificuldades de encontrá-lo de outra maneira.



Um aviso contém informações que você deve saber, pois, do contrário, poderá sofrer uma terrível fatalidade. Assim como uma nota, o aviso dá ênfase a um texto especial, mas esse texto informa sobre aspectos em potencial que poderiam causar problemas significativos em algum ponto. Se você não aproveitar mais nada de um capítulo, grave o significado por trás dos avisos em sua memória para poder evitar erros caros mais tarde.

Caixas de texto

Uma caixa de texto contém informações úteis que você não necessariamente precisa saber para trabalhar com aplicações web. Leia todas as caixas de texto em algum momento porque elas realmente são interessantes, mas não é preciso lê-las de imediato. As caixas de texto contém boas informações que complementam o assunto discutido no momento, mas que podem não ser exatamente sobre ele.

Convenções usadas neste livro

As seguintes convenções tipográficas são usadas neste livro:

Itálico

Indica termos novos, URLs, endereços de email, nomes e extensões de

arquivos.

Largura constante

Usada para listagens de programas, assim como em parágrafos para se referir a elementos de programas, como nomes de variáveis ou de funções, bancos de dados, tipos de dados, variáveis de ambiente, comandos e palavras-chave.

Largura constante em negrito

Mostra comandos ou outro texto que devam ser digitados literalmente pelo usuário.

Largura constante em itálico

Mostra o texto que deve ser substituído por valores fornecidos pelo usuário ou determinados pelo contexto.

Alguns textos deste livro recebem tratamento especial. Eis algumas convenções que você deve conhecer:

Códigos-fonte aparecem em parágrafos especiais na maioria dos casos para facilitar a leitura e o uso.

Às vezes, você verá um código-fonte em um parágrafo normal. A formatação especial deixa-o mais visível.

URLs, como *<http://blog.johnmuellerbooks.com>*, aparecem com uma fonte especial para facilitar sua localização. Este livro usa vários URLs para que você possa encontrar muitas informações relacionadas ao assunto, sem ter que procurá-las por conta própria.

Onde obter mais informações

Gostaria de ter certeza de que você terá a melhor experiência de leitura possível. Por favor, envie qualquer pergunta específica que você possa ter sobre o livro para John@JohnMuellerBooks.com. Você também pode dar uma olhada nas postagens de blog relacionadas a este livro em <http://blog.johnmuellerbooks.com/category/technical/security-for-web-developers/>. Essas postagens oferecem conteúdo adicional e respondem a perguntas comumente feitas pelos leitores. Se houver alguma errata para o

livro, as correções poderão ser encontradas no blog também.

Usando exemplos de código (de acordo com a política da O'Reilly)

Materiais suplementares (exemplos de código, exercícios etc.) estão disponíveis para download em https://github.com/oreillymedia/Security_for_Web_Developers.

Este livro está aqui para ajudá-lo a fazer seu trabalho. De modo geral, se este livro incluir exemplos de código, você poderá usar o código em seus programas e em sua documentação. Você não precisa nos contatar para pedir permissão, a menos que esteja reproduzindo uma parte significativa do código. Por exemplo, escrever um programa que use diversas partes de código deste livro não requer permissão. Porém vender ou distribuir um CD-ROM de exemplos de livros da O'Reilly requer permissão. Responder a uma pergunta mencionando este livro e citar o código de exemplo não requer permissão. Em contrapartida, incluir uma quantidade significativa de exemplos de código deste livro na documentação de seu produto requer permissão.

Agradecemos, mas não exigimos, atribuição. Uma atribuição geralmente inclui o título, o autor, a editora e o ISBN. Por exemplo: “*Security for Web Developers*, de John Paul Mueller (O'Reilly). Copyright 2016 John Paul Mueller, 978-1-49192-864-6.”

Se você achar que o seu uso dos exemplos de código está além do razoável ou da permissão concedida, sinta-se à vontade em nos contatar em permissions@oreilly.com.

Como entrar em contato conosco

Envie seus comentários e suas dúvidas sobre este livro à editora escrevendo para: novatec@novatec.com.br.

Temos uma página web para este livro na qual incluímos erratas, exemplos e quaisquer outras informações adicionais.

- Página da edição em português

<http://www.novatec.com.br/catalogo/7522484-seguranca-desenvolvedores-web>

- Página com material suplementar (exemplos de códigos, exercícios etc.).

<http://bit.ly/security-web-dev>

Para obter mais informações sobre os livros da Novatec, acesse nosso site:
<http://www.novatec.com.br>.

Agradecimentos

Agradeço à minha esposa Rebecca. Embora ela não esteja mais conosco agora, seu espírito está em todos os livros que escrevo, em todas as palavras que estão nas páginas. Ela acreditava em mim quando ninguém mais o fazia.

Russ Mullen, Billy Rios e Wade Woolwine merecem meu agradecimento pela revisão técnica deste livro. Todos os três revisores técnicos contribuíram muito para a exatidão e a profundidade do material que você vê aqui. Muitas vezes pude lançar ideias a eles e pedir que me ajudassem a pesquisar assuntos essenciais para o livro.

Meu agente Matt Wagner merece crédito por ter me ajudado a conseguir o contrato, em primeiro lugar, e por cuidar de todos os detalhes que a maioria dos autores realmente não leva em consideração. Serei sempre grato pela sua assistência. É bom saber que alguém quer ajudar.

Várias pessoas leram o livro todo ou parte dele para me ajudar a melhorar a abordagem e os scripts de teste e, em geral, ofereceram sugestões que todos os leitores gostariam de ter dado. Esses voluntários ajudaram de tantas maneiras que é impossível mencioná-los aqui. Em especial, sou grato aos esforços de Eva Beattie, Glenn A. Russell e Luca Massaron, que ofereceram sugestões em geral, leram o livro todo e se dedicaram de forma altruísta a este projeto.

Por fim, gostaria de agradecer a Meg Foley, Nicole Shelby, Jasmine Kwityn e ao restante da equipe de edição e produção.

¹ N.T.: “*Ransomware* é um tipo de malware que restringe o acesso ao sistema infectado e

cobra um valor de 'resgate' para que o acesso possa ser reestabelecido.” (Fonte: <https://pt.wikipedia.org/wiki/Ransomware>)

PARTE I

Desenvolvendo um plano de segurança

Nesta parte do livro você aprenderá a criar um plano de segurança para usar quando escrever aplicações. Ter um bom plano de segurança garante que sua aplicação realmente atenderá a metas específicas e que outras pessoas poderão discutir como implementar a segurança com a equipe de desenvolvimento. Sem um bom plano de segurança definido, com frequência, hackers terão fácil acesso à aplicação e causarão todo tipo de problemas à sua empresa. O capítulo 1 apresenta uma introdução que ajudará você a entender os componentes de um bom plano de segurança.

É importante perceber que o plano de segurança não está centrado apenas na aplicação, mas também em como os usuários a utilizam. Toda aplicação bem-sucedida tem o usuário em mente, conforme descrito no capítulo 2.

Além disso, você deve entender que as aplicações não existem mais no vácuo – elas interagem com fontes de dados online e dependem de códigos de terceiros. Com isso em mente, você também deve considerar o modo como as soluções de terceiros podem afetar sua aplicação, tanto de forma positiva quanto negativa. Usar soluções de terceiros também pode diminuir bastante seu tempo de programação. O capítulo 3 ajuda você a alcançar esse objetivo.

CAPÍTULO 1

Definindo o ambiente da aplicação

Os dados são o recurso mais importante que qualquer negócio possui. É literalmente possível substituir qualquer parte de um negócio, exceto os dados. Quando os dados são modificados, corrompidos, roubados ou apagados, um negócio pode sofrer perdas sérias. De fato, um negócio que cometa muitos erros com seus dados pode simplesmente deixar de existir. O foco da segurança, portanto, não está em hackers, aplicações, redes nem em outro elemento que alguém possa ter mencionado a você – está nos dados. Desse modo, este livro tem a ver com segurança de dados, que engloba vários outros assuntos, mas é importante ir direto ao ponto sobre o que você está realmente tentando proteger quando ler sobre esses outros assuntos.

Infelizmente, dados não têm muita utilidade se ficarem parados, sozinhos no escuro. Não importa quão sofisticado seja seu servidor, não importa a capacidade do banco de dados que armazena os dados, eles não terão muito valor até que você faça algo com eles. A necessidade de gerenciar dados traz as aplicações para o quadro geral, e o uso de aplicações para gerenciar dados é o motivo pelo qual este capítulo introdutório discute o ambiente das aplicações.

Contudo, antes de prosseguir, é importante decidir exatamente como as aplicações e os dados interagem, pois o restante do capítulo não será muito útil sem esse esclarecimento. Uma aplicação realiza apenas quatro operações sobre os dados, não importa quão incrivelmente complexa a aplicação possa se tornar. Essas operações podem ser definidas pelo acrônimo CRUD:

- Create (Criar)
- Read (Ler)
- Update (Atualizar)

- Delete (Apagar)

As próximas seções discutem dados, aplicações e CRUD, conforme eles se relacionam com o ambiente web. Você descobrirá como a segurança afeta todos os três aspectos do desenvolvimento web, tendo em mente que, apesar de os dados serem o foco, a aplicação realiza as tarefas CRUD necessárias. Manter seus dados protegidos significa entender o ambiente da aplicação e, desse modo, as ameaças aos dados gerenciados por ela.

Especificando as ameaças às aplicações web

Você pode encontrar listas de ameaças a aplicações web em toda a internet. Algumas das listas são abrangentes e não são necessariamente tendenciosas, algumas incluem o que o autor acha que são as ameaças mais importantes, outras apresentam as ameaças mais comuns, e você pode encontrar todo tipo de lista por aí. O problema com todas essas listas é que o autor não conhece sua aplicação. Um ataque de injeção de SQL só terá sucesso quando sua aplicação usar SQL de alguma maneira – talvez ela não use.

Obviamente, é preciso partir de algum lugar para ter ideias sobre o que você deve verificar, e essas listas são um bom ponto de partida. Entretanto, você deve considerar o conteúdo das listas tendo em vista a sua aplicação. Além do mais, não conte com apenas uma lista – use várias para que você tenha melhor abrangência das ameaças que podem, possivelmente, ameaçar sua aplicação. Com isso em mente, aqui está uma lista das ameaças mais comuns às aplicações web que você verá atualmente:

Buffer overflow (transbordamento de buffer)

Um invasor consegue enviar dados suficientes em um buffer de entrada para causar overflow (transbordamento) em um buffer da aplicação ou de saída. Como resultado, a memória fora do buffer é corrompida. Algumas formas de buffer overflow permitem ao invasor realizar tarefas aparentemente impossíveis porque a memória afetada contém código executável. A melhor maneira de resolver esse problema é realizar verificações de intervalo e de tamanho em qualquer dado, seja de entrada ou de saída, que sua aplicação manipule. Você pode ler mais sobre buffer

overflows em aplicações web em http://www.upenn.edu/computing/security/swat/SWAT_Top_Ten_A5.php e em https://www.owasp.org/index.php/Buffer_Overflows.

Injeção de código

Uma entidade adiciona código ao stream de dados que flui entre um servidor e um cliente (por exemplo, um navegador) no estilo de ataque *man-in-the-middle*. O alvo muitas vezes vê o código adicionado como parte da página original, mas ele pode conter qualquer código. É claro que o alvo pode nem mesmo ver o código injetado. Esse código pode ficar à espreita em background, pronto para provocar todo tipo de problema em sua aplicação. Uma boa maneira de enfrentar esse ataque é garantir que streams de dados criptografados, o protocolo HTTPS e a verificação de código sejam usados (quando possível). Oferecer um sistema de feedback do cliente também é uma boa ideia.



A injeção de código ocorre com mais frequência do que você imagina. Em alguns casos, ela nem mesmo faz parte de um ataque, mas poderia ser. Um artigo recente (<http://www.infoworld.com/article/2925839/net-neutrality/code-injection-new-low-isps.html>) discute como os ISPs (Internet Service Providers, ou Provedores de Serviço de Internet) estão injetando código JavaScript no stream de dados para colocar propagandas sobre uma página. Para determinar o tipo de propaganda a ser fornecida, o ISP também monitora o tráfego.

Cross-site scripting (XSS)

Um invasor injeta JavaScript ou outro código executável no stream de saída de sua aplicação. O receptor vê sua aplicação como a origem da infecção, mesmo quando ela não é. Na maioria dos casos, você não deve permitir que usuários enviem dados diretamente uns aos outros através de sua aplicação sem que seja feita uma verificação rigorosa. Uma formatação moderada para aplicações como blogs é obrigatória para garantir que sua aplicação não acabe servindo vírus ou algo pior junto com dados aparentemente inofensivos.



Poucos especialistas lembram você de que é preciso verificar seus dados de saída. Entretanto, você não sabe realmente se sua própria aplicação é confiável. Um hacker poderia modificá-la para gerar dados de saída contaminados. Verificações devem incluir tanto os dados de saída quanto os de entrada.

Upload de arquivos

Todo upload de arquivo, mesmo aqueles que possam parecer inócuos, é suspeito. Às vezes, hackers fazem upload de backdoors (porta dos fundos) usando os recursos de upload de arquivos de seu servidor, portanto o arquivo pode conter algo nocivo. Se for possível, desabilite uploads de arquivo em seu servidor. É claro que nem sempre é possível oferecer esse nível de segurança, desta forma você deve permitir somente determinados tipos de arquivo e então verificá-los para ver se não há problemas. Autenticar o arquivo o máximo possível é sempre uma boa ideia. Por exemplo, alguns arquivos contêm uma assinatura no início, que pode ser usada para garantir que o arquivo é legítimo. Não dependa apenas da extensão do arquivo – muitas vezes, os hackers fazem um arquivo parecer de outro tipo para passar pelo sistema de segurança do servidor.

Autenticação fixa

Desenvolvedores muitas vezes colocam informações de autenticação em arquivos de inicialização da aplicação para testes. É essencial remover esses dados fixos de autenticação e contar com uma área de armazenamento de dados centralizada para informações de segurança. Manter a área de armazenamento de dados em um local seguro, fora do servidor usado para aplicações web, é essencial para garantir que os hackers não poderão simplesmente ver as credenciais usadas para acessar a aplicação de determinadas maneiras. Se precisar de arquivos de inicialização para a aplicação, certifique-se de que esses arquivos fiquem fora do diretório webroot para garantir que os hackers não os encontrem acidentalmente.

Descoberta de arquivo/diretório oculto ou restrito

Quando sua aplicação permite a entrada de caracteres especiais como barra (/) ou barra invertida (\), é possível a um hacker descobrir arquivos e diretórios ocultos ou restritos. Esses locais podem conter todo tipo de informações que um hacker pode achar útil para atacar seu sistema. Desabilitar o uso de caracteres especiais sempre que for possível é uma ótima ideia. Além disso, armazene arquivos críticos fora do diretório webroot, em locais que o sistema operacional possa controlar diretamente.

Autenticação ausente ou incorreta

É importante saber com quem você está lidando, especialmente quando trabalhar com dados sensíveis. Muitas aplicações web dependem de contas comuns para algumas tarefas, o que quer dizer que é impossível saber quem acessou a conta. Evite usar contas de convidado para qualquer finalidade e atribua uma conta específica a cada usuário.

Autorização ausente ou incorreta

Mesmo que você conheça a pessoa com quem está lidando, é importante fornecer apenas o nível de autorização necessário para executar uma dada tarefa. Além disso, a autorização deve refletir o método de acesso do usuário. Um sistema desktop acessando a aplicação a partir da rede local provavelmente é mais seguro que um smartphone acessando a aplicação da lanchonete local. Contar com promoção de segurança para dar assistência em tarefas sensíveis permite manter os direitos em um nível mínimo no restante do tempo. Tudo que você puder fazer para limitar o que o usuário está autorizado a fazer ajuda a manter um ambiente seguro.

Criptografia ausente ou incorreta

Use criptografia para transmitir dados de qualquer tipo entre dois endpoints e evitar que hackers ouçam sua comunicação. É importante manter-se atualizado quanto às técnicas mais recentes de criptografia e contar com a melhor criptografia que o ambiente do usuário suporte. Por exemplo, o 3DES (Triple Data Encryption Standard) não é mais seguro, embora algumas empresas continuem a usá-lo. O atual AES (Advanced Encryption Standard, ou Padrão de Criptografia Avançada) é o mais seguro, mas você deve usar a chave mais longa possível para que seja difícil quebrá-la.

Injeção de comandos do sistema operacional

Um invasor modifica um comando de sistema operacional usado pela sua aplicação para executar tarefas específicas. Sua aplicação web provavelmente não deveria usar chamadas de sistema operacional, em primeiro lugar. No entanto, se você realmente precisa fazer chamadas do sistema operacional, garanta que a aplicação executará em uma sandbox.

Alguns especialistas enfatizarão validar dados de entrada para alguns casos e



deixar esse requisito de lado para outros. Sempre valide qualquer dado que receber de qualquer lugar. Não há como saber qual veículo um hacker usará para ter acesso ao seu sistema ou causar danos de outras maneiras. Dados de entrada são sempre suspeitos, mesmo quando são provenientes de seu próprio servidor. Ser paranoico é bom quando se está executando tarefas relacionadas à segurança.

Manipulação de parâmetros

Hackers podem fazer experimentos com parâmetros passados como parte do cabeçalho de requisição ou do URL. Por exemplo, ao trabalhar com o Google, você pode alterar o URL e os resultados de sua pesquisa. Não se esqueça de criptografar qualquer parâmetro que você passar entre o navegador e o servidor. Além disso, use protocolos seguros de páginas web, como HTTPS, quando passar parâmetros.



Hackers ainda podem manipular parâmetros se tiverem tempo e dedicarem esforço suficientes. Também é importante definir cuidadosamente os intervalos e os tipos de dados para os valores dos parâmetros a fim de reduzir os problemas em potencial apresentados por esse ataque.

Inclusão de código remoto

A maioria das aplicações web atualmente depende de bibliotecas incluídas, frameworks e APIs. Em muitos casos, a instrução de inclusão contém um path relativo ou usa uma variável contendo um path fixo para facilitar a alteração do local em que está o código remoto posteriormente. Quando um hacker consegue ter acesso à informação de path e alterá-la, é possível apontar a inclusão de código remoto para qualquer código que o hacker queira, proporcionando-lhe total acesso à aplicação. A melhor maneira de evitar esse problema em particular é usar paths completos fixos sempre que for possível, mesmo que essa ação dificulte a manutenção do código.

Muitos especialistas recomendarão o uso de bibliotecas e frameworks inspecionados para executar tarefas perigosas. Entretanto, esses add-ons simplesmente representam mais código. Hackers encontram métodos para corromper e contornar códigos de bibliotecas e de frameworks regularmente. Continuará sendo necessário garantir que sua aplicação e qualquer código do qual ela dependa interajam com elementos externos de forma segura, o que significa realizar testes exaustivos. Usar bibliotecas e frameworks reduz seu custo de suporte e garante que você faça correções de bugs rapidamente, mas os



bugs continuarão existindo e você deve estar sempre atento. Não há solução mágica quando se trata de segurança. O capítulo 6 contém mais informações sobre como trabalhar com bibliotecas e frameworks.

Sequestro de sessão (session hijacking)

Sempre que alguém fizer login em seu servidor web, este concederá uma sessão única a esse usuário. Um sequestrador de sessão entra na seção e intercepta dados transferidos entre o usuário e o servidor. Os três lugares mais comuns para procurar informações usadas no sequestro de uma sessão são: cookies, reescrita de URL e campos ocultos. Os hackers procuram informações de sessão nesses lugares. Ao manter as informações de sessão criptografadas, você pode reduzir o risco de alguém interceptá-las. Por exemplo, use o protocolo HTTPS para logins. Evite também ações como deixar seus IDs de sessão previsíveis.

Injeção de SQL

Um invasor modifica uma query criada por sua aplicação como resultado de entrada de usuário ou de outras entradas. Em muitos casos, a aplicação solicita dados de entrada de query, mas recebe elementos SQL em seu lugar. Outras formas de ataque de injeção de SQL envolvem o uso de escape ou de outros caracteres ou sequência de caracteres não esperados. Uma boa maneira de evitar ataques de injeção de SQL é evitar a geração de queries dinamicamente.

Essas podem parecer ameaças muito diferentes, mas se você pesquisar o suficiente na internet, poderá facilmente triplicar o tamanho dessa lista e mal tocar a superfície das maneiras pelas quais um hacker pode deixar sua vida interessante. À medida que avançarmos no livro, você conhecerá um número muito maior de tipos de ameaça e descobrirá maneiras de enfrentá-los. Não se

preocupe: na maior parte dos casos as correções acabam envolvendo bom senso, e uma única correção pode resolver mais de um problema. Por exemplo, observe a lista novamente e você perceberá que simplesmente usar HTTPS resolve vários desses problemas.

Considerando o aspecto da privacidade na segurança

Quando uma empresa se aprofunda em segurança, ela tende a focar primeiro na segurança de seus próprios dados. Afinal de contas, se os dados da empresa são perdidos, corrompidos, modificados ou tornam-se inutilizáveis, essa empresa poderia se ver fora do mercado. O próximo nível de verificação normalmente está em terceiros, por exemplo, nos parceiros. Com frequência, a segurança dos dados do usuário está em último lugar, e muitas empresas não pensam muito na segurança dos dados dos clientes. O problema é que muitos usuários e clientes veem a segurança de seus dados como o que há de mais importante. Toda a questão da privacidade se reduz à proteção dos dados do usuário, de modo que ninguém faça uso indevido ou exponha as informações sem o conhecimento e o consentimento do usuário. Em suma, ao criar uma aplicação, você deve considerar também a privacidade dos dados do usuário como uma questão de segurança, e uma questão importante.

Um artigo recente (<http://www.infoworld.com/article/2925292/internet-privacy/feds-vs-silicon-valley-who-do-you-trust-less.html>) destaca que usuários e clientes veem a indústria da tecnologia como uma péssima administradora de seus dados. De fato, a indústria de tecnologia está atrás do governo – as pessoas geralmente confiam mais no governo para guardar suas informações. Muitas empresas de tecnologia apoiam publicamente melhores políticas de segurança para outras entidades (como o governo) e, em particular, criam outras maneiras de contrariar qualquer noção de privacidade que o usuário ou o cliente possam ter. Essa dualidade deixa a situação muito pior do que seria se a indústria da tecnologia fosse aberta em relação à usurpação de dados de usuários e de clientes.

Para criar uma aplicação realmente segura, você deve estar disposto a proteger todos os aspectos dela, incluindo os dados de usuários e clientes. Essa ação exige que a aplicação obtenha e administre somente os dados necessários para executar suas tarefas e então descarte os dados quando não precisar mais deles. Confiança é algo que sua aplicação pode conquistar somente se obedecer ao mesmo conjunto de regras para trabalhar com todos os dados, independentemente de sua origem.

Entendendo a Software Security Assurance (SSA)

O propósito do software é interagir com dados. Entretanto, o software em si é um tipo de dado. Com efeito, os dados têm várias formas que talvez você não considere, e o efeito dos dados é mais amplo do que normalmente se imagina. Com a IoT (Internet of Things, ou Internet das Coisas), agora é possível que os dados tenham efeitos tanto abstratos quanto físicos de formas que ninguém poderia imaginar, até mesmo há poucos anos. Um hacker que tenha acesso à aplicação correta pode executar ações como danificar a rede elétrica ou envenenar o sistema de água. Em um nível mais pessoal, o mesmo hacker poderia elevar a temperatura de sua casa a um nível assustador, apagar todas as luzes ou espiar você pela sua webcam, somente para citar alguns exemplos. A questão principal da SSA é que o software precisa de algum tipo de regulamentação para garantir que não cause perdas, imprecisões, alterações, indisponibilidade ou uso indevido de dados e recursos que ele utiliza, controla e protege. Esse requisito aparece como parte da SSA. As próximas seções discutem a SSA com mais detalhes



A SSA atualmente não é um padrão. É um conceito que muitas empresas quantificam e definem por escrito com base em suas necessidades. Os mesmos padrões básicos aparecem em muitos desses documentos e o termo SSA refere-se à prática de garantir que o software permaneça seguro. Você pode ver como a SSA afeta muitas empresas, como Oracle (<http://www.oracle.com/us/support/assurance/overview/index.html>) e Microsoft (<https://msdn.microsoft.com/library/windows/desktop/84aed186-1d75-4366-8e61-8d258746bopq.aspx>), analisando sua documentação para SSA disponível online. De fato, muitas empresas de grande porte atualmente têm alguma forma de SSA definida.

Considerando o OSSAP

Um dos principais sites que você precisa conhecer para fazer da SSA uma realidade em aplicações web é o OWASP (Open Web Application Security Project ou Projeto Aberto para Segurança em Aplicações Web, em https://www.owasp.org/index.php/OWASP_Software_Security_Assurance_Process – veja a figura 1.1. O site detalha o processo necessário para fazer com que o OSSAP (OWASP Security Software Assurance Process, ou Processo de Garantia de Segurança de Software do OWASP) seja parte do SDLC (Software Development Lifecycle, ou Ciclo de Vida do Desenvolvimento de

Software). Sim, é uma enorme sopa de letrinhas, mas você deve conhecer esse grupo para criar um processo para sua aplicação que esteja de acordo com o trabalho feito por outras empresas. Além disso, as informações nesse site ajudarão você a desenvolver um processo de segurança para sua aplicação que realmente funcione, faça parte do processo de desenvolvimento e a não consumir muito tempo para criar seu próprio processo.



Embora o OSSAP ofereça um ótimo framework para garantir que sua aplicação atenda aos requisitos de SSA, não há nenhum requisito para você interagir com esse grupo de forma alguma. O grupo licencia sua abordagem para SSA. No entanto, atualmente, ele acabou de iniciar seus trabalhos e você verá muitos TBDs (To Be Determined, ou A ser definido) no site que o grupo planeja definir com o passar do tempo. É claro que você precisa de um plano para hoje, portanto o OWASP e seu OSSAP apresentam um lugar para você pesquisar soluções no presente e, possivelmente, obter ajuda adicional mais tarde.

O principal motivo para implementar SSA em sua aplicação como parte do SDLC é garantir que o software seja tão confiável e livre de erros quanto for possível. Ao falar com algumas pessoas, elas parecem achar que a SSA corrigirá todos os problemas de segurança em potencial que possam ser encontrados, mas simplesmente não é isso que acontece. A SSA deixará seu software melhor, mas você não encontrará nenhum software, em lugar algum, que esteja livre de erros. Supondo que você tenha conseguido criar um software livre de erros, continua sendo necessário considerar usuários, ambiente, rede e todo o software para outras questões de segurança. Consequentemente, a SSA é apenas uma parte do quadro muito maior de segurança, e implementá-la só corrigirá problemas de segurança até certo ponto. O melhor a fazer é ver a segurança como um processo contínuo.



Figura 1.1 – O site do OWASP dá informações sobre SSA para aplicações web.

Definindo os requisitos de SSA

O primeiro passo na implementação de SSA como parte de sua aplicação é definir os requisitos de SSA. Esses requisitos ajudam a determinar o estado atual de seu software, os problemas que exigem solução e a severidade desses problemas. Depois que os problemas estiverem definidos, você pode determinar o processo de correção e quaisquer outros requisitos necessários para garantir que o software permaneça seguro. Podemos dividir a SSA em oito passos:

1. Avaliar o software e desenvolver um plano para corrigi-lo.
2. Definir os riscos que os problemas de segurança representam aos dados e classificar esses riscos para corrigir os piores antes.

3. Fazer uma revisão completa de código.
4. Implementar as mudanças necessárias.
5. Testar as correções implementadas e conferir se elas realmente funcionam no sistema em produção.
6. Definir um sistema de defesa para proteger o acesso à aplicação e, desse modo, os dados que a aplicação gerencia.
7. Medir a eficiência das alterações feitas.
8. Treinar a gerência, os usuários e os desenvolvedores quanto aos métodos apropriados para garantir uma boa segurança à aplicação.

Uma estratégia alternativa de SSA

Quando se trata de segurança, você pode encontrar diversas maneiras de lidar com um problema em particular. Conforme a cultura de sua empresa e o método de trabalho diante de problemas de segurança, você pode encontrar uma técnica alternativa para garantir que a SSA funcione melhor. Alguns especialistas em segurança sugerem estes passos:

1. Definir os requisitos de segurança de modo que englobem os requisitos do produto (você só precisa executar este passo para softwares novos).
2. Definir e divulgar os requisitos para código seguro que os desenvolvedores devem usar para escrever código.
3. Fazer revisões automáticas de código à medida que os desenvolvedores criarem códigos novos.
4. Fazer uma revisão completa de código após concluir a aplicação.
5. Realizar testes de invasão e avaliação de vulnerabilidades na aplicação completa.
6. Avaliar os resultados dos testes para encontrar um equilíbrio entre segurança e riscos ao negócio. Planejar correções para qualquer vulnerabilidade identificada como arriscada demais para o negócio.
7. Implementar qualquer correção de segurança necessária.
8. Repetir o passo 5.

Classificando dados e recursos

Este processo envolve identificar os vários dados com os quais sua aplicação entra em contato de alguma forma, incluindo seu próprio código e informações de configuração. Após identificar todos os dados, classifique-os

para definir o nível de segurança exigido para protegê-los. Os dados podem ter vários níveis de classificação, e o modo como você classifica os dados depende das necessidades de sua empresa e do tipo dos dados. Por exemplo, alguns dados podem simplesmente causar inconveniências à empresa, enquanto outros podem, possivelmente, causar danos aos seres humanos. A definição de como as brechas de segurança nos dados afeta o ambiente de segurança como um todo é essencial.

Depois que o processo de classificação dos dados estiver concluído, é possível começar a usar as informações para realizar várias tarefas. Por exemplo, você pode considerar de que modo reduzirá as vulnerabilidades:

- criando padrões para programação;
- implementando treinamentos obrigatórios para desenvolvedores;
- contratando líderes de segurança nos grupos de desenvolvimento;
- usando procedimentos automatizados de testes que identifiquem especificamente problemas de segurança.

Todos esses métodos apontam para recursos com os quais a empresa interage e dos quais ela depende para garantir que a aplicação gerencie corretamente os dados. Classificar recursos quer dizer determinar o nível de ênfase a ser colocado em um recurso em particular. Por exemplo, não permitir que desenvolvedores recebam treinamento terá um impacto maior que impedir que usuários individuais da aplicação o recebam, pois os desenvolvedores trabalham na aplicação como um todo. É claro que treinamento é essencial para todos. Nesse caso, classificar recursos de todos os tipos ajuda a determinar onde e como o dinheiro será gasto a fim de obter o melhor ROI (Return on Investment, ou Retorno sobre Investimento), ao mesmo tempo em que os objetivos de segurança da aplicação sejam alcançados.

Fazendo a análise necessária

Como parte da SSA, você deve fazer uma análise de sua aplicação (incluindo modelagem de ameaças, falhas na interface de usuário e falhas na apresentação de dados). É importante saber exatamente que tipo de fragilidade seu código pode conter. A palavra em destaque aqui é *pode*. Até

realizar uma análise detalhada, você não tem como saber quais são os verdadeiros problemas de segurança de seu código. Aplicações web, em especial, tendem a esconder problemas porque, de modo diferente das aplicações desktop, o código pode aparecer em diversos lugares, e scripts tendem a ocultar problemas que aplicações compiladas não têm porque o código é interpretado em tempo de execução, e não em tempo de compilação.



É importante entender que segurança não está relacionada apenas ao código – diz respeito também às ferramentas necessárias para criar o código e às habilidades dos desenvolvedores que empregam essas ferramentas. Quando uma empresa escolhe as ferramentas incorretas para o trabalho, o risco de uma falha de segurança é muito maior, pois as ferramentas podem não criar um código que execute exatamente conforme esperado. De modo semelhante, quando desenvolvedores que usam a ferramenta não têm as habilidades necessárias, não é de surpreender que o software tenha falhas de segurança que um desenvolvedor mais habilidoso evitaria.

Alguns especialistas argumentam que há empresas que, na verdade, permitem trabalhos abaixo do padrão. Na maioria dos casos, a desculpa para permitir esse tipo de trabalho é que o processo de desenvolvimento da aplicação está com o cronograma atrasado ou a empresa não tem as ferramentas nem o conhecimento específico necessários. O fato de uma empresa empregar um software projetado para ajudar a tratar questões de segurança (como um firewall) não isenta o desenvolvedor da responsabilidade de criar um código seguro. As empresas precisam manter padrões de programação para garantir um bom resultado.

Lógica

Interagir com uma aplicação e os dados que ela gerencia é um processo. Embora usuários possam executar tarefas de maneira aparentemente aleatória, tarefas específicas seguem padrões que se devem ao fato de o usuário precisar seguir um procedimento para obter um bom resultado. Ao documentar e entender esses procedimentos, você pode analisar a lógica da aplicação de um ponto de vista prático. Usuários dependem de um procedimento em particular por causa da maneira como os desenvolvedores projetam a aplicação. Alterar o design necessariamente alterará o procedimento.

A questão principal da análise é procurar falhas de segurança no procedimento. Por exemplo, a aplicação pode permitir que o usuário permaneça logado, mesmo que nenhuma atividade seja detectada por um

longo período de tempo. O problema é que o usuário pode nem mesmo estar presente – outra pessoa poderia acessar a aplicação usando as credenciais do usuário e ninguém perceberia, pois todos achariam que o usuário está logado usando o mesmo sistema, como sempre.

No entanto, falhas de segurança nos dados podem assumir outras formas. Um número de identificação de uma peça pode ser constituído de diversos elementos quantificáveis. Para obter um bom número de identificação de peça, a aplicação poderia solicitar os elementos, em vez de pedir o número como um todo e compor o número de identificação a partir desses elementos. A ideia é deixar o procedimento mais limpo, mais claro e menos suscetível a erros de modo que o banco de dados não acabe com muitas informações ruins.

Dados

Talvez você ache que não pode fazer muitas análises em dados do ponto de vista de segurança, mas, na verdade, há muitas questões a considerar. De fato, a análise de dados é uma das áreas em que as empresas mais deixam a desejar porque a ênfase está em como administrar e usar os dados, e não em como protegê-los (é razoável supor que você deve abordar todas as três questões). Ao analisar dados, você deve considerar os seguintes pontos:

- Quem pode acessar os dados
- Qual é o formato usado para armazenar os dados
- Quando os dados são acessíveis
- Onde os dados são armazenados
- Por que cada item de dado é disponibilizado como parte da aplicação
- Como os dados são separados em componentes e qual é o resultado da combinação dos dados para uso da aplicação

Por exemplo, algumas aplicações falham na prática de ocultar dados (data hiding), que é uma funcionalidade essencial para qualquer boa aplicação. Ocultar dados significa dar ao usuário somente a quantidade de informações realmente necessária para executar uma tarefa específica.

As aplicações também formatam alguns dados incorretamente. Por exemplo,

é quase certo que armazenar senhas como texto causará problemas se alguém conseguir invadir o sistema. Uma opção melhor é armazenar o hash da senha usando um algoritmo seguro (um que não tenha sido quebrado). O hash não é importante para alguém que tenha feito uma invasão, pois a aplicação precisa da senha a partir da qual o hash foi criado.

Deixar todos os dados acessíveis o tempo todo também é uma péssima ideia. Dados sensíveis devem aparecer na tela somente quando alguém estiver disponível para monitorar seu uso e reagir imediatamente, caso o usuário faça algo inesperado.

Armazenar dados sensíveis na nuvem é uma ideia particularmente ruim. Sim, usar armazenamento em nuvem deixa os dados prontamente disponíveis, além de rápidos para serem acessados, mas também vulneráveis. Armazene dados sensíveis em servidores locais quando você tiver acesso direto a todos os recursos de segurança usados para manter os dados protegidos.

Desenvolvedores de aplicações também tendem a deixar muitas informações disponíveis. Use ocultação de dados para manter dados específicos de gerenciamento ocultos a outros tipos de usuários. No entanto, alguns dados não têm lugar na aplicação. Se ninguém realmente precisa de determinado dado para executar uma tarefa, não acrescente esse dado à aplicação.

Muitos dados atualmente estão em agregações, como parte de outros dados. É possível a um hacker aprender muito sobre sua empresa ao identificar a forma de agregação usada, separando o item de dado para descobrir suas partes constituintes. É importante considerar como os dados são reunidos e adicionar proteções que dificultem descobrir a origem desses dados.

Interface

Um grande problema com softwares atualmente é a inclusão de recursos desnecessários. Espera-se que uma aplicação atenda a um conjunto específico de objetivos ou execute um conjunto específico de tarefas. Invariavelmente, alguém tem a ideia de que o software, de alguma maneira, seria melhor se tivesse determinadas funcionalidades que não têm nada a ver com os objetivos principais que o software deve alcançar. O termo *excesso de recursos* (feature bloat) está presente há muito tempo. Normalmente, ele é

discutido em um sentido monetário – como a origem dos problemas de velocidade das aplicações, o responsável pela elevação dos custos de treinamento dos usuários e aquele que acaba com os cronogramas de desenvolvimento. Entretanto, os problemas de interface da aplicação – que, com frequência, são os mais afetados pelo excesso de recursos – têm um impacto significativo na segurança na forma de um aumento da superfície de ataque. Sempre que a superfície de ataque aumenta, você oferece mais oportunidades a um hacker para ter acesso à sua empresa. Livrar-se de recursos desnecessários ou transferi-los para uma aplicação totalmente diferente reduzirá a superfície de ataque, deixando sua aplicação muito mais segura. É claro que você economizará dinheiro também.

Outro problema em potencial é a interface com pistas (hint interface) – uma interface que revela os recursos de segurança da aplicação ao oferecer informações ou recursos demais a um hacker em potencial. Embora seja necessário oferecer uma maneira para o usuário recuperar uma senha perdida, algumas implementações, na verdade, possibilitam a um hacker obter a senha de um usuário e transformar-se nele. O hacker pode até mesmo impedir que o verdadeiro usuário acesse a conta alterando a senha (essa ação, porém, seria contraproducente, pois um administrador poderia facilmente restaurar o acesso do usuário). Um sistema mais conveniente é garantir que o usuário tenha realmente feito a solicitação antes de tomar qualquer atitude e, então, certificar-se de que o administrador enviará as informações de login de forma segura.

Restrições

Uma restrição é simplesmente um método para garantir que as ações atendam a critérios específicos antes que elas sejam permitidas. Por exemplo, desabilitar o acesso a elementos de dados, a menos que o usuário tenha o direito de acessá-los, é um tipo de restrição. No entanto, as restrições têm outras formas mais importantes. O tipo principal de restrição é determinar de que modo determinado usuário pode gerenciar os dados. A maioria dos usuários só precisa de acesso de leitura aos dados; apesar disso, as aplicações comumente concedem acesso de leitura/escrita, abrindo uma enorme brecha de segurança.

Dados também têm restrições a serem consideradas também. Ao trabalhar com dados, você deve definir exatamente o que os torna únicos e garantir que a aplicação não vá quebrar nenhuma regra em relação a essa unicidade. Com isso em mente, geralmente você deverá considerar os seguintes tipos de restrições:

- Garantir que os dados sejam do tipo certo
- Definir o intervalo de valores aceito pelos dados
- Especificar os tamanhos mínimos e máximos dos dados
- Listar qualquer valor inaceitável para os dados

Mergulhando em questões específicas de linguagem

O ambiente da aplicação é definido pelas linguagens usadas para criar a aplicação. Assim como toda linguagem tem funcionalidades que a fazem executar bem determinadas tarefas, toda linguagem também tem problemas em potencial que fazem dela um risco à segurança. Mesmo linguagens de baixo nível, apesar de sua flexibilidade, têm problemas causados pela complexidade. É claro que aplicações baseadas em web normalmente dependem de três linguagens em particular: HTML, CSS e JavaScript. As próximas seções descrevem alguns problemas específicos relacionados a essas linguagens em particular.

Definindo os principais problemas de HTML

O HTML5 tornou-se extremamente popular porque aceita uma variedade enorme de plataformas. A mesma aplicação pode funcionar bem no desktop de um usuário, em um tablet e em um smartphone, sem qualquer programação especial feita pelo desenvolvedor. Com frequência, bibliotecas, APIs e microsserviços oferecem conteúdo em um formato adequado ao sistema host automaticamente, sem intervenção do desenvolvedor. No entanto a flexibilidade oferecida pelo HTML5 também pode ser problemática. A lista a seguir descreve alguns problemas fundamentais de segurança que você pode ter ao trabalhar com HTML5:

Injeção de código

Com o HTML5 há inúmeras maneiras de um hacker injetar código malicioso, incluindo códigos-fonte que, normalmente, você não consideraria suspeito, por exemplo, um vídeo do YouTube ou um stream de música.

Monitoração de usuários

Como sua aplicação usa código de diversas origens na maioria dos casos, você poderá descobrir que uma biblioteca, uma API ou um microserviço fazem algum tipo de monitoração de usuário que um hacker poderia utilizar para aprender mais sobre sua empresa. Toda informação dada a um hacker facilita o processo de violação de seu sistema de segurança.

Dados de entrada contaminados

A menos que você forneça sua própria verificação de dados de entrada, o HTML5 permite passar qualquer dado que o usuário queira fornecer. Talvez você só precise de um valor numérico, mas o usuário poderia fornecer um script em seu lugar. Tentar verificar dados de entrada de forma completa para garantir que você está realmente recebendo o que pediu é quase impossível do lado cliente, portanto é preciso garantir que haja uma verificação robusta também do lado do servidor.

Superfícies de ataque com HTML5

O HTML5 acrescenta diversos recursos interessantes que os hackers simplesmente amam porque oferecem superfícies adicionais de ataque. Por exemplo, LocalStorage parece ser uma boa ideia até você considerar a facilidade com que um malware pode acessar a área de armazenamento local de um sistema. Você pode ver detalhes sobre essa superfície de ataque (incluindo alguns exemplos de ataque) em <http://www.slideshare.net/shreeraj/html5-localstorage-attack-vectors> e em <https://blog.whitehatsec.com/web-storage-security/>.

Outro recurso problemático é o WebSockets. Esse recurso permite comunicações bidirecionais entre um cliente e um servidor. O único problema é que o cliente pode estar executando um código não confiável. Você pode ler mais sobre os problemas desse recurso em particular em <http://www.slideshare.net/SergeyShekyan/bay-threat-2012-websockets> e em <http://blog.kotowicz.net/2011/03/html5-websockets-security-new-tool-for.html>, juntamente com exemplos de como o WebSockets pode ser usado para passar por cima de medidas de proteção, além de possíveis correções para o

problema.

Definindo os principais problemas de CSS

As aplicações dependem bastante de CSS3 para criar apresentações de boa aparência, sem que as informações para cada dispositivo estejam fixas no código. As bibliotecas de código CSS3 preexistente facilitam criar aplicações de aparência profissional que um usuário pode alterar para atender a qualquer necessidade. Por exemplo, um usuário pode precisar de uma apresentação diferente para um dispositivo em particular, ou exigir que a apresentação use um formato específico para atender a uma determinada necessidade. A lista a seguir descreve alguns problemas fundamentais de segurança que você pode ter ao trabalhar com CSS3:

Sobrecarga do design

Um motivo principal para um código CSS3 causar problemas de segurança é o exagero no design. O comitê de padrões originalmente projetou o CSS para controlar a aparência de elementos HTML, e não para afetar a apresentação de uma página web completa. Como resultado, os designers jamais pensaram em incluir segurança para determinados problemas porque não se esperava que o CSS funcionasse nessas áreas. O problema é que a parte da cascata do CSS não permite que o CSS3 tenha informações que não sejam de seus elementos pais. Como resultado, um hacker pode criar uma apresentação que pareça realizar uma tarefa, quando na verdade, faz outra. Algumas bibliotecas, como a jQuery, podem ajudar a solucionar esse problema.

Upload de CSS

Em alguns casos, um designer de aplicação permitirá que um usuário faça upload de um arquivo CSS para conseguir determinada aparência para a aplicação ou para fazê-la funcionar melhor em uma plataforma específica. No entanto, o CSS carregado também pode conter código que facilite a um hacker passar por cima de qualquer medida de segurança que você tenha implementado ou que oculte ações escusas. Por exemplo, um hacker poderia incluir URLs no CSS a fim de redirecionar a aplicação para

servidores não seguros.

Shaders CSS

Um uso especial de CSS pode apresentar alguns problemas sérios ao permitir acesso aos dados do agente de usuário e a dados entre domínios. Capítulos mais adiante no livro discutirão esse problema com mais detalhes, mas você pode ter uma visão geral do assunto em http://www.w3.org/Graphics/fx/wiki/CSS_Shaders_Security. A questão principal é que, às vezes, o ato de renderizar dados na tela abre brechas de segurança em potencial que talvez você não tenha considerado inicialmente.

Definindo os principais problemas de JavaScript

A combinação entre JavaScript e HTML5 criou todo o fenômeno de aplicações web. Sem a combinação dessas duas linguagens, não seria possível criar aplicações que executam bem em qualquer lugar, em qualquer dispositivo. Usuários não podiam nem pensar em pedir esse tipo de aplicação no passado porque era simplesmente impossível oferecê-lo. Atualmente, um usuário pode trabalhar em qualquer lugar usando um dispositivo que seja apropriado à localidade. No entanto, JavaScript é uma linguagem de scripts que pode apresentar falhas sérias de segurança. A lista a seguir descreve alguns problemas fundamentais de segurança que você pode ter quando trabalhar com JavaScript:

Cross-site scripting (XSS)

Esse problema apareceu antes no capítulo porque é extremamente sério. Sempre que você executar JavaScript fora de um ambiente de sandbox, é possível a um hacker executar todo tipo de truques sujos em sua aplicação.

Cross-site request forgery (CSRF)

Um script pode usar as credenciais do usuário armazenadas em um cookie para ter acesso a outros sites. Enquanto estiver nesses sites, o hacker poderá executar todo tipo de tarefa que a aplicação não foi projetada para realizar. Por exemplo, um hacker pode falsificar contas, roubar dados, realizar fraudes e executar vários tipos de atividades ilegais, tudo em nome do usuário.

Vulnerabilidades de navegadores e de plugin de navegadores

Muitos hackers contam com vulnerabilidades conhecidas de navegadores e de plugin de navegadores para forçar uma aplicação a executar tarefas para as quais não foi projetada. Por exemplo, o sistema de um usuário poderia repentinamente transformar-se em um zumbi, transmitindo código de vírus a outros sistemas. A extensão do que um hacker pode fazer é limitada pelas vulnerabilidades em questão. Em geral, você deve garantir que instalará todas as atualizações e que estará ciente de como as vulnerabilidades podem afetar a operação de sua aplicação.

Considerando o essencial para defesa nos endpoints

Um endpoint é um destino para o tráfego de rede, por exemplo, um serviço ou um navegador. Quando pacotes alcançam o endpoint, os dados que eles contêm são desempacotados e fornecidos à aplicação para processamento posterior. A segurança dos endpoints é essencial, pois eles representam um ponto importante de entrada nas redes. A menos que o endpoint seja seguro, a rede poderá receber transmissões com dados ruins. Além disso, a falta de segurança no endpoint pode causar danos a outros nós da rede. As próximas seções discutem três etapas para garantir a segurança nos endpoints: prevenção, detecção e correção.



É importante não subestimar o efeito da segurança dos endpoints nas aplicações e na infraestrutura da rede. Alguns cenários com endpoints tornam-se bastante complexos, e suas consequências são difíceis de detectar ou até mesmo de entender. Por exemplo, um artigo recente (<http://www.infoworld.com/article/2926221/security/large-scale-attack-hijacks-routers-through-users-browsers.html>) discute um ataque de roteador que depende de o invasor direcionar um usuário inocente a um site especial. O ataque está centrado no roteador do qual o usuário depende para fazer requisições de DNS (Domain Name System, ou Sistema de Nomes de Domínio). Ao obter controle total sobre o roteador, o invasor pode redirecionar o usuário para locais que esse invasor controla.

Evitando falhas de segurança

O primeiro passo para evitar uma armadilha é admitir que ela existe. O problema é que a maioria das empresas atualmente não acha que terá uma

falha de segurança; isso sempre acontece com outra empresa – aquela que é negligente quanto à segurança. Entretanto, de acordo com o *2014 Global Report on the Cost of Cyber Crime* (Relatório global sobre o custo de crimes cibernéticos de 2014, http://info.hpenterprisesecurity.com/LP_CP_424710_Ponemon_ALL) do Instituto Ponemon, o custo dos crimes cibernéticos foi de 12,7 milhões de dólares em 2014, muito acima dos 6,5 milhões de 2010. Obviamente, todas essas invasões não acontecem apenas em outras empresas – elas poderiam muito bem ocorrer em sua companhia, portanto é importante supor que algum hacker, em algum lugar, tem sua empresa como alvo. De fato, se partir da noção de que um hacker não só invadirá sua empresa, mas também roubará seus produtos, você poderá realmente começar a se preparar para o cenário do mundo real. Qualquer aplicação que você criar deve ser robusta o suficiente para:

- resistir a ataques comuns;
- informar invasões quando seu sistema de segurança não funcionar conforme esperado;
- evitar fazer suposições sobre os pontos em que as falhas ocorrerão;
- supor que, mesmo com treinamento, os usuários cometerão erros que provocarão uma falha.



Não suponha que falhas de segurança ocorrem apenas em algumas plataformas. Uma falha de segurança pode acontecer em qualquer plataforma que execute qualquer software que não seja um software customizado. Quanto menos preparados estiverem os desenvolvedores para uma plataforma em particular, mais devastadoras serão as falhas. Por exemplo, muitas pessoas consideram terminais POS (Point-of-Sale, ou Ponto de Venda) seguros contra ataques. No entanto, hackers atualmente estão atacando esses dispositivos vigorosamente (<http://www.computerworld.com/article/2925583/security/attackers-use-email-spam-to-infect-pos-terminals.html>) a fim de obter acesso a informações de cartões de crédito. O aspecto interessante sobre essa exploração de falha (exploit) em particular é que ela não funcionaria se os funcionários não estivessem usando os terminais POS incorretamente. Esse é um exemplo em que treinamento e políticas bem definidas poderiam ajudar a manter o sistema protegido. É claro que as aplicações ainda precisam ser robustas o suficiente para frustrar os ataques.

À medida que avançarmos no livro, você encontrará algumas técnicas úteis para diminuir as chances de haver falhas. Os pontos essenciais para evitar uma falha de segurança, depois de admitir que ela pode ocorrer (e provavelmente ocorrerá) são:

- criar aplicações que os usuários entendam e gostem de usar (veja o cap. 2);
- escolher fontes externas de dados cuidadosamente (consulte a seção “Acessando dados externos” na página 56 para ver os detalhes);
- criar aplicações que ofereçam barreiras naturais contra invasão (veja o capítulo 4);
- testar a confiabilidade do código que você criar e registrar cuidadosamente o tempo de inatividade (downtime) e as causas (veja o capítulo 5);
- escolher bibliotecas, APIs e microsserviços com cuidado (consulte a seção “Usando códigos e recursos externos” na página 51 para ver os detalhes);
- implementar uma estratégia abrangente de testes para todos os elementos da aplicação, mesmo aqueles que não sejam seus (consulte a Parte III para ver os detalhes);
- administrar os componentes de sua aplicação para garantir que suas defesas não se enfraquecerão depois que a aplicação for lançada (consulte a Parte IV para ver os detalhes);
- manter-se atualizado sobre as ameaças à segurança e as estratégias para vencê-las (veja o capítulo 16);
- treinar seus desenvolvedores para pensarem em segurança do início ao fim em todos os projetos (veja o capítulo 17).

Detectando falhas de segurança

A última coisa que uma empresa quer é ouvir falar de uma falha de segurança em segunda ou terceira mão. Ler sobre a incapacidade de sua empresa de proteger dados de usuário na imprensa especializada provavelmente é a pior maneira de começar um dia, embora seja assim que muitas empresas ficam sabendo de suas falhas de segurança. As empresas que assumem que uma

falha nos dados já ocorreu são as menos prováveis de sofrer danos permanentes com uma falha e as que têm mais chances de economizar dinheiro no final. Em vez de desperdiçar tempo e recursos corrigindo uma falha nos dados depois que ela ocorreu, sua empresa pode detectar a falha quando ela ocorrer e interrompê-la antes que se torne um problema. Detectar a falha quer dizer fornecer o código necessário como parte de sua aplicação e então garantir que esses métodos de detecção sejam projetados para funcionar com as ameaças atuais à segurança.

Sua empresa como um todo deverá ter uma equipe de resposta a falhas. Entretanto, sua equipe de desenvolvimento também precisa de indivíduos nos lugares certos para detectar falhas de segurança. A maioria das equipes de desenvolvimento atualmente precisará de especialistas em:

- rede;
- gerenciamento de banco de dados;
- design e desenvolvimento de aplicações;
- tecnologia móvel;
- computação forense;
- conformidade.

Cada aplicação precisa de uma equipe dessas e a equipe deve se reunir periodicamente para discutir requisitos e ameaças à segurança específicos da aplicação. Além disso, é importante analisar vários cenários de ameaça e determinar o que você fará quando uma falha realmente acontecer. Se estiver preparado, é mais provável que você detecte a falha com antecedência – possivelmente antes de alguém da gerência entrar gritando em seu escritório exigindo uma explicação.

Corrigindo um software com problemas

Quando uma falha de segurança ocorre, qualquer que seja a equipe definida por sua empresa, ela deve estar pronta para assumir a responsabilidade e trabalhar no processo de reparação. A organização como um todo precisa entender que não corrigir a falha de segurança e restaurar o sistema o mais rápido possível ao seu estado anterior à falha poderá fazer a empresa

fracassar. Em outras palavras, mesmo que você seja um ótimo funcionário, talvez seja preciso procurar um novo emprego.

A pessoa responsável pela segurança pode pedir ajuda à equipe de desenvolvimento para localizar o invasor. Software para SIEM (Security Information and Event Management, ou Segurança da Informação e Gerenciamento de Eventos) pode ajudar a analisar logs que apontem para a origem do problema. É claro que isso supõe que sua aplicação cria logs apropriados. Parte do processo de reparação consiste em criar funcionalidades de logging e de monitoração na aplicação, em primeiro lugar. Sem essas informações, tentar achar o culpado para que sua empresa possa interromper o ataque geralmente será uma causa perdida.

Seus procedimentos devem incluir uma estratégia para verificar atualizações ou patches (correções) para cada componente usado pela sua aplicação. Manter uma boa documentação para a aplicação é obrigatório se você quer atingir esse objetivo. Criar uma lista de recursos externos no momento em que houve uma falha é tarde demais; você deve ter essa lista em mãos antes que a falha ocorra. É claro que a equipe de desenvolvimento precisará testar qualquer atualização exigida pela aplicação para garantir que a falha não ocorrerá novamente. Por fim, é preciso garantir que os dados permaneçam seguros durante o processo e executar qualquer restauração que sua aplicação exigir.

Lidando com armazenamento em nuvem

Armazenamento em nuvem é um mal necessário em um mundo em que funcionários exigem acesso aos dados em qualquer lugar usando qualquer tipo de dispositivo que, por acaso, esteja à mão. Os usuários têm todo tipo de solução disponível para armazenamento em nuvem, mas um dos mais populares atualmente é o Dropbox (<https://www.dropbox.com/>), que havia angariado mais de 300 milhões de usuários até o final de 2014. O Dropbox (e a maioria das outras entidades para armazenamento em nuvem) tem uma história de altos e baixos quanto à segurança. Por exemplo, em 2011, o Dropbox teve um bug em que qualquer um podia acessar qualquer conta usando qualquer senha durante um período de quatro horas (veja o artigo da

InformationWeek em <http://www.darkreading.com/vulnerabilities-and-threats/dropbox-files-left-unprotected-open-to-all/d/d-id/1098442>). É claro que todos esses fornecedores dirão que os dados de sua aplicação estarão protegidos, agora que eles melhoraram a segurança. Não é uma questão de saber se um hacker achará um caminho para entrar no serviço de armazenamento em nuvem ou se o próprio serviço vai pisar na bola novamente, mas de saber quando isso ocorrerá.

A segurança de aplicações web é importante porque a maioria das aplicações, se não todas, em algum momento, terá uma aplicação web como base. Usuários querem que suas aplicações estejam disponíveis em todos os lugares, e o navegador é praticamente o único meio de oferecer esse tipo de funcionalidade em tantas plataformas diferentes, de modo eficiente. Em suma, você deve pensar nos problemas do armazenamento em nuvem desde o início. Há várias opções para lidar com armazenamento em nuvem como parte da estratégia de sua aplicação:



Um problema importante na maior parte do armazenamento em nuvem é que ele é público por natureza. Por exemplo, o Dropbox for Business parece ser uma ótima ideia e oferece recursos adicionais de segurança, mas o serviço continua sendo público. Um negócio não pode hospedar o serviço em sua nuvem privada.

Além disso, a maioria dos serviços em nuvem afirma que eles criptografam os dados em seus servidores, o que provavelmente é verdade. No entanto, o provedor do serviço normalmente guarda consigo as chaves de criptografia argumentando que devem conceder acesso aos dados às autoridades que tiverem mandatos judiciais. Por não ter as chaves de seus dados criptografados, você não pode controlar o acesso a eles, e a criptografia é menos útil do que você imagina.

Bloquear o acesso

Na verdade, é possível bloquear todo o acesso a dados na nuvem usando um firewall, uma política ou um recurso da aplicação. No entanto, a capacidade de bloquear o acesso de todos os lugares em que um usuário possa querer acessar dados na nuvem é extremamente difícil e os usuários são bastante determinados. Além disso, bloquear o acesso pode ter efeitos negativos nas reuniões de negócio. Por exemplo, parceiros podem optar por usar o armazenamento em nuvem como um método para troca de arquivos grandes. Uma estratégia de bloqueio também faz os usuários ficarem com

raiva, de modo que eles não trabalharão com sua aplicação ou acharão maneiras de contornar a funcionalidade que você procurava oferecer. Essa será a melhor opção se sua empresa tiver de administrar grandes quantidades de dados sensíveis, houver requisitos legais para proteger os dados ou simplesmente não precisar da flexibilidade do armazenamento em nuvem.

Permitir acesso sem controle

Você poderia optar por ignorar os problemas envolvidos no uso do armazenamento em nuvem. Entretanto, uma política como essa deixa sua empresa suscetível à perda e violação de dados e a todo tipo de outros problemas. Infelizmente, muitas empresas atualmente usam essa estratégia porque controlar o acesso dos usuários tornou-se muito difícil e a empresa não tem meios para usar outro tipo de abordagem.

Contar com locais seguros controlados pela empresa

Se você precisa que os usuários acessem dados na nuvem usando uma conta da empresa, você pode, pelo menos, monitorar o uso de arquivos e ter meios para recuperar os dados quando um funcionário sair da empresa. No entanto, os problemas básicos com o armazenamento em nuvem permanecem. Um hacker com o conhecimento certo ainda poderia acessar a conta e obter seus dados ou simplesmente optar por espionar você de outras maneiras. Essa opção funcionará bem se sua empresa não gerencia dados com medidas de proteção exigidas legalmente e você estiver disposto a trocar um pouco de segurança por conveniência.

Controlar o acesso dentro da aplicação

Muitos serviços em nuvem suportam uma API que permite interagir com o serviço de maneiras únicas. Embora essa abordagem consuma bastante tempo, ela tem a vantagem de permitir que você controle o local em que o usuário armazena dados sensíveis, ao mesmo tempo que lhe oferece a flexibilidade para usar armazenamento em nuvem para dados menos sensíveis. Considere essa solução quando sua empresa precisar interagir com um grande número de parceiros, ao mesmo tempo que precisar também administrar grandes quantidades de dados sensíveis ou críticos.

Contar com uma solução de terceiros

Você pode encontrar soluções de terceiros, como a da Accellion (<http://www.accellion.com/>), que oferece conectores para armazenamento em nuvem. O fornecedor oferece um serviço que atua como um ponto intermediário entre sua aplicação e o armazenamento de dados online. O usuário é capaz de interagir naturalmente com os dados, mas o serviço controla o acesso usando políticas que você define. O problema com essa abordagem é que agora você passa a ter uma camada adicional para considerar quando escrever a aplicação. Além disso, você deve confiar no terceiro que oferece o conector. Essa solução em particular funciona bem quando você precisa de flexibilidade, sem os custos usuais de desenvolvimento, e não quer criar sua própria solução que dependa de acesso a API.

Usando códigos e recursos externos

A maioria das empresas hoje em dia não tem tempo nem recursos necessários para criar aplicações completas do zero. Além disso, os custos para manter aplicações desse tipo seriam enormes. Para ter os custos sob controle, as empresas normalmente contam com códigos de terceiros em várias formas. Esse código executa tarefas comuns e os desenvolvedores o utilizam para criar aplicações, como se fossem peças de Lego. Por outro lado, um código de terceiros não deixa de ter desafios quanto à segurança. Você estará realmente dependendo de outra pessoa para escrever o código da aplicação que não só funcione bem e execute todas as tarefas necessárias, mas faça isso de forma segura. As próximas seções descrevem alguns dos problemas relacionados ao uso de códigos e de recursos externos.

Definindo o uso de bibliotecas

Uma biblioteca é qualquer código que você adiciona à sua aplicação. Muitas pessoas definem bibliotecas de forma mais ampla, mas, neste livro, o essencial é que as bibliotecas contêm código e elas se tornam parte de sua aplicação quando é usada. Uma biblioteca comumente utilizada é a jQuery (<https://jquery.com/>). Ela oferece diversas funcionalidades para executar

tarefas comuns em uma aplicação. Um fato interessante sobre a jQuery é que você encontrará os termos *biblioteca* (library) e *API* usados indistintamente, como mostra a figura 1.2.

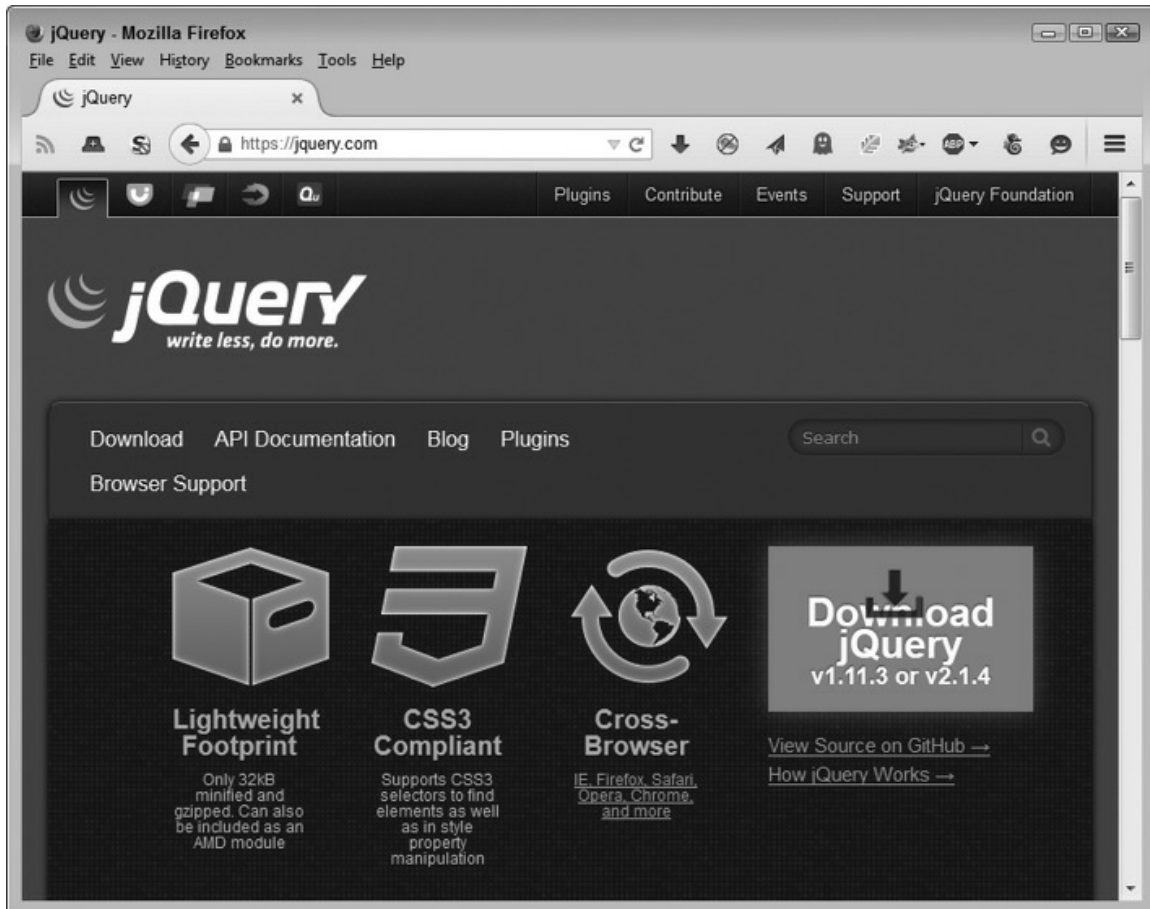


Figura 1.2 – Muitos sites usam os termos biblioteca (library) e API indistintamente.

Ao observar o site da jQuery, também vemos informações sobre configurações otimizadas da biblioteca. De fato, o modo como a jQuery se apresenta é um bom modelo para qualquer biblioteca que você queira usar. A biblioteca é totalmente documentada, e você pode encontrar vários exemplos de cada chamada à biblioteca (para garantir que seja possível encontrar um exemplo que seja ao menos semelhante àquilo que você quer usar). Acima de tudo, os exemplos são funcionais, portanto é possível realmente ver o código em ação usando os mesmos navegadores que você planeja usar em sua própria aplicação.

Como qualquer outro software, a jQuery também tem seus defeitos. À medida



que avançarmos no livro, você conhecerá outras bibliotecas e verá mais detalhes sobre cada uma; desse modo, você começará a perceber que recursos e segurança andam de mãos dadas. Pelo fato de ser uma biblioteca grande e complexa, a jQuery tem muito a oferecer, mas apresenta também uma superfície maior de ataque para os hackers explorarem.

Ao trabalhar com bibliotecas, o foco principal de seus esforços quanto à segurança é a sua aplicação, pois você fará download do código usado na aplicação a partir do servidor host. O código da biblioteca é executado durante o funcionamento, portanto você precisa saber se pode confiar na fonte a partir da qual esse código é obtido. O capítulo 6 discute as particularidades do uso de bibliotecas como parte da estratégia de desenvolvimento de uma aplicação.

Definindo o uso de APIs

Uma API (Application Programming Interface, ou Interface de Programação de Aplicativos) é qualquer código que você pode chamar como um serviço fora do processo. Você envia uma solicitação à API e ela responde com algum recurso que você requisitou. O recurso normalmente é um dado de algum tipo, mas as APIs podem executar outros tipos de tarefa também. A ideia é que o código fique em outro sistema e não faça parte de sua aplicação. Como as APIs funcionam com uma configuração do tipo solicitação/resposta, elas tendem a oferecer um suporte mais amplo a diferentes plataformas que as bibliotecas, mas também são mais lentas, pois o código não está no mesmo local em que está o sistema que o utiliza.

Um bom exemplo de APIs são os serviços oferecidos pela Amazon para atender às várias necessidades dos desenvolvedores (<https://developer.amazon.com/>). A figura 1.3 mostra apenas alguns desses serviços. Você deve se inscrever para cada API que queira usar, e a Amazon oferece uma chave especial para ser usada na maioria dos casos. Como você estará interagindo com os servidores da Amazon, e não estará apenas fazendo download de código para o seu próprio sistema, as regras de segurança são diferentes quando usamos uma API.

Cada API tende a ter uma vida própria e conta com diferentes abordagens

para questões como gerenciar dados. Consequentemente, você não pode fazer nenhuma suposição quanto à segurança de uma API quando comparada a outra, mesmo que ambas as APIs estejam no mesmo host.

As APIs também dependem de troca de informações. Qualquer troca de informação exige um esquema de segurança adicional, pois parte de seus dados invariavelmente vai parar no sistema host. Você precisa saber se o host oferece os meios para garantir apropriadamente a segurança dos dados que você transmitir como parte de uma solicitação. O capítulo 7 discute como trabalhar com APIs de forma segura quando usá-las como parte da estratégia de desenvolvimento de uma aplicação.

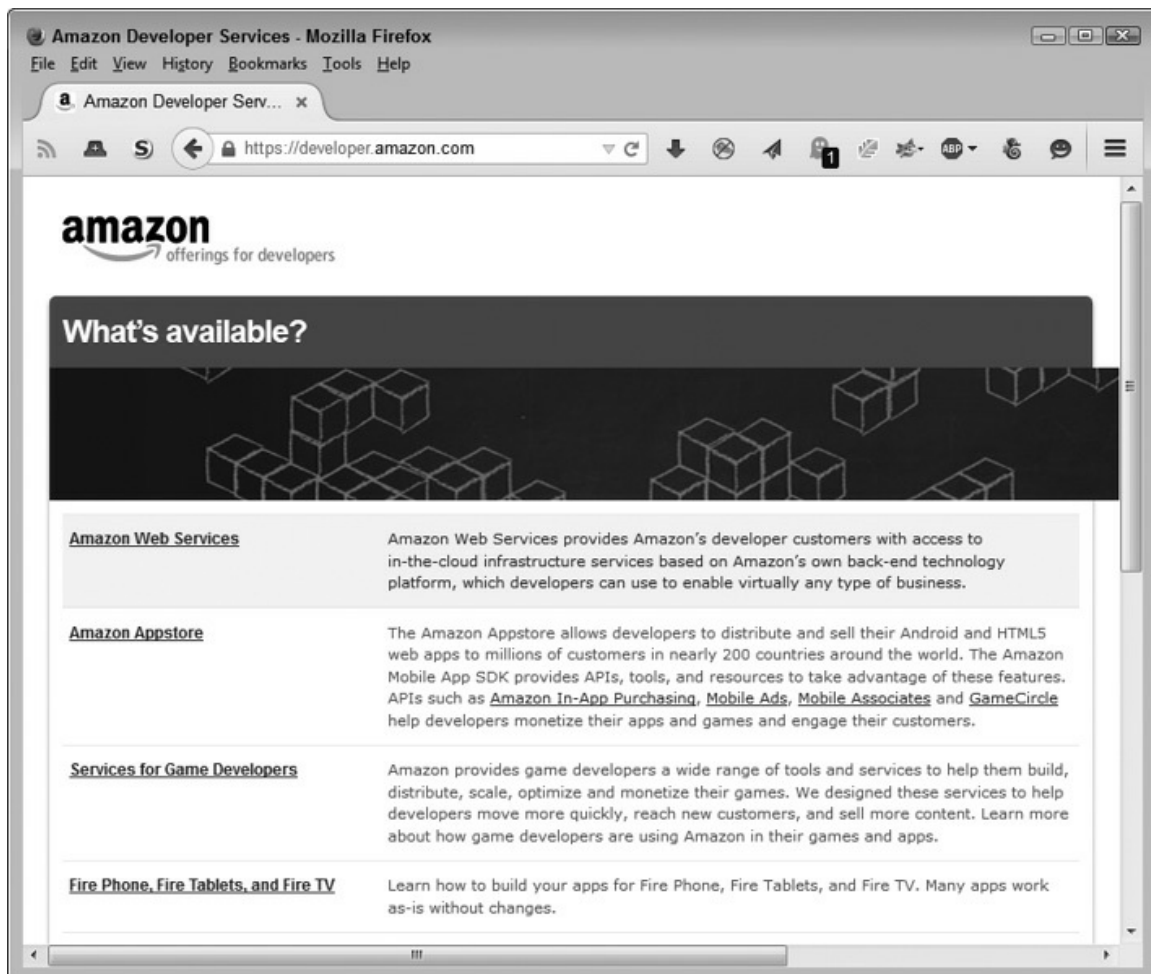


Figura 1.3 – A API da Amazon é um exemplo de código que executa em um servidor host.

Definindo o uso de microsserviços

Assim como uma API, os microsserviços executam em um sistema host. Você faz uma requisição e o microsserviço responde com algum tipo de recurso (normalmente, com dados). No entanto, os microsserviços diferem bastante das APIs. Com os microsserviços, a ênfase está em pequenas tarefas – um microsserviço típico executa bem uma única tarefa. Além disso, microsserviços tendem a focar intensamente na independência de plataforma. A ideia é oferecer um serviço que possa interagir com qualquer plataforma de qualquer tamanho e qualquer capacidade. A diferença de ênfase entre APIs e microsserviços afeta bastante a área de segurança. Por exemplo, as APIs tendem a ter mais ciência de segurança porque o host pode fazer mais suposições sobre a plataforma que faz a requisição e há mais a perder caso algo dê errado.

As ofertas atuais de microsserviços também tendem a ser soluções domésticas, pois a tecnologia ainda é nova. Dê uma olhada nos sites de API atuais que começarão a oferecer microsserviços, uma vez que a tecnologia se desenvolva. Enquanto isso, vale a pena analisar exatamente como os microsserviços diferem observando sites como da *Microservice Architecture* (<http://microservices.io/>). O site oferece exemplos de aplicações e uma discussão dos vários padrões em uso atualmente para código online, como mostra a figura 1.4.

As trabalhar com microsserviços, você deve garantir que o host seja confiável e que a tarefa única executada pelo microsserviço esteja claramente definida. Também é importante considerar como o microsserviço interage com qualquer dado fornecido, e você não deve partir do pressuposto que todo microsserviço interage com os dados da mesma maneira (mesmo que os microsserviços estejam no mesmo host). O uso de microsserviços implica eficiência e capacidade de trabalhar com uma variedade maior de plataformas, mas você também deve estar ciente da necessidade de fazer verificações adicionais de segurança. O capítulo 8 discute como usar microsserviços de forma segura como parte da estratégia de desenvolvimento de uma aplicação.

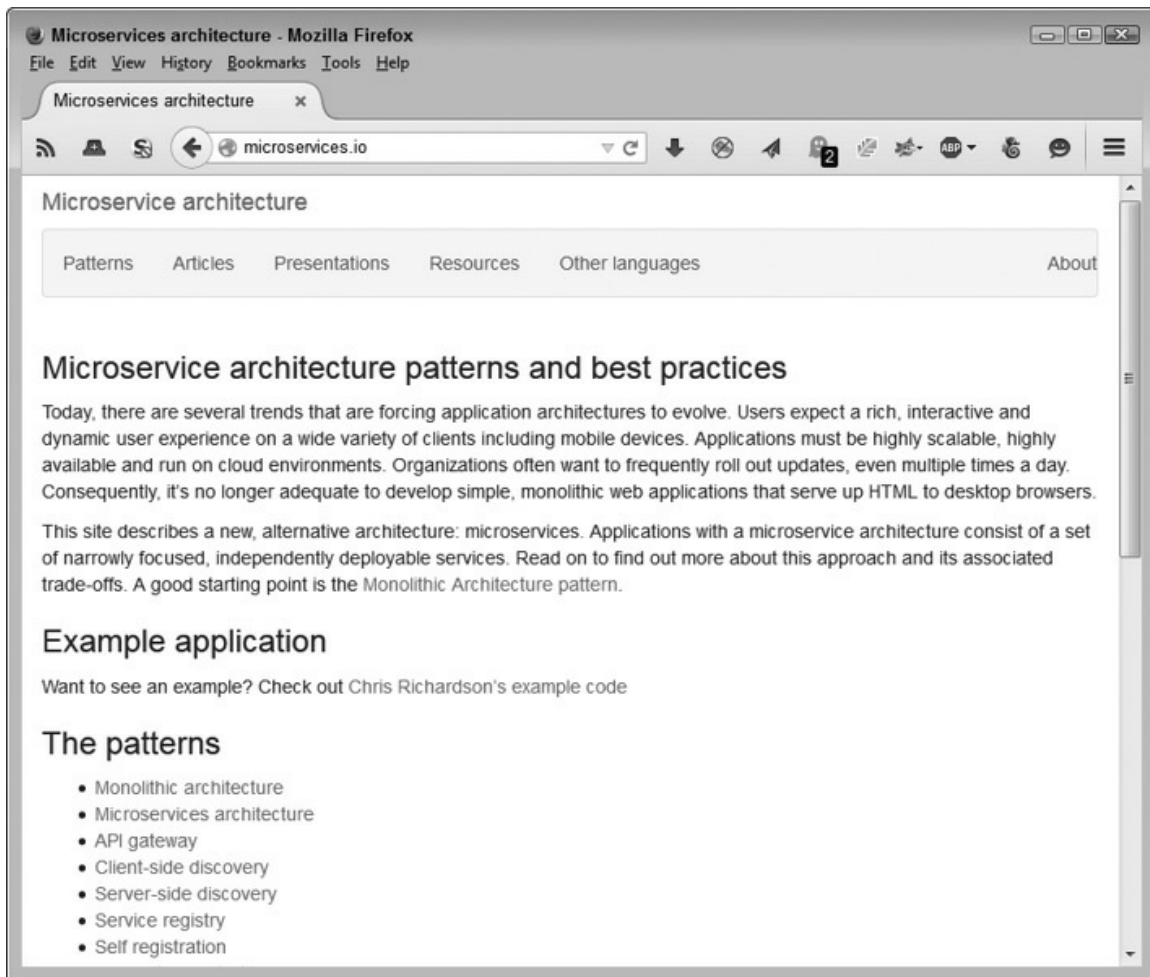


Figura 1.4 – Microserviços são uma tecnologia nova e vê-los à luz de outros padrões de uso pode ajudar.

Acessando dados externos

Dados externos assumem todo tipo de formato. Qualquer formato de dado externo é suspeito porque alguém pode adulterá-lo no simples ato de acessá-lo. Entretanto, classificar os dados de acordo com sua visibilidade é útil quando pensamos em requisitos de segurança.

Normalmente, você pode considerar fontes privadas de dados como relativamente seguras. Continua sendo necessário verificar os dados para ver se contêm elementos nocivos em potencial (por exemplo, scripts criptografados em campos do banco de dados). No entanto, na maior parte dos casos, a fonte de dados não estará tentando causar problemas propositalmente. A seguir, apresentamos as fontes mais comuns de

armazenamento de informações como parte de fontes de dados externas privadas:

- dados em hosts de sua própria empresa;
- dados em hosts de parceiros;
- fontes de dados calculados, criadas por aplicações que executam nos servidores;
- dados importados de sensores ou que tenham outras origens aceitas pela empresa.

Fontes pagas de dados também são relativamente seguras. Qualquer empresa que conceda acesso a dados pagos quer preservar seu relacionamento com ela, e reputação é tudo nessa área. Como ocorre com fontes locais e privadas, é preciso verificar se os dados não estão corrompidos e se estão livres de ameaças em potencial, por exemplo, se não contêm scripts. No entanto, como os dados também passam por uma rede pública, é preciso verificar se não houve manipulação de dados ou outros problemas em potencial, resultantes de causas como ataques man-in-the-middle.



Há muitos repositórios interessantes online que você poderá achar úteis quando criar uma aplicação. Em vez de gerar os dados por conta própria ou contar com uma fonte de dados paga, muitas vezes você poderá encontrar uma fonte de dados gratuita. Sites como o *Registry of Research Data Repositories* (Registro de repositórios de dados para pesquisa) atualmente oferecem APIs para que você possa procurar o repositório correto de dados com mais precisão. Nesse caso, você pode encontrar a documentação da API em <http://service.re3data.org/api/doc>.

Repositórios de dados podem ser problemáticos, e quanto mais público o repositório, mais problemático ele se torna. O uso que você faz de um repositório de dados faz diferença, mas garantir que você está realmente obtendo dados, e não algo diferente disfarçado, é essencial. Em muitos casos, você pode fazer download dos dados e analisá-los antes de utilizá-los. Por exemplo, o site da WHO (World Health Organization, ou Organização Mundial da Saúde) mostrado na figura 1.5 oferece meios para filtrar seus repositórios de dados e encontrar exatamente os dados de que você precisa; com isso, é possível fazer download somente desse conjunto de dados e

reduzir o risco de obter algo que você não queria.

Há muitos tipos de repositório de dados e muitas maneiras de acessar os dados que eles contêm. A técnica usada depende da interface do repositório de dados e dos requisitos de sua aplicação. Lembre-se de ler sobre repositórios de dados no capítulo 3. Os capítulos 6, 7 e 8 discutem o uso de dados externos conforme se aplicam a bibliotecas, APIs e microsserviços. Cada ambiente tem requisitos diferentes, portanto é importante entender exatamente como seu código afeta o método de acesso e, desse modo, a segurança necessária para usar os dados sem que haja perigo.

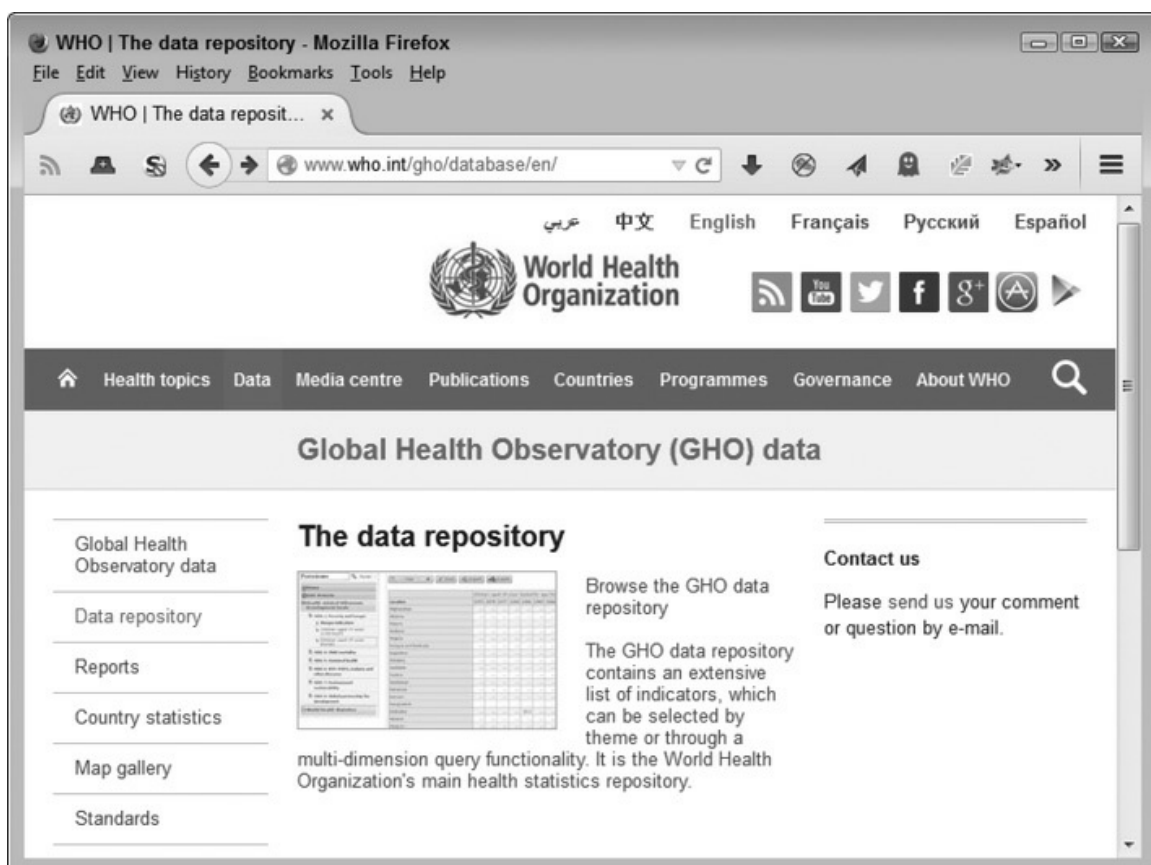


Figura 1.5 – É possível fazer download de alguns repositórios, como o banco de dados da OMS.

Permitindo acesso a outros

A maior parte deste capítulo discute a proteção de seus recursos ou o uso de dados e recursos fornecidos por outros. As empresas não existem no vácuo.

Ao criar uma fonte de dados, uma biblioteca, uma API, um microserviço ou outro recurso que outros possam usar, o terceiro normalmente solicita acesso. Assim que você concede acesso ao recurso a esse terceiro, você abrirá sua rede a possíveis ameaças de segurança que você nem sequer imagina. É provável que o parceiro de negócios ou outras entidades sejam bastante confiáveis e não tenham a intenção de causar problemas. No entanto, os vírus deles passam a ser seus vírus. Qualquer ameaça à segurança que eles enfrentem se tornará uma ameaça à segurança para você também. Se você tiver um número suficiente de terceiros usando seus recursos, há muitas chances de pelo menos um deles ter um problema de segurança que poderá acabar causando problemas a você também.

É claro que antes de poder fazer algo é preciso garantir que sua oferta externa seja tão boa quanto você acha que é. Garantir a segurança das aplicações às quais você dá suporte é essencial. Como fornecedor de recursos, você se transforma de repente em um ponto único de falha para várias entidades externas. Manter o ambiente seguro significa:

- testar todos os recursos regularmente para garantir que eles permaneçam viáveis;
- oferecer documentação conveniente para os recursos;
- garantir que desenvolvedores terceirizados cumpram as regras (tomando atitudes como incluir requisitos de segurança no contrato);
- executar testes de segurança conforme necessário;
- manter-se atualizado em relação às potenciais ameaças aos seus recursos;
- atualizar recursos do host para evitar vulnerabilidades conhecidas.

Desenvolvedores que oferecem seus recursos para uso externo têm outras questões a considerar também, mas elas são suficientemente comuns a ponto de serem consideradas como conhecidas para qualquer cenário de acesso externo. Além disso, você deve esperar que os terceiros testem os recursos para garantir que eles funcionarão conforme anunciado. Por exemplo, ao oferecer uma biblioteca, uma API ou um microserviço, você deve esperar que os terceiros realizem testes de entrada e de saída, em vez de simplesmente acreditarem em sua palavra ao afirmar que o recurso se

comportará conforme esperado.

Após passar pela fase inicial de oferecimento do recurso para um terceiro usar, você deve fazer sua manutenção para que as aplicações possam continuar a contar com ele. Além do mais, é importante partir do pressuposto que você vai enfrentar determinadas ameaças ao oferecer um recurso. Eis algumas questões que você deve considerar:

- Hackers tentarão usar seu recurso para ter acesso ao seu site.
- Desenvolvedores usarão o recurso incorretamente e tentarão fazer com que ele execute tarefas para as quais ele não foi projetado.
- De alguma maneira, o recurso será comprometido.

CAPÍTULO 2

Acolhendo as necessidades e expectativas dos usuários

A segurança não funcionará a menos que você seja capaz de convencer o usuário a acolhê-la. Qualquer desenvolvedor draconiano de dispositivos que se esforce para garantir a segurança sem a bênção do usuário em algum momento falhará porque os usuários são adeptos de encontrar maneiras de contornar a segurança. Em situações em que a segurança realmente seja completa o suficiente para frustrar até mesmo as tentativas mais árduas dos usuários, eles simplesmente se recusarão a usar a aplicação. Listas longas de aplicações que falharam atestam o fato de que não é possível garantir a segurança sem algum nível de assistência do usuário, portanto é melhor garantir que o usuário esteja de acordo com qualquer decisão que você tome.

Usuários têm dois níveis de requisitos para uma aplicação, e a segurança deve incluir ambos. Um usuário deve ter a liberdade de executar tarefas exigidas pelo seu trabalho. Quando uma aplicação falha em atender às necessidades do usuário, ela simplesmente falha como aplicação. As expectativas do usuário somam-se às suas necessidades. Usuários esperam que a aplicação funcione em seus dispositivos pessoais, além de funcionar em dispositivos fornecidos pelas empresas. Dependendo da aplicação, garantir que a aplicação e sua segurança funcionem na maior variedade possível de plataformas gera simpatia, o que facilita vender requisitos de segurança ao usuário.

Este capítulo discute tanto as necessidades quanto as expectativas conforme elas se relacionam com a segurança. Para muitos desenvolvedores, o objetivo de escrever código é criar uma aplicação que funcione bem e atenda a todos os requisitos. No entanto, o verdadeiro objetivo de escrever código é criar um ambiente em que o usuário possa interagir com os dados com sucesso e de

forma segura. Os dados são o foco da aplicação, mas o usuário é o meio para fazer as modificações orientadas a dados acontecerem.

Desenvolvendo uma visão de usuário à aplicação

Usuários e desenvolvedores muitas vezes estão em desacordo quanto à segurança porque veem as aplicações de maneiras significativamente diferentes. Os desenvolvedores veem um código escrito cuidadosamente, que faz todo tipo de tarefas interessantes; os usuários veem um meio para um fim. Com efeito, o usuário pode não ver realmente a aplicação. Tudo que interessa ao usuário é obter um relatório ou compor outro produto de acordo com determinado prazo final. Para os usuários, as melhores aplicações são invisíveis. Quando a segurança entra no caminho que torna a aplicação invisível, ela se torna um problema que o usuário deseja contornar. Em suma, deixar tanto a aplicação quanto sua segurança o mais invisível possível é sempre desejável, e quanto melhor você atingir essa meta, mais o usuário gostará de sua aplicação.



Eventos atuais realmente podem colocar desenvolvedores e usuários em desacordo. Eventos como o hack Ashley Madison (<http://www.computerworld.com/article/2982959/cybercrime-hacking/ashley-madison-coding-blunder-made-11m-passwords-easy-to-crack.html>) deixaram desenvolvedores especialmente preocupados e menos propensos a fazer acomodações. Em alguns casos, um especialista em interface ou experiência de usuário pode mediar uma solução entre os dois grupos que seja vantajosa para todos. Vale a pena pensar de forma mais ampla quando se trata de problemas de segurança, em que emoções podem estar à flor da pele e boas soluções podem ser menos aceitáveis a um ou outro grupo.

O problema com os desenvolvedores é que eles realmente não pensam como usuários. O aspecto interessante da tecnologia abstrata simplesmente é atraente demais para qualquer desenvolvedor resistir. Uma equipe de desenvolvimento deve incluir pelo menos um representante de usuário (que verdadeiramente represente usuários típicos em sua organização). Além disso, você deve incluir usuários como parte do processo de testes. Um esquema de segurança que não funcione é mais fácil de corrigir quando é identificado com antecedência no processo de desenvolvimento, como qualquer outro bug em sua aplicação. Quando o sistema de segurança é

desajeitado, inconveniente, óbvio ou simplesmente irritante, ele não funciona, mesmo que realmente proteja os dados da aplicação.



Apesar de este livro não discutir a metodologia de desenvolvimento DevOps, você deve considerar seu uso como parte do design e da estratégia de desenvolvimento de sua aplicação. O DevOps (amalgama entre *Development* e *Operations*, ou seja, Desenvolvimento e Operações) enfatiza comunicação, colaboração, integração, automação e cooperação entre os stakeholders (partes interessadas) de qualquer processo de desenvolvimento de aplicações. Você pode encontrar muitos recursos para DevOps em <http://devops.com/>. Várias pessoas tentaram descrever o DevOps, mas uma das dissertações mais esclarecedoras é o artigo no blog de Patrick Debois (<http://www.jedi.be/blog/2010/02/12/what-is-this-devops-thing-anyway/>). O artigo não é muito recente, mas continua a oferecer uma ótima visão geral do que é DevOps, quais os problemas que ele resolve e como você pode empregá-lo em sua própria empresa.

A segurança é um problema que uma empresa como um todo deve resolver. Como desenvolvedor, se você é o único que está tentando criar uma solução, então é quase certo que ela falhará. A visão que o usuário tem da aplicação é essencial para que os usuários se comprometam com a estratégia de segurança que você definir para uma aplicação. Entretanto, você também deve incluir:

- gerência (para garantir que os objetivos organizacionais sejam alcançados);
- departamento jurídico (para garantir que a proteção de dados atenda aos requisitos do governo);
- recursos humanos (para garantir que você não invada o espaço de outras pessoas);
- suporte (para garantir que qualquer treinamento necessário ocorra);
- todos os demais stakeholders envolvidos na definição das políticas de negócio que controlam o gerenciamento dos dados.

Afinal de contas, segurança tem tudo a ver com garantir que as regras de gerenciamento de dados sejam cumpridas. Não se trata somente de garantir que um hacker não possa, de alguma maneira, encontrar o diretório oculto utilizado para armazenar os dados. Segurança é proteger os dados de forma bem fundamentada, de modo que os usuários possam fazer alterações

responsáveis, ao mesmo tempo que alterações prejudiciais são evitadas.

Considerando os problemas de BYOD (Bring Your Own Device)

Usuários trarão seus próprios dispositivos de casa e os usarão para acessar sua aplicação – acostume-se a isso. Teoricamente, você poderia criar métodos para detectar quais dispositivos estão acessando sua aplicação, mas o fato é que os usuários encontrarão maneiras de contornar essa identificação em muitos casos. Criar aplicações que funcionem bem em um ambiente BYOD (Bring Your Own Device, ou Traga seu Próprio Dispositivo) é mais difícil que trabalhar com uma configuração exata de hardware, mas você pode conseguir bons resultados. A questão principal é pressupor que os usuários contarão com seus próprios dispositivos e criar o esquema de segurança da aplicação com esse objetivo em mente. As próximas seções descrevem os problemas que você enfrentará quando trabalhar com BYOD e apresentarão possíveis soluções a esses problemas.



Algumas empresas adotaram o BYOD por questões de economia, o que significa que os desenvolvedores nessas empresas não têm um dispositivo padrão para usar em testes. A empresa simplesmente supõe que o usuário terá um dispositivo adequado tanto para o trabalho quanto para o lazer. Se sua empresa aderiu a essa tendência, você não só precisará lidar com dispositivos BYOD como uma alternativa aos produtos da empresa como também deverá lidar com dispositivos BYOD como a única opção. Vale a pena saber quais tipos de dispositivo seus usuários possuem para que você possa ter uma base sobre a qual decidir quais tipos de dispositivo devem ser usados para testar a aplicação e como definir os critérios para os dispositivos que a utilizarão.

Entendendo a segurança de aplicações baseadas em web

A solução mais provável para possibilitar o BYOD é criar aplicações baseadas em web para todas as necessidades. Um usuário poderia então contar com um smartphone, um tablet, um notebook, um PC ou qualquer outro dispositivo que tenha um navegador apropriado para acessar a aplicação. Entretanto, é notoriamente difícil garantir a segurança de aplicações baseadas em web. Pense nos requisitos para oferecer algo

diferente de segurança baseada em senha para todos os dispositivos que existem por aí. De fato, a senha pode nem mesmo ser uma senha no verdadeiro sentido da palavra – você poderá se ver limitado a um PIN (Personal Identification Number, ou Número de Identificação Pessoal). O elo mais fraco de segurança para aplicações baseadas em web, na maioria dos casos, é o dispositivo móvel. Para deixar os dispositivos móveis mais seguros, você deve considerar os seguintes passos:

1. Envolver todos os stakeholders de uma aplicação – incluindo usuários, CIO (Chief Information Officer, ou Diretor de TI), CISO (Chief Information Security Officer, ou Diretor de Segurança da Informação), recursos humanos e outras pessoas fora da equipe de desenvolvimento – no processo de tomada de decisão para os recursos da aplicação. É preciso que esse seja o primeiro passo, pois esses indivíduos ajudarão você a criar uma estratégia que foque tanto nas necessidades do usuário quanto nas necessidades do negócio, ao mesmo tempo que permitirão que você enfatize os problemas em torno de expectativas não atendidas (e não das necessidades).
2. Desenvolver uma estratégia de segurança para dispositivos móveis e defini-la por escrito. O problema em criar acordos durante reuniões e não os formalizar é que as pessoas tendem a se esquecer do que foi combinado e isso se torna um problema mais adiante no processo. Depois que tiver uma estratégia formalizada, certifique-se de que todos estejam cientes dela e que a tenham lido. Em especial, isso é importante para os desenvolvedores que farão o design da aplicação.
3. Garantir que a gerência compreenda a necessidade de custear as medidas de segurança. A maioria das empresas atualmente sofre de falta de recursos quando se trata de segurança. Se uma equipe de desenvolvimento não tiver recursos para criar aplicações seguras, elas terão portas de entrada que os hackers explorarão. Os recursos financeiros para dar suporte aos esforços de desenvolvimento devem estar disponíveis antes de o processo de desenvolvimento começar.
4. Adquirir as ferramentas corretas para criar aplicações seguras. Sua equipe de desenvolvimento precisa das ferramentas apropriadas desde o

início, ou não será possível atingir o resultado desejado. Em muitos casos, os desenvolvedores não alcançam os objetivos de segurança porque não têm as ferramentas adequadas para implementar a estratégia de segurança. As ferramentas normalmente necessárias afetam as seguintes áreas:

- a. Autenticação de usuário ou no sistema
 - b. Criptografia de dados
 - c. Gerenciamento de dispositivos móveis
 - d. Proteção antivírus comum
 - e. Suporte a VPN (Virtual Private Network, ou Rede Privada Virtual) – quando necessário
 - f. Prevenção contra perda de dados
 - g. Prevenção contra invasão de host
5. Criar uma parceria com uma empresa que tenha conhecimento especializado em segurança (se for necessário). Em muitos casos, sua empresa não terá uma equipe de desenvolvimento com as habilidades necessárias. Adquirir esse tipo de habilidade de outra empresa que já tenha implantado diversas aplicações baseadas em web com sucesso fará sua empresa economizar tempo e esforço.
6. Começar o esforço de desenvolvimento. Dê início ao esforço de desenvolvimento somente após ter criado um sistema robusto de suporte para sua aplicação. Se seguir esses passos, você criará um ambiente em que a segurança fará parte da aplicação web desde o início, em vez de ser encaixada depois.



Um relatório da IDG Research Services de 2014 baseado em pesquisas com profissionais de TI e de segurança descreve diversos problemas em torno do uso de dispositivos móveis. A principal preocupação (mencionada por 75% dos que responderam à pesquisa) é o vazamento de dados – algo que a empresa solicita que os desenvolvedores evitem por meio de restrições à aplicação. Dispositivos perdidos ou roubados aparecem em seguida com 71%, seguidos de acesso não seguro à rede (56%), malware (53%) e Wi-Fi sem segurança (41%).

Considerando problemas de aplicações nativas

A reação automática aos problemas em torno de aplicações baseadas em web é usar aplicações nativas em seu lugar. Afinal de contas, os desenvolvedores entendem bem da tecnologia e é possível usar os recursos de segurança do sistema operacional para garantir que as aplicações protejam os dados conforme previsto originalmente. No entanto, os dias das aplicações nativas estão contados. Dar suporte a aplicações nativas está cada vez mais difícil à medida que o código se torna mais complexo. Além disso, oferecer acesso à sua aplicação a partir de uma variedade de plataformas significa que você terá as seguintes vantagens importantes:

- Melhor colaboração entre os funcionários
- Melhor serviço de atendimento ao cliente
- Acesso a informações corporativas de qualquer lugar
- Mais produtividade

É claro que há várias outras vantagens em dar suporte a várias plataformas, mas essa lista destaca o problema de usar aplicações nativas. Se realmente quiser usar aplicações nativas para garantir mais segurança, você deverá criar uma aplicação nativa para cada plataforma que queira aceitar, o que pode se tornar um empreendimento enorme. Para a maioria das empresas, simplesmente não vale a pena usar o tempo para criar as aplicações necessárias quando se analisa do ponto de vista de melhoria na segurança e mais controle da aplicação.



Algumas empresas tentam contornar o problema da aplicação nativa/baseada em web usando “gambiarras” que não funcionam bem em muitos casos. Por exemplo, usar uma interface iOS, Android ou baseada em web para sua aplicação nativa tende a resultar em erros e introduz problemas de segurança em potencial. Usar uma aplicação puramente baseada em web, na verdade, é melhor.

Usando navegadores customizados

Ao criar sua aplicação baseada em web, você pode optar pela solução do navegador customizado. Em alguns casos, isso quer dizer escrever uma aplicação nativa que inclua funcionalidades de navegador. A aplicação nativa ofereceria recursos adicionais, pois é possível oferecer mais segurança; ao mesmo tempo, disponibilizar a aplicação baseada em web para smartphones

com acesso a recursos menos críticos manteria os usuários satisfeitos. Algumas linguagens, como C#, oferecem recursos relativamente funcionais de navegador customizado já prontos. No entanto, é possível criar um navegador customizado usando praticamente qualquer linguagem de aplicação.



É uma boa ideia discutir o uso do *kill switch* (botão da morte) de smartphones e tablets com sua empresa como parte da estratégia de desenvolvimento da aplicação. Um kill switch possibilita transformar um smartphone roubado em uma peça inútil. De acordo com um artigo do *USA Today* (<http://www.usatoday.com/story/tech/2015/02/11/kill-switch-theft-down/23237959/>), o uso de kill switches reduziu drasticamente o roubo de smartphones em muitas cidades grandes. Um artigo recente da *PC World* (<http://www.pcworld.com/article/2367480/10-things-to-know-about-the-smartphone-kill-switch.html>) oferece as informações necessárias para ajudar a gerência a entender como os kill switches funcionam. Em muitos casos, você deve instalar um software para deixar o kill switch disponível. Usar um kill switch pode parecer drástico, mas é melhor que permitir o acesso de hackers a dados corporativos sensíveis.

Além da segurança direta, a solução com navegador customizado também deixa as opções indiretas de segurança mais simples. Embora o controle usado em uma aplicação para criar o navegador customizado provavelmente ofereça todas as funcionalidades, o desenvolvedor da aplicação pode optar por não implementar determinados recursos. Por exemplo, você pode optar por não permitir que um usuário digite URLs ou conte com o recurso de histórico para passar de uma página para outra. A aplicação continuaria exibindo as páginas, como qualquer outro navegador, mas o usuário não poderá controlar a sessão do mesmo modo como o faz em um navegador padrão. Ter esse nível adicional de controle possibilita acessar informações mais sensíveis porque o usuário não será capaz de executar algumas tarefas que normalmente resultam em downloads de vírus, vazamento de informações ou contaminação do ambiente da aplicação. É claro que há exceções a toda regra. Se um vírus que se associa a bibliotecas e funções de navegador para roubar informações já estiver no sistema, é possível que ele ainda possa ler o conteúdo administrado por um navegador customizado que use essa biblioteca.

Um ambiente de navegador customizado também dá ao desenvolvedor a

oportunidade de contar com técnicas de programação que normalmente não funcionariam. Desenvolvedores passam por constantes problemas ao fazer bibliotecas, frameworks, APIs e microsserviços funcionarem apropriadamente porque nem todo navegador oferece o suporte necessário. Por exemplo, para determinar se um navegador em particular realmente suporta os recursos de HTML5 que você quer usar, é preciso verificá-lo usando um site como o HTML5test (<https://html5test.com/>) para obter uma lista das áreas de problemas em potencial, como mostra a figura 2.1.

O problema com a solução do navegador customizado é que ele introduz disparidades no suporte ao usuário, dependendo do dispositivo que o usuário escolher. Quando houver esse tipo de disparidade, o desenvolvedor normalmente ouvirá reclamações. Os usuários vão querer acessar os dados mais sensíveis da empresa no dispositivo menos seguro, no lugar mais público. Imagine trabalhar com registros de pacientes em um Starbucks usando um smartphone. Os dados poderiam acabar em qualquer lugar, e é quase certo que haverá uma violação de dados. Em algumas situações, o desenvolvedor simplesmente precisa trabalhar com todos, do nível de gerência para baixo, a fim de definir uma lista razoável de medidas de precaução para manipulação de dados, o que pode significar que o uso da solução de navegador customizado não será popular, mas garantirá o uso de políticas prudentes para gerenciamento de dados.

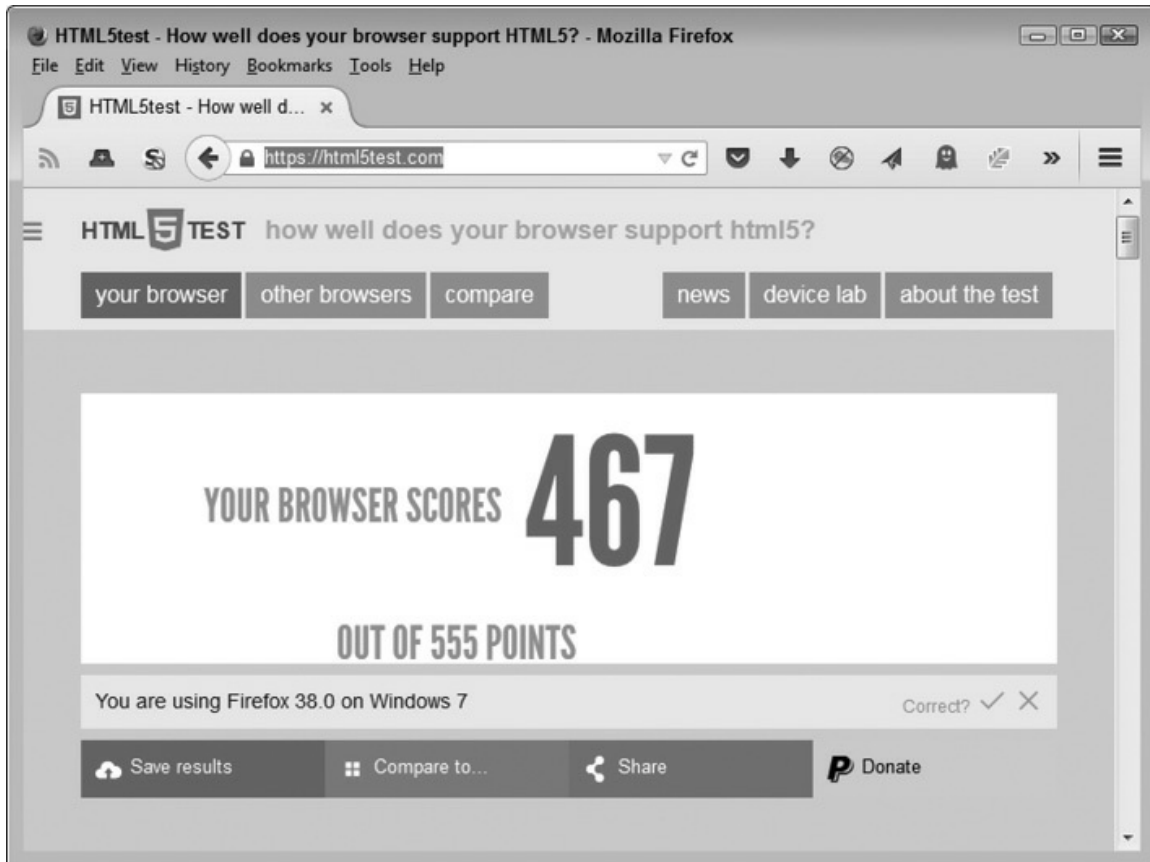


Figura 2.1 – Ao usar navegadores-padrão, você deve verificar se há problemas de suporte em potencial.

Verificando problemas de compatibilidade de código

O fenômeno BYOD implica que os usuários usarão todo tipo de dispositivo. É claro que isso é um problema. No entanto, um problema mais significativo é o fato de os usuários também terem softwares decrépitos nesses dispositivos porque softwares mais antigos são simplesmente mais confortáveis de usar. Como resultado do uso desses softwares antigos, sua aplicação parecerá ter problemas, mas o problema não é a aplicação – é uma questão de compatibilidade de código provocada pelo software realmente antigo. Com isso em mente, você precisará contar com soluções como o HTML5test (apresentado na seção anterior) para fazer verificações no software de um usuário de modo a garantir que ele atenda aos requisitos mínimos.

Outro método relacionado a essa questão é descobrir possíveis problemas de compatibilidade de código enquanto você escreve a aplicação. Por exemplo, a

figura 2.2 mostra o site W3Schools.com (<http://www.w3schools.com/tags/>), que disponibiliza a documentação do HTML5. Na parte inferior da página, você pode ver uma lista dos navegadores que suportam um recurso em particular e a versão necessária para aceitá-lo. Ao monitorar essas informações enquanto escreve sua aplicação, é possível reduzir os problemas de compatibilidade de código. No mínimo, você poderá informar aos usuários qual versão de determinado software eles devem ter para trabalhar com sua aplicação quando eles estiverem usando seus próprios dispositivos.

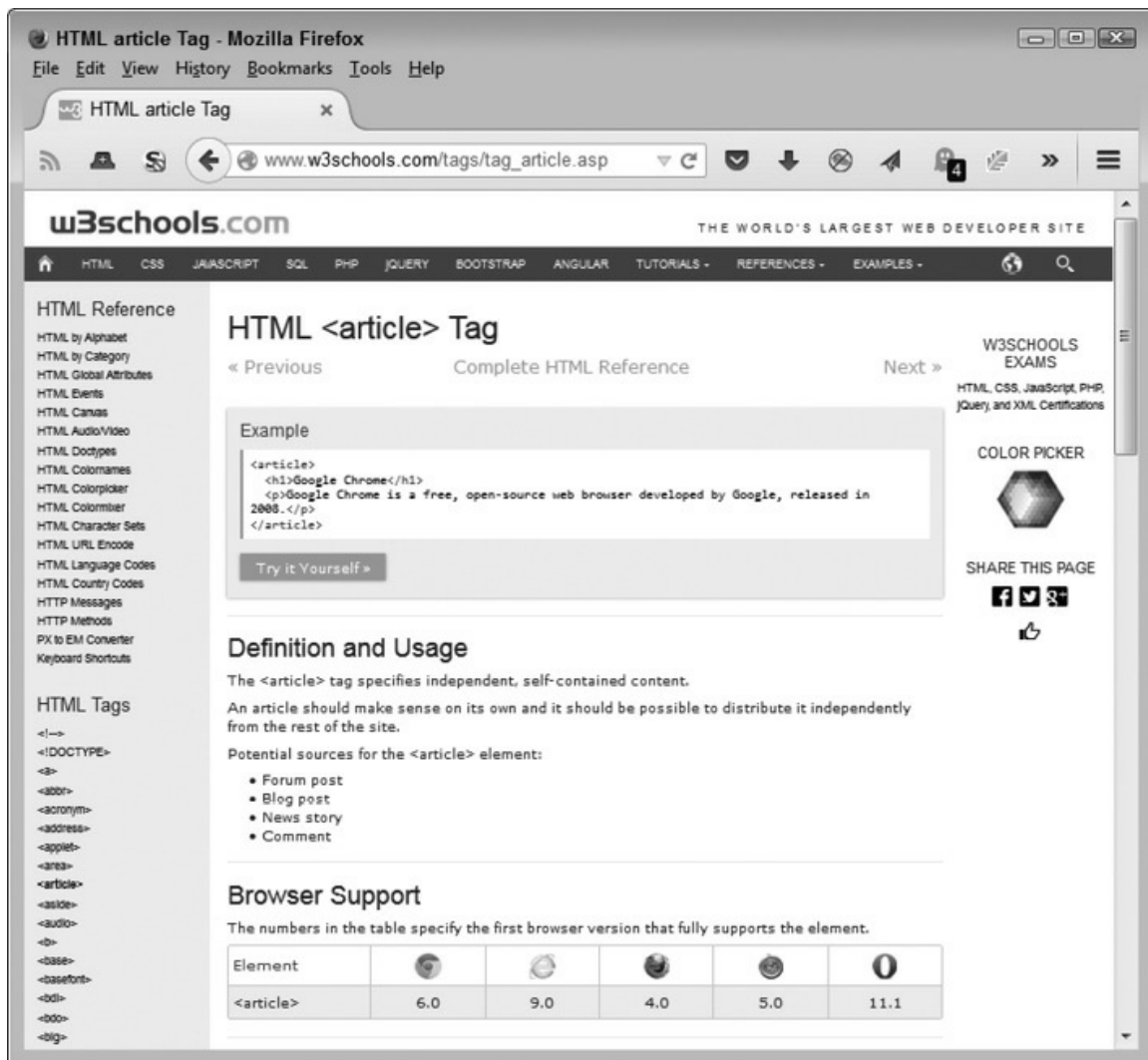


Figura 2.2 – Verifique se o recurso de código que você pretende usar realmente funciona com o software do usuário.

Também é importante observar que alguns sites também informam problemas de compatibilidade de forma sucinta. O site W3Schools.com também oferece

esse recurso. Observe que a lista de tags HTML mostrada na figura 2.3 informa quais tags o HTML5 suporta e quais não suporta. Ter essas informações à mão pode fazer você economizar durante o processo de implementação porque você não perderá um tempo considerável tentando descobrir por que um recurso em particular não funciona como deveria no sistema de um usuário.

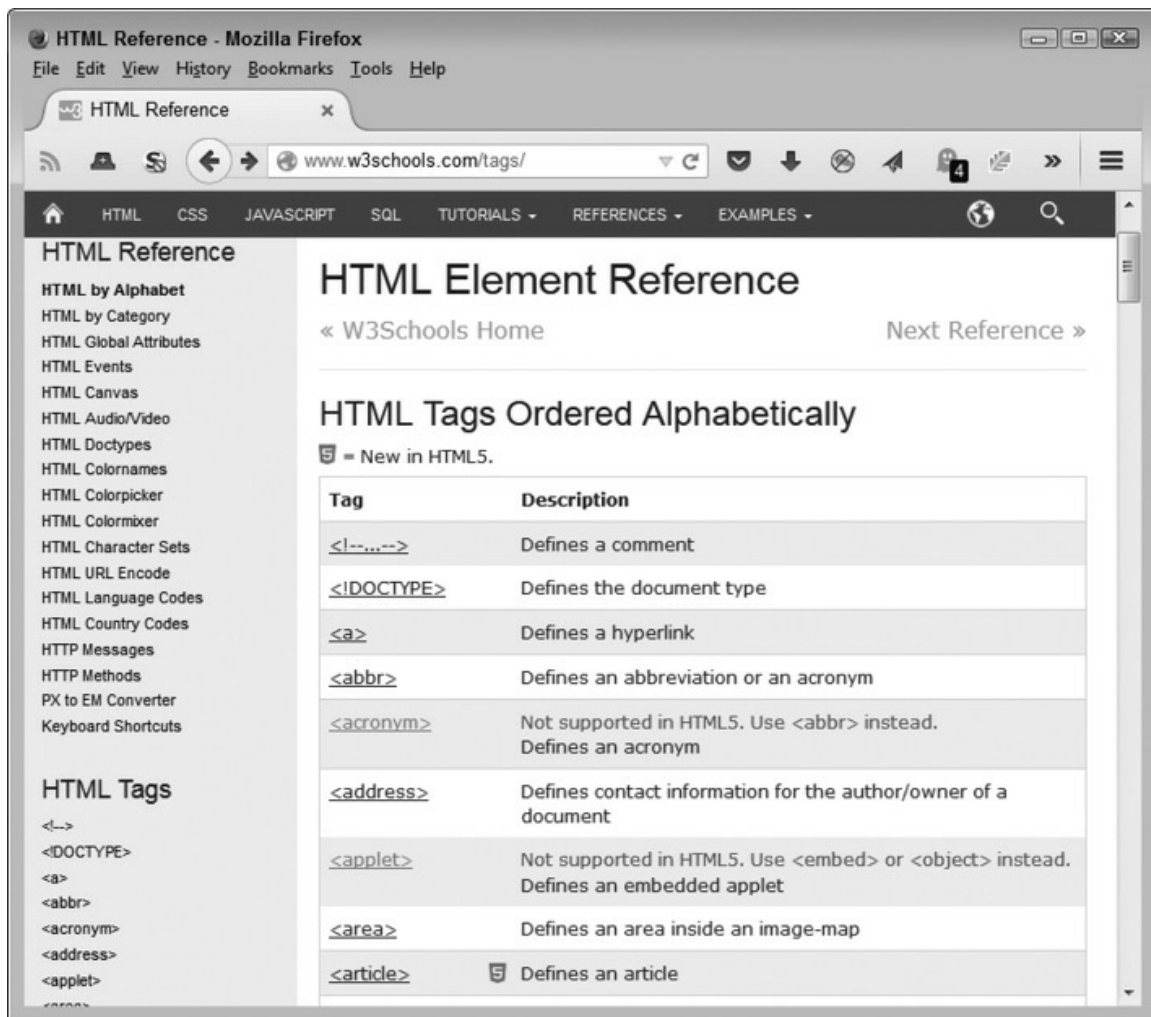


Figura 2.3 – Encontrar uma documentação que informe sobre questões ligadas a versões de forma sucinta é importante.

Um problema sério que a maioria dos desenvolvedores terá com compatibilidade de código é fazer o HTML5 ser apresentado de forma apropriada em navegadores mais antigos, para começar. Sites como http://www.w3schools.com/html/html5_browsers.asp oferecem algumas respostas que você pode usar. Por exemplo, o site mostra como usar `html5shiv`

para fazer o Internet Explorer aceitar elementos HTML5. O site [cdnjs](https://cdnjs.com/) (<https://cdnjs.com/>) contém uma grande variedade desses add-ins JavaScript úteis. Você pode encontrá-los em <https://cdnjs.com/libraries>. A figura 2.4 mostra apenas uma pequena lista das bibliotecas disponíveis. Infelizmente, você precisa encontrar exemplos para todas essas bibliotecas porque o site não oferece muitas informações úteis. A maior parte da documentação das bibliotecas está no GitHub. Por exemplo, você pode encontrar a documentação de *html5shiv.js* em <https://github.com/afarkas/html5shiv>.

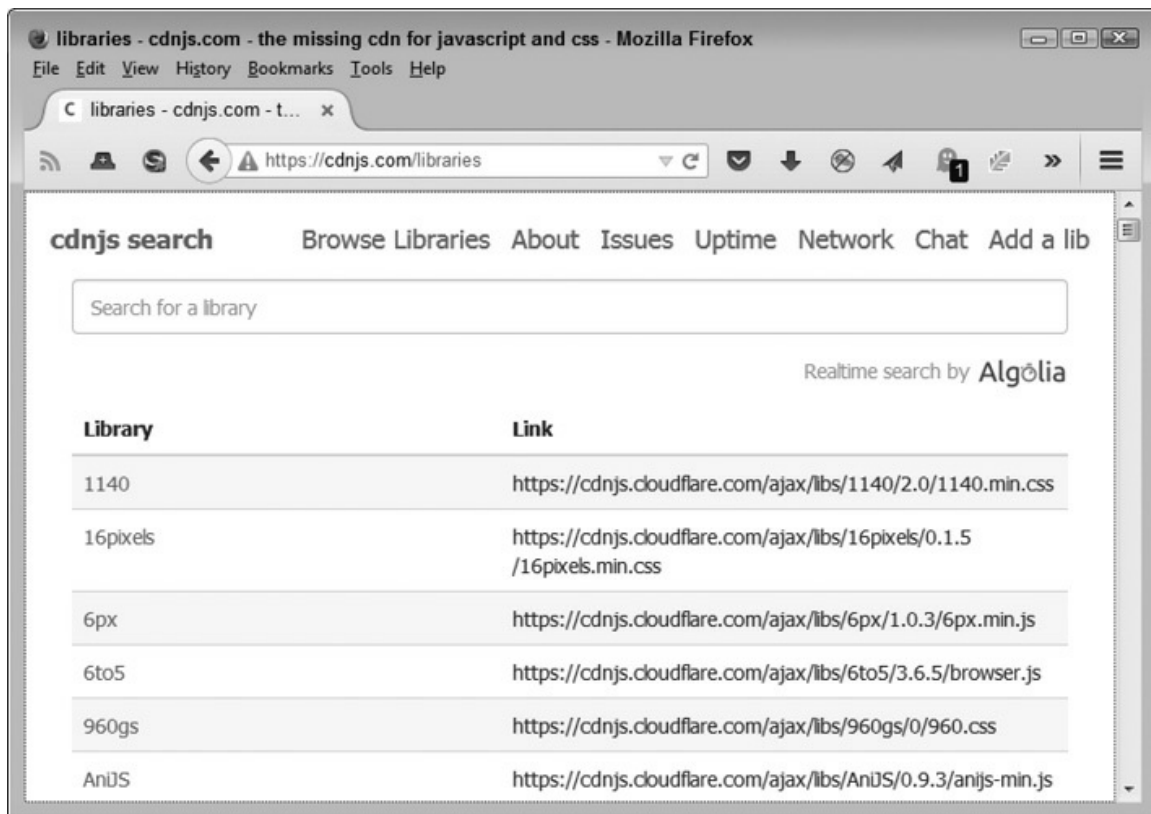


Figura 2.4 – O site cdnjs disponibiliza um grande número de bibliotecas úteis.

Você verá a abreviatura CDN usada em todos os lugares online. Uma CDN (Content Delivery Network, ou Rede de Fornecimento de Conteúdo) é uma série de serviços que oferece conteúdo web de diversos tipos. O propósito principal de uma CDN é oferecer conteúdo com alto nível de disponibilidade e muita velocidade. Ela também oferece conteúdo adequado à região, quando for necessário. Assim, o cdnjs é apenas uma CDN especificamente projetada para oferecer acesso a códigos JavaScript e disponibilizá-los a um grande número de desenvolvedores, de modo semelhante à maneira como a CDN do Google



(<https://developers.google.com/speed/libraries/>) executa essa tarefa.

Tratando atualizações quase contínuas de dispositivos

Sua aplicação deverá ser flexível o suficiente para tratar todos os tipos de cenários incomuns. Um dos cenários mais comuns atualmente é lidar com atualizações quase contínuas de dispositivos. Nesse caso, um usuário estará totalmente satisfeito usando sua aplicação em um smartphone em um dia, mas não conseguirá fazer nada funcionar no dia seguinte. A prática comum é o suporte culpar o usuário, mas em muitas situações o usuário não tem culpa. O problema, no fundo, é que as atualizações, com frequência, acontecem sem a permissão ou o conhecimento do usuário. Infelizmente, uma atualização pode introduzir os seguintes tipos de problema:

- Compatibilidade de código
- Brechas de segurança
- Perda de configurações
- Incapacidade de inicializar o dispositivo
- Danos aos dados

Uma forma de contornar esse problema é desabilitar as atualizações automáticas como parte da política de segurança de uma aplicação. Fazer as atualizações manualmente após tê-las testado como parte do processo de rollout (lançamento de versão) é a melhor maneira de garantir que sua aplicação continuará executando. Infelizmente, essa solução não funcionará. Os usuários não trarão seus dispositivos para serem testados, a menos que algo de errado já tenha ocorrido. Mesmo que os usuários estivessem dispostos a ceder seus dispositivos, é provável que você não terá os recursos necessários para realizar os testes exigidos.

Uma solução alternativa é projetar aplicações que verifiquem automaticamente se há atualizações durante a inicialização. Se o número da versão de um produto do qual o usuário dependa para trabalhar com a aplicação mudar, a aplicação poderá enviar essa informação a um administrador como parte de um lembrete para uma possível solução.

Criar designs flexíveis também faz parte da metodologia para lidar com atualizações constantes. Embora usar um truque sofisticado de programação para ajudar a manter os dados seguros pareça ser uma boa ideia, provavelmente não será. Atenha-se às melhores práticas para estratégias de desenvolvimento, use bibliotecas-padrão quando for possível e mantenha a segurança em um nível razoável para ajudar a garantir que a aplicação continuará funcionando depois que a atualização misteriosa ocorrer no dispositivo do usuário. Caso contrário, você poderá se ver gastando um tempo considerável durante aquelas preciosas horas do fim de semana, tentando corrigir o problema de segurança que está impedindo sua aplicação de funcionar.

Planejando alternativas a senhas

Senhas parecem ser a maneira direta de identificar usuários. Você pode alterá-las com relativa facilidade, deixá-las complexas o suficiente para reduzir as chances de alguém adivinhá-las e elas são totalmente portáteis. Entretanto, usuários veem as senhas como difíceis de usar, mais difíceis ainda de lembrar e como um lembrete incômodo do esquema de segurança que você implementou. (A equipe de suporte que precisa continuamente reiniciar senhas esquecidas tendem a concordar com os usuários nessa situação em particular.) As senhas deixam o sistema de segurança óbvio e incômodo. Os usuários prefeririam não ter de fornecer uma senha para acessar ou deixar funcionalidades específicas da aplicação disponíveis. Infelizmente, é preciso ter algum método para determinar a identidade do usuário. As próximas seções apresentam algumas ideias consolidadas (você pode implementá-las atualmente) que você poderá achar úteis quando estiver procurando a solução perfeita.

Uma palavrinha sobre NFC (Near Field Communication)

Um dos desenvolvimentos tecnológicos mais interessantes tem sido o uso de NFC (Near Field Communication, ou Comunicação por Campo de Proximidade) para todo tipo de requisito. A NFC é uma extensão da tecnologia RFID (Radio Frequency Identification, ou Identificação por Radiofrequência), usada principalmente em produtos. Você encontrará as etiquetas (tags) passivas praticamente em tudo que comprar. A etiqueta RFID também serve como dispositivo de segurança, mas seu propósito principal é identificar o produto. As ondas de rádio de um scanner alimentam a etiqueta e ela devolve as informações que contém. A NFC é um subconjunto de alta frequência da RFID; ela tem algumas características especiais que você poderá achar úteis quando criar soluções de segurança para sua aplicação, incluindo o seguinte:

- O mesmo dispositivo pode atuar como enviado (sender) e etiqueta (tag).
- Os dispositivos trocam informações de forma segura.
- Você pode reprogramar um dispositivo conforme necessário.
- O dispositivo contém tanto inteligência quanto uma memória local, o que o torna mais flexível que a RFID.

A NFC tem o potencial para eliminar senhas. Uma pessoa pode ter um chip em um crachá de identificação do tamanho de um cartão de crédito. Encostar o crachá em um dispositivo preparado para usar NFC faria o login necessário. Há muitos dispositivos com NFC atualmente, incluindo PCs, tablets e smartphones, assim essa solução poderia funcionar em qualquer lugar.

A tecnologia ainda não está madura, portanto é importante pesar suas opções com cuidado.

Felizmente, o W3C (World Wide Web Consortium) já está trabalhando em padrões para desenvolvimento web. Você pode ver a versão preliminar dos trabalhos em <http://www.w3.org/TR/nfc/>. Uma biblioteca, API ou microserviço que esteja de acordo com esse padrão pode oferecer os recursos necessários para criar aplicações robustas que atendam às necessidades tanto da empresa quanto dos usuários, ao mesmo tempo que reduz os custos de suporte e faz com que seja significativamente mais fácil para o usuário trabalhar com a aplicação. O melhor de tudo é que usar uma solução com NFC permite criar uma aplicação realmente segura.

Trabalhando com frases-senha

Senhas são difíceis de lembrar. Criar uma senha como !jS2Zd5L8 dificulta muito que os hackers a adivinhem e aumentam as chances de sua aplicação e seus dados permanecerem protegidos. No entanto, é difícil para algumas

peças memorizarem a senha, portanto, muitas vezes, elas a anotam em algum lugar. De fato, o usuário pode simplesmente manter um arquivo no próprio disco rígido para facilitar o acesso. É claro que os hackers sabem disso e procuram esses tipos de arquivo. Em suma, senhas complexas são uma ótima ideia do ponto de vista da segurança, mas podem ser muito mal planejadas do ponto de vista do usuário. Porém, é possível alcançar quase o mesmo nível de segurança usando uma frase-senha (passphrase), mais fácil de lembrar.

Uma frase-senha é um tipo de frase não totalmente legível aos seres humanos. Você pode usar letras, números e símbolos especiais combinados para criar uma senha complexa. Por exemplo, você pode criar uma frase-senha como I luv fl0w3rs!. A frase-senha contém letras maiúsculas e minúsculas, números e símbolos especiais. Não está no dicionário e resiste a ataques comuns baseados em força bruta. Provavelmente, a frase-senha será mais longa que a maioria dos usuários utilizará em uma senha. Apesar disso, ela é fácil de ser lembrada. Um usuário não precisará anotar a frase-senha.



Os usuários realmente têm razão em reclamar da quantidade de senhas que eles precisam lembrar atualmente. Uma das tecnologias que você pode considerar como parte da solução é o gerenciador de senhas (password vault). Um gerenciador de senhas permite armazenar várias senhas de usuário de forma segura, e o usuário pode continuar utilizando senhas complexas.

É importante escolher frases das quais o usuário se lembrará, mas que não estejam associadas à aplicação, à vida pessoal do usuário ou ao seu ambiente de trabalho. Não há motivos para oferecer vantagens aos hackers quando se trata de adivinhar uma frase-senha. Portanto, uma frase-senha como I Work in R00m 23a. não é, particularmente, uma boa frase-senha – seria muito fácil adivinhá-la.



Simplesmente dizer ao usuário para depender de frases-senha não funcionará. De fato, os usuários continuarão a usar senhas como secret e master porque as usaram por muito tempo. Usuários tendem a rejeitar tudo que os faça trabalhar mais, mesmo que seja só um pouco. Como consequência, você deve garantir o cumprimento das regras de complexidade como parte do código de sua aplicação, de modo que o usuário seja forçado a usar algo melhor que os suspeitos usuais quando se trata de senhas. Se um usuário tiver de usar uma senha realmente complexa, a ideia de usar uma frase-senha em seu lugar torna-se muito mais

atraente.

Usando soluções biométricas

Soluções biométricas contam com algumas características exclusivas de uma pessoa para gerar uma senha. A ideia por trás da biometria é que uma senha biométrica é única para uma pessoa em particular, não é fácil de roubar, o usuário não pode perdê-la e a senha é complexa o suficiente a ponto de os hackers não poderem adivinhá-la (pelo menos, não facilmente). As três soluções biométricas mais comuns em uso atualmente são:

- impressão digital;
- íris;
- reconhecimento de voz.

Essas três soluções têm limitações, e os hackers têm maneiras de superá-las, portanto você pode escolher outro sistema biométrico ou combinar um com outra forma de autenticação. Os fornecedores têm várias alternativas de sistemas biométricos atualmente. A lista a seguir descreve algumas dessas alternativas:

Batimento cardíaco

Uma das alternativas biométricas mais interessantes é combinar um monitor de batimento cardíaco com algoritmos de análise. De fato, a solução já existe na forma de uma pulseira da Nymi (<https://www.nymi.com/>). Essa solução depende de NFC para transmitir a senha do usuário a qualquer dispositivo que utilize NFC. A mesma pulseira pode ser usada para fazer login em um computador, habilitar uma funcionalidade da aplicação, abrir a porta de uma casa ou dar partida em um carro.

Reconhecimento facial completo

No filme *Minority Report*, câmeras analisam os rostos das pessoas e apresentam propagandas à medida que elas andam pela rua. O aspecto interessante é que essa tecnologia já existe na forma do Deepface do Facebook

(<https://research.facebook.com/publications/480567225376225/deepface->

closing-the-gap-to-human-level-performance-in-face-verification/). Basta olhar para seu computador (com uma webcam acoplada) e você poderá fazer o login no sistema e ter todas as funcionalidades da aplicação disponíveis. De fato, o Facebook recentemente declarou (<http://money.cnn.com/2014/04/04/technology/innovation/facebook-facial-recognition/>) que é capaz de fazer a análise tanto frontal quanto lateral, o que torna essa solução relativamente flexível quando comparada a outros sistemas biométricos.



O interessante é que todas as selfies que as pessoas estão tirando facilitará muito a criação de um banco de dados de reconhecimento facial por organizações, por exemplo, aquelas responsáveis pelo cumprimento da lei, tornando impossível que alguém se esconda. Pense só nas consequências do fato de as câmeras de qualquer loja serem capazes de identificar as pessoas baseando-se simplesmente em suas características faciais.

Formato da orelha

Você segura seu smartphone próximo à sua orelha, exatamente como faria para fazer uma chamada. No entanto, em vez de ouvir alguém falar, você fará login em uma aplicação. A solução já existe na forma da Ergo Lock Screen App (<http://www.descartesbiometrics.com/ergo-app/>).

Técnica de digitação

Toda pessoa tem uma maneira diferente de digitar. A velocidade com que você digita, o tempo que segura as teclas e até mesmo as pausas entre as letras o identificam como digitador. Ao digitar uma frase específica e monitorar o modo como você a digita, uma aplicação poderia criar uma autenticação de dois fatores que não pode ser diferenciada da simples digitação da senha. No entanto, um hacker que roubasse a senha não seria mais capaz de usá-la. Uma empresa que já implementou uma solução desse tipo é a Coursera, na forma do Signature Track (<http://blog.coursera.org/post/40080531667/signaturetrack>).

A promessa dos sistemas biométricos não está tão próxima da realidade da biometria. Por meio de descrições, já sabemos que os hackers criaram métodos para passar por cima das senhas biométricas. Se a senha de um usuário for comprometida, basta dar-lhe uma nova senha. No entanto, se a

impressão digital de um usuário for comprometida, não será viável cortar e costurar um novo dedo.

Por que usar autenticação de dois fatores?

Um problema com a maior parte das soluções de autenticação de um fator é que elas têm um ponto fraco que alguém pode explorar com relativa facilidade. Por exemplo, um hacker poderia roubar uma senha ou uma frase-senha usando um ataque de engenharia social ou até mesmo de força bruta. Ao usar uma autenticação de dois fatores, é possível reduzir o risco de alguém passar por cima do processo de autenticação. É claro que poderíamos argumentar que uma autenticação de três ou de quatro fatores seria melhor ainda, mas haveria um ponto em que ninguém mais conseguiria entrar em sua própria conta.

Há vários tipos de autenticação de dois fatores. Por exemplo, você pode fornecer tanto uma senha quanto um token a um usuário. Também é possível combinar senhas com soluções biométricas. Muitos bancos atualmente usam autenticação de dois fatores, e você pode usá-la opcionalmente em sites como Google, Facebook e Twitter.

O problema com a autenticação de dois fatores é o mesmo da autenticação de um fator – normalmente, os usuários não gostam de autenticação. A sensação é que deveria ser possível usar a aplicação sem fazer nada adicional. É claro que a autenticação é importante, e você deve usar a autenticação de dois fatores para dados críticos ou sensíveis. Porém, é importante considerar o ponto de vista do usuário sobre quais opções de autenticação funcionarão melhor. Criar uma solução flexível é essencial se você quiser que essa solução de segurança seja bem-sucedida.

Contando com cartões-chave

A maioria das pessoas vê os cartões-chave (key cards) como uma tecnologia ultrapassada, que continua sendo amplamente usada. Por exemplo, vá a um hotel e é provável que a recepcionista lhe dará um cartão-chave com uma fita magnética especialmente codificada. O cartão-chave substitui a chave que você receberia no passado. As empresas também usam cartões-chave para atender a diversas necessidades, incluindo controle de acesso a certas áreas, por exemplo, os estacionamentos. Uma combinação de cartão-chave com um PIN muitas vezes provê acesso a áreas críticas. Nem é preciso dizer que, provavelmente, você já deve ter usado mais de um cartão-chave em algum momento de sua vida.



A tecnologia de cartão-chave está constantemente sendo aperfeiçoada. Alguns cartões-chave modernos não se parecem nem um pouco com um cartão – eles se parecem mais com um dispositivo que um usuário pode colocar em um chaveiro ou usar em torno do pescoço. Ao contar com a tecnologia RFID ou NFC, não é nem mesmo necessário que o usuário passe o dispositivo; basta agitá-lo diante da fechadura para funcionar. A ideia por trás do cartão-chave, porém, não mudou. Você tem um dispositivo físico que contém informações de segurança com as quais o usuário conta para acessar recursos, em vez de usar uma senha.

Os PCs também usam a tecnologia de cartão-chave. Você pode utilizar o cartão-chave para controlar o acesso ao PC como um todo ou para controlar o acesso a uma aplicação específica. Ao usá-lo para uma aplicação específica, o desenvolvedor precisa fornecer um código que leia o cartão, determine sua validade e autentique o usuário.

A principal vantagem de usar um cartão-chave é que ele pode oferecer uma senha complexa e, possivelmente, outros detalhes que ajudem a identificar o usuário. A senha pode ser tão complexa quanto for necessária para evitar hackers. De acordo com a tecnologia usada, você pode até mesmo ser capaz de alterar informações do cartão-chave com relativa facilidade, de modo que uma mudança de senha não precisa ser uma tarefa custosa. Cartões-chave também tendem a ser grandes a ponto de os usuários não os perderem normalmente (embora você possa ter certeza de que haverá usuários colocando pelo menos alguns de seus cartões-chave no lugar errado). Como a tecnologia está presente há muito tempo, os cartões-chave também podem ter um custo relativamente baixo quando comparados a outras soluções de segurança. Os usuários também tendem a gostar dos cartões-chave (exceto quando os perdem) porque eles são rápidos e fáceis de usar.

A principal desvantagem da tecnologia de cartões-chave é que os usuários os colocam em lugares errados ou os esquecem em casa. Um cartão-chave perdido pode oferecer a um hacker o tipo de acesso necessário para realmente causar danos à sua empresa. Mesmo que o usuário simplesmente esqueça o cartão-chave em casa, fornecer um cartão-chave temporário é um custo adicional de suporte. É claro que há também a questão de obter o cartão-chave temporário de volta quando o usuário não precisar mais dele.

Um lugar interessante para ver diversas tecnologias de cartões-chave é na Secura Key (<http://www.securakey.com/>). Esse site apresenta várias opções de cartões-



chave disponíveis. Você pode encontrar até mesmo teclados, como o modelo IOGEAR (<http://www.iogear.com/product/GKBSR201/>), que oferece o leitor de cartão-chave necessário como parte do teclado. A questão principal é reduzir a interação do usuário necessária para usar sua aplicação a fim de diminuir os potenciais erros do usuário e o vazamento de dados.

Contando com chaves USB

Uma chave USB é essencialmente um pen drive que contém uma ou mais senhas ou tokens. Você conecta a chave USB em seu computador, inicia-o e ele usa os dados da chave USB para fazer login no sistema. A mesma chave poderia conter senhas usadas para acessar aplicações. A aplicação precisaria saber que a senha está na chave, mas a técnica possibilita fazer login na aplicação sem realmente fornecer uma senha ou qualquer outra informação. O Google atualmente está trabalhando em uma configuração de chave USB (<http://www.technologyreview.com/view/510106/googles-alternative-to-the-password/>), e você pode ter certeza que outras empresas farão o mesmo. As chaves USB têm algumas vantagens significativas sobre os cartões-chave:

- É possível alterar as senhas sem obter uma nova chave USB.
- A chave USB pode conter várias senhas.
- O custo geral das chaves USB é menor que o custo dos cartões-chave.
- É possível atualizar os tipos de credenciais que uma chave USB oferece.

É claro que a chave USB compartilha muitas das mesmas falhas do cartão-chave. Por exemplo, não importa se um usuário perde um cartão-chave ou uma chave USB, o resultado é o mesmo – o usuário não poderá mais fazer login na aplicação, enquanto outra pessoa poderá fazê-lo. De fato, dado que as chaves USB são menores que os cartões-chave, é muito mais provável que um usuário perca a chave USB e, como resultado, comprometa mais de uma senha.

Implementando uma estratégia de token

Geralmente, você implementará uma estratégia de token usando smartphones. Um site especial envia ao usuário uma mensagem SMS, uma imagem ou um

áudio que atuará como o meio para fazer login em um computador. Por exemplo, a Illiri (<http://www.illiri.com/>) envia um áudio ao smartphone do usuário que ele simplesmente reproduz ao computador para fazer login. De modo semelhante, a Clef (<https://getclef.com/>) faz o mesmo usando uma imagem. Em ambos os casos, você pode optar por enviar um token diferente sempre que o usuário fizer login no sistema, o que significa que mesmo que um hacker roube o token, ele será essencialmente inútil.

Na maioria dos casos, uma aplicação usa um token como um segundo método de autenticação em um sistema de autenticação de dois fatores. O primeiro método de autenticação continua sendo um cartão-chave, um sistema biométrico, uma senha ou uma frase-senha. Entretanto, teoricamente, é possível usar um token como o método principal de autenticação caso você queira fazer isso. Como ocorre com qualquer outro método de autenticação, esse vem acompanhado de alguns problemas que você precisa resolver:

- O usuário deverá estar o tempo todo com um smartphone.
- Perder um smartphone poderá possivelmente comprometer o sistema de token como meio para fazer login em uma aplicação.
- Essa configuração só funciona para fazer login em dispositivos que não sejam o smartphone, e muitos usuários atualmente querem usar seus smartphones no lugar de um computador sempre que for possível.
- O computador utilizado para login do usuário precisará ter o dispositivo necessário para aceitar o token.

Focando nas expectativas dos usuários

Até este ponto do capítulo, focamos nas necessidades do usuário. Nenhum usuário quer fazer autenticação, mas todo usuário precisa dela. As expectativas se enquadram na categoria “seria bom ter” quando se trata de desenvolvimento, mas implementá-las pode gerar simpatia, deixando o usuário um pouco mais aberto a realmente usar todos aqueles itens necessários adicionados à aplicação.

É claro que alguns usuários têm expectativas que não são realistas, por exemplo, ter uma aplicação que os faça parecer especialmente atraentes para

outras pessoas ou que faça todo o trabalho para eles enquanto jogam paciência e assistem a filmes o dia todo. Infelizmente, nem mesmo o melhor desenvolvedor do mundo poderá atender a expectativas desse tipo. As próximas seções descrevem algumas expectativas razoáveis dos usuários.

Deixando a aplicação fácil de usar

O capítulo procurou enfatizar a necessidade de a aplicação ser fácil de usar. Os usuários não querem saber dos detalhes sobre a solução de segurança que você implementou. De fato, a maioria dos usuários realmente não quer saber nada sobre a aplicação, além do fato de que ela é capaz de oferecer os dados necessários para executar uma tarefa que seja importante para o usuário. No longo prazo, tudo que você puder fazer para deixar a segurança invisível ao usuário final aumenta as chances de o usuário realmente participar do gerenciamento seguro dos dados, e a estratégia de segurança será bem-sucedida para manter os dados protegidos.

A contrapartida à necessidade de ser fácil de usar é criar um cenário de segurança que funcione conforme prometido. Basta uma violação nos dados para arruinar a reputação da empresa, custando-lhe muito dinheiro. De acordo com um artigo recente da *Computerworld* (<http://www.computerworld.com/article/2926775/security0/data-breach-costs-now-average-154-per-record.html>), uma violação de dados atualmente custa 154 dólares em média, por registro. O custo de cada registro perdido continuará a subir, portanto a necessidade de ter soluções seguras continuará a aumentar. A expectativa do usuário é ter uma aplicação fácil de usar, mas para essa aplicação a realidade é a necessidade de manter os dados seguros, independentemente do quanto isso custe.

Deixando a aplicação rápida

Muitos desenvolvedores não entendem a necessidade do usuário por velocidade a qualquer custo. É claro que parte do problema é que o tempo para conquistar a atenção do usuário diminuiu ao longo dos anos. De acordo com um artigo do site do *The Guardian* (<http://www.theguardian.com/media-network/media-network-blog/2012/mar/19/attention-span-internet->

consumer), atualmente, você tem entre um e cinco segundos para conquistar a atenção do usuário. Usuários querem gratificação instantânea e soluções rápidas. Pensamentos profundos e análises detalhadas não fazem mais parte do repertório do usuário. Infelizmente, quanto mais segurança você acrescenta a uma aplicação, mais lenta ela normalmente será. Se você perguntasse à maioria dos usuários quanto tempo eles querem gastar com segurança, a resposta seria zero. Os usuários realmente não se importam com a segurança – eles querem os dados rapidamente.

A expectativa do usuário é que não haja nenhum tempo de espera pelos dados necessários para realizar uma tarefa. Além disso, os dados devem ser precisos e devem estar em um formato necessário ao usuário desde o princípio. Qualquer coisa aquém disso tende a gerar frustração. A verdade é que a segurança deixará a aplicação mais lenta, mas você, como desenvolvedor, precisará se concentrar em manter os atrasos em um nível mínimo.

Além do mais, um ambiente seguro implica que nem todos os dados estarão disponíveis o tempo todo. Você não pode realmente permitir que um usuário veja os registros dos pacientes enquanto estiver no Starbucks local tomando um café. Ter esse nível de acesso é uma expectativa do usuário que não é razoável (isso não impede que os usuários peçam isso). Portanto, uma aplicação rápida é aquela que faz a autenticação do usuário rapidamente e então apresenta-lhe as opções permitidas para acesso aos dados. Ocultar opções não permitidas geralmente evitará, antes de tudo, que os usuários as procurem, mas você também precisa oferecer ajuda para explicar por que determinadas opções de manipulação de dados estão ausentes.

Criando um ambiente confiável

Acima de qualquer outro aspecto considerado, você deve deixar a aplicação confiável. A aplicação não pode se comportar de forma imprevisível, pois o usuário ficará frustrado e não a utilizará. Mesmo que a aplicação fique um pouco mais lenta, um usuário reclamará menos da velocidade que da execução incorreta de tarefas em uma aplicação que seja realmente mais rápida. Também é essencial que os recursos de segurança e de manipulação de dados funcionem perfeitamente. O capítulo 5 discute as várias técnicas

usadas para criar uma aplicação confiável. A Parte III deste livro discute os vários tipos de testes. É preciso levar os testes a sério para criar uma aplicação confiável que gere poucas reclamações de usuários.

A expectativa do usuário, nesse caso, é que a aplicação funcione sem falhas em todas as situações. Novamente, é uma questão de deixar a aplicação transparente para que o usuário só precise focar em concluir a execução de uma tarefa. No mundo real, você pode chegar perto da perfeição usando técnicas apropriadas de programação, técnicas de testes, análise de ameaças e verificações constantes de questões como atualizações de dispositivos. Portanto, nesse caso em particular, uma expectativa do usuário aproxima-se bastante de algo que você pode realmente oferecer.

Mantendo a segurança em perspectiva

Se você não aproveitar mais nada deste capítulo, é importante lembrar-se de que a segurança é uma questão de equilíbrio. A menos que você mantenha a segurança em perspectiva, sua aplicação poderá não proteger os dados suficientemente ou poderá sobrecarregar o usuário ao extremo implementando funcionalidades draconianas que não tenham um propósito útil. Usuários são rápidos para identificar não só as funcionalidades de que não gostam, mas também aquelas que não têm um propósito útil.

Embora possa parecer que a expectativa do usuário seja achar que as aplicações não precisam de segurança, a maioria deles percebe que a segurança tem seu lugar, especialmente se o usuário quiser preservar a saúde da empresa. Não é uma questão de vender a ideia da necessidade de segurança ao usuário – é mais uma questão de vender um nível específico de segurança a ele. Quando você mantém a segurança em perspectiva, é muito mais fácil entrar em acordo e ter o usuário do seu lado.

CAPÍTULO 3

Obtendo assistência de terceiros

Reinventar a roda é sempre uma péssima ideia. É totalmente possível que outra pessoa já tenha criado uma solução de segurança que atenda às suas necessidades a ponto de você poder usá-la em vez de criar sua própria solução de segurança. A solução de terceiros deve fazer parte de seu plano de segurança para que outros saibam que você tem uma solução em mente. Além disso, ao ter a solução de terceiros em seu plano, você pode incluir todo tipo de informações sobre ela e incentivar a discussão sobre a solução. Este capítulo discute como você pode adicionar vários tipos de soluções de terceiros ao seu plano de segurança.

É claro que, antes de poder fazer algo, você deve identificar a solução de terceiros que quer usar. Felizmente, há técnicas que podem ser utilizadas para reduzir o tempo necessário para executar o processo e pesquisar as soluções individuais a fim de garantir que elas realmente atendam às suas necessidades. Depois de identificar as soluções que você quer usar, é importante considerar os prós e contras de cada tipo de solução. Este capítulo agrupa as soluções entre as que existem na nuvem e aquelas que você adiciona à sua aplicação, por exemplo, uma biblioteca ou uma API. As próximas seções ajudarão você a perceber quais soluções de terceiros realmente podem atender às suas necessidades e reduzir o tempo e o esforço para criar uma solução funcional.



Este capítulo discute exemplos específicos, mas as informações fornecidas aplicam-se a uma classe como um todo. Por exemplo, embora o Capterra apareça como um possível site para avaliação de produtos, há outros sites desse tipo, e você deve escolher aquele que melhor combine com a filosofia usada no design e no desenvolvimento de sua aplicação. O exemplo específico simplesmente ajuda a ilustrar o princípio em questão.

Identificando soluções de terceiros para segurança

Encontrar uma solução de terceiros para segurança pode ser difícil. Insira o máximo de termos em que você possa pensar em uma ferramenta de pesquisa e você obterá uma lista de soluções individuais de segurança, muitas das quais podem nem mesmo estar relacionadas ao seu problema em particular. Ferramentas de pesquisa fornecem um palheiro, e não uma agulha. Infelizmente, elas também removem o ímã, de modo que encontrar a agulha é impossível. Você realmente precisa de uma maneira melhor de encontrar a solução de segurança de seus sonhos.

A ajuda pode estar em sites de avaliação como o Capterra (<http://www.capterra.com/network-security-software/>), como mostra a figura 3.1. Esse site oferece auxílio na forma de filtros que ajudam a entender os diversos fornecedores que disputam sua atenção. Cada uma das entradas tem uma pequena avaliação em que você pode clicar para ver detalhes adicionais. Em muitos casos, a avaliação inclui também mídias que podem incluir uma demo do produto ou outras informações úteis. O fato de as avaliações estarem em um só lugar, filtradas por critérios que você especifica e formatadas essencialmente da mesma maneira, simplifica o processo de pesquisa.



Figura 3.1 – Sites de pesquisa como o Capterra facilitam procurar recursos de terceiros.



Vale a pena conferir as informações encontradas no Capterra e em sites semelhantes de avaliação. Você pode fazer verificações até certo nível ao fazer uma pesquisa na web em busca de um produto que lhe interesse a fim de descobrir se alguém mais forneceu uma avaliação dele. Os esclarecimentos oferecidos por essas avaliações adicionais podem ajudar você a escolher melhor o produto.

Contar com avaliações de revistas também pode ser útil, dependendo da revista que você escolher. Por exemplo, a *SC Magazine* (<http://www.scmagazine.com/>) regularmente avalia produtos que podem ser úteis ao profissional da área de segurança. De fato, você pode receber as avaliações por email (<http://www.scmagazine.com/events/section/109/>). A revista até mesmo patrocina uma espécie de competição (<http://www.scmagazine.com/2014-sc-awards-us-finalists/section/3694/>) para que você possa determinar as soluções mais viáveis.



Sites de avaliação de qualquer tipo serão tendenciosos. Mesmo que o site avalie produtos honestamente, usando critérios que você possa analisar e processos que você possa conferir, uma avaliação ainda é uma questão de opinião pessoal e você deve considerar se a pessoa que está fazendo a avaliação tem conhecimento suficiente para oferecer as informações necessárias. Com o tempo, você desenvolverá a noção de quais sites em particular estão de acordo com a sua opinião do que é realmente um bom produto.

Em alguns casos, você pode encontrar organizações que oferecem descrições resumidas de seus participantes, o que pode prover informações consistentes. Por exemplo, a CSA (Cloud Security Alliance, ou Aliança para Segurança em Nuvem, em <https://cloudsecurityalliance.org/membership/solution-providers/>) se enquadra nessa categoria. Ao verificar esses sites, você poderá ter uma rápida visão geral dos participantes envolvidos em soluções específicas de segurança. É claro que são as empresas que escrevem os textos que você lê, portanto as informações são tendenciosas, o que é um problema. Você pode ter certeza de que os fornecedores que participam da organização deixaram de fora qualquer aspecto potencialmente negativo de seus diversos produtos. Leia esse tipo de informação com atenção e então pesquise os problemas em potencial por conta própria.

Alguns sites de revistas apresentam artigos que podem fornecer informações significativamente úteis. Por exemplo, o artigo “Improving the Security of Cloud Computing” (Melhorando a segurança da computação em nuvem) da MIT Technology Review (<http://www.technologyreview.com/news/424298/improving-the-security-of-cloud-computing/>) discute técnicas que você pode usar para manter seus dados seguros. As duas soluções oferecidas como parte do artigo apontam questões que talvez você não tenha considerado. No primeiro caso, você lerá sobre como cientistas de computação da Universidade da Califórnia em San Diego e o MIT demonstraram a necessidade de cada empresa ter seu próprio servidor virtual (como resultado, a Amazon mudou seu modo de trabalhar). No segundo caso, uma nova tecnologia divide seus dados em 16 partes. Você pode então recuperar os dados usando quaisquer dez partes restantes, caso algum tipo de falha ocorra. É importante lembrar que artigos normalmente têm um objetivo, portanto é preciso tê-lo em mente quando decidir sobre a viabilidade dos produtos apresentados no artigo para atender às necessidades

de sua empresa.



A parte boa sobre os sites organizacionais é que, com frequência, você encontrará outros tipos de informações úteis porque manter você envolvido é interesse do site. Alguns sites organizacionais também dão informações sobre os próximos eventos, em que você poderá encontrar informações adicionais sobre possíveis soluções, conhecer os fornecedores pessoalmente e descobrir novas técnicas das quais talvez ainda não tenha ouvido falar.

Considerando soluções para segurança em nuvem

As organizações armazenam muitos dados em nuvem atualmente para manter os custos sob controle e deixar os dados facilmente acessíveis a partir de qualquer lugar que a empresa possa precisar. A computação em nuvem, que inclui o armazenamento de dados em nuvem, veio para ficar, pois simplesmente faz sentido usá-la em vez de contar com soluções personalizadas.

O problema com a computação em nuvem é que ela abre sua empresa para todo tipo de problema. Por exemplo, você não tem como saber se a empresa hosting manterá seus dados protegidos. Há histórias demais na mídia sobre empresas que tiveram os dados hackeados, com consequências desastrosas. De fato, você pode facilmente encontrar histórias sobre como é fácil hackear áreas de armazenamento em nuvem, por exemplo, o artigo “Cloud Hacking Isn’t as Hard as Most Would Think” (Hacking na nuvem não é tão difícil quanto a maioria pensa) da *Tech Times* (<http://www.techtimes.com/articles/14800/20140903/cloud-hacking-isnt-hard-think.htm>). (Uma possível maneira de evitar parte dos infortúnios relacionados ao armazenamento de dados em nuvem é criptografá-los antes de enviá-los, usando técnicas robustas de criptografia. É claro que essa solução necessariamente envolve uma penalidade na velocidade de sua aplicação por causa da necessidade de criptografar e descriptografar dados continuamente.)

As próximas seções apresentam algumas ideias sobre como você pode usar soluções de terceiros para garantir a segurança de seus dados online. As seções consideram três cenários: repositórios de dados, compartilhamento de arquivos e armazenamento em nuvem. Talvez você precise usar combinações

de soluções para criar um pacote completo para sua empresa. Entretanto, essas soluções oferecem um bom ponto de partida e, em última instância, farão você economizar tempo e esforço. O melhor de tudo é que, como outra pessoa é responsável pela manutenção da solução, você economizará dinheiro também no longo prazo, pois não estará fazendo correções constantemente em uma solução personalizada.



Uma boa regra geral é lembrar que a melhor maneira de manter um segredo é não o contar a ninguém. Se você tem requisitos de armazenamento para dados que simplesmente não podem ver a luz do dia, armazená-los online provavelmente será a pior solução possível. Independentemente de como você protege seus dados, se alguém estiver convencido de que quer ter acesso a eles, essa pessoa terá. É sempre mais fácil hackear um sistema de segurança que implementá-lo. Consequentemente, se você tem dados que deva manter seguros do ponto de vista legal em todos os possíveis cenários, usar uma área de armazenamento local da empresa na qual você tenha controle é a melhor ideia. O artigo “Are My Files Really Safe If I Store Them in the Cloud?” (Meus arquivos estarão realmente seguros se eu armazená-los na nuvem?, <http://computer.howstuffworks.com/cloud-computing/files-safe-in-the-cloud.htm>) simplesmente narra de quantas maneiras diferentes os hackers podem acessar seus dados, e você pode estar certo de que eles descobrirão outras formas.

Entendendo os repositórios de dados

Um repositório de dados pode ter muitos sentidos. Basta tentar achar uma definição consistente online e você acabará encontrando um novo significado em cada site que verificar. O fato é que *repositório de dados* tem um significado um pouco diferente para cada um que usa o termo. No entanto, a maioria das pessoas, em geral, concorda que um repositório de dados é um local centralizado para armazenamento de dados com informações que uma empresa mantém como parte de sua base de conhecimento. Usando técnicas de mineração de dados (data mining), a empresa pode explorar essa base de conhecimento e criar novos conhecimentos a partir dela. É claro que há muitas outras implicações para os repositórios de dados, mas, no final das contas, estamos falando de muitos dados na maioria dos casos; alguns deles são mantidos, enquanto outros são armazenados por motivos históricos. Manter seguros dados desse tipo é uma tarefa enorme.

Repositórios de dados são abundantes. Você pode encontrar muitos

repositórios de dados abertos em sites como o *Open Access Directory* (OAD, em http://oad.simmons.edu/oadwiki/Data_repositories), como mostra a figura 3.2. Você pode até mesmo usar alguns desses repositórios em sua aplicação. Portanto, segurança não é simplesmente uma questão de manter seu repositório privado seguro, mas também de garantir que qualquer repositório público que você usar também seja seguro. Afinal de contas, um hacker não se importa muito com os meios usados para acessar sua aplicação e, em última instância, sua empresa – o que importa é que o acesso aconteça.

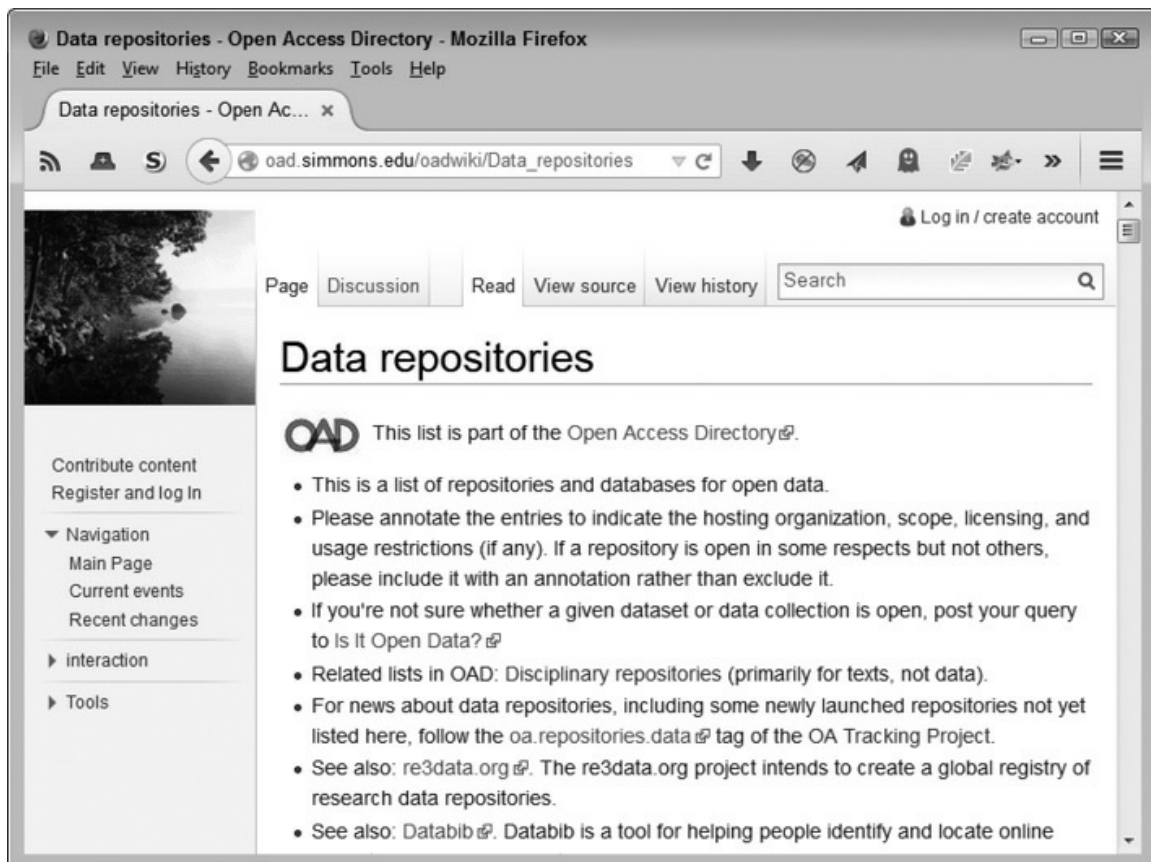


Figura 3.2 – Repositórios de acesso aberto oferecem informações essenciais para algumas organizações.

Poucos repositórios de dados contêm dados ou afetam apenas um projeto. De fato, não faz muito sentido criar um repositório desse tipo. A maioria dos repositórios de dados contém dados de um grande número de projetos. Por exemplo, o sourceforge.net (<http://sourceforge.net/>) contém mais de 90 mil projetos que incluem todo tipo de projetos de programação que você possa imaginar, incluindo softwares de modo kernel. Abordagens tradicionais para

manter esses repositórios de dados seguros, como contar com o administrador para fazer correções no servidor e administrar o acesso a ele, não funcionam nesse caso porque é fácil demais que itens de manutenção sejam negligenciados.

Um novo tipo de protocolo chamado SUNDR (Secure Untrusted Data Repository) da Secure Computer Systems Group (<https://www.yumpu.com/en/document/view/34269846/secure-untrusted-data-repository-sundr-uxenix>) ataca o problema do ponto de vista do cliente. Um cliente pode detectar modificações em um arquivo. Mesmo que o servidor não seja confiável ou esteja comprometido de alguma maneira, o cliente ainda poderá detectar a potencial brecha de segurança. O modo como esse sistema funciona depende de logs mantidos em um servidor de blocos (block server) e em um servidor de consistência (consistency server) usando métodos que seriam difíceis (mas jamais impossíveis) de falhar. Você pode ver um slideshow da tecnologia em <http://slideplayer.com/slide/3389212/>.

É importante lembrar que essas tecnologias novas andarão de mãos dadas com as tecnologias existentes, por exemplo, a segurança oferecida por um gerenciador de banco de dados. Elas também não isentam você da responsabilidade, como desenvolvedor, de incluir segurança como parte de sua solução. Entretanto, o que tecnologias como o SUNDR oferecem é outro nível de defesa – esse é no nível do servidor e não depende do servidor ou do administrador para ser mantido.

Lidando com problemas de compartilhamento de arquivos

Compartilhamento de arquivos geralmente significava criar uma configuração desajeitada que os usuários detestavam usar – isso quando ela funcionava. Para criar um cenário de compartilhamento de arquivos, a empresa precisava instalar um servidor de arquivos especial e criar uma VPN para acessá-lo. Para um desenvolvedor, isso significava escrever muito código para contornar os problemas associados ao acesso dos dados em ambientes desse tipo, além de uma quantidade igualmente grande de código para captura de erros para reduzir a frustração do usuário quando a

configuração não funcionasse conforme planejado. Transferir dados de redes privadas para soluções em nuvem mantidas por terceiros que têm os recursos monetários necessários para custear uma solução desse tipo parece ser uma opção óbvia. Uma empresa pode economizar 65% ou mais ao transferir dados para um provedor na nuvem em vez de hospedá-los em um servidor privado¹.

Os desenvolvedores ganham APIs bem projetadas e documentadas para facilitar o acesso aos dados. Os usuários ganham porque ficam muito menos frustrados ao usar uma solução de compartilhamento de arquivos acessível publicamente, e soluções como essa normalmente funcionam com todos os dispositivos que os usuários possam ter.



É essencial perceber que qualquer serviço de compartilhamento de arquivos disponível publicamente criará brechas de segurança em potencial. Não importa a altura das paredes construídas pelo host, um hacker aparecerá e cavará por baixo delas para acessar seus dados. Usar qualquer tipo de serviço de compartilhamento de arquivos implica riscos além dos que você esperaria ao usar uma VPN. Embora muitas empresas usem serviços de compartilhamento de arquivos com sucesso, tenha em mente que uma VPN geralmente é mais segura e talvez você precise optar por uma solução com VPN para algumas necessidades de armazenamento, apesar dos custos significativamente mais altos.

Quando a maioria das pessoas pensa em compartilhamento de arquivos em nuvem atualmente, elas pensam em produtos como o Dropbox (<https://www.dropbox.com/business>). É verdade que o Dropbox tem uma API (<https://www.dropbox.com/developers>) que os desenvolvedores podem usar para criar interfaces para suas aplicações. A API oferece funcionalidades completas e você pode usá-la para atender a várias necessidades, como mostra a figura 3.3. Sabemos que o Dropbox recentemente virou manchete de notícias sobre problemas de segurança², e a *PCWorld* até mesmo publicou um artigo (<http://www.pcworld.com/article/2918524/how-to-make-dropbox-more-secure-without-spending-a-cent.html>) que detalha soluções para corrigir esses problemas.

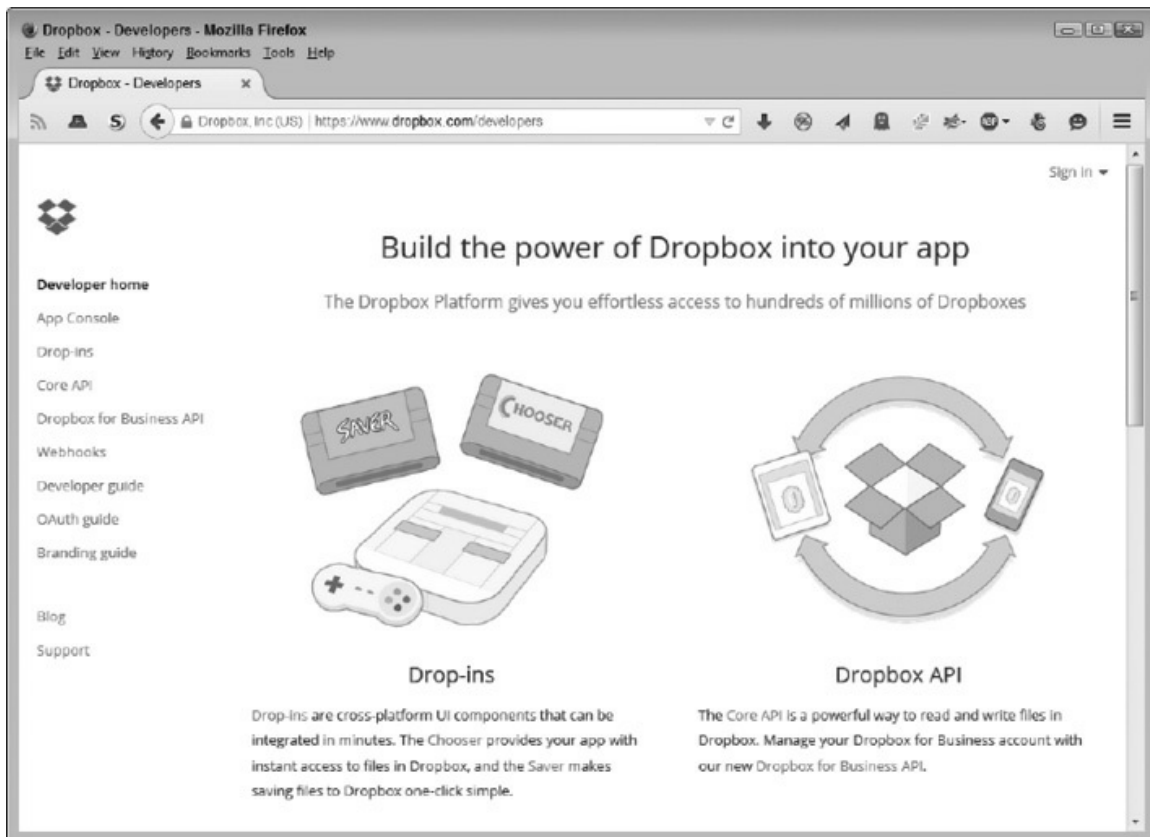


Figura 3.3 – A API do Dropbox simplifica a adição do compartilhamento de arquivos online à sua aplicação.

Na verdade, há algumas soluções que você pode experimentar quando criar uma solução de compartilhamento de arquivos para negócios de pequeno e médio porte. Todas elas têm problemas de segurança, portanto é preciso pesquisar cada uma a fim de determinar com quais desses problemas você prefere lidar. Algumas, como o Dropbox, apareceram nas manchetes do noticiário, mas elas também têm boas soluções viáveis para seus problemas de segurança. Eis uma lista das soluções mais comuns de compartilhamento de arquivos (além do Dropbox) em uso atualmente:

- Acronis Business (<http://www.acronis.com/en-us/business/overview/>)
- Box (<https://www.box.com/>)
- Carbonite Business (<http://www.carbonite.com/online-backup/business/how-it-works>)
- Citrix ShareFile (<http://www.sharefile.com/>)
- CrashPlan Pro (<http://www.code42.com/business/>)

- Engnyte (<https://www.egnyte.com/>)
- Google Drive (<http://www.google.com/drive/start/index.html>)
- Hightail (antigo YouSendIt) (<https://www.hightail.com/>)
- Microsoft OneDrive (<https://onedrive.live.com/about/en-us/>)
- MozyPro (<http://mozy.com/product/mozy/business>)
- SpiderOak (<https://spideroak.com/>)
- SugarSync for Business (<https://www.sugarsync.com/business/>)
- SyncPlicity (<https://www.syncplicity.com/>)

Escolher uma solução de terceiros para compartilhamento de arquivos pode ser difícil. Sua empresa terá objetivos específicos ao escolher uma solução de compartilhamento de arquivos, por exemplo, a relação custo/benefício. No entanto, como desenvolvedor, você tem outras preocupações. A seguir, apresentamos os aspectos que você deve considerar quando quiser saber sobre um serviço de compartilhamento de arquivos:

API bem documentada

Para escrever um código personalizado para a solução de compartilhamento de arquivos e deixar alguns tipos de acesso invisíveis, você precisa ter uma API bem documentada.

Segurança

Uma solução de compartilhamento de arquivos baseada em nuvem exige alguns recursos únicos de segurança para que seja bem-sucedida:

Cloud tiering (Níveis de armazenamento na nuvem)

O mais importante desses recursos é o cloud tiering, com o qual você pode definir como o arquivo é armazenado. Alguns arquivos aparecem somente no serviço de compartilhamento de arquivos, outros têm uma cópia local além da cópia na nuvem e outros aparecem apenas em drives locais, apesar de estarem disponíveis por meio do serviço de compartilhamento de arquivos.

Tipos de arquivo

Alguns tipos de arquivo podem ter requisitos especiais de armazenamento. Esses arquivos podem conter dados sensíveis e talvez você precise armazená-los localmente ou criptografá-los, mesmo que eles apareçam de maneira diferente de acordo com o nível (tier) do arquivo.

Frequência de acesso

Sua aplicação pode acessar arquivos somente com uma frequência específica. Quando houver acessos ao arquivo com uma frequência diferente, isso pode indicar uma falha de segurança. O serviço de compartilhamento de arquivos deve alertar você sobre esse problema em potencial.

Namespace global

Em alguns casos, um serviço de compartilhamento de arquivos oferecerá acesso de modo a criar silos de dados (data silos), em que a área de contabilidade terá seus dados totalmente separados da área de recursos humanos. Silos de dados têm um propósito, mas, às vezes, é necessário armazenar dados de modo que qualquer pessoa com direitos possa acessá-los de qualquer lugar usando qualquer dispositivo. Para ter esse tipo de acesso, você precisa de um namespace global.

Redundância

Para continuar escrevendo código sem estar preso à manutenção das necessidades do usuário, você precisa de uma solução de compartilhamento de arquivos que tenha alto nível de redundância. Caso contrário, você não terá como garantir que os dados necessários a um usuário estarão disponíveis a qualquer momento. Entretanto, como parte da redundância de dados, você precisa garantir que eles estejam armazenados de forma segura em cada local usando servidores virtuais individuais (vários tipos de ataques a arquivos compartilhados dependem do invasor conseguir acesso ao servidor do alvo – algo que se torna mais difícil quando o serviço de compartilhamento de arquivos conta com servidores virtuais associados a clientes individuais).

Considerando o armazenamento em nuvem

O termo *armazenamento em nuvem* (cloud storage) engloba várias necessidades. Duas delas foram especialmente mencionadas neste capítulo: repositórios de dados e serviços de compartilhamento de arquivos. No entanto, sua empresa pode ter outras necessidades de armazenamento em nuvem. Normalmente, será mais fácil se você puder criar uma aplicação que use um único host, mas isso nem sempre é possível. Ao escolher um host, você também deve considerar os seguintes requisitos adicionais de armazenamento em nuvem:

Arquivamento

Arquivamento de dados é diferente de outros tipos de armazenamento em nuvem, pois você deve garantir que os dados estejam seguros, em vez de estarem necessariamente acessíveis. Se um evento importante ocorrer, você quer garantir que seus dados permaneçam seguros. No passado, as empresas contavam com fitas e outras mídias armazenadas em outro lugar para garantir a segurança dos dados, mas, atualmente, o armazenamento em nuvem atende a essa necessidade. Qualquer aplicação que você crie pode precisar arquivar dados e garantir que o arquivamento ocorra de forma segura.

Armazenamento de configurações

Seus usuários vão querer usar todo tipo de dispositivo para acessar qualquer dado de várias maneiras. Os usuários não têm a mínima ideia de que as aplicações precisam de configurações para funcionar bem e que armazenar configurações localmente faz com que seja impossível criar aplicações flexíveis que funcionem em qualquer lugar. Infelizmente, as configurações armazenadas na nuvem também facilitam aos hackers adivinhar quais são os recursos da aplicação e efetuar um hack em sua empresa, desta forma é preciso garantir que o armazenamento de configurações esteja criptografado e armazenado em um host de modo a deixar qualquer brecha óbvia.

Armazenamento de mídias

Usuários exigem acesso a todo tipo de mídia atualmente, incluindo vídeo, áudio, fotos, apresentações e outros tipos. Sua aplicação talvez precise gerenciar mídias como parte de seu propósito, o que significa contar com

recursos como níveis de armazenamento (tiering) para garantir que a aplicação e o armazenamento em nuvem tratem os dados corretamente. Seu usuário deve ter acesso apenas às mídias necessárias, e não a todas as mídias que a empresa tem a oferecer.

Há outros tipos de armazenamento em nuvem que não serão discutidos neste livro. Por exemplo, a maioria dos desenvolvedores não se interessa muito por emails, a menos que esteja escrevendo uma aplicação para eles. A questão é que o armazenamento de dados engloba uma grande variedade de tipos e de requisitos que você deve considerar como parte de sua solução de segurança.

Escolhendo um tipo de produto

À medida que ler este livro, você aprenderá a usar vários tipos de produtos para deixar seus esforços de programação significativamente mais simples. É importante entender que você pode classificar produtos relacionados à códigos de três maneiras: bibliotecas, APIs e microsserviços. Cada tipo de recurso de programação tem um propósito específico e ajuda você no atendimento a necessidades específicas. É possível misturar e combinar tipos de produto em uma única aplicação, mas você deve garantir que os produtos sejam usados de modo que não afetem a segurança de maneira negativa. As próximas seções discutem os diferentes tipos de produto e ajudam a entender como você pode usá-los como parte da estratégia de uma aplicação. Acima de tudo, você poderá vislumbrar alguns dos problemas de segurança que cada tipo de produto pode ter.

Usando código de outras pessoas

Sempre que usar código de outras pessoas, você corre o risco de algum hacker descobrir uma maneira de passar por cima das medidas de segurança oferecidas por esse código. As vantagens de fazer isso são enormes. Ao criar uma única brecha de segurança, o hacker potencialmente pode ter acesso às aplicações escritas por todos que usem a biblioteca, a API ou o microsserviço ao qual o hacker conseguiu acesso. O mesmo hack funciona em todos os lugares, o que significa que o hacker pode selecionar quais sites vai invadir e quais dados vai acessar. Alguns hacks são tão bem-sucedidos que um hacker consegue ter controle total sobre uma fonte de dados e executa ações como deixar os dados como reféns até o dono fazer o pagamento.

A vantagem de usar código de outras pessoas é que você pode criar uma aplicação em

uma fração do tempo que seria gasto normalmente, usando poucas pessoas. Os custos de suporte e de manutenção também são significativamente mais baixos. Quando alguém descobre um problema na biblioteca, API ou microsserviço, o dono desse código faz a correção, e não você (supondo que o proprietário esteja disponível para realizar a correção; caso contrário, você deve mudar de biblioteca, API ou microsserviço). Sem realizar qualquer tarefa, você obterá automaticamente todas as vantagens da correção feita pelo terceiro.

O terceiro também oferece melhorias que deixam o código mais rápido, mais eficiente ou mais seguro. Quando novas tecnologias entram em cena, o terceiro oferece as atualizações necessárias para fazer sua aplicação funcionar de novas maneiras. Se os usuários começarem a exigir acesso a um novo tipo de dispositivo, o terceiro criará o código necessário para fazer esse dispositivo funcionar adequadamente. Em suma, há várias boas razões para usar código de outras pessoas, mas você deve ser cauteloso ao fazer isso.

Trabalhando com bibliotecas

Bibliotecas são códigos presentes em arquivos separados, mas que são carregados em sua aplicação. A biblioteca torna-se parte de sua aplicação. Na maioria dos casos, você pode fazer download das bibliotecas em um servidor local, mas em outros, isso não é possível. Fazer download de uma biblioteca para um servidor local tem a vantagem de não permitir que outros vejam como você está usando a biblioteca para criar uma aplicação. Essa prática também pode tornar a aplicação mais rápida, deixando o processo de carga mais simples e eficiente. Além disso, fazer download da biblioteca significa que o código da biblioteca permanecerá estável, o que possibilita criar aplicações mais confiáveis.

Do ponto de vista da segurança, pode parecer que fazer download de uma biblioteca em um drive local seja a melhor aposta. Você tem todas as vantagens de usar o código de outra pessoa e não precisa expor sua aplicação a possíveis hacks que se baseiam em ver como você usa a biblioteca. A desvantagem, do ponto de vista de segurança, é que usar uma cópia local também significa que você não terá atualizações automáticas. Isso quer dizer que sua aplicação poderia conter bugs que o desenvolvedor da biblioteca já corrigiu, facilitando o acesso de um hacker ao seu sistema.



O fato de alguém poder ver o que você está fazendo com uma biblioteca baixada depende da linguagem do código-fonte da biblioteca. JavaScript normalmente é legível em um editor de texto. Muitas pessoas ocultam seu código JavaScript (deixam o código virtualmente ilegível) de modo a dificultar que outros saibam o que elas estão fazendo com o código. Produtos como JScrambler (<https://jscrambler.com/en/>) e JavaScript Obfuscator/Encoder (<http://www.danstools.com/javascript-obfuscate/>) dificultam ver o que está acontecendo, porém um hacker determinado ainda poderá fazê-lo. Também é possível descompilar muitos tipos de arquivos de código usados na internet. A questão é que você pode dificultar as coisas ao fazer download de uma biblioteca e deixando-a ilegível, mas não pode fazer com que seja impossível determinar como sua aplicação funciona.

Ao trabalhar com bibliotecas, é preciso considerar a reputação do fornecedor, assim como a maneira como você usa a biblioteca em sua aplicação. Como uma biblioteca é diretamente integrada à sua aplicação, você deve considerar que tipo de acesso a biblioteca terá à parte interna de sua aplicação. É importante programar de forma defensiva quando usar bibliotecas porque você não deve dar informações demais a outras pessoas sobre o funcionamento de sua aplicação. Alguns exemplos de bibliotecas são:

- D3.js (<http://d3js.org/>)
- Google Web Toolkit ou GWT (<http://www.gwtproject.org/>)
- jQuery (<http://jquery.com/>)
- jQuery Mobile (<http://jquerymobile.com/>)
- jQuery UI (<http://jqueryui.com/>)
- MooTools (<http://mootools.net/>)
- PDF.js (<http://mozilla.github.io/pdf.js/>)
- QUnit (<http://qunitjs.com/>)
- SWFObject (<https://code.google.com/p/swfobject/>)
- YUI Library (<http://yuilibrary.com/>)

Acessando APIs

APIs são códigos que você acessa a partir de sua aplicação fazendo chamadas a um local centralizado. As APIs existem como uma entidade separada e você

aciona essa entidade a partir de sua aplicação. A questão é que as APIs são totalmente separadas de outros códigos de sua aplicação, portanto você precisa criar uma referência para ela e, então, fazer solicitações à API. Uma API normalmente está associada a algum tipo de serviço, por exemplo, armazenamento de dados. Use APIs para criar uma conexão do tipo cliente/servidor, com todos os problemas de segurança que uma conexão dessas implica.



Algumas APIs incluem acessos protegidos para ajudar a garantir que hackers não possam usar facilmente determinados exploits para ter controle da API ou usá-la de maneiras que seus criadores jamais imaginaram. Algumas APIs exigem uma combinação de nome e senha para que o serviço possa ser acessado, o que não é um problema, desde que a informação seja enviada de forma criptografada. Mas mesmo usando criptografia, os dados não estarão realmente seguros. Como alternativa, as APIs podem usar chaves de acesso. Infelizmente, quando as chaves são expostas, os endpoints das APIs continuam a dar informações alegremente a quem quer que as requisite. Em suma, problemas de segurança ocorrerão com uma API em algum momento, mas você pode empregar métodos para cercear as atividades dos hackers. Continua sendo importante manter-se vigilante e prestar atenção em atividades de hackers quando usar uma API.

Um problema em potencial com APIs é que elas podem ser lentas. Sua aplicação torna-se menos eficiente e pode haver atrasos. Esse é um aspecto importante a ser considerado porque a paciência não é a melhor das virtudes dos usuários e eles podem acabar fazendo algo que causará falhas inadvertidamente em sua aplicação enquanto esperam. Aplicações com falha sempre apresentam problemas de segurança que os hackers ficariam bem satisfeitos em explorar.

Também é possível aos hackers usar ataques man-in-the-middle para acessar seus dados quando uma API estiver sendo usada. Um ataque man-in-the-middle, em especial, é difícil de detectar porque as chamadas aparentemente funcionam e você não verá nenhuma diferença nos dados quando os recuperar posteriormente. Enquanto isso, o hacker usa os dados acessados para executar tarefas como obter números de cartões de crédito de clientes ou informações úteis sobre sua empresa. Ao trabalhar com uma API, você deve prestar bastante atenção a questões como criptografia de dados para garantir que seus dados permaneçam seguros. Alguns exemplos de APIs são:

- AccuWeather (<http://www.programmableweb.com/api/accuweather>)
- Amazon (<https://affiliate-program.amazon.com/gp/advertising/api/detail/main.html>)
- Box (<https://developers.box.com/>)
- Facebook (<https://developers.facebook.com/>)
- Flickr (<https://www.flickr.com/services/developer/>)
- Google (<https://developers.google.com/products/>)
- Pinterest (<http://www.programmableweb.com/api/pinterest>)
- Salesforce (<http://www.salesforce.com/us/developer/docs/api/index.htm>)
- Twitter (<https://dev.twitter.com/overview/documentation>)
- WordPress (http://codex.wordpress.org/WordPress_APIs)
- YouTube (<https://developers.google.com/youtube/>)



Uma das diferenças entre bibliotecas e APIs é que a maioria das bibliotecas oferece uso gratuito, enquanto muitas APIs cobram para serem utilizadas. Na maioria dos casos, você pode ter acesso a uma API em um nível suficiente para testá-la, mas para ter a funcionalidade completa, você precisa obter uma chave, o que significa pagar pelo nível de acesso necessário.

Além disso, muitos hosts de APIs exigem que você teste sua aplicação em modo sandbox e obtenha um certificado para a aplicação depurada resultante antes que o host permita à aplicação usar a API propriamente dita. Esses requisitos também aumentam o custo de usar APIs, o que pode torná-las menos populares que as bibliotecas.

Considerando os microsserviços

Microsserviços são uma tecnologia nova baseada em tecnologias existentes. Um microsserviço é um tipo de mistura entre web service, API e biblioteca, mas em um pacote pequeno. Com efeito, microsserviços têm a seguinte lista de recursos que você deve considerar:

- Dependem de uma aplicação pequena, de propósito único e baseada em serviço para criar uma aplicação totalmente funcional (cada aplicação de propósito único é um microsserviço).
- Usam a linguagem de programação mais apropriada para a tarefa.

- Acessam dados da aplicação usando a técnica mais eficiente de gerenciamento de dados para o microserviço em particular.
- Oferecem uma comunicação leve para cada microserviço.
- Dependem de protocolos como REST para se comunicar, de modo que o pipe não é inteligente, mas o microserviço é.
- Empregam gerenciamento descentralizado de aplicações, pois cada microserviço é monitorado separadamente.
- Cada microserviço é selecionado conforme for necessário para criar qualquer aplicação completa (desktop, navegadores móveis, aplicativos móveis nativos e até mesmo APIs).

Dada a maneira como os microserviços funcionam, você deve considerar alguns dos mesmos problemas das bibliotecas e APIs. Por exemplo, um microserviço poderia fazer parte do código de sua aplicação, portanto você deve considerar o modo como o microserviço interage com a aplicação. Além disso, como ocorre com uma API, dados são enviados e recebidos quando trabalhamos com um microserviço, por isso é possível que ataques man-in-the-middle causem problemas.

Do ponto de vista de segurança, os microserviços tendem a implementá-la de modo diferente das bibliotecas e das APIs. Em vez de oferecer uma única solução para tudo e para todos que usam os microserviços em um site como um todo, cada microserviço oferece um sistema individualizado de segurança baseado nas necessidades desse microserviço. Como resultado, os microserviços tendem a oferecer melhor nível de segurança, mas essa segurança também é irregular e mais difícil de ser trabalhada. Eis alguns exemplos de microserviços (cada um desses sites oferece acesso a diversos microserviços – você escolhe quais quer usar em uma aplicação em particular):

- Akana (<http://www.akana.com/solutions/microservices>)
- Archivemata (<https://www.archivemata.org/en/>)
- Gilliam (<http://gilliam.github.io/>)
- LSQ.io (<https://angel.co/lq-io>)

- Seneca (<http://senecajs.org/>)



Pelo fato de serem tão novos, você encontrará muitas discussões sobre o que, exatamente, é um microsserviço. Para alguns especialistas, o número de LOC (Lines of Code, ou Linhas de Código) importa. Um serviço que tenha um LOC entre 200 e 500 se enquadra como um microsserviço, mas se estiver acima disso, não será. (Um LOC menor que 200 aparentemente não oferece um nível suficiente de funcionalidade.) Consequentemente, um microsserviço como o Cloner (<https://www.npmjs.com/package/app-cloner-heroku>) atende aos requisitos (LOC de 305), mas um microsserviço como o Deploy Hooks (<https://devcenter.heroku.com/articles/deploy-hooks>) não atende (LOC de 1240).

-
- 1 “Moving Your Infrastructure to the Cloud: How to Maximize Benefits and Avoid Pitfalls” (Transferindo sua infraestrutura para a nuvem: como maximizar as vantagens e evitar as armadilhas, http://www.rackspace.com/knowledge_center/whitepaper/moving-your-infrastructure-to-the-cloud-how-to-maximize-benefits-and-avoid-pitfalls), “Cost Savings, Efficiencies Lead IT Pros to Cloud Computing” (Economia nos custos e eficiência levam profissionais de TI para a computação em nuvem, <http://searchcloudcomputing.techtarget.com/feature/Cost-savings-efficiencies-lead-IT-pros-to-cloud-computing>) e “To Find Cloud Cost Savings, All You Need Is a Little Patience” (Para identificar economias com a nuvem, tudo de que você precisa é de um pouco de paciência, <http://searchcloudcomputing.techtarget.com/feature/To-find-cloud-cost-savings-all-you-need-is-a-little-patience>) oferecem esclarecimentos adicionais sobre todo o quadro geral da economia.
 - 2 Por exemplo, veja “Dropbox Drops the Security Notification Ball, Again” (Dropbox pisou na bola na notificação de segurança novamente, <http://www.zdnet.com/article/dropbox-drops-the-security-notification-ball-again/>) e “Wary of Privacy Issues? Ditch Dropbox and Avoid Google, Says Edward Snowden” (Desconfiado de questões de segurança? Deixe de lado o Dropbox e evite o Google, diz Edward Snowden, <http://www.newsweek.com/wary-privacy-issues-ditch-dropbox-and-avoid-google-says-edward-snowden-276956>).

PARTE II

Aplicando práticas de programação bem-sucedidas

Nesta parte do livro, começaremos vendo as técnicas para desenvolver aplicações seguras. Cada um dos próximos capítulos discute uma parte em particular de uma aplicação típica baseada em web. O capítulo 4 começa com a interface de usuário, que é a parte mais importante em potencial, pois a segurança começa e termina com a cooperação do usuário. Se a interface tiver falhas, o usuário simplesmente não cooperará (pelo menos, não tão bem) e sua solução de segurança conterà falhas. O capítulo 5 anda de mãos dadas com o capítulo 4. É frustrante para um usuário interagir com uma aplicação que não seja confiável, e isso tende a criar mais problemas de segurança ainda, tanto direta quanto indiretamente.

Aplicações modernas não existem no vácuo. Se um desenvolvedor criasse toda aplicação do zero, todos sofreriam. O uso de bibliotecas (capítulo 6), APIs (capítulo 7) e microsserviços (capítulo 8) torna o processo de criação de uma aplicação muito mais rápido e fácil. Além disso, como o código oferecido por essas fontes de terceiros é analisado minuciosamente, ele tende a ser mais seguro que o código independente que você desenvolveria e, em geral, recebe atualizações mais rápido do que sua empresa poderia fornecer para códigos personalizados.

CAPÍTULO 4

Desenvolvendo interfaces bem-sucedidas

Aplicações contam com interfaces para interagir com o usuário. Quando a interface apresenta falhas, a opinião do usuário sobre a aplicação é ruim e ele fica insatisfeito. Toda solução de segurança que você conceber depende da boa vontade do usuário para torná-la uma realidade. Sim, você pode tentar medidas draconianas para forçar a participação do usuário, mas, com frequência, essas medidas resultam no usuário constantemente encontrando maneiras de driblar a segurança, em vez de trabalhar com ela. Na maioria dos casos, você terá melhores resultados quando o usuário estiver do seu lado, o que significa fazer um esforço para criar uma interface que o usuário possa apreciar (no sentido em que a aplicação seja tão fácil de usar que ela quase desaparece da vista). É claro que você precisa certificar-se de que a aplicação garanta o cumprimento das políticas. Desse modo, este capítulo discute tanto a cenoura (cooperação do usuário) quanto a vareta (políticas a serem cumpridas)¹ para a segurança da aplicação do ponto de vista da interface.

É importante deixar óbvia a parte da equação referente à cenoura. Um usuário vê diversos tipos de imagens atrativas, dicas úteis e uma interface clara como sinais de que o desenvolvedor realmente se importa com a aplicação e como o usuário a vê. Deixar a experiência de usuário agradável é essencial.

A parte da equação referente à vareta normalmente permanece oculta e é sutil. Por exemplo, em vez de pedir que o usuário digite o nome de um Estado, a aplicação evita entradas não permitidas criando uma lista de Estados e faz o usuário selecionar um item da lista. Embora as opções do usuário sejam limitadas, o que o usuário vê é a facilidade de uso.

Algumas decisões de interface são errôneas antes mesmo de você gerar

código para elas. Por exemplo, algumas técnicas de validação informam ao usuário que a entrada é inaceitável sem lhe dizer o porquê de essa entrada não ser adequada. Acima de tudo, quando o feedback não contém dicas úteis e exemplos, o usuário fica frustrado. Este capítulo também discute técnicas que você pode usar para deixar a interface mais sofisticada. Não é uma questão de algumas técnicas funcionarem e outras não – é mais uma questão de algumas técnicas deixarem o usuário menos angustiado.



É importante entender que poucas pessoas programariam um site manualmente hoje em dia. Poucas pessoas programariam um site sem usar bibliotecas, APIs, frameworks, microsserviços e várias outras ofertas de terceiros. No entanto, todos esses códigos-fonte de terceiros dependem das tecnologias descritas neste capítulo. Tudo que você obtém de terceiros é um código previamente empacotado que você mesmo poderia ter criado. Ver o código da maneira como é mostrado neste capítulo ajuda a entender as técnicas subjacentes usadas por códigos de terceiros para que você possa perceber melhor se esses códigos são seguros. Com isso em mente, o código deste capítulo foi projetado para ajudar você a entender os princípios de segurança, e não como algo que você realmente empregaria em seu site.

Acessando a interface de usuário

A maioria das empresas precisa investir tempo na avaliação da interface de usuário de suas aplicações porque elas criam oportunidades claras para os hackers terem acesso à sua rede. A seção “Especificando as ameaças às aplicações web” do capítulo 1, descreve alguns exploits comuns usados pelos hackers. Um número surpreendente desses exploits depende de algum elemento da interface de usuário para ajudar na tarefa. Quanto mais rígida você puder deixar sua interface sem inibir a interação com o usuário, melhor. As próximas seções descrevem algumas das técnicas que você pode usar para avaliar a interface de usuário.



A melhor maneira de trabalhar com os exemplos descritos neste capítulo é usar o código-fonte que pode ser baixado, em vez de digitá-lo manualmente. Usar o código-fonte baixado reduz os erros em potencial. Você pode encontrar os exemplos deste capítulo na pasta `\S4WD\Chapter04` no código-fonte disponível para download.

Criando uma interface clara

Uma parte essencial na criação de interfaces funcionais atualmente é deixá-las claras. As antigas e complexas interfaces criam problemas de segurança porque o usuário não tem certeza do que deve fazer ou, possivelmente, tem tarefas demais a fazer. As bibliotecas disponíveis hoje possibilitam criar interfaces claras que enfocam uma questão de cada vez em várias situações. Por exemplo, a interface com abas exibida na figura 4.1 mostra um exemplo de uma interface clara, que só executa uma tarefa de cada vez, como parte do arquivo *Tabs.html*.

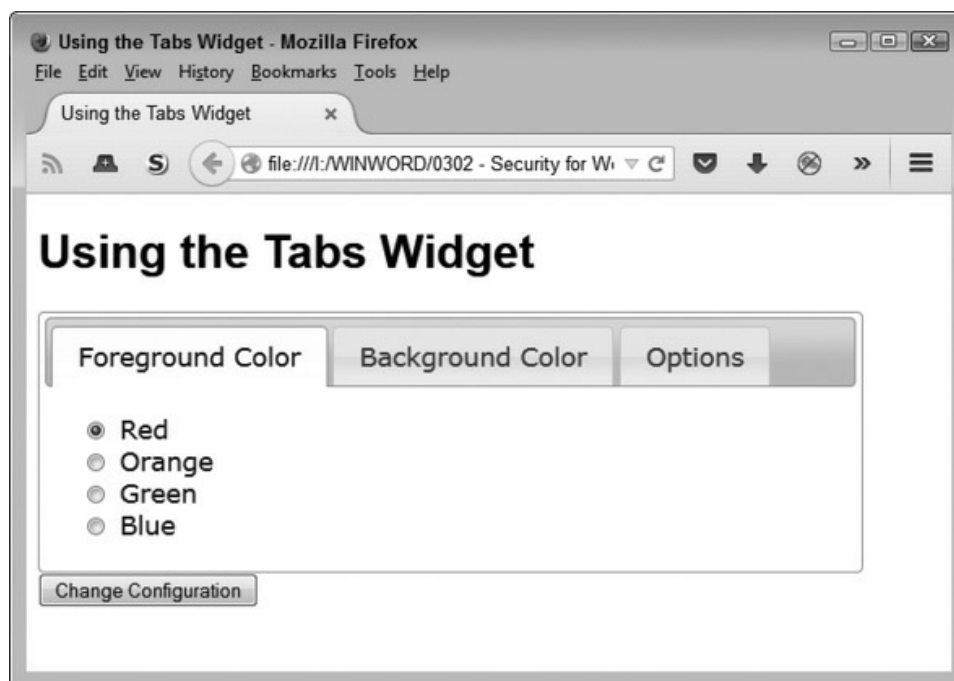


Figura 4.1 – Uma interface clara enfoca apenas uma questão, de maneira que o usuário possa entender.

A interface é bem clara. Ela faz uma única pergunta, oferece um número limitado de respostas e dificulta a entrada de dados indevidos pelo usuário. O usuário seleciona uma cor, clica no botão e passa para a próxima aba. Várias aplicações web atualmente usam esse tipo de interface. Você verá essa interface com frequência em páginas de inscrição. O exemplo conta com a biblioteca jQuery UI (<https://jqueryui.com/>) para executar sua tarefa. Eis o código-fonte desse exemplo:

```
<!DOCTYPE html>
```

```
<html>
<head>
  <script
    src="https://code.jquery.com/jquery-latest.js">
  </script>
  <script
    src="https://code.jquery.com/ui/1.9.2/jquery-ui.js">
  </script>
  <link
    rel="stylesheet"
    href="https://code.jquery.com/ui/1.9.2/themes/base/jquery-ui.css" />
  <title>Using the Tabs Widget</title>
  <style>
    #Configuration
    {
      width: 90%;
      text-align: center;
    }
    #div de Configuration
    {
      text-align: left;
    }
  </style>
  <script language="JavaScript">
    $(function()
    {
      $("#Configuration").tabs();
    });
  </script>
</head>
<body>
  <h1>Using the Tabs Widget</h1>
  <form id="ConfigForm" method="get" action="Tabs.html">
    <div id="Configuration">
      <ul>
        <li><a href="#Tab1">Foreground Color</a></li>
        <li><a href="#Tab2">Background Color</a></li>
```

```
<li><a href="#Tab3">Options</a></li>
</ul>
<div id="Tab1">
  <input id="FGRed"
    type="radio"
    name="Foreground"
    value="Red"
    checked="checked" />
  <label for="FGRed">Red</label><br />
  <input id="FGOrange"
    type="radio"
    name="Foreground"
    value="Orange" />
  <label for="FGOrange">Orange</label><br />
  <input id="FGGreen"
    type="radio"
    name="Foreground"
    value="Green" />
  <label for="FGGreen">Green</label><br />
  <input id="FGBlue"
    type="radio"
    name="Foreground"
    value="Blue" />
  <label for="FGBlue">Blue</label>
</div>
<div id="Tab2">
  <input id="BGRed"
    type="radio"
    name="Background"
    value="Red"
    checked="checked" />
  <label for="BGRed">Red</label><br />
  <input id="BGOOrange"
    type="radio"
    name="Background"
    value="Orange" />
  <label for="BGOOrange">Orange</label><br />
```

```

        <input id="BGGreen"
            type="radio"
            name="Background"
            value="Green" />
        <label for="BGGreen">Green</label><br />
        <input id="BGBlue"
            type="radio"
            name="Background"
            value="Blue" />
        <label for="BGBlue">Blue</label>
    </div>
    <div id="Tab3">
        <input id="Sounds"
            type="checkbox"
            name="Sounds"
            value="SpecialSounds" />
        <label for="Sounds">Use Special Sounds</label><br />
        <input id="Effects"
            type="checkbox"
            name="Effects"
            value="SpecialEffects" />
        <label for="Effects">Use Special Effects</label>
    </div>
</div>
<input id="ChangeConfig"
    type="submit"
    value="Change Configuration" />
</form>
</body>
</html>

```

Para criar essa interface, importe as bibliotecas jQuery (<https://jquery.com/>) e jQuery UI, e a folha de estilo associada da jQuery UI. A informação das abas está em uma <div>, cujo id é Configuration. A mágica usada para criar a interface é o resultado da chamada a `$("#Configuration").tabs()`.

Deixando as interfaces flexíveis

Reduzir a capacidade de o usuário inserir dados inválidos, deixando a interface clara e concisa, e manter a simplicidade restringem o usuário de várias maneiras. Há formas de restrições que ajudam a manter os dados seguros, ao mesmo tempo que simplificam a interação do usuário com a aplicação. Contudo, as interfaces também exigem flexibilidade. O usuário utilizará a aplicação em vários dispositivos, mas nem todos aceitarão a organização inicial oferecida por você. O ato de deixar o usuário organizar a tela conforme for necessário dá ao usuário uma sensação de poder, sem deixar nenhuma brecha na segurança. Sempre que possível, uma interface deve permitir que o usuário execute as seguintes ações:

Arrastar

Mover itens pela tela pode oferecer melhor visão da interface ao usuário e, potencialmente, reduzir entradas incorretas. No mínimo, reorganizar a interface permite ao usuário interagir com a aplicação da maneira que lhe seja mais conveniente.

Redimensionar

Há muitos motivos para o usuário precisar redimensionar um elemento da interface. Talvez o texto seja pequeno demais. Combinar a ação de redimensionar com a capacidade oferecida pela maioria dos navegadores de deixar o texto menor ou maior conforme for necessário ajuda o usuário a ver a informação de modo mais conveniente.

Selecionar

Criar feedbacks positivos para seleções é essencial. Você não deve pressupor nada sobre a capacidade do usuário de ver. A seleção deve oferecer vários tipos de feedback, por exemplo, usar uma borda mais larga, apresentar a seleção em uma cor mais clara (mais brilhante), contar com a mudança de atributos textuais e oferecer feedback na forma de cores. Essa é uma ocasião em que ter alguém na equipe para verificar os efeitos das seleções usando recursos de acessibilidade, como leitores de tela, é importante.

Ordenar

O usuário deve ver os dados em uma ordem que faça sentido a ele – e não necessariamente a você. Com isso em mente, ofereça várias maneiras de ordenar dados sempre que for possível para que o usuário possa realizar a análise necessária e fazer seleções com mais facilidade.

As bibliotecas jQuery e jQuery UI oferecem os meios para realizar todas as tarefas essenciais de interface com flexibilidade, além de várias outras tarefas (por exemplo, permitir entrada de dados arrastando e soltando). A figura 4.2 mostra um exemplo de uso dessas bibliotecas para mover itens pela tela.

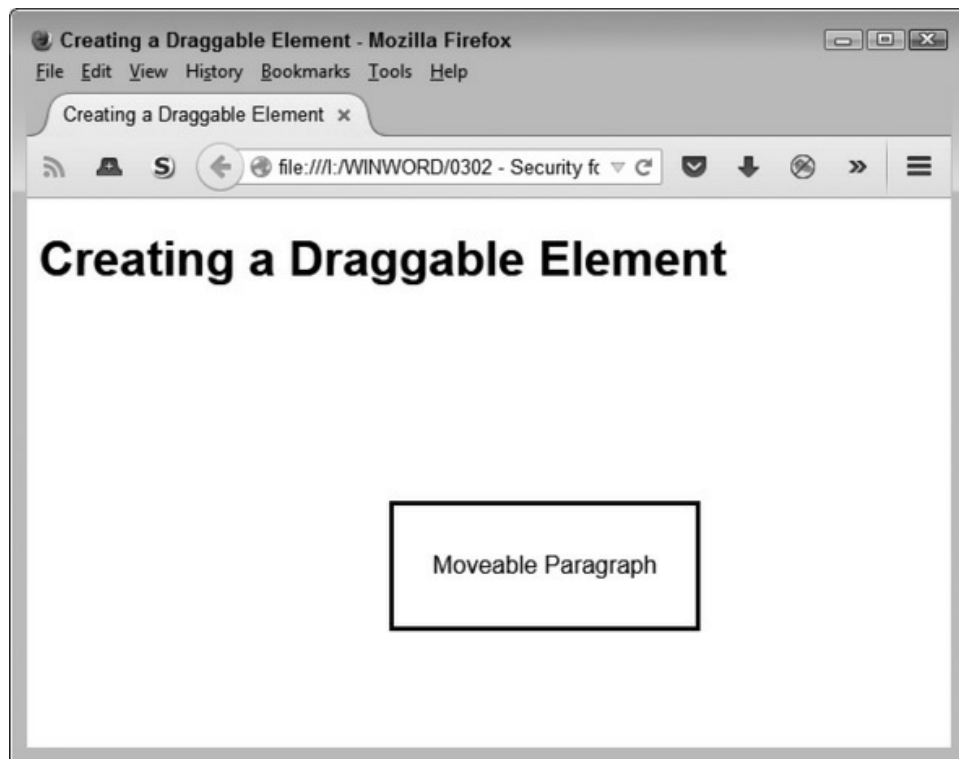


Figura 4.2 – Permitindo mover itens na tela.



Uma biblioteca pode ajudar você a criar um ambiente apropriado para a entrada de dados de usuário. No entanto, ela não pode eliminar a necessidade de validar todos os dados que você receber. Hackers gostam quando os desenvolvedores fazem pressuposições sobre entradas baseadas na interface. Sempre suponha que um hacker encontrará uma maneira de contornar os recursos de interface que você oferece a usuários honestos e valide todos os dados, independentemente de sua origem.

Criar uma interface com elementos que podem ser movidos é relativamente simples. O código a seguir mostra como fazer isso (esse código também pode

ser visto no arquivo *DragContent.html*):

```
<!DOCTYPE html>
<html>
<head>
  <script
    src="https://code.jquery.com/jquery-latest.js">
  </script>
  <script
    src="https://code.jquery.com/ui/1.9.2/jquery-ui.js">
  </script>
  <link
    rel="stylesheet"
    href="https://code.jquery.com/ui/1.9.2/themes/base/jquery-ui.css" />
  <style>
    #MoveMe
    {
      border: solid;
      width: 200px;
      height: 5em;
      text-align: center;
      line-height: 5em;
    }
  </style>
  <script language="JavaScript">
    $(function()
    {
      $("#MoveMe").draggable();
    });
  </script>
  <title>Creating a Draggable Element</title>
</head>
<body>
  <h1>Creating a Draggable Element</h1>
  <p id="MoveMe">
    Moveable Paragraph
  </p>
```

```
</body>  
</html>
```

Nesse caso, o parágrafo MoveMe (elemento `<p>`) é o alvo. Observe que deixar o parágrafo móvel não afeta o cabeçalho (elemento `<h1>`). Tudo que você precisa fazer é chamar `$("#MoveMe").draggable()` quando o formulário é carregado para deixar o elemento de parágrafo móvel.

Oferecendo auxílio ao usuário

Criar um ambiente em que até o menos capacitado dos usuários possa trabalhar bem é difícil para a maioria dos desenvolvedores, porque eles não conseguem nem ao menos imaginar como é não poder se relacionar bem com um computador. Muitas das pessoas a quem um desenvolvedor dá suporte com uma aplicação são bem inteligentes – elas simplesmente não entendem de computadores ou a maneira como as aplicações funcionam, portanto é importante dar-lhes suporte com vários tipos de ajuda.

O item de ajuda que você sempre deve oferecer é um texto explicativo. Acrescente o texto explicativo como parte da tag HTML associada a um elemento. A seguir, apresentamos os atributos mais comumente utilizados para um texto explicativo:

title

O atributo `title` permite acrescentar informações explicativas a praticamente qualquer elemento textual ou de controle na página. Quando um usuário passa o mouse sobre um elemento que tenha o atributo `title` definido, o navegador exibe as informações adicionais em um balão contendo o texto. É importante não deixar o texto longo demais nem usar nada além de texto no atributo `title` porque leitores de tela contam com esses atributos para descrever um elemento a alguém com necessidades visuais especiais.

alt

Use o atributo `alt` para oferecer uma descrição breve de um elemento de imagem. Quando o navegador não puder exibir a imagem, ele mostrará a descrição em seu lugar. Além disso, quando alguém com necessidades especiais não puder ver a imagem, os recursos de acessibilidade usados

normalmente utilizam o texto do atributo alt para apresentar a imagem. O ponto principal é que o atributo alt oferece uma descrição da imagem, além de atuar como substituto para ela, se necessário.

longdesc

Muitos navegadores ignoram o atributo longdesc. Este atributo oferece uma descrição muito mais extensa que o atributo alt para uma imagem. Na maioria dos casos, você deve limitar a descrição a aproximadamente um parágrafo de texto. A questão é oferecer informações suficientes para que alguém com necessidades especiais possa criar uma versão mental razoável da imagem – pelo menos dos pontos de destaque dela (e não do plano de fundo).

Informações de ajuda ao usuário podem estar em vários outros formatos. Um tipo de ajuda que a maioria dos sites não tem é oferecer um exemplo do que você quer como entrada em caixas de texto e em outros dados digitados como parte da entrada. Com frequência, você vê caixas de texto que contêm uma descrição do conteúdo desejado, e não um exemplo desse conteúdo. (Quando o usuário digita algo na caixa de texto, o texto de exemplo desaparece.) Por exemplo, em vez de oferecer Last Name (Sobrenome) como o texto da caixa, forneça um nome de verdade, por exemplo, Doe (como em John Doe), ou algo nessa linha.

Qualquer campo de entrada deve disponibilizar também um link de ajuda. Quando o usuário clicar no link, a aplicação exibe uma página de ajuda com informações adicionais. A questão é simplificar ao máximo para que o usuário forneça as informações corretas. Qualquer erro que você puder evitar desde o início reduzirá a frustração do usuário, deixará a entrada de dados mais precisa e, acima de tudo, evitará erros que, muitas vezes, resultam em problemas de segurança.

Definindo os problemas de acessibilidade

Acessibilidade pode ser uma questão complicada para lidar, especialmente quando trabalhamos com ferramentas modernas. No entanto, é uma questão importante a ser considerada porque, de acordo com a WebAIM (Web Accessibility In Mind), aproximadamente 20% das pessoas que acessam seu

site têm algum tipo de necessidade especial: visual, auditiva, de controle motor ou cognitiva. À medida que a população mundial continua a envelhecer, a porcentagem aumentará. De fato, você tem uma alta probabilidade de precisar de algum tipo de assistência para uma necessidade especial em algum momento.

Você pode estar se perguntando o que a acessibilidade tem a ver com segurança. Quando as pessoas não entendem uma interface, elas cometem erros. Porém, o treinamento normalmente ajuda nesse tipo de problema porque você pode instruir as pessoas quanto ao uso adequado do software (e, às vezes, elas realmente ouvem). Quando as pessoas não conseguem entender uma interface por causa de uma necessidade especial, elas continuarão a cometer erros, mas nenhum treinamento jamais resolverá o problema. Mesmo que o usuário esteja realmente motivado a usar o software de forma correta, necessidades de acessibilidade não atendidas as impedirão de fazer isso. Erros com frequência se traduzem em brechas de segurança do pior tipo. Uma entrada inesperada – aquela que você jamais imaginou que alguém pudesse fornecer, frequentemente cria os maiores problemas. É mais provável que uma aplicação que não tenha recursos de acessibilidade crie um ambiente no qual erros que comprometam a segurança ocorram com regularidade.

Felizmente, muitos fornecedores projetam bibliotecas, APIs, frameworks e microsserviços com algum nível de acessibilidade em mente. Eles evitam o uso de tabelas como recurso de formatação, por exemplo, pois isso causa problemas com os leitores de tela usados por aqueles que têm necessidades visuais. No entanto, nenhuma dessas ferramentas é perfeita, e algumas delas são menos perfeitas que outras. Consequentemente, é importante testar seu site para determinar qual é o nível de acessibilidade que ele oferece. A maioria dessas ferramentas de teste também oferece dicas sobre como corrigir os problemas de acessibilidade encontrados em seu site. A tabela 4.1 apresenta uma lista de ferramentas de teste e uma descrição de como você pode usá-las.

Tabela 4.1 – Sites para testes de acessibilidade

Site	URL	Descrição

Lynx	http://www.delorie.com/web/lynxview.html	É um navegador somente para Linux, OS X e Windows. Exibindo-o somente como texto, a ver seu site como um leitor com que seja possível identificar com mais facilidade. Saiba mais em http://lynx.browser.org/ .
Conjunto de ferramentas NIST Webmetrics	http://zing.ncsl.nist.gov/webmet	Um grupo de ferramentas do National Institute of Standards and Technology (Padrões e Tecnologia) que avaliam a usabilidade e a acessibilidade. O WebSAT (Web Static Analysis) e o WebVIP (Web Usability Inspection Program) atende a usabilidade. O WebVIP (Web Usability Inspection Program) ajuda a monitorar que você possa saber como encontrar os recursos do site nesse site, e o NIST as atualizações.
Opera	http://www.opera.com	Assim como o Lynx, este navegador também apresenta o site como um leitor de tela. Diferente do Lynx, este navegador também habilita e a desabilita recursos conforme for necessário. Por exemplo, você pode alterar o tamanho das imagens para ver a aparência. Para mais instruções, visite http://help.opera.com/Windows/ . O Opera está disponível para Windows, incluindo Windows, OS X e Linux.
O'Reilly XML.com	http://www.xml.com/pub/a/tools/ruwf/check.html	Este site oferece um verificador de XML também valida XHTML. Você pode validar XML diretamente ou um teste executado por essa ferramenta. Se o XML está bem formado – ele não pode realmente dizer algo. Na verdade, quer usar o W3C HTML Validator, uma verificação final de sua página. Este site realiza verificação intermediária e de teste.

W3C HTML Validation Service	http://validator.w3.org	Este site verifica o HTML e se está de acordo com as regras W3C (World Wide Web Consortium). O site quer dizer que o código é incorreto, mesmo que a maioria não perceba, portanto essa ferramenta verifica requisitos de usabilidade e supõe que passar no teste automaticamente acessível. O fato de que seu código está correto e seu código esteja correto é garantir que você pode acessibilidade.
Web Design Group HTML Validator	http://www.htmlhelp.com/tools/validator/	Este site verifica o HTML de uma alternativa, de seu computador, uma opção para validar um documento. Talvez você ache esse site mais exigente que o W3C HTML Validator, mas às páginas que ele verificar as mesmas informações, mas a validação completa de seu site.
WebAIM	http://webaim.org	Você pode encontrar ferramentas interessantes para verificar seu site. Por exemplo, você pode usar o Wave Accessibility Evaluation Tool (http://wave.webaim.org/) para verificar se tem algum problema importante. O problema sério para muitos desenvolvedores é o contraste das cores. O Color Contrast Analyzer (http://webaim.org/resource/contrast_analyzer) pode verificar se as pessoas com deficiência visual podem realmente enxergar as cores.

Ferramentas de teste são úteis para verificar se sua equipe de desenvolvimento fez um bom trabalho para criar uma aplicação que todos possam usar. No entanto, o esforço de desenvolvimento propriamente dito muitas vezes exige uma lista de verificação de tarefas a serem realizadas e dos possíveis problemas. A lista de verificação Section 508 em <http://webaim.org/standards/508/checklist> ajuda a criar uma lista desse tipo.

para testar os esforços de desenvolvimento de sua aplicação. O interessante é que, por mais difícil que possa ser ler essa lista de verificação, a documentação do governo em <http://www.section508.gov/> é muito mais difícil. A questão principal desses requisitos é garantir que qualquer pessoa, independentemente das necessidades especiais que possa ter, seja capaz de usar sua aplicação conforme planejado. Sem esses recursos presentes, você poderá descobrir que até 20% de seus usuários têm problemas que eles realmente não deveriam ter.



Se você já trabalhou com questões de acessibilidade em aplicações há tempo suficiente, deve conhecer o ícone Bobby Approved (Aprovado por Bobby) que costumava aparecer em alguns sites. Infelizmente, Bobby não está mais disponível – pelo menos, não como um serviço gratuito. Você pode ler a história de Bobby em <http://www.bobby-approved.com/>. O aspecto importante a ser observado é que a acessibilidade é uma questão muito importante e Bobby fez um ótimo trabalho em ajudar as pessoas a acrescentá-la em seus sites a ponto de a IBM, em algum momento, ter comprado o produto. Bobby continua presente – porém não é mais gratuito.

Oferecendo opções controladas

Segurança tem a ver com exercer controle. A aplicação deve ter controle sobre o modo como ela interage com os dados. Gerenciar dados, sem danificá-los ou causar vazamento, é essencial para criar uma interface segura. Uma das melhores maneiras de exercer controle é usar técnicas que garantam que as escolhas do usuário estejam limitadas somente àquelas que você espera. O uso de controles específicos de entrada limita as opções do usuário, mas também o deixa mais eficiente, porque o usuário precisa pensar menos sobre as opções disponíveis. Eis as opções específicas de controle de entrada mais comuns:

Botões de rádio

Deixa o usuário escolher uma opção entre várias. Selecionar uma nova opção sempre remove a seleção da opção anterior. O erro mais comum cometido pelos desenvolvedores é não definir uma das opções como default. Quando o dado é enviado ao servidor depois que o usuário falhar em selecionar uma opção, o servidor encontrará uma entrada de dado em

branco e poderia ter problemas em potencial.

Caixas de seleção

Permite ao usuário escolher entre nada e todas as opções disponíveis, marcando as opções desejadas. O erro mais comum cometido pelos desenvolvedores é permitir que opções conflitantes ocorram. Ao trabalhar com caixas de seleção, cada uma das opções deve ser mutuamente exclusiva.

Listas (list boxes)

Conforme a configuração, uma lista pode atuar como um botão de rádio ou como uma caixa de seleção. Uma vantagem das listas é que você pode configurá-las para ocultar as opções quando o usuário não tiver selecionado o controle de lista. Além dos problemas que os desenvolvedores frequentemente têm com botões de rádio e caixas de seleção, as listas podem ter problemas no preenchimento das opções. O usuário deve ter pelo menos uma opção disponível para que a lista atue como um controle. Fontes de dados inacessíveis podem ser fatais para esse controle.

Menus

Dependendo da configuração, menus podem permitir combinações complexas de comportamentos de botão de rádio e de caixa de seleção. É claro que os menus também são usados para criar opções de aplicações do lado cliente. Muitos desenvolvedores falham com menus ao deixá-los complicados demais. Um usuário não fará buscas demoradas por uma opção em particular, portanto deixar o menu complexo demais resulta em erros quando o usuário fica frustrado.

Além dos tipos de entrada padrão (incluindo tipos mais antigos, como as senhas), você também pode usar funcionalidades especiais de HTML5 para criar um ambiente controlado de opções. No entanto, nesse caso, a alternativa é oferecer ao usuário uma caixa de texto de entrada, o que significa que essas opções não são tão seguras quanto você imagina. Eis os controles específicos de HTML5:

color

Atua como um botão na maioria dos navegadores que o aceitam. Clicar no botão exibe o selecionador de cores (color picker) da plataforma. Quando o usuário escolhe uma nova cor, ela aparece no botão para que o usuário possa ver a opção atual. Versões mais recentes de Firefox, Chrome e Opera aceitam esse controle.

date

Exibe um selecionador de data (date picker) que o usuário pode utilizar para selecionar uma data específica. Os atributos min e max permitem definir o intervalo de datas possíveis. Versões mais recentes de Chrome, Safari e Opera aceitam esse controle.

datetime

Este tipo de entrada atualmente não é tratado por nenhum navegador, portanto você não deve usá-lo, apesar de as especificações dizerem que ele está disponível.

datetime-local

Oferece uma combinação dos controles de data e hora para que um usuário possa escolher tanto uma data quanto uma hora (com as restrições desses controles definidas). Ao especificar os atributos min e max, forneça tanto os valores de data quanto de hora como entrada. Versões mais recentes de Chrome, Safari e Opera aceitam esse controle.

email

Exibe um controle padrão de entrada de texto. No entanto, o controle valida automaticamente a entrada para garantir que o usuário forneça um endereço de email. Versões mais recentes de Internet Explorer, Firefox, Chrome e Opera aceitam esse controle.

month

Exibe um selecionador de data que o usuário pode utilizar para selecionar um mês e um ano específicos. Os atributos min e max permitem definir o intervalo de datas possíveis (apesar de estar trabalhando com meses, você deve fornecer datas para esses atributos). Versões mais recentes de Chrome, Safari e Opera aceitam esse controle.

number

Permite que um usuário digite qualquer valor numérico, mas não um valor alfabético ou um símbolo especial. Definir valores para os atributos min e max permite controlar o intervalo da entrada. Versões mais recentes de Internet Explorer, Firefox, Chrome, Safari e Opera aceitam esse controle.

range

Aparece como um slider (controle deslizante) na maioria dos navegadores. Definir valores para os atributos min e max permite controlar o intervalo da entrada. Use o atributo value para controlar a configuração do intervalo inicial. O atributo step controla quanto o slider desloca para cada mudança de posição para que um usuário possa selecionar apenas valores específicos. Versões mais recentes de Internet Explorer, Firefox, Chrome, Safari e Opera aceitam esse controle.

search

Não faz muito sentido usar esse controle HTML5 porque ele se comporta como uma tag `<input>` padrão. Versões mais recentes de Chrome e Safari aceitam esse controle.

tel

Exibe um controle-padrão de entrada de texto. No entanto, o controle valida automaticamente a entrada para garantir que o usuário forneça um número de telefone devidamente formatado. Versões mais recentes de Safari aceitam esse controle.

time

Exibe um selecionador de horário (time picker) que o usuário pode utilizar para selecionar um horário específico (mas não um fuso horário). Os atributos min e max permitem definir o intervalo de horários possíveis. Versões mais recentes de Chrome, Safari e Opera aceitam esse controle.

url

Exibe um controle-padrão de entrada de texto. No entanto, o controle valida automaticamente a entrada para garantir que o usuário forneça um URL devidamente formatado. Versões mais recentes de Internet Explorer,

Firefox, Chrome e Opera aceitam esse controle.



O controle de entrada url é o único controle que atualmente oferece suporte especial em smartphones (não encontrado em outros tipos de computador). Em alguns smartphones, o navegador adiciona uma tecla .com especial quando percebe o controle de entrada url.

week

Exibe um selecionador de data que o usuário pode utilizar para selecionar uma semana e um ano específicos. Os atributos min e max permitem definir o intervalo de datas possíveis (apesar de estar trabalhando com meses, você deve fornecer datas para esses atributos). Versões mais recentes de Chrome, Safari e Opera aceitam esse controle.



É importante lembrar que um hacker não joga de acordo com as regras nem se sente limitado pelos seus formulários de forma alguma. É possível driblar as proteções fornecidas por esses controles e submeter dados diretamente ao servidor em algum outro formato. A capacidade de passar por cima das proteções é o motivo pelo qual você deve conferir os intervalos e o tipo de qualquer dado recebido do cliente no servidor.

Para usar esses controles, utilize a tag `<input>`. Por exemplo, para criar um tipo de entrada numérico, você pode usar (consulte o arquivo *InputTag.html* para ver detalhes):

```
<input id="NumberInput" type="number" min=1 max=5 value=1 />
```

Nesse caso, o usuário vê um controle de entrada de texto, mas ele inclui um controle de setas para cima e para baixo, como mostra a figura 4.3. Além disso, o atributo value fornece um valor default. O usuário pode facilmente alterar o valor digitando um novo valor ou simplesmente usando o controle de seta para cima ou para baixo.

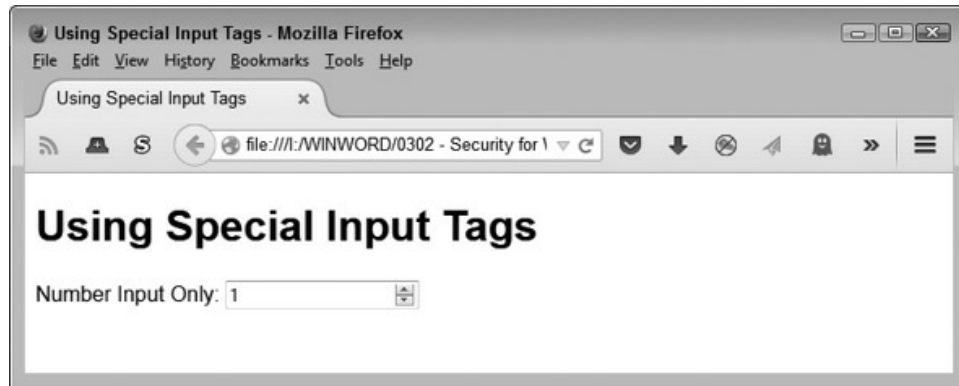


Figura 4.3 – Uma entrada numérica apresenta um controle de seta para cima e para baixo.

A entrada number não impede que o usuário digite um valor indesejado. No entanto, uma validação automática é realizada. Conforme o navegador, uma entrada incorreta recebe algum tipo de destaque. Por exemplo, no caso do Firefox, a borda se torna mais espessa e a caixa aparece em vermelho, como mostra a figura 4.4. A questão é que os acréscimos de HTML5 reduzem o trabalho a ser feito para usuários honestos, mas é possível que um hacker determinado passe por cima dele.



Alguns controles HTML5 são mais seguros que outros. Por exemplo, o tipo de entrada range normalmente aparece como um slider, de modo que o usuário não insere realmente um valor. Porém, você não deve contar com o fato de algum controle oferecer segurança absoluta.

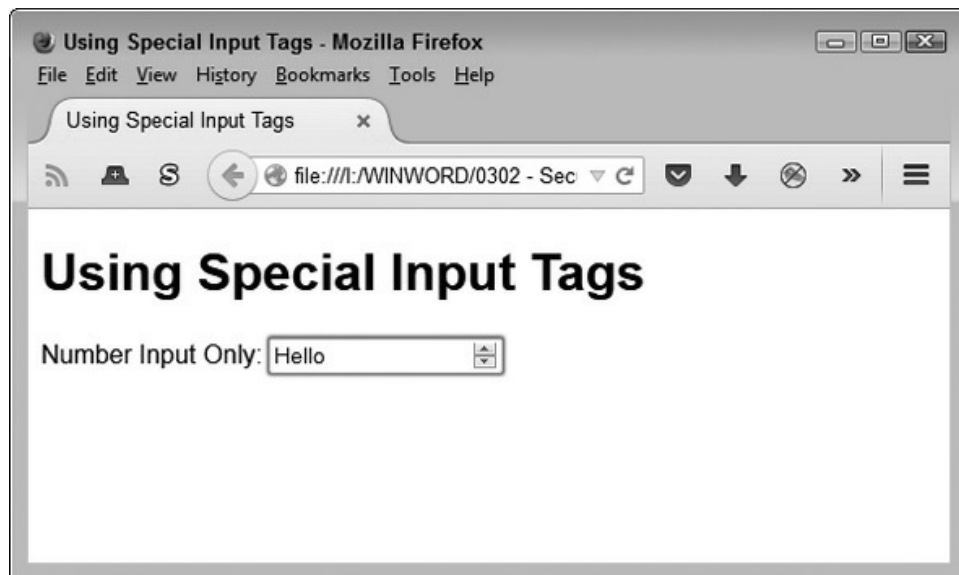


Figura 4.4 – Controles HTML5 oferecem validação automática.

Escolhendo um nível de solução para interface de usuário

Ao trabalhar com aplicações baseadas em web, você tem a opção de escolher o nível a ser utilizado para implementar a interface de usuário. De fato, a maioria das aplicações baseadas em web conta com vários níveis de controle de interface. Uma parte da página usa tags HTML, outra parte manipula a interface usando CSS (Cascading Style Sheets, ou Folhas de Estilo em Cascata) e uma terceira depende de JavaScript. É importante lembrar que você pode usar JavaScript tanto no cliente quanto no servidor, portanto, na verdade, há quatro níveis de soluções que você pode usar, conforme descrito nas próximas seções.

Implementando controles HTML padrão

A seção “Oferecendo opções controladas” na página 116 discute maneiras de controlar as opções do usuário. Muitas dessas opções aparecem na forma de controles HTML. A vantagem dos controles HTML é que eles funcionam automaticamente na maioria das ocasiões. Um navegador deve oferecer suporte para o nível de HTML exigido pelo controle, mas esse é o único requisito.

De modo diferente da maioria das demais soluções de controle, o navegador não precisa dar suporte a scripting em muitos casos para obter um efeito. É claro que, se quiser realizar um trabalho detalhado, você precisará ter algum nível de suporte a scripting. Como exemplo, se quiser usar controles especiais para garantir que um usuário preencha um formulário apropriadamente, você pode executar essa tarefa e submeter o formulário ao servidor para processamento sem usar scripts. Por outro lado, se quiser executar uma tarefa do lado cliente, por exemplo, uma validação do lado cliente, você precisará de suporte para scripting na maioria dos casos (alguns controles HTML5 oferecem suporte rudimentar à validação).

Há algumas desvantagens em usar controles HTML. A mais óbvia é que os controles HTML são rudimentares. Se quiser acrescentar mais sofisticação ao seu site, os controles HTML provavelmente desapontarão você. Além disso, você não pode ir além do básico. É impossível criar uma boa interface com abas (como a que mostramos na figura 4.1) usando apenas controles HTML.

Trabalhando com controles CSS

O principal propósito de CSS é formatar o conteúdo de uma página de modo que ele não dependa de um navegador, de um dispositivo ou de uma plataforma específicos. Além do mais, a formatação não afeta as necessidades de acessibilidade porque o usuário normalmente pode substituir a formatação elegante de CSS por uma formatação mais simples, que funcione melhor com dispositivos para acessibilidade. Apesar do que foi dito, é possível usar CSS em outras tarefas variadas por meio de técnicas inteligentes de programação. Uma dessas tarefas é criar controles de vários tipos usando uma combinação de HTML, CSS e JavaScript, com ênfase em CSS. Em outras palavras, você cria o controle propriamente dito usando CSS, em vez de depender de HTML ou de JavaScript para executar a tarefa.

Ter ferramentas disponíveis quando usar CSS para criar controles ajuda. Um lugar para procurar as ferramentas necessárias é o Dynamic Drive (<http://www.dynamicdrive.com/>). Ao visitar o site, você verá que ele oferece acesso a todo tipo de ferramentas, incluindo o Button Maker (<http://tools.dynamicdrive.com/button/>), que é o foco deste exemplo em particular. A figura 4.5 mostra como o Button Maker aparece na primeira vez que você o acessa. Utilize as configurações dessa página para gerar o CSS necessário para criar um microbotão. No entanto, você poderia criar facilmente outros tipos de controle da mesma forma, usando outras ferramentas.

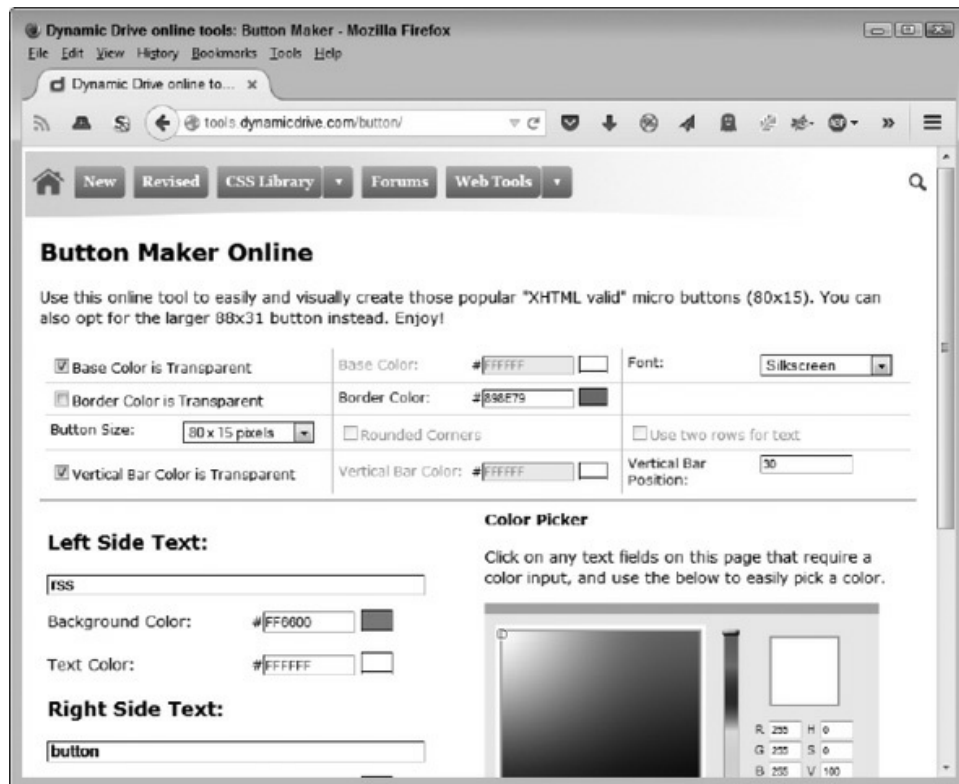


Figura 4.5 – A ferramenta Button Maker ajuda você a criar microbotões.

A saída do processo, nesse exemplo, é um pouco de CSS e um arquivo GIF (Graphic Interchange Format) chamado *MyButton.gif*. O código de exemplo (que se encontra em *TestButton.html*) é relativamente simples, como mostrado a seguir:

```
<!DOCTYPE html>
<html>
<head>
  <title>Testing a Micro Button</title>
  <style type="text/css">
    #MicroTest
    {
      border: none;
      background-color: transparent;
    }
  </style>
</head>
<body>
  <h1>Testing a Micro Button</h1>
```

```
<button id="MicroTest"
  onclick="alert('Clicked!')">
  
</button>
</body>
</html>
```

Observe como o código usa HTML e JavaScript como suporte. O botão propriamente dito é um arquivo GIF. O código CSS executa as tarefas de formatação exigidas. É claro que a formatação será mais complexa em uma aplicação de verdade. O resultado desse experimento é um botão como o que está na figura 4.6.



Figura 4.6 – O microbotão é totalmente funcional e elegante.

A principal vantagem de criar controles CSS é que tudo é gerado localmente, portanto eles tendem a ser bonitos, mas a aplicação também executa bem. Além disso, o CSS permite criar aplicações elegantes, muito mais divertidas de usar. A utilização de CSS também pode oferecer pistas visuais aos usuários, o que significa que você pode usá-las para reduzir erros – aumentando a segurança de sua aplicação.

O CSS também tem algumas limitações quando se trata de segurança. Para ter qualquer nível de validação, você deve empregar técnicas de HTML5 ou de JavaScript. O CSS não oferece qualquer tipo de potencial para validação porque nunca foi projetado para dar esse tipo de suporte. Lembre-se de que

os designers do CSS o projetaram visando a oferecer nada além de formatação básica de páginas. Somente por meio de uma programação inteligente você terá alguns dos efeitos interessantes oferecidos pelo CSS.

Criando controles com JavaScript

Quando se trata de controles, JavaScript oferece as opções que a maioria dos desenvolvedores usa para criar aplicações complexas. Sim, o desenvolvedor também conta com HTML5 para suporte a posicionamentos e com CSS para suporte à formatação, mas as tarefas essenciais de definição da interface ficam a cargo do JavaScript. Você tem acesso a dois níveis de suporte com JavaScript quando criar uma interface, e pode implementar os dois níveis em uma única aplicação. As próximas seções descrevem controles tanto do lado cliente quanto do lado servidor.

Contando com controles no cliente

Este capítulo contém alguns exemplos de uso de JavaScript de várias maneiras para criar configurações de controles do lado cliente. Em geral, você não criará os controles manualmente, mas usará uma configuração de biblioteca como a jQuery e a jQuery UI. Ao trabalhar com uma configuração do lado cliente, a aplicação carrega as bibliotecas de uma fonte local ou de um servidor remoto, mas depois que as bibliotecas são carregadas, tudo acontece localmente. Capítulos mais adiante no livro discutem as consequências de usar suporte de bibliotecas in-process (no mesmo processo) (veja os capítulos 6, 7 e 8 em especial). A principal vantagem do processamento no lado cliente é que você ganha velocidade e a aplicação é mais confiável em muitos casos porque você tem acesso constante a tudo que é necessário para fazer a aplicação executar.

JavaScript oferece as melhores oportunidades para validação de controles do lado cliente. Você pode configurar a validação para que funcione exatamente como quiser. Essa validação também pode oferecer mais funcionalidades que outros tipos de validação. No entanto, como um hacker pode ver seu código, é possível que ele elabore uma estratégia para contornar o que quer que você tenha implementado.

Um problema significativo de segurança ao usar controles do lado cliente é que você estará incorporando o código de outra pessoa à sua aplicação. Isso quer dizer que você tem uma relação de confiança entre sua aplicação e o código que você talvez nunca chegue a ver. Sim, você pode ver o conteúdo de bibliotecas como jQuery e jQuery UI, mas poucos desenvolvedores têm tempo para fazer isso e, certamente, não têm paciência para analisar um código essencialmente não documentado. Controles do lado cliente têm riscos em potencial associados a eles, que você deve considerar como parte da estratégia de segurança para a sua aplicação.

Contando com controles no servidor

Controles do lado servidor geralmente começam com um script PHP ou outro script no servidor. Em muitos casos, você pode chamar o script diretamente, passar-lhe os dados necessários e ver somente a saída do script em um navegador. Todo o processamento ocorre no servidor. Como o script é out-of-process (em outro processo), é mais fácil manter sua aplicação separada do código de terceiros, deixando-a mais segura. No entanto, você deve lidar com problemas de confiabilidade e de velocidade quando usar controles do lado servidor.

A parte do lado cliente dessa configuração normalmente depende de JavaScript. De fato, você pode incluir a saída do controle do lado servidor em uma página e atualizar esse conteúdo conforme for necessário usando algo como AJAX (Asynchronous JavaScript And XML, ou JavaScript Assíncrono e XML). Ao usar essa configuração, é possível agilizar a aplicação enviando apenas os dados ao servidor e atualizando apenas a parte da página afetada por eles.

Validando a entrada

De tudo que você pode fazer para garantir que sua aplicação permaneça segura, a mais importante é supor que todas as entradas que você receber são ruins. Ao pressupor que toda entrada que receber contém um vírus ou algum exploit que causará danos ao seu sistema, você começará a ver os dados de entrada de uma nova maneira. Sim, soa como paranoia, mas esse ponto de

vista é essencial para manter seus dados seguros. Validar dados de modo a refletir o ponto de vista de dados de entrada corrompidos é essencial no ambiente atual das aplicações. Mesmo assumindo esse ponto de vista, você poderá perceber que não está paranoico o suficiente (hackers são muito criativos para encontrar novas maneiras de fazer dados ruins parecerem bons) e que precisa contratar alguém para acrescentar mais paranoia ao seu comportamento. As próximas seções apresentam algumas ideias sobre como você pode proteger sua aplicação e seus dados associados validando absolutamente toda entrada de todas as formas que você puder imaginar.

Permitindo apenas entradas específicas

O problema que você vê com mais frequência quando se trata de validação de dados é que a aplicação não controlou bem os dados de entrada. Ao pensar nisso, você verá que esse é o problema subjacente de muitos dos exploits existentes. Jamais permita que um hacker passe um script ou outro conteúdo indesejado por meio de sua aplicação ao servidor. É claro que ninguém está disposto a permitir um conteúdo desse tipo, mas o conteúdo consegue chegar ao servidor, de qualquer modo. A lista a seguir apresenta algumas ideias dos tipos de verificações que você deve fazer em qualquer dado passado da aplicação para o servidor (e uma segunda verificação deve ser feita no servidor):

Tipo

Sempre verifique se o tipo do dado está correto. Strings são o tipo de dado mais difícil de verificar, pois podem conter de tudo, por definição. Use tipos de dados mais restritos sempre que for possível. Por exemplo, quando precisar de entradas numéricas, utilize um tipo de dado numérico para passá-lo, e não uma string.

Intervalo

Valide o intervalo de qualquer tipo de dado. Números são um tipo de dado óbvio nesse aspecto. No entanto, você pode conferir os intervalos de datas e horários também.

Expressões regulares

Determine se o formato dos dados está correto. Em alguns casos, é possível verificar strings usando expressões regulares que fazem a correspondência entre o padrão da string de entrada com um padrão prévio. Por exemplo, quando quiser um número de telefone como entrada, garanta que você realmente obtenha um número de telefone.

Caracteres especiais

Usuários normalmente não precisam enviar nenhum caractere especial à sua aplicação. Um hacker emprega caracteres especiais de várias maneiras – nenhuma delas é particularmente útil à sua aplicação. Portanto, eliminar conteúdos que incluam caracteres especiais é mandatório.

A melhor entrada possível de um usuário é o tipo de entrada que você espera desde o início. Por exemplo, se você quer que o usuário escolha entre vermelho, amarelo e verde, as únicas respostas aceitáveis são vermelho, amarelo e verde. Qualquer outra opção é inválida e você deve rejeitá-la. Quanto mais específica você puder deixar a entrada, mais fácil será manter sua aplicação e seus dados associados seguros.

Procurando dados de entrada furtivos

Hackers constantemente procuram maneiras de fornecer dados de entrada inesperados. A questão ao fazer isso é causar falhas na aplicação de maneiras específicas ou forçá-la a reagir de forma inesperada. Por exemplo, entradas furtivas é a causa-raiz de ataques de injeção de SQL. O hacker usa entradas de um formulário para criar uma situação em que a aplicação acabe se comportando de maneira inesperada. O resultado de um ataque desse tipo pode variar de dados corrompidos à execução de scripts que jamais deveriam fazer parte dos dados. Você pode encontrar informações adicionais sobre esse tipo de ataque em particular em <http://blogs.msdn.com/b/raulga/archive/2007/01/04/dynamicsql-sql-injection.aspx> e em https://www.owasp.org/index.php/SQL_Injection_Prevention_Cheat_Sheet.

Em determinada ocasião, pode parecer uma boa ideia permitir que o usuário forneça dados usados para criar scripts dinâmicos, mas você jamais deve fazer isso. Na verdade, é uma boa ideia não usar scripts dinâmicos, a menos que você



tenha certeza da fonte de dados usada para criar esse script.

Solicitando novas entradas

Um dos exploits favoritos empregado pelos hackers é procurar situações em que uma aplicação execute verificações rigorosas nos dados de entrada durante a primeira passagem, mas então essas verificações se tornam mais brandas nas passagens subsequentes. A situação é bem fácil de criar por causa do modo como muitas linguagens lidam com loops. Somente um desenvolvedor cuidadoso evitará a situação durante uma segunda tentativa de entrada ou em tentativas subsequentes. Toda passagem pela entrada de dados deve contar com o mesmo código para verificar e conferir a entrada. Independentemente de o usuário estar entrando com os dados pela primeira ou pela terceira vez, as verificações devem oferecer a mesma proteção.

No entanto, o problema assume outro significado quando você começa a questionar as motivações do usuário durante a entrada de dados. Por exemplo, você deve considerar simplesmente por que um usuário precisaria de dez tentativas para fornecer seu nome correto em um formulário. Provavelmente, você poderá supor que o usuário saiba seu nome – não é uma informação que eles se esqueçam repentinamente. Novamente, uma necessidade constante de fornecer dados óbvios à aplicação pode sinalizar algo diferente de um usuário esquecido. Você deve se perguntar se o formulário é claro o suficiente. Quando tiver certeza de que o formulário está claro e que você implementou a validação apropriada, comece a procurar outras causas para o problema, como um hacker tentando vários métodos para fazer sua aplicação falhar. Em suma, a maioria das aplicações deve permitir determinado número de tentativas e então encerrá-las enquanto espera a atenção do administrador.

Jamais suponha que os usuários estejam totalmente treinados. Alguns usuários exigem muito tempo de treinamento e jamais têm a imagem completa do que deveriam fazer exatamente. A menos que você queira passar seus dias procurando hackers ocultos em lugares em que eles nunca vão, considere o



treinamento dos usuários como um requisito de segurança fundamental. Um grande número de tentativas em um formulário pode apontar para a necessidade de mais treinamento, e não para um problema em potencial com um hacker.

Usando validação tanto do lado cliente quanto do lado servidor

A velocidade da aplicação é uma preocupação fundamental ao criar uma aplicação baseada em web. Um usuário pode perder o foco em um intervalo de tempo curto, de um a cinco segundos – não é muito tempo para fazer muito em termos de validação. No entanto, é essencial realizar validações para obter o tipo correto de entrada do usuário. Com isso em mente, usar validação do lado cliente parece ser o caminho certo. Uma rotina JavaScript pode rapidamente verificar entradas incorretas e informar isso ao usuário antes de os dados fazerem todo o trajeto até o servidor. O fato é que você realmente quer a validação do lado cliente.

Porém, a validação do lado cliente causa seu próprio conjunto de problemas. Os dados devem deixar o sistema host em algum momento e fazer o caminho até o servidor. Entre o instante em que o navegador envia os dados e o servidor os recebe, um hacker tem todo tipo de oportunidade para interceptá-los e alterá-los. Para garantir que os dados realmente sejam do tipo correto e tenham o conteúdo certo, você deve realizar uma segunda validação no servidor. Se ninguém adulterou os dados, a segunda validação acontecerá rapidamente e o usuário terá uma resposta em um intervalo de tempo razoável.

Algumas pessoas consideram exagerado o uso de validações tanto no lado cliente quanto no lado servidor. Contudo, você realmente deve executar ambas as validações. De fato, às vezes você terá ambas, queira ou não. Por exemplo, ao usar determinados controles HTML5, você terá uma validação automática. É verdade que a validação nem sempre é a mais exata ou completa, mas ela oferece certo nível de segurança ao usuário. Lembre-se

sempre de que a validação do lado cliente serve para a proteção do usuário e você a utiliza para manter o usuário satisfeito. Tudo que você puder fazer para reduzir a frustração do usuário também reduzirá os custos de suporte da aplicação e as preocupações com segurança.

A validação do lado servidor deve abranger todos os dados de entrada e de saída de forma completa. Não é suficiente verificar dados provenientes do cliente e então supor que ninguém adulterará o servidor. Hackers são motivados a atacar o servidor porque isso permite afetar muitas pessoas com apenas uma invasão. Consequentemente, você deve fazer validações em todas as entradas em todas as rotinas do servidor para garantir que a aplicação e seus dados permaneçam seguros.

O problema encontrado com maior frequência na validação é a velocidade da aplicação. Não são só os usuários que têm problemas quando uma aplicação é lenta – a carga do servidor também aumenta. Desse modo, você deve manter um equilíbrio quando se trata de validações. A segurança geralmente é uma questão de definir o nível de risco que você está disposto a aceitar ao perseguir metas específicas. É importante que a gerência entenda que você deve manter um equilíbrio como parte do processo de design e de desenvolvimento da aplicação.

Alguns desenvolvedores supõem erroneamente que colocar um hardware adicional resolve um problema. A adição de hardware reduz a confiabilidade do sistema como um todo. Uma redução de confiabilidade também é um risco à segurança. Se um hacker tiver cinco servidores para atacar, e não apenas um, a probabilidade de encontrar um servidor que não tenha os patches apropriados ou que esteja carregado a ponto de ter problemas é maior. Aumentar a quantidade de hardware usada para servir uma aplicação é parte da solução, mas você deve equilibrar isso com o aumento da velocidade da aplicação, sem criar grandes brechas de segurança.

Esperando o inesperado

Por vezes os usuários são impressionantes. Eles parecem encontrar a única brecha que você não pensou em corrigir. Um dia tedioso pode se transformar em uma sessão simplesmente para ver o que é preciso para quebrar uma

aplicação. Na verdade, o usuário pode nem mesmo estar disposto a quebrar a aplicação; talvez seja apenas uma questão de fazer algo sem prestar muita atenção, enquanto fala ao telefone. A questão é que os usuários encontrarão problemas que você jamais esperaria encontrar em sua aplicação. Isso pode resultar em todo tipo de problemas para proteger a aplicação e seus dados associados. Às vezes um problema pode fazer a aplicação ou, quem sabe, até mesmo o servidor em que ela estiver executando, falhar. Você pode ficar impressionado com o que os usuários conseguem fazer, mas, no mundo real, usuários entediados podem gerar eventos inesperados.

Hackers também farão o inesperado em sua aplicação. Entretanto, nesse caso, o inesperado é mais planejado e metódico. Um hacker continuará explorando suas defesas à procura de um caminho de entrada até outro alvo aparecer, o hacker ficará entediado ou ele conseguirá um acesso de entrada. Na maioria dos casos, o hacker encontrará um caminho de entrada quando estiver realmente determinado a fazer isso. Nunca se trata de saber se o hacker terá acesso à sua aplicação, mas de quando ele será bem-sucedido em seus esforços. O uso de medidas de segurança atrasa os hackers e, com uma monitoração apropriada, você pode descobrir o hacker espreitando seu sistema antes que seja possível que ele cause algum dano de verdade.

Muitas das tarefas inesperadas que os usuários e os hackers fazem estão associadas a entradas de dados. Por exemplo, você pode esperar uma string como entrada, mas recebe um script em seu lugar. Os usuários podem clicar combinações inesperadas de teclas que resultam em uma string corrompida que fazem a aplicação ou o servidor falharem. Às vezes, é simplesmente uma questão de supor que o usuário fará uma ação quando outra bem diferente parece ser mais apropriada. Por exemplo, um usuário pode responder com uma string quando um número é exigido (ou vice-versa). Os hackers, é claro, procuram todo tipo de maneiras nefastas de fornecer dados de entrada inesperados e fornecerão propositalmente dados de entrada que você não estava esperando.

¹ N.T.: A abordagem “da cenoura e da vareta” é uma expressão que se refere a uma política de oferecer recompensas e punições para levar a um comportamento. Seu nome baseia-se na imagem de uma pessoa conduzindo uma carroça com uma cenoura pendurada diante

de uma mula, segurando uma vareta atrás. A mula seguirá a cenoura porque quer a comida como recompensa, ao mesmo tempo que se move para fugir da vareta atrás dela, pois não quer a punição da dor, fazendo a carroça se mover. (Baseado em: https://en.wikipedia.org/wiki/Carrot_and_stick.)

CAPÍTULO 5

Implementando um código confiável

Você pode estar se perguntando por que este livro contém um capítulo sobre como implementar um código confiável quando o assunto é segurança. Há uma inter-relação entre a velocidade, a confiabilidade e a segurança de uma aplicação. Cada um desses elementos tem um papel para transformar uma boa aplicação em uma ótima aplicação. Não é possível dar mais destaque a um aspecto em relação a outro sem que sua aplicação seja diminuída de alguma maneira. É claro que as empresas tomam a decisão conscientes de enfatizar um elemento em relação a outro, mas normalmente para atingir objetivos específicos. Por exemplo, talvez haja um requisito legal para proteger aplicações a qualquer custo, caso em que você provavelmente precisará sacrificar um pouco de velocidade e de confiabilidade para atingir o objetivo. No entanto, para a maioria das aplicações, o equilíbrio entre os três elementos é crucial. Até entender as interações entre velocidade, confiabilidade e segurança, é quase impossível conseguir uma aplicação com o nível máximo possível de segurança implementado, e é impossível criar uma aplicação que funcione bem em um ambiente em que todos os três elementos são valorizados.

Este capítulo mostra a confiabilidade conforme ela se relaciona com a segurança. Em outras palavras, analisa como equilibrar a confiabilidade para atingir objetivos específicos de segurança em sua empresa. O capítulo também considera a confiabilidade como uma ciência estatística, embora não vá deixar você entediado com as especificidades dos cálculos de confiabilidade. A questão é entender o conceito de risco calculado no que diz respeito ao estado da segurança em uma aplicação. Qualquer pessoa que diga que uma aplicação não cria riscos para a integridade tanto dos dados quanto da organização não entende realmente nem de confiabilidade nem de

segurança.

Como parte do tratamento de questões de confiabilidade, o capítulo a explora de vários pontos de vista. Por exemplo, para criar uma aplicação confiável, você deve definir protocolos de equipe que garantam que todos entendam os conceitos necessários. Especialistas em confiabilidade também aprendem a cada iteração dos cálculos que fazem – de modo semelhante, você deve incorporar um ciclo de feedback para fazer alterações de maneira que a empresa veja a confiabilidade em seu ambiente único. Há também as questões ligadas ao uso de soluções prontas. Você não criará muitas aplicações do zero, portanto é importante conhecer os efeitos que o uso de uma solução pronta terá em sua aplicação.



A ideia principal a ser extraída deste capítulo é que a aplicação mais confiável do mundo é aquela que executa tarefas sem nenhum tipo de potencial para falhas. Isso excluiria o uso de qualquer processamento, entradas ou recursos externos, pois todos esses três itens têm pontos de falha. Uma aplicação desse tipo não teria usuários e deveria executar em um hardware com confiabilidade extremamente alta. No entanto, mesmo com todos esses aspectos implementados, ainda seria impossível criar uma aplicação que seja 100% confiável. Toda aplicação tem potencial para falhar, tornando-a menos confiável.

Falta de confiabilidade acaba com negócios

As manchetes de maior destaque na imprensa especializada geralmente são sobre falhas de segurança. Hackers entrando em registros de saúde supostamente seguros ou obtendo acesso a números do Seguro Social de milhares de usuários de cartões de crédito (somente no EUA) tendem a ser notícias de destaque. Parece que vemos muito menos notícias sobre confiabilidade. No entanto, a falta dela pode provocar resultados terríveis – até mesmo falência de negócios.

Considere o caso do Knight Capital Group (<http://dealbook.nytimes.com/2012/08/02/knight-capital-says-trading-mishap-cost-it-440-million>). Uma falha no software usado para interagir com a Bolsa de Valores de Nova York fez a empresa comprar aproximadamente sete bilhões de dólares em ações que ela não queria. Vender rapidamente as ações fez a empresa ter um prejuízo líquido de 440 milhões de dólares. Entretanto, os prejuízos logo aumentaram. À medida que as pessoas perderam a confiança no Knight Capital Group, a empresa começou a sangrar mais ainda no vermelho. Posteriormente, outra empresa, a Getco, comprou o Knight Capital Group por uma ninharia (<http://www.reuters.com/article/2012/12/19/us-knightcapital-getco-idUSBRE8BI0OF20121219>) – tudo por causa de uma falha de

software.

Diferenciando confiabilidade e segurança

Algumas pessoas acham que confiabilidade e segurança são iguais. No entanto, confiabilidade e segurança são duas medidas totalmente diferentes do desempenho da aplicação. Sim, as duas medidas interagem, mas de maneiras que muitas pessoas realmente não entendem muito bem. Uma aplicação confiável não é necessariamente segura e vice-versa. As próximas seções analisam a interação entre confiabilidade e segurança com mais detalhes.



A melhor maneira de trabalhar com os exemplos descritos neste capítulo é usar o código-fonte que pode ser baixado em vez de digitá-lo manualmente. Usar o código-fonte baixado reduz os erros em potencial. Você pode encontrar os códigos-fonte dos exemplos deste capítulo na pasta `\S4WD\Chapter05` no código-fonte disponível para download.

Definindo os papéis da confiabilidade e da segurança

Confiabilidade é uma medida da frequência com que uma aplicação falha. É uma medida estatística. Usamos a confiabilidade para responder à pergunta sobre qual a probabilidade de uma aplicação falhar, dado um certo conjunto de circunstâncias. A confiabilidade também informa quando o ambiente da aplicação está mudando. Por exemplo, quando você acrescenta pessoas à equipe e a carga da aplicação aumenta, a confiabilidade pode diminuir, a menos que você projete a aplicação para escalar bem. Porém, mesmo quando o software é perfeito, o hardware alcançará um ponto limite e a confiabilidade ainda assim diminuirá. Desse modo, a confiabilidade é uma medida do sistema como um todo. O hardware, o usuário, a plataforma, o ambiente de funcionamento, as técnicas de gerenciamento e vários outros aspectos afetam a confiabilidade e mudam o que você vê como defeitos ou falhas da aplicação.

A segurança é uma medida do nível de esforço necessário para quebrar uma aplicação. De modo diferente da confiabilidade, não há nenhum método disponível para medir a segurança estatisticamente. O que você tem é um

potencial para danos que só pode ser quantificado pela determinação do hacker que está tentando causar esses danos. Se os hackers estiverem realmente determinados a invadir sua aplicação para provocar danos, é quase certo que eles encontrarão meios para isso. Ao lidar com segurança, você também deve considerar os efeitos da monitoração e da capacidade de uma equipe reagir rapidamente às brechas.

Em ambos os casos, uma aplicação falha quando a pressão aplicada sobre ela se torna maior que a sua capacidade de resistir. Uma aplicação com falha não consegue cumprir sua principal responsabilidade, que é gerenciar os dados corretamente. Como essa falha ocorre exatamente depende da aplicação e do modo como ela falha. As falhas de confiabilidade tendem a causar danos nos dados – falhas de segurança, por outro lado, tendem a causar brechas nos dados ou resultam no comprometimento da integridade do sistema.

É importante mostrar a diferença entre ser apenas seguro e ser também confiável. O exemplo a seguir, *RangeCheck1.html*, mostra um código que é seguro do lado cliente (você também deve verificar os dados no lado servidor):

```
<!DOCTYPE html>
<html>
<head>
  <title>Performing a Range Check</title>
  <script language="javascript">
    function testValue()
    {
      value = document.getElementById("Data").value;
      if (value == "")
      {
        alert("Please type a number!");
        return;
      }
      if ((value < 0) || (value > 5))
      {
        alert("Value must be between 0 and 5!");
      }
      else
```

```

    {
        alert("Value = " + value);
    }
}
</script>
</head>
<body>
    <h1>Performing a Range Check</h1>
    <input id="Data" type="number" value="0" min=0 max=5 /><br />
    <button id="Test" onclick="testValue()">
        Test
    </button>
</body>
</html>

```

Nesse caso, um usuário normal que use as setas para cima e para baixo na tag `<input>` jamais fornecerá dados fora do intervalo (como mostra a figura 5.1). No entanto, o código JavaScript em `testValue()` também garante que os dados jamais estarão fora do intervalo, mesmo quando forem digitados. Usar um tipo de entrada `number` quer dizer que se alguém digitar um valor como “Hello”, `testValue()` receberá um valor vazio. É possível testar essa condição também e dar uma resposta apropriada ao usuário.

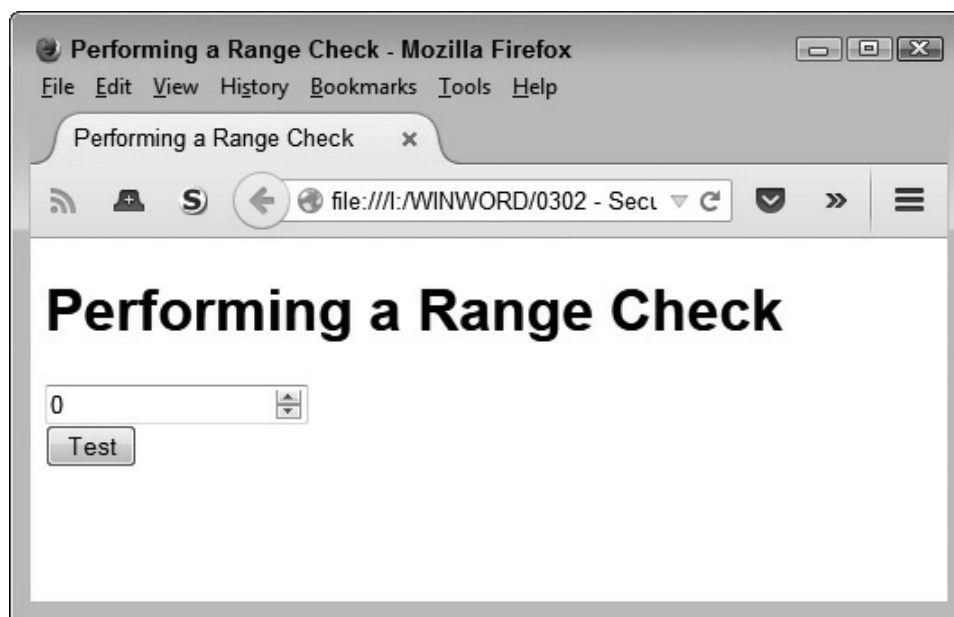


Figura 5.1 – Verificações de intervalo são simplificadas com o uso de

controles apropriados.

O problema com esse código é que ele é seguro, mas não é confiável. Se as condições mudarem, o código não funcionará mais conforme deveria. O exemplo *RangeCheck2.html* a seguir deixa o código mais confiável associando a verificação de intervalo aos atributos min e max da tag <input>:

```
<!DOCTYPE html>
<html>
<head>
  <title>Performing a Range Check</title>
  <script language="javascript">
    function testValue()
    {
      inputObj = document.getElementById("Data");
      value = inputObj.value;
      if (value == "")
      {
        alert("Please type a number!");
        return;
      }
      if ((value < inputObj.getAttribute("min")) ||
          (value > inputObj.getAttribute("max")))
      {
        alert("Value must be between 0 and 5!");
      }
      else
      {
        alert("Value = " + value);
      }
    }
  </script>
</head>
<body>
  <h1>Performing a Range Check</h1>
  <input id="Data" type="number" value="0" min=0 max=5 /><br />
  <button id="Test" onclick="testValue()">
    Test
```

```
</button>  
</body>  
</html>
```

As verificações básicas funcionam como antes. Contudo, se alguém decidir alterar os valores de min e max aceitos pela tag `<input>`, o código automaticamente responde mudando as condições de verificação. Há menos pontos de falha nesse exemplo.

Por outro lado, para ter um código que seja tanto seguro quanto confiável, é preciso pagar um preço em termos de velocidade. Observe a quantidade adicional de linhas de código no segundo exemplo e o maior número de chamadas de função. É provável que você não perceba a diferença de velocidade nesse exemplo em particular, mas quando começar a acrescentar esses tipos de verificação em toda a aplicação, você poderá notar uma séria degradação na velocidade. O código é mais confiável e seguro, mas o usuário talvez não fique satisfeito com o resultado.

Evitando brechas de segurança em códigos confiáveis

Assim como um software seguro não é automaticamente confiável, um software confiável não é automaticamente seguro. Com efeito, você perceberá que quanto mais confiável for o software, maior será o risco de ele não ser realmente seguro. O problema está em um dos objetivos divergentes da confiabilidade e da segurança. Uma aplicação confiável é sempre acessível, utilizável e previsível, o que causa mais do que alguns problemas de segurança. Eis alguns casos em que a confiabilidade tem objetivos divergentes em relação à segurança:

- Uma senha complexa deixa o software mais seguro, mas isso significa que o software poderá não estar disponível quando o usuário esquecer a senha, deixando-o não confiável.
- Verificações que garantam que ninguém adulterou os dados da aplicação podem fazer o software ficar indisponível quando as verificações detectarem uma adulteração.
- Falsos positivos em verificações de segurança deixam o comportamento da aplicação imprevisível.

- Gerenciar configurações de segurança aumenta a complexidade da interface da aplicação, deixando-a menos utilizável.
- Negação de acesso a recursos necessários por causa de restrições de segurança (como aquelas criadas por uma política) torna a aplicação menos acessível, além de deixá-la menos previsível.

De fato, há muitas situações em que a segurança e a confiabilidade entram em conflito. Você deve obter um tipo de equilíbrio entre ambos para garantir que os dados da aplicação permaneçam razoavelmente seguros e a aplicação continue a executar de forma confiável.



Os resultados de brechas de segurança em um código confiável realmente podem ser assustadores. Em uma lista dos 10 hacks extremos da InfoWorld (<http://www.infoworld.com/article/2933868/hacking/10-extreme-hacks-to-be-truly-paranoid-about.html>), aparelhos médicos apareceram em segundo lugar. Esses aparelhos são testados entre 5 e 10 anos para garantir que continuem funcionando, independentemente do que mais possa acontecer. No entanto, ninguém faz patches no software durante o período de testes (e patching implicaria em testes adicionais). Além disso, os aparelhos médicos devem ser fáceis de usar, portanto tudo que sequer lembre uma segurança abrangente é deixado de fora do processo de desenvolvimento. Como resultado, é muito fácil matar uma pessoa fazendo um hacking no aparelho médico de alguém. De todos os exemplos de softwares confiáveis com sérios problemas de segurança, os aparelhos médicos estão no topo da lista. Eles também têm a honra de serem o software com os resultados mais devastadores quando são hackeados.

Usando os exemplos da seção anterior como ponto de partida, é possível ver como uma verificação de intervalo pode interferir na capacidade de o usuário fornecer valores fora do intervalo previsto. É claro que a verificação de intervalo deixa a aplicação mais segura. Porém, é importante considerar o que acontece quando a pessoa que configura a verificação de intervalo da aplicação executa a tarefa incorretamente, e o usuário deixa de ser capaz de inserir um valor perfeitamente válido. A aplicação continuará segura, porém não será mais confiável.

Em algumas situações, um designer pode ver as possíveis ramificações de uma limitação desse tipo como indesejadas e deixar a aplicação mais confiável excluindo a verificação de intervalo. Afinal de contas, se o propósito do software é evitar que um reator nuclear se torne crítico, e o

software impede a entrada de um valor que evitará que o reator se torne crítico, a segurança do software não será mais importante porque não haverá mais ninguém presente para discutir a questão.

Uma correção intermediária para uma situação desse tipo existe, mas ela aumenta a complexidade do software e, assim, afeta mais ainda a confiabilidade. Além do mais, como a correção exige acréscimo de código, a velocidade da aplicação também é afetada. Contudo, ao incluir as várias verificações de segurança durante o funcionamento normal e permitir que um gerente ou um administrador passe por cima delas, desabilitando-as durante uma emergência, o usuário poderá inserir o valor correto para salvar o reator, apesar de o software normalmente não permitir esse valor.

A questão é que, normalmente, você poderá encontrar uma solução alternativa para o problema de “somente segurança” ou “somente confiabilidade”. Em geral, é uma péssima ideia focar apenas em um ou em outro porque os hackers (ou os usuários) farão você pagar o preço em algum momento.

Focando nas funcionalidades da aplicação

Deixar uma aplicação ao mesmo tempo segura e confiável faz um desenvolvedor ficar extremamente satisfeito, mas o usuário não se importa com isso. Os usuários sempre concentram sua atenção em ter uma tarefa que lhes interesse cumprida. A tarefa do usuário pode envolver a geração de um relatório. (Na verdade, o relatório pode não ser o foco – o foco pode envolver conseguir dinheiro de investidores. O relatório simplesmente ajuda o usuário a alcançar esse objetivo.) Se digitar o relatório manualmente for mais fácil e mais transparente que usar sua aplicação, o usuário digitará o relatório manualmente. Os usuários não se importam com a ferramenta usada para realizar uma tarefa, motivo pelo qual você vê usuários tentando gerar saídas complexas usando um smartphone. Como desenvolvedor, você pode se orgulhar secretamente das incríveis estratégias contidas em seu código, mas o usuário não dá a mínima para isso. A questão é que um usuário não virá até você para dizer que a aplicação não é confiável. A resposta do usuário sempre envolverá sua tarefa – qualquer que seja ela. Cabe a você determinar se o

problema que está fazendo o usuário falhar se deve ao fato de a aplicação que você criou não ser confiável quanto a algum aspecto importante.

O equilíbrio entre confiabilidade e segurança torna-se mais evidente quando focamos na funcionalidade da aplicação. Não é simplesmente uma questão de executar a tarefa, mas executá-la do modo que o usuário imaginava originalmente. No mundo atual, isso significa aspectos como:

- contar teclas – quanto menos melhor;
- permitir que a aplicação execute em qualquer plataforma;
- garantir que a aplicação esteja sempre disponível;
- reduzir o número de passos não relacionados à tarefa a zero;
- deixar óbvias as respostas às perguntas ou evitar totalmente as perguntas.

Desenvolvendo protocolos de equipe

Qualquer esforço em direção à criação de uma aplicação confiável deve considerar a equipe toda. Uma equipe de desenvolvimento precisa fazer o design e implementar a aplicação com a confiabilidade em mente desde o início. A equipe também precisa manter a questão do equilíbrio em mente durante esse processo. Não importa se uma aplicação é confiável e segura se ninguém a usa porque ela é lenta.

Os protocolos de equipe podem envolver todo tipo de questões. Por exemplo, em 1999, O Mars Climate Orbiter incendiou-se ao entrar na atmosfera de Marte porque um grupo que trabalhava no software usava unidades métricas enquanto outro grupo usava unidades inglesas ou imperiais (<http://www.wired.com/2010/11/1110mars-climate-observer-report/>). O problema foi a falta de comunicação – ela parte do protocolo que deve ser criado para o desenvolvimento bem-sucedido de uma aplicação.

Para criar protocolos apropriados para sua empresa, é preciso dividir as tarefas de várias maneiras. A equipe que cria uma aplicação deve considerar as questões de confiabilidade a partir de três níveis diferentes:

Erros acidentais de design ou de implementação

Quando a maioria das pessoas pensa em problemas de confiabilidade, elas

pensam em falhas que fazem a aplicação funcionar de maneira diferente do modo como a equipe de desenvolvimento originalmente a projetou para funcionar. A aplicação falha em executar tarefas corretamente. No entanto, esses erros também poderiam ser do tipo que abrem a aplicação ao acesso de hackers ou simplesmente não oferecem a flexibilidade necessária. As equipes de desenvolvimento superam esse problema por meio de treinamento de desenvolvedores e práticas seguras de desenvolvimento bem como o emprego de ferramentas criadas para localizar esses tipos de problemas de confiabilidade.

Mudança de tecnologia

Uma aplicação torna-se obsoleta no dia em que você termina de trabalhar nela. Com efeito, às vezes, a aplicação está obsoleta antes de você concluí-la. Mudanças de tecnologia agem contra o software para deixá-lo não confiável. Há dois níveis de mudança que devem ser considerados:

À prova de futuro

Para criar um ambiente em que uma aplicação possa manter sua qualidade técnica, você deve ser à prova de futuro. A melhor maneira de alcançar esse objetivo é criar a aplicação como componentes que interagem, mas que também sejam entidades separadas, para permitir a atualização de um componente sem necessariamente atualizar o sistema todo. Esse é o motivo pelo qual os microsserviços têm se tornado tão populares, mas você pode colocar em prática as estratégias de codificação em módulos usando designs monolíticos também.

Melhoria dos hackers

Hackers inovam e aperfeiçoam seus trabalhos. Um problema de segurança ou de confiabilidade que não afetava seu projeto no início pode se tornar bastante problemático antes de você concluir a aplicação. Talvez você nem saiba que o problema existe até um hacker apontá-lo. A melhor maneira de lidar com esse problema é garantir que você mantenha suas ferramentas e técnicas atualizadas para lutar contra as melhorias dos hackers.

Intenção maliciosa

Há uma boa chance de alguém em sua equipe de desenvolvimento não estar satisfeito ou ter conseguido um emprego em sua organização com o objetivo de encontrar maneiras de explorar falhas de software. Esse membro da equipe pode até mesmo introduzir falhas ou backdoors (porta dos fundos) com o intuito de explorá-los depois que sair da empresa. É claro que você não deseja criar uma atmosfera de Grande Irmão¹, pois fazer isso reprime a inovação e tende a criar mais problemas que soluções; entretanto, você também deve garantir que qualquer pessoa da equipe da gerência realmente administre a equipe de desenvolvimento.

A comunicação entre os membros da equipe é essencial, mas garantir que não haja falha de comunicação é mais importante ainda. Suposições geram todo tipo de problemas, e os seres humanos são especialmente bons em preencher lacunas em informações com suposições. É claro que as suposições de um membro da equipe podem não ser as mesmas de outro membro da mesma equipe. Desenvolver o tipo de comunicação de que as equipes precisam inclui as seguintes melhores práticas (você pode encontrar outras boas práticas, ferramentas e recursos no site Cyber Security and Information Systems Information Analysis Center em <https://sw.csiac.org/databases/url/key/2>):

Treinamento em confiabilidade e segurança

Membros da equipe não podem se comunicar, a menos que falem a mesma língua e entendam as preocupações com a confiabilidade no mesmo nível. A única forma de alcançar esse objetivo é garantir que os membros da equipe recebam treinamento adequado. Ao criar um programa de treinamento, garanta que ele seja consistente, independentemente de usar instrutores externos ou internos.

Requisitos de confiabilidade

Criar uma aplicação sem antes definir o que você quer é um pouco semelhante a construir uma casa sem ter um projeto. Assim como você não começaria a fazer as fundações de sua casa sem ter um projeto à mão, você não pode iniciar nenhum esforço de programação sem antes definir os papéis que a confiabilidade e a segurança terão no funcionamento geral da

aplicação. Certifique-se de que qualquer definição criada inclua métricas e objetivos específicos para cada fase do desenvolvimento. A definição também deve especificar como a análise tanto da segurança quanto da confiabilidade será usada, as auditorias no código e os testes.

Design confiável

Assim como você inicia qualquer projeto identificando as ameaças de segurança com que uma aplicação vai se deparar, você também deve identificar os problemas de confiabilidade e especificar os métodos para superá-los. O processo de design deve incluir especificidades sobre como lidar com problemas de confiabilidade. Embora muitas empresas estejam cientes da necessidade de especialistas em segurança para ajudar a solucionar problemas de segurança, poucas estão cientes de que há um serviço semelhante oferecido por especialistas em confiabilidade, como a Reliability Consulting Services (<http://www.reliasoft.com/consulting/>).

Código confiável

O processo de implementação deve ser feito tanto com a segurança quanto com a confiabilidade em mente. Quão preparado você está não importa, a menos que coloque o que aprendeu e projetou na prática.

Tratamento seguro do código-fonte

Para lidar com ameaças como intenções maliciosas, sua empresa deve dar um tratamento seguro ao código-fonte. Isso quer dizer que apenas as pessoas com treinamento e credenciais apropriados podem ver o código-fonte. Além do mais, isso também significa que você deve fazer revisões tanto de design quanto de código, para garantir que a aplicação permaneça fiel aos objetivos originais do design.

Testes de confiabilidade

É essencial testar o código para garantir que ele realmente atenda aos objetivos de confiabilidade que são parte dos requisitos e do design da aplicação. Testes de confiabilidade podem incluir todo tipo de problemas, como a forma como a aplicação responde quando houver carga ou quando ela perde acesso a um recurso necessário. A tarefa é testar os pontos de

falha da aplicação e conferir se a aplicação trata cada um deles com sucesso.



Testes cegos (blind testing, em que você pede a alguém externo ao grupo, sem experiência com a aplicação, que tente usá-la) pode produzir algumas informações interessantes e úteis sobre possíveis brechas de segurança da aplicação. O teste muitas vezes mostra os tipos de combinação de teclas que os hackers experimentarão usar para tentar provocar falhas na aplicação. Esses testes também podem apontar áreas em que os recursos da aplicação não são autoexplicativos e exigem mais trabalho, documentação ou treinamento adicionais.

Documentação relacionada à confiabilidade e segurança

É importante documentar os requisitos, o design, as técnicas de programação e de testes e outros processos que você usou para garantir que a aplicação seja confiável e segura. A documentação deve expressar os aspectos relacionados ao equilíbrio entre esses fatores e explicar como você os tratou como parte dos requisitos e do design da aplicação.

Prontidão para questões ligadas à confiabilidade e segurança

Imediatamente antes do lançamento da aplicação, a equipe de desenvolvimento deve garantir que nenhuma nova ameaça tenha surgido no cenário, que a aplicação deva tratar para funcionar com sucesso. Confiabilidade e segurança lidam com riscos, e essa fase determina o risco que essas novas ameaças representam. Tratar ameaças de baixo risco como parte de uma atualização, em vez de adiar o lançamento da aplicação, também pode ser uma boa solução.

Resposta a questões ligadas à confiabilidade e segurança

Após o lançamento da aplicação, a equipe de desenvolvimento deve responder a quaisquer novas ameaças à confiabilidade e à segurança prontamente. É importante entender que fontes fora da equipe de desenvolvimento podem informar esses problemas e esperam que a equipe de desenvolvimento forneça uma resposta rápida.

Verificação de integridade

Garantir que a aplicação continue a funcionar como deveria quer dizer

deixar o código seguro usando algum tipo de técnica de assinatura (para garantir que ninguém o modifique). Além disso, a equipe de desenvolvimento deve continuar a testar o código em relação a novas ameaças e verificar se a aplicação gerencia os dados de forma segura. A ideia é continuar procurando problemas em potencial, mesmo que você esteja certo de que não haja nenhum. Hackers esperam que sua equipe se descuide na identificação de novas ameaças até que seja tarde demais.

Pesquisa em segurança e confiabilidade

É importante que alguns indivíduos tenham como tarefa encontrar novas ameaças e, à medida que elas surjam, descobrir métodos para lidar com elas. Testar a aplicação não é um problema, mas saber como testá-la quanto às ameaças mais recentes é a única maneira de garantir que seu teste está realmente fazendo o que deveria.

Criando um ciclo de feedback de lições aprendidas

A confiabilidade é baseada em análise estatística de eventos no decorrer do tempo. Quanto mais tempo passar e quanto mais eventos forem registrados, mais exata será a previsão. O tempo médio entre eventos de falha é o MTBF (Mean Time Between Failures, ou Tempo Médio entre Falhas). A maioria dos textos sobre software parece não abordar o assunto desse ponto de vista, mas softwares, como tudo o mais, têm padrões de falha, e é possível analisar esses padrões para criar uma imagem de quando você pode esperar que elas ocorram. É claro que, como qualquer estatística, não é possível determinar momentos exatos – somente o curso geral da atividade para uma dada aplicação em um ambiente específico.

Algumas pessoas veem o MTBF como uma medida incorreta da confiabilidade do software porque acham que ela indica literalmente o próximo instante em que um bug de software, um problema de ambiente ou outro fator farão o software falhar. O aspecto importante a ser lembrado é que o MTBF é baseado em um ambiente específico. Como o software raramente funciona da mesma forma no mesmo ambiente de uma empresa para outra, tentar criar um MTBF para uma aplicação que seja apropriado em qualquer empresa não funcionará. Um MTBF de uma aplicação para uma empresa específica funciona porque é improvável que o ambiente dessa empresa mude significativamente. Quando isso ocorrer, o



valor do MTBF não será mais válido.

Quanto maior o MTBF de uma aplicação, menos tempo será gasto em seu suporte e menores serão os custos de manutenção. Além disso, um MTBF alto também indica uma situação em que as brechas de segurança por falhas na aplicação são menos prováveis. As falhas na aplicação não ocorrem só por causa de bugs no software – elas também ocorrem por falhas no uso, problemas de ambiente (por exemplo, falta de memória), causas que não podem ser reproduzidas (por exemplo, raios cósmicos causando picos na alimentação) e outras causas. Como não é possível administrar todas essas causas de falhas, o software jamais poderá ser 100% confiável. Em algum momento, até mesmo o melhor software do mundo terá uma falha.



Raios cósmicos realmente afetam os computadores. De fato, quanto maior a altitude do local de armazenamento do computador, maiores serão os efeitos. O tamanho dos transistores nos chips também afeta a incidência de erros de software. Você pode descobrir mais sobre esse efeito interessante em “Cosmic Rays and Computers” (Raios cósmicos e computadores, <http://www.nature.com/news/1998/980730/full/news980730-7.html>), “Effect of Cosmic Rays on Computer Memories” (Efeitos dos raios cósmicos nas memórias do computador, <http://www.ncbi.nlm.nih.gov/pubmed/17820742>) e “Should Every Computer Chip Have a Cosmic Ray Detector?” (Todo chip de computador deveria ter um detector de raios cósmicos?, <https://www.newscientist.com/blog/technology/2008/03/do-we-need-cosmic-ray-alerts-for.html>). As informações são interessantes e podem, finalmente, explicar alguns daqueles erros impossíveis de reproduzir, que você viu no passado.

Um ciclo de feedback para analisar a causa das falhas pode ajudar a aumentar o MTBF em determinada empresa. Ao analisar as causas das falhas, você pode criar um plano para melhorar o MTBF pelo menor custo possível. Eis algumas ideias a serem consideradas como parte da análise das causas de

falhas no software:

Qualidade

À medida que a qualidade do software melhora, o MTBF aumenta. É possível melhorar a qualidade do software encontrando e removendo bugs, melhorando a interface de usuário para diminuir as chances de um usuário cometer erros e acrescentando verificações para garantir que os recursos necessários estejam disponíveis antes de usá-los.

Pontos de falha

Reduzir o número de pontos de falha em uma aplicação aumenta o MTBF. A melhor maneira de reduzir os pontos de falha é deixar a aplicação menos complexa, organizando as rotinas e deixando-as mais eficientes. No entanto, você também pode tomar atitudes como aumentar a redundância, quando for possível. Por exemplo, ter duas fontes para os mesmos dados diminui a probabilidade de que a perda de uma única fonte cause falhas na aplicação.

Treinamento

Um treinamento melhor reduz erros operacionais e melhora o MTBF. Há um ponto em que o retorno por causa do treinamento diminui, mas a maioria dos usuários atualmente não recebe treinamento suficiente para o software que deveriam usar para realizar tarefas úteis. Além disso, treinar o pessoal de suporte para lidar com erros de modo mais preciso e os desenvolvedores a identificar as verdadeiras causas das falhas ajudarão a melhorar o processo de feedback.

Criar listas completas de falhas, juntamente com suas causas, é a melhor maneira de começar a entender a dinâmica de sua aplicação. À medida que a base de conhecimento de uma aplicação aumenta, é possível ver padrões previsíveis e usá-los como meio para determinar os pontos em que devemos investir tempo e recursos fazendo correções. É claro que não é possível corrigir as causas de alguns erros. Sim, é possível proteger todo o seu hardware para se livrar daqueles raios cósmicos irritantes, mas as chances de uma empresa gastar dinheiro com isso são extremamente baixas (e o retorno provavelmente será menor ainda).

Considerando os problemas de soluções prontas

Conforme os capítulos anteriores, a maioria das aplicações atualmente depende de soluções prontas para executar tarefas comuns. Tentar criar uma aplicação totalmente do zero consumiria tempo demais e seria inviável do ponto de vista financeiro. Não há realmente um bom motivo para reinventar a roda. No entanto, usar essas soluções prontas afetará a confiabilidade de sua aplicação. Você estará contando com um código escrito por outra pessoa para fazer sua aplicação funcionar, portanto esse código também deve fazer parte do cálculo da confiabilidade. As seções a seguir discutem algumas questões que você deve considerar quando trabalhar com soluções prontas.

Lidando com bibliotecas externas

Bibliotecas externas criam vários problemas interessantes de confiabilidade. A questão mais importante é que a aplicação geralmente fará download de uma cópia da biblioteca sempre que começar a executar. Se você mantiver sua aplicação executando, esse processo não ocorrerá com frequência, mas a maioria das aplicações baseadas em web executa no sistema do cliente, o que significa que o cliente precisará fazer download da biblioteca sempre que iniciar a aplicação. A velocidade torna-se um problema por causa do download da biblioteca. No entanto, a aplicação pode nem mesmo iniciar se algo impedir que o download do cliente tenha sucesso. Por exemplo, considere o uso da jQuery UI para criar um efeito de acordeão (accordion), como mostra a figura 5.2.

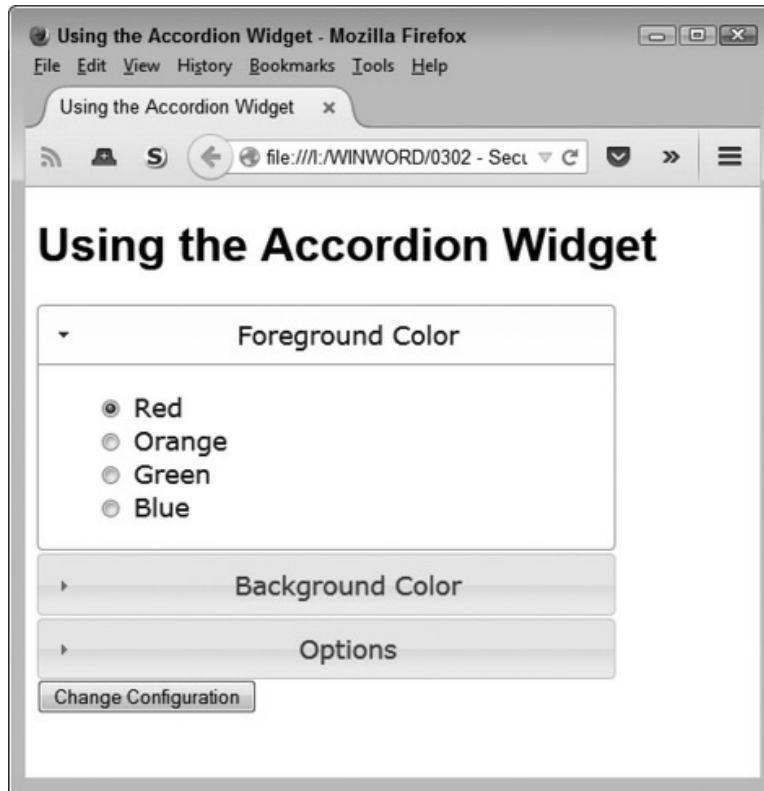


Figura 5.2 – Até mesmo um exemplo simples com jQuery UI exige download de código.

Esse exemplo nem mesmo iniciará caso os arquivos da biblioteca da qual ele depende tornarem-se inacessíveis por algum motivo. O código a seguir (que está no arquivo *Accordian.html*) aparece no cabeçalho para criar a conectividade necessária:

```
<head>
  <script
    src="https://code.jquery.com/jquery-latest.js">
  </script>
  <script
    src="https://code.jquery.com/ui/1.9.2/jquery-ui.js">
  </script>
  <link
    rel="stylesheet"
    href="https://code.jquery.com/ui/1.9.2/themes/base/jquery-ui.css" />
```

Se algum dos três arquivos mostrados nesse exemplo não estiver presente, a aplicação não executará. Esse exemplo também mostra duas abordagens

diferentes para selecionar as bibliotecas. Observe que a biblioteca jQuery usa a versão mais recente, o que significa que você terá automaticamente as correções de bugs e outras atualizações. No entanto, a biblioteca jQuery UI (e seu arquivo CSS associado) usa uma versão específica da biblioteca. Você não obterá as atualizações nesse caso, mas poderá ter correções de bugs que ainda poderão provocar falhas em sua aplicação, caso você já tenha incluído código para contornar o problema.

Uma das melhores maneiras de aumentar a confiabilidade no uso de bibliotecas externas é fazer download da biblioteca em drives locais e utilizar essa cópia, em vez de usar a cópia do site do fornecedor. Essa estratégia garante que você sempre terá a mesma versão da biblioteca e que haverá menos chances de a biblioteca não estar disponível. É claro que você precisará disponibilizar espaço no disco rígido local para armazenar a biblioteca, mas o maior uso de recursos locais é um preço baixo a ser pago pela estabilidade resultante.

Como tudo o mais, há um preço a pagar pelo uso de uma cópia local da biblioteca. O mais importante desses problemas é a falta de atualizações. A maioria dos fornecedores disponibiliza atualizações com relativa frequência, incluindo correções de bugs, melhorias na confiabilidade e na velocidade. É claro que a biblioteca normalmente conterá também novos recursos. É preciso encontrar um equilíbrio entre os novos recursos e os recursos antigos de que você pode precisar para fazer sua aplicação executar. Para ter os ganhos de confiabilidade, você deverá abrir mão de algumas melhorias de segurança em potencial e outras atualizações que você realmente queira usar em sua aplicação.

Uma abordagem intermediária é fazer download da biblioteca localmente, monitorar as melhorias nas atualizações e programar as atualizações em sua aplicação para que coincidam com as atualizações necessárias de segurança e de recursos na maioria das bibliotecas que você usa. Isso significaria realizar atualizações de acordo com o seu cronograma, e não conforme o cronograma do fornecedor. Embora essa alternativa possa parecer a solução perfeita, isso não é verdade. Hackers muitas vezes contam com exploits “dia zero” (zero-day)² para causar o máximo de danos no maior número de aplicações. Como

sua aplicação não receberá as atualizações necessárias automaticamente, você ainda estará diante da possibilidade de um ataque “dia zero” devastador.

Lidando com APIs externas

APIs externas oferecem os meios para acessar dados e outros recursos usando uma fonte externa. Uma API normalmente é um conjunto de classes que você instancia como objetos e usa para fazer chamadas. O código não executa no sistema do cliente. Em vez disso, ele executa como um RPC (Remote Procedure Call, ou Chamada de Procedimento Remoto) OOP (Out-Of-Process) no servidor. Um ciclo de requisição/resposta entre cliente/servidor ocorre com o cliente fazendo requisições ao servidor. Criar um link para a API é muito semelhante a criar um link para uma biblioteca, exceto que, normalmente, você deverá fornecer algum tipo de chave para ter acesso, como mostrado a seguir para o arquivo *GoogleAPI.html* (consulte o capítulo 7 para ver detalhes sobre este exemplo):

```
<head>
  <script type="text/javascript"
    src="https://maps.googleapis.com/maps/api/js?key=Your Key Here&sensor=false">
```

A consulta exige que o cliente monte uma requisição e a envie ao servidor. Ao trabalhar com a API do Google, você deve fornecer dados como a longitude e a latitude da interface, juntamente com o tipo preferido de mapa. A API, na verdade, escreve os dados diretamente no local em que a aplicação especificou como parte da requisição, como mostrado a seguir:

```
// Esta função exibe o mapa na tela
function GetMap()
{
  // Cria uma lista de argumentos para enviar ao Google
  var MapOptions =
  {
    center: new google.maps.LatLng(
      Latitude.spinner("value"),
      Longitude.spinner("value")),
    zoom: 8,
    mapTypeId: google.maps.MapTypeId.ROADMAP
```

```

}
// Especifica o local para colocar o mapa e
// as opções do mapa para o Google
var map = new google.maps.Map(
    document.getElementById("MapCanvas"), MapOptions);
};

```

O resultado da chamada é um mapa exibido no canvas do cliente. A figura 5.3 mostra um exemplo típico da saída dessa aplicação.

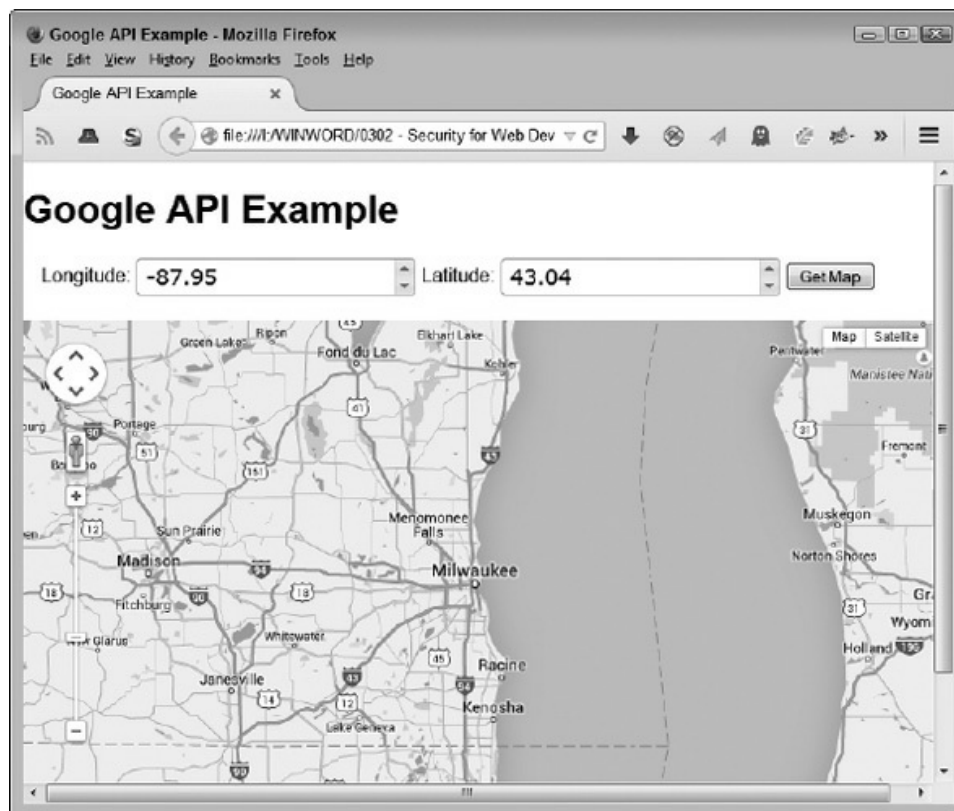


Figura 5.3 – A API do Google desenha diretamente no canvas do cliente.

Utilizar APIs externas torna possível usar o código de outra pessoa com menos preocupações com segurança e, potencialmente, menos problemas de velocidade (conforme seja mais rápido solicitar os dados ou fazer download do código). Contudo, do ponto de vista da confiabilidade, APIs externas apresentam todo tipo de riscos. Qualquer interrupção na conexão significa que a aplicação vai parar de funcionar. É claro que é relativamente fácil para um desenvolvedor criar um código que trate esse problema e apresente uma mensagem ao usuário. Corrigir o problema de conexões lentas, dados

corrompidos e outros problemas que possam refletir a condição da conexão ou o trabalho dos hackers é mais difícil. Nesse caso, a frustração do usuário torna-se um problema porque ele poderia contribuir com o caos, fornecendo dados de entrada aleatórios para receber algum tipo de resposta da aplicação.

Uma maneira de contornar o problema da confiabilidade, nesse caso, é medir o tempo das chamadas. Quando a aplicação perceber que as chamadas estão demorando mais que o normal, é possível responder informando o usuário sobre a conexão lenta, entrando em contato com o administrador ou reagindo ao problema de alguma outra forma. A questão é garantir que você monitore quanto tempo cada chamada demora e agir de forma apropriada.

Trabalhando com frameworks

Frameworks representam uma solução intermediária entre bibliotecas e APIs quanto à confiabilidade. Na maioria dos casos, frameworks são códigos compactados. Um dos frameworks mais comumente usados é o MooTools (<http://mootools.net/>). No entanto, há vários outros frameworks disponíveis por aí e você deve encontrar o framework adequado às suas necessidades. Na maioria das vezes, os frameworks podem executar no servidor, no cliente ou em uma combinação de ambos. Conforme o modo de usar o framework, você poderá ter os mesmos problemas de confiabilidade encontrados em bibliotecas, APIs ou em uma combinação de ambos.

Diferenciando frameworks de bibliotecas

Embora um framework como o Dojo (<https://dojotoolkit.org/>) e uma biblioteca como a jQuery se pareçam bastante – e você os utiliza da mesma forma, na maioria das vezes – o Dojo é um framework e a jQuery é uma biblioteca. Do ponto de vista de segurança, confiabilidade e velocidade, as duas entidades são diferentes.

Uma biblioteca é um código puro que é baixado e executado como parte de sua aplicação. Você chama as funções diretamente, e o código-fonte delas às vezes está disponível para que você possa alterar seu comportamento.

Os frameworks oferecem um meio para interagir com um comportamento, o que significa que algumas tarefas não são visíveis ao desenvolvedor – ele pede que o framework execute a tarefa e o framework define como executá-la. O código do fornecedor continua sendo baixado e também se torna parte de sua aplicação, mas a tecnologia subjacente é diferente. Algumas pessoas definem um framework como uma

forma empacotada de biblioteca que oferece uma estrutura, além do código.

A parte interessante sobre os produtos como o MooTools é que você pode executar muitas das mesmas tarefas executadas pelas bibliotecas. Por exemplo, você pode criar uma versão do exemplo com acordeão usando o MooTools. O código a seguir está bem simplificado, mas serve para ilustrar a questão:

```
<!DOCTYPE html>
<html>
<head>
  <title>MooTools Accordion Demo</title>
  <script src="MooTools-More-1.5.1.js"></script>
  <style>
    .toggler
    {
      color: #222;
      margin: 0;
      padding: 2px 5px;
      background: #eee;
      border-bottom: 1px solid #ddd;
      border-right: 1px solid #ddd;
      border-top: 1px solid #f5f5f5;
      border-left: 1px solid #f5f5f5;
    }
  </style>
  <script type="text/javascript">
    window.addEvent('domready', function()
    {
      var accordion = new Fx.Accordion('h3.atStart', 'div.atStart',
        {
          opacity: false,
          onActive: function(toggler, element)
          {
            toggler.setStyle('color', '#ff3300');
          },
        },
      );
    });
  </script>
</html>
```

```

        onBackground: function(toggler, element)
        {
            toggler.setStyle('color', '#222');
        }
    }, $('accordion'));
});
</script>
<body>
    <h2>MooTools Accordion Demo</h2>
    <div id="accordion">
        <h3 class="toggler atStart">Section 1</h3>
        <div class="element atStart">
            <p>Section 1 Content</p>
        </div>
        <h3 class="toggler atStart">Section 2</h3>
        <div class="element atStart">
            <p>Section 2 Content</p>
        </div>
        <h3 class="toggler atStart">Section 3</h3>
        <div class="element atStart">
            <p>Section 3 Content</p>
        </div>
    </div>
</body>
</html>

```

Você obtém o arquivo *MooTools-More-1.5.1.js* do site Builder em <http://mootools.net/more/builder>. Não se esqueça de incluir as bibliotecas essenciais. O Google oferece um site para hospedagem em <https://developers.google.com/speed/libraries/>, mas não inclui os recursos adicionais como o Fx.Accordion.

O framework exige que você configure uma série de cabeçalhos e divisões com o conteúdo para o acordeão, como mostra a parte em HTML do exemplo. A aparência desses itens, quando estiverem selecionados ou não, depende do CSS que você configurar. O script define dois eventos: `onActive`

(quando o item é selecionado) e `onBackground` (quando o item deixa de ser selecionado). Nesse caso em particular, o MooTools comporta-se de modo muito semelhante a uma biblioteca, e você pode ver a saída na figura 5.4.

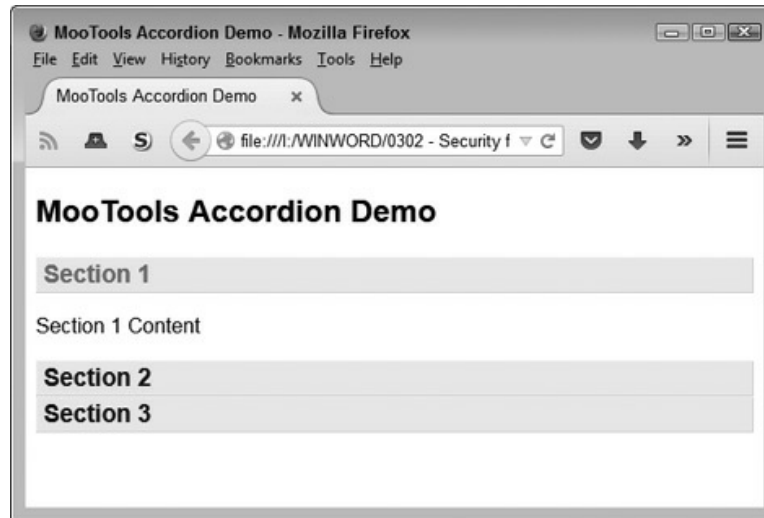


Figura 5.4 – MooTools é um framework que oferece funcionalidades tanto de biblioteca quanto de API.

Acionando os microsserviços

Em vários aspectos, um microsserviço é simplesmente uma API mais granular. O conceito de usar porções menores de código para que seja possível contar com quaisquer recursos necessários para executar bem uma tarefa específica faz sentido. Por causa do modo como os microsserviços funcionam, muito do que você sabe sobre APIs do ponto de vista da confiabilidade também se aplica aos microsserviços. Por exemplo, chamadas a um microsserviço em particular podem demorar mais que o esperado, aumentando a frustração do usuário. A forma de contornar esse problema é implementar timeouts na aplicação. Contudo, no caso de um microsserviço, você sempre pode tentar fazer a chamada usando uma fonte secundária quando estiver disponível.

No entanto, os microsserviços também diferem das APIs quanto a alguns aspectos simplesmente porque são menores e mais personalizados. Com isso em mente, apresentamos alguns dos problemas de confiabilidade que você deve considerar quando estiver lidando com microsserviços:

- Usar mais microsserviços de procedências diferentes aumenta o potencial de uma perda de conexão e reduz a confiabilidade da aplicação.
- Escolher microsserviços que realmente otimizem o serviço oferecido para a sua necessidade específica aumentará a confiabilidade da aplicação porque haverá menos riscos de erros significativos.
- Contar com uma interface consistente para chamadas a microsserviços reduz possíveis erros de programação, aumentando a confiabilidade da aplicação.
- Ter mais de um local de procedência que possa executar o mesmo serviço reduz o número de pontos de falha únicos, aumentando a confiabilidade da aplicação.
- Manter os serviços pequenos significa que a perda de um único serviço não implicará a indisponibilidade de todos os serviços, como ocorreria com uma API, portanto a confiabilidade é maior nesse caso também.

¹ N.T.: “‘Grande Irmão’, tradução literal de ‘Big Brother’ no original, que na verdade significa ‘Irmão Mais Velho’, em linguagem coloquial inglesa, é um personagem fictício no romance 1984 de George Orwell. Na sociedade descrita por Orwell, todas as pessoas estão sob constante vigilância das autoridades, principalmente por teletelas (telescreen), sendo constantemente lembradas pela frase propaganda do Estado: ‘o Grande Irmão zela por ti’ ou ‘o Grande Irmão está te observando’ (do original ‘Big Brother is watching you’).” (Fonte: https://pt.wikipedia.org/wiki/Grande_Irmão)

² N.T.: Um exploit “dia zero” (zero-day) é a exploração de uma vulnerabilidade ainda não documentada, para a qual ainda não existe uma correção.

CAPÍTULO 6

Incorporando bibliotecas

Seria difícil encontrar uma aplicação web que não dependa de uma biblioteca – pelo menos, não de uma que não seja significativa. Bibliotecas são o epítome da não reinvenção da roda. De fato, assim como as rodas, há bibliotecas de todo formato, tamanho e cor por aí. Sim, até as cores entram em ação quando consideramos que muitas bibliotecas incluem temas que você pode usar para deixar seu site mais elegante. Consequentemente, é provável que você já tenha gasto um bom tempo trabalhando com bibliotecas de alguma maneira. De fato, vários dos exemplos do livro em capítulos anteriores contavam com bibliotecas simplesmente porque reescrever o código do zero não faz sentido.

Este capítulo não tenta persuadir você sobre a importância de usar bibliotecas. É provável que você já tenha se convencido de suas vantagens. No entanto, talvez você não tenha considerado todas as ramificações de usar o código de outra pessoa em sua aplicação. Sempre há consequências para o uso de bibliotecas. O objetivo é usar bibliotecas de forma segura para que você tenha todas as vantagens de poder contar com o código de outra pessoa sem criar grandes brechas de segurança em sua aplicação. (Tenha ciência de que você criará algumas brechas de segurança; é impossível evitá-las.)

Obtendo velocidade máxima com bibliotecas

Você pode encontrar sites que dizem para você não usar nenhuma biblioteca porque elas incorrem em penalidades na velocidade (por exemplo, veja <http://www.giftspeed.com/dont-use-javascript-libraries/>). O fato é que bibliotecas podem deixar seu site lento, especialmente quando elas não são usadas de forma inteligente. O impacto na velocidade pode ser grande o suficiente para fazer as pessoas darem o clique que as levarão para o próximo site, o que significa que seu site deixou de oferecer um serviço ou outro recurso às pessoas que o acessaram.

Este livro geralmente usa as versões não compactadas das bibliotecas disponíveis para facilitar o trabalho com os exemplos e executar tarefas, como usar um depurador, para analisar as aplicações mais facilmente. Contudo, do ponto de vista da velocidade, normalmente você deve usar a versão compactada da biblioteca. A versão compactada contém o mesmo código, mas carrega mais rápido e, às vezes, você ganha um pouco de agilidade também ao fazer chamadas.

Também é importante perceber que você não usará todo o código da biblioteca. Muitas bibliotecas, como a jQuery UI (<http://jqueryui.com/download/>), oferecem um builder que permite criar uma versão personalizada da biblioteca carregada. Depois de descobrir quais partes de uma biblioteca você realmente usará, utilize um builder para criar uma versão que tenha somente essas partes. A biblioteca carregará mais rápido e você também reduzirá sua superfície de ataque, o que representa uma melhoria na segurança também.

Considerando os usos de bibliotecas

O modo de usar uma biblioteca afeta diretamente sua segurança – pelo menos, até certo ponto. Por exemplo, o envolvimento direto da biblioteca com seu código JavaScript tende a abrir mais brechas de segurança em potencial que usar a biblioteca para formatar a página. É claro que qualquer uso de uma biblioteca poderia criar potenciais lacunas de segurança que custarão tempo, esforço e recursos à sua empresa. As seções a seguir discutem como podemos usar bibliotecas para executar várias tarefas e analisam como a forma de uso poderia criar certos tipos de brechas de segurança.



A melhor maneira de trabalhar com os exemplos descritos neste capítulo é usar o código-fonte que pode ser baixado, em vez de digitá-lo. Usar o código-fonte baixado reduz os erros em potencial. Você pode encontrar os códigos-fonte dos exemplos deste capítulo na pasta `\S4WD\Chapter06` no código-fonte disponível para download.

Melhorando o CSS com bibliotecas

CSS tem tudo a ver com formatar conteúdo de páginas. Dado o estado atual do CSS, é impressionante ver os tipos de sofisticação que podem ser adicionados a um site. Desenvolvedores web usam CSS para fazer uma

página maçante ganhar vida. É claro que há usos comuns para o CSS também. Por exemplo, suponha que você precise ler um texto com várias colunas sem contar com CSS para formatar o conteúdo. Você acabará com uma confusão. Usar apenas tags HTML para formatar o conteúdo fará com que seja praticamente impossível usar as páginas na grande variedade de dispositivos da qual as pessoas dependem hoje em dia.

O CSS sozinho provavelmente poderia fazer tudo que é necessário para criar páginas fantásticas. No entanto, é importante perceber que até mesmo o melhor desenvolvedor web não tem tempo para criar todo o código CSS manualmente. Há diversas maneiras de empacotar código CSS em uma biblioteca, mas uma das mais interessantes é a função `css()` oferecida pela jQuery UI, conforme encontrada no arquivo *Transform.html*:

```
<!DOCTYPE html>
<html>
<head>
  <script
    src="https://code.jquery.com/jquery-latest.js">
  </script>
  <script
    src="https://code.jquery.com/ui/1.9.2/jquery-ui.js">
  </script>
  <script
    src="jquery.transform.js">
  </script>
  <link
    rel="stylesheet"
    href="https://code.jquery.com/ui/1.9.2/themes/base/jquery-ui.css" />
  <title>jquery.transform.js Demonstration</title>
  <style type="text/css">
    #TransformMe
    {
      border: double;
      border-color: Blue;
      border-width: thick;
      position: absolute;
```

```

    width: 120px;
    height: 40px;
    top: 90px;
    left: 75px;
}
#TransformMe p
{
    margin: 0;
    padding: 0;
    height: 40px;
    font-family: "Comic Sans MS", cursive, sans-serif;
    font-size: large;
    color: Red;
    background-color: Yellow;
}
#Rotate
{
    position: absolute;
    top: 190px;
    left: 75px;
}
</style>
<script type="text/javascript">
    $(function()
    {
        $("#Rotate").click(function()
        {
            $("#TransformMe").css("transform", "rotate(45deg)");
        });
    })
</script>
</head>
<body>
    <h1>jquery.transform.js Demonstration</h1>
    <div id="TransformMe">
        <p>Some Text</p>

```



```

</div>
<div>
  <input type="button"
    id="Rotate"
    value=" Rotate 45 Degrees " />
</div>
</body>
</html>

```

A funcionalidade encontrada nesse exemplo depende do script *jquery.transform.js*. No entanto, antes de passar para o scripting, observe que a página conta com CSS padrão para posicionar o conteúdo. Na maioria dos casos, as aplicações que você criar contarão com uma combinação de fontes tanto locais quanto de biblioteca para formatação. O conteúdo inclui um parágrafo formatado como um rótulo (label) e um botão.

Clicar no botão dispara o evento click que executa uma única linha de código. O CSS para TransformMe é modificado de modo a incluir um transform de 45deg. O texto deixa de ser horizontal e passa a ter um ângulo como mostra a figura 6.1.



Figura 6.1 – O uso de uma transformação pode modificar a aparência do texto na tela.

O problema em permitir que um script afete a formatação da página é que

you cannot be sure how the page will look after the script terminates. The page could contain hidden elements or simply not present the information as expected. Hidden elements could contain code to redirect the user to a infected site or perform other actions that you would not want associated with your application.

Interagindo com HTML usando bibliotecas

Some library functions interact directly with the HTML of your application in two common ways. The first consists in interacting with an existing element. In this case, the content appears inside the element according to the library function called. The second option is to create new elements and associate them with the existing document. The new elements can have any type of content, including tags that could potentially cause problems for the end user.

An important concern when using a library to interact directly with the HTML of your site is that the library could fill various elements with confusing, incorrect or previously contaminated data. For example, instead of creating the expected link, it could redirect a user to another site – maybe one that downloads a virus or other malicious software on the user's computer. Libraries that work with HTML can cause subtle problems that you may not even notice immediately because everything will appear to work correctly. At least until a user complains about incorrect redirection (and considering how URLs currently work, they probably won't), you won't know about the problem until several machines have been infected, causing serious problems.

Adding new elements to a page is a common practice. An example that can be found in *ViewJSON.html* adds new elements to the existing document, based on the content of a JSON (JavaScript Object Notation, or JavaScript Object Notation) file called *Test.json*. The code below shows how the example performs this task:

```
<!DOCTYPE html>  
<html>
```

```
<head>
  <title>Viewing JSON Files</title>
  <script
    src="https://code.jquery.com/jquery-latest.js">
  </script>
  <script language="JavaScript">
    function ViewData()
    {
      // Obtém os dados de disco
      $.getJSON("Test.json",
        function(data)
        {
          // Cria um array para armazenar os dados
          var items = [];
          // Faz parse dos dados observando
          // cada entrada do objeto Users
          $.each(data.Users,
            function(key, value)
            {
              items.push("<li>" +
                value.Name + "<br />" +
                value.Number + "<br />" +
                (new Date(
                  parseInt(value.Birthday.substr(6)))
                  ).toDateString()
                + "</li>");
            });
          // Coloca o resultado em uma lista não ordenada
          $('<ul/>', {html: items.join("")}).
            appendTo('body');
        });
    }
  </script>
</head>

<body>
  <h1>Viewing JSON Files</h1>
  <input id="btnView"
```

```
type="button"
value="View JSON Data"
onclick="ViewData()" />

</body>
</html>
```

Nesse caso, a biblioteca abre o arquivo *Test.json* e lê seus dados. O código remove a formatação do arquivo JSON e acrescenta formatação de tags HTML. Então, ele usa a função `join()` para acrescentar os novos elementos à página. A figura 6.2 mostra a saída desse exemplo.

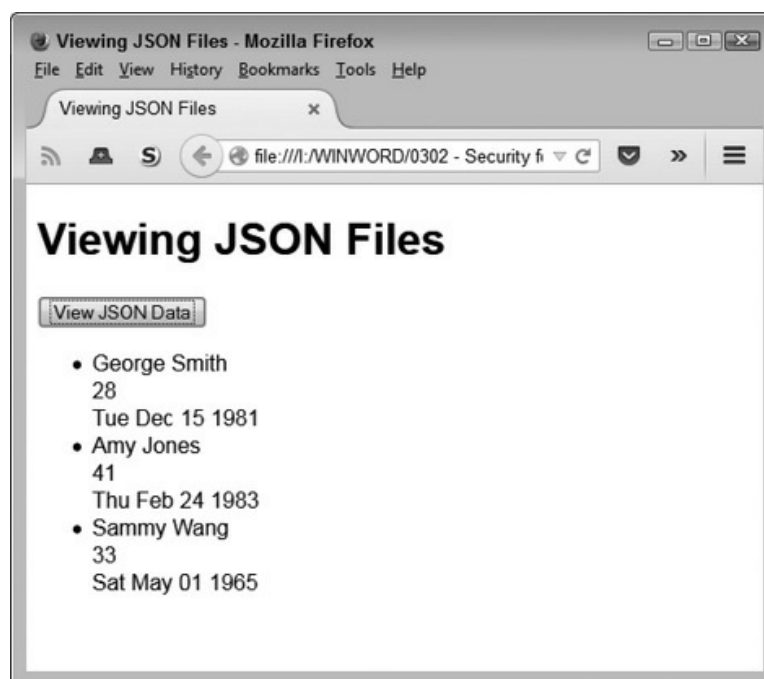


Figura 6.2 – Injetar dados de uma fonte externa é sempre problemático.

Observe que o exemplo evita o uso da chamada `document.write()`, que poderia expor os usuários a todo tipo de conteúdo indesejado. O uso da função `join()` oferece uma alternativa mais segura, que deverá manter a página livre de problemas de segurança.

Estendendo o JavaScript com bibliotecas

A maioria das pessoas pensa nas maneiras como as bibliotecas estendem o CSS ou o HTML. A apresentação das informações está na linha de frente. Entretanto, as bibliotecas podem afetar o scripting subjacente também. Por

exemplo, a maioria das pessoas está familiarizada com o uso de `alert()` para exibir mensagens simples. A maior parte das bibliotecas contém alternativas que um hacker poderia usar para exibir outros tipos de informações na tela ou afetar a aplicação de outras maneiras. Por exemplo, o arquivo *DialogBox.html* contém um exemplo de criação de uma caixa de diálogo personalizada como esta mostrada a seguir:

```
<!DOCTYPE html>
<html>
<head>
  <title>Creating a Simple Dialog Box</title>
  <script
    src="https://code.jquery.com/jquery-latest.js">
  </script>
  <script
    src="https://code.jquery.com/ui/1.9.2/jquery-ui.js">
  </script>
  <link
    rel="stylesheet"
    href="https://code.jquery.com/ui/1.9.2/themes/base/jquery-ui.css" />
  <style type="text/css">
.Normal
{
  font-family: Arial, Helvetica, sans-serif;
  color: SaddleBrown;
  background-color: Bisque;
}
.Emphasize
{
  color: Maroon;
  font-style: italic;
  font-size: larger;
}
  </style>
</head>
<body>
  <h1>Creating a Simple Dialog Box</h1>
```

```
<div id="DialogContent"
  title="Simple Dialog Example"
  hidden>
  <p class="Normal">
    This is some
    <span class="Emphasize">interesting</span>
    text for the dialog box!
  </p>
</div>
<script type="text/javascript">
  $("#DialogContent").dialog();
</script>
</body>
</html>
```

Nesse caso, o exemplo exhibe a caixa de diálogo imediatamente, quando você abre a aplicação, como mostra a figura 6.3. A ameaça representada por essa funcionalidade está no fato de um usuário poder abrir sua aplicação esperando uma experiência e vivenciar algo totalmente diferente. Como a caixa de diálogo aparece como parte da aplicação, o usuário fornecerá todo tipo de informação solicitada pela caixa de diálogo ou ela fará o usuário fazer qualquer coisa.



Figura 6.3 – A funcionalidade de caixa de diálogo da jQuery UI poderia ser usada para criar uma brecha de segurança.

Um hacker poderia habilitar essa funcionalidade usando algo como injeção de script (código). Por exemplo, o hacker poderia esconder o código na forma de um comentário em uma página de blog (motivo pelo qual a moderação manual de comentários é uma boa ideia). O ataque subjacente é um ataque de engenharia social – deixar o usuário à vontade apresentando-lhe a caixa de diálogo como parte da aplicação e então usar essa situação para fazer algo sórdido.

Diferenciando bibliotecas armazenadas internamente de bibliotecas armazenadas externamente

Talvez você ache que todas as bibliotecas JavaScript estejam em um site de terceiros. É verdade que muitas bibliotecas JavaScript populares, como a jQuery, realmente estão no site de outra pessoa. Contudo, muitas empresas têm suas próprias bibliotecas também. Os fatores que afetam as bibliotecas armazenadas internamente são:

- Elas estão em sistemas de propriedade da empresa e são mantidas por ela.

- A empresa controla o código-fonte e pode modificá-lo conforme for necessário para resolver problemas encontrados pelos desenvolvedores ou que tenham sido apontados por outros.
- Entidades externas não podem acessar a biblioteca sem permissão da empresa.
- A conectividade com a biblioteca depende de linhas internas, e não do acesso pela internet.

Bibliotecas armazenadas internamente têm muitas vantagens não encontradas em bibliotecas armazenadas externamente em um site de terceiros. Em sua maior parte, dado o mesmo nível de programação, uma biblioteca armazenada internamente é mais rápida, mais confiável e mais segura que uma biblioteca armazenada externamente. (Por exemplo, um hacker pode facilmente fazer download de uma cópia de uma biblioteca armazenada em um site externo, mas é menos provável que faça isso com uma biblioteca armazenada internamente.) Além do mais, como a biblioteca contém apenas o código necessário a uma empresa, é bem possível que a biblioteca armazenada internamente use menos recursos porque será menor que sua contrapartida genérica. É claro que essas afirmações fazem várias pressuposições sobre a biblioteca armazenada internamente, que podem não ser verdadeiras. Eis alguns contrapontos a serem considerados:

- Bibliotecas armazenadas internamente são caras para criar e manter, portanto a empresa pode não as manter atualizadas conforme necessário para deixá-las seguras.
- Poucas empresas podem manter uma equipe de desenvolvimento equivalente em habilidades a um fornecedor terceirizado, portanto a qualidade da biblioteca provavelmente sofrerá.
- Bibliotecas de terceiros recebem testes de um número significativo de testadores, de modo que até mesmo os menores erros são identificados. Bibliotecas armazenadas internamente em geral são pouco testadas, a ponto de nem mesmo as falhas importantes serem eliminadas.

A questão principal é que bibliotecas armazenadas internamente têm sucesso quando visam a funcionalidades especiais não oferecidas pelas contrapartidas

de terceiros e a equipe que as desenvolve toma o cuidado necessário para realizar bem a tarefa. Ao fazer o design de sua aplicação, você deve pesar as vantagens que as bibliotecas armazenadas internamente oferecem em relação aos riscos que elas representam. Além disso, você deve definir melhor os riscos apresentados pelas bibliotecas de terceiros.

Definindo as ameaças de segurança causadas por bibliotecas

Qualquer código JavaScript que você criar provavelmente conterá algum erro que um hacker pode explorar para diversos propósitos. As bibliotecas não são especiais quanto a esse aspecto. Entretanto, como você estará incorporando um código não criado pela sua empresa em uma aplicação criada por ela, há uma chance de o código que, normalmente, funcionaria de forma correta acabe tendo uma falha de segurança por causa de suposições feitas por ambos os lados. Com isso em mente, as duas ferramentas que todo desenvolvedor precisa conhecer quando se trata de bibliotecas são as ferramentas de teste especificamente projetadas para as necessidades relativas à segurança e um bom especialista em segurança.



Nem sempre é possível dizer que a biblioteca que você mantém é totalmente segura. Você pode dizer que escreveu o código de modo a atender às melhores práticas de segurança e que um especialista em segurança revisou o código, mas, mesmo com as melhores intenções e ferramentas robustas, simplesmente não é possível garantir que uma biblioteca (ou qualquer outro código, se for o caso) seja segura. Esse problema surge com relativa frequência quando conversamos com pessoas que não são programadores – em especial com a gerência de uma empresa que queira usar sua biblioteca para desenvolvimento. É importante que todos entendam que você adotou todas as medidas de segurança apropriadas, mas que ninguém pode garantir que um código qualquer seja totalmente seguro.

Mesmo após os testes, você deve estar ciente de que há um potencial para falhas de segurança em uma biblioteca, na aplicação à qual ela oferece suporte ou como resultado da combinação de ambas. Com isso em mente, eis uma lista dos tipos mais comuns de ameaças que você poderá encontrar ao trabalhar com bibliotecas (sem ênfase na origem propriamente dita da falha):

Cross-site scripting (XSS)

O problema mais comum de segurança com que os desenvolvedores se deparam é o XSS. Há três maneiras simples de driblar o XSS:

- Nunca transmita dados não confiáveis na mesma resposta HTTP como HTML ou JavaScript. De fato, é melhor se o documento HTML principal permanecer estático.



Talvez você esteja se perguntando como seu servidor poderia acabar enviando dados não confiáveis a alguém. É mais fácil do que você imagina. O artigo “ScriptGard: Automatic Context-Sensitive Sanitization for Large-Scale Legacy Web Applications” (ScriptGard: sanitização automática sensível ao contexto para aplicações web legadas de larga escala, <http://www.cs.berkeley.edu/~prateeks/papers/scriptgard-ccs11.pdf>) descreve como é difícil garantir a integridade dos dados, mesmo com o uso de sanitizadores corretos. A suposição segura é que qualquer dado que você não tenha criado pessoalmente não é confiável.

- Quando tiver de transmitir dados não confiáveis do servidor para o cliente, garanta que sejam codificados em JSON e que tenham um Content-Type igual a application/json.
- Depois que os dados estiverem prontos para exibição, utilize as chamadas a `Node.textContent()`, `document.createTextNode()` ou `Element.setAttribute()` (apenas o segundo parâmetro) para garantir que a página os apresente apropriadamente.

Chamadas de função perigosas

Só porque JavaScript aceita uma chamada em particular, isso não significa que a chamada seja segura. Usar `setInnerHTML()` ou `.innerHTML =` pode injetar um código de script indesejado. Conte com a chamada a `setInnerText()` em seu lugar.

Modificar o documento diretamente

Embora você possa ver `document.write()` aparecer algumas vezes no livro por questões de conveniência, usar essa chamada em ambiente de produção é um convite ao desastre. O código poderia escrever qualquer dado em qualquer lugar. Em seu lugar, use chamadas que adicionam, removem ou atualizam elementos do DOM (Document Object Model, ou Modelo de Objetos de Documentos).

Criando scripts durante a execução

Sempre que transformar uma string em um script, você estará convidando um hacker a fornecer a string. Usar chamadas como `eval()`, `setTimeout()` ou `setInterval()` com um argumento do tipo string ou `new Function()` deixa seu código significativamente menos seguro.

Código que executa, mas provoca falhas de segurança

Algumas chamadas de JavaScript oferecem todo tipo de corda para você se enforcar. Para evitar essa situação, use o modo estrito de JavaScript para garantir que apenas chamadas seguras funcionarão.

Conteúdo que diz ser algo que não é

Hackers amam mandar conteúdos que não são exatamente o que você acha que sejam. A melhor maneira de evitar esse problema é seguir uma Política de Segurança de Conteúdo (Content Security Policy), que significa incluir as tags apropriadas de conteúdo, como `script-src 'self'` e `object-src 'self'`.



Fazer scanning de uma biblioteca usando um produto como a JSLint (<http://www.jshint.com/>) pode ajudar a garantir a qualidade de seu código. É menos provável que um código de alta qualidade contenha erros que causem problemas de segurança. No entanto, é importante perceber que a JSLint (e ferramentas como ela) não verificam especificamente problemas de segurança. De fato, não é possível verificar problemas de segurança em seu código. Para conferir se há problemas de segurança, você deve começar testando seu código especificamente para verificar se eles existem. Se o código exige um nível maior de confiança do que os testes oferecem, você deverá empregar também os serviços de um especialista em segurança para conferir manualmente o código em busca de potenciais problemas.

Habilitando o modo estrito

Navegadores mais recentes oferecem suporte para ECMAScript 5, que inclui o modo estrito (strict mode) de JavaScript. Para usar esse recurso de segurança, você deve garantir que seus usuários não estejam com aquele navegador da época dos dinossauros que está no computador desde a primeira vez que eles o usaram. Por exemplo, o IE 9 não tem suporte para esse recurso, mas o IE 10 tem. Você pode descobrir quais navegadores têm suporte para o modo estrito em <http://kangax.github.io/compattable/es5/>. É

essencial que você desenvolva uma política organizacional que exija que os usuários tenham determinados navegadores para trabalhar com a aplicação.

O modo estrito faz muitas mudanças óbvias e sutis no modo como o JavaScript funciona. O aspecto importante é que ele facilita depurar aplicações JavaScript porque, em vez de falhar silenciosamente ou agir do modo estranho, sua aplicação JavaScript agora vai gerar erros quando encontrar um problema. Isso quer dizer que aquelas chamadas de biblioteca estranhas que costumavam morrer sem fazer nada agora informarão algo sobre os problemas. O modo estrito ajudará você a lidar com os seguintes problemas principais:

Elimina o uso de with

O uso da instrução `with` pode criar abertura para problemas de segurança em sua aplicação. As versões mais recentes de JavaScript tornaram esse recurso obsoleto por ser muito perigoso. O modo estrito gerará um erro sempre que você tentar usar `with` em seu código.

Evita variáveis indesejadas

Um problema importante em JavaScript é que você pode criar novas variáveis acidentalmente ao atribuir um valor a uma variável com um nome digitado erroneamente. Em alguns casos, esse erro pode criar variáveis globais indesejadas, resultando em brechas de segurança. O modo estrito força você a declarar todas as variáveis antes de usá-las, portanto não é mais possível criar variáveis acidentalmente.

Desabilita coerção usando this

Alguns códigos existentes fazem coerção de variáveis locais para o estado global atribuindo um valor a elas quando estão em um estado sem atribuição ou nulo. Por exemplo, você não pode atribuir um valor a uma variável em um construtor sem antes instanciar um objeto chamando `new`.

Evita duplicatas

É bem fácil criar duplicatas de propriedades em objetos ou argumentos nomeados em funções. O modo estrito lança um erro se você tentar criar uma duplicata em qualquer uma das situações.

Notifica tentativas de alterar valores imutáveis

Não é possível alterar um valor imutável em JavaScript. No entanto, no passado, a tentativa falhava silenciosamente, portanto era possível supor que o código estava em um estado quando, na verdade, estava em outro. O modo estrito lança um erro após qualquer tentativa de alterar um valor imutável.

O modo estrito está no ECMAScript 5 e em versões mais recentes. Você não precisa instalar nada especial para ter suporte a esse modo. No entanto, você deve incluir a string "use strict";. Versões mais antigas de JavaScript ignoram essa string, portanto você não verá nenhuma mudança no modo como códigos mais antigos executam. O arquivo *StrictMode.html* contém o seguinte exemplo:

```
<!DOCTYPE html>
<html>
<head>
  <title>Working with Strict Mode</title>
  <script language="javascript">
    function testStrict(Name)
    {
      "use strict";

      try
      {
        this.Name = Name;
        alert(this.Name);
      }
      catch (e)
      {
        alert(e);
      }
    }
  </script>
</head>
<body>
  <h1>Working with Strict Mode</h1>
  <button id="Test" onclick="testStrict('George')">
```

```
Test
</button>
</body>
</html>
```

Sem a verificação do modo estrito (você pode simplesmente comentar a instrução para testar), o código exibe o valor erroneamente atribuído a `this.Name`, que é George. No entanto, com a verificação de modo estrito habilitada, você verá a mensagem de erro mostrada na figura 6.4.

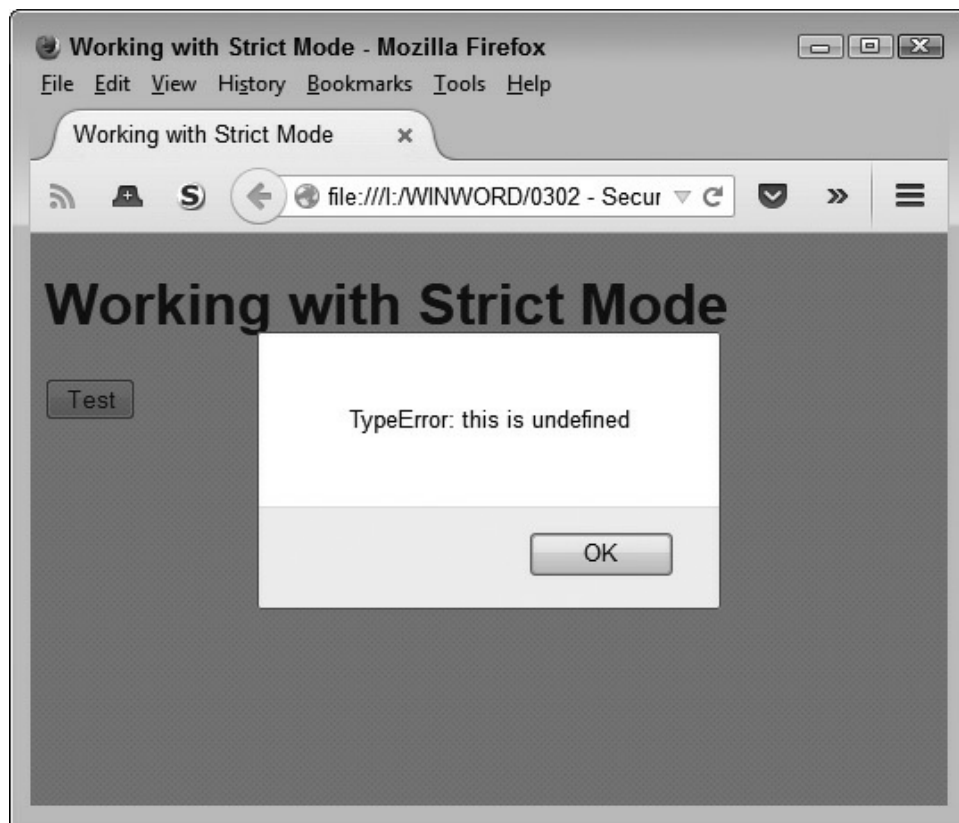


Figura 6.4 – A verificação do modo estrito evita atribuições errôneas.



Nunca use o modo estrito globalmente. Pode parecer uma boa ideia à primeira vista, mas você descobrirá rapidamente que alguns códigos de biblioteca falharão. A ideia por trás do modo estrito é garantir que você tenha um controle rigoroso sobre o código que criar. Ao usar verificações de validação, você pode monitorar o comportamento das bibliotecas que usar.

Desenvolvendo uma CSP (Content Security Policy)

Um dos principais problemas enfrentados pelos desenvolvedores é criar

aplicações que não sejam suscetíveis a vários tipos de ataques de injeção de script e de conteúdo, por exemplo, XSS. O problema resiste a uma solução simples por causa da maneira como os navegadores funcionam. Os navegadores efetivamente confiam em qualquer conteúdo proveniente de uma origem específica; sendo assim, o navegador confia em qualquer dado injetado no mesmo nível que o dado original. A CSP procura remediar esse problema criando uma espécie de recurso de lista branca.

Para navegadores com suporte a CSP (veja a tabela em <http://caniuse.com/contentsecuritypolicy> para ver uma lista de navegadores compatíveis), a CSP oferece os meios para evitar que os navegadores reconheçam scripts e conteúdos de sites não aceitos. A política aparece na forma de uma série de cabeçalhos que você adiciona no início da página. Quando um navegador vê os cabeçalhos, ele os usa para determinar quais scripts e conteúdos são seguros para carregar.



Quando um navegador não oferece suporte para CSP, ele ignora os cabeçalhos que você fornece e age como faria normalmente. Isso quer dizer que os usuários devem contar com navegadores que tenham suporte para CSP se você quiser usá-la como uma forma de bloquear conteúdos indesejados. Caso contrário, o navegador funcionará como sempre, e continuará a carregar conteúdo de sites não aceitos.

Um cabeçalho é constituído de três elementos básicos: nome da política, tipo e fonte de dados. A seguir, apresentamos um exemplo de um cabeçalho CSP que contém apenas um tipo de dado:

```
Content-Security-Policy: script-src 'self'
```

Nesse caso, a CSP determina que o navegador deve executar apenas os scripts que se encontram embutidos na página. Vamos supor que você queira oferecer suporte para jQuery também. Nesse caso, você estenderia a política como mostrado a seguir:

```
Content-Security-Policy: script-src 'self' 'https://code.jquery.com'
```

Você não precisa fornecer a localização específica do arquivo de código-fonte, mas apenas informações do site de host. Cada entrada é separada da próxima com um espaço. Uma CSP pode definir vários tipos de conteúdo. Por exemplo, talvez você decida que quer dar suporte a scripts encontrados

na página local, mas não queira dar suporte a objetos. Nesse caso, utilize o cabeçalho de CSP a seguir:

```
Content-Security-Policy: script-src 'self'; object-src 'none'
```

Observe que os tipos de conteúdo aparecem separados por um ponto e vírgula (;). A CSP aceita várias diretivas diferentes de tipos de conteúdo. Você pode encontrar um guia de referência rápida contendo uma lista dessas diretivas, valores de exemplo e descrições em <http://contentsecurity-policy.com/>. O aspecto importante a ser lembrado é que usar CSP pode fazer você evitar aquela brecha sutil em seu código, que um hacker, de outro modo, poderia explorar.

Incorporando bibliotecas de forma segura

Algumas empresas simplesmente adicionam uma biblioteca à sua caixa de ferramentas porque ela oferece algumas funcionalidades de que a empresa precisa, sem pensar muito sobre como essa biblioteca é composta ou nem se é uma boa ideia usá-la. Em alguns casos, as empresas que não pensam muito no uso de uma biblioteca acabam se arrependendo disso depois porque a biblioteca conterá muitas brechas de segurança, executará de forma não confiável ou lenta ou, na verdade, acabará custando mais tempo à empresa do que custaria se ela tivesse simplesmente criado sua própria biblioteca. As próximas seções ajudarão você a determinar se incorporar em sua aplicação aquela biblioteca interessante que você encontrou é realmente uma boa ideia.

Adotando a solução de sandbox

É bem provável que, mesmo que você verifique o JavaScript de terceiros e limpe completamente os dados que usar, ainda haverá algo tentando se esgueirar. Lembre-se de que é mais fácil passar pelas paredes que você constrói que construí-las. Quando se trata de ameaças de segurança a JavaScript, você deve considerar que os hackers simplesmente aparecerão com novas maneiras de passar pela sua segurança sempre que você tiver uma nova estratégia à prova de balas em seu arsenal.

Uma maneira em potencial de contornar esse problema é contar com sandboxing. Essencialmente, isso quer dizer que a biblioteca de terceiros executa com um nível mais baixo de privilégio que o seu código. Usar sandboxing não garante que sua aplicação vá permanecer segura, mas reduz o potencial para falhas de segurança. Você

pode ler sobre técnicas de sandboxing em <http://www.slideshare.net/phungphu/a-two-tier-sandbox-architecture-for-untrusted-javascript>. As técnicas descritas no slideshow são complexas, mas funcionam. Como tudo o mais, elas se reduzem a determinar o nível de risco que você está disposto a tolerar quando executar uma aplicação. O capítulo 10 discute o uso de sandboxes em detalhes.

Fazendo uma pesquisa completa sobre a biblioteca

Sempre que optar por acrescentar uma biblioteca à sua aplicação, você estará expressando confiança no criador dessa biblioteca. Mesmo com a ajuda de um competente especialista em segurança, tomar uma biblioteca e analisar cada pedacinho de código que ela contém (supondo que o código esteja totalmente disponível) mostrará que isso consome mais tempo que criar a biblioteca do zero. (Certifique-se de que você não adquiriu a versão minimizada da biblioteca, que é difícil de analisar porque o fornecedor remove os caracteres em branco.) Consequentemente, você deve considerar se a biblioteca apresenta poucos riscos a ponto de fazer parte de uma solução de aplicação criada por você. Quando estiver pesquisando sobre uma biblioteca, considere as perguntas a seguir como parte de sua pesquisa:

- Há alguma história na mídia especializada que discuta a biblioteca de forma negativa? (Se a biblioteca já é conhecida por ter brechas de segurança não corrigidas, talvez você queira reconsiderar o uso dela em sua aplicação.)
- Há outras empresas usando a biblioteca sem que haja problemas?
- A empresa que criou a biblioteca é proativa em responder a perguntas sobre a integridade da biblioteca?
- A empresa que criou a biblioteca oferece correções consistentes de bugs e atualizações?
- Que tipo de perguntas as pessoas estão fazendo como parte do suporte online oferecido para a biblioteca?
- Há quanto tempo a biblioteca está presente no mercado? (Bibliotecas com uma vida longa tendem a ter menos problemas e a longevidade está relacionada a um bom suporte por parte do criador.)

Definindo exatamente quais são os usos da biblioteca

Ninguém usa todos os recursos oferecidos por todas as bibliotecas encontradas em uma aplicação. Alguns sites de pesquisa afirmam claramente que muitos desenvolvedores usam uma biblioteca devido a apenas uma ou duas funcionalidades que ela contém. Documentar o modo como você usa uma biblioteca em particular ajuda a atenuar riscos associados à utilização da biblioteca ao focar sua atenção nessas áreas. Mais importante ainda, manter um controle das áreas específicas de uso diz quando você deve executar uma atualização e como é possível reduzir o tamanho de uma biblioteca removendo seus recursos não usados. Quanto mais você souber como usar uma biblioteca para executar tarefas específicas da aplicação, mais você reduzirá os riscos ao usá-la.

Mantendo o tamanho da biblioteca reduzido e o conteúdo focado

É essencial manter bibliotecas de terceiros reduzidas e focadas em necessidades específicas sempre que for possível. Isso quer dizer remover códigos que você jamais usará porque esses códigos tendem a:

- deixar o download mais lento;
- aumentar os problemas de segurança;
- reduzir a confiabilidade;
- provocar atualizações desnecessárias.

Felizmente, as melhores bibliotecas de terceiros estão do seu lado. Elas querem que você crie uma versão personalizada da biblioteca para usar em seu site. Por exemplo, quando usar a jQuery UI, você pode criar uma versão personalizada com o builder especial mostrado na figura 6.5.



Figura 6.5 – Usar o builder da jQuery UI ajuda a criar uma versão personalizada da biblioteca.

Usar um builder quer dizer que a biblioteca que você criar conterá apenas os elementos da biblioteca de que você realmente precisará. É claro que a desvantagem é que você deve fornecer hospedagem para a biblioteca, o que significa que você não terá atualizações automáticas à medida que o criador as disponibilizar. Se usar uma opção como essa, é essencial verificar a necessidade de atualizar sua cópia da biblioteca regularmente.

Executando os testes necessários

Antes de colocar qualquer biblioteca em um ambiente de produção, você deve criar uma suíte de testes para determinar se a biblioteca funciona conforme esperado. Você pode começar usando uma suíte de testes específica do fornecedor, mas é importante também testar usando uma suíte de testes projetada por você, que simule como sua aplicação usará a biblioteca. O processo de testes deve incluir verificações de dados de entrada válidos e inválidos. É importante garantir que você obtenha a saída esperada com entradas válidas e algum tipo de exceção quando usar entradas inválidas. Os capítulos 11 e 12 apresentam detalhes sobre como executar vários níveis de testes em sua aplicação e nas bibliotecas que ela usa.

Diferenciando bibliotecas de frameworks

A caixa de texto “Diferenciando frameworks de bibliotecas” na página 152 do capítulo 5 apresenta uma visão geral de como as bibliotecas e os frameworks se diferenciam do ponto de vista de confiabilidade. É claro que há mais aspectos nessa diferença que simplesmente preocupações com confiabilidade. É importante entender melhor como os dois diferem exatamente e por que você deve se importar com isso.

Frameworks oferecem templating para um site de um modo diferente das bibliotecas. Ao usar uma biblioteca, uma aplicação está simplesmente contando com outra fonte de código. O uso de frameworks implica outro nível de participação, em que a aplicação agora usa fontes externas para suporte adicional. Pelo fato de um framework se integrar de modo mais completo a uma aplicação e oferecer mais recursos, você também deve considerá-lo como tendo um padrão mais alto em relação a trabalhar com uma biblioteca (embora você possa considerar ambos como de alto padrão, de qualquer maneira).

Tentar descobrir se um framework é seguro pode ser bem difícil. De fato, testar pode ser quase impossível. Com isso em mente, especialistas em segurança de terceiros estão tentando quantificar o nível de segurança de frameworks e de bibliotecas de templating. Um dos melhores lugares para procurar informações é o [mustache-security](#)

(<https://code.google.com/p/mustache-security/>). Esse site apresenta sete níveis de verificação para frameworks, como mostra a figura 6.6.

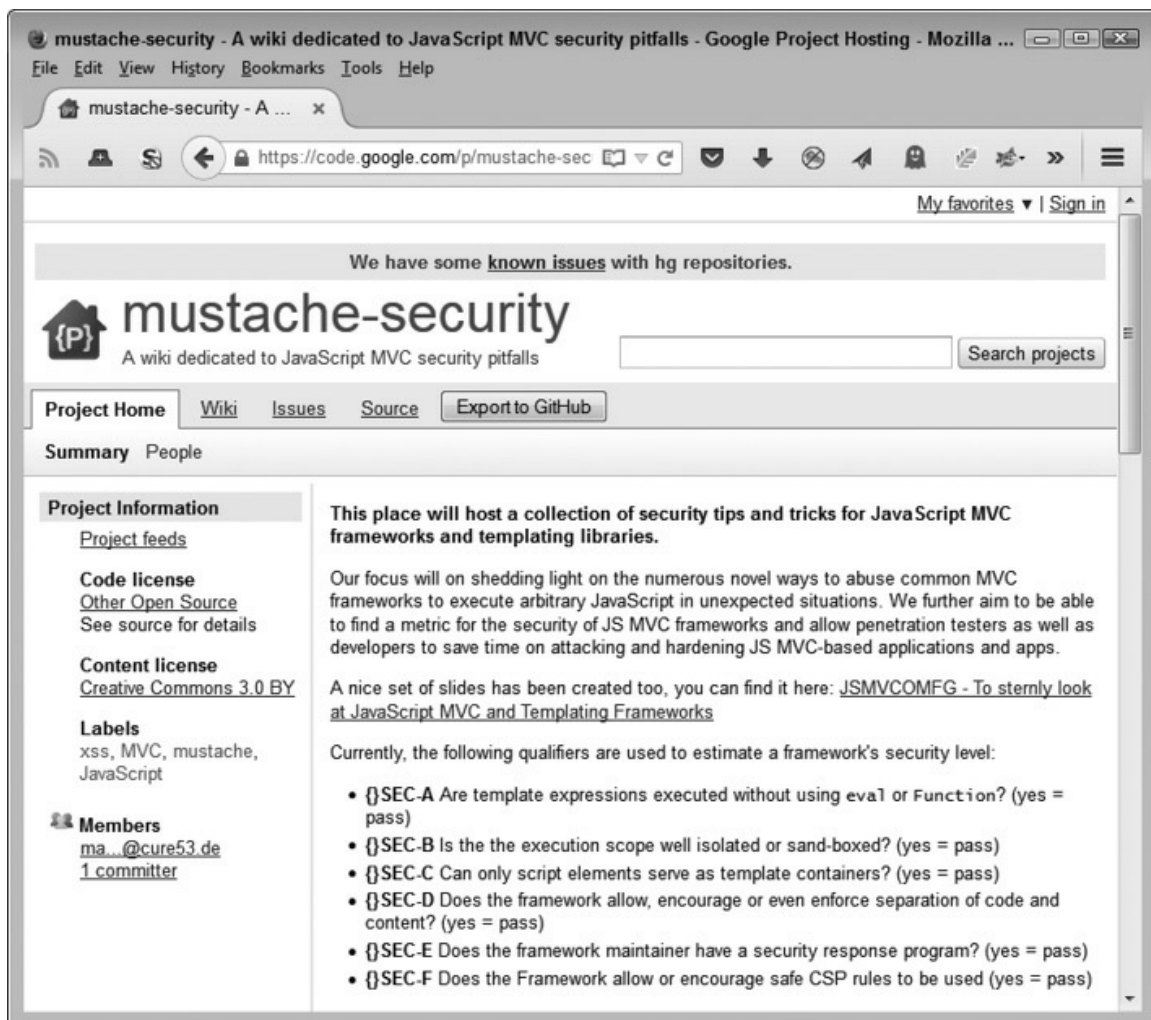


Figura 6.6 – Sites como o mustache-security oferecem sete níveis de verificação para frameworks.

Observando a parte inferior da página, você pode ver que as notícias sobre segurança em frameworks não são boas. A maioria dos frameworks atualmente contém problemas sérios de segurança que você deve considerar antes de usá-los no desenvolvimento de aplicações. A figura 6.7 mostra o resultado para alguns dos frameworks e bibliotecas de templating mais populares disponíveis atualmente.

mustache-security - A wiki dedicated to JavaScript MVC security pitfa...

File Edit View History Bookmarks Tools Help

mustache-security - A ... x

https://code.google.com/p/mu

Spot a mistake? Let us know! We go for fail if unclear - rather too harsh than too lax.

Framework	SEC-A	SEC-B	SEC-C	SEC-D	SEC-E	SEC-F
VueJS	Fail	Fail	Fail	Fail	Fail	Fail
AngularJS 1.0.8	Fail	Fail	Fail	Fail	PASS	Fail
AngularJS 1.2.0	Fail	PASS	Fail	Fail	PASS	PASS
AngularJS 1.4.0	Fail	PASS	Fail	PASS	PASS	PASS
CanJS	Fail	Fail	PASS	Fail	Fail	Fail
Underscore.js	Fail	Fail	PASS	Fail	Fail	Fail
KnockoutJS	Fail	Fail	Fail	Fail	Fail	Fail
Ember.js	Fail	PASS	PASS	Fail	PASS	TBD
Polymer	TBD	TBD	TBD	TBD	TBD	TBD
Ractive.js 0.4.0	Fail	Fail	Fail	Fail	Fail	Fail
Ractive.js 0.7.2	Fail	Fail	PASS	Fail	Fail	Fail
jQuery	TBD	TBD	TBD	TBD	PASS	TBD
JsRender	Fail	Fail	Fail	Fail	Fail	Fail
Kendo UI	Fail	Fail	Fail	Fail	Fail	Fail

Figura 6.7 – A maioria dos frameworks e das bibliotecas de templating atualmente tem um longo caminho a percorrer antes de se tornar segura.

O fato interessante sobre sites como esse é que com frequência eles oferecem exemplos específicos de como o framework falha. Por exemplo, clique no link para o AngularJS na tabela mostrada na figura 6.7 e você verá especificidades sobre como o framework falhou na área de segurança, como mostra a figura 6.8. As informações também incluem soluções alternativas que você pode usar para ajudar a atenuar os problemas, que é o motivo principal pelo qual você deve investir tempo pesquisando sobre os frameworks e garantir que compreende exatamente em que pontos eles deixam a desejar.

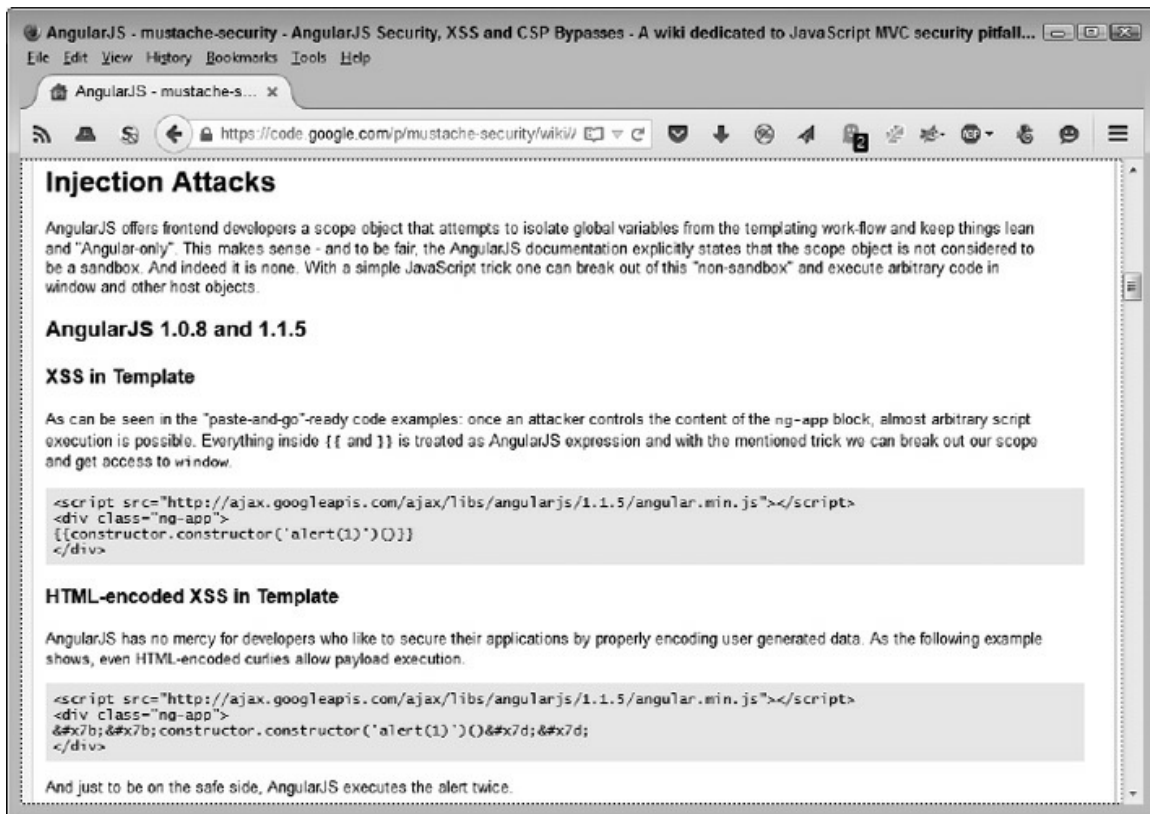


Figura 6.8 – Alguns sites de segurança oferecem códigos de exemplo que mostram problemas de segurança em frameworks.

CAPÍTULO 7

Usando APIs com cuidado

As APIs oferecem um meio de usar aplicações maduras no servidor de outra pessoa para atender às suas necessidades específicas. Trabalhar com uma API significa enviar uma solicitação a uma caixa-preta e receber algum tipo de resposta. A solicitação pode ser tão simples quanto uma chamada de função, mas, com frequência, exige que você forneça dados. A resposta pode estar na forma de dados, mas também poderia ser outro serviço. Por exemplo, há APIs para executar tarefas físicas como abrir portas. De fato, APIs provavelmente podem atender a quaisquer necessidades genéricas que você possa ter. Este capítulo discute as implicações de segurança do uso de APIs, incluindo como as APIs diferem das bibliotecas.

Parte de descobrir quais são as ameaças de segurança que as APIs representam é ver exemplos de códigos. Este capítulo usa um exemplo simples como parte do processo de explicação. Embora você possa usar a API alvo para um número significativo de tarefas da aplicação, a questão é criar algumas para uso em discussões e então ver como o uso de uma API poderia criar problemas para qualquer aplicação. Veja o exemplo deste capítulo como um ponto de discussão, em vez de vê-lo como uma técnica de programação.



Embora este capítulo discuta APIs específicas tanto em exemplos de código como na forma de exemplos dos pontos em que as APIs podem dar errado, o propósito não é dar destaque a nenhuma API em particular. Toda API que você usar pode ter defeitos sérios, que permitiriam a um hacker ter acesso ao seu sistema ou provocar outros tipos de problemas em sua aplicação. Os exemplos específicos ajudam você a entender como todas as APIs podem dar errado de modo geral. Em suma, você não deve ver os exemplos específicos deste capítulo como uma indicação de APIs particulares a serem evitadas.

É claro que o resultado de toda essa exploração é determinar como você pode

usar APIs de modo seguro como parte da estratégia de desenvolvimento de sua aplicação. Como as APIs são muito vantajosas, você não as verá desaparecer em curto prazo, mesmo que elas possam não ser uma boa ideia do ponto de vista de segurança. Na verdade, você pode contar com o fato de que o uso de APIs vai se expandir, pois os desenvolvedores têm problemas constantes de prazo que as APIs podem resolver. À medida que mais pessoas usam aplicações baseadas em web e essas aplicações se tornam mais complexas, você pode contar com o uso de APIs para expandir e atender a necessidades específicas. Conforme o uso aumenta, cresce o interesse em criar exploits por parte dos hackers que querem acessar seus dados e usá-los de maneiras que você jamais imaginou. Em suma, no mínimo, a influência das APIs em segurança aumentará, portanto seu cuidado ao usá-las também deve aumentar.

Diferenciando APIs de bibliotecas

Alguns desenvolvedores acham que todos os tipos de códigos externos são iguais. No entanto, APIs e bibliotecas diferem de modo significativo. Por exemplo, as APIs representam uma chamada out-of-process (fora do processo), pois executam em outro servidor, enquanto as bibliotecas representam uma chamada in-process (no processo) porque são incluídas como parte de sua aplicação. Só a questão de out-of-process *versus* in-process muda significativamente a maneira como você deve encarar os problemas de segurança. As próximas seções ajudam a definir as diferenças entre APIs e bibliotecas de modo mais completo; assim você pode compreender melhor as implicações de segurança ao usar uma em relação à outra.

Algumas pessoas usam os termos *biblioteca* e *API* indistintamente, mas ambos têm significados específicos neste livro. O capítulo 6 define as bibliotecas em detalhes, mas, essencialmente, uma biblioteca é um código externo que você incorpora em uma aplicação, usando qualquer técnica exigida pela linguagem host. Você deve usar as bibliotecas diretamente em seu código, como parte da aplicação. Como resultado, uma biblioteca é sempre uma técnica de implementação in-process, mesmo quanto o código-fonte da biblioteca estiver em um servidor de terceiros. Para usar a biblioteca, o código no servidor de terceiros é baixado em seu servidor e torna-se parte da aplicação, executando no



mesmo processo que ela.

Este livro também diferencia APIs usadas para executar tarefas e aquelas usadas como parte do esforço de desenvolvimento de uma aplicação. Por exemplo, as APIs web JavaScript descritas em <http://www.w3.org/standards/webdesign/script> definem os elementos da linguagem JavaScript. Esse tipo de API define uma linguagem usada para scripting. Sim, essa é uma API legítima, mas não no contexto usado neste livro. O contexto do livro está mais na linha de um web service, e não da especificação de uma determinada linguagem.

Considerando as diferenças em popularidade

Tanto bibliotecas quanto APIs podem oferecer vantagens ao desenvolvedor fazendo-o economizar tempo de desenvolvimento. De fato, a maioria das aplicações baseadas em web atualmente usa uma combinação de bibliotecas e APIs para executar tarefas. Entretanto, as APIs parecem ganhar em popularidade mais rapidamente que as bibliotecas por alguns bons motivos:

Uso reduzido de recursos

As APIs executam no servidor de outra pessoa, portanto não é preciso se preocupar em obter os recursos necessários. No entanto, o fato de o código executar no servidor de outra pessoa também significa que você perderá o controle de seus dados em algum momento e jamais verá o código da API propriamente dito para poder avaliar as ameaças de segurança que ele representa.

Requisitos de programação reduzidos

Usar uma API muitas vezes é mais fácil e exige menos código que usar uma biblioteca. É claro que você também tem menos controle sobre uma API. A contrapartida é a flexibilidade e o preço é a complexidade. Dado que os desenvolvedores estão cada vez mais sobrecarregados, os requisitos

reduzidos de programação das APIs pode deixá-las bem atraentes. Contudo, a flexibilidade reduzida também representa uma fonte em potencial de problemas de segurança porque você não pode mais personalizar o uso do código a fim de atender aos seus requisitos particulares.

Curva de aprendizado menor

Normalmente é mais fácil aprender a usar uma API que aprender a usar uma biblioteca porque a API, em geral, está em um formato cliente/servidor. O principal problema é determinar como formatar os dados para que a API possa entender o que você quer que ela faça. É na formatação de dados que está o maior problema de segurança. Um hacker poderia provocar uma má-formação de sua consulta, fazendo o servidor falhar ou forçá-lo a responder de formas inesperadas. Como a origem da consulta estará ligada à sua aplicação, o host a culpará em vez de acusar o hacker que provocou o problema.

Definindo as diferenças quanto ao uso

Não é possível usar APIs em todos os casos em que você pode usar uma biblioteca (e vice-versa). Algumas situações exigem uma abordagem integrada. Por exemplo, você não pode implementar facilmente uma interface de usuário usando uma API – isso realmente pede o uso de uma biblioteca. (Você pode obter os dados para a interface de usuário, como descrito no exemplo da API do Google mais adiante, mas isso é diferente de realmente criar os elementos da interface.) A lista a seguir descreve usos típicos de APIs e suas implicações quanto à segurança:

Consulta de dados

O uso mais comum para APIs é fazer requisições e obter dados como resultado. A requisição normalmente contém os parâmetros dos dados requisitados. Uma fonte de dados pode incluir bancos de dados, sensores, cálculos e análises. Sempre que lidar com dados, ataques man-in-the-middle são comuns. No entanto, um hacker poderia simplesmente optar por modificar os dados, a fim de enganar você ou usar os dados recebidos como um método para ter acesso à sua rede.

Sistemas de Supervisão e Aquisição de Dados (SCADA)

A IoT (Internet of Things, ou Internet das Coisas) conta com interfaces web para executar várias tarefas de SCADA (Supervisory Control and Data Acquisition, ou Sistemas de Supervisão e Aquisição de Dados). Por exemplo, com o termostato correto, é relativamente fácil ajustar a temperatura de sua casa usando uma aplicação web de seu smartphone, não importa onde você estiver. Um hacker poderia facilmente conectar-se à mesma interface e abaixar a temperatura de sua casa até que o encanamento congele ou deixá-la tão alta que será preciso abrir as portas quando você chegar em casa. É claro que o SCADA engloba todo tipo de atividade, incluindo controles industriais. Nesse caso, uma violação de segurança afeta não só os dados, mas tem efeitos físicos também.

Monitoração

Uma solução intermediária entre consulta de dados e SCADA é o ato de monitorar sistemas. Por exemplo, a polícia pode querer monitorar determinadas câmeras de rua durante um protesto ou outro evento. As APIs possibilitam criar aplicações que possam monitorar qualquer câmera em qualquer lugar, conforme for necessário, para oferecer proteção à população em geral. Um hacker poderia enganar a mesma API fazendo-a fornecer informações falsas.



Seria fácil pensar que uma aplicação usada por você ou por sua empresa não se classifique realmente como um sistema de monitoração. No entanto, sistemas de segurança doméstica, babás eletrônicas, dispositivos médicos e vários tipos de outras aplicações atuam como monitores. Consequentemente, o domínio dos softwares de monitoração pode ser muito maior do que você imaginou de início.

Multimídia

Uma API não precisa funcionar apenas com dados textuais ou outros tipos de dados abstratos. Ela também pode executar tarefas relacionadas a outros sentidos, em que o visual e o auditivo são os mais comuns. O controle e a manipulação de multimídia criam um ambiente em que os usuários interagem com aplicações de maneiras não tradicionais e os resultados muitas vezes são surpreendentes. Ao combinar multimídia com ciência aplicada a dados, é possível reconhecer novos padrões e objetos em

situações cotidianas, aumentando a capacidade de um usuário executar tarefas como visualizar cenas em ultravioleta que, de outro modo, seria impossível. No entanto, é importante perceber que a multimídia é apenas outra forma de dados, e dados de todo tipo são facilmente interceptados, corrompidos ou manipulados por hackers de maneiras indesejadas.

Localização

Embora a geolocalização seja a forma mais comum de aquisição, consulta e manipulação de localização, é importante entender que há APIs para trabalhar com todo tipo de informações de localização. A localização, assim como a multimídia, é um tipo especial de dado. Nesse caso, ela oferece um ponto bidirecional ou tridimensional no espaço, relativo a um espaço-alvo em particular, por exemplo, a Terra. A propósito, algumas APIs acrescentam uma quarta dimensão à equação: o tempo. Pense no que aconteceria se hackers decidissem modificar os dados de localização de modo que dois objetos, por exemplo, carros, tentassem ocupar o mesmo espaço ao mesmo tempo (resultando em um acidente).

Estendendo JavaScript usando APIs

É importante considerar a segurança do ponto de vista do mundo real. As próximas seções descrevem alguns problemas para localizar APIs apropriadas a serem usadas em testes e em desenvolvimento de aplicações. Daremos uma olhada então em um exemplo que mostra a API do Google Maps em ação. Essa aplicação simples demonstra a necessidade de considerar questões como requisições malformadas, erros de servidor e devolução errônea de dados. O exemplo é interessante porque mostra como criar uma aplicação que trabalha com o canvas do navegador para desenhar parte da interface de usuário baseada nos dados devolvidos pela API.



A melhor maneira de trabalhar com os exemplos descritos nesta seção é usar o código-fonte, que pode ser baixado em vez de digitado. Usar o código-fonte baixado reduz os erros em potencial. Você pode encontrar o exemplo com a API do Google Maps na pasta `\S4WD\Chapter07\GoogleAPI` do código-fonte disponível para download.

Localizando as APIs apropriadas

JavaScript oferece um ambiente de programação extensível com suporte tanto para APIs quanto para bibliotecas. Desenvolvedores têm tirado proveito de ambas as formas de extensão para criar alguns códigos compartilháveis impressionantes, que executam tarefas que você nem imagina que sejam possíveis. Por exemplo, é possível usar uma aplicação JavaScript para interagir com o recurso de vibração de um dispositivo móvel a fim de criar efeitos especiais (dê uma olhada na técnica em <https://developer.mozilla.org/en-US/docs/Web/Guide/API/Vibration>). Com o recurso correto, você pode encontrar uma API para executar praticamente qualquer tarefa que imaginar. A lista a seguir apresenta algumas ideias de onde procurar a API de seus sonhos:

- {API}Search (<http://apis.io/>)
- 40 useful APIs for web designers and developers (40 APIs úteis para web designers e desenvolvedores, <http://www.webdesignerdepot.com/2011/07/40-useful-apis-for-web-designers-and-developers/>)
- API Directory (<http://www.programmableweb.com/apis/directory>)
- API For That (<http://www.apiforthat.com/>)
- APIs Dashboard (<http://www.programmableweb.com/apis>)
- APIs Data.gov [clique no link “browse the current catalog for APIs” (navegue no catálogo atual para ver as APIs), <https://www.data.gov/developers/apis>]
- Guide to Web APIs (<https://developer.mozilla.org/en-US/docs/Web/Guide/API>)
- MashApe (<https://www.mashape.com/>)
- Public APIs (<https://www.publicapis.com/>)
- Web API Interfaces (<https://developer.mozilla.org/en-US/docs/Web/API>)

Essa é só uma amostra dos sites de pesquisa de APIs online. A maioria deles oferece um texto descritivo das APIs; alguns sites também oferecem avaliações. Antes de selecionar uma API em particular, leia o máximo que

puder sobre ela. Execute testes com a API em um sistema de testes. Não se esqueça de verificar os tipos de dados que a API pode realmente transferir entre seu sistema de testes e o host usando um sniffer de pacotes. Pode soar um pouco como paranoia testar APIs dessa maneira (veja a seção “Acessando APIs de forma segura a partir de JavaScript” na página 199 deste capítulo para ver mais detalhes), mas executar os testes ajuda a reduzir seus riscos. Lembre-se de que a segurança é sempre um equilíbrio entre os riscos e as vantagens obtidas ao assumir o risco.



Alguns fornecedores de API têm tantas APIs que eles oferecem sua própria lista. Por exemplo, o Google oferece uma variedade de APIs e você pode encontrá-las listadas no *GData API Directory* (<https://developers.google.com/gdata/docs/directory>). É importante perceber que muitas listas de hosts específicos contêm informações tendenciosas, e ler sobre as capacidades e deficiências da API em outro site é sempre recomendável.

Criando um exemplo simples

Criar uma aplicação segura que dependa de APIs é mais difícil do que você possa imaginar, pois mesmo as melhores APIs dependem de certa dose de confiança. Você troca o risco de confiar no host pelos dados ou outros recursos que ele oferece. Para ver como os problemas de segurança entram em cena, esta seção trabalha com um exemplo simples que utiliza a API do Google Maps. Para deixar a API utilizável, o exemplo também conta com as bibliotecas jQuery (<http://jquery.com/>) e jQuery UI (<http://jqueryui.com/>) para exibir os dados e a interface de usuário associada. Raramente você usará uma API sem também contar com bibliotecas para interagir com os dados de alguma maneira.



Para usar esse exemplo, você deverá obter uma chave de desenvolvedor. O Google oferece dois tipos de chave: paga e gratuita. Você só precisa da chave gratuita nesse exemplo. A chave paga oferece muito mais flexibilidade e, provavelmente, você precisará dela para qualquer aplicação completa que criar. No entanto, para experimentos, a chave gratuita servirá. Você pode obter essa chave em <https://developers.google.com/maps/licensing>. Certifique-se de entender totalmente os termos do serviço antes de começar a trabalhar com a API do Google Maps. Você também pode encontrar uma assistência adicional para o uso da API do Google Maps com JavaScript em <https://developers.google.com/maps/documentation/javascript/tutorial>.

É melhor criar o código desse exemplo em vários passos. O primeiro passo é adicionar as referências necessárias à jQuery mostradas a seguir:

```
<script
  src="https://code.jquery.com/jquery-latest.js">
</script>
<script
  src="https://code.jquery.com/ui/1.9.2/jquery-ui.js">
</script>
<link
  rel="stylesheet"
  href="https://code.jquery.com/ui/1.9.2/themes/base/jquery-ui.css" />
```

Embora o código faça uma referência a uma biblioteca externa, o código da biblioteca é incluído como parte da aplicação como um elemento in-process. O capítulo 6 descreve todos os problemas de segurança associados ao uso de bibliotecas. Além disso, também é necessário acrescentar uma referência à API do Google Maps, como mostra o código a seguir:

```
<script type="text/javascript"
  src="https://maps.googleapis.com/maps/api/js?key=Your Key Here&sensor=false">
</script>
```



Esse exemplo não funcionará a menos que você substitua as palavras “Your Key Here” pela chave que você receber do Google. Consequentemente, esse passo em particular é importante porque é um passo que você deve executar, mesmo que estiver usando o código baixado do site do livro.

As referências tanto para a biblioteca quanto para a API dependem da tag `<script>`, mas o modo como elas usam a tag é diferente. Observe que a biblioteca acessa um arquivo sem informações adicionais. A API exige informações adicionais na forma de uma chave e de uma opção de configuração de sensor nesse caso. À medida que avançarmos na seção, você verá outras diferenças.

Agora que você tem todas as referências necessárias definidas, é hora de criar um canvas para desenhar o mapa. O canvas é simplesmente uma `<div>`, como mostrado a seguir:

```
</div>
<div id="MapCanvas">
```



```
</div>
```

Você deve fornecer informações de estilo que especifiquem o tamanho da `<div>`; do contrário, o mapa não aparecerá na tela, mesmo quando o Google enviá-lo a você. O exemplo usa as seguintes informações de estilo:

```
#MapCanvas
{
  height: 90%;
  width:100%;
}
```

Além do canvas, o exemplo oferece duas caixas de texto para entrada e um botão que você pode usar para solicitar um novo mapa. Não há nada muito complexo na interface, mas ela faz seu trabalho (o exemplo está previamente configurado para mostrar Milwaukee no estado de Wisconsin no centro do mapa – você pode alterar a longitude e a latitude para qualquer outra localização desejada):

```
<div id="Input">
  <label for="longitude">
    Longitude:
  </label>
  <input id="longitude"
    value="-87.95"
    type="text" />
  <label for="latitude">
    Latitude:
  </label>
  <input id="latitude"
    value="43.04"
    type="text" />
  <input id="submit"
    value="Get Map"
    type="button" />
</div>
```

O código desse exemplo usa muitos dos truques da jQuery e da iQuery UI que você já viu em outras aplicações. Por exemplo, a aplicação cria um spinner para os controles de longitude e de latitude para facilitar mover o

centro do mapa de forma incremental. Mover um grau inteiro a cada vez não deixaria a aplicação muito útil, portanto os dois spinners mudam as entradas em um décimo de grau a cada vez (mesmo essa configuração pode ser grande demais e talvez você queira alterá-la). Observe o uso da opção `step` para executar essa tarefa. No código a seguir, a função `GetMap()` é a mais importante porque é ela que realmente exibe o mapa na tela:

```
$(function()
{
    // Controla a latitude atual usando um
    // controle do tipo spinner
    var Latitude = $("#latitude").spinner(
    {
        min: -90,
        max: 90,
        step: .1,
        change: function(event, ui)
        {
            if (Latitude.spinner("value") < -90)
                Latitude.spinner("value", -90);
            if (Latitude.spinner("value") > 90)
                Latitude.spinner("value", 90);
        }
    });

    // Controla a longitude atual usando um
    // controle do tipo spinner
    var Longitude = $("#longitude").spinner(
    {
        min: -180,
        max: 180,
        step: .1,
        change: function(event, ui)
        {
            if (Longitude.spinner("value") < -180)
                Longitude.spinner("value", -180);
            if (Longitude.spinner("value") > 180)
                Longitude.spinner("value", 180);
        }
    });
});
```

```

    }
  });
// Esta função exibe o mapa na tela
function GetMap()
{
  // Cria uma lista de argumentos para enviar ao Google
  var MapOptions =
  {
    center: new google.maps.LatLng(
      Latitude.spinner("value"),
      Longitude.spinner("value")),
    zoom: 8,
    mapTypeId: google.maps.MapTypeId.ROADMAP
  }
  // Especifica o local para posicionar o mapa e
  // as opções do mapa para o Google
  var map = new google.maps.Map(
    document.getElementById("MapCanvas"),
    MapOptions);
};
// O exemplo oferece dois métodos para obter um
// mapa: durante a carga da página ou clicando em Get Map
$(window).load(
  function()
  {
    GetMap();
  });
$("#submit").click(
  function()
  {
    GetMap();
  });
})

```

As latitudes variam de 90 graus norte a -90 graus sul, portanto o exemplo reflete esse requisito. De modo semelhante, as longitudes variam de 180 graus oeste a -180 graus a leste de Greenwich, na Inglaterra. Você pode ler mais sobre latitude



e longitude em <http://geography.about.com/cs/latitudelongitude/a/latlong.htm>.

A função `GetMap()` executa a tarefa propriamente dita de obter o mapa. Para isso, sua aplicação deve criar uma lista de opções de mapa. O exemplo mostra uma lista simples, porém típica. A mais importante dessas opções é o lugar para centralizar o mapa. Nesse caso, o mapa se centraliza automaticamente em Milwaukee, Wisconsin, mas você pode mudar as configurações para qualquer lugar que desejar. O exemplo usa um fator de zoom de oito, e você verá um mapa de estradas. A API do Google Maps disponibiliza uma série de tipos de mapas que você pode testar.

Há dois momentos em que `GetMap()` é chamada. Quando a aplicação é carregada, você vê Milwaukee, Wisconsin (a menos que tenha alterado as configurações default). Após alterar a entrada, você também pode clicar em Get Map (Obter mapa) para exibir uma nova localidade. A figura 7.1 mostra uma saída típica dessa aplicação.



Este capítulo não oferece uma visão detalhada da API do Google Maps, que é relativamente complexa. O que ele faz é oferecer uma visão simples de algo que você poderia facilmente expandir em uma aplicação completa mais tarde. Muitas empresas atualmente usam mapas para todo tipo de propósitos interessantes. O artigo em <http://blog.smartbear.com/software-quality/bid/242126/using-the-google-maps-api-to-addcool-stuff-to-your-applications> apresenta outras ideias sobre como você pode usar a API do Google Maps com sua aplicação baseada em navegador.

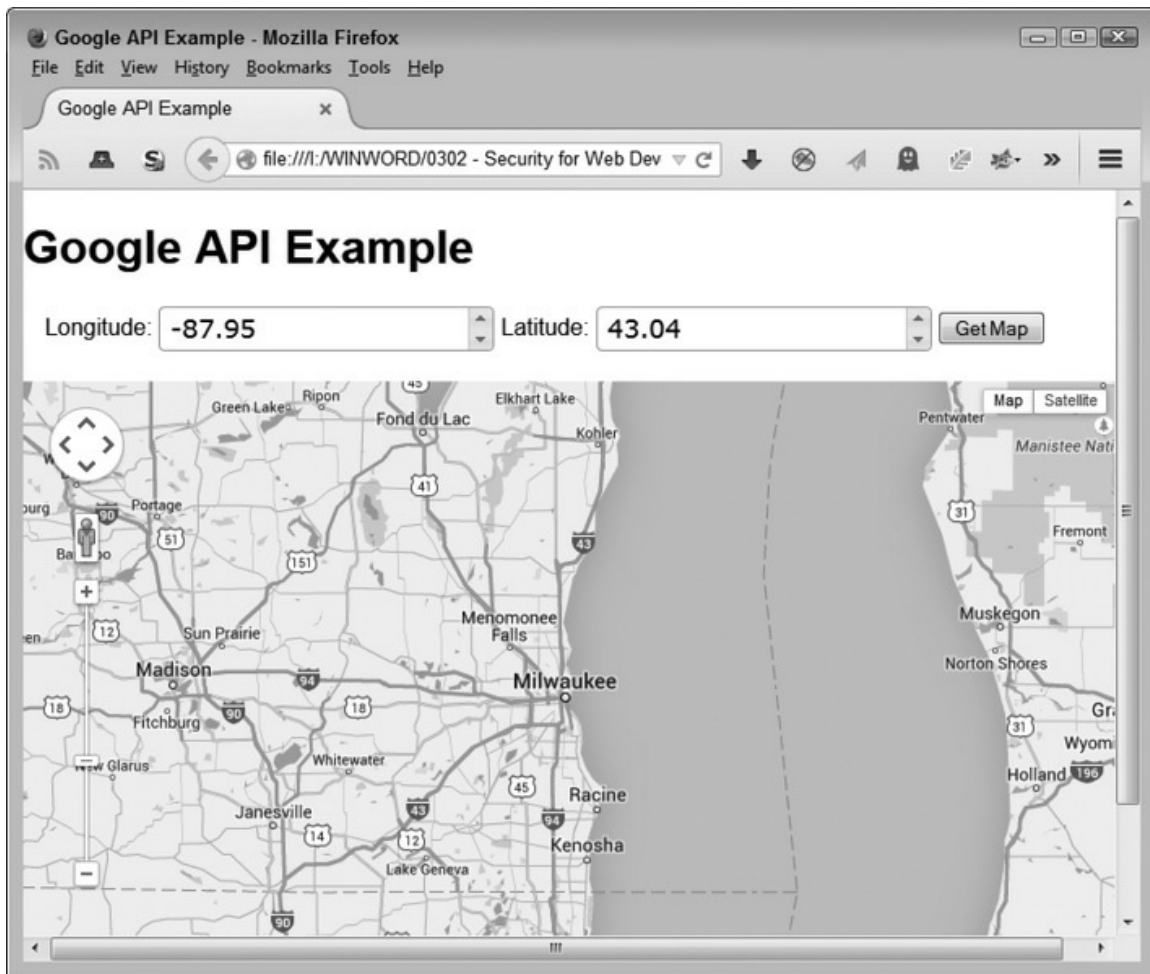


Figura 7.1 – O exemplo pode mostrar qualquer localidade para a qual você tenha uma longitude e uma latitude.

Definindo as ameaças de segurança causadas por APIs

É importante saber quais são os tipos de ameaça causadas por APIs que você vai encarar como desenvolvedor. Embora as próximas seções possam não conter as ameaças mais recentes das APIs, elas apresentam vários tipos dessas ameaças e oferecem o tipo de informação necessário para ajudar a pesquisar as ameaças mais recentes à sua aplicação. Esses exemplos do mundo real tornam possível entender os tipos de problema que você encontrará.

Arruinando seu bom nome com o MailPoet

Escrever um blog representa um investimento importante de tempo para muitas pessoas. Eu uso meu blog (<http://blog.johnmuellerbooks.com/>) para me comunicar com meus leitores e divulgar quais foram meus últimos esforços de autossuficiência às pessoas. Você pode encontrar um blog praticamente para qualquer finalidade ou assunto. As pessoas colocam suas reputações em seus blogs como uma maneira de ajudar o mundo como um todo. Desse modo, quando um plugin de software de blogging como o MailPoet (<http://www.mailpoet.com/>) tem um problema, ele pode afetar as reputações de todo blogueiro que o usa. É exatamente isso que aconteceu em 1º de julho de 2014.

A vulnerabilidade permite ao invasor colocar qualquer arquivo PHP no sistema (<http://blog.sucuri.net/2014/07/remote-file-upload-vulnerability-on-mailpoet-wysija-newsletters.html>) usando um upload remoto, o que significa que seu blog pode agora servir ataques de phishing (<http://wptavern.com/wordpress-mailpoet-plugin-security-vulnerability-immediate-update-recommended>) com as informações que ele costumava oferecer. À primeira vista, o hack fazia o site causar uma falha em alguns casos, mas essa falha foi corrigida, portanto, a menos que esteja atento, você poderá nem mesmo saber que houve um hack.

Um problema com essa vulnerabilidade em particular é que a entidade que descobriu o problema (Sucuri) aparentemente não deu tempo suficiente ao MailPoet para fazer uma correção e distribuí-la (<http://www.mailpoet.com/sucuri-hack-lessons-learned/>) antes de revelá-la, portanto há muitas acusações contra ele ainda. Esse plugin em particular foi baixado 1,7 milhão de vezes, e é usado em vários blogs por muitos indivíduos, assim tem um potencial significativo de causar problemas. A ameaça do MailPoet é um exemplo da necessidade de verificar a segurança da API antes de iniciar um desenvolvimento e realizar verificações constantes enquanto continuar a usar a API.

Há um problema significativo atualmente com a notificação de problemas de segurança. Alguns hackers white hat estão ansiosos demais para ganhar fama ao informarem um problema importante, não dando tempo suficiente ao host da API para corrigir os problemas antes de divulgar a informação – o que resulta em exploits zero-day (dia zero) sobre os quais outros hackers adoram saber. De



modo semelhante, alguns hosts de API são lentos para corrigir bugs que poderiam provocar problemas de segurança, o que significa que um hacker black hat os descobre antes de o problema ser corrigido (novamente, resultando em um exploit zero-day). Os hackers white hats que notificam o problema ficam apontando o dedo em acusação, dizendo a todos que fizeram o fornecedor tomar conhecimento do problema. Uma correção em potencial para esse problema seria criar um método padronizado para tratamento de notificação e correção de problemas de segurança para que todos pudessem trabalhar em conjunto, a fim de deixar o software mais seguro. Infelizmente, uma solução como essa não está nem em fase de discussão atualmente, quando escrevemos este livro.

Desenvolvendo uma imagem do snapping

O Snapchat (<https://www.snapchat.com/>) é uma ótima maneira de trocar fotos com outras pessoas de modo não permanente. O propósito do serviço é permitir um fluxo mais livre de informações em um ambiente com menos riscos. Na verdade, ele funciona quando todos obedecem às regras, mas nem todos o fazem (conforme explicado em *Security Lessons Courtesy of Snapchat* (Lições de segurança como cortesia do Snapchat, <http://blog.smartbear.com/apis/readyapi/security-lessons-courtesy-of-snapchat/>).

A API é extremamente fácil de hackear por causa de uma falha em uma biblioteca de terceiros. Em vez de realmente corrigir o problema, o Snapchat respondeu recentemente advertindo os usuários (<http://www.computerworld.com/article/2846038/snapchat-to-ask-users-to-stop-using-unauthorized-apps.html>) a não fazer download de programas de terceiros que permitam passar por cima das regras definidas. O resultado de não oferecer uma API sólida nesse caso é o vazamento de mais de 90 mil imagens que, provavelmente, são muito mais reveladoras do que a maioria gostaria de saber. Haja perda de reputação! É claro que o volume usual de extorsão ocorrerá e é pouco provável que alguém seja capaz de rastrear as implicações monetárias, pois ninguém estará disposto a falar disso.

A conclusão sobre esse hack de API em particular é que simplesmente não vale a pena usar certos serviços – não importa quão atraentes eles possam ser. Essa ameaça em particular mostra a necessidade de verificar a reputação do fornecedor antes de iniciar o desenvolvimento usando a API. Além do mais, isso mostra que você precisa pesquisar a API antes de usá-la, pois muitos hackers conheciam essa falha antes de a imprensa especializada começar a causar alvoroço com ela.

Perdendo seu dispositivo com o Find My iPhone

O serviço Find My iPhone (<https://www.apple.com/icloud/find-my-iphone.html>) foi criado para proteger seu iPhone em caso de perda. Você pode localizar seu iPhone, ver onde ele esteve e bloqueá-lo afastando-o de olhares curiosos. No entanto, a Apple não protegeu muito bem o acesso ao iCloud – o serviço que dá suporte ao Find My iPhone. Ele é vulnerável a ataques de força bruta – em que o invasor fica experimentando senhas até que uma tenha sucesso.

Alguns hackers descobriram essa falha e a usaram para bloquear os iPhones das

pessoas

(<http://ifwnewsletters.newsletters.infoworld.com/t/9545692/122348447/80127>

Por uma pequena quantia de cem dólares, a vítima podia conseguir que seu iPhone fosse desbloqueado. Se a vítima resistisse, os hackers prometiam apagar todos os seus dados. A única maneira de contornar o problema sem pagar era restaurar as configurações de fábrica no dispositivo. Felizmente, essa história teve um final feliz: os hackers foram presos (<http://www.computerworld.com/article/2490607/cybercrime-hacking/the-hackers-behind-those-iphone-ransom-attacks-have-been-arrested-in-russia.html>). Ninguém disse quantas pessoas pagaram a taxa, quantos dados foram perdidos ou quantos telefones foram hackeados, mas para gerar todo esse alvoroço na imprensa, deve ter sido mais que alguns.

Essa ameaça em particular é um exemplo da necessidade de testar entradas e saídas. Ao pensar em como a API funcionava, é possível que os desenvolvedores tivessem descoberto a abordagem de força bruta mais cedo e tivessem evitado os problemas em potencial.

Deixando suas informações mais importantes vazarem com o Heartbleed

Você depende do SSL (Secure Sockets Layer) para oferecer uma comunicação segura entre o cliente e o servidor. Infelizmente, como qualquer outro software, o SSL pode conter vulnerabilidades. Nesse caso, o uso de um recurso chamado heartbeat, em versões de 1.0.1 a 1.0.1f do OpenSSL (<https://www.openssl.org/>), permite que um invasor tenha acesso às senhas de usuários e a chaves privadas de longo prazo usadas para criptografar comunicações.

De acordo com a InfoWorld (<http://www.infoworld.com/article/2608736/data-security/heartbleed-to-blame-for-community-health-systems-breach.html>), o Heartbleed foi o culpado pela violação nos *Community Health Systems* (<http://www.chs.net/media-notice/>), disponibilizando registros de 4,5 milhões de pacientes. O preço dessa única violação pode ter custado até 150 milhões de dólares (<http://venturebeat.com/2014/08/25/community-health-systems-breach-could-cost-up-to-150-million/>). O interessante sobre o Heartbleed é que ele tem uma séria capacidade de permanência. De acordo com a ComputerWorld (<http://cwonline.computerworld.com/t/8953013/1084055737/677451/17/>), em 24 de junho de 2014, ainda havia 300 mil servidores suscetíveis ao Heartbleed (que é um tempo demorado para aplicar um patch, considerando que a vulnerabilidade foi descoberta em abril). A vulnerabilidade do Heartbleed é tão séria que deu origem a um site próprio para informações (<http://heartbleed.com/>).

Você deve ver essa ameaça como uma advertência para não confiar em nenhuma API em particular para permanecer seguro. Hackers estão constantemente sondando o ambiente para ver o que podem descobrir. Nesse caso, testes constantes da API poderiam ter resultado em informações sobre a vulnerabilidade antes que ela se tornasse um problema. Certamente contar com apenas um meio de proteger informações não é a atitude mais inteligente a ser tomada.

Sofrendo com o Shellshock

Alguns especialistas estão afirmando que o Shellshock (<http://www.troyhunt.com/2014/09/everything-you-need-to-know-about.html>) é pior que o Heartbleed (<http://www.eweek.com/news/why-shellshock-bug-is-way-nastier-than-heartbleed.html>). É difícil dizer se um é pior que o outro quando ambos são incrivelmente ruins e afetam um grande número de servidores. Nesse caso, um bug no utilitário bash, usado pelos administradores de Linux, pode ser hackeado criando uma variável de ambiente malformada e então executando um script do bash. O acesso a essa funcionalidade pode ser oferecido por meio de CGI (Common Gateway Interface) ou PHP.

O principal fator limitante desse hack é que ele não é explorável automaticamente (<http://www.infoworld.com/article/2687975/security/four-no-bull-facts-to-know-about-the-shellshock-bash-bug.html>) – deve haver uma interface externa para acessar o servidor. Há informações de que empresas maiores como Yahoo! (<http://www.businessweek.com/news/2014-10-06/yahoo-says-no-data-stolen-in-shellshock-hack>) e WinZIP (<http://arstechnica.com/security/2014/10/white-hat-claims-yahoo-and-winzip-hacked-by-shellshock-exploiters/>) tenham sido hackeadas com o Shellshock. O potencial para hacks com Shellshock é enorme porque o número de servidores afetados é maior que aqueles afetados pelo Heartbleed. Além disso, um patch viável demorou para ser disponibilizado (<http://www.infoworld.com/article/2687750/security/attacks-against-shellshock-continue-as-updated-patches-hit-the-web.html>); contudo, há um disponível atualmente (<http://www.eweek.com/security/shellshock-vulnerability-finally-patched-as-exploits-emerge.html>).

Esse é um exemplo da necessidade de manter seus dados localizados sob trava (lock) e chave. Ao oferecer uma interface externa para dados críticos, as empresas dão abertura à perda de dados. Manter dados sensíveis protegidos é a única maneira de garantir que eles realmente permaneçam seguros. É claro que você precisa deixar alguns dados acessíveis, mas fazer isso deve ser a opção de último recurso e você deve deixar apenas os dados necessários disponíveis.

Acessando APIs de forma segura a partir de JavaScript

É possível acessar APIs de forma segura usando JavaScript se você executar alguns testes antes. É claro que testar exige tempo e você pode enfrentar resistência para investir tempo e garantir que a API funcione conforme anunciado. Tudo que você realmente precisa fazer é perguntar qual é o valor dos dados da empresa quando comparado com o tempo exigido para testes. A resposta normalmente é óbvia e, considerando os dados, o tempo para testar sempre compensa. Com isso em mente, as próximas seções descrevem algumas medidas que você pode adotar para usar APIs de forma segura.

Verificando a segurança da API

A seção “Definindo as ameaças de segurança causadas por APIs” na página 194 descreve algumas das ameaças que as APIs podem representar ao desenvolvedor. Infelizmente, a imprensa especializada muitas vezes expõe essas ameaças depois que você implementou uma solução usando a API, dificultando mudar a API que você está usando sem incorrer em um alto custo. No entanto, verificar se a API está limpa no momento procurando histórias sobre ela é sempre um bom lugar para começar.

Também vale a pena dar uma olhada na reputação do fornecedor em termos de segurança. Novos fornecedores não estão testados e, desse modo, não são confiáveis. Use-os sabendo que haverá um risco bem maior que utilizar um fornecedor conhecido: um que já tenha produtos no mercado e tenha sido testado durante um bom período de tempo. Para compensar parcialmente a escolha de fornecedores novos *versus* os mais experientes está o fato de que os hackers também têm como alvo fornecedores com uma parcela maior de participação no mercado, a fim de aumentar as chances de obter algo realmente útil em troca de seus esforços.

O modo como a API é construída, a documentação fornecida e a quantidade de entradas requisitadas para as chamadas apontam para o nível de segurança oferecido. Pode ser incômodo inscrever-se para um serviço oferecido pelo Google, mas o fato de precisar usar uma chave especial para acessar a API pode reduzir a superfície de ataque a ela, pelo menos um pouco. Além disso, as APIs do Google não pedem que você forneça dados sensíveis; apesar

disso, é possível que um invasor extraia alguns dados simplesmente observando suas requisições. Por exemplo, pedir mapas em longitudes e latitudes específicas poderia indicar um novo empreendimento na área ou outro motivo para o aumento de atenção.

A API do Google é uma boa opção para o exemplo deste capítulo porque o Google tem uma reputação relativamente boa quanto à segurança. Além do mais, é fácil verificar se você está realmente interagindo com o Google na maioria dos casos e se os resultados recebidos são do tipo correto. A documentação do Google ajuda você a executar os tipos de verificação que facilitam conferir se a API é segura.



Não se esqueça de manter as necessidades de comunicação em mente quando analisar a segurança das APIs. É importante garantir que o fornecedor ofereça criptografia de dados SSL e que ambas as partes garantam que sabem com quem estão conversando ao empregar autenticação SSL mútua.

Porém, não deixe uma comunicação segura se tornar um potencial problema de segurança por si só. Hackers podem e roubam certificados de modo que eles podem fornecer o tipo de informação secreta necessária para conferir se uma comunicação é segura. Você nunca sabe realmente quem está do outro lado da linha. Tudo que você pode fazer é oferecer a segurança necessária e esperar que ela seja robusta o suficiente para manter a maioria dos hackers afastados.

Testando entradas e saídas

Uma maneira simples de verificar a utilidade e a segurança de uma API é simplesmente testar quantidades conhecidas. Verifique entradas e saídas conhecidas para garantir que a API funciona conforme anunciado. Você pode ficar surpreso ao descobrir que algumas APIs realmente não funcionam dessa maneira. Em alguns casos, o problema não está em erros de programação, mas em não entender como a API funciona.

É claro que muitas empresas realizam testes de entrada/saída durante a fase inicial de design e de desenvolvimento de uma aplicação. Contudo, criar um *test harness*¹ permite que você execute testes aleatoriamente. A vantagem dessa abordagem é que os testes podem ajudar a detectar corrupção de dados em potencial ou outros possíveis problemas de segurança. Executar testes a intervalos regulares ajuda a garantir que a API permaneça viável. Os testes

também tendem a invalidar qualquer conclusão que bisbilhoteiros possam tirar sobre as consultas que você faz na API.

O exemplo com a API do Google Maps neste capítulo também deve incluir um código para verificar dados tanto de entrada quanto de saída. Essas verificações foram deixadas de fora para deixar a aplicação mais fácil de entender. Contudo, em uma aplicação de verdade, você deve testar os dados antes de submetê-los e conferir se você obtém o tipo de informação esperado como valor de retorno. É claro que não é possível verificar os dados especificamente, pois o mapa mudará a cada chamada.



É importante verificar se as APIs que você usa permanecem consistentes ao longo do tempo. Em alguns casos, um fornecedor disponibilizará uma atualização de API que introduz mudanças de última hora. A mudança pode ser tão sutil que você não a perceberá à primeira vista, mas um hacker poderá notá-la enquanto procura brechas de segurança em potencial para explorar. Realizar testes de entrada e saída regularmente ajudará a atenuar esse tipo de problema ao verificar se a API continua funcionando conforme esperado, mesmo quando o fornecedor faz essas atualizações inesperadas.

Mantendo os dados localizados e seguros

Quanto mais dados você envia a uma API, maior a probabilidade de alguém interceptá-los. Os dados não precisam ser interceptados no caminho para representarem um problema. Se os dados acabarem ficando no servidor do host, então qualquer um que acesse o servidor provavelmente poderá acessar os dados também. A melhor maneira de manter um segredo é não o contar a ninguém. Compartilhar o mínimo possível de dados com uma API é uma boa ideia.

O uso de uma chave codificada com a API do Google é uma boa medida de segurança porque ela identifica positivamente sua empresa. No entanto, a chave é importante por outro motivo. Em alguns casos, outras APIs usam identificadores que um hacker pode rastrear até chegar à sua empresa. Qualquer um que interceptar a chave do Google não saberá nada mais sobre sua empresa além do que já sabia antes.

Um exemplo de uma API que costumava exigir um dado de entrada que um hacker poderia associar a uma empresa era a API da Amazon.com. Algumas

APIs costumavam depender do identificador relacionado que uma empresa também associava a links para compra de produtos. Um hacker poderia descobrir esse identificador com relativa facilidade e começar a rastrear as chamadas de API de uma empresa também.

Programando de forma defensiva

Quando escrever sua aplicação, pressuponha sempre que as entradas e saídas estão erradas, que o usuário tratará tudo de forma incorreta e que há um hacker espiando sobre seu ombro. Sim, os desenvolvedores precisam desenvolver um verdadeiro senso de paranoia para evitar alguns dos problemas predominantes nas aplicações atualmente. À medida que o livro avançar, você verá a utilidade de técnicas como sandboxing de API para determinar como a situação pode ficar ruim quando se trata de interagir com a API. O importante é não pressupor nada sobre o código – até mesmo do código criado por você.

¹ N.T.: Em testes de software, um *test harness* ou um *framework automatizado de testes* é uma coleção de softwares e de dados de teste configurados para testar uma unidade de programa executando-a em condições variadas e monitorando seu comportamento e suas saídas. (Baseado em: https://en.wikipedia.org/wiki/Test_harness.)

CAPÍTULO 8

Considerando o uso de microserviços

Microserviços são uma tecnologia relativamente nova que separa aplicações monolíticas enormes em componentes pequenos. Cada um desses componentes pequenos atua independentemente e executa bem apenas uma tarefa. Por causa das tecnologias das quais os microserviços dependem e da maneira como eles são empregados, os microserviços tendem a oferecer mais segurança que algumas das outras tecnologias descritas até agora no livro. No entanto, assim como qualquer outra tecnologia, os microserviços apresentam oportunidades para os hackers causarem problemas. É importante lembrar que qualquer tecnologia tem lacunas que os hackers explorarão para executar tarefas. O objetivo do desenvolvedor é minimizar essas lacunas e garantir o máximo possível que medidas de proteção estejam implantadas para ajudar no processo de monitoração.

Pelo fato de os microserviços serem tão novos, o capítulo começa gastando um pouco mais que o tempo usual para explicá-los. Este livro não apresenta uma visão completa dos microserviços, mas você deverá ter informações suficientes para entender as implicações para a segurança devido ao uso de microserviços no lugar de tecnologias mais antigas utilizadas no passado. Além disso, é importante considerar o papel que as pessoas terão nesse caso. Uma atitude hostil à implantação de microserviços pode causar problemas de segurança que você deve considerar durante a fase de desenvolvimento.

O capítulo discute como você pode criar um microserviço próprio (mas não fornece o código-fonte, pois este é um livro sobre segurança e não sobre criação de microserviços). O exemplo foca em uma combinação de Node.js e Seneca para criar um microserviço simples e então acessa esse

microserviço a partir de uma página. O ponto principal do exemplo é discutir como os microserviços funcionam para que você possa entender melhor as informações de segurança apresentadas a seguir na próxima seção. O motivo para usar a combinação de Node.js e Seneca é que essas aplicações executam nas plataformas Mac, Windows e Linux. Outros microserviços como o Docker executam apenas em sistemas Linux atualmente.

O capítulo termina revendo a importância de ter vários paths para acesso ao microserviço. Uma das vantagens de usar microserviços é que você pode empregar várias cópias do mesmo microserviço para reduzir o risco de uma falha na aplicação. Em suma, os microserviços podem ser mais seguros e mais confiáveis que as aplicações monolíticas que eles substituem.



A melhor maneira de trabalhar com os exemplos descritos neste capítulo é usar o código-fonte que pode ser baixado, em vez de digitá-lo manualmente. Usar o código-fonte baixado reduz os erros em potencial. Você pode encontrar os códigos-fonte dos exemplos deste capítulo na pasta `\S4WD\Chapter08` no código-fonte disponível para download.

Definindo microserviços

As aplicações que funcionam em uma única plataforma irão se afastar da maioria dos usuários em algum momento. Sim, elas continuarão existindo para necessidades especiais, mas as aplicações comuns, com as quais a maioria dos usuários conta no cotidiano, não se importarão com a plataforma, os requisitos da linguagem de programação ou qualquer outro aspecto que as aplicações precisam considerar atualmente. Os microserviços funcionam bem no ambiente de programação atual porque definem uma nova maneira de olhar para o código. Em vez de se preocupar em como criar o código, onde colocá-lo ou qual linguagem usar, o desenvolvedor pensa em apenas uma tarefa que o código deve executar. Essa tarefa pode nem mesmo se encaixar, necessariamente, na aplicação no momento – ela pode simplesmente representar algo interessante que a aplicação talvez precise fazer quando estiver trabalhando com os dados. No novo mundo do desenvolvimento de aplicações, elas executarão em qualquer lugar, a qualquer momento, por causa de tecnologias como a dos microserviços. As próximas seções apresentam uma boa visão geral do que são exatamente os microserviços e

por que você deve se importar com eles.

Especificando as características dos microsserviços

Muitos desenvolvedores estão acostumados a lidar com designs monolíticos, que dependem intensamente de técnicas de POO (Programação Orientada a Objetos). A criação de qualquer aplicação começa pela definição de todo tipo de objetos e pela consideração de todo tipo de problemas. As técnicas atuais de design de aplicação exigem muito trabalho prévio somente para começar, e elas estão associadas a plataformas específicas. Os microsserviços são diferentes. Em vez de uma porção enorme de código, você escreve porções extremamente pequenas de código e toma muitas decisões à medida que avançar, em vez de tomá-las no início do processo. Os microsserviços têm as seguintes características:

São pequenos

Cada microsserviço executa apenas uma tarefa.

São independentes de linguagem

Todo microsserviço depende da linguagem que seja mais adequada à tarefa que ele realiza, sem nenhuma consideração para as necessidades de outros microsserviços.

São independentes de transferência de dados

Embora a maioria dos microsserviços atualmente dependa de JSON (JavaScript Object Notation, ou Notação de Objetos JavaScript) para transferir dados, você pode usar qualquer método de transferência de dados que funcione melhor para o microsserviço.

Usam fila de mensagens

A comunicação tipicamente ocorre com um sistema assíncrono de mensagens de modo que nenhum microsserviço possa provocar atrasos na aplicação como um todo.

Pipe burro (dumb pipe)

Um problema com muitos métodos de comunicação atualmente é que a

inteligência está no pipe. Os microsserviços dependem de um pipe burro e de serviços inteligentes. A maioria dos microsserviços depende de REST (Representational State Transfer, ou Transferência de Estado Representativo) para comunicação.

São descentralizados

Cada microsserviço é separado de todos os demais microsserviços e da aplicação como um todo. Uma falha em um microsserviço normalmente não afetará a operação da aplicação. Cada microsserviço tem uma monitoração separada.

São independentes de plataforma

Qualquer aplicação pode usar qualquer microsserviço, independentemente da plataforma em que a aplicação está executando e da plataforma usada pelo microsserviço.

Você pode estar se perguntando qual é o tamanho dos microsserviços – o que executar bem uma única tarefa quer dizer. Pense em uma string por um momento. Ao trabalhar com uma aplicação monolítica, você tem um único objeto que pode converter a string para letras maiúsculas, invertê-la e transformá-la em um número. Ao trabalhar com um microsserviço, você cria um único microsserviço para executar cada tarefa. Por exemplo, um microsserviço iria converter a string em letras maiúsculas, outro iria invertê-la e outro ainda iria transformá-la em um número. Quando pensar em microsserviços, pense de modo focado e reduzido.

Do ponto de vista de um desenvolvedor, os microsserviços representam o que há de mais flexível e modular. É possível trabalhar com uma única função de cada vez sem provocar distúrbios em outras partes da configuração. Além do mais, como as atualizações são muito pequenas, não é como uma API em que você faz um enorme investimento de tempo e esforço. Quando cometer um erro, corrigi-lo será um problema muito menor.

Diferenciando microsserviços de bibliotecas

É importante perceber que os microsserviços não executam in-process como as bibliotecas. Um microsserviço executa em um servidor, como uma API.

Isso quer dizer que você não tem os riscos de segurança presentes nas bibliotecas ao usar microsserviços. É possível separar totalmente o código de sua aplicação do microsserviço.

A sintaxe para chamada de um microsserviço também difere de uma biblioteca, pois você cria uma requisição JSON e a envia para o servidor. A resposta também tem formato JSON. O uso de JSON possibilita trabalhar com dados de forma rica, sem recorrer ao XML. Trabalhar com JSON é muito mais fácil que trabalhar com XML porque o JSON é nativo de JavaScript e oferece uma sintaxe mais leve. Você verá como isso funciona mais adiante neste capítulo. Por enquanto, basta saber que os microsserviços funcionam de modo diferente das chamadas de biblioteca em sua maior parte.

Do ponto de vista de segurança, os microsserviços tendem a ser mais seguros que as bibliotecas porque eles não executam in-process e você pode se proteger contra a maioria das formas de entradas errôneas de dados usando as abordagens com as melhores práticas para trabalhar com JSON. É claro que os hackers podem frustrar qualquer esforço para deixar a aplicação mais segura, e os microsserviços não são exceção.

Diferenciando microsserviços de APIs

APIs geralmente exigem que você crie um objeto e então execute chamadas nesse objeto. As requisições podem assumir vários formatos como REST, cabeçalhos de requisição HTML ou XML. As respostas podem envolver manipulação direta de objetos da página (como no caso do exemplo de API do Google Maps mostrado no capítulo 7). O processo é desajeitado porque você trabalha com uma porção grande de código monolítico que poderia conter todo tipo de inconsistências.

Assim como as APIs, os microsserviços executam out-of-process. No entanto, de modo diferente das APIs, os microsserviços não são porções enormes de código monolítico. Cada microsserviço é pequeno e poderia executar em seu próprio processo, possibilitando isolar uma função de outra com total garantia. Trocas de dados ocorrem usando apenas uma abordagem, o JSON, que provavelmente é a melhor abordagem a ser usada atualmente porque é mais simples que trabalhar com XML.

Considerando as políticas dos microsserviços

Agora você sabe que os microsserviços têm muito a oferecer ao desenvolvedor, à área de TI em geral e à empresa como um todo. Usar microsserviços faz sentido porque a tecnologia permite criar aplicações que funcionam em qualquer dispositivo, em qualquer lugar, sem impor dificuldades ao desenvolvedor. Infelizmente, cenários de desenvolvimento de aplicações monolíticas tendem a criar feudos em que uma hierarquia de gerentes administra seu próprio conjunto particular de recursos. Como os microsserviços são pequenos, facilmente utilizáveis para todo tipo de propósito e tendem a não se importar com a proveniência dos dados necessários, eles derrubam as paredes entre grupos organizacionais – irritando os feudos que governavam no passado. Como em qualquer situação desse tipo, certo nível de brigas e até mesmo de sabotagem pode ocorrer.

A parte referente à sabotagem da equação é a que você precisa considerar como desenvolvedor. É pouco provável que alguém vá gastar seu tempo tentando acabar com um projeto de microsserviços de propósito, porém a sutil relutância em executar as tarefas ou fazê-las corretamente pode acabar igualmente com o projeto. Todas as empresas têm uma atitude do tipo “nunca fizemos isso dessa maneira aqui antes” quando se trata de novas tecnologias – a inércia tem um papel a ser desempenhado em todos os empreendimentos humanos, portanto você não deve se surpreender se perceber que precisa vencer a inércia antes de iniciar seu primeiro projeto.

Do ponto de vista da segurança, as falhas induzidas no projeto durante essa etapa inicial criam aberturas das quais os hackers estão totalmente cientes e é quase certo que irão explorá-las se sua empresa tornar-se um alvo (ou, às vezes, por pura coincidência). Com tudo isso em mente, muitas vezes seguir um processo ao incorporar estratégias de microsserviços em sua caixa de ferramentas de programação pode ser útil (você nem sempre seguirá esses passos na ordem exata listada, mas eles ajudam a vencer parte da relutância envolvida no trabalho com microsserviços):

1. Monte uma equipe de desenvolvimento responsável pelo desenvolvimento dos microsserviços, que seja diferente da equipe que, no momento, mantém a aplicação monolítica.

2. Crie alguns microsserviços um pouco menos granulares para novos recursos da aplicação, para começar.
3. Desenvolva microsserviços que ofereçam recursos de negócio autocontidos desde o início para que você não precise se preocupar tanto com interações.
4. Dê tempo suficiente para que as equipes existentes descubram como usar os microsserviços e comecem a incorporá-los em aplicações existentes. No entanto, não mova totalmente as aplicações existentes para microsserviços até que você tenha sucesso suficiente e todos concordem que fazer a mudança seja uma boa ideia.
5. À medida que o desenvolvimento avançar com os microsserviços iniciais e você puder ver os pontos que precisam de mudança, crie microsserviços mais granulares para ter melhores resultados.
6. Padronize templates de serviços para que seja possível criar microsserviços com um mínimo de discussões entre os grupos. Um template padronizado também tende a reduzir problemas de segurança porque ninguém precisa fazer suposições.
7. Crie microsserviços suficientemente granulares para desenvolver uma aplicação completa, mas não foque nas necessidades de uma aplicação existente – considere criar uma nova aplicação.
8. Obtenha as ferramentas necessárias para realizar uma monitoração granular, agregação de log, obtenção de métricas de aplicações, implantação automatizada e criação de painéis de controle de status para dados como status de sistema e relatório de logs.
9. Implemente uma aplicação pequena baseada somente em técnicas de desenvolvimento de microsserviços. A ideia é criar uma aplicação completa que mostre que os microsserviços são realmente capazes de fazer o trabalho. Desenvolver uma aplicação pequena tende a reduzir o potencial de falhas para um grupo de desenvolvimento que esteja começando a aprender o ofício.
10. Aos poucos, dê treinamento a diferentes indivíduos para que as divisões abruptas entre conjuntos de habilidades diminua.

11. Separe os silos entre vários grupos. Comece a criar microsserviços que criem código, recursos e dados de todo grupo disponível para todos os demais grupos, sem considerar qual grupo originou o item.
12. Passe o desenvolvimento da aplicação monolítica existente para um projeto baseado em microsserviços.
13. Partindo de um pequeno projeto monolítico, passe totalmente o projeto monolítico para um ambiente de microsserviços, se for possível. Execute a tarefa aos poucos e use métricas após o acréscimo de cada microsserviço para garantir que a aplicação realmente funcione mais rápido, execute de forma mais confiável e permaneça segura.
14. Elimine microsserviços mais antigos do sistema à medida que os substitui por microsserviços mais granulares e mais funcionais.

Fazendo chamadas a microsserviços usando JavaScript

A seção anterior ajudou a entender o que é um microsserviço, mas não mostrou como um microsserviço funciona. As próximas seções apresentam um exemplo simples de como você pode compor um microsserviço e usá-lo em uma aplicação. É claro que você precisa de muitos microsserviços para criar uma aplicação totalmente funcional, mas esse exemplo é um bom ponto de partida.

Criando uma configuração de microsserviço para JavaScript

Antes de poder começar a trabalhar com microsserviços, você precisa de uma configuração que os aceite. É claro que você tem todo tipo de opções para criar uma instalação utilizável, mas um caminho é bem mais fácil que os demais – usar uma combinação de Node.js e Seneca.

Comece instalando uma cópia do Node.js em seu sistema, se você ainda não tem uma instalada. O download necessário à instalação está em <https://nodejs.org/download/>. Ter instruções de instalação pode ajudar. Você pode encontrar instruções para uma instalação em Mac em <http://blog.teamtreehouse.com/install-node-js-npm-mac>, para uma instalação em Windows em <http://blog.teamtreehouse.com/install-node-js-npm-windows> e para uma instalação em Linux em <http://blog.teamtreehouse.com/install-node-js-npm-linux>. Certifique-se de que a pasta que você usar permite acesso ao

desenvolvedor, o que significa não usar a pasta *C:\Program Files* em sistemas Windows, por exemplo. (Instale o produto em *C:\nodejs* em sistemas Windows, se for possível.) Não se esqueça de executar os testes sugeridos após a instalação para garantir que sua instalação de Node.js esteja funcionando apropriadamente.

Este livro usa Seneca para microsserviços porque ele funciona bem em sistemas Mac, Windows e Linux. É claro que você pode usar qualquer gateway de API que esteja de acordo com suas necessidades. Após instalar o Node.js, você pode usar o Node Package Manager (NPM) para instalar o Seneca, usando as instruções que se encontram em <http://senecajs.org/install.html>. Verifique se você está na pasta que planeja usar para criar seu código quando instalar o Seneca para que os arquivos dele estejam no lugar certo. O código-fonte disponível para download inclui os arquivos do Seneca para você.

Usar o NPM (<https://www.npmjs.com/>) facilita instalar todo tipo de pacote para Node.js e reduz a complexidade de criar vários tipos de aplicação. É importante observar que as instruções de instalação do Seneca mostram um prompt \$. Ao trabalhar com plataformas diferentes de Linux, você verá outro tipo de prompt, mas o comando `npm install Seneca` é o mesmo em todas as plataformas. Um fato interessante é que o prompt simplesmente exibirá um indicador de ocupado durante o processo de instalação – você não verá nenhum tipo de saída que informe que algo está realmente acontecendo até o final, quando verá a estrutura de diretórios do Seneca. (Se você não conseguir fazer a instalação padrão do NPM funcionar de forma alguma, é sempre possível obter o código-fonte diretamente de <https://github.com/rjrodger/seneca> e instalá-lo dessa maneira, mas a instalação manual definitivamente é mais difícil.)

Entendendo o papel do REST na comunicação

Os microsserviços dependem de REST, que é um estilo de arquitetura de comunicação, porque ele é mais leve que os protocolos como o SOAP (Simple Object Access Protocol, ou Protocolo Simples de Acesso a Objetos). Usar SOAP tem suas vantagens em algumas situações, mas apresenta problemas em cenários de internet, como o uso significativo de banda e a necessidade de um nível mais formal de comunicação entre cliente e servidor. As aplicações que dependem de REST para comunicação são chamadas de aplicações RESTful. O uso de REST em microsserviços oferece as seguintes vantagens:

- Desacopla consumidores de produtores

- Oferece comunicação sem estados (stateless communication)
- Permite o uso de cache
- Permite o uso de um sistema de camadas
- Oferece uma interface uniforme

Você tem uma série de opções para usar REST com microsserviços. No entanto, o método mais simples (e o método usado no exemplo) é contar com um URL especialmente formatado. Por exemplo, *http://localhost:10101/act?say=hello* é o URL usado no exemplo. Nesse caso, você entra em contato com o localhost usando uma porta especial, 10101, e envia uma mensagem usando um act. A mensagem é interpretada como um par JSON nome/valor, {say:"hello"}. O exemplo mostra como tudo isso funciona, mas a ideia é que você envie uma requisição e obtenha uma resposta JSON de volta. Usar REST para comunicação simplifica a tarefa.

Transmitindo dados usando JSON

Os microsserviços dependem de JSON para transferir tanto requisições quanto respostas. Sim, você também pode enviar dados usando REST, mas, em última instância, a informação acaba em formato JSON. Há três motivos principais para usar JSON na transferência de dados:

Dados limpos

O formato de dados para JSON é simples. Os dados aparecem em dois formatos: pares nome/valor ou uma lista de valores. Como o formato dos dados é muito rígido e simples, há menos chance de erros na transferência de dados e, sendo assim, menos problemas de confiabilidade e de segurança.

Eficiência

Como JSON evita toda a aparência cheia de tags de HTML e de XML, ele tende a ser menor que outros tipos de transferência de dados. A informação continua em formato texto, mas o formato em si é bem eficiente, o que significa que você desperdiçará menos recursos transferindo os dados.

Escalabilidade

O formato rígido de dados usado por JSON significa que transferências de dados são padronizadas, o que facilita expandir sua aplicação conforme for necessário. Usar uma única estrutura de dados quer dizer que você pode plugar seu código em qualquer lugar em que for necessário.

Normalmente, JSON usa cinco formatos de dados distintos. De modo diferente de XML, você não cria hierarquias complexas de dados que podem ter praticamente qualquer formato imaginável. A seguir apresentamos os cinco formatos com que você pode contar para transmitir dados:

Objeto

Um objeto é um par nome/valor. O par aparece entre chaves ({}), e é separado por dois-pontos (:). Você pode criar objetos complexos separando vários pares nome/valor com uma vírgula. Por exemplo, {say:"hello"} é um par nome/valor.

Array

Um array é constituído de um ou mais valores entre colchetes ([]). Por exemplo, ["One", "Two", "Three"] é um array contendo três valores de string.

Valor

Um valor é um único item. JSON reconhece string, número, objeto, array, true, false e null como valores.

String

Uma série de caracteres entre aspas (a maioria dos textos diz que você deve usar aspas duplas). JSON reconhece caracteres de controle precedidos por uma barra invertida (\). Esses caracteres são: backspace (\b), formfeed (\f), quebra de linha (\n), carriage return (\r) e tab horizontal (\t). Você também pode especificar caracteres Unicode usando \u e um valor hexadecimal de quatro dígitos. Por exemplo, \u00BC é o símbolo de um quarto (¼).

Número

Um número é uma série de caracteres numéricos sem aspas, com ou sem ponto decimal. Os sinais de mais e de menos mostram valores positivos e negativos. Você também pode especificar números usando notação

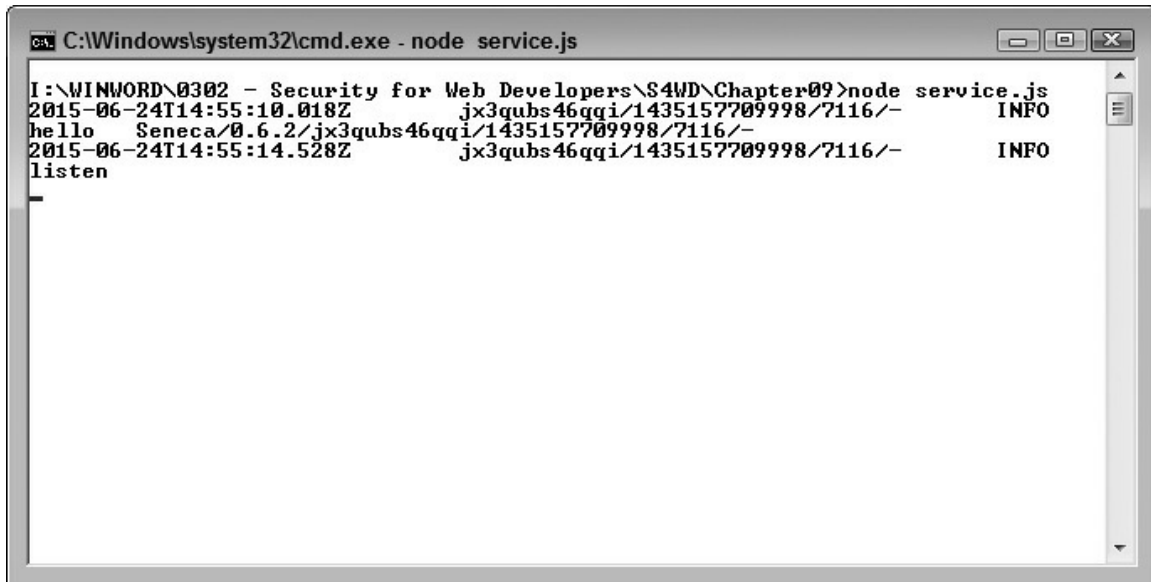
científica ao adicionar um e ou um E. Por exemplo, -123e20 é uma apresentação perfeitamente aceitável de um valor.

Criando um microsserviço usando Node.js e Seneca

Você pode encontrar vários exemplos de uso de Node.js e de Seneca para criar um microsserviço online. Infelizmente, a maioria deles é intrincada e difícil de usar. Alguns estão simplesmente desatualizados. O melhor exemplo está em <http://senecajs.org/>. O código-fonte do servidor funciona exatamente como mostrado. No entanto, um exemplo mais simples ainda é o que está em *service.js*, como mostrado aqui:

```
require('seneca')()
  .add(
    { say:"hello"},
    function( message, done )
    {
      done( null, {message:'hello'} )
    })
  .listen()
```

Nesse exemplo, `require('seneca')` carrega a biblioteca Seneca na memória. O código então adiciona um padrão de correspondência de `{ say:"hello"}` como um objeto JSON. A `function()` associada ao padrão de correspondência gera outro objeto JSON, `{message:'hello'}`. O exemplo usa propositalmente tanto aspas simples quanto aspas duplas ao criar objetos JSON para mostrar que isso é possível, apesar de as especificações oficiais aparentemente não dizerem isso. O último passo é informar o serviço que fique ouvindo com `listen()`. Você pode acrescentar um número de porta à função `listen()`. Se não fornecer um número de porta, o serviço ouvirá na porta default, que é 10101. Para iniciar o serviço, digite `node server.js` e tecle Enter no prompt de comandos. Você verá mensagens de inicialização como mostra a figura 8.1.



```
C:\Windows\system32\cmd.exe - node service.js
I:\WINWORD\0302 - Security for Web Developers\S4WD\Chapter09>node service.js
2015-06-24T14:55:10.018Z      jx3qubs46qqi/1435157709998/7116/-      INFO
hello  Seneca/0.6.2/jx3qubs46qqi/1435157709998/7116/-
2015-06-24T14:55:14.528Z      jx3qubs46qqi/1435157709998/7116/-      INFO
listen
```

Figura 8.1 – O microserviço está ouvindo à espera de requisições.

O processo de inicialização faz log de dois passos. O primeiro é o processo de inicialização do Seneca (em que o Seneca diz “hello” na terceira linha da saída na figura 8.1). O segundo é colocar o microserviço em modo de escuta (como mostra a quinta linha). Sempre que o microserviço fizer uma chamada ou executar outra tarefa (que não seja uma simples saída), você verá uma ou mais entradas de log serem adicionadas à janela. Do ponto de vista de segurança, isso possibilita monitorar a funcionalidade do microserviço e identificar se alguém está tentando fazer algo indesejado com ele.

É claro que você vai querer testar o microserviço. Abra uma janela de seu navegador e digite `http://localhost:10101/act?say=hello` como endereço. O microserviço exibe um objeto JSON simples, como mostra a figura 8.2.

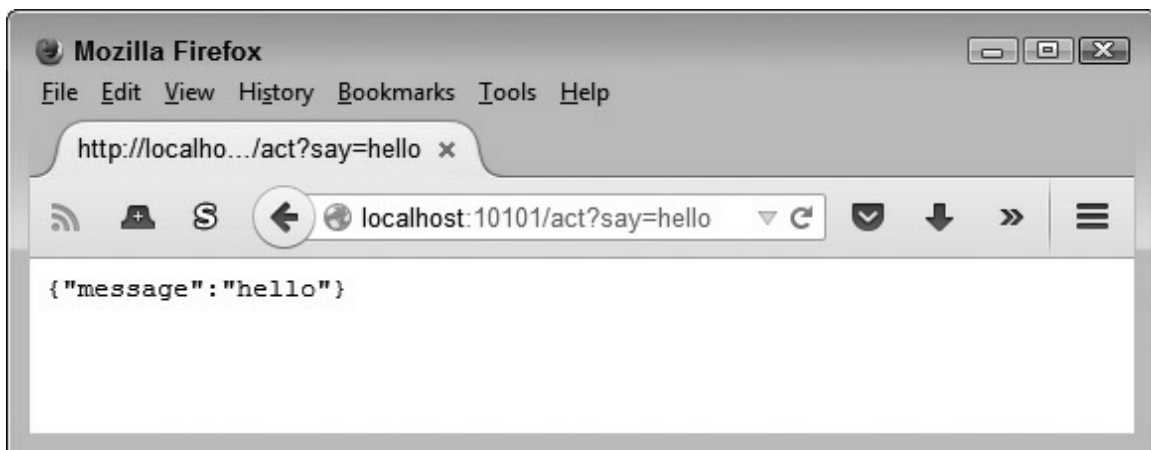


Figura 8.2 – Esse exemplo simples gera um objeto JSON.

Ao olhar novamente para a janela do console, você não verá nada. Isso ocorre porque a função gera um objeto JSON simples e não fez nenhuma chamada para fora do ambiente. No entanto, experimente digitar `http://localhost:10101/act?say=goodbye` como uma requisição. Agora você verá alguma atividade na janela do console, como mostra a figura 8.3.



```
C:\Windows\system32\cmd.exe - node service.js
I:\WINWORD\0302 - Security for Web Developers\S4WD\Chapter09>node service.js
2015-06-24T15:04:59.822Z    dsgugo7qgn2s/1435158299802/5168/-    INFO
hello    Seneca/0.6.2/dsgugo7qgn2s/1435158299802/5168/-
2015-06-24T15:05:02.902Z    dsgugo7qgn2s/1435158299802/5168/-    INFO
listen
2015-06-24T15:05:09.902Z    dsgugo7qgn2s/1435158299802/5168/-    WARN
act
- seneca: No matching action pattern found
for { say: 'goodbye' }, and no default result provided (using a default$ proper
ty).    act_not_found    {args={ say: 'goodbye' },plugin={}}    Error: seneca: N
o matching action pattern found for { say: 'goodbye' }, and no default result pr
ovided (using a default$ property).
    at errormaker (I:\WINWORD\0302 - Security for Web Developers\S4WD\Chapter09\
node_modules\seneca\node_modules\eraro\eraro.js:94:15)
    at Seneca.api_act (I:\WINWORD\0302 - Security for Web Developers\S4WD\Chapte
r09\node_modules\seneca\seneca.js:1228:15)
    at Seneca.delegate.act (I:\WINWORD\0302 - Security for Web Developers\S4WD\C
hapter09\node_modules\seneca\seneca.js:1956:11)
    at Seneca.delegate.act (I:\WINWORD\0302 - Security for Web Developers\S4WD\C
hapter09\node_modules\seneca\seneca.js:1956:11)
    at Object.handle_request (I:\WINWORD\0302 - Security for Web Developers\S4WD
\Chapter09\node_modules\seneca\node_modules\seneca-transport\lib\transport-util
s.js:183:16)
    at Object.handle (I:\WINWORD\0302 - Security for Web Developers\S4WD\Chapter
09\node_modules\seneca\node_modules\seneca-transport\transport.js:451:10)
    at next (I:\WINWORD\0302 - Security for Web Developers\S4WD\Chapter09\node_m
odules\seneca\node_modules\seneca-transport\node_modules\connect\lib\proto.js:14
9:15)
    at IncomingMessage.<anonymous> (I:\WINWORD\0302 - Security for Web Develop
ers\S4WD\Chapter09\node_modules\seneca\node_modules\seneca-transport\transport.js:
407:11)
    at IncomingMessage.emit (events.js:104:17)
    at _stream_readable.js:908:16
```

Figura 8.3 – Erros geram uma saída textual extensa.

A saída inclui uma stack trace, que você pode ignorar nesse caso, mas poderia ser útil quando você estiver trabalhando com microsserviços complexos. A informação mais importante aparece no início, nesse caso. Você pode ver um aviso informando que não há nenhum padrão que corresponda a `{ say: 'goodbye' }`. Observe que a requisição REST é traduzida em um objeto JSON. A saída com erro informa exatamente o que aconteceu, portanto é mais difícil para alguém passar despercebido com uma requisição inválida.

Na verdade, esse é um bom exemplo de experimentação porque você pode ver o resultado de uma tentativa de enganar a parte da comunicação REST da

funcionalidade do microserviço, sem se preocupar com outros elementos encobrindo os resultados. Quando acabar de trabalhar com o exemplo, tecle Ctrl-C ou Ctrl-Break para interromper o serviço. O serviço parará e você verá o prompt de comandos reaparecer.

Definindo as ameaças de segurança causadas por microserviços

Em muitos aspectos, os microserviços espelham as APIs quando se trata de preocupações com segurança. Por exemplo, é possível que um microserviço possa sofrer um ataque man-in-the-middle. A capacidade de um hacker verdadeiramente se beneficiar de um ataque como esse é menor do que no caso de uma API porque um microserviço é pequeno, autocontido e executa apenas uma tarefa. No entanto, a ameaça continua presente e um hacker realmente só precisa de uma via de ataque para arruinar seu dia. Embora você possa ver todo tipo de artigos falando sobre a segurança natural oferecida pelos microserviços, eles têm problemas de segurança e você deve conhecê-los também. As próximas seções apresentam uma mistura de vantagens e problemas que você deve considerar quando se trata de segurança de microserviços.

Falta de consistência

A principal ameaça em potencial representada pelos microserviços é a falta de consistência, que parece assombrar praticamente toda biblioteca e API já criada. Os desenvolvedores de bibliotecas e APIs começam com a simples ideia de criar uma interface consistente e fácil de usar, mas, com o tempo, a biblioteca ou a API tornam-se um emaranhado de estratégias conflitantes que deixam um nó górdio fácil de desfazer, em comparação. Tentar corrigir qualquer base de código é incrivelmente difícil porque os desenvolvedores usam bibliotecas e APIs como um código único. Essas inconsistências provocam problemas de segurança porque os desenvolvedores que usam as bases de código acham que o código deve funcionar de uma maneira, quando, na verdade, eles funcionam de outra. O resultado é uma incompatibilidade entre a base de código e a aplicação que depende dela. Hackers aproveitam

esses erros como um meio para ter acesso à aplicação, seus dados ou ao sistema em que ela executa.

Os microsserviços também podem sofrer de falta de consistência. É essencial que você crie um template para descrever exatamente como chamar os microsserviços o mais cedo possível no processo de desenvolvimento. Assim como no caso das bibliotecas e das APIs, os hackers poderiam usar as inconsistências como meio de passar por cima de qualquer sistema de segurança que você tenha implantado. De modo diferente de bibliotecas e APIs, a inconsistência afetaria apenas um microsserviço, em vez de afetar toda a base de código. Corrigir o microsserviço também seria mais simples porque você estará olhando apenas para uma chamada, e não para uma API completa. Publicar um novo microsserviço também é mais fácil que publicar toda uma nova biblioteca ou API. Consequentemente, superar uma inconsistência de microsserviço é relativamente fácil.

Considerando o papel da máquina virtual

Cada microsserviço normalmente executa em seu próprio ambiente de máquina virtual (VM). Isso quer dizer que um microsserviço normalmente não pode corromper outro. Mesmo que um hacker consiga ter acesso a um microsserviço, o volume de danos que o hacker pode causar normalmente é mínimo. Contudo, é bem possível executar vários microsserviços na mesma máquina virtual – nesse caso, seria possível a um hacker experimentar várias técnicas para ter acesso a toda a API. Para maximizar a segurança, evite empilhar microsserviços, como mostra o exemplo *service2.js* a seguir (para acessar esse exemplo, você deve usar a porta 999, por exemplo, <http://localhost:999/act?say=goodbye>):

```
require('seneca')()
  .add(
    { say:"hello"},
    function( message, done )
    {
      done( null, {message:'hello'} )
    }
  )
  .add(
```

```
{ say:"goodbye"},  
function( message, done )  
{  
  done( null, {message:'goodbye'} )  
})  
.listen(999)
```

A melhor prática é executar cada serviço em uma máquina virtual separada a fim de garantir que cada serviço tenha seu próprio espaço de endereçamento e seu próprio processo. Separar cada microserviço oferece menos oportunidades para erros e também para problemas resultantes de truques de código que podem ser empregados pelos hackers.

Usando JSON para transferências de dados

Não há metodologias perfeitas para a transferência de dados. Sim, JSON é leve, fácil de usar e menos suscetível a problemas de segurança que outras tecnologias como XML. Porém, os hackers ainda têm uma série de métodos para causar problemas com JSON. As próximas seções descrevem os problemas predominantes.

Considerando os perigos de eval()

É possível que alguém possa enviar uma resposta ao cliente contendo uma tag `<script>` com um script que poderia fazer praticamente de tudo (em alguns casos, você nem precisa da tag – basta passar o script). Felizmente, a maioria dos navegadores atualmente detecta essas tentativas e se recusa a transferir as informações. Por exemplo, experimente acessar `http://localhost:999/act?<script>alert('Hello'); </script>=goodbye` e você poderá ver algo como a saída exibida na figura 8.4. Além disso, o próprio microserviço recusou-se a processar a requisição. No entanto, esse script em particular é simplista por natureza – uma tentativa mais sofisticada poderia ter sido bem-sucedida.



Figura 8.4 – Problemas de scripting podem infestar os microsserviços, como fazem com APIs.

Defendendo-se contra CSRF

Um CSRF ou XSRF (Cross-site Request Forgery) é uma tentativa de um invasor fazer um usuário executar código sem querer ou sem saber. O código executa com o nível de privilégio e as credenciais do usuário, portanto tem-se a impressão que o usuário executou o código, e não outra pessoa. Na maioria dos casos, esse ataque em particular funciona contra microsserviços quando a série a seguir de eventos ocorre:

1. Um usuário faz login em uma aplicação que depende de microsserviços.
2. O usuário executa várias tarefas, em que cada uma depende de REST para comunicação em background.
3. Um invasor envia ao usuário um URL especialmente formatado para parecer exatamente como as outras mensagens REST, mas que faz algo que o invasor quer que o usuário faça. O URL poderia aparecer como parte de uma mensagem de email ou de outra comunicação, ou até mesmo como um link em um site.
4. O usuário inicia a requisição enviando o URL para o microsserviço, como faria normalmente.
5. A requisição maliciosa executa e o usuário pode nem mesmo perceber que ela ocorreu.

Um exemplo do mundo real desse exploit ocorreu em 2008 como um exploit do



µTorrent. O motivo desse exploit em particular ser tão importante para os desenvolvedores de microsserviços é que ele funciona em um sistema em que um servidor web executa em background. Quando o exploit ocorre, ele compromete todo o sistema, e é por isso que você deve manter esses microsserviços separados em suas próprias VMs. Você pode ler sobre ele em <http://xssniper.com/blog/2008/04/21/csrf-pwns-your-box/>.

Por causa do modo como esse exploit funciona, o que você realmente precisa fazer é garantir que a aplicação faça logout do usuário automaticamente após um período de inatividade. Além disso, você deve procurar por padrões incomuns de uso e definir limites para o que um usuário pode fazer sem aprovação. Por exemplo, um usuário pode transferir mil dólares em um banco, mas transferir dez mil dólares exige a aprovação de um gerente. Usar fluxos de trabalho de vários passos, em que um invasor precisaria interagir mais com o usuário para fazer o exploit funcionar, às vezes pode evitar esse tipo de exploit também, mas então você enfrentará a ira do usuário por ter de executar mais passos a fim de concluir uma dada tarefa.

Definindo a TLS

O calcanhar de Aquiles tanto de microsserviços quanto de APIs é que ambas funcionam enviando mensagens de um lado para outro, em vez de executar processamentos em um único computador. A pedra angular do desenvolvimento de uma correção para esse problema é a TLS (Transport Layer Security, ou Segurança da Camada de Transporte). É essencial garantir que a camada de transporte entre clientes e servidores permaneça segura durante o processo de troca de mensagens. Isso significa usar tecnologias como HTTPS e REST para garantir que as comunicações permaneçam tão seguras quanto possível.

Um problema com o encapsulamento de absolutamente todas as chamadas e respostas em HTTPS e REST é que a aplicação pode ficar extremamente lenta. O melhor método para superar esse problema é contar com balanceamento de carga para terminar comunicações de clientes, ao mesmo tempo em que um canal permanece aberto para atender às necessidades de processamento do backend. Manter o canal de processamento do backend aberto reduz o overhead e ajuda a reduzir os efeitos do uso de HTTPS e de

REST.



Um dos problemas com o uso de HTTPS com redes voltadas ao público é que você deve ter um certificado de um CA (Certificate Authority, ou Autoridade de Certificação) – uma proposta cara que pode evitar que algumas empresas usem HTTPS. Quando você controla os dois lados do canal de comunicação, é possível criar seu próprio certificado para atingir o mesmo objetivo a um custo significativamente reduzido.

O uso de HTTPS e de TLS bidirecional garante que tanto o cliente quanto o servidor definam a identidade um do outro durante cada ciclo de requisição/resposta. A autenticação reduz as chances de alguém poder implementar um ataque man-in-the-middle com sucesso a fim de ter acesso não autorizado aos dados. A maioria das comunicações atualmente ocorre com TLS unidirecional, em que o cliente confere a identidade do servidor, mas o servidor simplesmente pressupõe que o cliente não está comprometido. Dada a natureza da comunicação dos microsserviços, você realmente precisa implementar TLS bidirecional para conferir a identidade tanto do cliente quanto do servidor.

Criando paths alternativos para microsserviços

Um aspecto que muitos desenvolvedores terão problema para compreender é a natureza descentralizada dos microsserviços. Cada microsserviço é separado dos demais. Você não precisa pensar na plataforma de que um microsserviço precisa, qual linguagem ele usa ou em que lugar ele está fisicamente. É possível ter dois microsserviços escritos em duas linguagens diferentes, em duas plataformas diferentes, em dois lugares diferentes, executando a mesma tarefa. Como os ambientes usados pelos dois microsserviços são muito diferentes, é improvável que um problema que afete um microsserviço vá também afetar o outro. Consequentemente, é uma boa ideia manter ambos ao alcance para que você possa alternar entre eles conforme for necessário de modo a manter sua aplicação executando. É disso que trata esta seção – considerar as implicações de ter vários paths para acessar múltiplos microsserviços.

Quando pensar em microsserviços e nos paths que eles empregam, considere também outros aspectos, como portas. Você pode criar microsserviços que

funcionem em portas diferentes. Normalmente, você pode contar com o microsserviço na porta 999 para fazer o trabalho necessário a uma aplicação. No entanto, se o microsserviço na porta 999 ficar sobrecarregado, for comprometido ou simplesmente não funcionar, você pode alternar para o mesmo microsserviço em uma porta diferente. Seu código permanece igual – somente a porta muda. Usar essa abordagem confere resiliência, confiabilidade e flexibilidade à sua aplicação. Também significa que você tem opções caso algo como um ataque DDOS (Distributed Denial-of-Service attack, ou Ataque Distribuído de Negação de Serviço) ocorra.

PARTE III

Criando estratégias de testes úteis e eficientes

É improvável que você escape de problemas de segurança em potencial com sua aplicação, mas você pode reduzi-los desde que invista tempo pensando e executando técnicas apropriadas de teste. Esta parte do livro é toda dedicada a ajudar você a reduzir o risco de que alguém ache sua aplicação convidativa o suficiente para tentar invadi-la. O capítulo 9 começa ajudando-o a pensar como um hacker, o que é um exercício importante se você realmente quiser perceber as brechas de segurança de sua aplicação. Não é como se alguém marcasse as brechas de segurança em vermelho com um sinal enorme que diz: “Corrija-me”, portanto esse processo de raciocínio é extremamente importante.

O capítulo 10 discute técnicas de sandboxing. Manter o código em uma sandbox não o obriga a se comportar, mas reduz os danos que ele pode causar. As sandboxes deixam seu sistema e seus recursos muito menos acessíveis e podem fazer toda a diferença quando se trata de segurança.

Os capítulos 11 e 12 discutem testes de vários tipos. O capítulo 11 concentra-se em testes internos (in-house), que muitas vezes ajudam a localizar os principais problemas, mas podem não ser suficientes para encontrar problemas sutis que poderão ser custosos mais tarde. O capítulo 12 discute testes de terceiros – uma opção especialmente importante para negócios de pequeno porte, que podem não contar com um especialista em segurança. Independentemente de como você realiza os testes, garantir que eles sejam completos é essencial para saber quais são os riscos de usar um produto em particular com sua aplicação.

CAPÍTULO 9

Pensando como um hacker

A maioria dos desenvolvedores passa seu tempo em um mundo em que é importante considerar como tudo funciona (ou seja, eles focam em como tudo funcionará quando o código estiver correto). Toda a ideia de pensar em algo não funcionando (por exemplo, tentar determinar maneiras em que algo poderia dar errado quando o código está incorreto), de certo modo, é estranha. Sim, desenvolvedores lidam com bugs o tempo todo, mas a linha de raciocínio é diferente. Quando você pensa como um hacker, você pode usar um código que é perfeitamente aceitável como está escrito – o código pode não ter um bug, mas pode ter uma brecha de segurança.

Este capítulo contém um processo que ajuda você a ver o código como faria um hacker. Você usará ferramentas para procurar brechas de segurança em potencial, criará um sistema de testes para usar enquanto tenta causar falhas no código e contará com brechas comuns para facilitar um pouco mais a sua vida. Hackers amam o fenômeno BYOD (Bring Your Own Device, ou Traga seu Próprio Dispositivo) porque agora você tem todos aqueles sistemas sem segurança por aí usando sistemas operacionais com os quais a área de TI não tem muita experiência em trabalhar. É claro que há sempre aquela pessoa que, em última instância, testa as aplicações: o usuário. Os usuários podem encontrar mais maneiras de causar falhas em aplicações que um desenvolvedor gostaria sequer de pensar, mas os testes de usuário podem ser importantes para identificar aquelas suposições que você fez, e que realmente não eram válidas.

De fato, é a necessidade de pensar nessas linhas que levam muitas empresas a contratar um especialista em segurança para pensar em todas as maneiras ardilosas com que os hackers causarão falhas em aplicações perfeitamente funcionais, em um esforço para obter uma vantagem tangível que possa ser

usada para executar tarefas específicas. Entretanto, este capítulo supõe que você não tem realmente ninguém com a função de especialista em segurança em sua empresa no momento. Você pode considerar isso quando a aplicação estiver próxima de ser concluída, mas então, com frequência, será tarde demais para corrigir problemas importantes sem incorrer em elevados custos. Pensar como um hacker quando observar sua aplicação pode fazer sua empresa economizar dinheiro, tempo, esforços e, acima de tudo, evitar constrangimentos.

Definindo a necessidade de scans de segurança web

A ideia básica por trás de scans de segurança web é que eles informam se seu site está limpo no momento e, às vezes, ajudam você a considerar potenciais brechas de segurança. Se você tem uma instalação de grande porte, comprar um scanner de segurança web é uma boa ideia. No entanto, negócios menores muitas vezes podem fazer uso dos scanners de segurança web gratuitos online. É claro que esses produtos não são totalmente gratuitos. Quando o scanner de segurança web encontra um problema em seu sistema, a empresa proprietária do scanner provavelmente vai querer que você o adquira para limpar seu sistema e protegê-lo depois. (Não há nada gratuito no mundo da informática.)



Nenhum scanner de segurança web é capaz de detectar problemas em potencial em seu site com 100% de precisão. Isso é especialmente verdadeiro para scanners remotos – aqueles que não estão executando realmente em um sistema que tenha o acesso necessário ao servidor que você quer testar. Os scanners fazem um trabalho notável em localizar problemas existentes, mas não podem encontrar tudo.

Você também deve estar ciente de que os scanners de segurança web podem produzir resultados falso-positivos. Isso quer dizer que o scanner pode informar que há um problema de segurança quando, na verdade, ele não existe.

A melhor medida de proteção contra problemas inexistentes ou falso-positivos é usar mais de um scanner de segurança web. Contar com um único scanner de segurança realmente é um bom ponto de partida, mas é provável que você acabe trabalhando mais que o necessário para criar uma instalação totalmente funcional com o mínimo possível de brechas de segurança.

Antes de olhar mais atentamente para os scanners de segurança web, é

importante ver quais tarefas ele executa. Em nosso caso, o exemplo usará o Sucuri (<https://sitecheck.sucuri.net/>) – um scanner gratuito capaz de detectar uma série de problemas de segurança, principalmente em torno de infecções em potencial. A página principal mostrada na figura 9.1 permite inserir o URL de seu site e clicar em Scan Website! para iniciar o processo de scanning.

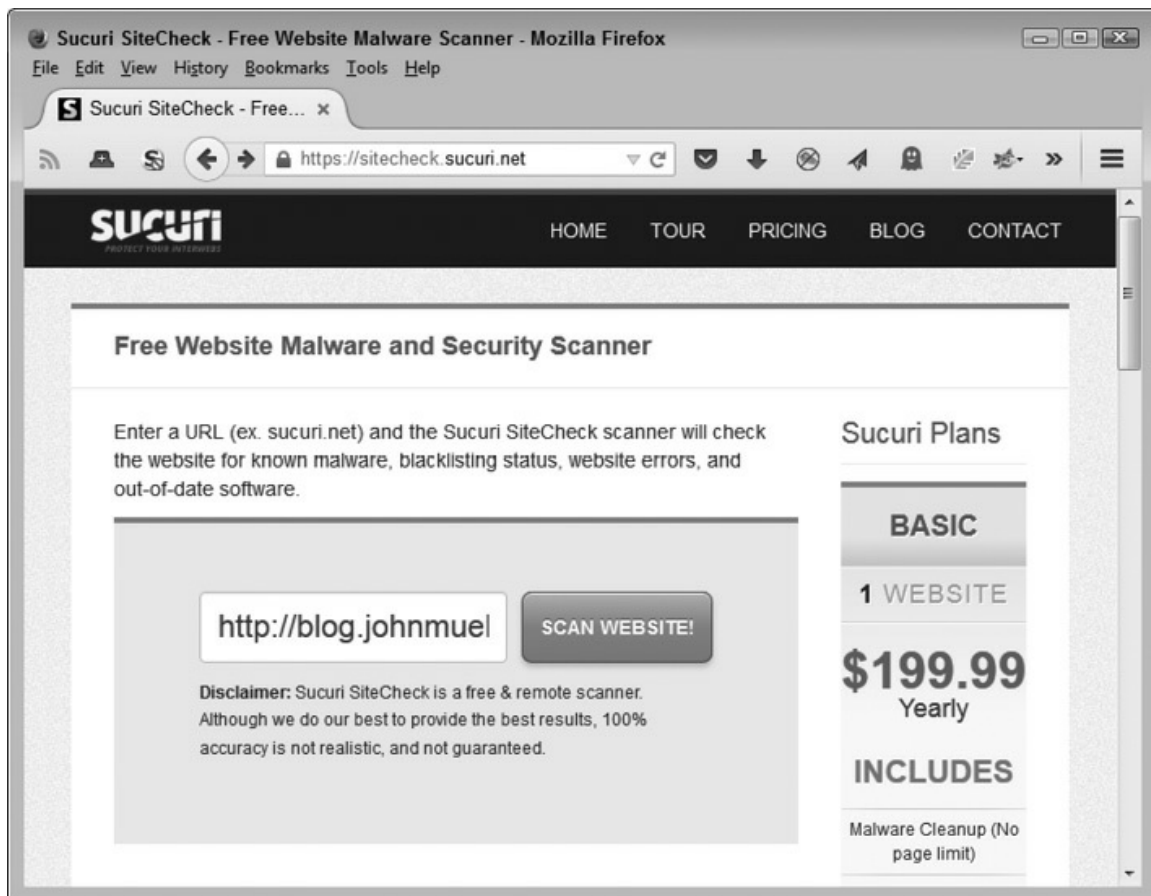


Figura 9.1 – Iniciar o scan envolve fornecer o URL de seu site e clicar em um botão.



Se você tiver um site hospedado, não se esqueça de verificar junto ao provedor de serviços de hospedagem. Muitos deles oferecem scanners de segurança web de baixo custo. Por exemplo, você pode encontrar a oferta para os usuários do GoDaddy em <https://www.godaddy.com/security/malwarescanner.aspx>. O provedor tem um profundo interesse em garantir que seu site não seja comprometido, portanto, a oferta de baixo custo beneficia também o provedor.

Após o scan ter sido concluído, você terá uma visão geral do que o scanner encontrou e uma listagem mais detalhada das verificações propriamente ditas,

como mostra a figura 9.2. É claro que, se você quiser uma verificação mais detalhada, é possível comprar um dos planos do Sucuri listados à direita da página. Contudo, para ver o que um scanner pode fazer por você, a versão gratuita servirá.

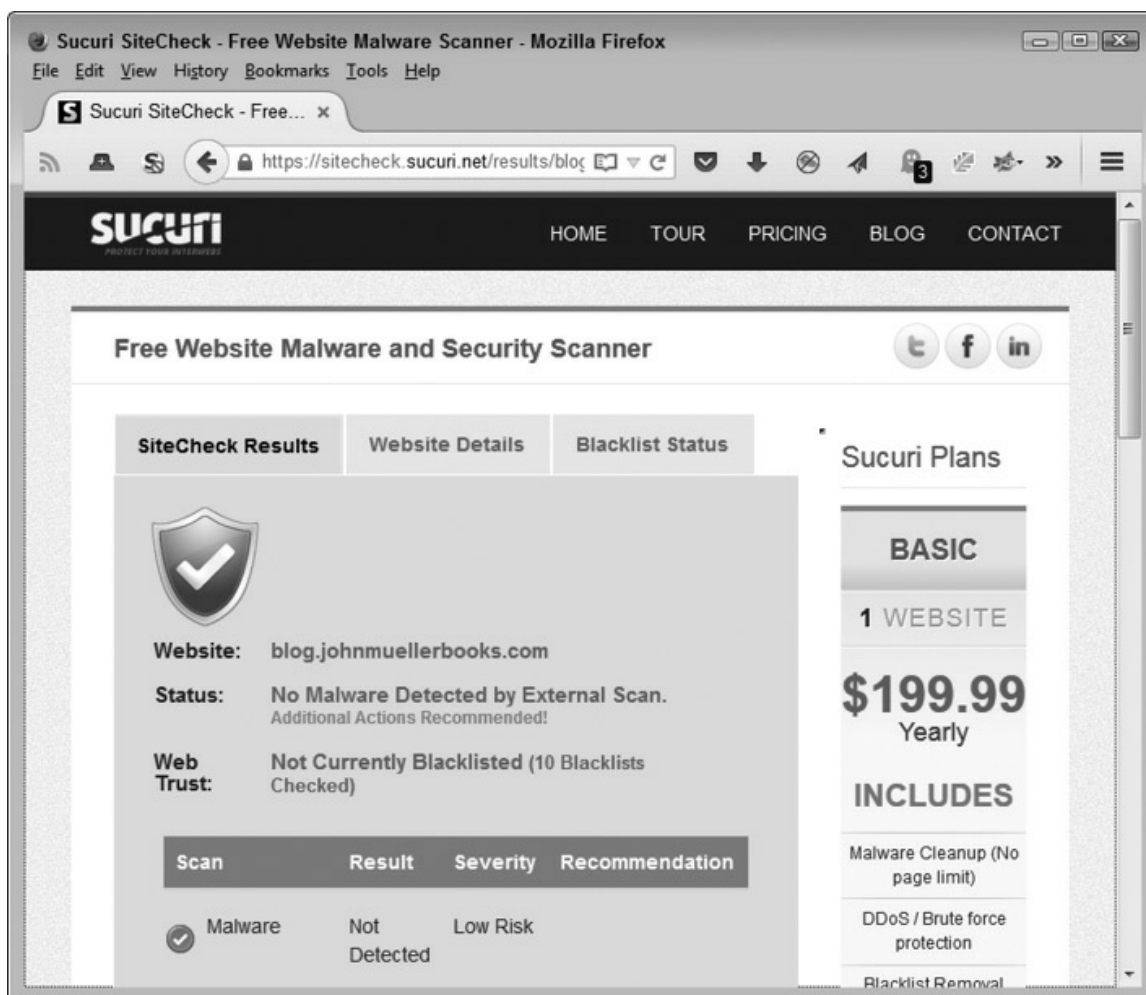


Figura 9.2 – O scan completo mostra qualquer problema com seu site.

Pode ser interessante ver as informações de detalhes do website (Website Details). A figura 9.3 exibe as informações de meu site. O interessante é que um hacker poderia empregar um scanner como esse para começar a procurar áreas de interesse em potencial em seu site, por exemplo, qual versão de um software em particular você usa. O scanner de segurança web ajuda você a entender os tipos de informação que os hackers têm disponíveis. Ver esse tipo de informação que todos podem encontrar muitas vezes é um pouco assustador.

As informações detalhadas podem ser úteis por outro motivo. Quando usar um site hospedado, como muitos negócios de pequeno porte fazem, é difícil saber se o provedor dos serviços de hospedagem realmente mantém o software atualizado. O scanner de segurança web Sucuri (e outros) sonda o site cuidadosamente e gera listas de informações de versões que você nem sequer veria ao usar o painel de controle (dashboard) de seu site. Por exemplo, é possível que uma biblioteca continue funcionando bem, mas esteja desatualizada e tenha o potencial para criar uma brecha de segurança em seu sistema. Um telefonema para a empresa de hospedagem normalmente pode esclarecer rapidamente a questão, mas você precisa conhecer o problema antes de poder ligar para alguém e falar sobre ele.

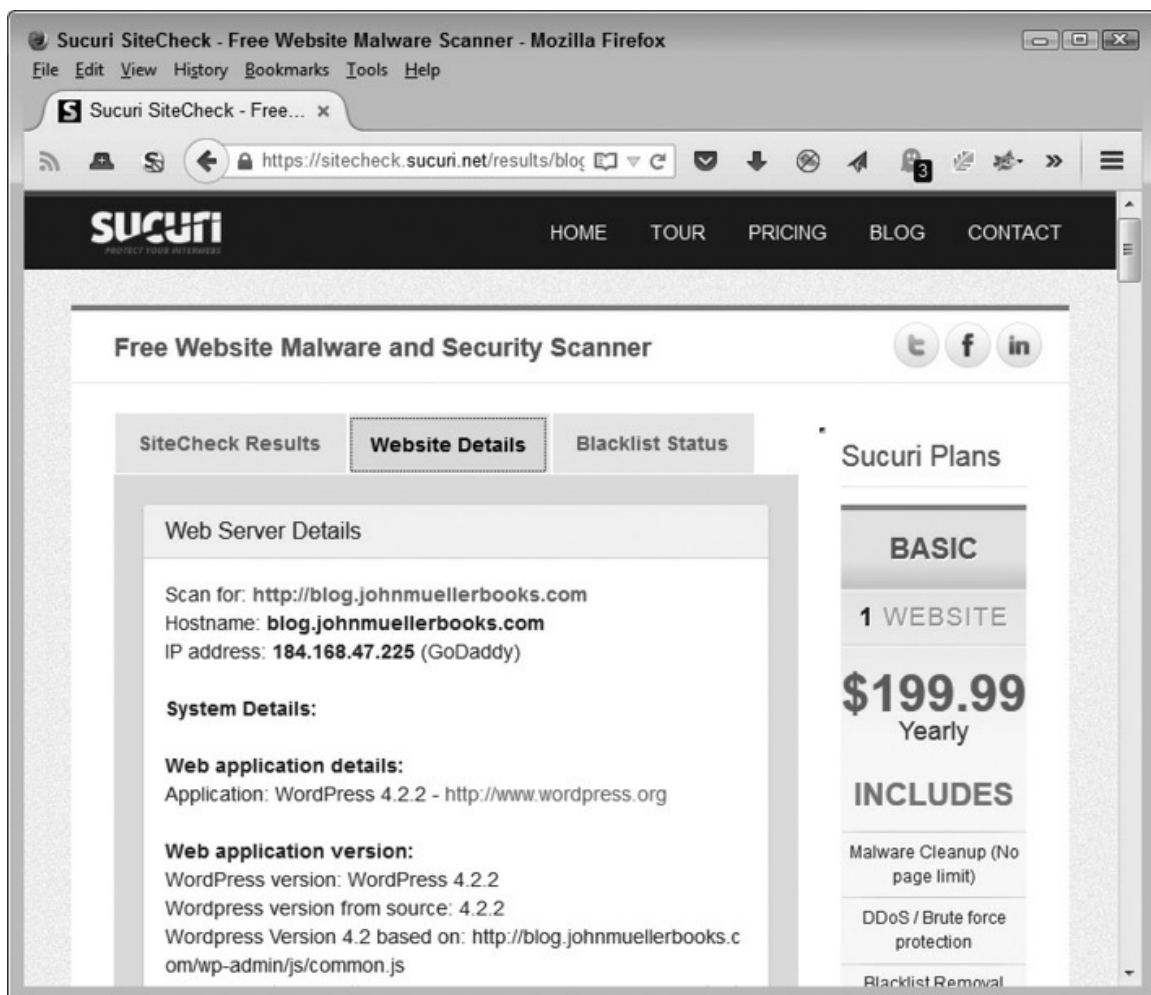


Figura 9.3 – As informações detalhadas podem oferecer insights sobre o que os hackers podem aprender sobre sua configuração.



É possível encontrar listas de scanners de segurança web online. Uma das melhores listas é a Application Security Scanner List (<http://projects.webappsec.org/w/page/13246988/Web>) do Web Application Security Consortium. Os autores classificaram a lista em comercial, SAAS (Software-As-A-Service, ou Software como Serviço) e ferramentas gratuitas de código aberto. Outra boa lista de ferramentas está no OWASP (Open Web Application Security Project, ou Projeto Aberto para Segurança em Aplicações Web, <https://www.owasp.org/index.php/Phoenix/Tools>). Você pode encontrar ferramentas para todo tipo de finalidade nesse site.

Criando um sistema de testes

Quando pensar como um hacker, com frequência é uma boa ideia ter um sistema que você realmente não se preocupe em arruinar. De fato, é mais do que provável que o sistema seja arruinado em algum ponto do caminho, portanto é importante criar um sistema desse tipo com a facilidade de restauração em mente. As próximas seções apresentam uma boa visão geral dos aspectos que você deve considerar quando criar uma instalação de testes para sua empresa.

Considerando os usos dos sistemas de testes

Você precisa de um sistema de testes para poder se sentir à vontade em explorar todas as possíveis vias para hacking. Até entender completamente quão vulnerável é a maioria das aplicações atualmente, você não poderá realmente corrigir sua própria aplicação. Os desenvolvedores cometem todo tipo de erros que parecem inocentes na ocasião, mas que provocam problemas de segurança posteriormente. Algo tão pequeno como o tipo errado de comentário em seu código pode provocar muitos problemas. Usar um sistema de produção para fazer experimentos é simplesmente uma péssima ideia porque as técnicas descritas neste capítulo poderiam criar aberturas para um ataque de verdade. Use um sistema que você possa arruinar e restaurar conforme for necessário, para garantir que as lições sejam realmente aprendidas.

A ideia é testar cada aplicação em condições adversas, mas de forma segura, que não comprometa realmente nenhum dado de verdade ou um sistema de produção. Dar um vírus a um sistema de testes dirá como sua aplicação

reagirá a esse vírus sem verdadeiramente testar o vírus em um ambiente de produção. É claro que seus sistemas de teste não podem estar conectados à sua rede de produção (senão o vírus poderia sair e infectar sistemas de produção). Você também pode usar esse ambiente para testar vários exploits e determinar o que é preciso para causar falhas em sua aplicação como faria um hacker.

Porém, um sistema de testes não precisa testar apenas a sua aplicação. Você também pode usá-lo para testar a eficiência de medidas de proteção ou as vulnerabilidades de configurações específicas. Testar o sistema como um todo ajuda a garantir que sua aplicação não quebre facilmente por causa de vulnerabilidades em outras partes da configuração do sistema. É essencial testar todo o ambiente porque você pode estar certo que qualquer hacker faria isso.

Obtendo o treinamento necessário

Antes de um desenvolvedor poder realmente testar algo, é importante saber o que deve ser testado e entender como o teste deve ser feito. Você pode iniciar o processo de treinamento usando o OWASP Security Shepherd (https://www.owasp.org/index.php/OWASP_Security_Shepherd), que mostra os dez principais riscos de segurança às aplicações. O Security Shepherd pode oferecer instruções a apenas uma pessoa ou você pode usá-lo em um ambiente de sala de aula para ensinar vários alunos de uma só vez. A aplicação oferece um ambiente competitivo, portanto, em alguns aspectos, é um jogo de segurança, em que a aplicação mantém uma pontuação com o resultado de cada sessão. A aplicação tem suporte para mais de 60 níveis e você pode configurá-la para que leve em consideração a curva de aprendizado do usuário e sua experiência.

Depois de aprender o básico, é preciso investir tempo invadindo o software como um hacker faria para que você possa ver o outro lado do ambiente de segurança. É claro que você deve executar essa tarefa em um ambiente seguro e permitido, o que significa não invadir o seu próprio software. O OWASP projetou especificamente a aplicação WebGoat (https://www.owasp.org/index.php/Category:OWASP_WebGoat_Project)

para oferecer vulnerabilidades que um desenvolvedor de aplicação pode usar para descobrir como os exploits funcionam em uma instalação com uma aplicação de verdade. Os vídeos em <http://webappsecmovies.sourceforge.net/webgoat/> conduzem você passo a passo no processo de hacking para que você possa entender melhor o que os hackers fazem com sua aplicação. O treinamento inclui vídeos sobre:

- princípios gerais;
- qualidade do código;
- concorrência;
- parâmetros não validados;
- falhas no controle de acesso;
- falhas de autenticação;
- falhas no gerenciamento de sessões;
- Cross-site scripting (XSS);
- buffer overflows (transbordamentos de buffer);
- falhas de injeção;
- armazenamento sem segurança;
- Denial of Service (DOS, ou Negação de serviço);
- configuração;
- web services;
- segurança com AJAX.

O último passo é um desafio do WebGoat em que você demonstra suas novas habilidades quebrando o esquema de autenticação, roubando todos os cartões de crédito e então desfigurando o site. A questão principal é entender completamente o hacking do ponto de vista de um hacker, mas em um ambiente seguro (e com permissão).

É importante entender como aperfeiçoar suas habilidades de testes de segurança. O artigo em <http://blogs.atlassian.com/2012/01/13-steps-to-learn-perfect-security-testing-in-yourorg/> apresenta 13 passos que você pode seguir para melhorar seus métodos de teste. Essa informação, em conjunto com as

ferramentas desta seção, deve ajudar você a testar qualquer aplicação, mesmo que sua empresa não tenha recursos para realizar testes mais profundos.

Criando o ambiente correto

Qualquer ambiente que você criar deve corresponder ao ambiente da aplicação propriamente dito. Isso significa ter acesso a cada um dos sistemas operacionais aos quais você planeja dar suporte, aos navegadores que os usuários planejam usar e às medidas de proteção que cada sistema empregará (o que pode ser nenhuma, em alguns casos). Certifique-se de que os sistemas sejam fáceis de acessar porque você poderá se ver reconfigurando-os para simular os tipos de sistema que os usuários têm. Além disso, você deve se planejar para reconstruir os sistemas de testes regularmente.



Por causa do tipo de testes que você realiza usando o sistema de testes, você deve prover segurança física para seu laboratório. Caso contrário, alguns indivíduos mal-intencionados poderiam deixar um vírus, ou outro código sórdido, livre no ambiente de produção. Você também não deve permitir acesso à instalação de segurança a funcionários descontentes ou a outros que possam usá-la para deixar sua vida mais interessante. O ambiente de testes deve refletir seu ambiente de produção do modo mais semelhante possível, mas você também deve mantê-lo fisicamente separado; do contrário, terá de enfrentar as consequências.

Como parte de manter os ambientes de produção e de testes separados, considere o uso de um cabeamento de cor diferente no ambiente de testes. Você não vai querer conectar um sistema de produção ao ambiente de testes acidentalmente ou vice-versa. Além do mais, é essencial identificar os sistemas de testes com etiquetas para que ninguém possa usá-los no ambiente de produção.

Usando máquinas virtuais

A maioria das empresas não terá hardware suficiente para executar uma única máquina para cada configuração de sistema operacional e de navegador com que se espera que o usuário vá contar quando estiver trabalhando com a aplicação. A solução é usar máquinas virtuais para que você possa configurar um único computador físico de modo a hospedar vários computadores virtuais. Cada um dos computadores virtuais representará uma única configuração de usuário para testes.

O uso de máquinas virtuais é uma boa ideia por outro motivo. Quando o

sistema virtual em algum momento sucumbir aos ataques que você fizer, bastará simplesmente parar a máquina virtual, apagar o arquivo que armazena esse computador e criar uma nova cópia a partir de uma configuração de base armazenada no disco rígido. Em vez de horas para configurar um novo sistema de testes, você pode criar uma nova instalação em apenas alguns minutos.

Máquinas virtuais podem resolver muitos problemas com sua configuração, mas você também precisa considerar a necessidade de um hardware mais sofisticado para fazê-las funcionar de forma apropriada. Um sistema sem capacidade não produzirá o tipo de resultado necessário para avaliar problemas de segurança. O número de sistemas que você pode criar virtualmente em uma máquina física é limitado pela quantidade de memória, pelos ciclos de processamento e por outros recursos que o sistema deve dedicar ao processo.

Também é essencial que o software de operação das máquinas virtuais ofereça suporte para todas as plataformas que você deseja testar. Produtos como o VMWare (<http://partnerweb.vmware.com/GOSIG/home.html>) oferecem suporte para a maioria dos principais sistemas operacionais. É claro que esse suporte tem um custo adicional.

Como alternativa ao uso de máquinas virtuais, algumas empresas usam sistemas mais antigos que continuam viáveis, mas que não são tão úteis quanto eram no ambiente de produção. Por causa do modo como as aplicações web funcionam, esses sistemas antigos normalmente oferecem toda a capacidade de processamento necessária e permitem que a empresa continue a tirar algum proveito de uma compra mais antiga. É claro que manter esses sistemas antigos também implica em custo, portanto é preciso pesar a diferença de custo entre uma configuração de máquina virtual e o uso de sistemas mais antigos (ou talvez criar uma combinação dos dois).

Obtendo as ferramentas

A menos que você queira escrever suas próprias aplicações para ferramentas de segurança (definitivamente, é uma péssima ideia), você precisará obtê-las de alguém. Felizmente, sites como a McAfee

(<http://www.mcafee.com/us/downloads/free-tools/index.aspx>) oferecem todo tipo de ferramentas gratuitas que você possa querer para executar tarefas como:

- detectar malwares no sistema host;
- avaliar se o sistema é vulnerável a ataques;
- executar análise forense após um ataque;
- usar as ferramentas SASS (Software Application Security Services, ou Serviços de Segurança de Aplicativos e Software) da Foundstone para deixar as aplicações mais seguras;
- determinar quando uma invasão ocorreu;
- fazer scan do sistema em busca de várias vulnerabilidades;
- estressar o sistema para testá-lo.

Configurando o sistema

Começar com uma configuração limpa é importante para conduzir uma análise forense do sistema após atacá-lo. Antes de instalar um software ou fazer qualquer configuração, certifique-se de que o sistema esteja totalmente zerado (escreva zeros em tudo). Vários produtos de software permitem que você execute essa tarefa. Escrever zeros em todo o disco rígido garante que qualquer dado que você vir será o resultado dos testes executados, e não informações deixadas por uma instalação anterior.

É importante criar uma instalação limpa para cada plataforma à qual você pretenda dar suporte desde o início. Não se esqueça de fazer uma cópia da imagem da configuração para poder restaurá-la posteriormente. A configuração deve incluir o sistema operacional, os navegadores e qualquer software de teste necessário para dar suporte ao ambiente de testes. Talvez você queira também incluir softwares que o usuário normalmente instale no sistema se a aplicação web interagir com o software de alguma maneira.

Produtos como o Norton Ghost (<http://www.symantec.com/page.jsp?id=ghost>) facilitam bastante criar imagens de cada configuração. Certifique-se de que você tenha uma estratégia de criação de imagens em mente antes de fazer algo com a configuração limpa que você criar. Você precisará de imagens limpas depois para



restaurar a configuração após arruiná-la quando atacá-la.

Além dos softwares-padrão, talvez você queira instalar softwares para acesso remoto para poder acessar o sistema de um local remoto, fora da área de testes fisicamente protegida. Acessos externos devem ocorrer por meio de uma rede fisicamente separada para garantir que não haja riscos de contaminar o ambiente de produção. O uso de software para acesso remoto permite que mais de um testador de software acesse os sistemas conforme for necessário a partir de seus locais usuais de trabalho, em vez de acessá-los fisicamente de dentro da área protegida.



Algumas empresas disponibilizam estações de trabalho que acessam os sistemas de testes usando um controlador KVM (Keyboard, Video, and Mouse, ou Teclado, Vídeo e Mouse). Usar uma configuração de KVM com um transmissor permite alternar facilmente entre sistemas de testes a partir de um local remoto. No entanto, o uso de um software de acesso remoto provavelmente é mais flexível e mais rápido.

Restaurando o sistema

Não demorará muito para que seu sistema de testes exija uma restauração. Vírus, adwares¹ e cavalos de Troia (Trojans) que você usa para testá-lo são apenas uma fonte de problemas. Executar exploits e determinar a facilidade com que sua aplicação falha em algum momento causarão danos ao sistema operacional, à aplicação de teste, aos dados do teste e às medidas de proteção do sistema. Em suma, você precisa de algum método para restaurar rapidamente o sistema a um estado conhecido.

A restauração também inclui a reconfiguração do sistema para simular outro ambiente. Vale a pena usar imagens para criar vários ambientes de teste prontamente. Conforme mencionamos, usar máquinas virtuais economiza bastante tempo porque você não precisa recriar sempre o disco rígido do zero. No entanto, certifique-se de que você tenha também a imagem de cada sistema operacional em um disco rígido diferente, em um DVD ou em outra

mídia de armazenagem porque o disco rígido em algum momento será corrompido.

Definindo as fontes mais comuns de brechas de segurança

Todo dia vemos o aumento de novas brechas de segurança. É improvável que alguém possa acompanhar todas elas. Entretanto, algumas dessas brechas exigem atenção especial, enquanto outras são do tipo que você poderá ver no futuro. É claro que é interessante ter um método organizado para localizar brechas de segurança e a lista de verificação (checklist) oferecida pelo OWASP

(https://www.owasp.org/index.php/Web_Application_Security_Testing_Cheat) é um início na direção correta.

Depois que conhecer os potenciais vetores de ataque à sua aplicação, é conveniente classificá-los para saber quais vulnerabilidades devem ser corrigidas primeiro. Um ataque que permita a alguém acessar todas as fotos do piquenique da empresa tem prioridade muito menor que um ataque que permita o acesso a informações de cartões de crédito de todos os seus clientes. Um dos melhores sistemas que você pode usar para classificar potenciais vetores de ataque é o CVSS (Common Vulnerability Scoring System, ou Sistema de Pontuação de Vulnerabilidades Comuns, em <http://www.first.org/cvss/v2/faq>). O uso desse sistema ajuda a criar uma lista organizada de problemas com os quais você pode lidar para deixar sua aplicação mais segura.

É claro que se você souber o que deve testar também ajuda. Com isso em mente, as próximas seções descrevem as fontes mais comuns de brechas de segurança atualmente (quando este livro foi escrito). (Essas seções são baseadas em informações que você já viu sobre hacks básicos no capítulo 1.) É provável que você encontre mais brechas quando o livro estiver publicado, pois os hackers são, acima de tudo, criativos em seus métodos para causar falhas em softwares.

Não é possível classificar todas as possíveis fontes de brechas de segurança que os hackers usarão. Na verdade, os hackers muitas vezes contam com um desvio



de atenção (de modo muito semelhante aos mágicos) para evitar que você descubra o que está acontecendo. Uma notícia recente sobre a companhia aérea polonesa LOT

(<http://www.computerworld.com/article/2938486/security/cyberattack-grounds-planes-in-poland.html>) serve para enfatizar o resultado de um desvio de atenção. Nesse caso, as autoridades gastaram tempo e recursos consideráveis para que sistemas de voo não fossem afetados por ataques em potencial. No entanto, eles não foram tão cuidadosos com os sistemas em terra. Hackers conseguiram invadir os sistemas em terra e fizeram com que fosse impossível criar planos de voo para passageiros de saída. Se os ataques mantiveram as aeronaves presas por afetarem seus sistemas de voo ou porque afetaram o controle em terra não importa. O que importa é que os aviões não podiam decolar. Os hackers conseguiram um resultado desejado desviando a atenção para outro lugar. A lição é que você deve olhar para todos os lados – e não apenas para os lugares em que você acha que os hackers atacarão com base nas estatísticas mais recentes.

Evitando ataques de injeção de SQL

Existem muitas formas de ataques de injeção de SQL. Por exemplo, você poderia criar confusão com dados de formulário em uma aplicação para determinar se o formulário é capaz de enviar comandos ao servidor no backend. Entretanto, é melhor começar com um método popular para verificar o potencial para um ataque de injeção de SQL.

Vamos supor que você tenha um URL como *<http://www.mysite.com/index.php?itemid=10>*. Você provavelmente já deve ter visto URLs estruturados assim em muitos sites. Uma maneira de verificar se há uma vulnerabilidade é simplesmente adicionar uma aspa simples após o URL, transformando-o em *<http://www.mysite.com/index.php?itemid=10'>*. Ao teclar Enter, o site enviará uma mensagem de erro de SQL de volta. A mensagem varia conforme o sistema, mas a ideia é que um servidor no backend receba seu URL como uma requisição SQL formatada de modo semelhante a: `SELECT * WHERE itemid='10'`. O acréscimo de outra aspa simples deixa a query inválida, o que gera o erro.

Quando encontrar um problema em sua aplicação que alguém poderia potencialmente explorar, é importante deixar toda a equipe de desenvolvimento ciente disso. Fazer uma demonstração da exploração de falha para que todos possam ver como ela funciona e o que você precisa fazer para corrigi-la é uma



parte essencial do treinamento dos desenvolvedores em qualquer empresa. Produtos como o BeEF (Browser Exploitation Framework, <http://beefproject.com/>) podem ajudar você a encontrar o vetor de ataque e então mostrá-lo a outras pessoas em sua empresa.

Agora você pode começar a manipular o URL para ver o que é possível fazer. Por exemplo, vamos supor que você queira determinar quantas colunas essa query gera e o que são essas colunas. É possível usar a cláusula SQL `ORDER BY` para executar essa tarefa. Altere o URL de modo que ele inclua a cláusula `ORDER BY` assim: `http://www.mysite.com/index.php?itemid=10' ORDER BY 1`. Isso equivale a digitar `SELECT * WHERE itemid='10' ORDER BY 1` como um comando. Ao incrementar o valor de `ORDER BY` em 1 a cada requisição, em algum momento você verá um erro novamente. Suponha que você veja o erro quando tentar `ORDER BY 10`. O resultado da query, na verdade, tem nove colunas nesse caso.

Um hacker continuará a acrescentar comandos SQL ao URL básico para saber mais sobre os resultados da query no banco de dados. Por exemplo, usar a cláusula `SELECT` ajuda a determinar quais dados aparecem na tela. Você também pode solicitar informações especiais como parte da cláusula `SELECT`, por exemplo, `@@version`, para obter o número de versão do servidor SQL (dando uma ideia de quais vulnerabilidades o servidor SQL pode ter). A questão é que o URL original oferece acesso direto ao servidor SQL, possibilitando que alguém assuma o controle desse servidor sem muito trabalho.

Você pode ver outro tipo de ataque de injeção de SQL detalhado em http://www.w3schools.com/sql/sql_injection.asp. A causa subjacente de todos esses ataques é que um desenvolvedor usou dados diretamente de uma página ou de uma requisição sem antes verificá-la para saber se há problemas e, possivelmente, sem remover informações errôneas.

Entendendo o Cross-Site Scripting

O XSS é semelhante à injeção de SQL em muitos aspectos por causa do modo como a exploração ocorre. No entanto, a técnica propriamente dita é diferente. Os dois tipos de XSS são: não persistente (em que a exploração de falha depende do usuário visitar um link especialmente criado) e persistente (em que o código do ataque é armazenado em uma área de armazenagem secundária, por exemplo, em um banco de dados). Os dois ataques dependem de colocar código JavaScript em um local que você não esperaria.

Como exemplo de uma forma não persistente de XSS, considere este link: *<http://www.mysite.com/index.php?name=quest>*. Parece um link totalmente inofensivo, com uma entrada nome/valor acrescentada. Entretanto, se você adicionar um script a ele, por exemplo, *[http://www.mysite.com/index.php?name=quest<script>alert\('XSS'\)</script>](http://www.mysite.com/index.php?name=quest<script>alert('XSS')</script>)*, o usuário poderia ver uma caixa de diálogo aparecer com a mensagem XSS. Esse exemplo não causa nenhum dano, mas o script poderia facilmente executar qualquer tarefa que seja possível em um script. Por exemplo, você poderia compor o script de modo que ele redirecionasse o usuário a outro site em que a página faria o download de um vírus ou de outro software nocivo.



O exemplo mostra a tag `<script>` em formato de texto simples. Um exploit de verdade codificaria a tag `<script>` de modo que o usuário não pudesse reconhecê-la facilmente. O que o usuário veria seria um URL longo, com um conjunto extremamente complexo de valores % que também aparece regularmente em URLs válidos.

Um ataque de XSS persistente é mais difícil de implementar, mas também causa muito mais danos. Por exemplo, considere o que acontece quando um usuário faz login em uma aplicação. O servidor envia um ID de sessão na forma de um cookie de volta ao computador do usuário. Toda requisição depois disso usa o ID de sessão para que a aplicação possa manter informações de estado sobre o usuário. Vamos supor que o invasor envie um script especialmente criado ao servidor como parte do processo de login; esse script é armazenado no banco de dados com um nome que é quase certo que o administrador vá querer analisar. Quando o administrador clica no nome do usuário, o script executa e envia o ID de sessão do administrador ao invasor. O invasor agora tem os privilégios do administrador durante o restante da sessão, possibilitando executar qualquer tarefa de nível de administrador.

Você pode obter informações mais detalhadas sobre XSS persistente em <https://www.acunetix.com/blog/articles/persistent-cross-site-scripting/>.

Nos dois casos, a melhor defesa contra o XSS é sanitizar qualquer entrada que você receber. O processo de sanitizar a entrada remove qualquer script, caractere estranho ou outras informações que não façam parte da resposta esperada a uma solicitação. Por exemplo, se você espera uma entrada numérica, a resposta não deve conter caracteres alfabéticos.



Uma saída que codifique os dados fornecidos pelo usuário pode ajudar a reduzir o potencial para XSS. Um hacker ainda poderia contornar a codificação, mas seria preciso tempo e esforço adicionais para isso. Tudo que você puder fazer para dificultar a vida do hacker aumentará o desejo dele de hackear outra pessoa, mas você também deve se lembrar de que um hacker determinado sempre violará sua segurança.

Enfrentando problemas de negação de serviço (denial-of-service)

A ideia por trás de um ataque de DOS (Denial of Service, ou Negação de Serviço) é relativamente simples. Você encontra uma porta aberta em um servidor e fica enviando pacotes sem sentido para ela em um esforço para sobrecarregar o serviço associado. A maioria dos servidores tem serviços que oferecem portas abertas, como:

- servidores de DNS;
- servidores de email;
- servidores de FTP;
- servidores de Telnet;
- servidores web.

É claro que quanto mais portas você oferecer, maiores serão as chances de sobrecarregar seu servidor. Uma primeira linha de defesa é fechar portas que não sejam necessárias, evitando instalar serviços de que você não precisa. Um servidor de aplicações pode exigir apenas um servidor web, portanto esse é o único serviço que você deve instalar. Como desenvolvedor de aplicações, você pode recomendar que outros serviços não sejam instalados (ou que

fiquem, no mínimo, inativos). Ao criar uma aplicação privada, usar uma porta não padrão também pode ajudar, assim como exigir autenticação.

Hackers normalmente procuram serviços que não tenham um número máximo de conexões, portanto garantir que você mantenha o número máximo de conexões com um valor que seu servidor possa tratar é outro passo na direção certa. Em um ataque de DOS, é importante dar ao servidor algo para fazer, por exemplo, executar uma busca complexa quando estiver trabalhando com um servidor web. A autenticação ajudaria a evitar que um hacker faça requisições sem a devida autenticação.

Apesar disso, é possível derrubar qualquer servidor se você tiver sistemas suficientes enviando um fluxo interminável de requisições sem valor. Algumas das defesas contra ataques DOS incluem procurar padrões na requisição e então simplesmente negá-las, em vez de gastar recursos do sistema tentando resolver a requisição. Você pode encontrar várias ferramentas para ataques de DOS para testar seu sistema em <http://resources.infosecinstitute.com/dosattacks-free-dos-attacking-tools/>.

Além de procurar padrões em ataques e tentar resolvê-los por conta própria, você também pode tentar:

- comprar equipamentos especializados projetados para ajudar a atenuar ataques de DOS;
- contar com seu ISP para identificar e atenuar ataques de DOS;
- obter os serviços de um provedor de atenuação de ataques na nuvem.

Aproveitando-se da localização previsível de recursos

É possível invadir um sistema conhecendo a localização de recursos específicos e então usar esses recursos para obter informações suficientes para acessar o sistema. Alguns sites também se referem a esse tipo de ataque como *navegação forçada* (forced browsing). É claro que a melhor maneira de evitar esse tipo de ataque é manter os recursos em locais imprevisíveis. Além disso, você pode garantir que o esquema de autenticação funcione e que os recursos sejam adequadamente protegidos. No entanto, vamos dar uma olhada em como esse ataque em particular funciona.

Um exemplo desse tipo de ataque envolve identificar um URL que aponte para um recurso válido e, por sua vez, usar esse URL para acessar outro recurso de propriedade de outra pessoa. Vamos supor que você tenha sua agenda para determinado dia em *http://www.mysite.com/Sam/10/01/2015* e que você queira acessar a agenda de Amy para o mesmo dia. Alterar o URL para *http://www.mysite.com/Amy/10/01/2015* pode proporcionar o acesso necessário se o administrador não tiver configurado a autenticação no servidor corretamente.

Como outro exemplo, alguns servidores colocam informações em diretórios específicos. Por exemplo, você pode ter acesso autorizado a *http://www.mysite.com/myapp/app.html*. Porém, você poderia alterar o URL para ver se *http://www.mysite.com/system/* existe. Se obtiver uma resposta igual a 200 do servidor, o diretório existe e você pode começar a consultá-lo em busca de informações úteis. É claro que isso supõe que o administrador não protegeu adequadamente o diretório do sistema e que esse sistema está em um diretório-padrão. O administrador poderia ter alterado o nome do diretório de sistema, além de garantir que o acesso a ele só seja permitido àqueles que tenham as credenciais apropriadas.

Evitando a divulgação de informações que não deveriam ser reveladas

A divulgação não intencional de informações que não deveriam ser reveladas pode ocorrer de várias maneiras. No entanto, a exploração de falhas sempre envolve a obtenção de acesso não autorizado a um banco de dados centralizado por parte do hacker. Houve uma época em que a fonte de informações teria sido algo como o NIS (Network Information System, ou Sistema de Informações da Rede). Porém, hoje em dia, a informação pode vir de qualquer fonte que não seja segura ou que tenha vulnerabilidades que um hacker possa explorar. Há tantas dessas fontes que não é realmente possível criar um exemplo simples que ilustre todas elas. Você pode vencer esse tipo de hack se:

- aplicar todos os patches necessários ao sistema operacional, aos serviços, ao ambiente da aplicação, às bibliotecas, às APIs e aos microsserviços,

conforme for preciso;

- configurar os roteadores de fronteira e os firewalls para bloquearem requisições que poderiam solicitar informações de uma fonte sensível;
- restringir o acesso a todas as fontes com informações sensíveis;
- jamais deixar senhas fixas no código ou colocá-las em um lugar em que alguém possa encontrá-las facilmente;
- usar autenticação de dois fatores para qualquer fonte de informações sensíveis;
- fazer auditorias para procurar brechas de segurança em potencial (mesmo que você ache que o sistema esteja totalmente protegido);
- usar ferramentas de avaliação para determinar se é possível acessar a fonte de informações sensíveis de qualquer lugar que não seja um local designado para isso.

Testando em um ambiente BYOD

O fenômeno BYOD continua ganhando força. É importante perceber que o BYOD não desaparecerá. De fato, é provável que que você já saiba que, em algum momento, o BYOD será o método preferido para os usuários utilizarem. As empresas dirão aos usuários para que tragam qualquer dispositivo necessário para executar suas tarefas e deixarão tudo nas mãos do usuário. Parece um desastre na prática, mas é nessa direção que os usuários estão indo.

De acordo com a Gartner, Inc, em 2017, metade das empresas não fornecerá mais nenhum dispositivo aos usuários (veja <http://www.gartner.com/newsroom/id/2466615>). Além disso, por volta de 2020, 75% dos usuários pagarão menos de cem dólares por um smartphone (veja <http://www.gartner.com/newsroom/id/2939217>), portanto obter um dispositivo inteligente o suficiente para executar a maioria das tarefas nem será tão caro assim. Criar e administrar aplicações será mais difícil porque você precisará garantir que a aplicação realmente funcione em todos os lugares e em qualquer dispositivo. É claro que ela deve funcionar sem comprometer a segurança das empresas. As próximas seções ajudarão você a

oferecer certo nível de testes e de isolamento para o ambiente BYOD.



Também é importante perceber que os usuários agora estão participando fortemente do BYOA (Bring Your Own Application). O motivo de esse novo movimento por parte dos usuários ser tão importante é que ele introduz outro nível de imprevisibilidade ao ambiente da aplicação. Você nunca sabe quando outra aplicação causará problemas à sua aplicação. De fato, a aplicação de terceiros poderia fornecer o caminho para a próxima violação de segurança importante que sua empresa sofrerá. Os usuários definitivamente continuarão usando aplicações como Dropbox, Google Docs e CloudOn, pois elas são convenientes e executam em qualquer lugar. Para garantir a integridade do ambiente da aplicação, é preciso continuar vendo essas aplicações como contaminações, apenas esperando para arruinar seu dia.

Um dos motivos pelos quais o BYOA é um problema é que a empresa perde o controle sobre seus dados. Se o usuário armazenar dados da empresa em uma conta pessoal no Dropbox, a empresa não poderá recuperar facilmente esses dados em caso de emergência. Em suma, o BYOA abre brechas sérias de segurança que poderiam ser um problema para qualquer dado da aplicação que você queira proteger.

Configurando uma zona de acesso remoto

Quando trabalhar em um ambiente BYOD, a melhor suposição que você pode fazer é que o ambiente do dispositivo não é seguro e que você não pode deixá-lo seguro facilmente. Com isso em mente, uma sessão de BYOD normalmente tem quatro fases:

1. O cliente e o servidor criam um túnel seguro para dificultar que outros escutem. O propósito é evitar ataques man-in-the-middle.
2. O usuário fornece duas formas de autenticação. Um processo de autenticação de dois fatores faz com que seja menos provável que alguém se passe pelo usuário com sucesso.
3. O cliente faz uma ou mais requisições para as quais o servidor faz a mediação usando um módulo de mediação de serviços. O módulo de mediação de serviços só trata requisições para serviços válidos. O módulo de mediação de serviços faz log automaticamente de todas as requisições, sejam elas bem-sucedidas ou não.
4. Um módulo de separação de serviços oferece acesso apenas aos dados

públicos. Ele impede o acesso a dados sensíveis. O cliente vê apenas os dados que a empresa considera aceitáveis para um ambiente BYOD.

A zona de acesso remoto faz parte das fases 1 e 2. Ela é constituída de um firewall externo e de uma VPN e um gateway para autenticação. A zona de acesso remoto oferece um primeiro nível de defesa contra invasões.

As informações obtidas do usuário devem aparecer como parte da estratégia de sua aplicação. Um acesso BYOD é diferente de um acesso local a partir de um sistema desktop de propriedade da empresa, pois você não tem ideia de onde esse acesso ocorre ou se o dispositivo é seguro. Quando um usuário acessa sua aplicação dessa maneira, você deve oferecer-lhe um papel que esteja de acordo com o dispositivo usado. Isso quer dizer que você não deve deixar que nenhuma informação sensível apareça no dispositivo e pode, possivelmente, limitar o acesso a outros tipos de dados. Por exemplo, sua empresa pode decidir que é aceitável obter uma listagem das vendas para um cliente em particular, mas a lista é somente para leitura, o que significa que o usuário não poderá fazer alterações na lista de vendas quando estiver em campo. Para fazer essas alterações, o usuário precisaria fazer login na conta a partir de um sistema local.

O uso de uma zona de acesso remoto também implica que sua empresa deve configurar o MDM (Mobile Device Management, ou Gerenciamento de Dispositivos Móveis). O MDM é um conjunto de produtos e serviços que ajuda a garantir que os dispositivos móveis sejam tão seguros quanto possível. Por exemplo, o MDM poderia verificar os dispositivos móveis para conferir os requisitos de patch e garantir que o usuário os tenha instalado no dispositivo antes de você permitir que a aplicação tenha acesso (você poderia simplesmente instalar o patch no dispositivo automaticamente). Mesmo com um MDM implantado, você não pode supor que um dispositivo seja seguro quando:

- o dispositivo informa valores que implicam uma configuração segura, mas sua empresa não implementa os valores especificados. Um desenvolvedor de malware não saberá quais valores devem ser informados e simplesmente informará todos eles.
- outras autoridades podem sobrescrever as configurações do dispositivo.

Por exemplo, se um fornecedor de smartphone puder fazer patch automaticamente do dispositivo sem que isso esteja em seu controle, você deve supor que alguns desses patches poderiam conter vírus ou outros malwares.

- não é possível garantir configurações no dispositivo móvel. O dispositivo pode não oferecer o suporte necessário, sua aplicação pode não estar configurada para dar suporte ao dispositivo ou malwares interferem nas atualizações.

Verificando hacks entre aplicações

Um hack entre aplicações (cross-application hack) é aquele em que uma aplicação tem acesso aos dados ou a outros recursos usados por outra aplicação. Em muitos casos, o hack ocorre quando duas aplicações têm acesso à mesma fonte de dados, o que é conhecido como XARA (Cross-Application Resource Access, ou Acesso a Recursos entre Aplicações). Um hack recente desse tipo ocorreu em sistemas operacionais OS X e iOS (leia sobre ele em <http://oversitesentry.com/xara-an-old-way-to-hack-cross-application-resourceaccess/>).

Outro tipo de problema entre aplicações é o CAS (Cross-Application Scripting, ou Scripting Entre Aplicações). Nesse caso, o bug provoca um problema em que o código JavaScript executa em determinados tipos de aplicação. Você pode ler mais sobre esse tipo de exploração de falhas para Android em <http://news.softpedia.com/news/Apache-Cordova-for-Android-Patched-Against-Cross-Application-Scripting-453942.shtml>.

A melhor maneira de verificar se sua aplicação não está possivelmente expondo dados por meio desse tipo de exploração de falhas é manter-se atualizado quanto a qualquer notícia para as plataformas às quais você dê suporte. É preciso ter um especialista em segurança para encontrar possíveis problemas desse tipo. Infelizmente, exceto por um patch de um fornecedor de sistema operacional ou de navegador, não haverá muito o que fazer sobre esse hack em particular, a não ser permanecer vigilante verificando suas áreas de armazenagem de dados em busca de possíveis danos. É claro que esse tipo de problema em particular dá mais crédito à criação de uma zona de acesso

remoto (veja a seção anterior neste capítulo).

Lidando com equipamentos e softwares realmente antigos

Empresas de pesquisa adoram criar imagens lindas de como será exatamente o futuro. É claro que alguns desses sonhos realmente se tornam verdadeiros, mas eles não levam em consideração a variedade de equipamentos dos usuários. Por exemplo, o fato de descobrir que (na época em que este livro foi escrito) 95% de todos os caixas eletrônicos americanos ainda dependem de Windows XP como sistema operacional (veja <http://info.rippleshot.com/blog/windows-xp-stillrunning-95-percent-atms-world>) pode surpreender algumas pessoas. A marinha americana continua usando Windows XP em cem mil desktops (veja <http://arstechnica.com/information-technology/2015/06/navy-re-upswith-microsoft-for-more-windows-xp-support/>). Sim, softwares velhos, arcaicos e decrépitos ainda existem por aí e você poderá se deparar com eles sendo usados para executar sua aplicação.

De acordo com a NetMarketShare (<http://www.netmarketshare.com/operating-system-market-share.aspx?qprid=10&qpcustomd=0>), o Windows XP ainda está instalado em até 14,6% dos sistemas por aí. É importante perceber que sua principal preocupação ao criar uma aplicação pode não ser o novo smartphone reluzente, com o sistema operacional atualizado, todos os patches e a tecnologia mais recente de navegador. A verdadeira questão com que se preocupar pode ser aquele velho e decrépito sistema Windows XP, com Internet Explorer 8 que seus usuários insistem em utilizar. É interessante observar que o Internet Explorer 8 ainda detém aproximadamente 25% da parcela de mercado de desktops (veja <http://www.netmarketshare.com/browsermarket-share.aspx?qprid=2&qpcustomd=0>).

É claro que você sempre pode tentar forçar os usuários a fazerem um upgrade. No entanto, se você já lidou com todo o fenômeno de BYOD por tempo suficiente, saberá que seus usuários irão simplesmente ignorar você. Sim, eles podem mostrar um sistema novo e reluzente, mas continuarão a

usar o sistema que quiserem – aquela velharia que está causando grandes problemas à sua aplicação.

Assim, o único meio eficiente de lidar com equipamentos desatualizados é verificar os dados do navegador durante as requisições. Fazer isso permite que você decida se vai permitir que a requisição seja bem-sucedida. Quando o navegador for velho demais, você poderá simplesmente exibir uma mensagem dizendo ao usuário para fazer um upgrade de seu sistema. O artigo em <http://sixrevisions.com/javascript/browser-detection-javascript/> descreve como fazer uma verificação da versão do navegador. Você pode ver as informações de seu navegador em <https://www.cyscape.com/showbrow.asp>. O único problema com essa abordagem é que é possível impedir a verificação em alguns casos, e alguns navegadores também dão informações incorretas de versão.

Contando com testes de usuário

Ninguém é capaz de testar o software de forma mais violenta e imprevisível que um usuário. A maioria dos desenvolvedores já viu usuários tentarem fazer o software executar tarefas que ele obviamente não deveria, pois o usuário não percebe que o software não deveria fazer isso. É essa falta de conexão entre o que o software parece fazer e o que o usuário faz o software executar que possibilita os testes sérios que somente um usuário pode fazer.

A parte importante do processo é aquela que causará as maiores dificuldades aos desenvolvedores. Qualquer tentativa de criar uma abordagem organizada para testes de usuário resultará somente em falhas de testes. Os usuários precisam ter liberdade para brincar com o software. Na verdade, um tempo para brincar com o software é bom para todos, mas é um elemento essencial nos testes de usuário. As próximas seções descrevem como você pode fazer o usuário brincar com seu software, o que resultará naquelas sérias deficiências de segurança que somente os usuários (e aparentemente alguns hackers) parecem encontrar.

O interessante é que você não precisa mais realizar seu próprio teste de usuário. Se você realmente quiser ter uma base ampla de usuários de teste, mas não tem recursos para fazer isso por conta própria, é sempre possível contar com sites



como o User Testing (<http://www.usertesting.com/>). Teoricamente, o site fornecerá avaliações honestas de sua aplicação em um tempo muito rápido, por exemplo, uma hora (embora uma citação que estava na página principal dissesse que os resultados seriam entregues em 20 minutos). A maioria desses sites de testes de terceiros oferece testes para aplicações web, aplicativos móveis e de protótipos.

Alguns sites de teste de terceiros, como o Applause (<http://www.applause.com/web-app-testing>), oferecem especificamente testes “em campo”, em que sua aplicação realmente é usada na internet por usuários desconhecidos, como ocorreria no ambiente de produção. É claro que a diferença está no fato de os testes ocorrerem sem nenhum risco ao seu equipamento, e o Applause oferece as ferramentas necessárias para obter resultados mensuráveis. Em geral, você vai querer deixar esse nível de testes para quando sua aplicação estiver quase pronta para ser lançada e você quiser realizar uma verificação de sanidade.

Deixando o usuário correr solto

Às vezes, a melhor maneira de testar sua aplicação é dar ao seu usuário um conjunto de passos e um objetivo. Observar como o usuário interage com a aplicação pode dizer muito sobre a percepção dele em relação à aplicação e os pontos em que você precisa fazer alterações. Na verdade, filmar o usuário trabalhando pode ajudar, mas você deve estar ciente de que o ato de filmar mudará o comportamento do usuário. Captura de teclado (keylogging) e outras técnicas também são úteis para monitorar o que o usuário faz com a aplicação sem ficar realmente em cima do usuário observando (o que, definitivamente, mudaria o comportamento do usuário, e seu teste falharia).

Felizmente, não é preciso recorrer apenas aos recursos que você tem à mão. Sites como o Mashable (<http://mashable.com/2011/09/30/website-usability-tools/>) oferecem uma variedade de ferramentas de testes que pode ser usada para verificar se sua aplicação tem problemas. O site tem boa documentação para as ferramentas e informa por que cada ferramenta é importante. Acima de tudo, o site ajuda a entender a importância de tipos específicos de testes que talvez você não tenha considerado. Por exemplo, Check My Colours (<http://www.checkmycolours.com/>) confere se as pessoas com várias necessidades visuais podem realmente ver seu site. Sim, usar as cores erradas

realmente pode ser um problema, e testar isso pode ajudar a evitar problemas de segurança em potencial, causados por erros do usuário.

Outro bom lugar para encontrar ferramentas de teste é o The Daily Egg (<http://blog.crazyegg.com/2013/08/08/web-usability-tools/>). Algumas das ferramentas nesse site são as mesmas que estão no site Mashable, mas você verá esclarecimentos adicionais sobre elas. Algumas das ferramentas são únicas. Por exemplo, a lista de ferramentas de testes para velocidade de páginas é melhor em The Daily Egg e inclui o GTMetrix (<http://gtmetrix.com/>), que pode ajudar a localizar a origem da lentidão em sua página.

Desenvolvendo passos reproduzíveis

Parte dos testes de usuário consiste em obter um conjunto de passos reproduzíveis. A menos que você obtenha esse tipo de informação, não será realmente possível corrigir os problemas da aplicação. Esse é o motivo pelo qual você precisa de softwares específicos de testes que registrem o que o usuário faz em detalhes. Pedir que o usuário reproduza os passos que resultaram em um erro nunca funciona. Em muitos casos, o usuário não tem ideia de como o erro ocorreu e simplesmente acha que o computador não gosta dele. Tentar fazer o usuário reproduzir os passos mais tarde provavelmente resultará em frustração e fará o usuário não gostar da aplicação (o que, normalmente, resulta em mais erros e em mais problemas de segurança). Desse modo, você precisa identificar os passos necessários para reproduzir um erro na primeira tentativa.

Para criar um ambiente em que você possa obter o melhor nível de testes, além de garantir que o usuário tenha uma boa chance de encontrar erros em potencial, é necessário executar tipos específicos de testes. Deixar o usuário correr solto para ver o que eles podem fazer sem qualquer tipo de orientação é útil, mas o caos dificilmente gera resultados previsíveis. Quando trabalhar com testes de usuário, você deve considerar os seguintes tipos de teste:

Funcionalidade

É importante testar todas as funcionalidades da aplicação. Isso quer dizer pedir que os usuários testem formulários, manipulem arquivos e façam

tarefas de cálculo, busquem informações usando recursos da aplicação e experimentem usar qualquer recurso de mídia que sua aplicação oferecer. À medida que o usuário testar essas várias funcionalidades, certifique-se de que os testes também verifiquem bibliotecas, APIs e microsserviços dos quais sua aplicação dependa para executar a maioria das tarefas.

Interface de usuário e usabilidade

A interface de usuário deve manter o usuário envolvido e oferecer suporte a qualquer pessoa com necessidades especiais. Como parte desse nível de testes, você deve conferir a navegação, a acessibilidade, a usabilidade em vários tipos de navegadores, as mensagens de erro e avisos, informações de ajuda e qualquer outra documentação oferecida, além dos layouts.

Segurança

Embora você tenha testado a aplicação para determinar se ela tem algumas das brechas de segurança comuns listadas na seção “Definindo as fontes mais comuns de brechas de segurança”, é preciso que os usuários a testem também. Veja se o usuário consegue provocar uma falha na aplicação dos mesmos modos que você conseguiu. Procure maneiras pelas quais o método do usuário para interagir com a aplicação crie novas condições para brechas de segurança.

Carga e escalabilidade

É impossível testar totalmente uma aplicação a fim de determinar como ela atua quando houver carga. É preciso garantir que a aplicação escale bem e que seu desempenho degrade elegantemente à medida que a carga aumentar. Porém, você deve verificar, sobretudo, se a carga não causa problemas em que uma falha de segurança possa ocorrer. É importante saber se a aplicação continuará funcionando adequadamente, não importando a carga aplicada a ela. É claro que a aplicação executará mais lentamente quando a carga exceder as expectativas, mas isso não é o mesmo que provocar falhas (isto é, provocar uma falha que poderia permitir que alguém entre).

Dando voz ao usuário

Entrevistar um usuário ou permitir que os usuários discutam a aplicação em uma sessão de reclamações após terem trabalhado com ela é uma boa maneira de descobrir mais problemas em potencial com a aplicação. Em muitos casos, um usuário ficará com medo de executar determinados passos por causa da percepção de que a aplicação falhará. O problema ocorre mesmo em um ambiente de testes, em que se espera que o usuário cause falhas na aplicação. Talvez você não queira ouvir o que os usuários têm a dizer, mas é melhor ouvir durante a fase de desenvolvimento que colocar a aplicação em ambiente de produção e descobrir que ela tem problemas sérios depois.

Como alternativa às abordagens de confronto para obter feedback dos usuários, você também pode contar com pesquisas. Uma pesquisa anônima poderia ajudar a obter informações que o usuário, de outra maneira, poderia se sentir constrangido em fornecer. É importante considerar os efeitos do estresse e dos riscos percebidos na qualidade do feedback que você receber dos usuários.

Usando pentesters externos

Testes de invasão contam com serviços de terceiros para determinar se uma aplicação, site ou até mesmo uma empresa como um todo é suscetível a vários tipos de invasão. O invasor sonda as defesas usando as mesmas técnicas utilizadas por um hacker. Como o invasor é um profissional da área de segurança, o nível dos testes provavelmente será melhor que aquele que uma empresa pode ter por conta própria. A maioria das empresas usa serviços externos de testes de segurança pelos seguintes motivos:

- Localizar as vulnerabilidades não identificadas por auditorias internas.
- Oferecer aos clientes e a terceiros uma auditoria independente de segurança incluída na aplicação.
- Garantir que os testes de segurança estejam completos porque a empresa não tem nenhum profissional de segurança na equipe.
- Validar a eficiência dos procedimentos de gerenciamento de incidentes.
- Treinar a equipe de tratamento de incidentes.
- Reduzir os custos de segurança para a empresa como um todo.



É uma péssima ideia permitir que alguém faça um teste de invasão em sua aplicação, site ou empresa sem ter a documentação legal estabelecida, por exemplo, um NDA (Nondisclosure Agreement, ou Acordo de Confidencialidade). Não suponha que você possa confiar em alguém, especialmente em alguém que esteja testando intencionalmente a segurança de sua instalação. Certifique-se de ter tudo por escrito e que não haja nenhuma chance de problemas de compreensão desde o início.

Considerando a empresa de testes de invasão

É claro que, como tudo o mais, os testes de invasão são acompanhados de alguns riscos. Por exemplo, é provável que você espere que o pentester terceirizado seja totalmente honesto com você a respeito das vulnerabilidades encontradas em sua aplicação, mas muitas vezes não é isso que acontece. Em alguns casos, o terceiro falha em documentar completamente as vulnerabilidades, mas, em outros, o pentester pode estar avaliando sua empresa a fim de determinar se uma invasão de verdade seria uma boa ideia. É importante garantir que você esteja lidando com um pentester de boa reputação e conferir previamente os tipos de serviços prestados. Você deve investir mais tempo verificando os serviços de segurança se perceber que:

- houve divulgação, abuso ou perda de informações sensíveis obtidas durante um teste de invasão;
- há vulnerabilidades ou pontos fracos não identificados;
- a disponibilidade da aplicação ou do site-alvo sofreram impactos;
- os testes não ocorreram no prazo;
- o relatório é excessivamente técnico e difícil de compreender;
- o gerenciamento do projeto parece desorganizado, feito às pressas ou sem a devida consideração.

Quando se trata de testes de invasão, a tendência é obter aquilo pelo qual pagamos. Sua empresa deve considerar o custo-benefício entre preço, execução no prazo e qualidade dos testes quando contratar um terceiro para realizar testes de invasão. Quanto menos você pagar, mais você tenderá a esperar, menos você receberá pelo valor pago e maiores serão as chances de que algo negativo ocorrerá como resultado dos testes.

Em alguns casos, uma compensação na quantidade de testes funciona melhor que outras vantagens. Testar apenas uma aplicação, em vez de testar o site todo, custará menos e você será capaz de contratar uma empresa de segurança mais qualificada para realizar a tarefa. É claro que, se você testar apenas uma aplicação, não será possível verificar questões como a existência de relacionamentos confiáveis entre o sistema-alvo e outro sistema.

Gerenciando o projeto

Antes de permitir que alguém execute testes de invasão, você precisa de uma proposta que defina o escopo, os objetivos e os métodos de teste. Qualquer teste de invasão deve incluir ataques de engenharia social, pois a maioria dos hackers os emprega. De fato, as pessoas de sua empresa são o elo mais fraco em sua segurança. Uma aplicação pode oferecer um sistema quase perfeito de segurança, mas uma única revelação feita pela pessoa errada pode frustrar todas as suas tentativas de manter um ambiente seguro.

Garanta que a metodologia de testes esteja bem documentada e esteja de acordo com as melhores práticas do mercado. Por exemplo, muitas empresas de segurança seguem o OSSTMM (Open Source Security Testing Methodology, ou Metodologia Aberta para Testes de Segurança); dê uma olhada no site do ISECOM (Institute for Security and Open Methodologies, ou Instituto para Segurança e Metodologias Abertas em <http://www.isecom.org/>) para ver mais detalhes. Se o pentester usa outra metodologia, verifique se você entende exatamente o que é essa metodologia e quais tipos de vantagens ela traz.

A proposta deve definir o tipo de feedback que você receberá como parte dos testes. Por exemplo, é importante decidir se os testes de invasão incluem revelações completas, com uma demonstração de como, exatamente, o sistema é invadido, ou se o relatório simplesmente informa que uma área em particular é vulnerável a ataques.

Definir o horário do ataque também é importante. Talvez você não queira permitir testes durante os horários mais movimentados do dia para evitar que haja riscos às vendas. Por outro lado, talvez você precise permitir que os testes sejam realizados em horários menos convenientes se quiser realmente

ver os efeitos da carga no sistema.

Incluindo o essencial

Qualquer empresa que você contratar para testes de invasão deve ter um único ponto de contato. É importante que você possa acessar esse contato a qualquer momento. O líder de sua equipe de desenvolvimento e o contato na equipe de testes de invasão devem ser capazes de se comunicar um com o outro a qualquer momento para interromper os testes, caso haja necessidade. O contato também deve ter os detalhes completos sobre como, exatamente, os testes estão ocorrendo, e deve conhecer as especificidades sobre qualquer problema em potencial que possa ocorrer durante a fase atual de testes.

Você também deve saber se o pentester tem seguro para cobrir qualquer perda que possa ocorrer durante os testes. O seguro deve incluir o pagamento pelo tempo gasto na recuperação dos dados, o tempo de inatividade (downtime) do sistema e qualquer perda em potencial de receita que sua empresa possa ter.

A equipe de testes também deve demonstrar competência. Você não vai querer que qualquer pessoa faça testes de invasão em seu sistema. A equipe definida na proposta também deve ser a equipe que faz os testes propriamente ditos. Verifique se há certificações como:

- GIAC Certified Incident Handler (GCIH)
- Certified Ethical Hacker (CEH)
- OSSTMM Professional Security Tester (OPST)

Obtendo o relatório

O resultado de qualquer teste de invasão é apresentado na forma de um relatório. Para garantir que você obtenha um relatório que realmente possa ser usado, não se esqueça de pedir um relatório de exemplo antes que os testes comecem. O relatório deve incluir um resumo dos testes, detalhes sobre qualquer vulnerabilidade, uma avaliação de riscos e detalhes de todas as ações envolvidas no teste de invasão. O relatório deve conter informações técnicas suficientes para que você possa realmente corrigir os problemas

encontrados durante os testes, mas deve ser legível pela equipe de gerência para que você possa negociar o tempo e os recursos necessários para efetuar as correções.

Juntamente com o relatório, você também deve receber arquivos de log com todas as ações executadas durante a fase de teste. Os arquivos de log devem mostrar todos os pacotes enviados e recebidos durante os testes para que você possa retornar depois e seguir o processo de testes, passo a passo.

A empresa de segurança contratada também deve manter um arquivo do processo de testes. Você precisa saber como eles planejam manter esse arquivo. Parte do motivo para verificar o processo de arquivamento é garantir que seus dados confidenciais permanecerão confidenciais. É importante que o arquivamento seja feito como parte de um armazenamento offline, e não em um drive de rede no sistema do fornecedor.

¹ N.T.: “O termo adware é frequentemente usado para descrever uma forma de malware (software malicioso), geralmente aquela que apresenta anúncios indesejados para o usuário de um computador. Os anúncios produzidos por adware são, por vezes, em forma de um pop-up.” (Fonte: <https://pt.wikipedia.org/wiki/Adware>)

CAPÍTULO 10

Criando uma zona de segurança para APIs

Qualquer API que você criar ou usar como parte de sua aplicação tem o potencial para criar uma grande variedade de problemas. No entanto, de modo diferente das bibliotecas, você pode deixar o uso de uma API muito mais seguro porque uma API executa em seu próprio espaço de endereçamento e em seu próprio processo. Colocar a API em uma sandbox ou em um ambiente virtual (essencialmente, em um ambiente protegido) possibilita:

- controlar precisamente as ações que a API pode executar, os recursos que ela pode acessar e as maneiras como ela interage com sua aplicação. É claro que você também pode deixar a API sem recursos ou fazer com que seja impossível que ela conclua uma tarefa, deixando a sandbox ou o ambiente virtual restrito demais. Há um equilíbrio que você deve alcançar entre riscos (à segurança) e a capacidade de realizar tarefas úteis.
- controlar como a aplicação interage com a API. Por exemplo, você diminuirá a probabilidade de entradas errôneas ou maliciosas provocarem algum tipo de efeito desastroso. As entradas da aplicação são estritamente controladas, e entradas não esperadas tendem a ter um efeito reduzido ou a não ter efeito algum. É claro que esse tipo de proteção também pode dificultar fazer experimentos com a API ou realizar determinados tipos de teste.

Este capítulo ajuda você a entender o conceito de uma sandbox/ambiente virtual para API e determinar exatamente como você pode usar um ou outro em seu próximo projeto de programação para manter seu código seguro. Como parte de trabalhar com sandboxes de API, este capítulo também

discute algumas soluções de sandboxing e ambientes virtuais. Nem toda solução funciona em todas as situações, portanto é uma boa ideia conhecer algumas delas em potencial.

A última seção deste capítulo discute uma alternativa ao sandboxing: a virtualização. Apesar de parecer que um ambiente virtual e uma sandbox sejam essencialmente iguais, pois em grande medida atingem os mesmos objetivos, na verdade, são tecnologias diferentes; assim, essa última seção ajuda a entender as diferenças em detalhes. Um ambiente virtualizado tende a fazer com que, para a API, ela pareça executar em um ambiente totalmente aberto. No entanto, um ambiente virtualizado é totalmente isolado de tudo o mais, e a API funciona por meio de um esquema rigoroso de acesso.



Como ocorre com qualquer solução de segurança, usar sandbox de API não é uma solução para todos os males. Ela não manterá todos os hackers afastados e não evitará que todas as APIs causem danos à sua aplicação ou aos dados associados a ela. Usar uma sandbox de API reduz os riscos. Você deve determinar se a sandbox de API reduz suficientemente os riscos ou se você precisa de medidas adicionais para manter sua aplicação segura. A segurança não tem a ver com apenas uma técnica ou uma medida. Um sistema seguro conta com uma série de camadas de segurança para fazer seu trabalho.

A questão principal deste capítulo é ajudar você a criar um ambiente em que uma API possa executar sem realmente se conectar a outras partes da aplicação. O ambiente é rigidamente controlado. O capítulo mostra dois métodos para atingir essa meta e ajuda você a entender quando escolher uma tecnologia em detrimento de outra. Escolher o tipo certo de zona de segurança é essencial se você quiser obter resultados úteis nos testes de sua aplicação e, em última instância, do uso dela.



Este livro usa o termo *zona de segurança* para se referir às metas alcançadas tanto pelas sandboxes quanto pelos ambientes virtuais. É essencial lembrar que as metas das duas tecnologias são basicamente as mesmas, mas como as tecnologias atingem essas metas é significativamente diferente. Por causa das diferenças, você pode achar que escolher uma tecnologia em relação à outra traz vantagens para certas necessidades do desenvolvimento da aplicação.

Entendendo o conceito de zona de segurança de API

Uma zona de segurança de API oferece um método seguro e flexível para

testar ou usar qualquer API que você criar. É sempre uma boa ideia usar uma zona de segurança de API durante os testes para garantir que você possa se recuperar rapidamente de erros. Você pode decidir continuar usando uma zona de segurança de API após passar a API para o ambiente de produção por causa dos recursos de segurança que ela oferece.

O uso de uma sandbox oferece mais segurança. No entanto, muitas empresas não estão preocupadas com segurança quando implementam um ambiente de sandbox. Além de segurança, uma sandbox pode oferecer as seguintes vantagens:

- Custos reduzidos ao controlar o uso de recursos
- Acesso controlado a API de terceiros
- Ambiente melhor para testes e desenvolvimento na forma de mais controle
- Tempo reduzido para colocação no mercado (time to market)
- Cenários simulados de erros para testes
- Desempenho monitorado de APIs

As empresas também usam ambientes virtuais para finalidades que não são simplesmente garantir a segurança da API. Como ocorre com uma sandbox, um ambiente virtual reduz custos ao controlar o uso de recursos e o acesso a APIs de terceiros, além de melhorar o ambiente de testes e de desenvolvimento. Um ambiente virtual tem muitas vantagens do ponto de vista organizacional, incluindo:

- tempo reduzido de recuperação após uma falha de API;
- mais velocidade e controle de banda de rede;
- uso melhorado de energia;
- diminuição do uso de hardware;
- provisionamento mais rápido;
- aprisionamento tecnológico (vendor lock-in)¹ reduzido;
- mais tempo de uptime (mesmo quando eventos de segurança e de confiabilidade são raros);

- vida mais longa para o aplicativo.

A escolha entre usar uma sandbox ou um ambiente virtual pode se reduzir aos recursos adicionais que cada um oferece se a API funcionar igualmente bem em cada opção. Porém, é essencial considerar os aspectos de segurança de cada opção antes, se você quiser manter o tipo de segurança exigido pelas aplicações atualmente.

Definindo a necessidade de uma zona de segurança de API

Segurança tem a ver com gerenciamento de riscos. Fazer requisições a qualquer API implica riscos, pois você não sabe se a API não foi comprometida, supondo que, antes de tudo, ela estava livre de problemas em potencial. A necessidade de vigiar qualquer API que você usar torna-se aparente quando lemos a respeito de danos causados pelas APIs online. Qualquer API que você usar seria capaz de enviar dados que poderiam ser um vírus, um script ou outro malware disfarçado. Por causa do modo como as APIs são usadas, você só descobriria o problema quando todos comessem a reclamar que a aplicação ficou inativa ou quando encontrar informações confidenciais em um lugar em que você não esperaria encontrá-las (supondo que outra pessoa não as encontre antes).

Problemas inversos ocorrem quando você é dono do código de uma API. Sua API pode oferecer um serviço importante para seus clientes. Ela pode oferecer acesso a todo tipo de dados confidenciais, alguns dos quais o cliente deve acessar, enquanto outros devem permanecer ocultos. O tipo incorreto de entrada poderia fazer sua API reagir de maneiras inesperadas – causando danos à API, seus dados e ao servidor em geral. Consequentemente, você deve encontrar maneiras de sanitizar os dados de entrada, mas também deve oferecer proteção quando o processo de sanitização não funcionar tão bem como previsto.

Independentemente de sua API enviar ou receber dados como resultado de uma requisição, usar a API em uma sanbox ou em um ambiente virtual só faz sentido porque fazer isso reduzirá seus riscos. Manter os vilões totalmente afastados pode não funcionar, mas é possível minimizar seus riscos e os

efeitos de qualquer brecha de segurança que você tiver. As próximas seções desenvolvem argumentações para trabalhar com APIs em uma sandbox ou em um ambiente virtual.

Garantindo que sua API funcione

Segurança tem a ver com gerenciamento de riscos. Garantir que sua API funcione conforme previsto reduz os riscos ao diminuir o potencial para alguém fazer a API operar de uma maneira que você não havia considerado. Com frequência, os hackers fazem seu trabalho sujo procurando maneiras de fazer o código se comportar mal – fazendo algo que o proprietário não quer que o código faça. Usar uma zona de segurança de API permite testar seu código de maneiras que você normalmente não poderia testá-lo sem criar problemas para o servidor de testes. Você pode executar os seguintes tipos de testes para garantir que sua API funcione conforme esperado:

- Verificar se as entradas corretas oferecem as respostas corretas.
- Usar verificações de intervalos para garantir que a API responde corretamente a dados fora do intervalo.
- Fornecer dados do tipo incorreto para garantir que a API sanitize os dados de acordo com o tipo.
- Verificar o tamanho dos dados usando entradas que sejam longas demais ou curtas demais.
- Fornecer dados inválidos de propósito que os hackers poderiam usar, como scripts, tags ou dados binários.
- Deixar dados necessários de fora.
- Acrescentar entradas extras.
- Criar entradas com dados nulos.
- Sobrecarregar a API.
- Deixar a API sem recursos.

Possibilitando um desenvolvimento rápido

Colocar sua API diretamente no servidor quer dizer que você terá uma cópia

da API para todos usarem. Naturalmente, no instante em que algum desenvolvedor decidir experimentar algo radical, que faça a API ou o servidor falhar, todos os demais desenvolvedores precisarão interromper seu trabalho também. Você quer que seus desenvolvedores se sintam à vontade para fazer experimentos porque um hacker certamente se sentirá à vontade para fazer isso. Criar um ambiente restrito, em que todos tenham que jogar de acordo com as regras, é quase certo que ocultará brechas de segurança que um hacker usará para invadir seu servidor.

A técnica de sandboxing possibilita fazer um desenvolvimento rápido, diminuindo as chances de um servidor falhar. Se a API tiver problemas, em geral você poderá se recuperar rapidamente. O ambiente de sandbox também facilitará simular todo tipo de condições que possam ocorrer no mundo real, por exemplo, falta de recursos ou recursos totalmente ausentes. Entretanto, você ainda terá normalmente uma única cópia da API executando, portanto não terá um ambiente de desenvolvimento tão livre assim.

Usar um ambiente virtual permite que cada desenvolvedor (ou grupo de desenvolvedores) tenha uma cópia individual da API. É claro que você usará mais recursos do servidor para ter esse cenário, mas a questão é que você acabará com um ambiente em que nenhum desenvolvedor poderá se intrometer nos esforços de programação de outros desenvolvedores. Todos estarão isolados dos demais. Se um desenvolvedor achar que pode ser uma boa ideia atacar a API de determinada maneira e o ataque for bem-sucedido (causando falha na API ou no ambiente virtual), esses esforços não afetarão mais ninguém. A questão é ter um ambiente em que os desenvolvedores se sintam à vontade para fazer o que lhes vier à mente em busca das soluções perfeitas para suas aplicações.

O custo-benefício a ser considerado nesse caso é pesar recursos *versus* flexibilidade. A abordagem com sandboxing é mais amigável aos recursos e, provavelmente, é melhor opção para empresas menores, que podem não ter muitos recursos de hardware disponíveis. O ambiente virtual é uma abordagem melhor para empresas que estejam trabalhando com APIs maiores, que exijam a atenção de vários desenvolvedores. Evitar que os desenvolvedores fiquem no caminho uns dos outros torna-se uma

consideração importante nesse caso, e o uso de hardware adicional provavelmente não será um grande problema.



Pense de forma mais ampla quando se tratar de aquisição de hardware para desenvolvimento e testes. O sistema que já não funciona bem para uso geral pode ainda ter um tempo de vida suficiente para ser usado no desenvolvimento, em especial se seu objetivo é testar em ambientes restritos. Em suma, não pense sempre em hardwares novos quando se tratar de configurar um ambiente de testes – às vezes, hardwares mais antigos também funcionam bem e talvez você tenha muitos deles à mão como resultado de upgrades em equipamentos.

Garantindo a melhor integração possível

Um problema com testes de integração em muitos casos é que o teste procura resultados positivos em vez de procurar resultados negativos. Os desenvolvedores querem verificar se:

- a API funciona de acordo com a especificação;
- a funcionalidade da API atende às expectativas de qualquer lógica de negócios que a API deva garantir.

O fato é que há uma série de soluções que você pode usar para testes de integração que também ajudarão a encontrar problemas de segurança em potencial. As próximas seções descrevem as técnicas mais populares: sandboxing, criação de um ambiente virtual, uso de simulação (mocking) e contar com virtualização de API.

Testes de integração com sandboxing

O uso de um ambiente com sandbox possibilita realizar testes próximos à realidade. Em alguns casos, você poderá obter falso-positivos quando o ambiente com sandbox não corresponder ao ambiente do mundo real de alguma forma. Além disso, usar o ambiente com sandbox normalmente não mostra as verdadeiras características de velocidade da API porque há outra camada de software entre a API e a aplicação. Problemas de velocidade tornam-se exagerados quando muitos desenvolvedores começam a acessar a API com requisições em um ambiente que não corresponda ao ambiente de produção.

Se descobrir que os testes criados por você geram resultados não confiáveis ou



inconsistentes, você precisará criar um ambiente melhor para testes. Quando não for possível fornecer valores específicos e obter saídas específicas, o processo de testes gera falso-positivos que poderiam exigir dias para serem solucionados. Falso-positivos são especialmente problemáticos quando trabalhamos com problemas de segurança. Você não pode ter um conjunto particular de condições que gere um problema de segurança uma vez e não na próxima.

Testes de integração com virtualização

O uso de um ambiente virtual como alternativa permite fazer os mesmos testes próximos à realidade. Por causa do modo como um ambiente virtual funciona, é possível simular o ambiente de produção com mais facilidade, portanto você não terá tantos falso-positivos. A velocidade dos testes normalmente não é um problema porque cada desenvolvedor tem um ambiente virtual privado para usar. No entanto, a menos que você tenha o tipo certo de configuração, não poderá realizar testes nem de velocidade da aplicação nem de carga de forma confiável porque o ambiente virtual tende a distorcer os resultados. A recuperação após um teste de integração com falha é definitivamente mais rápida em um ambiente virtual.

Simulação para testes de integração

Alguns desenvolvedores usam simulação (mocking) para testes de integração de API. Uma API simulada aceita entradas e fornece saídas conforme um script. Em outras palavras, a API não é real. O propósito da API simulada é testar a parte da aplicação separadamente da API. Usar uma API simulada permite começar a criar e a testar a aplicação antes que a equipe implemente a API.

A vantagem da simulação é que qualquer teste é executado rapidamente. Além do mais, a resposta que a aplicação recebe é exata e inalterável, de modo diferente de uma API de verdade, em que as respostas poderiam possivelmente variar. (Um simulador [mock] não realiza nenhum processamento do mundo real, portanto não há motivo para respostas variadas.) Contudo, erros no simulador podem criar falso-positivos quando a aplicação é testada.

Há exemplos de APIs simuladas online. Por exemplo, se quiser testar sua

aplicação baseada em REST, você pode usar o Mocky (<http://www.mocky.io/>) para executar a tarefa. Use o Mocky para criar uma resposta previamente definida que uma aplicação pode consultar, conforme mostra a figura 10.1.

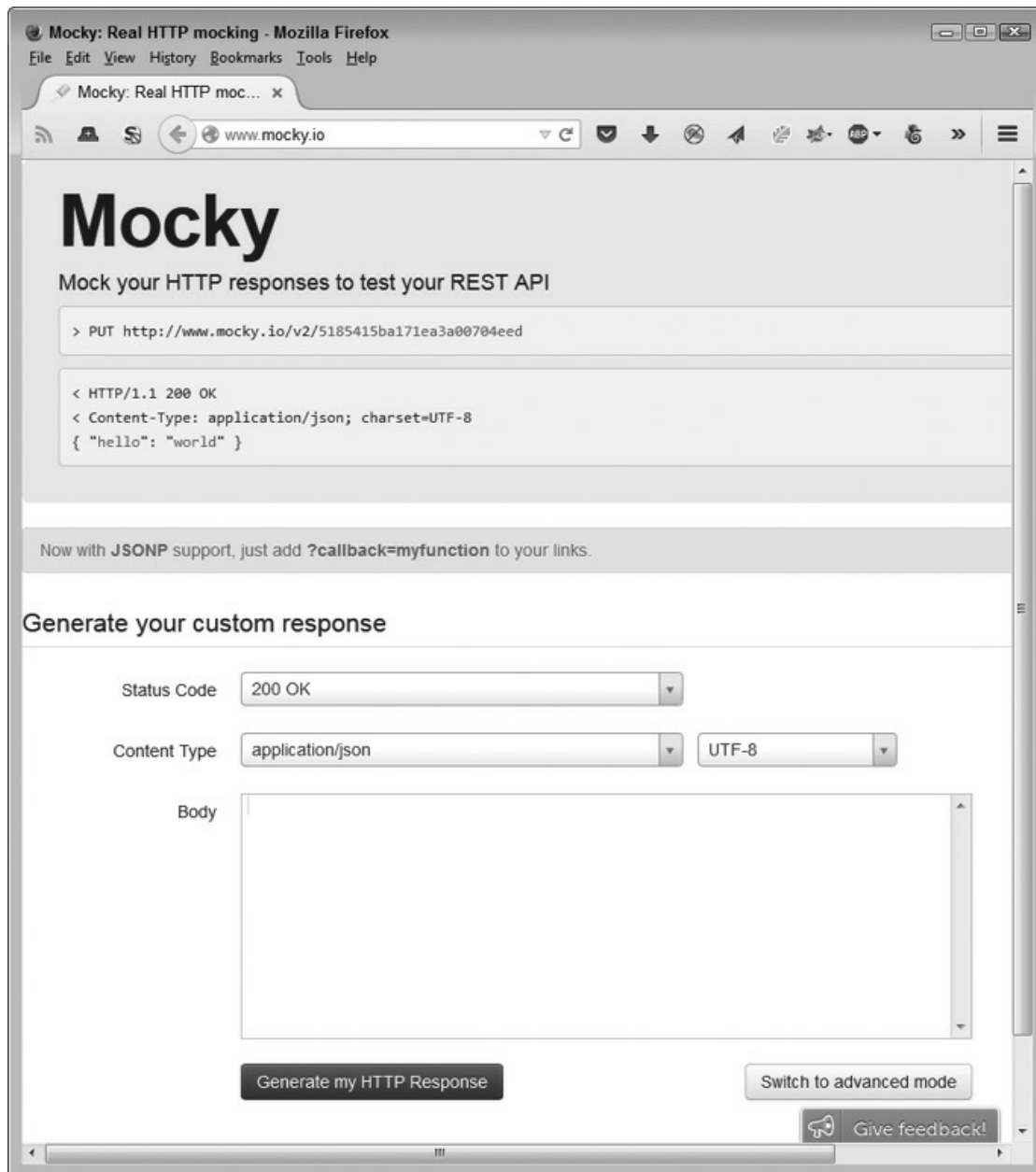


Figura 10.1 – O Mocky oferece um formulário simples que você deve preencher para configurar os testes.

O Mocky permite criar vários tipos de resposta também. Você não só controla a resposta HTTP, como também controla o tipo e o valor da resposta.

Uma série de simuladores oferece suporte tanto para SOAP quanto para REST. Por exemplo, mockable.io/ (<https://www.mockable.io/>) tem suporte tanto para SOAP quanto para REST, como mostra a figura 10.2. Para usar esse serviço, você deve criar uma conta. Uma conta para testes (trial) permite realizar testes, mas qualquer dado que você gerar não será privado. Para criar uma configuração privada, você deve adquirir uma conta paga.

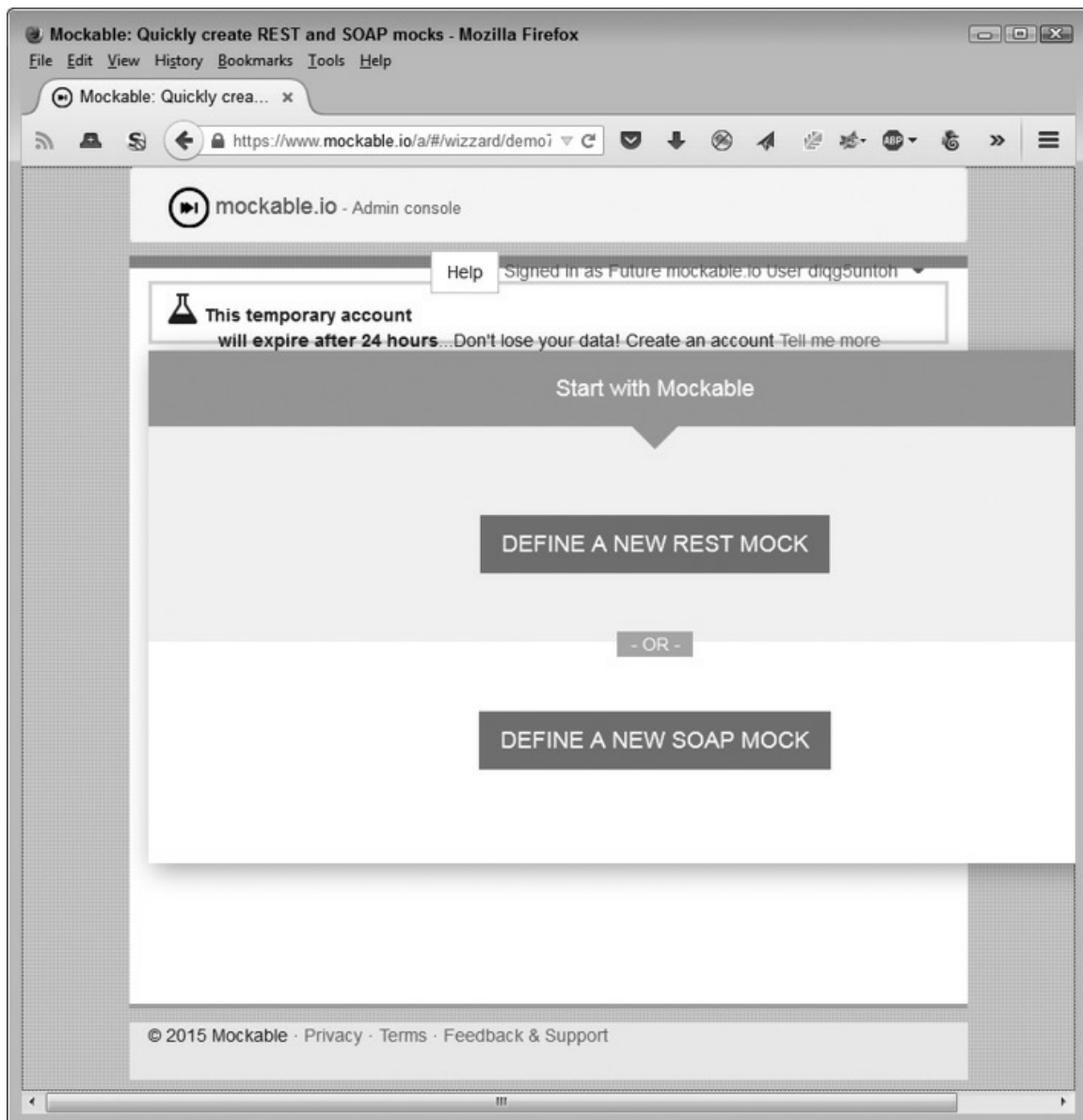


Figura 10.2 – Use simuladores como o mockable.io quando precisar de suporte para SOAP.

Nesse caso, depois de selecionar o tipo de simulador desejado, você deve

preencher um formulário para executar o processo propriamente dito de testes. Como mostra a figura 10.3, a lista de parâmetros do simulador que você pode alterar é extensa.

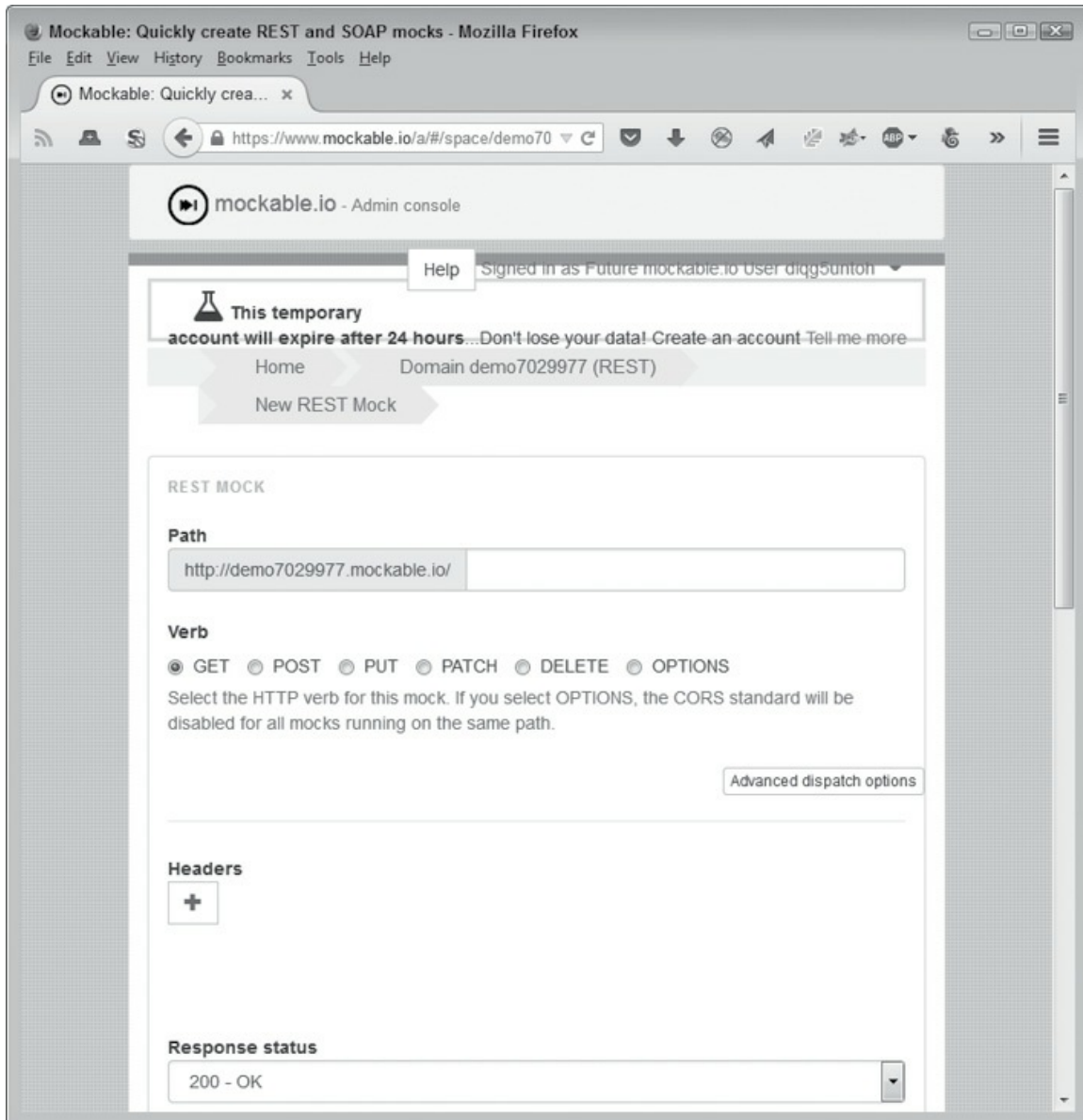


Figura 10.3 – Alguns simuladores oferecem uma boa dose de controle sobre a interface de testes.

Nesse caso, você também pode selecionar itens como o verbo usado para criar a chamada. A questão principal é que um simulador oferece um ambiente em que você fornece uma entrada específica de forma específica e obtém um resultado específico de volta.

Os simuladores mais sofisticados, como o Apiary (<https://apiary.io/>), oferecem uma variedade de serviços para facilitar o processo de simulação. Você pode usar esse serviço para gerar documentação, integrar exemplos de código, executar tarefas de depuração e automatizar testes. Naturalmente, à medida que a complexidade do simulador aumentar, o mesmo acontecerá com o preço.

Integração usando virtualização de API

Um ambiente de sandbox possibilita fazer algo que normalmente você não faria facilmente – virtualizar uma API ausente. É totalmente possível que seu cronograma de desenvolvimento não coincida com os cronogramas de alguns dos terceiros dos quais você depende para obter serviços. Quando esse problema ocorre, as empresas muitas vezes precisam esperar ansiosamente pela API da qual dependem. É claro que é sempre possível reinventar a roda e criar sua própria versão da API, mas a maioria das empresas não tem essa opção e, realmente, não é uma boa escolha, mesmo quando você tem os recursos necessários.



Jamais confunda virtualização de API com virtualização de servidor ou com o uso de máquinas virtuais. Nos dois últimos casos, você cria todo um sistema que imita todas as partes de uma plataforma padrão como uma entidade virtual. A virtualização de API simula apenas a API. Você não pode usá-la para realizar tarefas como executar diretamente uma aplicação.

Virtualização é o processo de criar uma representação de uma API que você pode usar para testes e outros propósitos. A representação atua como uma espécie de caixa-preta, que você pode usar em vez de ter a API verdadeira. Em última instância, essa caixa-preta oferece acesso às APIs de verdade, mas também poderia prover acesso a um simulador. A questão é que a camada de virtualização de API oferece um meio consistente de interagir com a API e integrar uma aplicação com ela, mesmo que a API não esteja totalmente concluída. A figura 10.4 mostra a aparência da camada de virtualização de API.

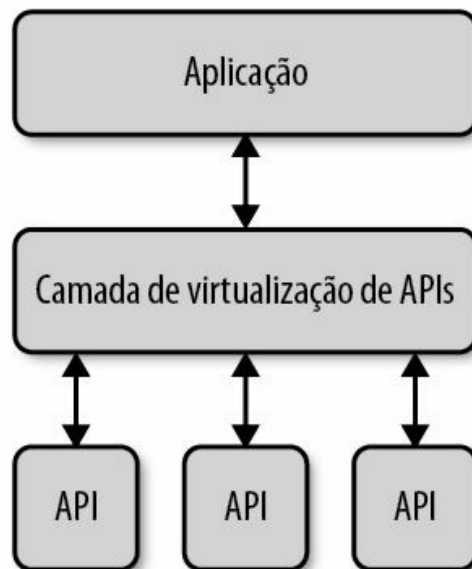


Figura 10.4 – A camada de virtualização de APIs separa a aplicação das APIs.

O uso de virtualização de APIs deixa sua aplicação mais robusta contra os tipos de falhas que os hackers amam. A camada de virtualização permite que a aplicação continue executando mesmo quando a API em si não esteja funcionando corretamente. Por exemplo, você poderia criar um simulador para substituir a API quando uma falha ocorrer e a aplicação simplesmente continuará executando. O usuário pode nem mesmo perceber qualquer diferença. Mais tarde, quando a API estiver online novamente, você poderá executar qualquer recuperação necessária em background. A questão é que um hacker não conseguirá fazer a aplicação falhar simplesmente sobrecarregando a API.

Em alguns casos, você pode criar um ambiente melhor ainda combinando um produto para sandbox com virtualização de APIs. Por exemplo, a SmartBear oferece dois produtos: o VirtServer e o Ready API! (<http://smartbear.com/product/ready-api/overview/>), que permite criar uma abordagem de duas camadas para trabalhar com APIs (leia mais sobre ele em <http://smartbear.com/product/ready-api/servicev/features/share-virtual-services/>). A ideia é ter uma flexibilidade adicional quando trabalhar com interações complexas de API.

Verificando se a API se comporta quando houver carga

Testes de carga são importantes por motivos de segurança porque os hackers muitas vezes sobrecarregam uma API ou uma aplicação para fazê-la falhar de maneira específica. Depois que a API ou a aplicação falhar, o hacker usa vários truques para utilizar o software prejudicado como ponto de entrada para a rede. Consequentemente, saber como sua API vai se degradar quando houver carga é essencial, pois você vai querer que ela falhe elegantemente e de um modo que não abra brechas enormes para o hacker poder entrar. É essencial ter em mente que toda API vai falhar em algum momento se você sobrecarregá-la o suficiente. Apesar das promessas que você pode ter lido no passado, não há nenhuma maneira de fazer um software escalar até o ponto em que ele possa aceitar uma carga infinita (supondo que seja sequer possível testar algo como isso).

Para efetuar um teste de carga, você deve ser capaz de simular uma carga, o que normalmente significa acrescentar softwares ao sistema que farão as chamadas necessárias, reduzirão os recursos e executarão logging conforme necessário. A maioria dos desenvolvedores usa um ambiente com sandbox para testes de carga porque ele proporciona o ambiente de teste mais realista possível. Procure ferramentas que funcionem em todas as plataformas às quais você deva dar suporte. Alguns produtos, como o LoadUI (<http://www.loadui.org/>), oferecem versões para testes em Windows, Mac e Linux. Antes de adquirir qualquer ferramenta, experimente fazer download da versão de teste (trial) para ver se ela funciona conforme prometido.

Um ponto de partida para qualquer teste de carga de API é determinar como o teste será feito. Você precisa obter dados estatísticos que informem como a API deve ser sobrecarregada para simular um ambiente do mundo real. Para garantir que a API funcione conforme anunciado, você precisa determinar:

- o número médio de requisições que a API recebe por segundo;
- o número máximo de requisições que a API recebe por segundo;
- a distribuição do throughput por endpoint (loais que fazem chamadas à API);
- a distribuição do throughput por usuário ou grupo de trabalho.

Também é importante decidir como o tráfego será gerado para testar a API. Você pode decidir começar de modo simples. No entanto, é importante testar a API usando vários tipos de tráfego para garantir que ela se comportará bem em todas as circunstâncias:

- Geração repetitiva de carga (em que o software de teste usa a mesma sequência de requisições)
- Padrões de tráfego simulado (uma versão aleatória da geração de carga repetitiva ou uma reprodução dos dados de log de acesso à API)
- Tráfego real (em que você tem um grupo de teste acessando a API com requisições de verdade)

Após confirmar que a API funciona em condições-padrão, é preciso variar essas condições para simular um ataque de um hacker. Por exemplo, um hacker poderia decidir inundar a API com uma requisição específica, esperando que ela falhe de determinada maneira. O hacker também poderia optar por ficar acrescentando mais e mais zumbis em um ataque DDoS (Distributed Denial-of-Service attack, ou Ataque Distribuído de Negação de Serviço) que simplesmente sobrecarregue a API até o ponto em que ela falhe. Seja criativo. É importante determinar o que causará uma falha em sua API e fazê-la falhar e então ver como a API falha. Saber como a API falha ajudará você a determinar o nível de risco que cada tipo de hack representa, de modo que você possa se preparar melhor para as consequências.

Mantendo a API segura contra hackers

Após ter executado todos os demais testes descritos neste capítulo, você pode achar que sua API é segura. O problema é que você não pensou em todas as maneiras com que o hacker atacará. Se alguém estiver realmente determinado a causar uma falha em sua API, essa pessoa achará uma maneira de fazer isso. O capítulo 9 discute a necessidade de pensar como um hacker. É claro que esse é um bom ponto de partida, mas depois que você tiver ideias sobre como alguém poderia causar uma falha em sua API, é preciso testá-las. A única maneira de fazer isso de forma segura é tentar criar uma zona de segurança para a API.

Nesse caso, testar em um ambiente tanto com sandbox quanto virtual ajudará a obter os melhores resultados. Cada ambiente oferece vantagens específicas ao testador de software, que ajudarão a garantir que a API se comportará bem em produção.



Testar sua aplicação e as APIs associadas em um ambiente de produção nunca é uma boa ideia, a menos que isso seja absolutamente necessário. Muitos desenvolvedores tentaram fazer isso e descobriram que os procedimentos de testes, na verdade, causam danos aos dados que a aplicação deveria proteger. Além disso, geralmente você não será capaz de agendar um horário para testes que seja conveniente aos usuários. Como os usuários acrescentam variações ao ambiente de testes, você precisa testar em um horário em que os usuários não estejam ativamente envolvidos na execução de tarefas. Caso contrário, você não poderá ter certeza se um teste que falhou é resultado do teste ou de uma entrada de usuário.

Desenvolvendo com uma API em sandbox

Visualizar uma sandbox de API é relativamente simples. É a mesma ideia por trás de uma caixa de areia (sandbox) para crianças. A API tem acesso a tudo que estiver contido na sandbox, mas não poderá acessar nada fora dela. Manter uma API em uma sandbox tem implicações importantes para a segurança porque é menos provável que uma API, mesmo que esteja sendo atacada por um hacker, possa fazer algo para prejudicar a aplicação ou seus dados. Ao manter a API isolada, ela pode causar danos somente aos recursos que já foram separados para ela. Isso quer dizer que a API é efetivamente incapaz de causar danos de uma maneira não controlada, a menos que o designer implemente a sandbox de forma incorreta ou ineficiente.

Nada vem de graça. Ao colocar uma API em uma sandbox, também é possível que a API não funcione mais como esperado ou que funcione lentamente no ambiente com menos recursos. É essencial criar a sandbox da API de modo que a funcionalidade da API permaneça robusta. Criar uma sandbox de API balanceada pode ser difícil e exige um conhecimento profundo de como exatamente a API funciona com a aplicação para executar tarefas específicas. (Não é necessário entender todos os detalhes da API porque é improvável que a aplicação a utilize totalmente.)

Muitos artigos criam muito alvoroço em torno das sandboxes, mas na verdade não é muito diferente de trabalhar com qualquer sistema operacional moderno. Houve uma época em que a aplicação tinha acesso completo a todo o computador. Atualmente, uma aplicação tem acesso controlado aos recursos do sistema, que incluem memória e tempo de processamento. A aplicação precisa compartilhar recursos com outras aplicações executando no sistema, portanto o sistema operacional controla o acesso a esses recursos. O que você tem com uma sandbox é uma camada adicional de controle, mas o controle é semelhante em funcionalidade e efeito a qualquer outro tipo de controle de aplicações.

Após determinar bons motivos para desenvolver uma sandbox de API, é hora de desenvolver uma para usá-la com sua aplicação. Na verdade, você precisa de uma sandbox para cada API que usar em sua aplicação. Algumas aplicações usam uma boa quantidade delas e contaminar o ambiente de uma API com problemas gerados por outra API não ajuda. Manter tudo separado e incapaz de destruir outros ambientes é sempre uma boa ideia. Em alguns casos, pode ser que você precise ser criativo quanto ao modo de criar uma aplicação para que as APIs continuem funcionando como deveriam, mas a ideia de manter as APIs em ambientes totalmente separados e controlados é ótima.

Origem da sandbox

Talvez você ache que a sandbox seja uma ideia relativamente nova. No entanto, o conceito de sandbox existe há um bom tempo, desde os anos 1970, na forma do sistema operacional Hydra. O sistema operacional multiprocessador da Carnegie Mellon para minicomputadores tinha uma sandbox em que os desenvolvedores podiam fazer experimentos com aplicações de IA (Inteligência Artificial) com relativa segurança (http://research.microsoft.com/en-us/people/gbell/Computer_Structures_Principles_and_Examples/csp0366.htm).

Cada aplicação executava como uma aplicação de usuário, o que significava que ela não tinha acesso às funcionalidades de baixo nível do sistema operacional que pudessem provocar uma falha no sistema.

Outros desenvolvedores perceberam a funcionalidade oferecida pelo Hydra e a reproduziram em outros ambientes. Por exemplo, a Sun disponibilizou o conceito de Dynamic System Domains (Domínios de Sistemas Dinâmicos,

<http://www.filibeto.org/~aduritz/truetrue/e10000/dynamic-sysdomains.html>) para oferecer proteção contra erros e isolamento de aplicações (entre outros objetivos). O FreeBSD na realidade trancava as aplicações colocando-as em *jails*, isto é, celas (<https://www.freebsd.org/doc/handbook/jails.html>). Desse modo, a ideia de sandbox está presente há muito tempo – contudo, usá-las em APIs é relativamente novo.

Atualmente, muitas aplicações executam em uma sandbox. Por exemplo, em 2012, a Apple começou a exigir que os desenvolvedores incluíssem recursos de sandbox como parte das aplicações que estivessem na Mac App Store da Apple (<https://developer.apple.com/app-sandboxing/>). O navegador Chrome (<https://tools.google.com/dlpage/res/chrome/en-GB/more/security.html>) do Google, o Internet Explorer (<http://securityintelligence.com/internet-explorer-ie-10-enhanced-protected-mode-epm-sandbox-research/>) da Microsoft e o Safari (<http://www.cnet.com/news/safari-matches-rivals-with-sandboxed-flash-for-better-security/>) da Apple têm sandboxes. O único navegador sem uma sandbox é o Firefox (<http://www.extremetech.com/computing/178587-firefox-is-still-the-least-secure-web-browser-falls-to-four-zero-day-exploits-at-pwn2own>) do Mozilla.

Usando uma solução de prateleira

Criar sua própria sandbox poderia ser bem divertido, mas, provavelmente, seria um desperdício de tempo. Você deve focar em sua aplicação em vez de focar em criar um ambiente para usá-la de forma segura. É por isso que você precisa de uma boa solução de prateleira (off-the-shelf) em que possa confiar. A lista a seguir descreve algumas das soluções disponíveis para sandbox:

AirGap (<https://spikes.com/isolation-is-the-new-prevention.html>)

Essa é uma ideia interessante. Em vez de permitir que o navegador execute no sistema do cliente, ele executa em um servidor na nuvem. O usuário conta com uma aplicação de visualização, que oferece isolamento do navegador. Você vê a janela do navegador e interage com ela como faria normalmente, mas a única aplicação que está executando no sistema do cliente é o visualizador. O fornecedor promete isolamento físico, de conexão, de sessão e de malwares por meio dessa abordagem. Você precisaria testá-la com sua aplicação para ver se a abordagem com o visualizador realmente funcionaria.

Uma solução como o AirGap funciona bem para muitos usuários. No entanto,



you should consider the place where the browser is running. If you are in a regulated market or your application deals with sensitive information, AirGap will not serve you. The browser really needs to run on the host system to function correctly and maintain the confidentiality that the legal system (or common sense) requires.

Sandboxie (<http://www.sandboxie.com/>)

You can use Sandboxie to run any application on the host system, not just browsers. This means you can use Sandboxie to run any type of task that other solutions might not handle. Since it runs on the client system, all its sensitive data remains completely protected from prying eyes. The provider offers a trial version for you to test. When you opt to purchase a license, you have the option to acquire a home (personal use) or small business (for small businesses) or an enterprise (corporate) version. Only the enterprise version has a support package. Sandboxie only works with the Windows operating system.

Spoon.net (<https://spoon.net/browsers>)

Sometimes, you need to test many different browsers, but downloading and installing all of them on your system is difficult. Configuring all these browsers as needed also implies additional steps. You can avoid all this difficulty by accessing the browser through the environment provided by Spoon.net. To use this product, you must install a plugin for your browser and then choose the browser you want to use, as shown in Figure 10.5. The browser options are not limited to those that your platform normally accepts. For example, even if you are on a Windows system, you will be able to load Safari or Firefox Mobile to perform the necessary tests.

Using sandboxes from other providers

There are situations where you use a sandbox, whether you want to or not. For example, when writing an application that uses the eBay API, you must first use the sandbox (<https://go.developer.ebay.com/developer-sandbox>) for

escrever o código de sua aplicação e então testá-lo. Somente depois que o eBay certificar a aplicação é que você poderá executá-la no ambiente de produção. Por causa dos altos riscos envolvidos no ambiente da API, provavelmente você perceberá que passará muito mais tempo usando sandboxes de terceiros para testar o seu código.

O acesso à maioria dessas sandboxes depende de chaves especializadas. Fornecedores como o eBay querem saber quem está usando sua sandbox e com qual finalidade. A chave identifica você unicamente e é necessário tê-la para fazer uma chamada de sandbox. Quando trabalhar com o eBay, é fácil obter a chave, criar uma conta de usuário e então entrar na sandbox para começar a executar testes na aplicação, como mostra a figura 10.6.

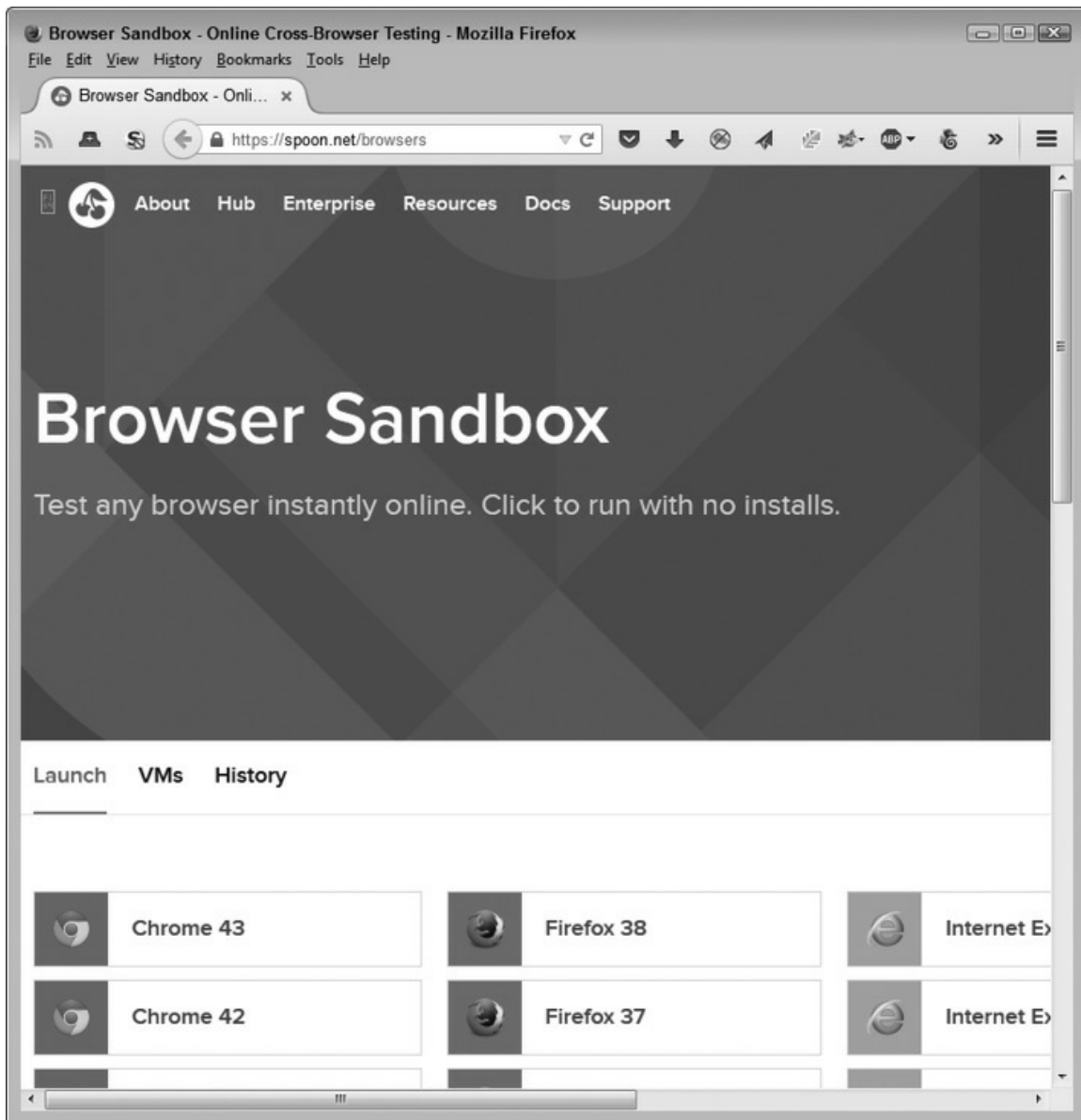


Figura 10.5 – Spoon.net permite usar qualquer um dos navegadores mais comuns.

Pode haver um problema quando você precisar usar várias APIs em sua aplicação e cada uma dessas APIs tiver seu próprio ambiente de sandbox. Tentar fazer com que a aplicação seja certificada quer dizer fazer com que cada um dos fornecedores a aprove. Não se esqueça de criar um plano para conseguir que sua aplicação seja aprovada por todos os fornecedores antes de começar a implementação. Em alguns casos, talvez você precise dividir sua aplicação em partes possíveis de serem testadas separadamente para obter a aprovação necessária.

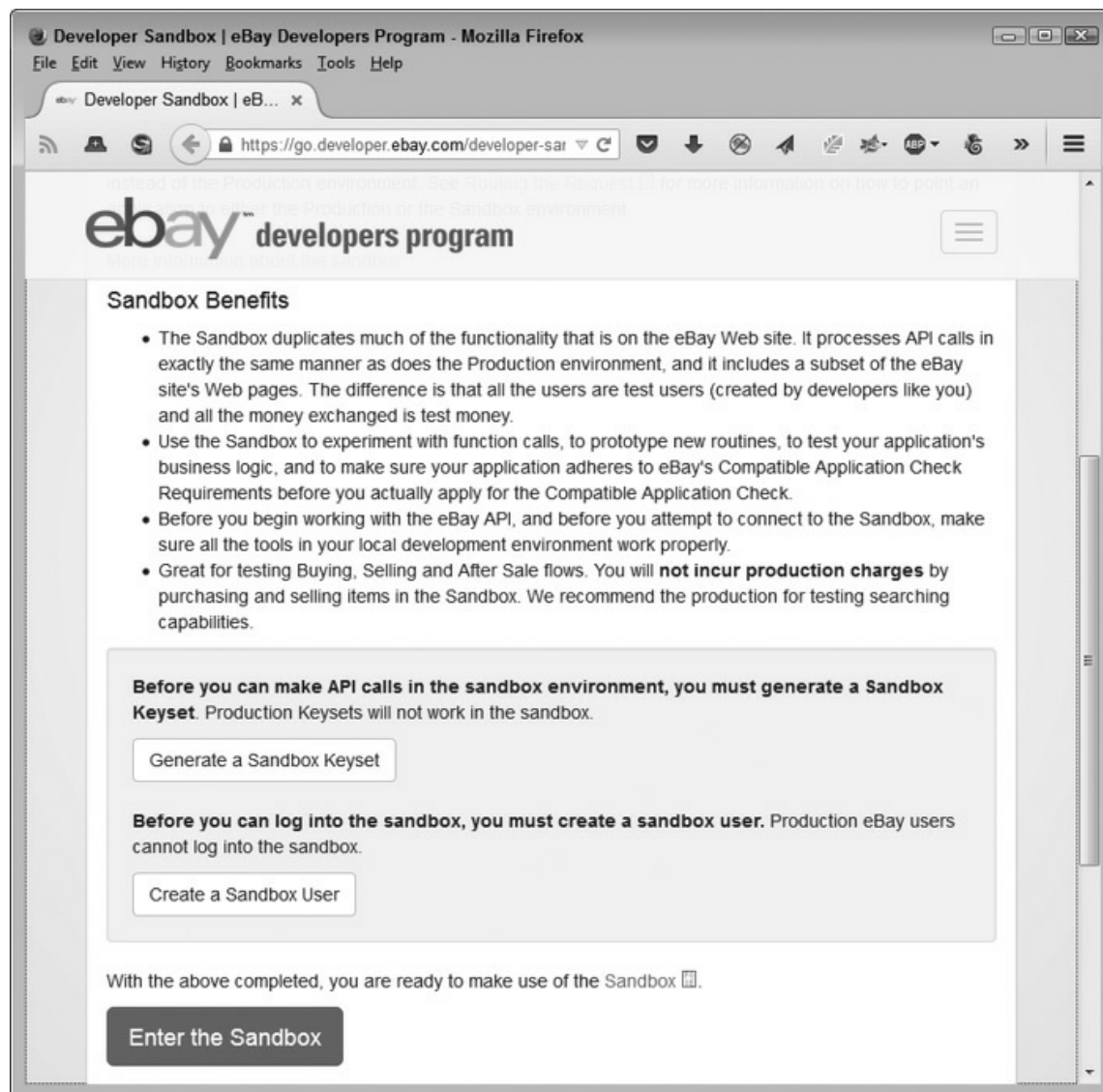


Figura 10.6 – A maioria dos fornecedores exigirá algum tipo de controle de acesso para usar suas sandboxes.

O uso de várias APIs em uma única aplicação tornou-se tão comum e tão complexo que muitos desenvolvedores agora lançam mão de agregação de APIs para ajudar. O uso de agregação de APIs permite reduzir a complexidade com a criação de mashups de APIs. Trabalhar com agregação de APIs está fora do escopo deste livro, mas você pode encontrar informações excelentes sobre ela nos artigos “Api Aggregation: Why It Matters and Eight Different Models” (Agregação de APIs: por que ela é importante e oito modelos diferentes, <http://www.programmableweb.com/news/api-aggregation-why-it-matters-and-eight-different-models/2013/12/13>) e “Extending REST APIs with API Aggregator” (Estendendo APIs REST com o Agregador de APIs, <http://tech.3scale.net/2013/04/18/accelerate-your-mobile-api-with-nginx-and->



lua/). É claro que há muitos outros recursos que você pode consultar para obter informações sobre agregação, mas esses artigos são um ótimo ponto de partida.

Considerando ambientes virtuais

Às vezes, sua API exige um ambiente virtual para funcionar. Um ambiente virtual dá a impressão de ser o ambiente verdadeiro sem deixar que a API execute tarefas que poderiam causar problemas. Ao longo do capítulo, vimos comparações entre sandboxing e ambientes virtuais. Do ponto de vista de segurança, ambos oferecem proteção contra atividades de hackers, mas fazem isso de maneira significativamente diferente. No entanto, a questão é sempre separar a API, a aplicação, os componentes da aplicação ou qualquer outro item de software que você queira isolar do sistema operacional. O isolamento permite interceptar e limpar dados, assim como manter o software sob controle de modo que os hackers causem menos danos. As próximas seções discutem os ambientes virtuais com mais detalhes

Definindo o ambiente virtual

Um ambiente virtual pode se referir a qualquer software que ofereça meios para isolar outros softwares. Pense nisso como se fosse um contêiner, pois é isso que ele é. Seções anteriores do capítulo descreveram vários tipos de ambientes virtuais. Contudo, esta parte do capítulo descreve o tipo de ambiente virtual que você usa para desenvolvimento, em vez de utilizá-lo para executar uma aplicação como um usuário.

O motivo básico para usar virtualização é isolar todo o ambiente de

desenvolvimento ou parte dele para que você possa brincar com a aplicação. Encontrar problemas de segurança significa realmente fazer experimentos com o software de maneiras que você jamais usaria em um sistema operacional-padrão (em especial, não em um conectado à rede, em que seu erro poderia derrubar todos).



Algumas linguagens têm suas próprias ferramentas para ambientes virtuais. Por exemplo, usuários de Python têm à disposição uma série de ferramentas para criar ambientes virtuais. Essas ferramentas facilitam instalar e configurar ambientes específicos para linguagens específicas. Você pode ler sobre os recursos oferecidos por Python em <http://docs.pythonguide.org/en/latest/dev/virtualenvs/>. A questão é procurar ferramentas específicas de linguagens sempre que for possível para criar ambientes virtuais projetados para funcionar com essa linguagem.

Ambientes virtuais também podem incluir muito mais que uma sandbox inclui. É possível criar ambientes virtuais com sistemas operacionais específicos, ferramentas de desenvolvimento, instalações de aplicações e navegadores. O ambiente virtual emula todas as partes da experiência de desenvolvimento para que você possa criar uma configuração de servidor Apache usando um ambiente virtual e uma configuração de servidor IIS usando outra. Os dois ambientes virtuais podem existir no mesmo computador sem problemas com conflitos.

Diferenciando ambientes virtuais de sandboxing

Os ambientes virtuais e as sandboxes têm muitas semelhanças. O restante do capítulo descreveu as semelhanças mais comuns, como manter sua API segura. As diferenças básicas entre a maioria dos ambientes virtuais e o sandboxing estão no fato de os ambientes virtuais serem:

Repetíveis

Quando o arquivo de ambiente virtual é transferido para outro sistema, esse sistema executa o ambiente virtual exatamente como executaria no sistema original. Isso quer dizer que todo desenvolvedor vê a aplicação do mesmo modo, usando um ambiente de desenvolvimento predefinido, portanto o resultado visto em um sistema é repetível em outro sistema. Diferenças entre configurações dificultam (às vezes, impossibilitam) verificar a

presença de problemas de segurança.

Móveis

Conforme o software de virtualização que você usar, é possível transferir o ambiente virtual praticamente para qualquer lugar. Desde que o desenvolvedor que recebe o arquivo de ambiente virtual tenha o software correto, é possível recriar o ambiente virtual exato necessário para trabalhar com o projeto.

Recuperáveis

Os ambientes de desenvolvimento virtuais dependem de um tipo especial de arquivo que define como criar a emulação necessária. Se seus experimentos provocarem uma falha no ambiente virtual, tudo que você precisa fazer é fechar essa cópia do ambiente virtual e iniciar outra. A recuperação só demora o tempo necessário para o software carregar, o que pode ocorrer em apenas alguns segundos.

Recriáveis

Há ocasiões em que você precisará de várias cópias do mesmo projeto executando em seu sistema. Você pode criar quantas cópias de um ambiente virtual forem necessárias para seus propósitos. A natureza especial dos ambientes virtuais implica que toda cópia que você criar começará exatamente com os mesmos recursos, os mesmos softwares carregados, o mesmo ambiente, e assim por diante. Consequentemente, toda cópia é um clone de todas as demais cópias de seu sistema.

Implementando a virtualização

Um dos ambientes de desenvolvimento virtuais mais interessantes é o Vagrant (<https://www.vagrantup.com/>). Esse produto em particular executa em qualquer sistema Windows, OS X ou Linux comum (há versões de Linux que não funcionam com o Vagrant, portanto não se esqueça de conferir a lista para ter certeza de que o Vagrant tem suporte para sua distribuição de Linux). Para usar esse produto, você deve instalá-lo em seu sistema, configurar o tipo de ambiente que você quer usar para o desenvolvimento e colocar o arquivo resultante junto com o código de seu projeto. Sempre que quiser iniciar o

ambiente de desenvolvimento necessário a um projeto em particular, basta executar o comando `vagrant up` e ele configurará o ambiente correto para você. O produto criará o mesmo ambiente, não importa para onde você mova os arquivos do projeto, de modo que você poderá criar um único ambiente para todos de sua equipe de desenvolvimento usarem. O Vagrant é a melhor opção para o desenvolvedor individual ou uma empresa de pequeno porte – ele não é caro e é fácil de usar.

Outro produto útil para ambientes virtuais é o Puppet (<https://puppetlabs.com/>). Esse produto depende do VMware (<http://www.vmware.com/>) com o Puppet executando as tarefas de gerenciamento. A combinação de Puppet com o VMware é mais adequada a ambientes corporativos, pois o VMware vem acompanhado de uma lista enorme de ferramentas que você pode usar com ele, além de oferecer suporte para instalações maiores.

Contando com a virtualização de aplicações

Existem vários tipos de ambientes virtuais para atender praticamente a qualquer necessidade em que você possa pensar. Por exemplo, o Cameo (<http://www.cameyo.com/>) permite compactar qualquer aplicação em um único executável (arquivo `.exe`) que você pode copiar para qualquer computador Windows e simplesmente executá-lo sem qualquer instalação. Se precisar dar suporte a outras plataformas, você poderá copiar a aplicação para os servidores em nuvem do Cameo e executá-la a partir de lá.

A virtualização de aplicações é uma solução de empacotamento. Você cria um ambiente virtual que atende especificamente às necessidades apenas daquela aplicação. A figura 10.7 mostra a aparência do ambiente virtual de uma aplicação. No que concerne à aplicação, ela está funcionando em seu ambiente nativo, mesmo que esteja usando um sistema operacional totalmente diferente para executar as tarefas.

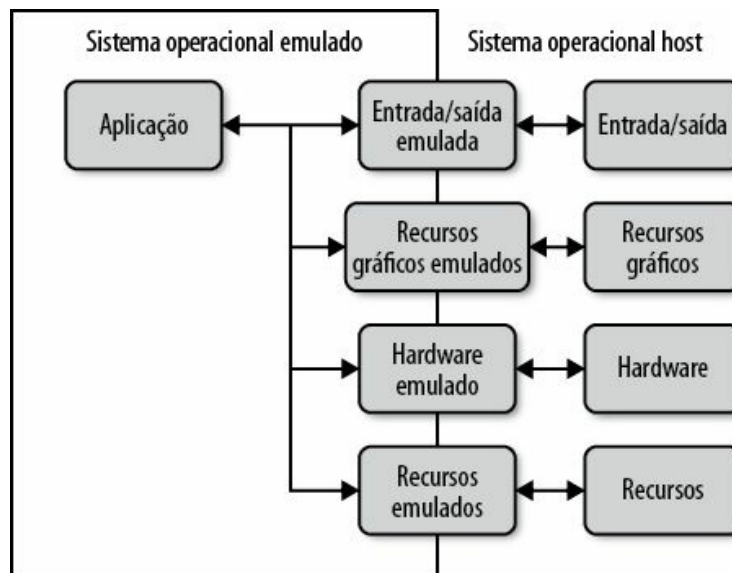


Figura 10.7 – A virtualização de aplicações é um tipo de tecnologia de empacotamento.

Outro produto que oferece um ambiente virtual para aplicações é o Evalaze (<http://www.evalaze.de/en/evalaze-oxid/>). Esse produto enfatiza o ambiente de sandbox que ele oferece, juntamente com a virtualização de aplicações. Você pode usá-lo para executar uma aplicação a partir de qualquer lugar, incluindo de pen drives. É uma ótima solução se sua aplicação exigir um navegador específico e você quiser ter certeza de que o ambiente do usuário esteja configurado corretamente.

Pesquisar um tipo apropriado de virtualização exigirá tempo e você deve conduzir sua pesquisa com a mente aberta. Algumas das soluções de aparência estranha que existem por aí podem ser exatamente o que você precisa para criar um ambiente seguro para sua aplicação.

¹ N.T.: “Em economia, aprisionamento tecnológico corresponde ao termo inglês *vendor lock-in*. O aprisionamento tecnológico decorre de particularidades em produtos ou serviços que tornam seus usuários dependentes dos fornecedores, impedindo-os de trocar de fornecedor sem custos adicionais substanciais.” (Fonte: https://pt.wikipedia.org/wiki/Aprisionamento_tecnol%C3%B3gico)

CAPÍTULO 11

Verificando se há brechas de segurança em bibliotecas e APIs

Antes de saber se um software em particular vai se comportar de determinada maneira, é preciso testá-lo. Sim, você tem uma especificação e criou o código de acordo com ela. Você até mesmo depurou o código para garantir que não contém nenhum erro evidente. No entanto, ainda não sabe se o código funcionará conforme previsto originalmente até executar algum nível de testes nele. Testes garantem que o software se comportará conforme esperado.

Este capítulo discute os testes no sentido tradicional, mas também apresenta tipos não tradicionais de testes. Uma suíte de testes-padrão fornecerá entradas específicas e então validará as saídas esperadas. Entretanto, o mundo real nem sempre fornece o tipo de entrada que as bibliotecas e APIs esperam, portanto é importante testar esses tipos de entrada também. É claro que, quando os testes são feitos dessa maneira, você não sabe o que esperar como saída. O design da biblioteca ou da API deve conter funcionalidades que ajudem a se recuperar de entradas errôneas de modo que o sistema não cause falhas, mas você não sabe se essa funcionalidade está presente até executar o nível exigido de testes.

Testes podem e devem ocorrer em vários níveis. Os testes de unidade (unit testing), que são testes de partes individuais do código, devem ocorrer antes. Um desenvolvedor começa executando esse tipo de teste imediatamente após escrever as primeiras porções de código. A seguir, temos os testes de integração (integration testing), quando os componentes da aplicação são reunidos. Por fim, a aplicação é testada como um todo, com o software completo instalado. Todos esses níveis de teste (e outros) exigem testes de

segurança também – verificar entradas inesperadas e determinar se o código age de forma aceitável. Durante esse período, os desenvolvedores também criam um *test harness* – rotinas que automatizam o processo de testes para que ele seja mais confiável e consistente.

Cada linguagem de programação também pode ter problemas específicos que os desenvolvedores devem verificar. A última parte deste capítulo discute como verificar problemas específicos de linguagens de programação quando você executar seus testes de segurança. De modo diferente de outras áreas de teste, problemas específicos de linguagem tendem a girar em torno de segurança, pois frequentemente os hackers procuram essas diferenças como meios para fazer a aplicação falhar de forma específica, ou erros de lógica que podem ser empregados pelo hacker para ter acesso. Monitorar possíveis falhas da linguagem é tão importante quanto monitorar falhas em bibliotecas e APIs de terceiros. É preciso testar todas as partes de uma aplicação de todos os ângulos possíveis para garantir que sua aplicação se comportará de maneira previsível.

Testar além do óbvio

O código depende de procedimentos. Sim, o código pode ser orientado a eventos ou pode não incluir o conceito de estado, mas, lá no fundo, ele depende de um conjunto de passos para executar tarefas. Esse conjunto de passos é um procedimento. Se o procedimento está ou não completo não interessa a esta discussão.

Tive meu primeiro contato com testes que vão além do óbvio no ensino fundamental. É difícil me lembrar da maioria dos eventos daquela época de minha vida, mas um evento se destacou em todos aqueles anos como uma habilidade a ser lembrada para a vida toda. A professora havia pedido a todos nós que escrevêssemos um procedimento para fazer uma torrada. Parece ser um pedido bem simples. No entanto, todos falhamos. O fato é que todos pensaram em pegar o pacote de pão e colocar as fatias individuais na torradeira. Esperar que a torrada saltasse não foi um problema, nem passar manteiga nela depois.

O ponto problemático foi retirar as fatias da embalagem. Quando a professora seguiu nosso procedimento ao pé da letra, ela, é claro, tentou colocar as fatias individuais de pão com embalagem e tudo na torradeira. Todos nós havíamos feito uma suposição que significava que o procedimento tinha falhas. Os computadores são literais, assim como minha professora. Fazemos suposições sobre o que o computador fará baseados no senso comum e na experiência – nenhum dos quais está presente no computador.

Testar além do óbvio quer dizer procurar aquela suposição do pão na embalagem que todos fizeram quando o código foi desenvolvido. Brechas de segurança muitas vezes são resultado de suposições que o desenvolvedor não deveria ter feito.

Criando um plano de testes

Todo cenário de testes exige um plano de testes. Embora você queira testar além do óbvio, é preciso ter algum tipo de método estruturado para realizar os testes. Caso contrário, os testes se tornam inconsistentes e incompletos. Para ter um propósito útil, os testes precisam ser metódicos, embora devam ser flexíveis o suficiente para permitir testes adicionais à medida que se tornar evidente que serão necessários. Com isso em mente, as próximas seções ajudam a definir um plano de testes do ponto de vista de desenvolvimento.



É importante observar que os planos de teste muitas vezes têm várias fases e diretrizes. Todo stakeholder (pessoa-chave) em um esforço de desenvolvimento de aplicação vai querer realizar algum nível de testes para garantir que a aplicação atinja metas e objetivos específicos. Esses pontos de vista podem ser conflitantes, mas na maioria dos casos eles simplesmente se complementam. Por exemplo, um DBA talvez queira conferir se a aplicação interage com o banco de dados de forma consistente com as diretrizes da empresa.

Considerando metas e objetivos

Testes de aplicação não podem ser bem-sucedidos, a menos que a equipe de desenvolvimento defina tanto as metas quanto os objetivos do teste. Uma meta é tão simples quanto determinar se a aplicação atende aos requisitos técnicos e de negócios que uma empresa exige dela. Um objetivo é determinar se a aplicação é capaz de executar com sucesso um conjunto de tarefas no ambiente de negócios fornecido a ela. Por exemplo, um objetivo pode ser adicionar novos usuários em um banco de dados sem causar erros, duplicar usuários ou deixar de fora informações essenciais. As próximas seções discutem metas e objetivos a que os testes devem atender para oferecer um resultado útil.

Definindo as alternativas de testes

Testar não é a única maneira de verificar uma aplicação. A questão principal dos testes

é verificar se a aplicação funciona de determinada maneira. Outras técnicas podem atingir as mesmas metas. As alternativas mais comuns para testar são: revisão de código (code review), análise estática (static analysis), verificação de modelo (model checking) e provas (proofs). Cada uma dessas alternativas tem seu lugar como parte de sua estratégia para melhorar a segurança da aplicação em geral e garantir que a aplicação se comportará de maneira confiável.

A revisão de código (code review) é uma verificação realizada por especialistas humanos. Uma equipe percorre o código e verifica se o design e a implementação atendem aos requisitos da especificação. A revisão de código pode ajudar a localizar problemas de implementação em potencial, que métodos automatizados podem deixar passar. De fato, é totalmente possível que uma aplicação vá compilar e executar com falhas significativas que a revisão de código revelará. Você pode ler mais sobre revisão de código em https://www.owasp.org/index.php/Code_Review_Introduction.

A análise estática (static analysis) é o mesmo processo da revisão de código, mas usa ferramentas automatizadas no lugar de revisores humanos. As vantagens da análise estática são consistência e velocidade. Ferramentas automatizadas verificam todas as facetas da aplicação exatamente da mesma maneira e tendem a não cometer erros resultantes de cansaço. Além disso, ferramentas automatizadas podem ser mais rápidas que seres humanos. No entanto, essas ferramentas também podem deixar passar erros que os seres humanos veriam quase de imediato. Você pode ler mais sobre análise estática em https://www.owasp.org/index.php/Static_Code_Analysis.

A verificação de modelo (model checking) confere se as propriedades da aplicação satisfazem aos requisitos da especificação. Na maioria dos casos, isso quer dizer conferir se a aplicação oferece soluções para certos aspectos, por exemplo, algoritmos. Esse tipo de teste é automatizado, mas exige dados substanciais de entrada pelos seres humanos para ser executado. Você pode ler mais sobre verificação de modelo em <https://www7.in.tum.de/um/25/target.html>.

Provas (proofs) frequentemente aparecem como alguma forma de testes para estressar a aplicação. Uma prova verifica se a aplicação executará conforme necessário, mesmo quando cargas substanciais são impostas à aplicação.

Definindo as metas

As metas definem uma condição que uma pessoa ou entidade pretende atender. Você pode definir todo tipo de metas para testes de aplicação, como calcular o valor exato de pi em 42 picossegundos. É claro que a meta não é alcançável porque não é possível calcular um valor exato para pi. Algumas

empresas definem os mesmos tipos de meta para aplicações e ficam decepcionadas quando se torna óbvio que o processo de testes não atingiu a meta. Metas verdadeiras são possíveis de serem alcançadas usando recursos disponíveis no tempo alocado.

Além disso, uma meta deve definir uma medida para o sucesso. Não é só uma questão de saber se o teste teve sucesso – é preciso saber quão bem-sucedido foi o teste. Para obter uma resposta para quão bem-sucedido foi o teste, as metas que você especificar devem estabelecer uma medida que defina um intervalo de resultados esperado, por exemplo, a aplicação calculou o resultado correto no tempo alocado em 99% das vezes.

As metas exatas que você define para sua aplicação dependem do que você espera dela e do tempo que você tem para implementar as metas. Contudo, é possível generalizar as metas nas seguintes categorias:

Verificação e validação

O processo de verificação e validação trazem as falhas à tona. No entanto, ela também garante que a aplicação forneça a saída esperada, considerando entradas específicas. O software deve funcionar conforme definido pela especificação. Do ponto de vista de segurança, você deve testar o software tanto para entradas esperadas quanto para entradas não esperadas e conferir se ele responde corretamente em cada caso.

Abrangência por prioridade

A mais segura das aplicações no mundo teria todas as funções testadas de todos os modos possíveis. No entanto, no mundo real, restrições de tempo e de orçamento impossibilitam testar completamente uma aplicação. Para testar o software do modo mais completo possível, você deve gerar seu perfil a fim de determinar os pontos em que o software gasta a maior parte de seu tempo e concentrar seus esforços aí. Contudo, outros fatores entram em cena. Mesmo que uma funcionalidade não seja usada regularmente, você ainda precisará lhe conceder uma ampla abrangência se ela não puder falhar (uma falha teria um resultado catastrófico). A priorização da abrangência deve incluir fatores que dizem respeito somente à sua aplicação.

Equilibradas

O processo de testes deve achar um equilíbrio entre requisitos escritos, limitações do mundo real e expectativas dos usuários. Quando realizar um teste, você deve verificar se os resultados são repetíveis e independentes de quem testa. Evitar posturas tendenciosas no processo de testes é essencial. Também é possível que o conteúdo da especificação e a expectativa dos usuários não sejam totalmente compatíveis. Falhas de comunicação (ou, às vezes, sua total ausência) impedem que a especificação inclua totalmente o ponto de vista do usuário sobre o que a aplicação deve fazer. Como resultado, talvez você também precise considerar expectativas não registradas por escrito como parte do processo de testes.

Rastreáveis

A documentação completa do processo de testes é muito importante para repetir esse processo posteriormente. Ela deve descrever tanto os sucessos quanto as falhas. Além do mais, deve especificar o que foi testado e como a equipe de testes fez os testes. A documentação também inclui o testing harnesses e outras ferramentas que a equipe usou para realizar os testes de software. Sem um conjunto completo de tudo o que a equipe de testes usou, é impossível recriar o ambiente de testes posteriormente.

Determinísticas

Os testes que você fizer não devem ser aleatórios. Qualquer teste deve incluir recursos específicos de software e você deve saber quais são esses recursos. Além disso, os testes devem mostrar resultados específicos, considerando o fornecimento de entradas específicas. A equipe de testes sempre deve saber com antecedência como os testes devem funcionar exatamente e definir os resultados que eles devem fornecer para que os erros sejam óbvios.

Testando o desempenho

Muitas pessoas acham que desempenho é o mesmo que velocidade, mas o desempenho inclui mais que apenas a velocidade. Uma aplicação que seja rápida, mas executa a tarefa incorretamente, é inútil. Do mesmo modo, uma aplicação que execute bem uma tarefa, mas disponibiliza a informação que

ela processa as entidades erradas, também é inútil. Para ter um bom desempenho, uma aplicação deve executar tarefas de modo confiável, seguro e rápido.

Cada um desses elementos do desempenho influencia os demais. Aumentar a segurança necessariamente deixará a aplicação mais lenta porque o desenvolvedor acrescenta mais código para executar verificações de segurança. De modo semelhante, a confiabilidade deixará a aplicação mais lenta porque mais verificações são acrescentadas nesse caso também. Verificações de segurança podem diminuir a confiabilidade da aplicação ao reduzirem os riscos às custas da funcionalidade – uma aplicação confiável é aquela que oferece todas as funcionalidades esperadas em todas as circunstâncias (ela não falha). Em suma, todos os elementos do desempenho funcionam contra os demais, como mostra a figura 11.1.

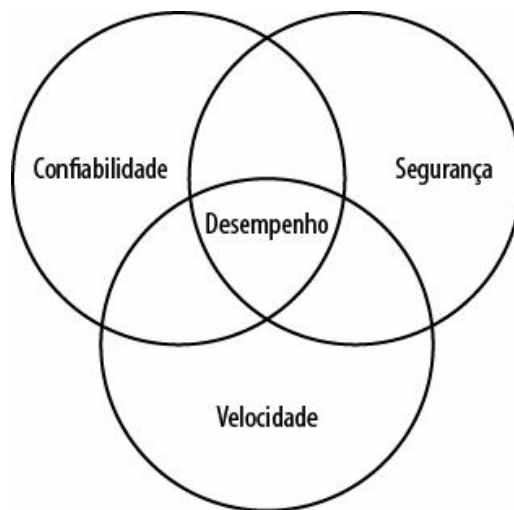


Figura 11.1 – O desempenho inclui velocidade, confiabilidade e segurança.

Para testar o desempenho de uma aplicação, você deve verificar o equilíbrio entre velocidade, confiabilidade e segurança. Aplicações balanceadas têm bom desempenho e não representam um peso para o usuário, mas tratam os dados de forma confiável e eficiente.

Testando a usabilidade

Muitos cenários de testes falham em testar a usabilidade. Determinar a facilidade com que o usuário pode interagir com o software é essencial, pois a meta do software é ajudar os usuários a serem mais produtivos (o fato de

esses usuários serem humanos ou máquinas não é importante). Uma interface confusa ou inútil causa problemas de segurança ao impedir que os usuários interajam com o software corretamente. O processo de testes deve considerar as necessidades físicas e emocionais do usuário, além dos passos necessários para executar uma tarefa. Por exemplo, pedir a um usuário daltônico que clique no botão vermelho pode não levar ao resultado desejado. Falhar em diferenciar o botão que não seja pela cor quase certamente causará problemas de entrada de dados que, em algum momento, resultará em problemas de segurança.



É fácil se tornar complacente quando executar os passos dos testes. Um usuário normalmente pode no mínimo contar com entradas pelo teclado e pelo mouse, portanto é preciso testar ambos. Contudo, os usuários podem ter uma variedade maior de opções de acesso. Por exemplo, pressionar uma combinação de teclas com Control pode executar tarefas de forma diferente em relação a simplesmente usar teclas-padrão, portanto você deve testar esse tipo de entrada também. Não é essencial testar todo tipo de entrada em todas as situações possíveis, mas você deve saber se a aplicação é capaz de lidar com os vários tipos de entrada corretamente.

Testando o tipo da plataforma

Softwares comportam-se de modo diferente de acordo com a plataforma usada. Ao longo deste livro você viu que os usuários podem e usarão vários tipos de dispositivos para interagir com qualquer aplicação que você criar. Não é possível testar sua aplicação em todas as plataformas concebíveis, pois algumas plataformas não estarão nem mesmo disponíveis no período de testes. Consequentemente, você deve criar tipos de plataforma – dispositivos que se enquadram em categorias específicas de acordo com as capacidades e os recursos. Por exemplo, você poderia agrupar os smartphones em duas ou três categorias, de acordo com a funcionalidade oferecida pela sua aplicação.



É pouco provável que uma empresa conheça todos os tipos de dispositivo com que os usuários contarão para executar tarefas relacionadas ao trabalho. É importante fazer uma pesquisa durante o processo de design da aplicação a fim de obter uma lista dos possíveis dispositivos de usuário. Você pode usar essa lista quando criar cenários de testes para tipos específicos de dispositivos.

Quando trabalhar com problemas associados a tipos de plataforma, é

especialmente importante observar como os dispositivos diferem tanto fisicamente quanto no modo como a interface funciona. As diferenças no nível de padronização do navegador também podem ser muito significativas. Qualquer problema que poderia fazer sua aplicação funcionar de modo diferente na plataforma alternativa é um tópico de teste. Você deve garantir que essas diferenças não farão a aplicação se comportar de maneira não prevista.

Implementando princípios de testes

Um princípio de teste é uma diretriz que você pode aplicar a todas as formas de testes. Os princípios afetam todos os aspectos dos testes de aplicações e são encontrados em todos os níveis. Quando realizar testes de unidade em APIs, você deve aplicar os mesmos princípios usados no teste da aplicação como um todo durante os testes de integração. Os seguintes princípios são comuns a todos os tipos de testes:

Fazer o software falhar

O objetivo do teste é fazer a aplicação falhar. Se testar a aplicação para ver se ela é bem-sucedida, você jamais encontrará os erros na aplicação. O processo de teste deve expor o máximo possível de erros, pois os hackers definitivamente procurarão erros para explorar. Os erros que você não encontrar serão aqueles que o hacker usará contra você.

Testando com antecedência

Quanto mais cedo você encontrar um erro, menor será o custo para corrigi-lo. O custo de correção de um bug aumenta com o tempo, como mostra a figura 11.2. Quanto mais cada bug custar para ser corrigido e quanto mais tarde você encontrar cada um deles, menos bugs você poderá corrigir por causa de considerações sobre custo e tempo, e menos segura será a aplicação.

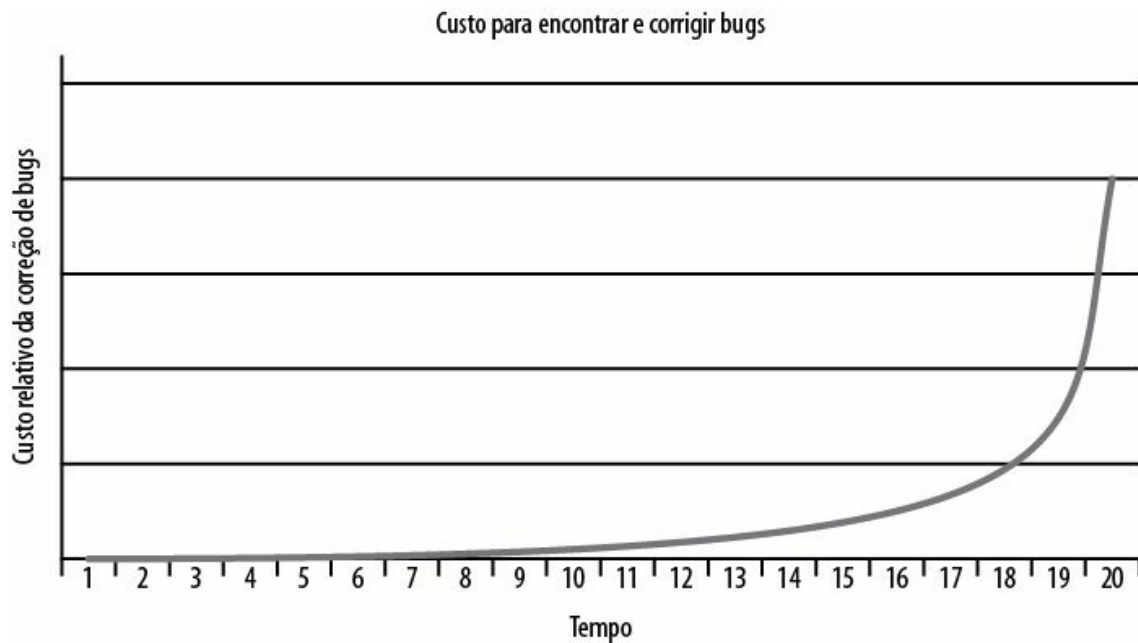


Figura 11.2 – Encontrar bugs com antecedência proporciona mais tempo e dinheiro para encontrar outros bugs.

Deixar o contexto dos testes dependente

O contexto de uma aplicação ajuda a determinar como você irá testá-la. Por exemplo, uma aplicação usada para uma necessidade crítica quanto à segurança exige testes diferentes quando comparada a uma aplicação usada em um site de e-commerce. A abordagem de desenvolvimento também afeta o contexto dos testes. Uma aplicação desenvolvida com a abordagem de cascata (waterfall) exige testes diferentes da aplicação que dependa de uma abordagem ágil (agile). Usar o contexto correto para testes ajuda a melhorar a segurança da aplicação aumentando as chances de encontrar os bugs mais prováveis de causar problemas.

Criar casos de teste eficientes

Quanto mais completos e exatos forem os casos de teste que você criar, mais eficiente será o teste e maior será a segurança em potencial. Os casos de teste devem incluir os requisitos tanto do usuário quanto da arquitetura da aplicação. Cada caso de teste é constituído da entrada exata para o código da aplicação e uma descrição da saída específica esperada como resultado dessa entrada. É essencial que as entradas e saídas usem quantidades mensuráveis para evitar ambiguidade.

Rever os casos de teste regularmente

Usar os mesmos casos de teste repetidamente cria um test harness que, em algum momento, deixará de encontrar erros. É essencial rever os casos de teste à medida que possíveis problemas da aplicação se tornem conhecidos para que o teste continue a pressionar mais a aplicação e possa encontrar mais bugs. Como parte do processo de revisão, você também deve fazer um teste exploratório para localizar bugs em potencial em que ninguém ainda pensou, com os quais nenhum usuário chegou a se deparar e nenhum hacker explorou.

Usar vários testadores de software

Algumas empresas contam com diferentes testadores de software durante as fases de teste, como testes de disponibilização de versão, aceitação, integração e de unidade. Usar testes em níveis funciona, mas empregar diferentes testadores de software ao longo do processo de desenvolvimento proporcionará melhores resultados. Por exemplo, contar com usuários para ajudar a testar durante os estágios iniciais da aplicação poderia ajudar a localizar problemas de segurança no design da interface em um momento em que é fácil e não é custoso corrigi-los.

Executar testes tanto estáticos quanto dinâmicos

Usar testes estáticos faz uma sondagem da aplicação em profundidade e mostra o entendimento do desenvolvedor sobre o domínio do problema e a estrutura de dados. O uso de testes dinâmicos faz uma sondagem da aplicação em largura e revela a capacidade da aplicação de tratar entradas extremas. Usar testes tanto estáticos quanto dinâmicos ajuda a garantir que o processo de teste resultará no máximo de bugs possíveis no período de tempo alocado para isso.

Procurar defeitos em conjunto

Erros tendem a ocorrer em conjunto (clusters). A probabilidade de encontrar erros em um segmento particular de código é diretamente proporcional ao número de erros já encontrados nesse segmento de código.

Fazer uma avaliação dos testes

Cada caso de teste exige uma avaliação no final para determinar seu sucesso ou sua falha. Quando o número de casos de teste for baixo, ou eles tiverem natureza crítica, você deve contar com inspetores que sejam seres humanos. Quando o número de casos de teste for alto, você deve incluir inspeção automática, além da inspeção feita por seres humanos.

Evitar o mito da ausência de erros

Só porque uma aplicação executa sem erros detectáveis não quer dizer que ela esteja livre de erros. As aplicações podem ter todo tipo de erros não testáveis, como uma incapacidade de atender às necessidades do usuário. Além disso, os testes só podem verificar o que o desenvolvedor os fizer verificar. Um teste pode não encontrar um erro simplesmente porque não testa esse erro. As aplicações geralmente têm erros, muitos dos quais não podem ser detectados.

Terminar o processo de testes

Teoricamente, você pode testar continuamente uma aplicação à procura de erros (e continuar a encontrá-los). No entanto, os testes normalmente chegam ao fim em algum momento, baseado na combinação entre dinheiro, tempo e qualidade do software. Quando o risco de usar uma aplicação torna-se baixo o suficiente e os usuários concordam que a aplicação é suficientemente utilizável, o processo de testes normalmente para, mesmo que ainda haja problemas a serem considerados. Como consequência, qualquer software que você empregar para qualquer finalidade provavelmente conterá erros que representarão riscos à segurança.

Entendendo as limitações dos testes

Não é possível oferecer um quadro completo do desempenho do software com os testes. Eles podem ajudar você a averiguar fatos específicos sobre o software, mas não garantem que o software não falhará. É essencial definir expectativas realistas sobre o processo de testes e executar esse processo sabendo que o resultado estará incompleto. Eis algumas limitações que você deve considerar:

Testadores de software não são clarividentes

Os testes só podem mostrar a presença de erros, mas nunca a sua ausência. O processo de testes determina a presença de problemas conhecidos. Um testador de software não pode testar problemas desconhecidos ou não descobertos.

Testar não é uma ferramenta de tomada de decisão

Testes podem apenas ajudar você a determinar o estado do software. Eles não podem ajudar a determinar se o software é seguro para ser usado ou se você deve lançá-lo com alguns bugs.

Os usuários encontrarão um ambiente que não funciona

Os testes só permitem determinar se o software funcionará razoavelmente bem em um ambiente específico. Se um usuário instalar o software em um ambiente com condições diferentes, o software poderá falhar. De fato, qualquer mudança no ambiente que você criar para o software poderia causar falhas que poderiam não ser encontradas durante os testes.

Causas-raiz são invisíveis aos testes

Os testes têm a ver com determinar o efeito de uma falha, dada uma entrada específica – eles não dizem qual é a origem do problema que provocou inicialmente a falha. Tudo que você sabe depois do processo de testes é que há uma falha; é necessário determinar qual é a sua origem.

Contando com ferramentas de teste para preencher as lacunas

Técnicas manuais de testes, juntamente com testing harnesses e scripts, podem proporcionar uma vantagem significativa na localização de possíveis brechas de segurança em sua aplicação. No entanto, a maioria dos desenvolvedores também conta com ferramentas para ajudar a preencher as lacunas deixadas por esses recursos há muito tempo existentes. As ferramentas de teste permitem procurar tipos específicos de problemas de segurança que os testes-padrão podem não localizar, por exemplo, injeção de SQL, XSS (Cross-site Scripting), buffer overflow (transbordamento de buffer), problemas em aplicações flash/flex e de exposições a Web 2.0. Eis algumas das ferramentas que você deve considerar para acrescentar à sua caixa de ferramentas de desenvolvedor para preencher as lacunas:

WebInspect (<http://www8.hp.com/us/en/software-solutions/webinspect-dynamic->

analysis-dast/

Oferece uma ferramenta DAST (Dynamic Application Security Testing) que procura hacks comumente usados de forma automática. A ferramenta simula o comportamento de sua aplicação quando estiver sob ataque desses hacks para que você possa identificar facilmente possíveis brechas de segurança. A ferramenta funciona tanto em aplicações web quanto em serviços.

AppScan (<http://www.ibm.com/developerworks/downloads/r/appscan/>)

Executa grande variedade de testes, incluindo DAST, análise em tempo de execução e análise estática de contaminação (static taint analysis) tanto em aplicações quanto em serviços. A ênfase dessa ferramenta está no gerenciamento de vulnerabilidades durante o ciclo de vida do software.

Burp Proxy (<https://portswigger.net/burp/proxy.html>)

Instala um servidor proxy em seu sistema que intercepta todo o tráfego entre a aplicação e outros endpoints, possibilitando analisar o tráfego de requisições e respostas da aplicação. O uso dessa ferramenta permite procurar atividades suspeitas que normalmente não atrairiam atenção, mas poderiam servir como precursores de um ataque por parte de um hacker.

Paros (<http://sourceforge.net/projects/paros/>)

Cria um ambiente de servidor proxy que possibilita analisar e editar todo o tráfego de entrada e de saída para sua aplicação. Essa ferramenta também oferece recursos de scanning, que facilitam a execução de testes automatizados.

Testando bibliotecas internas

Quando testar bibliotecas internas – aquelas das quais você é o proprietário e controla – você tem acesso ao código-fonte e pode realizar testes de unidade em cada um dos elementos antes mesmo de terminar a biblioteca. Essa abordagem dá a você um controle adicional sobre o processo de testes, permite testar e corrigir bugs quando os custos são mais baixos e garante que você possa realizar testes tanto estáticos quanto dinâmicos para localizar possíveis problemas com mais eficiência.

É claro que o fato de ser dono do código também significa que você está escrevendo a biblioteca ao mesmo tempo que o restante da aplicação. Outros desenvolvedores vão querer executar testes também. Isso significa que será necessário empregar simulação (mocking), conforme descrito no capítulo 10,

para garantir que o desenvolvimento possa progredir conforme previsto. À medida que testes de unidade individuais são executados e você tem certeza de que os elementos da biblioteca funcionam como previsto, é possível substituir os elementos simulados por elementos reais.

Como parte do processo de testes, você também pode criar um testing harness para a biblioteca, conforme descrito na seção “Criando um test harness para bibliotecas”. Entretanto, em vez de criar o test harness completo de uma só vez, você deve criá-lo por partes, à medida que os verdadeiros elementos da biblioteca se tornam disponíveis. Criar o test harness dessa maneira o ajudará a monitorar como os elementos da biblioteca estão interagindo com a aplicação e a fazer alterações conforme for necessário (quando o custo de fazer isso ainda é baixo).

Testando APIs internas

Como no caso das bibliotecas internas, você é o proprietário do código de uma API interna. Pelo fato de uma API não fazer parte da aplicação – ela executa em um processo diferente – você precisa criar uma configuração de servidor para interagir com a API. No entanto, você não vai querer usar um servidor de produção para essa tarefa, pois o código que você criar terá todo tipo de problemas. Vale a pena configurar a aplicação de modo que seja possível alterar apenas um item da configuração para fazer a aplicação apontar para a API de produção depois que você disponibilizar a API.

Qualquer API que você criar também precisará contar com simulação para que os desenvolvedores da aplicação possam começar a escrever o código da aplicação enquanto você continua a trabalhar na API. À medida que o código real se torna disponível, é preciso substituir os elementos simulados pelos elementos reais. Vale a pena desenvolver o conjunto todo de scripts de testes para uma API já no início, usando as técnicas que se encontram na seção “Criando scripts de testes para APIs”, estando ciente de que os elementos simulados fornecerão respostas previamente definidas. De fato, as respostas previamente definidas darão uma pista sobre quais elementos continuam sendo simulados.

É essencial testar suas APIs internas usando os mesmos critérios das APIs

externas e configurar os ambientes de teste e de desenvolvimento para que correspondam ao seu ambiente de produção. Caso contrário, você poderá acabar em uma situação em que uma API interna se transforma em um caminho para um hacker ter acesso à sua rede. Até mesmo softwares desconectados estão sujeitos a vários tipos de hacks (conforme descrito em capítulos anteriores).

Testando bibliotecas externas

Uma biblioteca externa (incluindo frameworks e todo tipo de outras estruturas de programação semelhantes a bibliotecas) é aquela que é de propriedade de outra pessoa. A biblioteca está completa antes mesmo de você começar a escrever sua aplicação e, teoricamente, o terceiro testa e mantém o código da biblioteca. Entretanto, a biblioteca é apenas uma caixa-preta na maioria dos aspectos. Sua capacidade de realizar um teste estático completo é limitada aos módulos voltados ao público que o terceiro disponibiliza. Por causa das complexidades de trabalhar com a maioria das bibliotecas de terceiros, um teste estático completo está fora de questão, e só restam os testes dinâmicos.

Antes de começar a escrever sua aplicação, durante as fases de proposta e de design, você deve garantir que qualquer biblioteca de terceiros escolhida seja segura e atenda totalmente aos critérios de sua aplicação. As bibliotecas mais populares oferecem um test harness ou você pode encontrar um test harness de terceiros para elas. Entretanto, quando trabalhar com uma biblioteca menor e menos conhecida, você precisa configurar os testes por conta própria.

Seria fácil supor que só porque uma biblioteca, um framework, uma API ou um microsserviço é popular, ele é automaticamente seguro para uso. Mesmo com produtos tão conhecidos como a jQuery, você pode encontrar problemas de segurança em sites como http://www.cvedetails.com/vulnerabilitylist/vendor_id-6538/Jquery.html. Além disso, mesmo que o produto seja supostamente seguro, usá-lo incorretamente pode provocar todo tipo de problemas de segurança. Leia artigos como “How to Safely and Wisely use jQuery: Several Key Issues” (Como usar a jQuery de forma segura e sábia: várias questões fundamentais, <http://resources.infosecinstitute.com/safely-and-wisely-use-jquery/>) para descobrir os pontos que poderiam causar problemas posteriormente. Todo código



tem vulnerabilidades, todo código tem problemas de uso e nenhum código é totalmente seguro – continue repetindo essas três frases a si mesmo e você não terá problemas.

Testando APIs externas

APIs externas são populares exatamente porque outras pessoas são proprietárias do código e as APIs nem mesmo executam no sistema local. Você não fará download do código nem fará nada com ele, exceto fazer chamadas. O canto da sereia das APIs externas seduz até mesmo o mais cuidadoso dos desenvolvedores, dando-lhes uma falsa sensação de segurança. Capítulos anteriores informaram tudo sobre o potencial assustador que as APIs têm de causar infortúnios. No mínimo, você precisa criar scripts de teste para APIs externas com muito mais cuidado do que para qualquer outro código que usar porque, de modo diferente das bibliotecas externas (e, por extensão, dos frameworks), você jamais verá o código. Não há nenhuma possibilidade de realizar um teste estático, portanto é melhor que seus testes dinâmicos englobem toda a API antes de tomar uma decisão sobre usá-la.

De modo diferente das bibliotecas, é improvável que você vá encontrar uma suíte com scripts prontos para uma API. Para conferir se a API funciona conforme previsto, você realmente precisa criar uma configuração de scripting e enviar entradas de todo tipo para a API. É essencial monitorar as respostas recebidas, especialmente para entradas errôneas. Você não sabe como uma API responderá a entradas errôneas. Consequentemente, não saberá como implementar sua aplicação de modo a reagir a uma resposta a uma entrada errônea. Em outras palavras, você precisa saber como a API reagirá quando receber dados que estejam fora do intervalo ou que sejam

possivelmente do tipo errado.

A suposição que a maioria dos desenvolvedores faz é que a entrada errônea virá dos usuários da aplicação. No entanto, entradas errôneas à API podem vir de um ataque man-in-the-middle prejudicial ou de outros tipos de hacks. A entrada errônea poderia também vir de outras origens em seu sistema, refletindo algum tipo de infecção ou outros problemas. Ao estar ciente de como uma API reage a entradas errôneas, você pode criar um tipo de indicador de segurança que informe que há algo errado. Considere isso como uma estratégia do canário na mina¹. Entradas errôneas, na maioria dos casos, não ocorrem por acaso – há um motivo e conhecer o tipo de entrada errônea que resulta em uma resposta inesperada pode fornecer pistas quanto à origem de um problema.

Estendendo os testes aos microsserviços

Teste os microsserviços usando as mesmas técnicas utilizadas com as APIs. Assim como no caso das APIs, você é capaz de executar somente testes dinâmicos, a menos que, por acaso, você seja o proprietário do código do microsserviço. Além disso, é essencial monitorar respostas a entradas de dados não esperadas, especialmente quando você planeja usar vários microsserviços para executar a mesma tarefa (com as alternativas oferecendo backup a um microsserviço principal que você selecionar). As respostas recebidas podem variar entre os microsserviços, o que quer dizer que o seu código de tratamento de erros será mais complicado.

A principal questão a ser considerada com os microsserviços, porém, é que o desenvolvedor os mantenha propositalmente pequenos. Você não pode testar os recursos mais comumente usados porque todo recurso é comumente usado. Em suma, seus scripts de teste agora devem testar todos os microsserviços completamente, o que poderia representar uma carga maior ao grupo de testes.

Testando bibliotecas e APIs individualmente

Em geral, o primeiro nível de testes funciona com bibliotecas e APIs individualmente. O processo de testes para microsserviços é semelhante ao de

APIs, exceto que você não precisa de um test harness complexo, pois os microsserviços, definitivamente, são mais simples que as APIs. As próximas seções descrevem estratégias que você pode usar quando realizar testes de unidade tanto em bibliotecas quanto em APIs (e, por extensão, em microsserviços). É importante observar que as APIs e os microsserviços podem ser testados diretamente ou como parte de uma camada de virtualização de APIs.

Criando um test harness para bibliotecas

Um test harness é um conjunto de instruções no código da aplicação ou como parte de um acréscimo especial ao código, e que executa vários testes. Como as bibliotecas existem como parte da aplicação, as instruções para testar a biblioteca também aparecem como parte da aplicação.

Instruções de teste normalmente aparecem em código de depuração como um tipo de função `assert()`, ou fazem uso de logging ou de saídas na tela. JavaScript não tem uma função `assert()` (há rumores de que uma função desse tipo será acrescentada). No entanto, você pode usar o objeto `error` para criar uma função semelhante à de asserção que ofereça o mesmo tipo de informações. Ao usar uma configuração com `assert()`, você cria asserções no código semelhantes a:

```
assert(typeof myArg === "string");
```

A função `assert()` terá o seguinte aspecto:

```
function assert(condition, message)
{
  if (!condition)
  {
    message = message || "Assertion failed";
    if (typeof Error !== "undefined")
    {
      throw new Error(message);
    }
    else
    {
      throw message;
    }
  }
}
```

```
}  
}  
}
```

Nesse caso, quando o valor de um argumento ou outra condição do código falha, o teste termina com um erro. Você pode optar por fazer log do erro ou trabalhar com ele de outras maneiras, mas saberá que o teste falhou. Testar a condição não é um problema para nenhum navegador. Contudo, talvez você encontre navegadores que deem suporte ao objeto `error` de modo distinto, portanto simplesmente lançar a mensagem (em vez de lançar novamente o objeto `error`) é uma boa opção de fallback.

O elemento que está no código executa o teste, mas você ainda precisa de dados de entrada para esses testes. Para obter essa parte do quebra-cabeça, normalmente é necessário empregar um teste manual com scripts, que é sempre suscetível a erros, ou um produto de terceiros para gerar os scripts das entradas exigidas. Os testes executados na biblioteca informarão exatamente até que ponto ela atende aos requisitos de sua aplicação.

Criando scripts de teste para APIs

Uma API oferece suporte por meio de chamadas. É possível testar uma API sem nem mesmo usar a aplicação. Tudo de que você precisa é de um script que faça chamadas e confira as respostas. Vários produtos de terceiros realizam essa tarefa, ou você pode criar uma aplicação simples para fazer o teste manualmente. Usar um script garante que você sempre terá os mesmos resultados de teste, portanto usar um produto para scripting normalmente é a melhor opção. Qualquer teste que você criar deve verificar, no mínimo, as seguintes condições:

Intervalo

A chamada responde corretamente dentro de seu intervalo e oferece uma resposta de erro correta quando os valores são muito altos ou muito baixos.

Tipo

Verifica se o usuário fornece o tipo correto de dado como entrada e oferece uma resposta de erro correta quando a entrada for do tipo incorreto.

Tamanho

Valida o tamanho do dado de modo que não seja possível a alguém enviar um script no lugar da string esperada.

Caracteres

Testa caracteres inválidos na entrada para garantir que o usuário não possa enviar caracteres de controle ou outros caracteres incorretos como parte da entrada.

Depois que você começar a usar a API a partir da aplicação, é preciso realizar testes de integração, que consistem em fornecer entradas à aplicação e então exigir que ela faça as chamadas necessárias. Novamente, você pode usar um produto para scripting para facilitar essa tarefa.

Estendendo as estratégias de teste aos microsserviços

Como ocorre com as APIs, você pode querer usar algum tipo de produto de scripting para fazer chamadas e conferir as respostas quando trabalhar com um microsserviço. No entanto, é preciso garantir que o microsserviço seja verificado completamente, pois cada um representa uma porção separada de código. Não é possível fazer suposições sobre um microsserviço do mesmo modo que podemos fazer com uma API.

Ao fazer testes de integração, você deve determinar o perfil de desempenho da aplicação. Todo microsserviço deve receber pelo menos um teste. No entanto, os microsserviços que você planeja usar com mais frequência devem receber mais testes. O propósito dessa estratégia é conferir códigos que tenham mais probabilidade de causar problemas e manter os custos dos testes baixos.

Desenvolvendo estratégias de resposta

O foco de todo teste está na resposta fornecida pela biblioteca, API ou microsserviço a uma dada entrada. A menos que o código reaja de maneira apropriada, a aplicação não funcionará conforme previsto originalmente. Acima de tudo, os hackers procuram discrepâncias no comportamento para poder explorá-las. Ao definir respostas, você deve considerar dois tipos delas:

direta e simulada (mocked). As próximas seções discutem cada tipo de resposta.

Contando com resultados diretos

Uma resposta direta é proveniente da biblioteca, da API ou do microserviço ativo. Nesse caso, você obtém a resposta propriamente dita à entrada fornecida. Se o código em teste estiver funcionando corretamente, a resposta recebida deverá corresponder exatamente às especificações que definem os casos de teste. Os resultados diretos na verdade testam o código que você planeja usar com sua aplicação.

Contando com resultados simulados

Resultados simulados são provenientes de softwares que simulam a biblioteca, a API ou o microserviço. Usar resultados simulados permite começar a trabalhar com a aplicação e testá-la antes que o código da biblioteca, API ou microserviço esteja pronto para uso. A utilização dessa abordagem economiza tempo e permite que o desenvolvimento avance muito mais rápido.

Contudo, há outro aspecto a ser considerado. Você pode usar simulação para testar a viabilidade de seu testing harness para uma biblioteca ou dos scripts de testes para uma API. Como você já sabe que o software de simulação oferecerá uma resposta exata para uma dada entrada, será possível validar se o software de teste está funcionando conforme previsto. Não vale a pena realizar um teste, a menos que você possa contar com o fato de que o software de teste executará a tarefa corretamente.

Realizando testes de integração

Após ter testado os elementos individuais de uma aplicação em busca de problemas, você deve começar a integrar as bibliotecas, APIs e microserviços à aplicação. O método mais fácil, mais completo e menos complexo de testes de integração é usar uma abordagem em fases, em que as bibliotecas, as APIs e os microserviços são adicionados à aplicação em grupos pequenos, de forma organizada. O uso de testes de integração em

fases ajuda a localizar e a corrigir problemas rapidamente. Fazer a transição de uma aplicação de dados simulados para dados reais, uma parte de cada vez, pode parecer demorado à primeira vista, mas o processo possibilita localizar erros rapidamente, economizando tempo no final. O objetivo dos testes de integração é criar uma aplicação completa, que funcione conforme previsto e não contenha brechas de segurança.



Não é possível criar uma aplicação à prova de balas. Toda aplicação conterá falhas e potenciais brechas de segurança. Os testes eliminam os problemas mais óbvios e você deve testar (e retestar) toda alteração feita em qualquer parte da aplicação, pois as brechas de segurança aparecem nos lugares menos esperados. No entanto, nunca se torne complacente achando que encontrou todos os problemas possíveis da aplicação – é quase certo que um hacker aparecerá mostrando que você estava errado.

Os desenvolvedores passaram a contar com vários modelos de testes de integração. Alguns desses modelos são projetados para deixar a aplicação pronta e executando o mais rápido possível, mas fazem sua tarefa somente quando nada dá errado. Qualquer pessoa que escreve software sabe que algo sempre dá errado, portanto o modelo big-bang de testes de integração só causará problemas e não permitirá que você verifique totalmente os problemas de segurança. Com isso em mente, a seguir estão três modelos de teste que usam uma abordagem em fases e oferecem mais chances de identificar problemas de segurança:

Bottom up

Nesse caso, a equipe de desenvolvimento acrescenta e testa os recursos de baixo nível antes. A vantagem desta abordagem é que você pode conferir se há uma base robusta para as aplicações que realizam tarefas, por exemplo, monitoração. Os dados brutos tornam-se disponíveis antes no processo de testes, o que deixa todo o processo de testes mais realista.

Top down

Usar top down quer dizer testar todos os recursos de alto nível antes, verificando os ramos do primeiro nível em seguida e trabalhando de cima para baixo, até os recursos de mais baixo nível. A vantagem dessa abordagem é que você pode verificar se os recursos da interface de usuário

funcionam conforme esperado e se a aplicação atenderá às necessidades do usuário desde o princípio. Esse tipo de teste funciona melhor com aplicações para apresentações, em que a interatividade com o usuário tem alta prioridade.

Sanduíche

É uma combinação de bottom up e top down. Ela funciona melhor em aplicações que exijam algum uso de fontes de dados para executar a maioria das tarefas, mas a interatividade com o usuário também é um aspecto importante. Por exemplo, você pode usar essa abordagem em uma aplicação de CRM (Customer Relationship Management, ou Gerenciamento de Relacionamento com Clientes) para garantir que a interface de usuário apresente os dados de um banco de dados de forma correta, antes de passar para outras funcionalidades.

Testando problemas específicos de linguagem

Uma enorme brecha em algumas suítes de teste é a falta de testes para problemas específicos de linguagem. Esses testes, na verdade, procuram falhas no modo como uma linguagem trata requisições específicas. A linguagem pode funcionar exatamente conforme projetada, mas uma combinação de fatores atua em conjunto para gerar um resultado incorreto ou inesperado. Em alguns casos, o ataque baseado em deficiências de linguagem na verdade usa a linguagem de uma forma não prevista originalmente pelos designers da linguagem.

Toda linguagem tem deficiências. Por exemplo, em muitas linguagens, o problema de segurança de threads entra em cena. Quando usada sem multithreading, a linguagem funciona sem erros. Porém, quando usada com multithreading e apenas em algumas circunstâncias, a linguagem repentinamente deixa a desejar. Ela pode gerar um resultado incorreto ou simplesmente agir de maneira inesperada. Muito mais traiçoeiras são as deficiências em que a linguagem aparentemente funciona de modo correto, mas consegue fornecer aos hackers informações necessárias para eles se infiltrarem no sistema (por exemplo, transferindo dados entre threads).

As próximas seções descrevem os problemas mais comuns específicos de linguagem que você deve considerar nos testes de sua aplicação. Você encontrará deficiências em HTML5, CSS3 e JavaScript – linguagens comumente usadas em aplicações web. Entretanto, se você usa outras linguagens em sua aplicação, é preciso verificar suas deficiências também.



Muitas suítes de teste verificam se as saídas estão corretas, dada uma entrada específica. Além disso, elas podem fazer verificações de intervalo para garantir que a aplicação se comporta corretamente no intervalo de valores que ela deve aceitar e oferece uma resposta apropriada, indicando erro quando os valores estão fora do intervalo. O motivo pelo qual a maioria das suítes de testes não verifica deficiências de linguagem é porque esse problema está mais relacionado à segurança que praticamente qualquer outro aspecto que você testar. Ao fazer testes para identificar problemas específicos de linguagem, o que você realmente deve procurar é o efeito das deficiências na segurança de sua aplicação.

Projetando testes para problemas de HTML

Ao trabalhar com HTML, você deve considerar que a linguagem oferece a interface de usuário básica e que ela também oferece os caminhos pelos quais muitos problemas de segurança específicos de usuário ocorrerão. Com isso em mente, você precisa garantir que o HTML usado para apresentar informações gerenciadas pela sua aplicação seja testado e que ele funcionará com uma grande variedade de navegadores. Levando isso em consideração, eis algumas questões específicas da linguagem que você deve analisar:

- O HTML é bem-formatado e não depende de tags ou de atributos que não tenham suporte por parte da maioria dos navegadores.
- O documento está corretamente codificado.
- Qualquer código no documento executa o tratamento de erros necessário e verifica se as entradas estão corretas.
- A saída do documento tem a aparência esperada quando entradas específicas são fornecidas.
- A seleção de elementos da interface de usuário reduz o potencial para confusão e entradas errôneas.

Existem várias ferramentas disponíveis para testes de HTML. Dois dos melhores produtos são: Rational Functional Tester (<http://www->



03.ibm.com/software/products/functional) e Selenium (<http://www.seleniumhq.org/>). Os dois produtos automatizam testes de HTML e ambos oferecem o método gravar/reproduzir (record/playback) de criação de scripts para executar os testes. O Rational Functional Tester – um produto da IBM – também tem estratégias especializadas de testes, como o storyboarding. Se você precisar executar uma verificação manual de uma técnica nova, experimente o W3C Markup Validation Service em <https://validator.w3.org/dev/tests/>. O site oferece testes específicos a diferentes versões de HTML.

Parte dos testes de HTML está em garantir que todos os seus links funcionam conforme previsto e que seu HTML seja bem-formatado. Produtos como o WebLight (<http://www.illumit.com/weblight/>) automatizam a tarefa de conferir seus links. Um produto semelhante é o LinkTiger (<http://linktiger.com/>). Os dois produtos verificam links indisponíveis, mas cada um oferece recursos adicionais que você pode precisar em seus testes, portanto é uma boa ideia consultar as especificações de ambos.

Projetando testes para problemas de CSS

O propósito original do CSS era criar um meio de formatar conteúdo que não envolvesse o uso de tabelas e de outros truques. O problema com esses truques é que ninguém utilizava uma abordagem-padrão, e os truques tendiam a deixar a página inutilizável para pessoas com necessidades especiais. Contudo, o CSS deixou de ser apenas uma simples formatação. As pessoas encontraram maneiras de criar efeitos especiais com o CSS. Além disso, o CSS atualmente oferece certo nível de funcionalidades de programação. Como resultado, tornou-se importante testar o CSS de forma tão completa quanto você testaria qualquer outra parte de sua aplicação. O CSS tem o potencial de ocultar problemas de segurança. Com isso em mente, você precisa executar testes específicos de CSS como parte dos testes de segurança de sua aplicação – os seguintes critérios devem ser atendidos:

- O CSS é bem-formatado.
- Não há elementos não-padrão no código.
- Efeitos especiais não causam problemas de acessibilidade.
- A escolha de cores, fontes e outros elementos visuais reflete as melhores práticas para pessoas com necessidades especiais.

- É possível usar um formato alternativo de CSS quando o usuário tiver necessidades especiais.
- Dado um evento ou uma entrada de usuário em particular, o CSS oferece uma saída consistente e repetível.



À medida que o uso de CSS aumenta, o mesmo acontece com a necessidade de ter boas ferramentas de teste. Se você é usuário do Node.js, uma das melhores ferramentas de teste que você pode obter é o CSS Lint (<http://csslint.net/>). Use o CSS Lint para verificação de código. Quando quiser verificar a aparência, utilize outro produto que compare imagens de tela, como o PhantomCSS (<https://github.com/Huddle/PhantomCSS>). Quando a imagem de tela de seu site mudar de forma imprevisível, o PhantomCSS pode ajudar a identificar a alteração e apontar a causa. Se precisar de uma ferramenta de validação manual para verificar uma técnica que queira usar, conte com o W3C CSS Validation Service em <https://jigsaw.w3.org/cssvalidator/>.

Projetando testes para problemas de JavaScript

JavaScript oferecerá a maior parte do código funcional de sua aplicação. Com isso em mente, teste o código JavaScript usando a mesma abordagem utilizada em outras linguagens de programação. Você deve verificar se, para uma dada entrada, vai obter uma saída específica. Eis alguns problemas que devem ser considerados como parte de sua suíte de testes:

- Garantir que o código esteja de acordo com os padrões.
- Testar o código usando um conjunto completo de tipos de entrada para garantir que ele trata entradas errôneas sem falhar.
- Executar testes assíncronos para garantir que sua aplicação pode tratar respostas que chegam depois de um intervalo de tempo não determinístico.
- Criar grupos de teste para que você possa validar diversas asserções usando o código simples de asserção do exemplo mostrado na seção “Criando um test harness para bibliotecas” (ou um `assert()` oferecido como parte de uma biblioteca de teste). Grupos de teste multiplicam o efeito do uso de asserções para testes.
- Verificar se o código é responsivo.

- Verificar o comportamento da aplicação para garantir que uma sequência de passos produza um resultado desejado.
- Simular condições de falhas (como a perda de um recurso) para garantir que a aplicação degrade de forma elegante.
- Executar qualquer teste necessário para garantir que o código não seja suscetível a hacks recentes que podem não estar corrigidos no sistema do cliente. Usar um analisador específico para segurança, por exemplo, o VeraCode (<http://www.veracode.com/>), ajuda a localizar e a corrigir bugs que possam oferecer uma entrada aos hackers que se aproveitam de problemas de segurança recentemente encontrados.



A ferramenta que você usar para testar JavaScript depende, em parte, das ferramentas utilizadas para outros tipos de testes na empresa e da experiência dela com outras ferramentas. Além disso, você deve escolher uma ferramenta que funcione bem com outros produtos usados em sua aplicação. Algumas empresas dependem do QUnit (<https://qunitjs.com/>) para testar JavaScript por causa das outras suítes (como jQuery e jQuery UI) do fornecedor. Em alguns casos, uma empresa usará o RhinoUnit (<https://code.google.com/archive/p/rhinounit/>) para obter o framework de testes de JavaScript baseado em Ant. Muitos desenvolvedores profissionais gostam do Jasmine (<http://jasmine.github.io/>) juntamente com o Jasmine-species (<http://rudylattae.github.io/jasmine-species/>), pois a suíte de testes tem um bom comportamento. Se você estiver trabalhando bastante com o Node.js, pode também querer investigar a dupla Vows.js (<http://vowsjs.org/>) e kyuri (<https://github.com/nodejitsu/kyuri>). Outro favorito entre os desenvolvedores que usam Node.js é o Cucumis (<https://github.com/noblesamurai/cucumis>), que oferece funcionalidade de testes assíncronos.

Um dos problemas que você deve pesquisar como parte dos estágios de especificação e de design é a disponibilidade de suítes de testes existentes. Por exemplo, o site test262 do ECMAScript Language (<http://test262.ecmascript.org/>) pode oferecer ótimos insights para sua aplicação.

Considerando a essência de um ataque

É importante entender que muitos ataques não parecem que estão realmente fazendo algo, pois você não sabe o que o invasor está tentando fazer. Por exemplo, um ataque ao protocolo TLS (Transport Layer Security) que dependa da criptografia RC4 (Rivest Cipher 4) pode dar a impressão de que alguém está tentando transformar o sistema em

um zumbi. Se você é um desenvolvedor que está tentando descobrir o que está acontecendo com a aplicação, deve conhecer o tipo de ataque que está ocorrendo, portanto deve garantir que haja um profissional de segurança em sua equipe.

Nesse caso, um ataque depende da geração de uma requisição com informações repetidas, por exemplo, um cookie. Quem envia os dados incluirá esse cookie em todas as requisições, mas elas serão criptografadas de modo que a informação pareça diferente. Somente após obter amostras suficientes, será possível quebrar a criptografia e começar a obter dados do navegador comprometido.

No mínimo dois grupos quebraram o RC4 o suficiente para obter dados de um navegador usando TLS. A primeira técnica exige algo em torno de duas mil horas, enquanto a segunda exige apenas 75 horas. Obviamente, a menos que a informação obtida tenha valor significativo, provavelmente o tempo médio empregado pelo hacker não compensa. Mesmo assim, o simples fato de as técnicas existirem quer dizer que você deve pensar sobre o que um ataque está tentando fazer.

A parte mais interessante desse ataque é que os invasores contam com JavaScript para implementá-lo (é impressionante para uma linguagem que os desenvolvedores consideravam ser útil apenas para scripting no passado). Os invasores fazem upload do código no navegador-alvo usando injeção de script ou um ataque man-in-the-middle. Você pode ler mais sobre esse ataque em particular em <http://www.computerworld.com/article/2948937/security/encrypted-web-and-wi-fi-at-risk-as-rc4-attacks-become-more-practical.html>.

A correção para esse tipo de ataque consiste em usar um cifrador mais moderno como parte do protocolo TLS. O TLS aceita vários cifradores, incluindo o AES (Advanced Encryption Standard, ou Padrão de Criptografia Avançada). Para manter sua aplicação razoavelmente segura, é preciso garantir que o servidor esteja configurado para não aceitar RC4 como cifrador, exigindo um cifrador mais moderno em seu lugar.

¹ N.T.: Referência à antiga prática de levar canários às minas de carvão para saber se tudo estava bem. Enquanto o canário cantasse, não havia perigo, mas se a ave silenciasse, era sinal de um possível vazamento de gases tóxicos, permitindo emitir um alerta para que a mina fosse evacuada.

CAPÍTULO 12

Usando empresas terceirizadas de testes

Testes feitos por terceiros envolvem contratar uma entidade externa para realizar testes de vários tipos em sua aplicação, incluindo testes de segurança. O terceiro pode oferecer uma variedade de serviços – alguns bem abrangentes. É claro que você precisa saber se pode confiar em um terceiro antes de iniciar qualquer tipo de teste. Depois que os testes começarem é preciso garantir que o terceiro receba as informações apropriadas de sua empresa, tenha o nível correto de monitoração e ofereça um nível desejável de resultados. O terceiro pode realizar testes semelhantes àqueles que você usa, porém é necessário saber se o nível de habilidade do terceiro é melhor que o oferecido por sua própria empresa, ou não haverá motivo para contratar o terceiro.

Há muitas razões para querer contar, pelo menos parcialmente, com testes de terceiros. O motivo mais comum para contratá-los é o tempo. No entanto, muitas empresas não têm as habilidades e outros recursos para realizar um serviço completo de testes de forma apropriada. Às vezes, as empresas contratam um terceiro para preservar a honestidade dos testadores de software internos e garantir que eles não deixaram passar nada. Trabalhar com terceiros é um processo que comumente segue os quatro passos, assim descritos:

1. Localizar o serviço de testes de terceiros que você quer usar.
2. Criar um plano de testes (usando o terceiro como um recurso) que defina exatamente como o terceiro testará o software.
3. Implementar o plano de testes após o terceiro ter tido a chance de comprometer-se com ele.

4. Validar a aplicação com base em relatórios e outros resultados que você receber do terceiro.



Embora este livro tenha dedicado um período de tempo considerável trabalhando com aplicações para desktop, navegadores e aplicativos móveis, é importante lembrar que os princípios descritos aplicam-se a todos os tipos de aplicações também – algumas das quais quase sempre exigem testes de terceiros. Por exemplo, as indústrias automobilísticas provavelmente poderiam ter evitado o recente recall de muitos automóveis conectados se tivessem empregado terceiros para garantir que o processo de testes tivesse testado completamente todos os aspectos da conectividade (veja <http://www.computerworld.com/article/2953832/mobile-security/senators-call-for-investigation-of-potential-safety-security-threats-from-connected-cars.html>). O fato é que os hackers sabem como invadir esses veículos e o mesmo acontece com muitos testadores de software terceirizados (consulte <http://www.computerworld.com/article/2954668/telematics/hacker-shows-he-can-locate-unlock-and-remote-start-gm-vehicles.html>).

Testadores de software terceirizados também podem verificar tipos robustos de hacks exóticos que poderão se tornar bem comuns no futuro (consulte <http://www.computerworld.com/article/2954582/security/researchers-develop-astonishing-web-based-attack-on-a-computer-s-dram.html>). Embora técnicas como *rowhammering* pareçam bem distantes hoje em dia, você pode apostar que os hackers as empregarão amanhã e que será necessário ter um testador de software terceirizado habilidoso para garantir que sua instalação não seja vulnerável a esse tipo de ataque.

Localizando serviços de testes oferecidos por terceiros

Não é possível consultar o guia telefônico para encontrar uma lista de empresas que façam testes de software. Seria bom se você pudesse simplesmente encontrar um terceiro interessado nas páginas amarelas e saber que ele faria um ótimo trabalho de testes em sua aplicação. Infelizmente, encontrar uma empresa terceirizada de testes de software exige um pouco mais de pesquisa de sua parte e você deve garantir que verificará se o terceiro é capaz de fazer o trabalho que você está lhe pedindo para fazer. As próximas seções ajudam a entender alguns dos problemas em torno da procura por uma empresa terceirizada de testes de software e descreve como garantir que ela realmente tenha seus melhores interesses em mente.

Definindo os motivos para contratar um terceiro

Há muitos motivos para contratar um terceiro para fazer testes em sua aplicação. Os motivos variam conforme a empresa, o tipo de aplicação e o domínio do problema que a aplicação resolve. Antes de contratar um terceiro para realizar testes, é importante saber por que você o está contratando. Se não tiver essa informação essencial, o esforço de testes estará condenado desde o início, porque você terá expectativas não definidas e possivelmente não realistas do processo de testes. Em suma, você não pode saber se o teste foi bem-sucedido porque não tem ideia do que esperar dele. Com isso em mente, os motivos mais comuns para contratar um terceiro são:

Qualidade

Usar alguém especializado em testes pode melhorar a qualidade de seu software porque uma empresa dedicada a testes conhecerá todas as técnicas mais recentes para realizar testes específicos ao seu problema. Esses testadores de software também realizam tarefas de testes todos os dias, portanto conhecem bugs e problemas em potencial que ocorrem comumente com softwares que usam uma linguagem de programação, um sistema operacional ou uma plataforma em particular, e tratam de necessidades específicas de usuários. Também é possível que a qualidade dos relatórios que você receber do processo de testes seja melhor que qualquer relatório produzido pela sua própria empresa, porque os testadores de software preenchem esse tipo de relatório todos os dias.

Custo

Um testador de software profissional pode executar as tarefas de teste mais rápido e com mais precisão que alguém que execute a tarefa em tempo parcial. O testador de software também pode evitar testes que não resultem em nada em sua aplicação em particular – um problema que ocorre regularmente nas empresas (desperdiçando tempo e dinheiro). Além disso, é possível terceirizar o teste em um lugar no mundo em que os custos para contratar alguém para executar essa tarefa sejam bem mais baixos que no lugar em que você está.

Treinamento

Um custo oculto, que muitas empresas não consideram, é o tempo e os gastos com treinamento de uma equipe para executar as tarefas de testes. Mesmo que os membros da equipe executem a tarefa em tempo parcial, eles não terão a experiência que os testadores de software profissionais têm e poderão precisar de treinamento adicional para cada novo tipo de aplicação testada. A única situação em que vale a pena ter uma equipe de testes é quando sua empresa desenvolve software suficiente para manter a equipe ocupada com testes em tempo integral.

Tempo

Instalar um laboratório de testes, treinar testadores de software e esperar que eles se tornem proficientes na realização do trabalho necessário exige tempo que muitas empresas simplesmente não têm. Para lançar um produto no mercado a tempo, é necessário fazer testes durante o processo de escrita, usando técnicas que sejam as mais eficientes, em sistemas que se assemelhem ao máximo do sistema de produção e com relatórios entregues aos desenvolvedores que ajudem a localizar e a corrigir problemas o mais rápido possível.

Expertise

Sua empresa pode ter feito recentemente uma atualização de plataforma, equipamento ou de outros recursos que exijam outro ciclo de testes. A aplicação ou os serviços relacionados não mudaram, porém o ambiente em que eles funcionam mudou. Se você descobrir que a instalação nova causa problemas para sua equipe de testes, pode ser hora de obter ajuda de terceiros para descobrir o que essas mudanças significam para o software e definir novas formas de verificação. O terceiro também pode oferecer o treinamento necessário.

Considerando a variedade de possíveis serviços de testes

A principal vantagem de usar uma empresa terceirizada de testes é garantir que você extrairá o máximo dos serviços prestados. No entanto, o melhor serviço de testes do mundo não poderá conhecer tudo sobre todos os tipos possíveis de testes. É preciso definir exatamente os tipos de testes que você

quer fazer e então usar essas especificidades quando escolher uma empresa de testes. Como primeiro passo, é preciso definir o ambiente:

- Plataforma
- Sistema operacional
- Linguagem de programação
- Domínio do problema
- Tipo de empresa
- Tipo de usuário

Nem toda empresa de testes executa todos os tipos de testes disponíveis. Depois de avaliar os fatores ambientais, considere os tipos de testes que você deve fazer. Para obter os melhores resultados, é importante que a empresa terceirizada de testes ofereça recursos para executar todos os testes necessários. Eis os tipos mais comuns de testes:

- De segurança
- Funcionais
- De automação
- De desempenho
- De integração



Existem vários tipos de testes em inúmeros níveis. Você pode encontrar listas mais extensas de tipos de testes em <http://www.softwaretestinghelp.com/types-of-software-testing/>, <http://www.aptest.com/testtypes.html> e <http://www.testingexcellence.com/types-of-software-testingcomplete-list/>. No entanto, até mesmo essas listas são incompletas. Lembre-se de que também é necessário considerar técnicas de testes tanto estáticos quanto dinâmicos quando você é o proprietário do código, e se a empresa terceirizada de testes poderia se envolver em ambos os níveis.

Algumas empresas de teste oferecem apenas um serviço: testes. No entanto, você pode precisar de outros serviços para ter sucesso na tarefa de desenvolvimento de sua aplicação. Uma empresa que oferece um serviço completo normalmente cobra muito mais que uma empresa que oferece apenas testes. O número de serviços nem sempre afeta o preço final, mas isso pode acontecer e você deve escolher apenas os serviços que sejam realmente

necessários. Eis alguns serviços adicionais comuns:

Orientação a desenvolvedores

Em alguns casos, os testes podem revelar problemas que seus desenvolvedores simplesmente ainda não conhecem. A orientação aos desenvolvedores ajuda sua equipe a localizar e a corrigir problemas mais rapidamente. Além disso, oferece um nível de treinamento específico à sua aplicação, o que significa que as informações aprendidas são específicas à empresa e são focadas (aumentando seu valor).

Gerenciamento de programas

Talvez você precise de alguma ajuda para gerenciar os esforços de teste. Assim como sua equipe de desenvolvimento pode não ter experiência com testes, a equipe de gerenciamento pode não ter experiência em administrar determinados aspectos do ciclo de vida do produto. Algumas empresas de teste podem ajudar sua empresa a superar esse problema.

eLearning

Uma empresa como um todo pode exigir treinamento adicional para fazer a aplicação funcionar melhor. Por exemplo, muitos usuários não estão cientes dos problemas em torno da segurança, de modo que cometem erros que podem gerar uma brecha de segurança. O uso de módulos de eLearning ajuda as pessoas a descobrir as habilidades necessárias para usar a aplicação com sucesso durante períodos em que o usuário tem motivação e disponibilidade para fazer o treinamento.

Também é importante considerar as especificidades dos testes de segurança em sua aplicação. A empresa terceirizada de testes pode oferecer alguns tipos de testes de segurança, mas não outros. Além do mais, você deve saber o que o terceiro deve oferecer como parte do serviço. Eis alguns aspectos a serem considerados:

Ambiente

Você deve definir se o teste é em ambiente de staging (ensaio) ou de produção. Testes em ambiente de staging funcionam melhor durante as fases iniciais do desenvolvimento da aplicação. Testes em ambiente de

produção são melhores durante as fases finais de desenvolvimento e quando a aplicação é inicialmente disponibilizada no ambiente de produção.

Nível de testes

Algumas empresas terceirizadas de testes executam uma suíte básica de testes para garantir que os riscos de segurança de sua aplicação permaneçam baixos. Entretanto, esse teste básico pode não fornecer informações suficientes para corrigir os problemas de sua aplicação. Talvez você queira que o teste ofereça exploits completos para prova de conceito, com códigos de exemplo, para que você possa ver exatamente como os hacks funcionam e efetuar uma correção melhor. (Às vezes, o terceiro também pode ajudar você a fazer uma triagem em sua aplicação.)

Uso de recursos

Testes usam recursos de seu sistema. Para evitar a perda de funcionalidades da aplicação ou de disponibilidade durante os testes, você deve especificar as regras para o uso de recursos. O testador de software terceirizado pode configurar as ferramentas de testes para que elas estejam de acordo com as diretrizes que você definir. Além do mais, não se esqueça de especificar o uso de recursos como uma função do dia e da hora. Por exemplo, talvez você queira que o testador de software terceirizado foque em testes à noite e nos finais de semana, quando o sistema em teste tem menos uso e você pode afrouxar algumas das regras de uso de recursos.

Condições para interrupção dos testes

Você não vai querer que os testes de segurança resultem em violação dos dados ou outros problemas de segurança. Em alguns casos, os testes de segurança podem realmente resultar em um problema de segurança. Definir condições de interrupção ajuda a evitar que o cenário de testes de transforme em uma situação em que você realmente precise executar uma recuperação de dados ou fazer uma declaração pública informando uma violação de dados.

Garantindo a legitimidade do terceiro

Qualquer pessoa pode vir até você e dizer que a empresa oferece serviços

superiores de testes em tempo hábil e com o menor custo possível. Isso não quer dizer que elas realmente cumpram suas promessas. Pode significar que elas têm uma ótima equipe de marketing, capaz de escrever um material acima da média, mas nada do que está escrito poderá se concretizar. Uma empresa terceirizada de testes ruim poderia custar muito a você, como permitir que um concorrente lance um produto antes, causar violações nos dados que afastem os clientes ou fazer com que seja impossível para as pessoas em sua empresa realizarem seu trabalho. Consequentemente, garantir que o terceiro possa realmente cumprir suas promessas é uma parte essencial na busca por um fornecedor.

Testes de segurança é uma das áreas em que você realmente não quer correr riscos, portanto certifique-se de que o terceiro contratado tenha todas as certificações necessárias. O ASTQB (American Software Testing Qualifications Board, <http://www.astqb.org/>) é um bom ponto de partida para começar a procurar certificações exigidas de terceiros. Outro lugar para olhar é o International Software Certifications Board (<http://www.softwarecertifications.org/>). Independentemente do grupo em que você decida se basear, certifique-se de que a empresa que contratar tenha testadores de software com as certificações apropriadas. Além disso, obtenha garantias por escrito de que apenas testadores de software com certificação trabalharão em seu projeto (caso contrário, a empresa poderá fazer uma substituição por mão de obra abaixo do padrão, supervisionada marginalmente pelas pessoas que têm as certificações).

Também vale a pena dar uma olhada na estabilidade dos terceiros. Um relatório anual pode dizer muito sobre a saúde de uma empresa que, por sua vez, informa se a empresa tem sucesso no cumprimento de tarefas. Empresas de sucesso tendem a fazer um trabalho melhor de testes e você vai querer o melhor para sua aplicação. Explore também as estatísticas relacionadas às empresas, como tempo de *turnover* (rotatividade de pessoal, que indica a satisfação dos funcionários) e verifique se a empresa já teve notícias ruins associadas ao seu nome na imprensa.

Avaliações de clientes anteriores podem ajudar. No entanto, é essencial lembrar que as avaliações são simplesmente opiniões, e os clientes às vezes

são extremamente tendenciosos. Além disso, uma empresa pode plantar avaliações positivas para fazer parecer que oferece serviços melhores do que realmente o faz. Dê uma olhada nas avaliações e garanta que a empresa realmente fez o trabalho para o cliente, quando for possível. Entre em contato com o cliente para ver que tipo de testes foram feitos (novamente, se for possível).

Entrevistando o terceiro

A essa altura você sabe exatamente por que quer a ajuda de terceiros, sabe se achou uma empresa que ofereça os serviços de que você precisa e se a empresa é legítima. É hora de ter uma discussão com o pessoal da empresa e determinar se você pode estabelecer um relacionamento profissional com ela. Essa entrevista não é como uma entrevista para contratar alguém em sua empresa – você está contratando toda uma empresa para realizar testes em sua aplicação a fim de garantir que ela seja segura. Isso significa envolver todos os stakeholders de sua empresa na entrevista e determinar qual é o melhor candidato para a tarefa. Os stakeholders incluem a gerência, é claro, mas também desenvolvedores, instrutores, usuários e qualquer um que seja afetado pela aplicação. A pergunta a que você deve responder é se a empresa que você está contratando realmente pode realizar os níveis de teste exigidos para obter uma aplicação utilizável. Se precisar de outros serviços, também será necessário entrevistar a empresa para saber que tipo de suporte você pode esperar dela (é aí que os usuários poderiam se sair bem, se a empresa oferecer suporte para treinamento).

Realizando testes em uma instalação para testes

Antes de assinar na linha pontilhada, certifique-se de que a empresa que você está contratando é capaz de executar o trabalho. Faça a empresa oferecer algum tipo de demonstração limitada que percorra um processo de testes genérico, se for possível. Você deve ter uma visão geral do processo ou alguma outra evidência de que a empresa realmente é capaz de fazer o trabalho. Caso contrário, será realmente difícil saber o que você está adquirindo, até que seja tarde demais para voltar atrás na negociação. Uma

demonstração de proficiência é sempre uma boa ideia, mesmo que seja um *dry run*¹ do processo que a empresa planeja empregar.

Criando um plano de testes

Não é possível testar qualquer porção de software sem algum tipo de plano de testes. Você conheceu esse requisito nos capítulos anteriores. No entanto, criar apenas um plano genérico não será suficiente quando você estiver trabalhando com terceiros. É necessário criar um plano que detalhe o que você espera de uma empresa de testes de software. O documento deve separar o trabalho entre sua empresa e a empresa de testes. Além disso, o plano deve considerar o que é um teste bem-sucedido nessas condições. Você não deve se preocupar apenas com o estado da aplicação no que se refere a ter passado/falhado no teste, mas também se a colaboração entre sua empresa e a empresa terceirizada de testes atendeu a metas específicas, necessárias para garantir que a aplicação seja razoavelmente segura. As próximas seções descrevem como criar um plano de testes.

Especificando as metas da empresa terceirizada nos testes

O capítulo 9 discute a necessidade de ter metas para os testes de aplicação. O capítulo 11 enfatiza a necessidade de ter metas e as descreve com alguns detalhes. No entanto, essas são as metas para sua aplicação e para sua empresa como um todo. Uma empresa terceirizada de testes é um parceiro no processo de testes, o que significa que ela trabalhará em apenas uma parte do processo. As metas de uma empresa terceirizada de testes são semelhantes às suas metas, no sentido em que os testes ainda devem refletir princípios específicos; contudo, as metas agora incluem também a necessidade de informar claramente todos os critérios e as metodologias de testes. Além do mais, você deve definir metas que incluam os conceitos da estratégia de gerenciamento de sua empresa. Consequentemente, as metas descritas nos capítulos 9 e 11 são um bom ponto de partida, mas você deve expandi-las para garantir que o terceiro saiba exatamente o que é necessário e como deve responder às solicitações de relatórios.

Gerando um plano de testes por escrito

Sempre crie um plano de testes por escrito. No entanto, quando trabalhar com uma empresa terceirizada, você deve analisar o plano de testes por escrito para garantir que o papel do terceiro nos testes esteja claro e que ele compreenda totalmente todos os requisitos de teste. Quando houver dúvidas, a empresa terceirizada de testes deve ser capaz de fornecer informações que ajudem a esclarecer os requisitos de teste. De fato, essa informação tem valor inestimável como treinamento para futuros planos de testes. Uma empresa terceirizada de testes deve ser capaz de ajudá-lo a criar um plano de testes que esteja totalmente de acordo com as melhores práticas do mercado e a obter os resultados específicos que você deseja.

Como parte do plano de testes, você deve listar os testes a serem conduzidos usando a lista de serviços da empresa terceirizada de testes como entrada. Em outras palavras, o plano de testes deve definir claramente quais serviços da empresa terceirizada fazem parte do processo de testes de modo que não haja ambiguidades depois de os testes começarem. Certifique-se de entender exatamente quais testes serão executados e por que eles são necessários para garantir que a aplicação funcione conforme previsto. Uma empresa terceirizada de testes deve fazer a articulação entre a necessidade de testes e outros serviços de forma clara; caso contrário, o processo de testes poderá falhar por falta de comunicação.



É importante desenvolver um relacionamento de confiança com a empresa terceirizada de testes durante esse processo inicial. É claro que a confiança começa ao ter certeza de que a empresa terceirizada de testes realmente sabe como realizar testes de aplicação no domínio do problema. Verificar a legitimidade da empresa terceirizada é um bom ponto de partida, mas certifique-se de que seus especialistas já estejam disponíveis para administrar tarefas que especifiquem como o processo de testes deve ocorrer. Obter informações da empresa terceirizada de testes será útil somente se você souber que ela realmente está do seu lado (e que não está tentando enganar você de alguma forma).

Listando as saídas dos testes e os requisitos de relatório

O plano de testes deve conter uma seção que descreva exatamente os resultados esperados. Além disso, o plano deve detalhar o que fazer quando

os testes falharem para que as respostas sejam organizadas e bem avaliadas antes de o processo de testes começar. A empresa terceirizada normalmente pode ajudar a fornecer informações adicionais sobre saídas esperadas e, possivelmente, sobre tipos adicionais de testes individuais para que você possa verificar se sua aplicação realmente está segura contra as ameaças mais recentes. É pouco provável que sua própria equipe conheça todas as ameaças em potencial, portanto parte da tarefa da empresa terceirizada de testes é garantir que sejam feitos testes para problemas de segurança que você não conhece, além de testar problemas já conhecidos. O plano de testes deve incluir saídas detalhadas para todos os testes.

Quando um ciclo de testes estiver completo ou algum outro evento ocorrer, o plano de testes deve especificar algum tipo de relatório por escrito. O relatório não precisa estar no papel; basta estar em um formato permanente para que todos possam revisá-lo e analisá-lo conforme for necessário. Os relatórios normalmente estão associados aos testes ou a outros eventos significativos, e o plano de testes deve detalhar exatamente quais testes geram relatórios específicos.



Lembre-se de adquirir cópias de relatórios de exemplo da empresa terceirizada de testes. Certifique-se de entender o conteúdo desses relatórios. Em muitos casos, a empresa terceirizada de testes pode fazer alterações no relatório para que ele seja mais adequado ao estilo de testes de sua empresa. Além disso, talvez você precise de relatórios em um formato específico para atender aos requisitos de regulamentação ou legais, portanto certifique-se de que a empresa terceirizada de testes esteja ciente desses requisitos e que possa fornecer os relatórios no formato necessário.

Considerando os requisitos de testes

Testes são mais que simples entradas e saídas produzidas em um ambiente específico. Como parte de seu plano de testes, você deve considerar como a empresa de testes os realiza. Isso pode se tornar um ponto problemático em algumas situações. Por exemplo, talvez você só tenha os Estados Unidos em mente como mercado. Se sua empresa de testes estiver em Hong Kong, eles estarão acostumados a falar o inglês britânico. As diferenças são pequenas, mas o suficiente para deixar os testes menos úteis em algumas situações.

Considere fazer parsing de palavras que variam um pouco entre as duas formas de inglês, por exemplo, *color* e *colour* para a palavra cor. Seu código talvez não faça parse corretamente da palavra *colour*. O inverso também vale. Uma empresa de testes deve dar suporte a todos os idiomas que você pretende tratar.

Considerar o modo como você testa uma aplicação é importante sob todos os aspectos. Por exemplo, se sua aplicação foi projetada para atender a requisitos específicos de saúde e a empresa de testes não coloca ênfase suficiente em testes de acessibilidade, a aplicação poderá oferecer de tudo, exceto o essencial, que é facilitar o acesso. Não importa se a aplicação é segura quando o usuário não for capaz de interagir com ela.

Criar essa parte do plano de testes é especialmente difícil porque você já treinou as equipes de testes de sua empresa para identificar os requisitos especiais de sua aplicação. É fácil fazer suposições sobre como a empresa de testes verá os testes com base no que você já faz. A questão principal é não pressupor nada quando se trata de testes – registre tudo por escrito.

Implementando um plano de testes

Depois que tiver um plano de testes definido, você deverá considerar como ele será implementado. Ter metas é um bom ponto de partida, mas definir um processo para atingir essas metas é necessário para que os testes sejam concluídos com sucesso. Ao implementar um plano de testes, você deve considerar a necessidade de efetuar uma monitoração. Não se trata de espiar, mas de direcionar – para ajudar o terceiro a realizar os testes da maneira esperada, você deve fornecer a direção necessária. O ato de implementar o plano também pode fazer com que problemas inesperados surjam. Você precisa de um procedimento definido para tratar esses problemas, ou os testes poderão se tornar lentos até serem interrompidos. As próximas seções descrevem as técnicas necessárias para implementar um plano de testes.

Determinando a participação da empresa nos testes

É importante envolver sua empresa no processo de testes. Caso contrário, você não poderá ter certeza de que a empresa de testes está executando a

tarefa corretamente. Entretanto, monitorar o processo de testes difere substancialmente da participação direta, ou seja, de colocar a mão na massa. Você pode até mesmo optar por dividir o processo de testes em áreas funcionais – aquelas em que sua empresa já tem expertise para executar e aquelas em que a empresa de testes é especializada. O plano de testes deve definir exatamente o nível de participação de cada entidade.

Implementar o plano de testes com frequência mostra que os melhores planos no papel ainda não são totalmente úteis no mundo real. Você pode descobrir que o plano de testes não inclui todas as áreas de participação (deixando áreas a serem divididas posteriormente) ou que ele não considera a logística para executar os testes da maneira especificada. O plano de testes pode não definir o nível exigido de comunicação de forma adequada ou você pode descobrir que o processo de comunicação é prolixo, causando desperdício de tempo. Executar um *dry run* do processo de testes normalmente identifica esses pontos antes que eles se tornem um problema.

Iniciando o processo de testes

Depois de todo o planejamento e dos dry runs, é hora de iniciar o processo de testes. Você poderá tentar se convencer que o primeiro dia será bom, mas ficará desapontado. É mais inteligente supor que alguns problemas ocorrerão durante o processo de testes e que você precisará lidar com eles antes. Para o desenvolvedor, isso quer dizer que será preciso informar erros à empresa de testes, como arquivos de código-fonte faltando, o mais rápido possível. Você também poderá descobrir que a empresa de testes não tem acesso a recursos online como bibliotecas, APIs e microsserviços, necessários para executar a aplicação. A melhor ideia é reservar os primeiros dias de teste para resolver esses detalhes do sistema para que uma correria geral a fim de iniciar os testes não complique ainda mais os problemas enfrentados durante a inicialização.

Conforme a configuração, o propósito e os recursos de sua aplicação, você também deve se planejar para tratar vários problemas de segurança. A empresa de testes pode não ter acesso a recursos online pagos ou você poderá ter de conceder direitos adicionais a recursos de propriedade de sua empresa.

Certifique-se de que terá todas as pessoas necessárias para fazer a aplicação estar disponível, pelo menos nos primeiros dias de teste. Por exemplo, se sua aplicação exigir acesso a banco de dados, garanta que um DBA esteja disponível para ajudar a criar os níveis de acesso necessários. Coordenar o esforço é um bom trabalho para DevOps, supondo que sua empresa tenha alguém designado para essa tarefa. Não se esqueça de incluir a equipe de gerenciamento conforme for necessário.



Jamais dê carta branca aos recursos de sua empresa à empresa de testes. A empresa de testes deve ter acesso ao mínimo de recursos necessários para fazer a aplicação executar. Além do mais, você deve conceder o mínimo de direitos a esses recursos. Se os testadores de software não precisarem de acesso de administrador, limite esse acesso a algum nível de usuário apropriado. O principal motivo para limitar o acesso é proteger seus recursos. No entanto, mesmo supondo que a empresa de testes seja totalmente confiável, é importante que os testadores de software verifiquem a aplicação usando os mesmos direitos de todos os demais (um fato especialmente importante durante os testes de segurança).

Fazendo a monitoração necessária nos testes

Monitorar a empresa de testes é parte essencial para garantir que os testes ocorram conforme planejado. Você deve assegurar que a empresa de testes execute todos os testes exigidos (e não simplesmente se comprometa a fazê-los), mas, ao mesmo tempo, não realize testes sem autorização (que poderão ser possivelmente cobrados). Também vale a pena monitorar os recursos durante os testes. Por exemplo, o DBA deve garantir que o processo de testes não danifique nenhum dado do banco de dados, mesmo que esses dados sejam usados especificamente para testes. Algumas falhas de aplicação não são imediatamente aparentes, a menos que você faça a monitoração necessária.

Tratando problemas inesperados nos testes

Testes são um processo antagônico. Afinal de contas, você está forçando uma aplicação a falhar. Qualquer processo antagônico produzirá, ocasionalmente, resultados inesperados. Você pode passar o máximo de tempo que quiser afirmando que uma dada entrada deve gerar determinada saída, mas a ênfase

da afirmação está na palavra *deve*. O motivo para realizar testes é encontrar aquelas ocasiões em que os resultados sejam diferentes do esperado.

É claro que os testadores de software tratam a maioria desses resultados inesperados usando o sistema que você tiver implantado para informar bugs. Contudo, alguns resultados inesperados têm origens externas, como falhas de comunicação entre os membros da equipe de testes. Nesse caso, informar o problema como um bug não ajudará porque o problema não está na aplicação – é resultado de alguma origem externa atuando na aplicação ou em seu processo de testes. Para tratar esse tipo de problema de testes, é preciso ter um processo definido que funcione fora do sistema de registro de bugs.

Uma maneira comum de abordar tais problemas é realizar uma reunião extraordinária. O entrave é que as reuniões podem perder o foco, são difíceis de agendar e podem não gerar resultados úteis, pois os stakeholders precisam de tempo para pesquisar acerca dos problemas. Algumas empresas contam com fóruns privados ou wikis para tratar esses tipos de assunto. De fato, existem produtos para tratar essa questão de várias maneiras. Você pode ver uma lista de 25 ferramentas comuns para colaboração entre desenvolvedores em <https://blog.profitbricks.com/top-25-collaboration-tools-for-developers/>.

Usando os relatórios resultantes

A empresa terceirizada de testes poderia simplesmente dizer que seu software é absolutamente impressionante e que funciona totalmente sem falhas, de forma raramente vista na comunidade de desenvolvimento. No entanto, um relatório verbal não oferece meios de realizar análises mais tarde, nem de criar um plano para corrigir os problemas encontrados pela empresa terceirizada de testes. Você precisa de relatórios de vários tipos para apresentar os fatos aos vários membros de sua empresa. Muitos desses membros não terão nenhuma habilidade de programação, portanto você precisará de relatórios que descrevam o resultado de forma simples e outros que forneçam uma análise detalhada, que possa ser usada para fazer correções. As próximas seções descrevem o uso de relatórios como uma forma de usar os resultados de testes de terceiros de maneira eficiente.

Discutindo o relatório gerado com o terceiro

Os relatórios que descrevem o desempenho da aplicação durante os testes podem variar e as partes envolvidas também variam. Por exemplo, a maioria das aplicações atualmente depende de bibliotecas, APIs e microsserviços de terceiros. Você pode optar por informar problemas com esses elementos de terceiros ao fornecedor que os criou como uma forma de fazer sua aplicação funcionar melhor. É claro que você não vai querer compartilhar o relatório completo, incluindo os relatórios relacionados ao seu código pessoal, com esses fornecedores. Uma empresa terceirizada de testes pode tratar a necessidade de entrar em contato com terceiros a respeito de seus códigos, mas você precisa ter alguma política definida para administrar essa questão. Se a empresa terceirizada de testes realizar o trabalho, o stakeholder do terceiro poderá simplesmente fazer um upload de novas versões de seus códigos para a empresa de testes – simplificando enormemente o papel desempenhado por você na tarefa. (Você ainda precisará saber de qualquer alteração no código de terceiros porque essas alterações podem afetar outras aplicações.)

Os relatórios gerados pela empresa de testes devem atender a vários públicos-alvo em sua empresa. Uma equipe de gerenciamento talvez queira ter apenas uma visão geral que ofereça informações de status e os números globais. Usuários talvez queiram saber apenas dos problemas que afetam a usabilidade e o tipo de interface que a aplicação apresenta. Desenvolvedores querem relatórios detalhados que especifiquem os locais exatos dos erros e possíveis correções para eles. Um DBA talvez queira conhecer apenas as especificidades dos problemas que afetam o banco de dados. Em suma, os relatórios têm vários formatos para atender às necessidades de públicos-alvo específicos.

Apresentando o relatório à empresa

O processo de testes da aplicação gera a maior parte dos relatórios necessários automaticamente, como parte da execução do teste. Um sistema de encaminhamento pode garantir que todos que precisam de um relatório obtenham um de alguma forma, por exemplo, um email. No entanto, enviar

um relatório a um stakeholder não é exatamente o mesmo que fazê-lo ler esse relatório. É claro que a questão é por que alguém deveria se importar com esse problema, dado que o stakeholder deveria assumir a responsabilidade por ler o material. O problema é que os stakeholders tendem a ignorar as informações até que elas sejam realmente necessárias, normalmente tarde demais quando se trata de desenvolvimento de aplicações. Os stakeholders precisam permanecer envolvidos para garantir que a aplicação se comporte conforme esperado quando os testes forem concluídos.

De acordo com o modelo de desenvolvimento usado, talvez seja necessário apresentar relatórios à empresa frequentemente, por exemplo, a cada semana. Alguns modelos exigem uma reunião às segundas-feiras. Toda segunda-feira você deve apresentar um relatório do processo do projeto, qualquer problema que houver e então apresentar uma demonstração em seguida. Os stakeholders que podem ver o progresso a cada semana tendem a se envolver mais no projeto e oferecer feedbacks melhores, o que significa que você terá menos perguntas sobre a adição de recursos no final do processo, quando não é mais possível adicioná-los.

Usar as segundas-feiras como o dia da apresentação dá tempo à equipe de desenvolvimento, de testes e outras equipes para reagir a sugestões ou às necessidades para o desenvolvimento da aplicação. Em vez de tentar se apressar para fazer uma correção no domingo, as várias equipes têm a semana toda para gerar uma versão atualizada da aplicação que impressionará os stakeholders na segunda-feira. Como resultado, as várias equipes conseguem descansar e serão capazes de fazer um trabalho melhor.



O pior dia para apresentar relatórios à empresa é sexta-feira. Para começar, ninguém está prestando atenção, porque as pessoas já estão pensando nos planos para o fim de semana. Mesmo que prestem atenção, elas estarão cansadas após uma semana de trabalho e menos dispostas a dar opiniões criativas realmente necessárias para projetar uma aplicação atraente que faça um trabalho útil. As várias equipes também se sentirão obrigadas a trabalhar no fim de semana para iniciar as correções enquanto a necessidade de implementá-las está fresca na memória. Se quiser obter informações práticas sobre sua aplicação, evite fazer seus relatórios às sextas-feiras.

Atuando sobre as recomendações dos testes

O resultado de testes, relatórios, discussões, e assim por diante, é que as várias equipes precisam corrigir os problemas encontrados pelos vários testadores de software. Alguns problemas são mais importantes que outros e são mais prováveis de causar problemas após a implantação da aplicação, portanto você deve trabalhar neles antes. Parte da interação entre a equipe de testes e o stakeholder consiste em atribuir valores a cada problema, que definam a sua importância. Por exemplo, você não vai querer fazer uma correção de baixa prioridade e encerrar o problema antes de corrigir aquele problema importante que faz a aplicação falhar nos computadores de praticamente todas as pessoas.

No entanto, apenas corrigir o problema não o encerra. Em alguns casos, o problema tem mais de uma resposta e você pode optar por criar versões da aplicação que implementem cada resposta a fim de determinar qual delas é a melhor. Os vários stakeholders precisam avaliar as possíveis correções e decidir qual delas funciona melhor para eles (pode não ser a mesma correção em todos os casos). Criar e exercitar todas as correções úteis possíveis para um problema faz parte de atuar nas recomendações dos testes.

Também é importante que as correções sejam verificadas pelos testadores de software. No entanto, você pode decidir que a correção não será testada imediatamente. Talvez uma interação faça com que seja difícil ou impossível testar a correção sem corrigir outros problemas da aplicação antes. É essencial olhar para as dependências de um problema em particular e garantir que elas também sejam tratadas, assim como o problema original.

¹ N.T.: *Dry run* é a execução mental de um processo antes de colocá-lo em prática.

PARTE IV

Implementando um ciclo de manutenção

Softwares podem falhar e falham. No mínimo, ficam velhos e param de interagir com hardwares mais novos. Em alguns casos, os usuários ultrapassam a capacidade do software ou percebem que ele não atende mais às suas necessidades. O código novo, empolgante e cuidadosamente polido que você entrega hoje logo ficará enferrujado e decadente, como uma sombra do código que havia sido ontem – uma peça de museu que deixará as pessoas boquiabertas se ele se tornar suficientemente famoso. É difícil ouvir falar que seu código, em algum momento, acabará no lixo após ter investido tanto tempo e esforço desenvolvendo-o. É claro que você pode evitar esse destino simplesmente fazendo a manutenção de seu código, que é o ponto principal desta parte do livro. Se realmente quiser que sua aplicação ofereça serviços pelo máximo de tempo possível, você deverá ter um plano definido para mantê-lo viável desde o princípio.

Esta parte do livro divide a tarefa de manutenção do software em três partes distintas: determinar quando fazer uma atualização no software (capítulo 13), decidir como fazer a atualização (capítulo 14) e usar relatórios para monitorar o requisito da atualização (capítulo 15). Ao seguir esse processo de três partes, você pode manter sua aplicação polida e reluzente de modo que as pessoas continuem a admirá-lo e a usá-lo com sucesso.

CAPÍTULO 13

Definindo claramente os ciclos de upgrade

Sua aplicação está constantemente exposta a ameaças online, e o mesmo ocorre com todos os outros códigos dos quais sua aplicação depende. Para reduzir os riscos resultantes de ameaças conhecidas, você deve fazer upgrades regularmente em sua aplicação. Os seguintes upgrades ajudam a melhorar sua aplicação de várias maneiras, e todos eles afetam a segurança:

- Melhoria em usabilidade para reduzir erros de usuário
- Correção de erros explícitos de programação
- Melhorias de velocidade que evitam que usuários façam algo inesperado
- Correções em códigos funcionais que reduzem o potencial para brechas de segurança
- Correções necessárias para upgrades em bibliotecas, APIs e microsserviços de terceiros

Este capítulo vê o processo de upgrade a partir de vários níveis. Você não pode simplesmente fazer correções de qualquer maneira, sem esperar que surjam problemas em vários lugares, principalmente usuários irritados. Os upgrades exigem certa dose de planejamento, testes e então de implementação. As próximas seções discutem os três níveis de upgrade e ajudam a criar um plano que faça sentido para suas próprias aplicações.

Desenvolvendo um plano detalhado de ciclo de upgrade

Parte do processo de introdução de upgrades em sua aplicação é criar um plano para implementar as alterações. O problema é que, com frequência,

você não saberá dos upgrades necessários aos softwares de terceiros, não entenderá os requisitos para criar o upgrade, não poderá colocar os upgrades em uma ordem em particular e encontrará problemas com os planos de implementação de upgrades. É essencial criar uma maneira organizada de avaliar as atualizações, que é o motivo para criar um plano de ciclo de upgrades. A figura 13.1 mostra um processo que pode ser seguido na execução dessa tarefa.

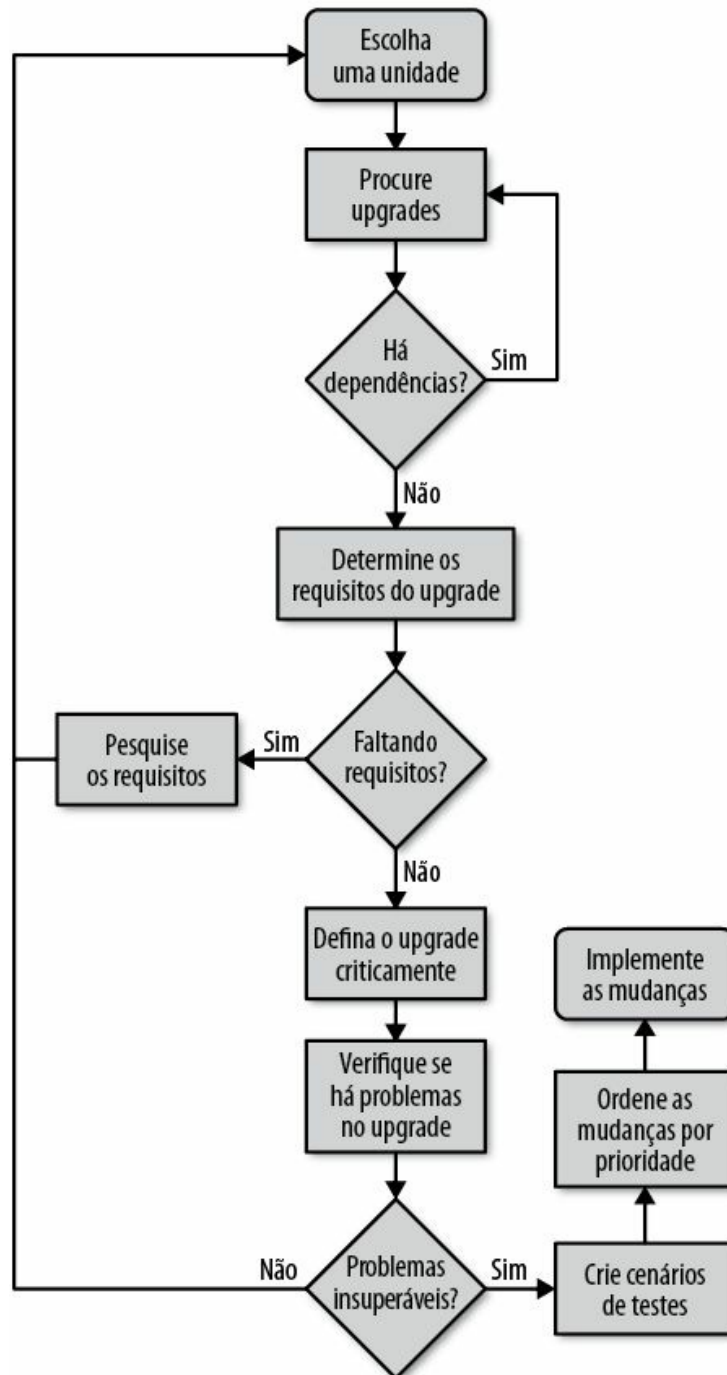


Figura 13.1 – Seguir um processo organizado ao criar um plano de ciclo de upgrades economiza tempo e dinheiro.

A ideia é trabalhar com uma unidade de cada vez e garantir que você testará tudo que está associado a essa unidade. Mesmo que uma dependência em particular seja testada como parte do teste de outra unidade, você também deve testá-la com a unidade em observação porque essa unidade pode usar a

dependência de modo diferente ou afetá-la de outra maneira.

Agora que você tem uma ideia do tipo de processo que pode seguir, é preciso conhecer os detalhes da execução da tarefa. As próximas seções discutem os problemas em torno de um plano de ciclo de upgrades. Após ler esta seção, você deverá ter uma ideia melhor de como criar um plano que atenda especificamente às necessidades de sua aplicação.

Procurando upgrades

Os registros de problemas (trouble tickets), coletados pela equipe de suporte como resultado de reclamações de usuários e interrupções do sistema, normalmente informam o que você precisa saber sobre upgrades para o código de sua própria aplicação. Se os registros de problemas não informarem tudo que for preciso, no mínimo oferecerão informações suficientes para fazer a pesquisa necessária e descobrir exatamente quais upgrades você deve criar. No entanto, softwares de aplicações de terceiros são uma questão diferente. A menos que um desenvolvedor ou outra parte interessada localize uma correção para um problema como resultado de depuração de código interno, um upgrade poderá permanecer invisível. É importante procurar upgrades em softwares de terceiros e garantir que você tenha uma lista completa deles. Caso contrário, não será possível preservar o estado de segurança de sua aplicação, e o risco desconhecido em potencial para uma brecha de segurança aumenta. Muitas empresas acabam não percebendo que deveriam fazer um upgrade.



Alguns terceiros oferecem notificações automáticas para atualizações. O uso de notificações automáticas é uma maneira de garantir que você tome conhecimento dos upgrades, mas você não poderá depender delas. Uma notificação informa que uma atualização está disponível, mas você também deve ser proativo e verificar se há atualizações sobre as quais não ouviu falar. Mesmo que um fornecedor seja diligente em oferecer notificações, elas ainda podem se perder ou acabar na pasta da lixeira.

Conferir o site do fornecedor pode informar se há upgrades que devam ser incorporados em sua aplicação quando trabalhar com bibliotecas, APIs e microsserviços públicos. Em muitos casos, os fornecedores oferecem versões beta de upgrades para que você possa começar a trabalhar com eles com

antecedência. Vale a pena investir tempo e analisar exatamente o que o upgrade afeta para que você saiba a importância que deve atribuir a ele no que diz respeito à sua aplicação. Um upgrade que não afete nenhuma das funcionalidades de sua aplicação é menos importante que um upgrade que afete todas as funcionalidades dela. No entanto, você ainda deve incorporar o upgrade em algum momento, mesmo que tenha certeza de que sua aplicação não será afetada, pois os upgrades muitas vezes têm interações com dependências que até mesmo o fornecedor não conhece (ou não testou completamente).

Artigos na imprensa especializada e comunicados do fornecedor à imprensa também informam sobre upgrades que estão por vir. Além disso, você pode ficar atento a artigos que mencionem qualquer deficiência no produto que você está usando e nos planos do fornecedor para corrigi-las. Todas essas informações alimentam seus planos de upgrade. Você sabe com antecedência o que provavelmente acontecerá para que possa começar a pensar no assunto bem antes de fazer o trabalho.

Não menospreze os efeitos da comunicação pessoal quando chegar a hora de fazer um upgrade. Alguém que trabalhe para o fornecedor muitas vezes pode informar você sobre um upgrade antes que qualquer outra pessoa tome conhecimento. Ficar sabendo sobre um upgrade com antecedência dá a você uma vantagem competitiva, pois será possível se preparar para o upgrade antes de todos os demais. Tempo é um recurso crítico no mercado de computação, e a empresa que lança upgrades com frequência assim que eles sejam necessários é aquela que preserva e atrai novos clientes. A capacidade de planejar também reduz o estresse da equipe e possibilita a todos criar upgrades melhores (upgrades feitos às pressas podem acarretar mais trabalho que os problemas que eles corrigem).

À medida que tomar conhecimento dos upgrades necessários (seja para seu próprio código ou para códigos fornecidos por terceiros), é preciso colocar informações sobre eles em uma lista. Todos que estão em seu grupo de desenvolvimento devem ter acesso à lista e devem se manter informados quanto aos requisitos de upgrades futuros. Incentive sua equipe a se comunicar a respeito de upgrades em potencial e sobre os efeitos que eles

terão em sua aplicação. Tudo que você puder fazer para manter as pessoas pensando nos upgrades e como realizá-los adequadamente reduzirá o trabalho que deverá ser feito depois, quando o tempo for crítico.

Determinando os requisitos do upgrade

Conhecer as especificidades de um upgrade é um bom passo inicial. No entanto, não é bom começar fazendo algo concreto a respeito do upgrade. Você deve determinar os requisitos para fazer o upgrade. Em alguns casos, um upgrade pode causar os mesmos problemas de logística criados por uma nova aplicação. Prever quanto tempo um upgrade complexo demorará e quantos recursos ele exigirá pode ser quase impossível. A lista a seguir apresenta algumas ideias do que deve ser considerado como parte da especificação de requisitos de um upgrade:

Determinar os requisitos para recursos

Nem todos os upgrades exigem recursos adicionais – o fato surpreendente é que a maioria exige. Os recursos podem incluir de tudo: memória, ciclos de processamento, espaço em disco rígido, fontes de dados, espaço disponível na tela, tipos de entrada (por exemplo, sensores), banda de rede, e assim por diante. A lista pode se tornar bem extensa e você deve considerá-la cuidadosamente, pois mudanças em recursos podem ter efeitos colaterais desagradáveis, como a incapacidade de executar a aplicação em um dispositivo com que o usuário contava no passado.

Identificar qualquer mudança necessária na plataforma

Ficar de olho em funcionalidades obsoletas é sempre uma boa ideia. Funcionalidades obsoletas provocam todo tipo de problemas porque, às vezes, elas não levam em consideração plataformas mais antigas. Você deve considerar exatamente como uma mudança de plataforma afetará os usuários. Talvez um upgrade exclua um sistema operacional mais antigo, porém popular, que os usuários amam. Em vez de ouvir reclamações intermináveis sobre a mudança de plataforma, você deve ter informações sobre elas e considerar os efeitos em sua empresa, na aplicação e nos usuários antes de iniciar o processo de implementá-la.

Considerar o tempo necessário para criar, testar e implantar o upgrade

O requisito mais difícil de upgrade a ser considerado é o tempo necessário para criar, testar e implantar o upgrade. Não importa o cuidado com que você tente planejar esse requisito, com muita frequência, ele deixa a desejar. A maioria das pessoas prevê que poderá realizar a tarefa em muito menos tempo do que realmente será necessário. Quando determinar esse requisito do upgrade, certifique-se de usar números realistas e acrescentar tempo e recursos para aquelas emergências não planejadas que sempre ocorrem em um projeto de software.



Há muitas causas para problemas quando tentamos calcular um requisito de tempo. Um dos problemas que você não verá discutido em livros com muita frequência é a necessidade de levar em consideração o elemento humano. As pessoas de sua equipe terão muitas tarefas pressionando-as enquanto trabalham no projeto. Se acharem que podem adiar seu projeto por um tempo para realizar uma tarefa mais crítica, definitivamente elas o farão. Esse é o fator procrastinação, e você deve lidar com isso de forma positiva como parte de seu processo de planejamento. A menos que os membros de sua equipe achem que seu projeto tenha um aspecto crítico, o fator procrastinação sempre aparecerá em seu caminho para causar problemas. O projeto que poderia ser concluído no prazo repentinamente torna-se um projeto atrasado, totalmente por causa do fator humano que você não havia levado em consideração em seu planejamento.

Criar uma lista de pessoas e habilidades necessárias para criar o upgrade

Um problema comum com projetos de upgrade é o fato de o planejamento original não incluir uma lista de pessoas e habilidades necessárias para executá-lo. A tendência é pensar que um pequeno subconjunto de pessoas, ou talvez apenas uma pessoa, possa realizar o upgrade sem qualquer problema. Infelizmente, não é isso que acontece. Um upgrade exige planejamento nos mínimos detalhes, do mesmo modo que um novo projeto. Você precisa ter acesso aos mesmos especialistas exigidos pelo projeto original para garantir que o upgrade atenda aos mesmos padrões elevados.



Muitas vezes nos esquecemos de incluir os usuários como parte da equipe em um upgrade. Como resultado, o upgrade causa falhas nas interfaces de usuário e provoca outros problemas que um usuário veria rapidamente, porém um desenvolvedor deixaria passar por completo. Lembre-se de incluir todas as pessoas necessárias como parte de seu processo de upgrade para garantir que acabará com uma atualização que realmente funcione e permaneça segura.

Definir o efeito em potencial do upgrade nas fontes de dados

Os upgrades podem afetar fontes de dados de várias maneiras. Por exemplo, você poderá descobrir que precisa acrescentar novos campos nos bancos de dados ou que deverá acessar uma fonte totalmente nova de dados. A fonte de dados pode ser uma entrada de um sensor ou outra informação não tradicional que sua aplicação agora exige para poder funcionar de forma apropriada. Pense em todas as entradas que a aplicação poderia exigir quando considerar as ramificações de um upgrade.

De modo diferente das aplicações antigas, você não pode mais considerar apenas a sua aplicação como parte da lista de requisitos ou as necessidades de sua empresa como o ambiente de operação. A lista de requisitos de seu upgrade deverá considerar o uso de bibliotecas, APIs e microsserviços de terceiros como parte do quadro geral. Além disso, você deve considerar os ambientes em que essas partes de sua aplicação executam como parte da lista geral de requisitos.

Também é importante lembrar-se de seus usuários em tudo isso. Quando os usuários dependiam de sistemas desktop, era fácil criar um ambiente controlado em que as necessidades deles eram fáceis de serem consideradas. Atualmente, um usuário pode tentar usar sua aplicação em um smartphone no restaurante local. A aplicação pode ser usada enquanto o usuário estiver viajando de carro como passageiro (espera-se que ele não seja o motorista) ou quando estiver usando um transporte alternativo, por exemplo, o trem ou o ônibus local. Os requisitos de upgrade devem considerar todo tipo de dispositivos usados em todos os tipos de ambiente.

Definindo a criticidade do upgrade

Os upgrades afetam aplicações de várias maneiras e o risco de não disponibilizar um upgrade varia de acordo com a aplicação, a plataforma e as necessidades. A criticidade de um upgrade é diretamente proporcional ao risco representado por não o aplicar. À medida que um upgrade se torna mais crítico, sua prioridade na lista de upgrades aumenta. Parte do ciclo de planejamento envolve a atribuição de prioridades aos upgrades de modo que sua empresa possa aplicar os de mais alta prioridade primeiro. Do contrário,

you will not be able to be sure that the upgrade process will actually represent a reduction of risks to your company. When working with priorities, consider the following criteria:

Riscos à segurança

Security upgrades always have high priority because someone has already proven that they offer some method of causing damage to your system. However, not all security upgrades represent an immediate risk, since no known hacker uses a vulnerability for an attack or the attack requires a special condition, for example, physical access to the computer. When assigning priorities according to the need for the security upgrade, consider the level of risks that the security problem imposes on your application.

Potencial para danos

Upgrades that correct a problem that could result in some type of damage to the application or the data are normally at the top of the list of upgrades. According to the risk imposed by a problem, upgrades that correct problems can be more prioritized than security upgrades in many situations. In this case, you must be sure of the probability of this type of damage occurring and the respective result.

Potencial para erros

Errors created by user input or other problems can represent a significant risk, but you can also overcome many of these errors with additional training for users or by means of the implementation of new policies in the company. Even so, you must assign a priority according to the probability of the error occurring, the damage that the error could cause and the company's ability to overcome, identify and correct the error.

Eventos naturais

Some upgrades are aimed at possible natural events that can cause errors in applications. The presence of noise on the line can cause errors in a single bit in a flow of data, so a solution would be to include

rotinas de ECC (Error-Correcting Code, ou Código de Correção de Erros) para eliminar e corrigir erros simples de bits. Na maioria dos casos, com o surgimento de tecnologias que corrigem automaticamente a maior parte desses erros abomináveis, você poderá atribuir uma prioridade menor a eles. Ainda será necessário criar upgrades para corrigi-los, porém os problemas de segurança que definitivamente causarão transtornos devem vir antes.

É essencial atribuir valores à sua lista de prioridades. Esses valores facilitam determinar se uma mudança é crítica (porque corrige uma brecha de segurança) ou é simplesmente conveniente (porque faz uma mudança na interface, solicitada pelo usuário, deixando-a mais adequada para melhorar a usabilidade). A figura 13.2 mostra a possível inter-relação entre os vários critérios de prioridade quando usamos um sistema de prioridades de cinco níveis (seu sistema pode conter um número diferente de níveis).

É importante atualizar constantemente as prioridades dos upgrades que você deve criar para sua aplicação. À medida que criar novos requisitos de mudança, é importante aumentar as prioridades dos upgrades existentes de acordo com isso. De modo semelhante, o acréscimo de novos requisitos de mudança pode reduzir a prioridade de alguns upgrades existentes. Para que uma lista de upgrades necessários cumpra os propósitos para os quais foi criada, você deve mantê-la atualizada.

	Crítico	Essencial	Normal	Baixo	Acessório
Risco à segurança					
Potencial para danos					
Potencial para erros					
Eventos naturais					

Figura 13.2 – Criar um equilíbrio entre as prioridades é uma parte essencial do processo de testes.

Verificando upgrades para ver se há problemas

Sempre que você fizer um upgrade de um produto, há um potencial para problemas de compatibilidade ou para outros problemas passarem despercebidos até o upgrade ser transferido para o ambiente de produção. A essa altura, é realmente tarde demais para fazer algo sobre os problemas, exceto reagir a eles e esperar pelo melhor. Uma solução mais adequada é procurar possíveis problemas de upgrade durante os testes, o que significa ter um grupo de testes substancial ou contar com uma empresa terceirizada de testes (veja o capítulo 12 para conhecer os detalhes). No entanto, mesmo antes de chegar à fase de testes – até antes de o upgrade ser implementado – você pode usar várias técnicas para descobrir se o upgrade criará problemas. A lista a seguir descreve algumas das técnicas mais comuns:

Verificar o site do fornecedor

Os fornecedores muitas vezes oferecem descrições detalhadas dos problemas conhecidos em seus produtos, incluindo problemas de compatibilidade. Na verdade, para levar um produto adiante, às vezes o fornecedor pode introduzir propositalmente uma mudança significativa que, com certeza, causará problemas em sua aplicação. Depois que estiver ciente desses tipos de problema, você deverá considerar se as implicações relativas à segurança e à confiabilidade do upgrade valem a pena quando comparadas às novas funcionalidades obtidas.

Observar fóruns dedicados ao produto ou ao fornecedor

Em alguns casos, um fornecedor não está ciente de uma mudança que causa falhas ou não está disposto a admiti-la. Outros usuários informarão você sobre mudanças que causam falhas em potencial. Aquelas mensagens de erro estranhas que ficam aparecendo estão informando que há um problema em potencial com o software. Quando começar a ver padrões em mensagens de fóruns, é preciso investigar melhor o upgrade antes de usá-lo em sua própria aplicação e sofrer as mesmas consequências que outros estão vivenciando.

Ler a imprensa especializada

As piores mudanças causadoras de falhas normalmente aparecem na

imprensa especializada em algum lugar. Desenvolvedores especializados que também têm uma carreira como escritores estão sempre procurando mudanças em produtos que fazem com que seja quase impossível usá-lo corretamente. Em alguns casos, você até encontrará soluções para contornar os problemas. Quando uma mudança causadora de falhas aparecer na imprensa especializada, você pode ter certeza de que a maioria dos fornecedores já está trabalhando em uma correção para o problema e pode ser melhor esperar pela correção, em vez de implementar o upgrade imediatamente.

Realizar testes diretamente no novo código do terceiro

O uso de testing harnesses e de scripts de teste (veja o capítulo 11) é uma parte essencial da procura por problemas em potencial. Os testes podem não revelar todos os problemas, mas quando um problema for suficientemente significativo, você pode apostar que os testes oferecerão um indício.

Desenvolver um cenário de testes para um subconjunto da funcionalidade

Se você suspeita que uma mudança causadora de falhas está no código, mas não consegue encontrar uma fonte confiável de informações sobre ela, experimente criar uma pequena aplicação para testes que foque nessa funcionalidade em particular. Criar uma aplicação de teste que execute apenas uma parte da biblioteca, da API ou do microserviço não consumirá muito tempo e ajudará consideravelmente se você encontrar o problema. A aplicação de teste pode servir como um meio para desenvolver uma solução para contornar o problema ou para saber quando o fornecedor corrigiu o problema.

Obter informações de especialistas de empresas terceirizadas

Algumas empresas de testes e muitos consultores podem oferecer informações especializadas sobre bibliotecas, APIs e microserviços, pois eles trabalham com o código o dia todo de uma maneira que visa a causar falhas. Dedicar todo seu tempo de trabalho testando intensivamente o código com certeza proporciona vantagens que não serão obtidas se você apenas trabalhar com ele para criar uma aplicação.

Muitas empresas tomam a decisão de introduzir várias alterações como parte de



um único pacote consolidado de upgrade, acreditando que um único upgrade é bem melhor que vários. É verdade que um único upgrade pode ser menos desestabilizador quando tudo ocorrer conforme planejado. Porém, o fato é que o procedimento raramente ocorre conforme planejado e a inclusão de vários upgrades em um único pacote complica o processo de identificar o que deu errado. Usar uma abordagem em fases para introduzir alterações é sempre melhor. O ambiente de produção normalmente apresentará alguns problemas que você não viu no ambiente de testes, portanto fazer as alterações em etapas ajuda a localizar rapidamente esses novos problemas para que você possa eliminá-los antes que eles se tornem um grande problema.

É pouco provável que você vá encontrar uma atualização de código de terceiros que não tenha problemas. É quase certo que qualquer mudança no código introduza bugs, alterações que causarão falhas no código, mudanças de interface e de estratégia de design, além de todo tipo de problema que simplesmente é inevitável, porque uma mudança realmente pode ser representada como uma moeda de duas faces, com as boas alterações de um lado e as ruins de outro. Virar a moeda geralmente determina se você acabará com o lado bom ou com o lado ruim das alterações. (É claro que as moedas também têm uma borda e algumas alterações realmente são neutras.)

Em algum momento você precisará decidir se os problemas que encontrar causarão muitos problemas a ponto de o tempo gasto com o upgrade não valer a pena. Quando decidir que o upgrade não compensa pelo tempo de implementação gasto, você também deve considerar os problemas de dependência. O código que você rejeitou pode afetar outros códigos atualizados ou a sua aplicação de alguma forma que você não poderá prever sem fazer uma análise. Vale a pena investir tempo e determinar se os problemas realmente causarão sofrimento suficiente a ponto de deixar o upgrade inviável. Cancelar um upgrade de aplicação e esperar pelas correções é preferível a lançar um software com falhas, que irritará os usuários e possivelmente causará problemas de segurança.

Criando cenários de testes

Para testar um upgrade, muitos dos processos descritos nos capítulos 11 e 12 são seguidos. No entanto, o processo para upgrades exige que você realize

alguns testes obrigatórios a fim de garantir que o upgrade funcione conforme esperado. O uso de casos de teste pode ajudar a garantir que um upgrade seja totalmente testado antes de disponibilizá-lo. A lista a seguir apresenta ideias sobre os tipos de cenários de teste que você deve incluir como parte do processo de testes de um upgrade:

Conteúdo de registro de problema (trouble ticket)

Todo registro de problema a que seu upgrade visar deve incluir informações suficientes para criar um caso de teste. Quando esse não for o caso, você deve trabalhar junto ao desenvolvedor responsável para pesquisar uma correção para o problema. Todo bug que você corrigir em uma aplicação (seja um erro de implementação ou outro problema não relacionado a código) exige um caso de teste para garantir que o código alterado seja totalmente testado.

Ações necessárias (action items)

Às vezes, um problema não gera um registro de problema porque você não pode reproduzir esse problema ou não é capaz de corrigi-lo por outros motivos (por exemplo, por falta de acesso ao código). Mesmo quando você só suspeita de um problema, é importante tentar criar um caso de teste para identificá-lo. Uma mudança de código muitas vezes faz um erro intermitente se tornar visível, criando o potencial para corrigi-lo. No entanto, você só encontrará essas oportunidades por meio de testes.

Mudanças em código de terceiros

Toda mudança em código de terceiros deve gerar um caso de teste específico em sua aplicação. Você precisa saber se a mudança no código do terceiro não afetou sua aplicação de forma negativa. Infelizmente, com frequência, você encontrará esse tipo de problema durante os testes de integração, a menos que tenha um ótimo test harness ou conjunto de scripts de teste para conferir o código do terceiro.

Mudanças em ambiente e em configuração

Todo tipo de mudanças em ambiente e em configuração pode causar problemas. Por exemplo, a política de uma empresa pode mudar o modo

como a área de TI concede acesso a recursos aos usuários. A mudança é positiva porque deixa os recursos mais seguros e reduz o acesso a eles para que a aplicação execute mais rápido. No entanto, a falta de acesso também pode provocar problemas aos usuários, portanto você deve garantir que eles ainda tenham acesso aos recursos necessários. Outros tipos de mudanças de ambiente e de configuração podem ter efeitos semelhantes na aplicação, e você não os identificará até realizar os testes necessários.

Implementando as mudanças

A essa altura você já considerou todos os elementos de um upgrade. O ponto principal em todo esse trabalho é garantir que o upgrade:

- funcione conforme previsto;
- permaneça estável;
- ofereça a segurança apropriada;
- melhore a confiabilidade;
- crie uma ótima interface de usuário;
- sele qualquer brecha de segurança.

O processo de implementação deve começar pelos itens de mais alta prioridade em sua lista, em vez de tentar implementar tudo de uma só vez. Um problema que muitas empresas enfrentam é o fato de um upgrade se tornar muito pesado. Com frequência as mudanças ocorrem, mas à custa de testes e da garantia de que elas atendam ao conjunto de especificações definido para elas. Como resultado, o upgrade acaba precisando de um upgrade. Em algum momento, o código se torna tão macarrônico que ninguém o compreende totalmente e, com certeza, ninguém poderá garantir a sua segurança. Você acabará com o tipo de software que os hackers amam porque é fácil passar por qualquer defesa implementada no software e é mais fácil ainda ocultar o hack.

Criando um cronograma para teste de upgrade

Em algum momento você dará início à implementação de seu upgrade. Você

não vai querer esperar até que o processo tenha avançado muito para começar a testar. É importante começar a testar o mais cedo possível, especialmente quando trabalhar com uma empresa terceirizada de testes. Quanto mais cedo você puder encontrar problemas em potencial, mais baixo será o custo e menos tempo será consumido para corrigi-los. Considere o uso de simulação (mocking) para que você possa realizar testes de upgrade à medida que as funcionalidades se tornarem disponíveis, em vez de esperar o upgrade como um todo, por causa das dependências. Com isso em mente, as próximas seções mostram como criar e implementar um cronograma de testes para seu upgrade.

Executando os pré-testes necessários

Depois que o processo de upgrade começar, você deve iniciar o processo de testes imediatamente. Muitas pessoas se perguntam o porquê da pressa, mas a maioria das empresas fica sem dinheiro antes de acabar de corrigir tudo que é necessário. Para garantir que os testes assumam seu devido lugar, você deve começá-los imediatamente. Execute os seguintes níveis de testes durante o processo de implementação do upgrade:

Testes de unidade

Certifique-se de testar cada mudança à medida que concluir a implementação. Caso contrário, você não poderá ter certeza de que o código funcionará. Usar simulação ajuda a deixar o processo de testes viável nesse nível.

Testes de dependência

As mudanças que você fizer afetarão outros códigos. Dentro do razoável, teste as dependências em códigos relacionados para facilitar o caminho para os testes de integração com uma boa chance de sucesso.

Testes de configuração

É importante garantir que os parâmetros de configuração continuem proporcionando o efeito e o nível de flexibilidade esperados pelos vários especialistas de TI. Do contrário, a aplicação falhará durante os testes de integração, quando você começar a configurá-la de várias maneiras.

Testes de segurança

Você não pode simplesmente supor que os reparos feitos no código realmente funcionarão conforme esperado. Às vezes, uma correção de segurança não corrige realmente o problema. Você precisa executar testes no código que falhou para determinar se a correção acabou com a brecha. Além disso, é necessário certificar-se de que a correção não abriu novas brechas.



Alguns desenvolvedores supõem que fazer testes de segurança quer dizer procurar brechas no código. Sim, esse é um tipo de teste de segurança. No entanto, você deve testar todos os aspectos da segurança. Por exemplo, um nível comumente menosprezado nos testes de segurança é garantir que os usuários acabem assumindo os papéis apropriados e que tenham os níveis de acesso exigidos. Um gerente realizando uma tarefa de nível de usuário deve desempenhar um papel de usuário, e não um papel gerencial. Um hacker pode usar problemas de papéis para ter acesso ao sistema com um nível mais alto de privilégios do que deveria ser possível.

Testes de conectividade de dados

Aplicações administram dados. Se não fosse assim, não haveria muitos motivos para criar uma aplicação. Parte de seus testes deve garantir que a unidade em teste ainda consiga se conectar com as fontes de dados necessárias e administrá-las com sucesso. Às vezes, é surpreendente descobrir que uma correção em um problema de segurança ou em outro problema acaba deixando os dados inacessíveis ou prejudicam a capacidade da aplicação de gerenciar os dados de forma segura.

Executando os testes de integração necessários

À medida que criar o upgrade, você deve observar se há problemas de integração. Como você não estará criando o código do zero, problemas de integração podem ser mais difíceis de localizar. Você estará misturando códigos novos, com os quais todos trabalharam recentemente, com códigos antigos, que ninguém havia visto há um bom tempo. A capacidade de a equipe de desenvolvimento compreender os problemas de integração é menor se comparada à situação em que ela estava trabalhando exclusivamente com um código novo. Como resultado, você precisa realizar o tipo de testes de

integração descrito no capítulo 11, mas de olho naquelas interfaces entre o código novo e o código antigo.

Lembre-se de testar usando o testing harness e os scripts existentes a fim de garantir que a aplicação continue atendendo aos requisitos básicos. Talvez você precise fazer atualizações no testing harness e nos scripts para garantir que eles ofereçam as funcionalidades de testes para os novos recursos e remover testes de recursos obsoletos. Mantenha um log das alterações que você fizer no testing harness e nos scripts. Caso contrário, você poderá achar difícil reverter as alterações mais tarde, caso um de seus upgrades se mostre incorreto.



Ter alguém disponível que realmente tenha trabalhado com o código antigo e o entenda totalmente é conveniente. Dado que as pessoas mudam de emprego regularmente, talvez você precise encontrar um bom candidato e contratá-lo como consultor para ajudar com os problemas que surgirem no código mais antigo. Contratar alguém que tenha familiaridade com o código antigo permitirá economizar bastante tempo e esforços para entender esse código. Além do mais, alguém que tenha trabalhado na equipe de desenvolvimento anterior poderá explicar por que alguns códigos funcionam, como o fazem e por que a equipe implementou o código de determinada maneira. A documentação deveria fornecer esse tipo de informação, porém ela normalmente é inadequada e escrita de forma precária de modo que, às vezes, ela gera mais perguntas que respostas.

Passando um upgrade para o ambiente de produção

Depois que o teste terminar e você tiver certeza de que o upgrade funcionará conforme anunciado, é preciso passá-lo para o ambiente de produção. Se sua empresa é de grande porte, experimente passar o upgrade apenas a um grupo de usuários antes, de modo que, caso eles encontrem um erro, você possa corrigi-lo sem deixar toda a empresa offline. O aspecto importante a ser lembrado é que a aplicação deve permanecer confiável e segura com o upgrade instalado, mas deve oferecer a velocidade e a experiência de interface exigidas pelo usuário. Do contrário, o upgrade falhará, pois os usuários se recusarão a usá-lo. Eles tentarão usar a versão com a qual se sintam mais à vontade, mesmo que essa versão provavelmente provoque violação nos dados e outros problemas de segurança.

Parte do motivo para empregar usuários como testadores de software é garantir



que eles informem outros usuários sobre o novo upgrade. Quando os usuários se empolgam com um upgrade, você tem uma resposta melhor e levará menos tempo para que todos comecem a usar o novo produto. É claro que você deve garantir que a experiência do testador seja boa.

Do ponto de vista de segurança, muitas vezes fazer testes de invasão em um upgrade usando uma empresa terceirizada de testes pode ajudar. Sim, a maioria das pessoas diria que você deveria concluir todos os testes necessários no servidor de testes, e essas pessoas estarão certas na maioria das vezes. No entanto, testes de invasão não verificam apenas se a aplicação tem erros. Eles também possibilitam fazer uma verificação de sanidade nos seguintes itens:

- Conectividade com a rede
- Software de detecção de invasão
- Software de prevenção de invasão
- Confiabilidade de DMZ¹
- Atualizações de firmware necessários a todos os hardwares
- Configuração do servidor
- Updates necessários a todos os softwares
- Ambiente da aplicação
- Bibliotecas, APIs e microsserviços de terceiros
- Aplicação
- Processos de usuário
- Conectividade com banco de dados

Simplesmente não é possível conferir todos esses itens usando a instalação de teste. Por exemplo, você pode verificar a vulnerabilidade do servidor de produção a ataques usando a instalação de teste. Embora possa parecer que alguns desses problemas de teste estejam muito longe do escopo de passar um upgrade para o ambiente de produção, qualquer mudança na configuração do servidor, incluindo o software da aplicação, realmente exige um teste desse tipo. As interações entre os vários componentes tornam impossível saber se o servidor continua confiável e seguro após a mudança. Infelizmente,

muitas empresas deixam a desejar quanto ao trabalho nessa área porque ninguém parece entender o que aconteceu com a configuração do servidor. Em última instância, o servidor poderia agora estar vulnerável e você nem ficaria sabendo.

Testar o software não é a única tarefa que você deve considerar. Também é importante obter feedback dos usuários sobre as mudanças na interface. O usuário trabalha com a aplicação em um nível detalhado durante muitas horas, todos os dias. É provável que o usuário vá encontrar possíveis problemas de segurança resultantes das mudanças de interface, antes que a equipe de desenvolvimento possa fazê-lo. De fato, a experiência mostra que a equipe de desenvolvimento muitas vezes não tem ideia de que uma determinada mudança provoca um problema que é totalmente óbvio aos usuários da aplicação. Não é porque a equipe de desenvolvimento seja indiferente ou não faça o trabalho corretamente – é simplesmente o fato de que o usuário tem um ponto de vista bem diferente de como a aplicação deve funcionar.

Fazer uma verificação junto a administradores, DevOps, DBAs e outros indivíduos especializados também é prioridade em um upgrade. Você precisa descobrir se as mudanças afetam esses usuários especializados de alguma maneira negativa. Por exemplo, mudanças em configuração que demorem muito para se propagar pelo sistema representam possíveis problemas de segurança. Se demorar 24 horas para que a remoção de um usuário tenha efeito, um usuário insatisfeito poderá causar todo tipo de problemas que seriam difíceis de rastrear. É preciso apenas um problema desse tipo para provocar todo tipo de infortúnios a uma empresa, portanto garantir que os profissionais de TI de sua empresa possam fazer seu trabalho é parte essencial do upgrade.

¹ N.T.: “Em segurança de computadores, uma DMZ ou zona desmilitarizada (do inglês *demilitarized zone*), também conhecida como rede de perímetro, é uma sub-rede física ou lógica que contém e expõe serviços de fronteira externa de uma organização a uma rede maior e não confiável, normalmente a internet. Quaisquer dispositivos situados nesta área, isto é, entre a rede confiável (geralmente a rede privada local) e a rede não confiável (geralmente a internet), está na zona desmilitarizada.” (Fonte: [https://pt.wikipedia.org/wiki/DMZ_\(computação\)](https://pt.wikipedia.org/wiki/DMZ_(computação)).)

CAPÍTULO 14

Considerando as opções de update

Fazer um update (atualizar) quer dizer trazer novas informações, recursos ou elementos de interface à aplicação. Melhorar a precisão da aplicação também se enquadra nos domínios de um update. Um update pode nem mesmo afetar o código da aplicação de forma significativa, mas pode afetar a segurança de várias maneiras. Por exemplo, você pode fazer um update em um banco de dados de modo a incluir um campo que mostre quem foi o último a alterar um registro, o que pode melhorar a segurança ao possibilitar rastrear a origem de um erro ou de uma infecção que tenha ocorrido no sistema. Mudar prompts para que a informação que você quer de um usuário seja mais clara é uma correção de segurança que não exige necessariamente mudanças no código, especialmente quando esses prompts estão em um arquivo externo. A ideia por trás de um update é melhorar a aplicação de alguma forma para que uma correção no código se torne desnecessária. É importante saber diferenciar entre um upgrade e um update, portanto a primeira parte deste capítulo compara os dois processos e como eles afetam sua aplicação.

Um update nem sempre corrige um problema de segurança, assim como um upgrade às vezes é um exagero. Para usar os recursos da empresa de modo eficiente, você deve saber quando usar um update e quando realmente precisa fazer um upgrade. O processo de upgrade está descrito no capítulo 13, que apresenta detalhes sobre como ele funciona. Este capítulo compara updates e upgrades e ajuda você a entender como o processo de update funciona.

Updates podem se enquadrar em várias categorias. Por exemplo, você poderia optar por fazer um update no pacote de sua linguagem de programação. O novo pacote poderia oferecer recursos melhores de depuração, uma saída de código otimizada e executáveis mais rápidos. Nenhum desses recursos afeta diretamente o código de sua aplicação, porém

afetam seu funcionamento. O simples ato de recompilar os módulos de sua aplicação gera efeitos que todos podem ver.

Em alguns casos você precisa realizar updates de emergência. Convencer a gerência a fazer um update pode ser difícil porque os updates, muitas vezes, não afetam nem um pouco o código da aplicação. É difícil dizer a um gerente que você precisa de tempo, recursos e pessoas para fazer um update que não mudará uma única linha de código da aplicação. Tentar mostrar as vantagens da mudança também pode ser difícil, até você realmente fazer as mudanças e poder quantificar seus efeitos à gerência, de modo a poder demonstrar um retorno para o investimento.

Assim como no caso dos upgrades, é necessário testar seus updates. No entanto, os testes de alguns updates são significativamente diferentes dos testes de upgrade porque, novamente, você não estará lidando necessariamente com código quando trabalhar com um update. Em suma, este capítulo apresenta uma introdução completa para a realização de updates de vários tipos para que você possa avaliar melhor quando deve usar um upgrade e quando deve usar um update.

Diferenciando entre upgrades e updates

Tanto os upgrades quanto os updates afetam a segurança de sua aplicação. No entanto, um upgrade difere significativamente em escopo e em propósito de um update, portanto é importante conhecer a diferença entre eles. A maioria dos upgrades cria novas e significativas funcionalidades e afeta a base de código em um nível mais profundo que um update. De fato, muitos updates nem sequer tocam na base de código. Um update executa tarefas como:

- mudar o banco de dados administrado pela aplicação. Por exemplo, um banco de dados que forneça informações de catálogo pode ter um update em descrições que ajudarão um usuário a tomar decisões melhores.
- modificar recursos existentes para deixá-los mais amigáveis ou menos suscetíveis a gerar erros. Por exemplo, entradas de menu agora poderiam incluir teclas de acesso rápido para que os usuários não precisem adivinhar quais combinações de teclas devem usar.

- reconfigurar recursos da interface de usuário pode deixá-la mais adequada às necessidades do usuário. Um update pode incluir suporte a mais idiomas para que um usuário possa ver a interface em um idioma de preferência, em vez de vê-la em uma língua secundária ou terciária que seja menos clara.
- incluir opções mais específicas na interface de usuário, como mudar uma entrada de caixa de texto para um conjunto de botões de rádio para que o usuário possa escolher opções específicas. Todo elemento removido de uma aplicação, que possa dar margem ao acaso, inerentemente melhora a precisão e a segurança dessa aplicação.

Na maioria dos casos, os updates são mais sutis do ponto de vista do desenvolvedor. É bem provável que o desenvolvedor nem mesmo faça alterações em algumas situações. Por exemplo, você precisa dos serviços de um DBA para fazer um update no banco de dados em vez de contar com o desenvolvedor para isso. Um DBA pode executar a tarefa de forma mais rápida e precisa na maioria dos casos. Designers podem fazer muitas alterações na interface de usuário utilizando técnicas de implementação disponíveis atualmente, fazendo com que seja desnecessário ao desenvolvedor alterar a base de código. No entanto, os desenvolvedores devem se envolver no processo quando chegar o momento de testar o update, para garantir que ele funcione conforme esperado. Às vezes, um update também exige alguns ajustes no código para funcionar como deve.



Mesmo que um desenvolvedor não esteja envolvido em um update de forma significativa, a equipe sempre deve incluir os desenvolvedores na discussão. De fato, os updates, assim como os upgrades, devem ter total suporte de todos os stakeholders da equipe de desenvolvimento. Esse processo inclui usuários que façam testes para verificar se um update realmente funciona conforme necessário e determinar se ele não causará novos erros significativos de usuário nem representará problemas em potencial (por exemplo, deixando o sistema mais lento ao usuário ou provocando um enorme pico nos custos de treinamento).

Um traço que os upgrades e os updates têm em comum é a necessidade de testes. Algumas empresas cometem o erro de supor que os updates exigem testes mais leves ou, possivelmente, nenhum teste. O problema com esse ponto de vista é que colocar um update no servidor de produção

provavelmente revelará problemas que os testes mostrariam antes e que facilitariam a correção. É importante lembrar que os testes não verificam apenas a base de código; eles também verificam se há problemas como a incapacidade de o usuário interagir com a aplicação de forma apropriada em algum nível. Um update poderia abrir uma brecha de segurança possibilitando que um hacker interaja com a aplicação de uma nova maneira (especialmente quando o update envolve uma nova funcionalidade na interface de usuário). No entanto, a ênfase dos testes de update está nas áreas alteradas. Por exemplo, você ainda deve testar todos os aspectos da aplicação, mas se a interface de usuário mudar, o foco dos testes deve estar na interface de usuário e mais verificações devem ser feitas nessa área.



Algumas decisões de update estão fora da área de supervisão direta da equipe de desenvolvimento. Por exemplo, um update de terceiros em uma biblioteca, API ou microsserviço pode ocorrer automaticamente. Nesse caso, você ainda deve testar o update, mas o teste, na verdade, ocorre após o sistema de produção receber o update em muitos casos. Reverter o update também pode ser impossível, portanto aquela mudança causadora de problema poderia fazer uma empresa procurar um novo produto de terceiros ou criar um upgrade para corrigir o problema. Como resultado, é essencial monitorar todos os tipos de update e estar pronto para testá-los a qualquer momento.

Determinando quando o update deve ser feito

Um software de terceiros poderia apresentar um problema quando se trata de updates. Muitos fornecedores terceirizados tentaram esgueirar updates não anunciados pelas equipes de desenvolvimento quando a natureza de uma correção poderia se mostrar embaraçosa ao fornecedor. Updates de segurança às vezes se encaixam nessa categoria – eles simplesmente aparecem num passe de mágica um dia, e ninguém tem realmente certeza de quando o fornecedor terceirizado os disponibilizou. Esses updates podem se transformar em uma bola de neve ao fazer sua aplicação parar de funcionar ou, pior ainda, começar a agir de maneira imprevisível. Um dia você simplesmente descobre que a área de suporte começou a receber muitas ligações de usuários irritados que aparentemente não conseguem fazer a aplicação funcionar.

Os updates que você fizer em seu software ou aqueles feitos para você por um terceiro exigem certo nível de planejamento, mesmo quando o fornecedor terceirizado se esquece de mencionar o update ou por que ele foi feito. Você deve saber que a mudança, na verdade, está nos domínios de um update e não é um upgrade disfarçado. Com isso em mente, as próximas seções discutem alguns dos problemas que devem ser considerados quando você determinar se uma aplicação exige um update.

Trabalhando com updates de biblioteca

Updates de biblioteca podem ser os mais difíceis de fazer porque uma biblioteca se torna parte do código da aplicação. Como ela é importada diretamente na aplicação, algumas mudanças são difíceis de esconder. Além do mais, algumas alterações podem revelar problemas estranhos – bugs que sempre estiveram na biblioteca, mas jamais apareceram porque a aplicação alocava memória ou outros recursos de forma diferente. O update pode nem mesmo aparecer na aplicação após a inclusão da nova biblioteca porque as chamadas podem ter um nome um pouco diferente. Todos esses problemas são comuns a qualquer tipo de update de biblioteca que você queira fazer. As próximas seções dividem as bibliotecas entre aquelas que você incorpora de uma fonte de terceiros e aquelas que você desenvolve por conta própria.

Lidando com updates em bibliotecas de terceiros

Quando trabalhar com um update de terceiros, o primeiro passo é obter uma lista das mudanças – supondo que o fornecedor ofereça uma. Percorra a lista a fim de determinar quais mudanças realmente afetam sua aplicação. Use um teste de unidade para ajudar a determinar se a mudança documentada reflete a mudança propriamente dita na função antes de começar a fazer planos para o update.

Infelizmente, a lista não informará alguns dos problemas, por exemplo, mudanças de nomes, de uma maneira que parece ser uma mudança. Em muitos casos, o fornecedor lista uma mudança de nome como uma nova funcionalidade. Consequentemente, você também deve analisar a lista de novas funcionalidades à procura de recursos implementados como uma mudança em uma função que você já usa. Por exemplo, a nova

funcionalidade pode ter o nome `MySpecialFunction2()`, quando ela substitui `MySpecialFunction()`. A mudança poderia refletir novas funcionalidades, o que significa que você deve analisar o update cuidadosamente para garantir que compreende as ramificações em seu código.

Se você puder obter uma cópia do código, use técnicas de testes estáticos para entender melhor as mudanças que o fornecedor fez na biblioteca. Executar o código em um debugger para vê-lo em ação também pode ajudar. Talvez você não tenha essas opções disponíveis na maior parte das vezes, mas não faz mal considerá-las.

Desenvolvendo updates em bibliotecas internas

Realizar updates em bibliotecas internas permite ter mais controle sobre o processo de update. Você pode executar todos os níveis de testes necessários, pois é o proprietário do código. A questão mais importante é documentar completamente todas as mudanças criadas pelo update para não encontrar problemas de confiabilidade e de segurança mais tarde. Muitos problemas de segurança ocorrem quando um update muda o modo como uma função opera e o código continua a usar a antiga abordagem para trabalhar com a função. Essa incompatibilidade pode gerar brechas de segurança.

Uma situação pior ocorre quando um update aparece como uma função renomeada, mas a falta de documentação faz com que as pessoas que trabalham com a biblioteca continuem usando a função mais antiga, com bugs. O resultado é que a equipe que trabalha com a biblioteca sabe que ela contém uma correção de segurança, mas a equipe que trabalha na aplicação não tem ideia da existência dessa correção. Os hackers analisam essas mudanças de biblioteca quando possível e usam a incompatibilidade para descobrir problemas de segurança. Explorar a incompatibilidade é mais fácil que procurar erros de programação, pois a função renomeada atua como um sinal anunciando a correção ao hacker.

Empregue testes estáticos para garantir que a mudança documentada na realidade corresponda à mudança implementada na biblioteca. Após executar um teste estático, utilize testes de unidade para conferir se a correção funciona conforme anunciado. A seção “Criando um cronograma para testes

de update”, apresenta informações adicionais sobre o processo de testes.



A brecha de segurança que corresponde ao pior caso é aquela em que todos acham que há uma correção, mas ela não está presente, não funciona conforme anunciado ou os desenvolvedores não implementam a correção na aplicação. Sempre verifique se os desenvolvedores realmente implementaram os updates de forma consistente. Mesmo que uma correção não funcione como anunciado, uma implementação consistente facilita bastante a implementação de um update no futuro, pois todos estarão usando a função do mesmo modo.

Trabalhando com updates de API e microserviços

Tanto as APIs quanto os microserviços sofrem do mesmo problema – eles funcionam como caixas-pretas em sua maior parte. É possível introduzir furtivamente um update neles sem que ninguém saiba. Alguns fornecedores, na verdade, já fizeram isso no passado, e sabemos que casos assim ocorrem quando trabalhamos com códigos internos. A questão da caixa-preta pode causar todo tipo de problema quando a API ou o microserviço não funcionam conforme esperado. Rastrear o culpado pode ser difícil. Com isso em mente, é essencial garantir que a equipe de desenvolvimento documente por completo todas as mudanças no funcionamento interno de uma chamada de API ou de microserviço e que as saídas permaneçam iguais para uma dada entrada. As próximas seções discutem as diferenças entre updates de terceiros e internos.



Desenvolvedores de aplicações desenvolvem confiança em uma API ou microserviço com base na garantia de que, para uma dada entrada, a saída permanecerá a mesma. Um update pode deixar as chamadas mais seguras, mais rápidas ou confiáveis, mas não pode mudar a saída. Se algum elemento de uma interface mudar por causa de um update, você deverá criar novas chamadas para acessar esse elemento. Essa abordagem garante que os desenvolvedores precisarão usar as novas chamadas para obter as vantagens oferecidas pelo update.

Lidando com updates em API e microserviços de terceiros

Quando trabalhar com o código de uma API ou de um microserviço de terceiros, você não terá nenhum acesso ao código do update e não poderá realizar nenhum tipo de teste estático. O que você pode fazer é realizar um teste de unidade intensivo para garantir que as considerações a seguir sejam

verdadeiras:

- Cada uma das chamadas continua produzindo as saídas anunciadas para determinadas entradas, com o intervalo originalmente especificado para a chamada.
- A chamada não aceita entradas fora do intervalo originalmente especificado.
- Qualquer teste envolvendo código de exploit (para garantir que as melhorias de segurança realmente funcionam) falha.
- Testes de velocidade mostram que a chamada completa as tarefas com sucesso dentro do intervalo de tempo tolerado para a chamada original ou mais rapidamente.
- Testes de confiabilidade mostram que a chamada não falha quando houver carga.
- Qualquer alteração no código documentada realmente funciona conforme previsto.

Desenvolvendo updates para APIs e microsserviços internos

Sempre realize testes estáticos como primeira medida em updates internos. Durante o processo, você deve verificar se a documentação e as alterações no código são totalmente compatíveis. Um problema significativo com updates é que a equipe de desenvolvimento da API ou do microsserviço acha que o update funciona de uma forma quando, na verdade, ele funciona de outra. Também é uma boa ideia relacionar updates documentados com registros de problemas (trouble tickets) para garantir que as correções tratam os problemas descritos nesses registros. Pode parecer que esse processo de verificação visual consome tempo, mas realizá-lo muitas vezes torna os testes subsequentes muito mais fáceis e reduz significativamente a probabilidade de um problema de segurança esgueirar-se para o código por acidente. A natureza de caixa-preta do código da API ou do microsserviço faz com que a verificação estática seja mais importante ainda se comparada às bibliotecas.

Seu código interno também será beneficiado com os mesmos tipos de verificação realizados em códigos de terceiros (consulte a seção “Lidando com updates em APIs e microsserviços de terceiros”, para ver os detalhes).

No entanto, você pode fazer essas verificações com o código à mão para entender melhor por que determinadas verificações falham.

Um problema particularmente complicado ao trabalhar com updates em microserviços é que muitas empresas atualmente dependem de uma API de microserviços para fazer os vários microserviços se parecerem com uma API personalizada. O uso de uma API de microserviços reduz a carga de programação para os desenvolvedores e garante que toda plataforma tenha os recursos necessários usando os protocolos disponíveis àquela plataforma. A figura 14.1 mostra uma configuração típica de API de microserviços.

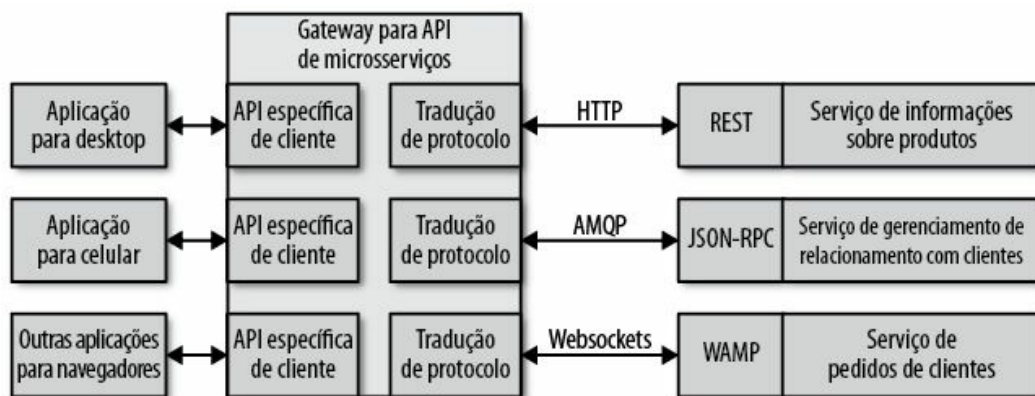


Figura 14.1 – Quando usar uma API de microserviços, teste a API também.

É essencial testar os vários microserviços diretamente para garantir que o update não mudou o modo como o microserviço funcionava originalmente. Entretanto, você deve testar também a API de microserviços para garantir que ela continue respondendo corretamente a todos os tipos de plataforma aceitos pela aplicação. Caso contrário, você não poderá ter certeza de que a API de microserviços continuará a oferecer os resultados corretos de forma segura e confiável.

Aceitando updates automáticos

Alguns fornecedores atualmente dificultam o máximo ignorar updates automáticos. Um update automático é aquele que um fornecedor empurra para um cliente de alguma maneira. Em alguns casos, o fornecedor simplesmente muda o software no servidor, de modo que você acaba usando o update automaticamente. Estes são os principais motivos pelos quais os

fornecedores gostam de updates automáticos:

- Menos problemas de segurança ou de confiabilidade por causa do uso de uma versão desatualizada
- Custos reduzidos de suporte
- Possibilidade de usar equipes menores
- Aumento de ROI

É claro que a pergunta não é se os updates automáticos são bons para o fornecedor, mas se são bons para a sua empresa. Em muitos casos, o uso de updates automáticos se mostra problemático porque a empresa não está preparada para o update, e mudanças desestabilizadoras normalmente causam problemas de suporte. Os casos mais severos resultam em perda de dados ou em falhas de segurança que depõem contra fazer um update, antes de tudo.

A maioria das empresas tenta evitar updates automáticos sempre que possível para reduzir o potencial de efeitos colaterais inesperados. Quando um fornecedor terceirizado insiste em updates automáticos e não há alternativa para um software de terceiros, você realmente precisa de alguém monitorando esse software para determinar quando um update automático surgirá. Testar o update imediatamente pode ajudar a evitar problemas com atualizações que causam falhas e, possivelmente, melhora também a capacidade de sua empresa de reagir a mudanças repentinas.

Fazendo update em pacotes de idiomas

As aplicações web se classificam em uma categoria, por assim dizer, especial, pois usuários que falam vários idiomas poderiam simplesmente encontrá-los enquanto procuram outras informações. Qualquer aplicação pública que você criar, mesmo que vise a apenas um grupo, estará no palco da internet mundial, em que qualquer pessoa em qualquer lugar poderá acessá-la. Teoricamente, você pode tentar bloquear o acesso, mas essa é uma estratégia contraproducente. Se descobrir que sua aplicação está repentinamente recebendo atenção de lugares que você jamais imaginou, talvez você precise de um update na aplicação que trate de questões específicas a idiomas. As próximas seções discutem alguns dos problemas que você deve considerar,

principalmente quando se trata de manter um ambiente seguro para a aplicação.

Criando uma lista de idiomas aceitos

Nem toda aplicação oferece suporte a todos os idiomas do mundo. Tentar fazer isso seria um pesadelo de logística e não é necessário, pois nem todos vão querer realmente usar sua aplicação. Contudo, talvez você descubra que precisa oferecer suporte a mais que apenas uma língua à medida que sua aplicação se tornar mais popular. Softwares de servidores modernos geralmente permitem monitorar os lugares que acessam sua aplicação. A localidade em si pode não informar tudo de que você precisa. Por exemplo, você pode ficar surpreso em saber que a Romênia tem várias línguas oficiais:

- Romeno
- Croata
- Alemão
- Húngaro
- Romani
- Russo
- Sérvio
- Eslovaco
- Ucraniano

No entanto, essas são apenas as línguas oficiais. Os usuários de sua aplicação podem falar uma língua diferente, por exemplo, italiano. Não é possível saber exatamente a quais idiomas você deve dar suporte com base apenas na localidade. A questão é que, se você de repente descobrir que sua aplicação está recebendo muitas solicitações da Romênia, saberá que os usuários provavelmente não falarão inglês de forma nativa (ou que simplesmente não falarão inglês). Se sua aplicação só oferece suporte para inglês, você poderá descobrir que o usuário fica confuso, comete erros e pode provocar problemas tanto de segurança quanto de confiabilidade.

Em alguns casos, você pode fazer suposições sobre suporte a idiomas com base



no ambiente da aplicação. Uma aplicação executando em um ambiente privado nos Estados Unidos, por exemplo, talvez precise oferecer suporte apenas a inglês e espanhol como opções. Quando a aplicação executar na Romênia, você poderá oferecer suporte a todas as línguas oficiais para garantir que todos possam acessar sua aplicação. Considerar o ambiente da aplicação é uma parte importante ao trabalhar com a lista de idiomas tratados.

Usar a localidade é um bom ponto de partida, mas você precisa obter estatísticas de idiomas de alguma outra maneira. Praticamente, o único meio eficiente de garantir que você tenha bons dados é oferecer formulários de feedback ou conduzir pesquisas junto aos usuários. À medida que acumular dados estatísticos sobre o uso da aplicação, você verá surgirem padrões e poderá começar a decidir a quais idiomas deverá dar suporte. Criar uma lista de idiomas aceitos é importante porque você precisa saber quais deles poderá incluir como parte de um update e deverá ter certeza de que cuidará das línguas mais populares antes.



Quando criar um update para suporte a vários idiomas, normalmente você vai querer oferecer URLs diferentes para cada idioma. Do contrário, será praticamente impossível encontrar o idioma de que alguém precisa. Além do mais, usar URLs diferentes permite identificar informações fornecidas pelo navegador e fazer uma seleção razoável do idioma para o usuário. Os dois métodos mais comuns para criar URLs únicos para cada idioma são:

URL específico de idioma

O mesmo domínio hospeda vários idiomas. Um URL como *<http://www.mysite.com/en/stuff.html>* ofereceria suporte a inglês, enquanto *<http://www.mysite.com/de/stuff.html>* daria suporte ao alemão. A vantagem dessa abordagem é que você economiza dinheiro ao não precisar comprar vários TLDs (Top-Level Domains, ou Domínios de Mais Alto Nível). Essa abordagem foca no idioma.

Vários TLDs

Cada idioma tem seu próprio domínio. Um URL como *<http://www.mysite.us/stuff.html>* oferece suporte a usuários nos Estados Unidos, enquanto *<http://www.mysite.ro/stuff.html>* daria suporte a usuários na Romênia. A vantagem dessa abordagem é que as ferramentas de pesquisa, como o Google, oferecerão um link diferente para cada idioma tratado, em vez de tratar todos os idiomas como conteúdo duplicado e exibir apenas um link. Essa abordagem foca no país. Você pode encontrar uma lista de TLDs na Wikipedia (*https://en.wikipedia.org/wiki/List_of_Internet_top-level_domains*).

Contando com especialistas de confiança em idiomas

Se optar por realizar um update específico para idiomas em sua aplicação, você só pode garantir que ele funcionará de forma confiável e segura quando parecer que você o escreveu na língua-alvo. Todos já trabalharam com instruções ou outros comunicados escritos originalmente em outra língua e então traduzidos para qualquer que seja o idioma que você fale. Quando o tradutor não é muito habilidoso, os resultados muitas vezes são hilários. O verdadeiro significado por trás das várias informações se perde e ninguém pode levá-las a sério. Contudo, os problemas de segurança que esse tipo de falha de comunicação provoca não são nada engraçados.

Porém, mesmo um ótimo tradutor não fará um trabalho perfeito se você não oferecer o ambiente correto para efetuar a tradução. Os passos a seguir oferecem esclarecimentos sobre as técnicas que podem ser usadas para criar uma tradução melhor para as aplicações e garantir que os usuários possam seguir suas instruções como resultado:

1. Coloque todas as strings que estejam fixas no código da aplicação em algum tipo de banco de dados. Identifique cada string de acordo com sua localização na aplicação.
2. Substitua cada string fixa no código por um controle de placeholder que você preencherá com a string no idioma correto quando a página carregar.
3. Crie um arquivo separado no banco de dados para cada idioma e garanta que o nome do banco de dados reflita o conteúdo do idioma.
4. Preencha o banco de dados de cada idioma com as strings traduzidas fornecidas pelo tradutor. Use exatamente os mesmos identificadores para cada uma das strings de modo que o identificador sempre corresponda ao prompt na tela.
5. Acrescente código na aplicação para executar a tarefa de preencher os controles com os prompts corretos obtidos do banco de dados.
6. Teste a aplicação inicialmente com o idioma original para garantir que os prompts funcionam conforme desejado.

É sempre uma boa ideia fazer uma segunda verificação nos updates de idiomas, especialmente quando ninguém na empresa fala esse idioma de forma nativa. Há



sempre um potencial para erros na tradução. Além disso, você deve considerar os costumes sociais das pessoas que falam a língua. A questão mais problemática é que algumas línguas não têm uma tradução direta para algumas palavras. Como resultado, você precisará escolher uma aproximação correta e garantir que haja pouco espaço para erros de interpretação nessa tradução aproximada. Fazer uma verificação independente da tradução evitará constrangimentos mais tarde, além de deixar a aplicação mais segura ao reduzir o potencial para erros de entrada.

Conferindo se os prompts específicos de um idioma funcionam na aplicação

Simplesmente traduzir os prompts não é suficiente em muitos casos. Você precisa contar com seu especialista em idiomas para garantir que todo prompt em todas as telas reflitam o modo como as pessoas que falam a língua nativamente esperam ver os prompts. Por exemplo, algumas línguas são lidas da esquerda para a direita, enquanto outras são lidas da direita para a esquerda. A orientação dos prompts é importante, assim como a formatação. Talvez você precise oferecer dicas e uma formatação diferente na aplicação para garantir que as pessoas que a estão usando possam realmente interpretar os prompts corretamente.

Garantindo que os dados apareçam no formato correto

Diferentes regiões do mundo formatam dados de maneiras distintas. Alguns países usam vírgula no lugar de ponto para a casa decimal. Esses mesmos países podem usar um ponto no lugar da vírgula para o indicador de milhar. Pense no que aconteceria se um usuário mudasse valores com base em uma formatação incorreta nessa situação. Um usuário poderia inserir valores que fariam a aplicação se comportar indevidamente, falhar ou simplesmente fazer algo estranho. O resultado poderia ser uma brecha de segurança ou uma abertura que um hacker poderia usar para provocar uma falha de segurança. Muitos problemas de segurança resultam de erros aparentemente tolos que um hacker explora para obter uma pequena vantagem que levará a uma recompensa valiosa.

A apresentação dos dados no formato correto garante que não haja erros de compreensão. Criar um problema de segurança em potencial simplesmente por causa de uma confusão com vírgulas e pontos pode parecer um problema que pode ser facilmente solucionado, mas, às vezes, ele se mostra absurdamente difícil. Você pode encontrar exemplos de indicadores de milhar e de ponto decimal de acordo com o idioma em <http://docs.oracle.com/cd/E19455-01/806-0169/overview-9/index.html>. A imagem gráfica em <http://www.statisticalconsultants.co.nz/blog/how-the-worldseparates-its-decimals.html> também é interessante.

Definindo os requisitos especiais para testes de suporte a idiomas

Testar sua aplicação torna-se muito mais difícil quando você começa a acrescentar vários idiomas. A menos que você planeje manter aqueles especialistas em idiomas presentes durante todo o tempo de vida de sua aplicação, será necessário ter outros meios de verificar se os updates não estão causando problemas de suporte a idiomas. Planejar de forma apropriada, alterar a documentação e fazer verificações na língua original são seus melhores pontos de partida. Sempre que fizer uma alteração na aplicação que afete um prompt na língua original, você deve considerar mudar os prompts em todos os demais idiomas tratados também. Para facilitar o máximo esse processo, utilize as seguintes sugestões:

Planejar

Não se esqueça de conferir cuidadosamente todas as alterações. Às vezes, um designer tentará ajustar os prompts quando eles realmente não precisam de nenhuma mudança. Fazer o mínimo de mudanças reduzirá os custos, garantirá que a aplicação permaneça confiável e ajudará a melhorar a segurança, reduzindo o potencial para interpretações errôneas.

Documentar

Toda mudança em prompts na língua original deve aparecer também em todas as outras línguas que sua aplicação tratar. Um problema de segurança e de confiabilidade nasce quando as versões para cada idioma saem de

sincronismo. É essencial que essa parte do processo de update execute suavemente, sem erros, ou você se verá contratando aquele especialista em idiomas novamente para acabar com a confusão.

Verificar

Garantir que toda mudança ocorra em todos os lugares é difícil porque você provavelmente não terá funcionários em sua equipe que falem o idioma ao qual você dará suporte. Uma boa maneira de conferir o update de idioma é usar capturas de tela. Compare a tela original com a tela atualizada para verificar se a mudança de prompt realmente ocorreu. O único problema com essa abordagem é que você não poderá ter certeza de que a mudança de prompt está totalmente correta.



Por causa do modo como a aplicação utiliza prompts, não será necessário testar o código subjacente para cada idioma. O que você muda para cada idioma é a interface e é aí que deverá estar o foco de seus testes para problemas específicos de idiomas. Garantir que você terá prompts adequados e que eles aparecem nos lugares corretos é a base para os testes específicos de idiomas.

Efetuando updates de emergência

Os upgrades são sempre bem planejados e cuidadosamente agendados. A maioria dos updates também recebe um planejamento e um cronograma apropriados. No entanto, quando um hacker acabou de expor alguma brecha terrível em sua aplicação, sobre a qual você precise agir imediatamente, às vezes um update não pode esperar – você deve criar uma correção provisória e implantá-la imediatamente, pois, do contrário, sofrerá as consequências. Ninguém gosta de emergências. Quanto mais crítica for a aplicação e quanto mais sensíveis forem os dados afetados, menos desejável será uma emergência. As seções a seguir ajudam no caso das emergências que surgem no meio da noite, espreitando e esperando que você baixe a guarda.

Evitando as emergências sempre que for possível

Todos passam por uma emergência em algum momento no processo de suporte à aplicação. Simplesmente faz parte do ofício. No entanto, você pode evitar a maioria das emergências se fizer planejamentos antes que elas

aconteçam. Eis alguns problemas que você deve considerar como parte do trabalho de evitar emergências:

- Fazer testes completos em cada update e upgrade antes da implantação.
- Implantar updates e upgrades somente quando a carga e a disponibilidade do sistema suportá-los.
- Evitar implantar updates e upgrades quando houver poucos funcionários na equipe ou quando pessoas-chave estiverem ausentes.
- Garantir que a aplicação tenha acesso a todos os recursos necessários de várias fontes, se for possível.
- Planejar-se para situações de sobrecarga e garantir que você tenha capacidade adicional disponível para atender a requisitos de picos.
- Supor que um hacker atacará no pior momento possível e planejar-se para o ataque (o máximo possível) com antecedência.
- Testar a segurança e a confiabilidade do sistema regularmente.

Criando uma equipe para respostas rápidas

Sua empresa precisa de uma equipe para respostas rápidas – um grupo de indivíduos que esteja de plantão para lidar com emergências. Não precisa ser o mesmo grupo de indivíduos o tempo todo – todos precisam descansar – mas você deve ter uma equipe para responder a emergências, disponível o tempo todo, especialmente naqueles momentos em que os hackers adoram atacar (por exemplo, nos fins de semana). Uma equipe de resposta a emergências deve incluir todos os indivíduos que uma aplicação exija para qualquer tipo de update. Se você precisa de um DBA para fazer um update normalmente, então será necessário ter um DBA em sua equipe de resposta a emergências. A equipe deve fazer treinamentos em conjunto especificamente para tratar emergências e garantir que esteja pronta para agir quando a emergência ocorrer.

Jamais faça um upgrade como medida de emergência. Upgrades afetam profundamente o código e você não poderá testá-lo apropriadamente com o tempo curto de resposta exigido por uma emergência. Quando disponibilizar um upgrade sem testes apropriados, é provável que você inclua alguns bugs interessantes que um hacker adoraria explorar. Sempre trabalhe de forma segura



e mantenha as medidas de emergência limitadas àqueles itens que você pode tratar como um update.

Executando testes simplificados

Conforme a natureza da emergência, talvez você precise disponibilizar o update com um mínimo de testes. Você deve ao menos testar o código afetado diretamente pelo update e, possivelmente, o código em torno dele também. Quando um hacker estiver derrubando sua porta, pode ser que você não tenha tempo de testar todas as partes do código como normalmente seria necessário. É importante lembrar que o update que você não disponibilizou poderia ter mantido o hacker afastado. O problema se reduz a um equilíbrio nos riscos – a emergência de um lado e o potencial para criar um problema consideravelmente pior de outro.

Criando um cronograma permanente de updates

Sempre trate updates de emergência como temporários. Você não tem como saber com que pressa a equipe de respostas rápidas trabalhou enquanto estava tentando disponibilizar o update. A equipe poderia ter cometido todo tipo de erros durante o processo. Mesmo que o update funcione, você não poderá deixar o update de emergência instalado, sem analisá-lo melhor. Deixar o update de emergência instalado abre brechas de segurança e de confiabilidade que talvez você não saiba que estão presentes e poderiam causar problemas consideráveis para você no futuro.



É uma boa ideia realizar uma análise da emergência após o fato, determinar o que a causou e considerar a rapidez com que a equipe de emergência respondeu a ela. Ao efetuar uma análise, você poderá criar uma abordagem baseada em lições aprendidas (lessons learned) para lidar com emergências no futuro. Você pode apostar que os hackers constantemente trabalham para aperfeiçoar suas habilidades, portanto você deve melhorar as suas também. Reservar tempo para compreender a emergência e procurar formas de lidar melhor com ela no futuro são maneiras valiosas de criar uma equipe melhor de resposta rápida.

Como parte do processo permanente de updates, lembre-se de realizar testes estáticos do update para conferir se a equipe de resposta rápida reagiu de forma apropriada e ofereceu uma boa correção para o problema. Lembre-se ainda de colocar o update permanente no cronograma o mais rápido possível para garantir que corrigirá qualquer problema adicional do qual alguém possa tirar proveito. O novo update deve contar com todos os procedimentos usuais de teste e seguir qualquer que seja o plano que você tenha adotado para updates permanentes.

Criando um cronograma para testes de update

Independentemente de como você cria o update ou do período de tempo envolvido, não vale a pena apressar um update para que ele seja implantado sem que haja tempo para alguns testes. Apressar um update para lançá-lo quase sempre causa problemas no final. De fato, os problemas criados pela pressa normalmente são piores que não disponibilizar o update. Os testes permitem uma verificação de sanidade que força você a reduzir a velocidade e verificar novamente seu processo de raciocínio para criar o update. Se usado corretamente, os testes garantem que seus companheiros de equipe estejam tão certos quanto você de que o update é robusto e que não causará problemas que acabarão se transformando em um pesadelo de segurança.

Os testes de update não seguem exatamente o mesmo padrão que os testes de upgrade, pois há menos mudanças na aplicação em um update e você pode focar a maior parte de sua energia nas funcionalidades que sofreram alteração. Além do mais, você pode realizar updates durante uma emergência e precisar lançar o update o mais rápido possível para evitar uma brecha de segurança importante. Com isso em mente, você deve executar os seguintes níveis de testes para garantir que o update não cause mais prejuízos que benefícios:

- Testes de unidade
- Teste de integração
- Testes de segurança

Quando o update serve para cuidar de uma emergência, esses três níveis

normalmente evitarão que ele cause mais danos que benefícios. No entanto, você deve executar os níveis adicionais de testes a seguir o mais rápido possível:

- Testes de usabilidade
- Testes de compatibilidade
- Testes de acessibilidade
- Testes de desempenho

Depois que a poeira baixar e houver tempo adicional para testar, você deve garantir que o update seja totalmente funcional. A melhor maneira de garantir a funcionalidade é realizar os seguintes níveis adicionais de testes:

- Testes de regressão
- Testes de internacionalização e localização
- Testes de conformidade



Você poderia realizar outros tipos de testes, mas somente se for necessário. Por exemplo, aplicações web raramente precisam de qualquer tipo de instalação, portanto não será necessário realizar nenhum teste de instalação na maioria dos casos. A exceção é quando uma aplicação web exige a instalação de uma extensão de navegador, um add-on ou um serviço do navegador, um agente de sistema ou outro tipo de software necessário.

CAPÍTULO 15

Considerando a necessidade de relatórios

Relatórios ajudam a documentar o status de sua aplicação. Pode parecer que uma nota no arquivo de código ou uma simples discussão no corredor seriam suficientes para iniciar uma tarefa, mas um processo formal de aceitação de informações de entrada sobre as aplicações realmente é a única maneira de garantir que todos compreendam o status da aplicação. Muitas pessoas associam relatórios desse tipo com problemas negativos, por exemplo, bugs. No entanto, você pode usar relatórios para todo tipo de necessidade. Por exemplo, um relatório positivo pode indicar que os usuários realmente gostam de uma nova funcionalidade, com base em estatísticas de uso. Um relatório como esse é como um tapinha nas costas de todos, sem se preocupar em quebrar a mão de alguém para fazer isso.

Em geral, você cria relatórios internos para a aplicação como um todo, mas pode usar relatórios para monitorar funcionalidades individuais da aplicação também. Você pode nem mesmo focar diretamente na aplicação, se for o caso, e monitorar o uso de banda de rede ou os requisitos de acesso a dados. A questão é que relatórios internos geralmente focam em seu código, nos usuários e nos recursos.

Os relatórios externos provêm de terceiros. Por exemplo, se contratar uma empresa terceirizada de testes (veja o capítulo 12), você deve esperar ver vários relatórios que detalhem o status de sua aplicação durante os testes. Um relatório de terceiros também pode listar os serviços que sua aplicação realmente usa quando tem acesso a uma biblioteca, API ou microsserviço. Ao monitorar o uso desses serviços, você poderá otimizar a aplicação e conseguir uma taxa reduzida no preço ao adquirir menos serviços.

Uma forma importante de relatório envolve procurar ativamente feedback de usuários (ou até mesmo designar representantes de usuário específicos para garantir que cada grupo de usuários tenha oportunidade de expor seus desejos). Você jamais satisfará a todos os usuários de sua aplicação. No entanto, poderá tentar atender às necessidades da maioria deles de forma razoável. O truque é garantir que os relatórios obtidos dos usuários realmente representem a visão da maioria, e não as angústias expressas por uma minoria. Manter os usuários satisfeitos, eficientes e bem controlados é sempre uma boa meta a ser atendida no desenvolvimento da aplicação.



Relatórios podem beneficiar sua empresa de todas as maneiras usuais. Por exemplo, você pode analisá-los à procura de usos incomuns ou de padrões de acesso. Padrões incomuns podem indicar atividade de hackers, mas também poderiam indicar a necessidade de um upgrade (veja o capítulo 13) ou de um update (veja o capítulo 14) na aplicação. A questão é ter a mente aberta para o potencial de usar relatórios de forma não padrão quando as oportunidades se apresentarem.

Usando relatórios para fazer alterações

Antes de poder fazer algo com um relatório, você precisa ter um. Alguns desenvolvedores acreditam que há um conjunto-padrão de relatórios que atenderá a todas as possíveis necessidades organizacionais. Infelizmente, não é isso o que acontece. Os relatórios variam porque as necessidades variam. Você deve considerar exatamente como criar os tipos de relatórios necessários e evitar relatórios inúteis, que simplesmente consomem o tempo de todos. Às vezes, você precisa de relatórios para acompanhar um upgrade ou um update e garantir que essas tarefas ocorram conforme planejado. É possível gerar relatórios manualmente, a partir de dados acessados facilmente, ou gerá-los de forma automática. Relatórios automáticos podem vir de várias fontes, por exemplo, de arquivos de log. O importante é garantir que os relatórios sejam consistentes para que as pessoas que os leiam possam contar com eles como fonte consistente de informações. As próximas seções descrevem como criar relatórios que realmente farão algo útil para sua empresa, facilitando monitorar problemas como preocupações com segurança durante o processo.

Evitando relatórios inúteis

As empresas, e até mesmo os indivíduos, geram uma variedade de relatórios no decorrer de sua vida, e apenas alguns deles são realmente úteis. Um relatório é uma espécie de ferramenta, portanto você deve precisar dele para que ele tenha uma função útil. Algumas pessoas veem os relatórios como informativos, mas esse não é o caso. Se você não puder atuar no conteúdo do relatório, a informação será inútil e você não deverá gastar tempo lendo-o. De fato, informações inúteis colaboram significativamente com o fenômeno de sobrecarga de informações.

Os relatórios que contêm informações úteis ajudam a executar tarefas específicas. Eles oferecem a base sobre a qual podemos agir. Com isso em mente, os relatórios que ajudam a melhorar a segurança de sua aplicação normalmente contêm tópicos como:

- bugs;
- velocidade;
- confiabilidade;
- uso;
- acesso a dados.

O tipo de relatório determina como você agirá sobre a informação que ele contém. Quando perceber que você não executa nenhuma ação em resposta a um relatório, ele terá falhas e exigirá mudanças, ou você realmente não precisa dele. As informações nos relatórios geralmente inspiram algum tipo de ação pelos seguintes motivos:

Mudança de padrão

Dados são constituídos de padrões. Você pode ver um fluxo no modo como os dados se apresentam. Na verdade, ramos inteiros da matemática hoje em dia são dedicados ao estudo dos padrões de dados. Quando você vê uma mudança em um padrão de dados, mesmo que essa mudança não aparente ser um problema, é preciso investigar sua origem. Alguns padrões apresentam problemas de confiabilidade e outros problemas de segurança. É claro que uma mudança de padrão poderia simplesmente resultar de uma

mudança no uso ou de outro fator, mas você precisa saber o porquê de o padrão ter se alterado.

Mudança de status

Uma mudança de status pode ter vários significados. No entanto, indicadores de status informam se o fluxo atual de informações mudou de alguma maneira, o que significa que você deve verificar se há possíveis problemas de interface, confiabilidade, segurança ou de recursos. Problemas de interface podem resultar em problemas de segurança mais adiante, e problemas com recursos normalmente acabam provocando problemas de confiabilidade.

Evento anômalo

A natureza constantemente tem potencial para gerar um evento anômalo. Por exemplo, a falha causada por raios pode se apresentar como um evento de informação de anomalia. No entanto, eventos anômalos também ocorrem quando hackers sondam um site à procura de possíveis pontos fracos. Alguns hacks podem se manifestar após a ocorrência de poucos eventos, por exemplo, três, portanto vale a pena rastrear a origem de cada evento.

Valores inesperados

Quando um relatório contém informações fora do intervalo esperado, você deve considerar o motivo pelo qual a aplicação gerou essas informações. O valor inesperado poderia ser proveniente de um bug ou ser o resultado de sondagens de um hacker. Talvez seja o resultado de limitações de recursos no servidor ou de dados ruins no banco de dados. O fato é que valores inesperados ocorrem regularmente, e as pessoas simplesmente os menosprezam, em vez de atuar sobre eles. Sempre verifique o porquê de valores inesperados, quando puder.

Tirar de sua mesa as informações sobre as quais você não pode atuar é uma parte essencial para manter uma aplicação segura. Ao limpar a bagunça, você pode começar a focar nas informações que realmente são necessárias para localizar e verificar problemas de segurança em sua empresa. O segredo é se perguntar como planeja atuar na informação contida em um relatório antes de criá-lo. Além do mais, livrar-se de relatórios antigos, que não servem para

mais nada, é essencial para futuramente ter controle sobre o fluxo de informações em qualquer empresa.

Controlando o instante e a frequência de relatórios de upgrades e updates

Do ponto de vista de segurança e de confiabilidade, você precisa de boas informações para atuar sobre mudanças na aplicação que simplesmente não funcionaram. Para obter as informações necessárias, é importante agendar os relatórios para aqueles momentos em que você faz upgrades e updates na aplicação. Esse agendamento pode ser tão simples quanto mudar o horário em que o sistema gera o relatório ou modificar sua frequência para obter as informações mais vezes.

O fluxo de informações deve aumentar proporcionalmente à significância das alterações que você fizer. No entanto, conforme mencionamos na seção “Evitando relatórios inúteis”, você também deve evitar a sobrecarga de informações. Quando você fica sobrecarregado, há uma tendência a não perceber padrões e eventos críticos apresentados pelos dados. É importante manter um equilíbrio entre as informações que você realmente pode usar e aquelas realmente necessárias. Quando começar a perceber que os relatórios se acumulam por dias sem que nenhuma ação seja tomada, você saberá que está se sobrecarregando de informações e deve reduzir seu fluxo de alguma maneira.



Muitas empresas não dão muita ênfase a softwares de filtragem. Se você puder filtrar as informações recebidas para cada indivíduo a fim de deixá-las mais focadas, será possível reduzir o nível de sobrecarga que a pessoa sofrerá e aumentar as chances de localizar problemas de confiabilidade e de segurança em um upgrade ou update de aplicação de modo muito mais rápido. O problema está em ajustar o software de filtragem para que ele realmente foque a atenção e não deixe informações críticas de fora. Para ajustar o filtro, alguém deve comparar os dados brutos com os dados filtrados e garantir que o filtro esteja funcionando corretamente. Depois que o filtro estiver ajustado, você poderá depender um pouco mais dele para reduzir a carga de trabalho dos indivíduos e deixá-los mais eficientes.

Aumentos no fluxo de informações por causa de upgrades e de updates são temporários. Você não precisa continuar a receber informações adicionais

depois de um período de tempo razoável. É claro que o problema é determinar o momento de poder reduzir o fluxo de informações. Normalmente, você pode começar a reduzir esse fluxo quando a frequência de bugs e de eventos incomuns diminuírem e os padrões de uso da aplicação voltarem ao normal. Ao usar essa abordagem, você poderá desviar parte de sua atenção a outros assuntos, como planejar o próximo update ou upgrade.

Usando relatórios gerados automaticamente

Softwares de monitoração e outras ferramentas de apoio em geral oferecem alguns relatórios. O fornecedor que criou o software normalmente desenvolve esses relatórios em resposta a solicitações do usuário, portanto os relatórios refletem exatamente os dados de que você precisa para fazer uma monitoração geral na aplicação. É importante analisar a lista de relatórios automaticamente gerados e selecionar apenas aqueles que atendam às suas necessidades. Selecionar todos os relatórios e esperar encontrar as informações de que você precisa é uma péssima ideia, pois resulta em sobrecarga de informações e na perda de oportunidades para corrigir problemas na aplicação.



O segredo nesse caso é que os relatórios gerados automaticamente oferecem informações gerais e genéricas – do tipo que todos os clientes do fornecedor podem usar para monitorar suas aplicações. O fornecedor não tem como saber que tipo de aplicação você executa, quando os usuários interagem com as aplicações, de quais dispositivos os usuários dependem ou qualquer fator ambiental que afete a execução da aplicação. Em suma, os relatórios gerados automaticamente atuam como ponto de partida e um possível catalisador para suas próprias ideias sobre as informações necessárias para proteger as aplicações e os dados que elas administram de maneira apropriada.

Os relatórios gerados oferecerão informações genéricas, porém, às vezes, você pode deixá-las mais específicas às suas necessidades selecionando opções de relatório (quando estiverem disponíveis). Em muitos casos, o fornecedor oferecerá os meios para personalizar um relatório para que ele forneça apenas as informações de que você realmente precisa, em vez de mostrar muitas informações desnecessárias. O importante é gerar o relatório uma ou duas vezes com todos os recursos habilitados para que você possa ver o que está disponível e determinar como você pode atuar sobre ele. Depois

que tiver essas informações, você poderá começar a personalizar o relatório a fim de deixá-lo menor e mais eficiente.

Usando relatórios personalizados

Os relatórios genéricos oferecidos pelo fornecedor juntamente com o produto são uma ótima maneira de começar a monitorar sua aplicação em busca de problemas, mas normalmente eles não oferecem informações suficientes sobre suas necessidades específicas. É claro que o fornecedor não sabe nada sobre suas necessidades específicas. A única maneira de obter informações diretamente relacionadas ao modo como sua aplicação em particular funciona em seu ambiente e com seus usuários é criar um relatório personalizado. Quando o software de monitoração que você usar não oferecer o recurso de criação de relatórios personalizados de que você precisa, é hora de escrever uma aplicação por meio da qual obterá as informações e as formatará de maneira que funcione. Esta seção descreve o tipo de relatório que você cria de forma totalmente manual (consulte a seção “Usando relatórios gerados automaticamente”, para ver uma discussão sobre relatórios personalizados).

É claro que algumas empresas quase enlouquecem com relatórios e têm tantos que ninguém realmente sabe para que eles servem. Ser restritivo quando se trata de relatórios personalizados é essencial. Caso contrário, você se verá mergulhado em informações que não quer, não precisa ou não entende, além de ficar com uma responsabilidade extra para a manutenção do software de geração de relatórios. As próximas seções descrevem métodos que podem ser usados para fazer com que seus relatórios personalizados reflitam melhor suas necessidades e gerem apenas as informações de que você precisa para manter sua aplicação segura.

Desenvolvendo relatórios manualmente

Em alguns casos, você desenvolverá relatórios manualmente, extraindo dados de logs que a aplicação cria e de qualquer outra fonte de dados personalizada que você possa ter. As fontes de dados variam bastante. Por exemplo, talvez você precise contar parcialmente com formulários de feedback de usuários para criar o relatório desejado. É essencial planejar com cuidado, pois os relatórios personalizados consomem tempo para serem criados e são custosos

para manter. Eles também podem ser frágeis quando dependem de dados de fontes de terceiros, pois o terceiro pode mudar os dados fornecidos como parte de seu produto.

Obter dados necessários para um relatório tem sérias desvantagens, especialmente quando trabalhamos com uma aplicação. Os dados não são obtidos gratuitamente – você sempre pagará por eles de alguma forma. Eis algumas questões que devem ser consideradas ao obter dados personalizados de qualquer fonte:

- Gerar dados de log exige acréscimo de código à aplicação ou ao produto, o que significa que você perderá velocidade em troca da obtenção de informações.
- Hackers podem usar dados de log para determinar melhor como sua aplicação executa, portanto você poderá dar às pessoas que você mais quer manter afastadas de seu sistema as chaves para se infiltrarem melhor nele.
- Criar logs e outras formas de informações sobre a aplicação ou o produto utiliza recursos que podem ser necessários para trabalhar com a aplicação. Como resultado, a aplicação poderá ter problemas de confiabilidade.
- O código para criação de dados poderia abrir uma potencial brecha de segurança que, do contrário, talvez não existisse – oferecendo aos hackers um método que eles não deveriam ter para acessar seu sistema.
- É muito mais fácil criar dados confusos ou incorretos que gerar exatamente o que você precisa, portanto é preciso tempo para testar a saída de dados e conferir sua exatidão.

Relatórios personalizados servem a um propósito. Porém, você deve ter cuidado ao criá-los. Crie apenas os relatórios necessários, que apresentem somente as informações sobre as quais você deve atuar, e mantenha o relatório ativo apenas pelo tempo absolutamente necessário. Seguir essa estratégia o ajudará a manter sua rede mais segura e reduzirá as penalidades por criar um relatório personalizado.

Desenvolvendo relatórios a partir de fontes de dados prontamente disponíveis

Às vezes, você pode criar um relatório personalizado a partir de fontes de dados prontamente disponíveis. Alguns produtos para monitoração oferecem APIs que permitem acessar os dados criados por eles. Se o produto para monitoração vai criar os dados independentemente de você usá-los ou não, sempre vale a pena usar os dados do monitor (ou de outra aplicação) em vez de criar dados próprios. Mesmo assim, você ainda precisa criar o código personalizado, garantir que ele apresentará os dados necessários no formato que você quer e então dar manutenção no código. Eis alguns dos outros problemas que devem ser considerados quando um relatório personalizado for criado com base em dados gerados por um terceiro:

- O terceiro poderia mudar a saída, o que exigiria uma mudança em seu relatório.
- Os dados poderiam se tornar indisponíveis.
- É mais difícil conferir se os dados são precisos, portanto você não pode ter tanta certeza de que o relatório está totalmente correto – ele poderia fornecer dados anômalos em alguns casos.
- Talvez você precise conseguir licenças ou vencer outros obstáculos para acessar fluxos de dados proprietários por meio de uma API.
- Os dados podem não estar no formato de que você precisa, portanto o código personalizado deverá manipular os dados para limpá-los e formatá-los antes que seja possível usá-los.
- Problemas de segurança conhecidos, relacionados ao modo como um terceiro administra os dados, poderiam dar aos hackers um acesso ao seu sistema.

Criando relatórios consistentes

Um problema que é difícil de resolver – mas é importante contemplar – é a consistência. Quando seus relatórios formatarem dados de várias maneiras diferentes e os apresentar em ordens distintas, pode haver confusão. Qualquer pessoa que estiver usando o relatório poderá cometer erros quando der uma olhada nele e poderá acabar supondo que os dados tenham um significado, quando têm outro. Os relatórios que você usar e criar devem considerar os

seguintes problemas de consistência:

- Os dados devem aparecer no mesmo formato.
- A ordem dos dados deve ser a mesma sempre que for possível.
- O relatório não deve referenciar dados usando várias abordagens conflitantes.
- Qualquer limpeza e filtragem de dados deve funcionar de forma consistente para gerar as mesmas informações em todos os relatórios (devem ser baseadas no uso do mesmo algoritmo).
- Dados apresentados graficamente devem espelhar os dados textuais.

Quando atualizar relatórios, lembre-se de consultar os usuários para ver se há problemas de consistência. As pessoas que dependem de informações para executar tarefas no mundo real provavelmente perceberão discrepâncias mais rapidamente que alguém que simplesmente administra, mantém ou gera o relatório. Alguns dados podem ter várias interpretações, dificultando decifrar possíveis problemas com sua aparência e layout.

Usando relatórios para executar tarefas específicas da aplicação

Como mencionamos várias vezes neste capítulo, um relatório é útil somente quando oferece uma saída sobre a qual é possível atuar. No entanto, alguns relatórios podem não visar a leitores humanos. É possível criar relatórios que melhoram a automação da aplicação. Aplicações de monitoração e de gerenciamento podem ler arquivos de log e usar os dados aí contidos para determinar quando devem realizar tarefas de gerenciamento da aplicação. Você não vai querer que as ações de manutenção ocorram o tempo todo ou com um cronograma específico – é mais importante realizar a manutenção somente quando for necessário para reduzir o uso de recursos e melhorar a disponibilidade da aplicação. Como outros softwares leem e atuam sobre as informações nesses relatórios automaticamente, você deve criá-los com um olho na consistência e outro na precisão. A aplicação não se preocupa com a beleza do relatório, mas, para ela, é importante que os dados contidos reflitam o seu verdadeiro estado.



A automação é uma parte excepcionalmente importante do TI moderno porque ajuda a reduzir custos e a garantir que as ações aconteçam de forma confiável. No entanto, hackers também adoram automação porque as pessoas tendem a se esquecer de que ela existe e o hacker muitas vezes pode usá-la como meio para conseguir acesso ao sistema. Mesmo que a automação usada seja totalmente precisa e confiável, você ainda deve monitorá-la para garantir que permaneça segura e continue a funcionar conforme desejado originalmente.

Criando relatórios internos

Relatórios internos são aqueles que você cria e usa dentro de sua própria empresa. Mesmo que a tendência seja estarem focados na aplicação como um todo, é possível que esses relatórios se concentrem em qualquer fonte de dados que você precise monitorar. Um relatório interno pode até mesmo focar nos usos de bibliotecas, APIs e microsserviços de terceiros. Os relatórios devem ajudar a executar algumas tarefas úteis, como monitorar o potencial para brechas de segurança. O ponto principal de qualquer relatório interno é fornecer dados para mudanças na aplicação ou determinar quando uma aplicação está bem e não precisa de qualquer mudança. As próximas seções ajudam a entender melhor o uso de relatórios internos.

Determinando quais fontes de dados serão usadas

Relatórios internos tendem a funcionar com dados específicos da empresa. Esses relatórios refletem a natureza exata das aplicações executando em seus sistemas e nos ambientes que você oferece. Você pode ver os padrões de utilização de seus usuários e eventos anômalos específicos em seu sistema. Em outras palavras, os relatórios internos tendem a focar em dados que um fornecedor não é capaz de oferecer como parte de um relatório genérico.

O problema com os relatórios internos é que todos parecem supor que as outras pessoas são confiáveis e que os dados incluídos no relatório jamais sairão da empresa. É importante perceber que muitas violações de dados que ocorrem atualmente se devem a um funcionário de confiança que deixou seu notebook no aeroporto ou a alguém com menos escrúpulos que vendeu os dados pelo melhor preço. Com isso em mente, você deve considerar que qualquer dado disponibilizado como parte de um relatório em algum

momento verá a luz do dia no mundo lá fora, quer você queira ou não. A lista a seguir apresenta alguns problemas que devem ser considerados quando você usar fontes internas de dados em um relatório:

- Certifique-se de que as pessoas que usam o relatório realmente precisam dos dados que você está incluindo.
- Defina protocolos, inclusive com penalidades, para trabalhar com dados sensíveis.
- Crie um plano para lidar com violações que envolvam dados sensíveis (procedimentos que você pretende executar além dos protocolos normais para violações em dados).
- Monitore os usuários de dados sensíveis para que seja mais fácil determinar qual é a origem de uma violação de dados.
- Proteja os dados sensíveis e mantenha-os separados dos dados menos sensíveis.



Às vezes, os dados sensíveis acabam se tornando públicos por causa de uma série de erros conectados. A razão para manter os dados sensíveis separados é diminuir as chances de que eles apareçam em um lugar público, por exemplo, em um relatório trimestral ou em uma apresentação pública. Disponibilizar dados sensíveis acidentalmente não é apenas constrangedor, mas pode provocar todo tipo de problemas de publicidade, sem mencionar que oferecerá lenha na fogueira para os hackers invadirem seu sistema.

Proteger seus dados é essencial nos dias de hoje. Muitas empresas aparecem na primeira página da imprensa especializada ou nas principais mídias de notícias após cometerem um erro aparentemente pequeno que acaba dando acesso aos hackers a informações sensíveis. Em muitos casos, a falha está no uso incorreto de uma fonte interna de dados que uma empresa deveria proteger diligentemente, disponibilizando-a a pessoas que realmente não precisam dela. A melhor pergunta que você pode fazer a si mesmo quando criar um novo relatório é se as pessoas que verão esse relatório realmente precisam ver os dados que você está disponibilizando.

Especificando usos para os relatórios

Relatórios internos podem usar e usam dados sensíveis. Se você esconder

todas as informações possivelmente sensíveis, a empresa jamais poderia funcionar de maneira apropriada. Há hora e lugar para disponibilizar os dados sensíveis de uma empresa. No entanto, você ainda deve reduzir os riscos de segurança associados aos dados sensíveis, garantindo que definirá quais são os usos para os relatórios que você gerar e descrever como os usuários devem interagir com eles. Uma empresa exige um conjunto de regras que possa ser imposto para lidar com dados sensíveis contidos em relatórios.

É importante considerar exatamente como os relatórios usam os dados. Técnicas de extrapolação e ciência de dados (data science) podem deixar os dados sensíveis mais prejudiciais ainda quando são disponibilizados no lugar errado. Novamente, você pode usar técnicas de limpeza e de filtragem de dados para deixá-los menos sensíveis e, possivelmente, aceitáveis para uso em apresentações para a empresa como um todo. Por exemplo, dados associados a indivíduos são mais sensíveis que dados não associados a eles. Técnicas de limpeza permitem reter o valor estatístico dos dados sem associá-los a indivíduos específicos.

Os relatórios também devem ter uma documentação associada a eles. Você deve saber quem criou o relatório e por quê. O relatório deve ter uma data de revisão para garantir que continua a ter um propósito pertinente para a empresa. Como parte da documentação, você deve definir alguns elementos para a segurança do relatório:

- Detalhar quem pode acessar e ler o relatório.
- Especificar quem pode gerar o relatório.
- Definir como os usuários podem ver o relatório e onde (alguns relatórios podem não ser adequados para download em dispositivos móveis, por exemplo).
- Criar um processo para garantir que todas as cópias do relatório sejam destruídas em algum momento (exceto uma cópia para arquivamento mantida em um local seguro).
- Fornecer informações de contato para que qualquer pessoa que tenha dúvidas saiba a quem pode fazer perguntas sobre o relatório e seu conteúdo.



O fenômeno BYOD (Bring Your Own Device, ou Traga seu Próprio Dispositivo) está criando problemas de segurança para muitas empresas quando se trata de relatórios contendo dados sensíveis. É improvável que você consiga evitar que os usuários tentem ver dados sensíveis em seus dispositivos móveis sem algumas medidas de segurança significativas implantadas. Você deve pressupor que alguns usuários retirarão o relatório do prédio, com ou sem sua permissão. Como resultado, é preciso estar preparado para a eventual brecha de segurança que ocorrerá quando o usuário perder seu celular ou outro dispositivo pessoal contendo seus dados sensíveis. Dificultar o acesso aos usuários que não queiram seguir as regras manterá alguns deles honestos, mas nem todos.

Contando com relatórios gerados externamente

Os terceiros com quem você interage podem precisar fornecer relatórios antes que você tenha se certificado de como o relacionamento beneficiará sua empresa, os usuários, os dados e a aplicação. Sem esse feedback, você só poderá ter palpites sobre como o terceiro vê sua empresa e o uso que ela faz dos serviços oferecidos pelos terceiros. A questão principal na geração de relatórios externos é garantir que você compreenda o retorno que está tendo sobre o investimento e que o terceiro esteja trabalhando conforme esperado.

Obtendo relatórios completos de terceiros

Qualquer tipo de produto de terceiros para suporte que você adquirir, por exemplo, para testes de aplicação ou de segurança, deve ter um relatório associado. O relatório deve detalhar o serviço prestado e quando ele foi feito. Deve informar a todos que o visualizarão o motivo pelo qual o terceiro realizou o serviço e os resultados (por exemplo, relatórios de teste devem detalhar se os testes foram bem-sucedidos ou se falharam e oferecer meios para acessar os resultados individuais). Qualquer relatório que você solicitar deve fazer parte de uma lista de produtos entregues que o terceiro deve concluir antes de receber o pagamento; caso contrário, é provável que você não vá receber todos os relatórios necessários.

Como ocorre com relatórios criados internamente, qualquer relatório obtido de terceiros deve oferecer uma motivação para uma ação. Terceiros tendem a gerar relatórios para impressionar, não para informar. O papel elegante e um palavreado persuasivo não ajudarão muito quando você estiver lidando com

uma brecha de segurança possivelmente provocada por um erro da parte do terceiro ou uma falta de atenção aos detalhes. Em suma, você deve encarar o relatório do mesmo modo que vê os relatórios internos – sempre questionando o valor que ele agrega a você e à sua empresa.

Sempre que for possível, trabalhe com o terceiro para gerar relatórios que atendam especificamente às necessidades de sua empresa, em vez de aceitar relatórios genéricos que o terceiro cria para todos. O relatório deve refletir o ambiente de sua empresa, seu uso de recursos, o quadro de funcionários e outros fatores que tornam sua empresa única. O relatório personalizado provavelmente será mais caro, mas você terá mais informações, e a economia em termos de tempo dos funcionários frequentemente gera um retorno adicional pelo custo extra.



O máximo possível de testes de verificação devem ser realizados nos dados do relatório. Em muitos casos, você não terá os recursos para verificar todos os aspectos dos relatórios, mas normalmente poderá fazer verificações pontuais para garantir que o terceiro não gerou um relatório contendo informações que ele acha que você quer ver. Para criar um ambiente realmente seguro, você precisa de fatos verificáveis que ajudem a entender melhor a dinâmica de sua aplicação e a encontrar maneiras de manter os hackers afastados.

Desenvolvendo relatórios a partir de dados brutos

Alguns terceiros oferecerão uma API que você poderá usar para acessar os dados brutos associados à sua aplicação. Em vez de confiar totalmente nos relatórios de terceiros, você pode usar os dados brutos para gerar seus próprios relatórios. O serviço normalmente custa mais e nem todos o oferecem, mas vale a pena verificar se ele existe. Quando criar relatórios personalizados, use as diretrizes apresentadas na seção “Usando relatórios personalizados”, para criá-los. O importante é garantir que você obtenha somente as informações necessárias no formato que você precisa para realizar as tarefas necessárias.

Mantendo os dados internos seguros

Um terceiro pode ter acesso a alguns dados internos de sua empresa. Os dados podem não ser sensíveis, mas você deve protegê-los porque sabe que

um terceiro tem acesso a eles. Quando trabalhar com um terceiro, você deve tratar todos os dados aos quais o terceiro tem acesso como dados sensíveis, seguindo as regras que se encontram na seção “Criando relatórios internos”. Além de seguir essas regras, você deve ter o máximo possível de informações sobre o terceiro. Só confie nele depois que executar os passos apropriados para verificar se pode realmente fazer isso. Eis algumas sugestões para garantir que sua aplicação, seus dados e os dados de sua empresa permaneçam seguros quando trabalhar com um terceiro:

- Verifique a identidade de qualquer terceiro que trabalhará com os dados.
- Faça uma verificação em segundo plano para garantir que o terceiro é confiável.
- Monitore qualquer acesso que o terceiro fizer a dados compartilhados e garanta que acompanhará de perto qualquer anomalia percebida.
- Garanta que o terceiro assine um NDA (Nondisclosure Agreement, ou Acordo de Confidencialidade).
- Monitore o terceiro para garantir que não haja nenhuma tentativa de acessar dados sensíveis.
- Crie um procedimento para tratar violações de dados por terceiros ou para administrar um problema relacionado à conformidade com os protocolos de segurança.

Possibilitando feedback de usuários

Feedback de usuários é um elemento essencial de qualquer cenário de relatórios, pois as aplicações são inúteis sem os usuários. Se não puder ter certeza de que os usuários de uma aplicação estão satisfeitos com ela, você não poderá realmente estar certo de que a aplicação é útil. Infelizmente, é um grupo menor de usuários mais expressivos que normalmente oferece feedback – não ouvimos nada da maioria silenciosa. Por esse motivo, você deve contar com várias técnicas para solicitar feedback de usuários e garantir que eles sejam realmente úteis. Depois que determinar isso, você pode começar a usar o feedback para fazer alterações na aplicação que melhorarão a segurança de várias maneiras. Por exemplo, mudanças na interface de

usuário que deixem a aplicação mais fácil de usar e mais compreensível reduzirão o potencial para problemas de segurança que ocorrem como consequência de erros do usuário.

Obtendo feedback de usuários

Feedback de usuários é um tipo de relatório e é uma parte essencial na manutenção de um ambiente seguro para sua aplicação. Em certo nível, os usuários que não estão satisfeitos com o desempenho de uma aplicação e acreditam que suas solicitações de serviços não serão respondidas tendem a encontrar soluções para seus problemas por conta própria – normalmente, com as brechas de segurança e os problemas de confiabilidade resultantes. Em outro nível, o feedback dos usuários informa como sua aplicação está realmente atendendo às necessidades dos usuários. Em muitos casos, os usuários precisam de ajuda e de serviços que eles não são capazes de verbalizar, em parte porque não sabem que a aplicação não está funcionando como você pretendia que ela funcionasse.

É possível obter feedback de usuários de várias maneiras. Alguns métodos funcionam melhor que outros para obter tipos específicos de feedback. Por exemplo, um formulário para comentários informará como um usuário se sente a respeito da aplicação, mas não dirá como ele está realmente interagindo com ela, pois seu feedback é tendencioso (mais sobre esse problema na seção “Determinando a usabilidade do feedback de usuário”). Quando trabalhar com um grupo de usuários, muitas vezes você precisará extrair uma opinião média com base nas informações variadas que receber. Cada usuário pode achar que sua opinião não é tendenciosa, mas todos apresentam distorções quando expressam uma. A distorção afeta as informações que você receber; por sua vez, ela afeta a usabilidade. A lista a seguir apresenta algumas ideias para obter feedback de usuários, mas é importante lembrar que alguns desses métodos são invasivos e outros tendem a gerar resultados tendenciosos:

- Crie formulários de feedback que o usuário possa enviar a você para solicitar novas funcionalidades ou simplesmente comentar as funcionalidades existentes.

- Patrocine grupos de discussão em que vários usuários possam conversar sobre os méritos relativos da aplicação e definir o que eles gostariam que mudasse.
- Faça log das interações do usuário com a aplicação acrescentando verificações no software.
- Monitore as interações do usuário com a plataforma da aplicação para que você possa entender melhor como o usuário trabalha no ambiente da aplicação.
- Monitore o acesso do usuário fora do ambiente da aplicação, por exemplo, acessos a um banco de dados, API ou microsserviço.
- Peça feedback do usuário sempre que um evento especial (por exemplo, uma falha por causa de um bug) ocorrer.



A maioria dos usuários torcerá o nariz para técnicas de monitoração que envolvam tecnologias como logging de teclas (key logging). A sensação é de haver alguém espionando. Quando usar uma abordagem secreta para obter dados de usuário, você deve considerar o custo em termos de sentimentos negativos do usuário em relação às vantagens proporcionadas pela obtenção dos dados. É importante considerar o custo-benefício não só para a aplicação atual, mas para todas as aplicações utilizadas por uma empresa, pois o usuário vai supor que você está monitorando todas elas ao mesmo tempo (mesmo que não esteja). A perda de privacidade e de confiança consequentes do uso de produtos como key loggers é um daqueles problemas que você deve considerar cuidadosamente antes de utilizar um. Também é importante considerar o fato de que os hackers muitas vezes usam o próprio software de monitoração, com o qual você está contando, para invadir sua rede.

Determinando a usabilidade do feedback de usuário

Obter feedback de usuário é útil, mas somente quando você considera as circunstâncias em que os dados são obtidos. Falar diretamente com o usuário sempre produzirá um resultado tendencioso. Monitorar o usuário pode gerar um resultado não tendencioso, mas se o usuário souber que a monitoração está ocorrendo, suas ações não refletirão as condições reais de uso, e os dados que você receber serão suspeitos. Fazer logging das interações do usuário por meio de verificações no software não oferece informações em profundidade como aquelas oferecidas pela monitoração. A questão é que você deve

considerar como as informações foram recebidas antes de poder tomar qualquer decisão sobre sua utilidade para realizar uma tarefa específica.

Para colocar em perspectiva, um usuário pode solicitar uma funcionalidade em particular porque parece ser interessante. No entanto, os testes mostram que a funcionalidade provavelmente aumentará o risco de uma violação nos dados por causa do modo como eles são acessados. Além disso, a monitoração mostra que o usuário não terá muitos benefícios com a funcionalidade – ela raramente será usada no mundo real. Consequentemente, você tem três informações que devem ser correlacionadas para determinar se a funcionalidade deve ser mantida. Remover a funcionalidade quase sempre gerará comentários por parte dos usuários, portanto você precisa ter um bom motivo para removê-la. Em situações como essa, você deve considerar que outros tipos de feedback úteis de usuário podem ajudar. Por exemplo, você pode trazer à tona os problemas com a funcionalidade em uma sessão em grupo de modo a poder contar com o feedback do grupo para ajudar a entender a necessidade da funcionalidade.

A segurança da aplicação muitas vezes está atrelada à participação do usuário. Ataques de engenharia social são devastadores, mas tentar frustrar os esforços de um funcionário insatisfeito pode ser mais difícil ainda. Parte do propósito do feedback de usuários é garantir que o usuário permaneça comprometido em trabalhar com a empresa como um todo, para manter a aplicação e seus dados seguros. Em alguns casos, você deverá pesar os efeitos de aceitar feedback de usuários que podem não parecer muito úteis no início para preservar a boa-vontade do usuário em relação à aplicação.



Alguns desenvolvedores veem as sessões de grupo como um processo doloroso, a ser evitado por qualquer indivíduo são. No entanto, sessões em grupo e as conclusões que elas proporcionam ajudam a fazer uma pressão social sobre o problema da segurança. Em alguns casos, a pressão social é suficiente para evitar que os usuários cometam alguns erros que eles cometeriam sem a introdução da pressão social. Quando estiver trabalhando com uma decisão difícil, uma sessão em grupo também pode evitar que você pareça ser o vilão quando chegar a hora de remover uma funcionalidade que poderia apresentar um potencial risco de segurança, com pouco valor para o usuário. Se usadas de modo apropriado, as sessões em grupo podem oferecer uma forma de feedback de usuário de valor inestimável, que também ajudarão a criar um ambiente mais seguro para sua aplicação.

Durante o processo de avaliação do feedback de usuários, você também deve considerar a política da empresa. O feedback de usuário é sempre tendencioso, mas o nível de distorção muitas vezes depende da percepção do valor da aplicação para o usuário e das políticas por trás do processo de tomada de decisão. Algumas situações exigem que você desconsidere parcialmente o feedback direto do usuário e procure obter feedback por meios indiretos (por exemplo, monitoração) para determinar em que ponto termina a política e começa a verdadeira distorção.

Não se esqueça de considerar todos os feedbacks de usuário com a assistência de sua equipe de desenvolvimento e mantenha a gerência informada de seu progresso. Contudo, é importante lembrar que tanto sua equipe de desenvolvimento quanto a gerência são compostas de pessoas, com suas próprias ideias tendenciosas e agendas políticas. É da natureza humana desconsiderar informações do usuário que você realmente não quer ouvir em favor de opiniões da equipe de desenvolvimento que correspondam mais de perto às suas próprias ideias. No entanto, às vezes o usuário realmente está certo e você deve considerar esse ponto de vista antes de tomar decisões.

Como parte do processo de tomada de decisão, considere também outras fontes de informação à sua disposição. Os relatórios que você gerar informarão não só sobre a aplicação, a plataforma, os serviços de terceiros e assim por diante – eles também informarão sobre o estado de sua empresa e o ajudarão a tomar decisões melhores sobre os feedbacks de usuário recebidos. Lembre-se de considerar as informações recebidas de fontes externas, por exemplo, de artigos da imprensa especializada. Uma notícia informando que determinado tipo de hack está acontecendo pode pesar mais contra a inclusão de funcionalidades específicas na aplicação, que facilitariam ao hacker ter acesso à sua aplicação. Em suma, use todos os seus recursos para compor o quadro-geral, em vez de ter uma visão míope apenas do feedback recebido do usuário.

PARTE V

Localizando recursos de segurança

Este livro tem apresentado uma variedade de informações sobre ameaças de segurança, códigos-fonte, correções e técnicas de monitoração. Você descobriu diversas maneiras pelas quais os hackers frequentemente têm acesso à sua empresa e dificultam a vida de todos. As brechas em dados criadas pelos hackers se apresentam de várias formas e têm efeitos significativamente diferentes. No entanto, você realmente precisa de mais informações sobre esses problemas e é disso que trata esta parte do livro.

O capítulo 16 discute os métodos que você pode usar para monitorar as ameaças de segurança. Essas ameaças mudam continuamente e você precisa acompanhá-las de perto. Contudo, você também não vai querer sucumbir à sobrecarga de informações, portanto obter as informações corretas o mais rápido possível é essencial. Este capítulo ajuda você a localizar as informações necessárias sem gastar muito tempo ou energia para isso.

O capítulo 17 discute o treinamento. Todos precisam de treinamento. De fato, se você observar muitos dos relatórios detalhados sobre brechas de segurança, logo descobrirá que se houvesse mais treinamento, elas poderiam ter sido evitadas, antes de tudo. É claro que você precisa conhecer os tipos de treinamento de segurança que deve obter e quem deve fazê-los. Este capítulo apresenta todos os detalhes para você.

CAPÍTULO 16

Monitorando as ameaças de segurança atuais

O conhecimento é uma ferramenta excepcionalmente eficaz nas mãos certas. Para estar sintonizado com os problemas de segurança que podem afetar sua aplicação, você precisa monitorar as ameaças de segurança atuais. Contudo, se você presta o mínimo de atenção ao noticiário na mídia, sabe que tentar estar a par de todas as possíveis ameaças de segurança pode se tornar um trabalho de tempo integral. Acrescente a isso as histórias da mídia especializada e você terá um verdadeiro trabalho de tempo integral e, possivelmente, uma tarefa que exigirá uma equipe para administrar o fluxo de informações. Está claro que você precisa de algum método para distinguir quais ameaças realmente exigem sua atenção, ou poderá acabar gastando tempo demais monitorando informações e não terá tempo suficiente para realmente evitar as ameaças.

Depois que você decidir que a ameaça de segurança é real, é preciso decidir que tipo de ação deve ser tomada. Lembre-se de que, de acordo com o capítulo 15, o único relatório realmente necessário é aquele que permite realizar algum tipo de ação. Podemos dizer o mesmo sobre relatórios de ameaças à segurança. A discussão sobre ameaças de segurança que você deve investigar de forma completa é aquela que exige algum tipo de ação de sua parte. Tudo o mais é excesso de informação e desperdiçará seu tempo. É claro que você precisa decidir entre um update ou um upgrade em sua aplicação quando a situação exigir. A questão é que você deve pensar em como reagir a ameaças de segurança e então executar as ações necessárias no momento certo (ou simplesmente colocar o relatório no arquivo, que é o lugar a que ele pertence).

Este capítulo não tem a ver apenas com pesquisas sobre ameaças à segurança – tem a ver com atuar sobre as informações relacionadas a ela. Para manter sua aplicação e os dados associados seguros, você deve conhecer as ameaças de segurança e então atuar sobre as informações recebidas. O pior que pode acontecer é você atingir uma situação de crise estando despreparado, sem um plano e sem as informações necessárias para lidar com a ameaça.

Evitando o sensacionalismo, a histeria e a impotência

Existe um problema no mercado de segurança. Na realidade, existem vários problemas, mas todos têm a mesma origem: sensacionalismo, histeria e impotência. O mercado de segurança floresce em cima dessas três palavras e muitos profissionais de segurança se deixam levar por elas também. Com frequência, a mídia participa disso porque manchetes sensacionalistas vendem conteúdo. A questão é que as pessoas que deveriam acalmar, tranquilizar e apaziguar você fazem de tudo, menos isso. É fácil se tornar um exemplo de incapacidade no mercado de segurança – todos querem deixar você paranoico sobre tudo, o tempo todo.

O sensacionalismo continua sem dar trégua porque é a base para vender falsas promessas – algumas relacionadas a produtos, outras não. Por exemplo, é provável que você já tenha ouvido falar várias vezes que um vírus ou um hack em particular acabaria com a internet como a conhecemos, mas ela continua existindo. O fato é que uma brecha de segurança causará danos e poderá arruinar seu dia (e até mesmo a sua carreira), mas os tipos de prognóstico que abalam o mundo raramente se tornam verdadeiros, se é que chegarão a isso. Quando ouvir uma afirmação que pareça bizarra, provavelmente ela o será.

Mesmo quando uma afirmação é verdadeira, as pessoas em posição de liderança no mercado de segurança muitas vezes promovem uma perspectiva histérica sobre ela. A manchete sensacionalista que você lê pode incitar a curiosidade para comprar conteúdo, mas não serve realmente às suas necessidades. A histeria jamais venceu uma guerra e não funciona particularmente bem no mercado de segurança. O que você deve realmente fazer é considerar os efeitos de uma ameaça em sua empresa, fazer o que puder para atenuá-la e então monitorar seus sistemas a fim de determinar quando (ou se) alguém conseguiu invadir. Uma mente tranquila normalmente faz você ganhar o dia.

Tudo se reduz a uma questão de fazer você se sentir impotente. Afinal de contas, a menos que você se sinta realmente impotente, os vários fornecedores de segurança não poderão cobrar para virem em seu resgate. Se este livro não ensinou mais nada, você deveria saber, a essa altura, que o conhecimento lhe dá o poder de assumir a

responsabilidade pela segurança de várias maneiras e que as ferramentas, embora sejam úteis, são apenas uma pequena parte de uma solução muito mais ampla. Você não está realmente impotente, pois tem todo tipo de recursos à sua disposição.

Desenvolvendo fontes para obter informações sobre ameaças de segurança

Pelo fato de haver tantas informações de segurança amplamente disponíveis, é preciso cultivar fontes de informações para ameaças de segurança que ofereçam o material na forma necessária e que se mostrem sucintas o suficiente para revelar ameaças sem desperdiçar muito tempo. Você vai querer informações de profissionais de segurança, pois eles têm o tempo e os recursos para pesquisar as informações de forma completa. No entanto, você precisa de informações de fontes administradas por profissionais com uma perspectiva semelhante à sua, e não sujeitas ao nível usual de sensacionalismo, histeria e impotência. As próximas seções analisam as fontes mais comumente usadas para informações sobre ameaças à segurança e ajudam a entender o papel que cada fonte desempenha para ajudar a tornar sua aplicação segura.

Lendo artigos relacionados à segurança escritos por especialistas

Você não tem tempo suficiente para fazer pesquisas sobre todas as fontes de ameaças à segurança em potencial. É claro que você poderia contratar sua própria equipe para ajudar, mas é preciso considerar a disponibilidade de especialistas em segurança que escrevem em revistas especializadas como uma boa segunda opção. Esses especialistas tendem a passar a maior parte de seu tempo envolvidos com problemas de segurança, portanto conhecem todos os detalhes das várias ameaças. Por exemplo, Jeremy Kirk recentemente revelou que *mesmo bancos de dados criptografados de registros médicos deixam vaziar informações* no artigo “Even Encrypted Medical Record Databases Leak Information” (<http://www.computerworld.com/article/2980593/data-privacy/even-encrypted-medical-record-databases-leak-information.html>). É o tipo de

artigo em que você deve prestar atenção porque a maioria dos negócios depende de bancos de dados para armazenar informações e os bancos de dados que armazenam informações sensíveis normalmente contam com algum tipo de criptografia. O tipo de artigo que você deve procurar tem as seguintes características em comum:

- Um título claro, sem uma linguagem sensacionalista (até o ponto em que for possível, de qualquer modo)
- Uma visão geral resumida do problema para que você possa decidir se a ameaça exige mais pesquisas
- Descrições breves dos métodos empregados pelos hackers para usar o hack
- Links para informações adicionais para que você não gaste horas simplesmente procurando detalhes
- Estatísticas sobre os efeitos da ameaça (possibilitando avaliar o risco para sua empresa de forma mais inteligente)
- Exemplos de estudos de caso que mostrem a viabilidade do ataque em um alvo do mundo real

Dar uma olhada superficial no artigo leva alguns segundos, mas, ao procurar essas características, você normalmente evitará ler um artigo que realmente não informe nada. O importante é obter as informações necessárias sem desperdiçar muito tempo com isso. Após um tempo, você passará a saber quais autores na mídia especializada atendem melhor às necessidades de seu negócio e começará a prestar mais atenção nos artigos escritos por eles – fazendo você economizar mais tempo. (De modo semelhante, você descobrirá quais autores escrevem artigos que são um desperdício total de tempo e os descartará.)

Quando lidar com artigos na imprensa, lembre-se de que a função principal do autor é atrair leitores para vender os produtos anunciados nas propagandas. É assim que uma revista online paga pelos serviços do autor. Como resultado, você precisa ter cuidado quando ver manchetes como “Is Your Connected Car Spying on You?” (Seu carro conectado está espiando você?, <http://www.bbc.com/news/business-29566764>). O material no artigo provavelmente é preciso do ponto de vista técnico, mas o autor usou uma linguagem para fazer a ameaça parecer mais significativa do que realmente é;



outra história desse tipo está relacionada a babás eletrônicas – “9 Baby Monitors Wide Open to Hacks That Expose Users’ Most Private Moments” (Nove babás eletrônicas totalmente abertas a hacks que expõem os momentos mais privados dos usuários, <http://arstechnica.com/security/2015/09/9-baby-monitors-wide-open-to-hacks-that-expose-users-most-private-moments/>). O propósito é conseguir visualizações, mas o efeito é gerar histeria. É claro que você pode estar se perguntando o que carros e babás eletrônicas têm a ver com aplicações web. O fato é que muitas aplicações web atualmente executam em carros e em babás eletrônicas para realizar todo tipo de tarefas – fazem de tudo, desde oferecer entretenimento até monitorar o ambiente. Mesmo que sua empresa não esteja envolvida na criação desse tipo de aplicação web, os mesmos princípios se aplicam.

O melhor motivo para usar artigos da imprensa especializada para localizar informações sobre ameaças de segurança é que você pode obter informações sobre muitas ameaças em um período de tempo extremamente curto. Simplesmente procurar manchetes claras de autores que você conhece permite identificar as ameaças mais importantes em apenas alguns minutos.

A principal desvantagem em usar artigos da imprensa especializada é a falta de profundidade e a necessidade de prestar atenção em uma escrita que é menos informativa do que poderia ser. Certifique-se de que vai contar intensamente com autores que ofereçam links para os detalhes sobre uma ameaça a respeito da qual você deve saber de tudo, sem precisar gastar horas procurando. Se começar a achar que o artigo da imprensa especializada está detalhado demais, você terá um bom motivo para suspeitar que ele pode ter mais a ver com venda de produtos que mantê-lo informado. Sempre procure artigos breves e informativos, com vários links.

Verificando sites de segurança

Sites de segurança tendem a oferecer informações detalhadas para uma grande variedade de ameaças de segurança. O site de segurança pode não

discutir todas as ameaças atuais, mas você pode estar certo de que encontrará a maioria das ameaças ativas no momento (e, provavelmente, uma história de todas as ameaças que os profissionais de segurança destruíram no passado). De fato, o simples peso de todas essas informações faz com que os sites de segurança sejam uma escolha ruim para sua primeira consulta. O que você realmente deve fazer é criar uma lista das ameaças para pesquisar e então acessar os sites de segurança para procurar informações sobre essas ameaças em particular.

Nem todos os sites de segurança são iguais – um fato que passa despercebido pela maioria das pessoas que navega pela internet procurando informações de segurança. De fato, os sites de segurança se apresentam em quatro formatos diferentes (sintetizados na figura 16.1), cada um com suas próprias vantagens e desvantagens.

Sites organizacionais

Sites organizacionais geralmente oferecem as informações mais concentradas em uma área em particular, mas não têm a abrangência de informações que outros tipos de site têm. Por exemplo, o SANS Institute (<http://www.sans.org/>) oferece fontes de pesquisa sensacionais. Ele também patrocina vários tipos de eventos de treinamento. Essa organização em particular (como muitas outras) é apoiada por fornecedores, mas ao menos você não tem o ponto de vista de apenas um fornecedor sobre como deve trabalhar com problemas de segurança.

Sites governamentais

Esses sites obviamente são tendenciosos em direção às necessidades do governo que os patrocina, mas em sua maior parte, não são tendenciosos quanto aos tipos de informação oferecidos. Esses sites são bons lugares para localizar informações sobre ameaças atuais de natureza internacional. Sites como o United States Computer Emergency Readiness Team (US-CERT, <https://www.us-cert.gov/>) e o National Vulnerability Database (<https://nvd.nist.gov/>) oferecem dicas e publicações em abundância, que podem ser obtidas sem custo.

Sites de comunidades

Entre os sites de segurança que você pode acessar, os sites de comunidades podem se mostrar os mais interessantes. Muitas vezes, eles oferecem insights que você não encontrará em outros lugares. Como qualquer outro site que você acessar, eles têm algum tipo de agenda, e essa agenda faz as informações que você receber serem tendenciosas, porém essas distorções normalmente não são fortes o suficiente para afetar a qualidade das informações. Sites como o Vmyths (<http://vmyths.com/>) podem ajudar a evitar a histeria que normalmente resulta da revelação de incidentes de segurança.

Sites de fornecedores

Quando acessar um site de fornecedor, com frequência você obterá informações em profundidade, sem paralelo com qualquer outra fonte, parcialmente porque o fornecedor deseja impressionar você. No entanto, como o fornecedor também está ativamente engajado em tentar frustrar várias ameaças de segurança, você também poderá obter insights melhores sobre o que precisa fazer para se preparar para uma ameaça em particular. Sites como a Symantec Security Response (http://www.symantec.com/security_response/) possibilitam verificar rapidamente aspectos como o nível atual de ameaças na internet e determinar quais ameaças apresentam mais potencial de problemas para a maioria das pessoas.



Sites de segurança de fornecedores existem para vender seus produtos de segurança. Você deve estar ciente desse fato quando acessar o site, pois os artigos conterão informações tendenciosas, que se inclinam a adquirir um produto que resolverá a ameaça de segurança. Às vezes, você realmente precisará de um produto de terceiros para resolver um problema, mas uma pesquisa cuidadosa, a atenção aos detalhes, uma configuração apropriada do sistema e um ou dois updates geralmente fazem mais para resolver a ameaça que comprar um produto. Lembre-se de analisar e validar os fatos obtidos em um site de segurança de um fornecedor antes de agir com base neles. Pense nos métodos que você já tem à sua disposição para resolver um problema antes de comprar mais um produto.



Figura 16.1 – Escolher a fonte de informações correta para ameaças à segurança é importante.

Alguns sites são especializados em consolidar informações de outras fontes. Por exemplo, o Security Focus (<http://www.securityfocus.com/>), o OSVDB (Open Source Vulnerability Database, <http://www.osvdb.org/>) e o CVE (Common Vulnerabilities and Exposures, <https://cve.mitre.org/>) oferecem índices com informações que também se encontram em outros sites. Usar um site que apresenta informações consolidadas pode fazer você economizar tempo, mas também abre a possibilidade de deixar passar uma notícia importante sobre segurança porque o proprietário do site tem uma posição tendenciosa, diferente das necessidades de sua empresa.

Um aspecto importante a considerar é que você realmente só precisa visitar um site de segurança quando achar que há ameaça de segurança sobre a qual deve pesquisar. Do contrário, seu tempo será mais bem gasto procurando informações gerais sobre ameaças à segurança com as quais você poderá ter de lidar. Certifique-se de visitar o tipo de site que seja mais adequado às suas necessidades para uma ameaça em particular. Por exemplo, uma ameaça internacional perpetrada por um país contra outro provavelmente receberá uma cobertura melhor em um site governamental. Casos de hackers tentando extorquir fundos de você após terem criptografado os dados de seu disco rígido geralmente são mais bem detalhados em sites de fornecedores. Se você achar que uma ameaça provavelmente seja totalmente falsa, não se esqueça

de visitar um site de comunidade antes para conferir.

Obtendo informações de consultores

Todas as fontes de informações discutidas até agora neste capítulo têm um aspecto em comum: elas oferecem informações gerais com base em uma fonte externa. No entanto, seu negócio não é uma entidade genérica e tem necessidades específicas de segurança de acordo com o tipo de aplicações que você executa, o ambiente em que elas são executadas e o nível de treinamento de seus usuários. Em alguns casos, as fontes gerais não oferecem informações suficientes porque elas simplesmente não podem fornecê-las – os autores dessas informações não sabem nada sobre sua empresa. Nesse caso, normalmente você deverá empregar os serviços de um consultor para avaliar as ameaças de segurança.



É importante lembrar que os consultores são recursos pagos, o que significa que eles poderiam (e provavelmente vão) ser bem mais tendenciosos que outras fontes que você usar. Afinal de contas, o consultor representa um negócio que visa ao lucro. Alguns consultores são bem éticos e jamais cobrariam a mais de propósito; no entanto, o fato de o dinheiro trocar de mãos e o consultor ter um relacionamento mais pessoal com você que outras fontes de informação provavelmente fará do consultor uma fonte de informações com a qual é preciso ser mais cauteloso. Você deve equilibrar o profundo conhecimento que um consultor tem sobre sua instalação em particular com o fato de ele ser mais tendencioso.

Um consultor pode oferecer informações específicas sobre ameaças específicas enfrentadas por sua empresa em particular. O conselho possibilitará lidar com as ameaças de modo muito mais rápido em relação a criar sua própria estratégia. É claro que a qualidade da informação tem um preço. Usar um consultor também é a maneira mais custosa de lidar com ameaças de segurança. Em geral, a maioria das fontes de informações apresentadas neste capítulo oferece algum nível de suporte gratuito. É melhor usar consultores quando:

- você já sabe que as ameaças são sérias;
- você não é capaz de criar uma estratégia para lidar com as ameaças rapidamente;

- os métodos necessários para tratar as ameaças estão fora dos domínios das habilidades que sua empresa tem a oferecer;
- tratar as ameaças exige coordenação entre vários grupos ou contato com outras empresas;
- há outras circunstâncias atenuantes que fazem com que seja inviável tratar a ameaça de segurança por conta própria.

Evitando sobrecarga de informações

Há tantos hackers atuando intensamente para deixar sua vida interessante que seria excepcionalmente fácil ficar sobrecarregado com informações sobre os problemas que afetam a segurança hoje em dia. De qualquer modo, a maioria das pessoas está diante de um caso terrível de sobrecarga de informações. É possível ter uma caixa de entrada completamente lotada de links para artigos que você realmente deveria ler, mas jamais consegue por causa de preocupações com cronogramas (como aquelas reuniões intermináveis que existem na maior parte das empresas). Mesmo assim, você precisa dedicar um pouco de tempo para tentar descobrir quais ameaças são assustadoras o suficiente para exigir uma ação de sua parte, motivo pelo qual você evita se sobrecarregar com informações, antes de tudo. Os passos a seguir sintetizam técnicas que podem ser usadas para evitar ser sobrecarregado com informações e, possivelmente, evitar totalmente as ameaças de segurança sobre as quais você lê:

1. Escolha fontes de informações sobre segurança cuidadosamente. Certifique-se de que as fontes de informações sobre segurança escolhidas estejam relacionadas às necessidades de sua empresa, apresentem as informações necessárias de forma sucinta e ofereçam apenas os detalhes de que você precisa, sem exageros.
2. Localize correções de segurança em potencial que não dependam de alterar sua aplicação. Em alguns casos, as melhores fontes de informação sobre segurança podem apresentar uma correção rápida para seu problema, como instalar um update no navegador ou outro tipo de update.
3. Crie uma abordagem baseada em tarefas para lidar com as ameaças. Se

você não puder resumir uma ameaça de segurança em uma única frase, é sinal de que você realmente não a entendeu bem o suficiente para saber se precisa fazer algo. Desenvolver o hábito de resumir a ameaça rapidamente em uma descrição constituída de uma só frase fará você economizar tempo e esforços a longo prazo.

4. Descarte fontes de informações que tendem a usar o sensacionalismo para descrever as ameaças de segurança. Você só precisa dos fatos, e nada além dos fatos. Quando uma fonte de informações começa a exagerar no sensacionalismo, você não poderá depender dela para conhecer as verdadeiras ameaças de segurança que você precisa corrigir.
5. Desenvolva habilidades para pensar de forma crítica. Algumas das pessoas mais inteligentes baseiam suas ações no pior que poderia acontecer se elas não agirem. Pensar em ameaças na forma de um cenário de pior caso ajuda a descobrir quais ameaças apresentam os maiores riscos para que você possa dedicar mais tempo em evitá-las.
6. Evite a ameaça do tipo fogo de palha. Algumas ameaças aparecem em uma manchete apenas uma vez e você nunca mais vai ler algo sobre ela. Mesmo quando as previsões sobre essas ameaças são sombrias, você deve se perguntar quão sombrias elas são se elas não permanecerão por tempo suficiente para gerar uma segunda manchete.

Como parte de sua estratégia para evitar a sobrecarga de informações, não se esqueça de investir o tempo necessário para entender como as ameaças realmente funcionam. Por exemplo, você já deve ter ouvido falar de algumas ameaças como o *rowhammer* (<http://www.rowhammer.com/>), que pode causar muitos estragos em um sistema. No entanto, quando ler mais sobre o assunto, você começará a entender que ele é um hack interessante, mas exige o tipo de acesso ao sistema que os hackers normalmente não têm quando lidam com aplicações web. Consequentemente, é provável que você possa ignorar o *rowhammer* por enquanto. É claro que um ótimo recurso de segurança teria informado isso a você desde o início, mas às vezes você deve efetuar a pesquisa por conta própria para evitar ser inundado com informações que realmente não poderá usar.

Lembre-se de empregar filtragem de dados quando puder. Realizar pesquisas



rápidas em suas fontes de informações reunidas em busca de palavras-chaves em particular pode ajudar a determinar se você deve considerar uma ameaça como algo que deve ser realmente vencido. Muitas das ferramentas disponíveis atualmente no mercado permitem filtrar e reorganizar informações de várias maneiras. Saber como usar essas ferramentas de modo eficiente fará você economizar tempo a longo prazo à custa de um pouco de esforço adicional para aprender a usar a ferramenta hoje.

Criando um plano para upgrades baseado em ameaças

Quando receber informações sobre uma ameaça em potencial com antecedência suficiente e a quantidade de código necessária para corrigir o problema for muito grande, você precisará realizar um upgrade em sua aplicação. É claro que, inicialmente, é preciso descobrir se há algo que você deva fazer. Algumas ameaças merecem atenção, mas são do tipo que não afetarão sua aplicação por exigirem algum tipo de acesso especial ou por afetarem o uso de um software que você nem mesmo tem. As próximas seções ajudam a determinar se um upgrade é necessário com base nas informações sobre ameaças de segurança e como abordar o upgrade depois que você souber que ele será necessário.

Previendo situações que não exigem nenhuma ação

Sempre que ler sobre uma ameaça de segurança e pesquisar suas implicações é preciso ter em mente que a documentação que você ler fará a ameaça parecer sombria. E a ameaça, de fato, é sombria para alguém. No entanto, a ameaça pode não significar nada para você. As empresas muitas vezes gastam tempo e dinheiro reagindo a ameaças que não representam nenhum tipo de problema para elas. Em suma, elas entram no jogo da histeria que cerca as ameaças de segurança quando não há necessidade de fazer isso. Um dos aspectos que você deve considerar quando analisar as ameaças de segurança do momento é verificar se elas realmente se aplicam a você. Em muitos casos, elas não se aplicam e não será necessário fazer nada a respeito delas. Você pode descartá-las despreocupadamente, sem pensar duas vezes.

É claro que o problema é chegar até o ponto em que você sabe que a ameaça

não o afetará. Uma ameaça não identificada representa um problema sério porque pode abrir as portas a um ataque e a tudo que se segue como consequência (como um site inativo ou dados violados). Avaliar o risco em potencial de uma ameaça de segurança em particular envolve:

- determinar se você está usando a tecnologia descrita como ponto de entrada da ameaça de segurança;
- considerar se o tipo de acesso exigido para invocar a ameaça realmente se aplica à sua empresa;
- definir as circunstâncias em que sua empresa poderia ser atacada e quando o ataque poderia ocorrer;
- rever as medidas de proteção implantadas no momento a fim de determinar se são suficientes para frustrar o ataque;
- garantir que você tenha uma estratégia definida, caso o ataque ocorra e você esteja enganado sobre a capacidade do hacker de fazê-lo em sua empresa.

Decidir-se entre um upgrade e um update

Os capítulos 13 e 14 discutem upgrades e updates que ocorrem naturalmente. No entanto, nesse caso, você está pensando em um upgrade ou em um update como consequência de uma ameaça à segurança. Muitas das mesmas regras se aplicam. Um upgrade sempre indica algum tipo de mudança significativa de código, enquanto um update pode não exigir uma mudança de código, mas pode refletir uma mudança na interface de usuário ou em algum outro elemento que pode ser alterado rapidamente, sem modificar o código subjacente. Como consequência, a primeira demarcação entre upgrades e updates é aquela que você normalmente aplicaria quando faz qualquer outra mudança.

Uma ameaça de segurança pode mudar o propósito e a urgência de um upgrade ou de um update. Em alguns casos, você deve enfrentar o problema por partes:

1. Abordar o problema específico em que a ameaça de segurança está focada como um update.

2. Fazer um upgrade ou um update em qualquer software de apoio (como firewalls ou o sistema operacional) para garantir que a ameaça de segurança não possa atacar a aplicação a partir de outra direção.
3. Treinar os usuários para evitar que realizem tarefas de modo a oferecerem aos hackers informações valiosas ou um acesso que o hacker possa usar para iniciar o ataque.
4. Modificar qualquer código em torno do código-alvo para garantir que a aplicação continue a funcionar exatamente como projetado para um upgrade após ter implantado o update.
5. Realizar um upgrade de qualquer dependência para garantir que a aplicação toda esteja estável.
6. Verificar se a aplicação após o upgrade continua a evitar a ameaça de segurança realizando testes abrangentes com a aplicação como alvo, como faria um hacker.

Nem toda ameaça de segurança exige essa longa lista de passos para que seja vencida, mas é importante considerar cada passo quando decidir como você deverá proceder. Talvez você descubra que um update será suficiente para frustrar o ataque ou que um upgrade no sistema operacional é tudo de que você realmente precisa. A questão é considerar todos os possíveis aspectos da ameaça de segurança em relação à sua aplicação e aos dados tratados por ela.



Quando estiver diante de um ataque dia zero (zero-day attack), é sempre melhor começar com um update e testá-lo usando o menor período de tempo possível. Os hackers não esperarão que você implante o update antes de atacar o seu sistema. De fato, os hackers esperam que você fique enredado com as políticas e os procedimentos organizacionais – dando-lhes mais tempo para atacar a sua aplicação. A velocidade é um elemento essencial para manter os hackers a distância, mas certifique-se de sempre testar qualquer software antes de implantá-lo a fim de garantir que você não estará, na verdade, piorando a situação. Atenuar a ameaça de segurança exige uma estratégia precisa e implementada o mais rápido possível.

Às vezes, você realmente deve tomar decisões rápidas quanto ao uso de um upgrade ou de um update para corrigir uma ameaça de segurança. Os passos nesta seção certamente ajudam, mas o diagrama de fluxo na figura 16.2 apresenta um método mais rápido ainda para tomar a decisão.

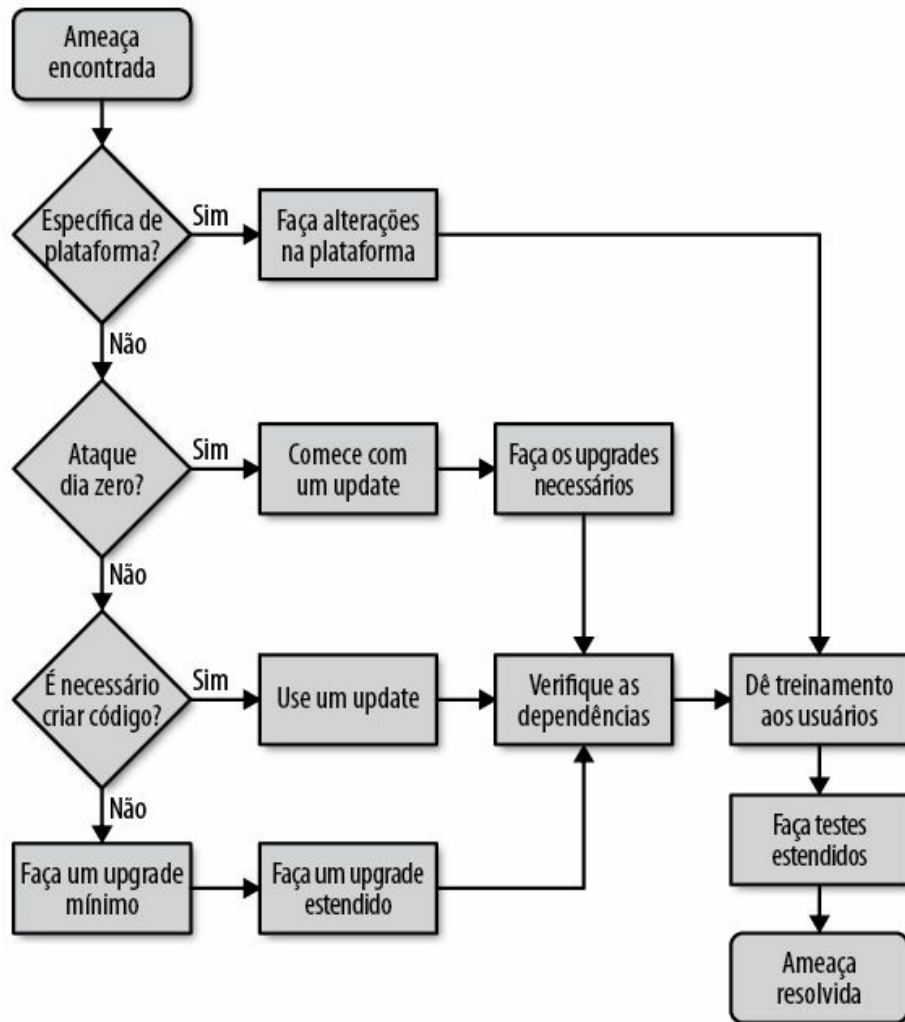


Figura 16.2 – Tomando a decisão entre um upgrade e um update rapidamente às vezes é necessário.

Definindo um plano de upgrade

O capítulo 13 descreve o processo para um upgrade-padrão em sua aplicação. Ele detalha um processo que poderia exigir meses para ser concluído. É claro que você terá tempo quando estiver realizando um upgrade em sua aplicação de modo a executar todas as tarefas de forma cuidadosa e completa. As ameaças de segurança raramente esperam tempo suficiente para que você possa esperar meses e criar um upgrade e implantá-lo. A única maneira de obter o nível de aviso prévio para realizar um upgrade de longo prazo é ter alguém no lugar certo para obter informações quando elas se tornarem disponíveis inicialmente (muitos hackers éticos dão de três meses a um ano

aos fornecedores para corrigirem os problemas antes de torná-los de conhecimento público).



Testes constantes em sua aplicação feitos por uma empresa especializada em testes de segurança podem dar mais tempo a você para fazer upgrades. Uma empresa de testes de segurança pode deixar você ciente das possíveis deficiências em sua aplicação bem antes que elas se tornem de conhecimento público. É claro que para ter esse nível de conhecimento prévio, você deve estar disposto a pagar pelo serviço, e o preço pode ser bastante alto.

Quando trabalhar com um upgrade em resposta a uma ameaça de segurança, você deve considerar metodologias que permitam criar o upgrade e testá-lo de forma incremental à medida que trabalhar, para poder disponibilizar o upgrade em um período de tempo menor. Técnicas de programação ágeis (<http://agilemethodology.org/>) podem ajudar nessa tarefa. É claro que se sua empresa não usa técnicas de programação ágeis normalmente, o tempo necessário para conhecê-las não será de muita ajuda. Mesmo assim, parte de seu plano de upgrade deve incluir alguma metodologia para criar o código e testá-lo em um intervalo bem menor de tempo que o normal. Após a ameaça ter passado, você deve retornar ao código e garantir que não introduziu outras brechas de segurança na correção da brecha representada pela ameaça de segurança.

No capítulo 14, a seção “Efetuando updates de emergência”, discute a necessidade de ter uma equipe de resposta rápida. Em muitos casos, usar essa equipe para criar o upgrade economizará tempo e, possivelmente, implicará em um custo mais baixo. A equipe já está acostumada a trabalhar em conjunto para criar updates rapidamente. Usá-la para criar um upgrade em resposta a uma ameaça de segurança faz sentido. No entanto, você deve garantir que a equipe tenha funcionários adequados para lidar com um upgrade. Talvez seja necessário acrescentar membros à equipe para oferecer serviços além daqueles normalmente exigidos por um update.

Um problema fatal enfrentado por muitas empresas são os gerentes que ficam histéricos e exigem atualizações constantes. O problema do “já conseguimos terminar?” deixa o progresso consideravelmente mais lento em um upgrade. A equipe de desenvolvimento responsável pelo upgrade deve deixar a gerência informada regularmente, mas a gerência também deve se dar conta da necessidade de deixar a equipe de desenvolvimento trabalhar em paz. Do



contrário, enquanto um membro importante da equipe está entregando outro relatório desnecessário, os hackers invadirão seu sistema e farão seus piores pesadelos se tornarem realidade.

Criando um plano para updates baseado em ameaças

Em alguns casos, você recebe informação de ameaças de segurança com tão pouco tempo de antecedência que precisará fazer um update agora e um upgrade mais tarde. Os updates fazem menos alterações no código da aplicação e, às vezes, nem afetam o código. Como parte do planejamento de um update baseado em informações sobre ameaças, você deve considerar se o update é uma emergência ou não. Além disso, você deve criar um plano que considere o efeito de qualquer software de terceiros do qual você dependa para fazer sua aplicação funcionar.

Verificar se os updates resolvem as ameaças

O foco deste capítulo está na velocidade de execução. Quando identificar uma ameaça de segurança, você deve atuar sobre a informação o mais rápido possível. Consequentemente, um update é o método preferido para atacar o problema, pois os updates exigem menos tempo, menos recursos e causam menos problemas em potencial durante os testes. Nesse caso, os updates implicam mais que apenas a sua aplicação e podem incluir:

- ajustes no software de segurança já instalado;
- alterações na plataforma do host;
- modificações no navegador do host;
- alterações em procedimento com base em treinamento de usuário e em outros meios;
- updates na aplicação subjacente por meios que envolvam pouco ou nenhum código;

- modificação em softwares dependentes como bibliotecas, APIs e microsserviços.

Infelizmente, os updates nem sempre corrigem o problema. Se a meta é evitar uma ameaça de segurança, então encontrar o curso de ação correto antes é a melhor maneira de proceder, pois o tempo desperdiçado tentando outras medidas simplesmente atrasará a abordagem que funciona. Os hackers não esperam. Às vezes, a velocidade de execução se reduz a executar correções mais detalhadas e simplesmente admitir que você precisa de mais tempo do que provavelmente é conveniente para fazer um reparo.

O cerne do problema é observar o tipo de ataque feito pelo hacker. Por exemplo, um ataque man-in-the-middle pode simplesmente exigir que você criptografe dados de entrada e de saída corretamente, com mais cuidado. Além disso, medidas adicionais de autenticação e de autorização podem ser necessárias. Um hack desse tipo não é corrigido facilmente com um update, pois você estará olhando para o modo como os dados são preparados, enviados e processados – três tarefas que o código subjacente deve executar.

Por outro lado, um ataque DDoS (Distributed Denial-of-Service Attack, ou Ataque Distribuído de Negação de Serviço) é adequado ao modelo de update, pois você pode alterar as configurações do software de segurança, oferecer recursos adicionais ao sistema e reconfigurar a plataforma para dificultar mais um ataque desse tipo. Você também pode corrigir hacks de engenharia social com treinamento de usuário, modificação da interface de usuário e alterações nas configurações de segurança da aplicação. O tipo de ataque determina se um update é uma boa opção. Em muitos casos, você descobrirá que pode, no mínimo, reduzir a probabilidade de um ataque por meio de um update, mas poderá não ser capaz de corrigi-lo totalmente.

Lidando com downtime de aplicação

A maioria das pessoas não quer falar sobre downtime (período de inatividade) da aplicação, especialmente em relação a problemas de segurança. No instante em que uma aplicação se torna inativa, as pessoas começam a fazer perguntas, e elas naturalmente levam a suspeitas (verdadeiras ou não). Em algum momento, a pergunta passa a ser se você é capaz de continuar mantendo uma aplicação ativa em uma situação de ataque ou se você deve deixá-la offline para evitar os tipos de danos que voltarão para assombrá-lo mais tarde. Em alguns casos, o hacker não lhe dará uma

opção: o próprio hack faz a aplicação falhar de forma específica. A questão é que os hackers provocam downtime na aplicação, e é uma situação que nunca é agradável para a equipe de TI que permite que o downtime ocorra.

O downtime ocorre, apesar das propagandas que você vê sobre 99,999% de uptime de algumas empresas. É claro que o downtime sempre aparecerá em um artigo importante na imprensa especializada se sua empresa for de grande porte, deixando-a mais constrangida ainda. É interessante observar que os artigos normalmente contêm um motivo nebuloso (e provavelmente incorreto) para o downtime (veja <http://www.zdnet.com/article/microsoft-explains-roots-of-thisweeks-office-365-downtime/>). No entanto, às vezes o downtime e a perda subsequente de dados têm outras origens, por exemplo, a Mãe Natureza (veja <http://www.computerworld.com/article/2974260/data-center/mother-nature-teaches-google-a-lesson.html>). A questão é que todos sofrem downtime e ele ocorre por vários motivos.

Quando um ataque é do tipo que você começa a se preocupar com perda de dados, perda da integridade do sistema ou danos graves, é hora de deixar a aplicação, e talvez todo o sistema, offline. Fazer isso lhe dará tempo para realizar qualquer update necessário e tentar deixar o sistema online novamente – espera-se que com melhores resultados. Embora o downtime seja visto pela gerência como uma solução negativa para qualquer problema, às vezes é a melhor solução para evitar danos sérios que exigirão semanas ou meses para serem reparados (normalmente com mais downtime envolvido).

Determinando se a ameaça é uma emergência

Quando falamos de ameaças à segurança, é importante determinar se a ameaça é uma emergência. O capítulo 14 discute os critérios para uma emergência de ponto de vista geral de um update. Eis alguns pontos que você deve levar em conta, além das considerações gerais, quando trabalhar com problemas de ameaças à segurança:

- Qualquer ameaça de segurança dia zero (zero-day) que tenha ativamente seu sistema ou sua aplicação como alvo
- Qualquer ataque em progresso
- Ameaças significativas, que podem ser confirmadas, feitas diretamente à empresa ou a um de seus funcionários

- Ataques contra bibliotecas, APIs ou microsserviços dos quais sua aplicação dependa

Definindo um plano de update

O capítulo 14 define os requisitos para um plano de update. Quando trabalhar com um update que trate uma ameaça de segurança, você deve usar a equipe de resposta rápida para garantir que o update esteja concluído e testado em tempo hábil (veja a seção “Criando uma equipe para respostas rápidas”). Também é importante garantir que os mesmos tipos de problemas discutidos na seção “Definindo um plano de upgrade”, sejam considerados. Por exemplo, usar técnicas de programação ágeis deixará o processo mais rápido e reduzirá o tempo necessário para obter um bom update.

Solicitando updates de terceiros

Um ponto sobre o qual muitas equipes de desenvolvimento se esquecem na pressa de disponibilizar um update de segurança é a necessidade de tratar as dependências. Se o código dependente for bem robusto e você está usando a versão mais recente, talvez não seja necessário verificar se há um update imediatamente, mas você deve fazer com que o terceiro tome conhecimento dos possíveis problemas de segurança em seu produto e pedir que esses problemas sejam tratados. Do contrário, qualquer correção que você criar pode continuar não funcionando – o hacker simplesmente escolherá outra via de ataque.

CAPÍTULO 17

Proporcionando o treinamento necessário

Aplicações seguras, que seguem as melhores práticas e empregam todas as estratégias anti-hacking mais recentes, são presas fáceis de usuários que não estão cientes da necessidade de preservar a segurança. Ataques de engenharia social continuam sendo a maneira mais eficiente de invadir uma empresa. Os usuários não são ineptos e não é o caso de não terem a habilidade de tomar boas decisões – é o fato de que, muitas vezes, eles não têm o treinamento apropriado e não há um requisito para que eles preservem o conhecimento adquirido. Simplesmente treinar uma pessoa para realizar bem a tarefa não resultará em nada, a menos que esse conhecimento seja testado periodicamente, com repercussões para as falhas (ou, quem sabe, com recompensas para o sucesso).

No entanto, desenvolvedores, administradores, gerentes e os demais funcionários de sua empresa também devem receber treinamento. Sem um treinamento adequado, você não pode esperar que sua empresa tenha a mínima chance de manter os hackers afastados. Os desenvolvedores precisam ficar de olho em novas ameaças e corrigir as brechas de segurança, os administradores precisam permanecer vigilantes em relação a potenciais brechas e a gerência precisa estar ciente da necessidade de oferecer recursos para manter as aplicações seguras. Em suma, todos têm uma tarefa a fazer que exige algum nível de treinamento.

Este capítulo discute algumas das maneiras pelas quais você pode ajudar a manter o elemento humano de uma aplicação treinado para que a aplicação tenha um melhor desempenho e os hackers estejam menos inclinados a atacá-la. Não, um treinamento não resolverá todos os problemas, mas o treinamento

em conjunto com aplicações seguras, um software de suporte apropriado e bibliotecas, APIs e microsserviços bem projetados darão um motivo para os hackers pensarem duas vezes. Um hacker determinado sempre conseguirá quebrar sua segurança, mas se você puder dar um bom motivo para ele procurar um alvo com defesas menos eficientes, certamente vale a pena fazer isso.

Criando um treinamento específico

Este capítulo usa o termo *treinamento* em sentido genérico, pois a maioria dos problemas discutidos afeta igualmente todas as formas de treinamento. Você deve reservar um espaço para treinamento, independentemente da forma com que ele seja conduzido.

Algumas formas de treinamento são realmente genéricas. Por exemplo, todo grupo em sua empresa exige treinamento em procedimentos de segurança organizacionais. Não importa se o grupo é formado por usuários, desenvolvedores, administradores ou gerentes – todos precisam conhecer as políticas organizacionais para garantir que o processo de segurança seja conduzido conforme esperado.

No entanto, alguns tipos de treinamento são bem específicos. Os usuários podem exigir treinamento específico para entrada de dados. Nesse caso, você deve dimensionar a área de treinamento para que acomode um grupo menor, constituído apenas de usuários. Os desenvolvedores – um grupo menor ainda na maioria dos casos – podem exigir um treinamento especial para acesso a bibliotecas, APIs ou microsserviços. Talvez algumas dessas informações sejam sensíveis, portanto você precisará de uma área de treinamento menor, que ofereça o nível de privacidade necessário.

Criar o treinamento específico às necessidades do grupo ou dos grupos que o recebem é importante. Garantir que você atenda às necessidades de privacidade e de tamanho e às necessidades técnicas desses grupos é importante. Considere como o grupo aprenderá como parte de sua busca por um espaço em particular. Por exemplo, se o grupo exige o uso de um sistema de projeção, você precisará de uma sala com o equipamento apropriado e instalações elétricas suficientes para acomodar essas necessidades. Este livro não tem a ver com fazer apresentações, porém tentar discutir sobre segurança sem antes atender às necessidades dos indivíduos do grupo condenará sua apresentação a falhar.

Criando um plano de treinamento interno para segurança

Conforme o tamanho de sua empresa, um plano de treinamento interno pode

ser menos custoso e melhor que usar um instrutor ou uma escola terceirizada. A vantagem dessa abordagem é que você pode ensinar sobre segurança conforme ela se aplica à sua empresa, o que quer dizer que seus funcionários terão mais informações pertinentes em um período de tempo significativamente menor. A desvantagem é que é improvável que um instrutor interno tenha a habilidade, a formação acadêmica ou a experiência oferecidas por um instrutor profissional. As próximas seções discutem os requisitos para um plano de treinamento interno em segurança.



Como ocorre com muitas áreas de segurança, você deve considerar o risco como parte da equação quando optar por um treinamento interno em segurança. Sim, seu instrutor conhecerá todos os requisitos de segurança de sua empresa, mas a documentação oferecida pela empresa pode estar incompleta ou simplesmente incorreta. O risco é oferecer informações incorretas aos funcionários, mesmo quando essas informações estejam mais alinhadas com o que a empresa acredita estarem corretas. Usar uma opção de treinamento externo com frequência faz os funcionários discutirem as tendências mais recentes em segurança e pode fazê-los mudar o plano de segurança para promover processos melhores de segurança dentro da empresa.

Definindo os treinamentos necessários

Antes de embarcar em uma cruzada de treinamento em sua empresa, você deve ter uma ideia do tipo de treinamento que seus funcionários precisam. Todos na empresa exigem algum nível de treinamento, mesmo aqueles que já têm algum tipo de treinamento profissional. Criar uma lista dos tipos de treinamento necessários permitirá personalizar as sessões de treinamento e obter um retorno melhor para o seu investimento. Além disso, como os recursos para treinamento em uma empresa normalmente são bem limitados, você deve extrair o máximo deles. Com isso em mente, considere os seguintes problemas como parte de qualquer plano de treinamento:

- O nível atual de habilidades de seus funcionários
- Requisitos gerais de treinamento (adotados em todo o mercado)
- Requisitos de treinamento específicos de sua empresa (como o uso de formulários ou processos)
- Disponibilidade dos funcionários

- Disponibilidade dos instrutores
- Familiaridade do instrutor com as necessidades específicas dos funcionários
- Possíveis locais para treinamento

A menos que você crie um plano utilizável para atender às metas de treinamento, como resultado você terá funcionários que aprenderão pouco, instrutores que ficarão frustrados e um retorno quase nulo pelo tempo investido. É importante entender que um treinamento interno só oferece vantagem quando você personaliza o local e as metas de modo a tirar proveito do ambiente interno. Por exemplo, uma sala silenciosa, em que o instrutor possa se reunir com os funcionários que exijam um auxílio especial individual, em geral, produz resultados melhores que tentar realizar a tarefa em uma sala de aula barulhenta.

Definindo metas razoáveis

Um problema importante com um treinamento interno é que a gerência tende a definir metas não realistas e o instrutor não tem experiência para conduzir esses gerentes na direção correta. Como os funcionários podem não estar dispostos a participar, às vezes eles acham que o instrutor deve lhes dar crédito simplesmente por participarem das sessões de treinamento. A falta de metas claramente definidas faz com que seja impossível a alguém obter o que quer ou até mesmo saber o que atingiu. A falta de metas razoáveis, claramente definidas, muitas vezes faz com que até mesmo uma sessão de treinamento bem organizada falhe. Em suma, a definição de metas é uma parte essencial para que o treinamento funcione. Eis alguns pontos a serem considerados como parte da definição das metas de treinamento em sua empresa:

- Desenvolver um plano baseado no que a empresa realmente precisa em termos de treinamento, em vez de considerar o que seria interessante ter.
- Especificar metas que todos possam compreender, usando uma linguagem que todos entendam, sem depender de jargão.
- Garantir que as metas estejam de acordo com o que os funcionários são

capazes de aprender, e não de acordo com o que você acha que os funcionários deveriam aprender.

- Criar metas que usem o tempo disponível de modo eficiente e reservar um tempo extra para assuntos mais complicados, pois os funcionários, definitivamente, terão dúvidas ou simplesmente não entenderão o material.
- Reservar tempo para os funcionários que exijam treinamento individual ou que não sejam capazes de comparecer em um horário específico de treinamento.
- Usar informações de todos os stakeholders na área de treinamento: gerência, instrutor e funcionários – todos devem ter direito a opinar quanto àquilo que as sessões de treinamento oferecem.

Somente quando um plano de treinamento inclui metas razoáveis é que você pode esperar conseguir algo. É importante entender que você pode não alcançar todas as suas metas, portanto ter níveis na lista de metas (em que atender um nível específico ainda é considerado uma vitória para todos) é essencial. É sempre possível criar outra sessão de treinamento para lidar com as metas mais difíceis posteriormente – o importante é conseguir resultados que possam ser demonstrados durante cada sessão de treinamento para que você possa apontar para as metas quando a gerência perguntar sobre os resultados do treinamento.



Pode parecer que o instrutor deveria definir as metas do treinamento, mas esse não é o melhor caminho em muitos casos. O instrutor gastará tempo se preparando para treinar os funcionários em novas técnicas, portanto coordenar as sessões de treinamento imporá uma carga desnecessária ao instrutor. Em muitos casos, você pode fazer mais designando outro indivíduo para coordenar os esforços de todos. Dessa maneira, os stakeholders podem se concentrar nas áreas específicas com que estão preocupados.

Usando instrutores internos

Muitas empresas exigirão que um funcionário faça um treinamento necessário. Em muitos casos, usar um instrutor interno funciona de forma aceitável, mas algumas situações exigem o uso de um consultor ou de um instrutor profissional para obtenção de bons resultados. Há vantagens e

desvantagens em usar um instrutor interno. As vantagens são:

Custo

É muito mais barato usar um funcionário interno que contratar um instrutor profissional na maioria dos casos.

Familiaridade

Um instrutor interno já terá familiaridade com as políticas da empresa e com os funcionários que as implementam, portanto é mais fácil para um instrutor interno oferecer uma atenção personalizada.

Disponibilidade

Usar um instrutor interno quer dizer que é possível agendar o treinamento de acordo com as necessidades da empresa, e não conforme as necessidades do instrutor. Além disso, funcionários com dúvidas podem ter acesso ao instrutor conforme for necessário.

Conveniência

Na maioria dos casos, usar um instrutor interno é mais conveniente que encontrar alguém lá fora com o conhecimento necessário. Contratar alguém de fora efetivamente quer dizer conduzir entrevistas para alguém que você não planeja manter como funcionário.

Volume de conhecimento

Uma empresa já sabe qual é o nível de conhecimento, quais são as credenciais e as habilidades do instrutor interno. Alguns instrutores externos se apresentam de uma forma durante a entrevista e fazem algo diferente na prática – às vezes fazem menos que o esperado.

Para ter essas vantagens, a gerência deve trabalhar com o instrutor e os funcionários devem estar dispostos a participar do processo de treinamento. Mesmo em condições ideais, usar um instrutor interno apresenta ao menos algumas destas desvantagens:

Falta de respeito

Os funcionários já têm familiaridade com o instrutor como um colega de trabalho. O ambiente de sala de aula exige alguém que inspire respeito e os

outros funcionários podem se recusar a agir dessa forma.

Perda de tempo

Um instrutor interno focará sua atenção na tarefa de treinamento, e não em atender a assuntos normais ligados ao negócio. Você não pode esperar que ele faça ambas as tarefas. Isso quer dizer que você perderá efetivamente um funcionário antes e depois das sessões de treinamento.

Falta de habilidade

Em geral, qualquer instrutor interno não terá as mesmas habilidades de alguém que seja um instrutor em tempo integral. A exceção é alguém que, por acaso, tenha uma formação para ser um instrutor.

Falta de experiência

Mesmo que alguém tenha as habilidades e o treinamento necessários, o fato de não realizarem tarefas de treinamento em tempo integral quer dizer que eles não têm o conhecimento resultante da experiência que um instrutor em tempo integral terá.

Monitorando os resultados

O treinamento é um processo de monitoração e ajustes. Você não pode simplesmente despejar informações nas cabeças das pessoas e esperar que elas sejam retidas (ao menos, ainda não). O motivo pelo qual as escolas devem aplicar exames é ajudar o professor a entender as deficiências no currículo atual e contribuir para fazer ajustes. O mesmo vale para treinamentos corporativos. Para atingir os melhores resultados, você deve monitorar o esforço de treinamento e fazer ajustes para permitir acomodar as diferenças entre as pessoas e suas compreensões sobre o assunto. É claro que exames relacionados ao trabalho podem assumir muitas formas. Eis algumas ideias para monitorar a eficiência do treinamento:

Exames escritos

Usar um exame para medir a eficiência do treinamento funciona na sala de aula até certo ponto, e funciona também no ambiente corporativo. É claro que algumas pessoas sabem como manipular um exame conforme sua

vontade (sem realmente se tornar competente), enquanto outras pessoas simplesmente não se dão bem com provas (apesar de serem bem competentes), portanto essa não deve ser sua única forma de medir o sucesso.

Testes práticos

Criar um cenário de teste e fazer o funcionário demonstrar o que aprendeu no treinamento é outra maneira de verificar se os resultados são os desejados. Muitas vezes, as pessoas que são reprovadas nos exames escritos se saem muito melhor nos testes práticos. No entanto, o teste prático ainda tem os mesmos problemas dos exames escritos – algumas pessoas simplesmente se saem melhor com eles sem realmente ter conhecimento do material.

Fatores práticos

O treinamento deve gerar um efeito demonstrável na eficiência e na produtividade geral do local de trabalho. Simplesmente observar como o treinamento afeta os negócios pode provar que o treinamento funciona. Por exemplo, uma queda significativa nos erros relacionados à segurança pode mostrar que o treinamento em segurança está atingindo a meta desejada.

Resultados monitorados

Observar o negócio como um todo pode não informar tudo que você precisa saber sobre a eficiência do treinamento. Às vezes, você terá que monitorar áreas específicas do negócio, por exemplo, menor incidência de ataques bem-sucedidos via email, para saber se o treinamento está funcionando conforme desejado.

Treinamento cruzado

Ensinar outra pessoa a executar uma tarefa corretamente é um método em que muitas empresas não pensam para avaliar o sucesso de uma situação de treinamento. Para treinar outra pessoa, o funcionário deve absorver bem o conhecimento a ponto de usá-lo com eficiência. Um cenário de treinamento cruzado mostra que um funcionário realmente entendeu o material e é capaz de explicá-lo para que outra pessoa entenda.



Um problema sério durante o processo de monitoração é que as pessoas se sentem ameaçadas e podem se tornar improdutivas. Para começar, parece que todos sentem essa necessidade de culpar alguém ou algo por uma falha em atingir uma meta específica de treinamento. No entanto, a falha simplesmente aponta para a necessidade de treinamento adicional, e não para a necessidade de culpar alguém (veja o artigo “Defining the Benefits of Failure” (Definindo os benefícios das falhas, <http://blog.johnmuellerbooks.com/2013/04/26/defining-the-benefits-of-failure/>). Uma falha pode ocorrer por outros motivos que não um erro da parte de alguém. Desperdiçar tempo no jogo de apontar culpados é improdutivo. É muito melhor encarar o fato de que uma falha ocorreu e oferecer o treinamento que remediará a situação.

À medida que monitorar os resultados do treinamento, você poderá marcar os itens em que os funcionários demonstraram um conhecimento adequado. Para os pontos em que os funcionários falharam em atingir as metas do treinamento, você poderá recriar a meta e experimentar alguma abordagem diferente para apresentar o material. Repetir a mesma abordagem no treinamento geralmente terá o mesmo resultado – a capacidade de criar vários métodos para aprendizagem do mesmo material é essencial para garantir que os funcionários realmente o absorverão.

Quando o treinamento estiver em curso, você poderá aproveitar a oportunidade para anotar as metas aprendidas e gerar novas metas com base no conhecimento acumulado pelos funcionários. É importante não sobrecarregar ninguém criando listas enormes que fazem a tarefa de aprender o novo material parecer inatingível. O que você deve fazer é definir passos menores com sucessos frequentes para que os funcionários possam manter uma atitude positiva em relação ao treinamento.

A meta geral de qualquer cenário de treinamento é garantir que sua aplicação e os dados associados a ela permaneçam seguros. Conforme mencionamos, a única maneira de conseguir um ambiente seguro em qualquer empresa é conquistar o apoio dos funcionários para isso. O treinamento é parte do processo de conquistar a confiança dos funcionários na aplicação e garantir que todos entendam o papel desempenhado pela segurança em melhorar o ambiente de trabalho. A única maneira de conseguir esse tipo de resultado é manter um ambiente favorável.

Obtendo treinamento de terceiros para os desenvolvedores

Um treinamento de terceiros normalmente promete informações genéricas de segurança e funciona bem para a maioria das necessidades da empresa. É claro que você pagará mais por esse treinamento genérico do que pagaria se usasse funcionários internos, portanto obterá informações que são menos pertinentes à sua empresa a um custo maior. No entanto, o treinamento recebido pelos funcionários será mais profissional e atualizado em relação àquilo que um instrutor interno pode oferecer na maioria dos casos. Você ganha na qualidade do treinamento.

Nem todos os treinamentos de terceiros são semelhantes. Você pode obter treinamento de terceiros de várias maneiras diferentes – cada uma com suas vantagens e desvantagens. A lista a seguir sintetiza as opções mais comuns de treinamento oferecidas por terceiros:

Treinamento de segurança feito internamente

Um consultor de segurança com experiência em treinamento vem até sua empresa e oferece seus serviços em uma área que você lhe disponibiliza. Você pode fornecer dados ao instrutor sobre os tipos de informações que seus funcionários precisam. Como resultado, você pode personalizar o treinamento até certo ponto e obter melhor equilíbrio entre informações genéricas e específicas da empresa. Além disso, seus funcionários aprendem no ambiente em que trabalham e usam o equipamento que normalmente utilizam. Uma desvantagem dessa opção é que você precisa disponibilizar uma área para o instrutor trabalhar, espaçosa o suficiente para acomodar os funcionários que você quer que sejam treinados. Além do mais, essa é uma opção que tende a ser mais cara.

Cursos online

Quando trabalhar com uma escola que ofereça cursos online, os funcionários normalmente podem seguir seu próprio ritmo, o que pode significar um nível melhor de treinamento. No entanto, os funcionários devem ter motivação própria para que essa opção funcione bem. A vantagem dessa opção de treinamento é que ela custa significativamente

menos que a maioria das outras opções e oferece um treinamento de alta qualidade na maioria dos casos. A desvantagem é que os funcionários não estão aprendendo em um ambiente de sala de aula e podem achar difícil que suas perguntas sejam respondidas em alguns casos. Você também poderá descobrir que o material do curso é um pouco mais limitado em relação às demais opções.

Centros de treinamento

Um centro de treinamento oferece um ambiente de sala de aula especializado, com instrutores que não fazem mais nada o dia todo além de dar aulas sobre segurança. Consequentemente, essa opção oferece o melhor treinamento genérico e os funcionários obtêm o melhor nível de interação possível com o instrutor. Os centros de treinamento normalmente mantêm turmas pequenas para que o instrutor possa dedicar um tempo pessoal a cada aluno e motivá-los a fazerem o melhor que puderem. Essa tende a ser uma opção de treinamento moderadamente cara, e você poderá descobrir que terá custos adicionais para os funcionários que frequentarem as aulas em um centro de treinamento que esteja em um lugar um pouco mais distante.

Faculdades e universidades

Na maioria dos casos, os alunos têm o mesmo nível de treinamento obtido se frequentassem a instituição para conseguir um diploma. As classes podem ter mais pessoas, o instrutor pode estar sobrecarregado e o material pode estar desatualizado. Em alguns casos, essa opção é gratuita ou tem um custo mínimo, com exceção dos materiais exigidos em sala de aula. Uma desvantagem dessa abordagem é que a instituição oferece aulas em horários que são mais convenientes para ela, e não para a sua empresa, portanto você poderá acabar perdendo funcionários quando eles fizerem os treinamentos.

Talvez você não ache uma opção perfeita com terceiros, mas normalmente poderá encontrar uma que seja suficientemente adequada. As metas, nesse caso, consistem em encontrar o nível de treinamento necessário (ou chegar o mais próximo possível disso) pelo preço que você puder pagar. As próximas seções oferecem informações adicionais sobre como instrutores terceirizados

podem ajudar seus funcionários a obter o tipo de treinamento de que eles precisam.



Alguns níveis de treinamento oferecem um certificado que um instrutor interno não pode conceder. O certificado pode não ter importância para seus negócios ou pode ser algo que você poderá exibir a possíveis clientes para mostrar que seus funcionários receberam o treinamento necessário em segurança. Ter o certificado deixa você mais atraente para alguns clientes e pode ser bom para seus negócios – possibilitando recuperar parte do dinheiro extra gasto com as necessidades de treinamento.

Especificando os requisitos de treinamento

Quando trabalhar com terceiros, você ainda deverá executar as tarefas descritas nas seções “Definindo os treinamentos necessários”, e “Definindo metas razoáveis”. No entanto, você deve agora realizar essas tarefas para alguém que não sabe nada sobre a cultura e as necessidades de sua empresa. Para obter os requisitos de treinamento necessários, você deve fornecer um esquema para a pessoa ou empresa que está oferecendo o serviço de treinamento. Qualquer funcionário envolvido em procurar, consultar e interagir com um instrutor terceirizado deve compreender totalmente os requisitos de treinamento e comunicá-los claramente ao terceirizado. Caso contrário, o treinamento que seus funcionários receberão não será adequado e você continuará a ter problemas de segurança, os quais poderiam ser evitados com um treinamento adequado. A lista a seguir descreve alguns aspectos que você deve considerar em qualquer conversa com um instrutor terceirizado:

- Discutir os horários de treinamento para garantir que sejam adequados à sua empresa
- Especificar o local do treinamento
- Garantir que o instrutor realmente oferece os serviços necessários
- Verificar se o instrutor tem as credenciais apropriadas
- Criar uma lista das necessidades do instrutor para que seja possível obter um resultado desejável
- Obter uma lista de qualquer equipamento especial necessário

O valor de instrutores terceirizados e de empresas de treinamento varia



enormemente. Certifique-se de registrar qualquer serviço prometido por escrito e que o serviço de treinamento obtido tenha um registro que prove a sua execução. Se possível, reserve tempo para conversar com clientes anteriores e descobrir se houve algum problema em potencial. É claro que as pessoas que o instrutor recomendar terão mais coisas positivas a dizer, pois, provavelmente, tiveram uma boa experiência. Mesmo assim, você poderá obter informações suficientes para tomar uma boa decisão se for cuidadoso ao falar com clientes anteriores. Como ocorre com qualquer outra transação comercial, é importante ser cauteloso ao contratar um terceiro para fornecer treinamento em sua empresa.

Contratando um instrutor terceirizado para sua empresa

Quando contratar um instrutor terceirizado para dar aulas no local, é preciso agendar várias reuniões para garantir que ele compreenda o que você quer e verificar se você tem tudo de que o instrutor precisa para fazer um bom trabalho. O primeiro dia em que o instrutor estiver no local para ensinar não deve ser um terrível desperdício de tempo para todos. Fazer sua lição de casa reduzirá os problemas do primeiro dia para que todos comecem com o pé direito. Eis alguns pontos que devem ser considerados para discussão com um instrutor terceirizado:

- Discutir problemas de segurança específicos que sua empresa tem e oferecer demonstrações de como esses problemas ocorrem na empresa. É essencial que o instrutor saiba exatamente quais áreas de segurança devem ser visadas e enfatizadas durante as sessões de treinamento.
- Garantir que o instrutor compreenda a história por trás de qualquer problema relacionado à segurança e saiba o que você já fez antes para atenuá-los. Esse passo garante que o instrutor não desperdiçará tempo tentando fazer algo que você já experimentou ou descartou.
- Abordar quaisquer restrições em potencial ao treinamento com o instrutor para garantir que os funcionários não se sintam desconfortáveis recebendo instruções que sejam contrárias à política da empresa.
- Verificar se o lugar físico selecionado para o treinamento atenderá às necessidades do instrutor. Além disso, verifique questões como o tipo de equipamento de que o instrutor precisa e quais equipamentos ele

providenciará.

Como ocorre com qualquer outra pessoa que visitar sua empresa, você deve garantir que o instrutor seja bem-vindo, mas não tenha acesso livre a áreas ou a informações sensíveis. Um instrutor não está acima dos requisitos de segurança de sua empresa. De fato, ele poderia estar no negócio especificamente para perpetrar ataques de engenharia social.

Sempre trate seu instrutor terceirizado com respeito, mas garanta também que ele receberá o nível apropriado de monitoração para poder manter sua empresa segura.

Usando cursos online

Cursos online oferecem pouco no que diz respeito à personalização e o currículo que você vir é o que terá. Garanta que a escola que disponibiliza os cursos online ofereça um bom acesso ao instrutor por algum meio assíncrono (por exemplo, email). Você deve se certificar de que seus funcionários possam entrar em contato com o instrutor quando for conveniente para ambos. Também é importante perceber que a comunicação em cursos online tem limites, portanto talvez você precise de um funcionário experiente de plantão para ajudar.

Conforme mencionado, a vantagem de usar um curso online é que todos podem prosseguir com o treinamento em um ritmo confortável. No entanto, considerando como é a natureza humana, muitas pessoas procrastinarão e se matarão para estudar para as provas na última hora. Essa forma de treinamento exige suporte adicional da gerência e monitoração por parte de um funcionário experiente. Você deve ver uma progressão clara no conhecimento dos funcionários à medida que o treinamento prosseguir. Caso contrário, você precisará perguntar se os funcionários estão participando totalmente do programa.

É importante perceber que nem todos conseguem aprender online. As pessoas têm todo tipo de maneira de aprender. De fato, algumas pessoas não aprendem bem em um ambiente de sala de aula – elas precisam de algum tipo de treinamento prático ou podem precisar ver alguém realizando uma tarefa. Você poderá descobrir que alguns funcionários simplesmente não são capazes de usar essa opção e precisarão contar com um instrutor interno ou com outro método



para obter o treinamento necessário. O problema não é o funcionário não tentar, mas simplesmente uma questão de habilidade para aprender usando uma abordagem específica.

Contando com centros de treinamento

Centros de treinamento podem oferecer o melhor tipo de treinamento, desde que você tenha funcionários dispostos a fazê-lo. O principal aspecto a lembrar, nesse caso, é que o treinamento que os funcionários receberem não só é caro, mas é também genérico. Os centros de treinamento ganham dinheiro oferecendo treinamentos intensivos com técnicas predefinidas. O treinamento é realmente bom, mas os funcionários devem estar dispostos a participar totalmente do processo e então estudar em seu tempo livre (após as aulas).

Para que essa opção funcione, você precisa, essencialmente, abrir mão de conseguir que o funcionário faça algo durante o período de treinamento. Interrupções só distrairão o funcionário e reduzirão a eficiência do treinamento (e considerando os preços cobrados por esses lugares, as interrupções realmente serão bem caras). Como o período de treinamento e o lugar não são negociáveis, talvez você precise compensar o funcionário de várias maneiras para garantir que o treinamento ocorra conforme previsto.

Quando o funcionário retornar do treinamento, lembre-se de testar completamente o novo conhecimento adquirido. Além disso, muitas vezes é conveniente que o funcionário faça uma apresentação das novas habilidades adquiridas. Usar essa abordagem ajuda a reforçar o conhecimento que o funcionário adquiriu e a ensinar novas técnicas a outros funcionários. A apresentação se enquadra na categoria de treinamento cruzado descrita na seção “Monitorando os resultados”. Você pode usar outras técnicas daquela seção para garantir que o dinheiro pago ao centro de treinamento valha a pena.

Usando faculdades e universidades locais

Nem toda empresa tem tempo, recursos ou fundos para usar qualquer uma das outras técnicas de treinamento descritas nesta seção. Seus funcionários ainda precisam de treinamento. Mesmo que você não tenha ninguém capaz de oferecer um treinamento interno e não possa pagar por nenhuma das demais opções listadas neste capítulo, ainda há uma boa chance de poder pagar pelos serviços de uma faculdade ou de uma universidade local. Dependendo de como a educação nessas instituições é tratada no lugar em que você mora, talvez seja preciso pagar um valor extra para custear o treinamento do funcionário, mas você provavelmente verá que é muito mais barato que outras opções.

Faculdades e universidades não ensinam nada rapidamente. O funcionário aprenderá no mesmo ritmo que todos os demais, portanto essa opção não oferecerá o tipo de treinamento necessário a curto prazo. Um funcionário que usar esta opção pode acabar afastado por um semestre ou um ano escolar inteiro a fim de obter as informações necessárias. Consequentemente, você deve fazer um planejamento prévio para usar essa opção.

O treinamento normalmente é um pouco mais flexível que frequentar um centro de treinamento. Por exemplo, o funcionário pode descobrir que as aulas ocorrem várias vezes durante o dia e à noite também. Mesmo assim, com frequência você achará que o horário não é tão flexível quanto outras opções de treinamento mencionadas neste capítulo – a instituição definitivamente atenderá às suas próprias necessidades antes de atender às suas.

Como no caso dos centros de treinamento, certifique-se de que o funcionário demonstre o conhecimento adquirido e faça apresentações para treinamento cruzado de outros funcionários. Contudo, por causa do período de tempo para o treinamento ser tão longo, normalmente você poderá realizar essas tarefas em etapas. Além disso, um período mais longo de treinamento significa que o funcionário não precisará se apressar tanto para adquirir as novas habilidades e há uma boa chance de que ele reterá mais informações, pois haverá mais tempo para absorver o material entre as sessões de treinamento.

Garantindo que os usuários estejam cientes da segurança

Todos precisam de algum nível de treinamento para garantir que a segurança seja boa em sua empresa. No entanto, os usuários muitas vezes exigem treinamento especializado porque não têm o conhecimento que sua equipe de desenvolvimento adquiriu durante a certificação ou quando estavam estudando para conseguir uma graduação na universidade. Além do mais, os usuários não têm a experiência que a equipe de desenvolvimento e outros funcionários adquiriram ao longo de anos trabalhando com problemas de segurança. Desse modo, você deve fazer alguns planos adicionais quando se trata de treinamento de usuários para criar um ambiente seguro. As próximas seções apresentam algumas ideias sobre como deixar seus usuários mais cientes da segurança.

Criando treinamentos específicos para segurança

Um problema com boa parte do treinamento de segurança em uso atualmente é o fato de não ser específico. Você deve criar um treinamento de segurança que seja específico para sua empresa, isto é, que reflita as políticas organizacionais. Do contrário, não poderá esperar que os funcionários sigam essas políticas quando executarem tarefas cotidianas, e o resultado, com frequência, será uma brecha de segurança que a empresa poderia ter evitado se seguisse as políticas.

O treinamento também deve considerar qualquer requisito legal ou outros requisitos. Nem todo negócio tem os mesmos requisitos de segurança. Para que os funcionários saibam como executar tarefas de forma segura em sua empresa em particular, você precisará deixá-los cientes de todos os requisitos legais e de outros requisitos que afetem sua empresa. É importante que os funcionários não só entendam os requisitos legais e os demais requisitos, mas também compreendam como sua empresa atende a esses requisitos.

Também é importante focar nas áreas em particular de sua empresa que apresentam problemas. Utilize exemplos reais de problemas de segurança (com nomes, datas e outras particularidades removidas para proteger os inocentes). Ao focar nas necessidades específicas de sua empresa, você deixará o treinamento mais pertinente e ajudará a garantir que os funcionários

vejam os benefícios práticos de seguir qualquer requisito definido pelo treinamento. Acima de tudo, ajude os funcionários a entender o custo da brecha de segurança para eles pessoalmente como parte do treinamento. É essencial que os funcionários compreendam o papel que eles desempenham ao seguirem as diretrizes organizacionais quando se trata de segurança.

Combinando treinamento com orientações por escrito

Ninguém se lembrará de todas as palavras ditas durante uma sessão de treinamento. Testes de vários tipos ajudam a memorizar, mas eles também têm seus limites. Além de treinamento, seus funcionários precisam de orientações por escrito que reforcem o treinamento. De fato, você deve usar essas orientações por escrito como parte de qualquer treinamento interno que fizer. Certifique-se de que as orientações por escrito sempre reflitam o treinamento recebido pelas pessoas. À medida que atualizar o treinamento, atualize também as orientações por escrito.



Sempre que fizer uma atualização, certifique-se de que todos recebam uma cópia das orientações atualizadas. Além disso, não deixe que funcionários mais antigos deixem de fazer o treinamento. Qualquer atualização no treinamento deve incluir um treinamento atualizado para funcionários antigos também. Alguém que já está há muito tempo na empresa pode ficar desatualizado quanto aos protocolos atuais de segurança e causar uma brecha de segurança inadvertidamente por falta de informação.

Não foque em deixar suas orientações por escrito cheias de pompa e floreios. Você precisa é de um guia simples e prático para as políticas da empresa. Os funcionários não têm nem tempo nem paciência para lidar com um guia cheio de termos legais ou jargões. Um guia que apresente as orientações de forma simples e concisa funciona bem melhor. Lembre-se de que a maioria das pessoas atualmente consegue focar a atenção por bem menos de um minuto, portanto se seus funcionários não conseguirem encontrar nenhuma resposta no guia que você escrever em menos de um minuto, eles não o usarão.

Resuma ao máximo as orientações por escrito. Quanto menos inchada a linguagem usada para transmitir informações importantes sobre segurança, mais as pessoas prestarão atenção a ela. Deixe tudo conciso e fácil de lembrar. Além disso, lembre-se de gerar o guia em formato tanto impresso

quanto eletrônico. Fazer isso permite que um funcionário coloque o guia em um dispositivo alternativo, por exemplo, em um smartphone, para mantê-lo sempre disponível.

Criando e usando lembretes alternativos para segurança

Os usuários se prendem a usar uma aplicação e ocasionalmente não pensam em segurança, a menos que você forneça um lembrete apropriado. Por exemplo, uma empresa poderia criar pôsteres com lembretes de segurança e colocá-los em lugares públicos. A técnica se parece com algo saído de um filme antigo ruim, mas, na realidade, funciona. Mostrar a necessidade de garantir a segurança diante do usuário, mesmo durante os intervalos, é uma forma de manter a percepção de que a segurança é importante.

A aplicação também deve incluir lembretes de segurança. Você não deve fazer com que sejam irritantes. O lembrete que diz: “Você tem certeza que quer fazer isso?”, e, em seguida, exibe outro lembrete dizendo: “Tem certeza mesmo?”, é simplesmente irritante e os usuários vão ignorá-los. Uma maneira eficaz de apresentar lembretes de segurança é oferecer uma dica do dia como parte da inicialização da aplicação. Nem todas as dicas precisam ter a segurança como foco. Às vezes, você pode acrescentar uma dica divertida no conjunto. A questão é fazer os usuários adivinharem o conteúdo da dica para que eles continuem a prestar atenção nela.

A equipe de TI também deve receber treinamento em relacionamento humano. Um dos maiores problemas em conquistar a atenção e a cooperação dos funcionários é que a área de TI tende a fazer com que todos se sintam como idiotas. Se você diz a alguém com frequência que ela é idiota, ela tenderá a acreditar em você e começará a agir dessa forma. Reforço positivo e uma boa atitude para ajudar outros funcionários realmente proporcionará melhores resultados que lhes dizer constantemente que eles não são capazes de entender os problemas de segurança.

Uma empresa teve um administrador de segurança criativo que, às vezes, organizava competições como “A primeira pessoa que conseguir localizar o princípio de segurança correto a ser aplicado em relação a dar sua senha às pessoas ganhará uma pizza no almoço”. As competições não eram frequentes e não custavam quase nada à gerência, mas mantinham o funcionário alerta para as



dicas de segurança e melhoraram bastante a segurança da empresa. Criatividade em fazer as pessoas continuarem a aprender mais sobre segurança é uma parte essencial para conseguir que elas cooperem para manter a aplicação e os dados seguros.

Verificando se os treinamentos são eficientes

Você poderá descobrir que as várias técnicas que experimentou para deixar os funcionários entusiasmados com segurança simplesmente não estão funcionando tão bem quanto planejado. É claro que o problema é descobrir como mudar a situação para que a segurança melhore. O primeiro ponto em que você deve pensar é que a segurança jamais será à prova de balas – tudo que você pode fazer é continuar a melhorá-la o máximo possível e lidar com cada nova ameaça à medida que ela surgir. No entanto, você pode se aproximar da perfeição. Parte de se aproximar da marca da perfeição é verificar a eficiência dos vários métodos usados. Obtenha estatísticas sobre o que está e o que não está funcionando usando as diversas técnicas descritas ao longo do livro (especialmente aquelas que estão na seção “Possibilitando feedback de usuários”, do capítulo 15). O que você deve verificar são as origens de possíveis falhas de segurança.

Esses pontos em potencial para falhas de segurança informam mais sobre como seu treinamento está funcionando. Você também pode testar os funcionários aleatoriamente usando as técnicas apresentadas anteriormente no capítulo. A questão não é localizar funcionários que estejam deixando a desejar no trabalho, mas identificar locais em que as estratégias atuais de treinamento não estão apresentando os resultados visados. À medida que encontrar essas áreas problemáticas, você deve considerar de que modo poderá passar as informações necessárias de maneira mais adequada, como proporcionar motivação para que as regras sejam seguidas e garantir que penalidades para falhas propositais em seguir procedimentos sejam implementadas. No entanto, o foco é sempre melhorar o treinamento para garantir que as pessoas realmente compreendam o que é necessário.

Sobre o autor

John Mueller é autor freelance e revisor técnico. Ele tem a escrita no sangue e já escreveu 99 livros e mais de 600 artigos até hoje. Os assuntos variam de rede à inteligência artificial e de gerenciamento de banco de dados à pura programação. Alguns de seus livros atuais incluem uma obra sobre Python para iniciantes, Python para cientistas de dados e um livro sobre MATLAB. Também escreveu um kit Java para e-learning, um livro sobre desenvolvimento de HTML5 com JavaScript e outro sobre CSS3. Suas habilidades para revisão técnica já ajudaram mais de 60 autores a melhorar o conteúdo de seus manuscritos. John já fez serviços de revisão técnica para as revistas *Data Based Advisor* e *Coast Compute*. Não deixe de ler o blog de John em <http://blog.johnmuellerbooks.com/>.

Quando John não está trabalhando no computador, você pode encontrá-lo no jardim, cortando madeira ou apreciando a natureza em geral. John também gosta de produzir vinho, assar biscoitos e tricotar. Quando não está ocupado com mais nada, ele faz sabonetes de glicerina e velas, que são práticos para cestas de presente. Você pode entrar em contato com John na internet em john@johnmuellerbooks.com. John também está criando um site em <http://www.johnmuellerbooks.com/>. Sinta-se à vontade para dar uma olhada e fazer sugestões sobre como ele pode melhorá-lo.

Colofão

O animal na capa de *Segurança para desenvolvedores web* é um abutre-barbudo (*Gypaetus barbatus*), também conhecido como abutre-das-montanhas, abutre-dos-cordeiros e quebra-ossos. O abutre-barbudo é uma ave de rapina, alimenta-se predominantemente de carniça e vive em áreas

montanhosas do sudeste da Europa, no Cáucaso, na África, no subcontinente indiano e no Tibete.

Ser uma ave carniceira quer dizer que o abutre-barbudo é capaz de sobreviver com uma dieta composta de restos de animais mortos. Em vez de se alimentar da carne, porém, a ave prefere a medula óssea, que compõe de 85% a 90% de sua dieta. Ocasionalmente, os abutres-barbudos são vistos atacando animais vivos, e eles parecem ter uma preferência por tartarugas. Eles foram observados capturando tartarugas, levando-as a grandes altitudes e derrubando-as para quebrar os cascos e poder pegar o corpo macio abaixo deles.

Os abutres-barbudos podem ter aproximadamente entre 95 e 125 centímetros e pesam até pouco mais de oito quilos. As fêmeas são um pouco maiores que os machos e os dois membros de um casal cooperam para construir o ninho enorme (até um metro de largura). Suas asas longas e estreitas facilitam distingui-los de outras espécies de abutres. Os abutres-barbudos não têm a cabeça desprovida de penas como muitos outros tipos de abutre, e a espécie recebeu seu nome por causa das longas penas pretas em seu queixo.

Seu tamanho grande e a presença majestosa conferiram ao abutre-barbudo um lugar na mitologia de muitas sociedades. Para os iranianos, essa ave é um símbolo de sorte e de felicidade, e acreditava-se que estar à sua sombra poderia prever a ascensão de uma pessoa à realeza. Diz-se que o dramaturgo grego Ésquilo foi morto quando uma “águia” derrubou uma tartaruga em sua cabeça, confundindo sua calva com uma pedra. Dada a região, a época e o comportamento, o abutre-barbudo é um provável candidato para a “águia” da história. Também na Grécia, o abutre-barbudo foi uma das poucas espécies de ave que podiam ser usadas para prever o futuro – essa prática é conhecida como augúrio. Por fim, tanto na Bíblia quanto na Torá, o abutre-barbudo está entre as aves que as pessoas eram proibidas de comer.

Muitos dos animais na capa dos livros da O'Reilly estão ameaçados; todos eles são importantes para o mundo. Para saber mais sobre como você pode ajudar, acesse animals.oreilly.com.

INTRODUÇÃO À ANÁLISE FORENSE EM REDES DE COMPUTADORES

CONCEITOS, TÉCNICAS E FERRAMENTAS
PARA "GRAMPOS DIGITAIS"



novatec

Ricardo Kléber M. Galvão

Introdução à Análise Forense em Redes de Computadores

Galvão, Ricardo Kléber M.

9788575226940

152 páginas

[Compre agora e leia](#)

Este livro apresenta conceitos e ferramentas sugeridos para a prática de atividades de computação forense em redes de computadores, a partir da experiência de seu autor em atividades de ensino e pesquisa, eventos da área, suporte pericial e atuação nos mais diversos cenários envolvendo o tema. É um livro que pode ser lido e aplicado em sua totalidade ou parcialmente, conforme a conveniência, diante de cenários em que o tráfego em redes de computadores (ou parte dele) necessita ser capturado e analisado. Para isso são apresentados desde detalhes dos pacotes que trafegam nas redes de computadores e, em particular na internet e redes locais a ela ligadas, que utilizam, na sua maioria, a pilha de protocolos TCP/IP, passando por conceitos, técnicas e ferramentas para captura e análise detalhada dos dados de cabeçalho e carga (payload) destes pacotes. Além da fundamentação na área de redes de computadores, necessária à compreensão de parte das técnicas apresentadas, o livro apresenta uma série de ferramentas com detalhes sobre instalação, sintaxe e exemplos de utilização para a prática dos conceitos e aplicação nos mais diversos cenários relacionados à computação forense em redes. Destina-se a profissionais (peritos criminais e judiciais, juízes, advogados e promotores com atuação na área), estudantes e

professores da área de computação, além do público em geral interessado em assuntos relacionados a crimes cibernéticos com a utilização de redes de computadores.

[Compre agora e leia](#)

O'REILLY®

Padrões para Kubernetes

Elementos reutilizáveis no design de aplicações
nativas de nuvem



novatec

Bilgin Ibryam
Roland Huß

Padrões para Kubernetes

Ibryam, Bilgin

9788575228159

272 páginas

[Compre agora e leia](#)

O modo como os desenvolvedores projetam, desenvolvem e executam software mudou significativamente com a evolução dos microsserviços e dos contêineres. Essas arquiteturas modernas oferecem novas primitivas distribuídas que exigem um conjunto diferente de práticas, distinto daquele com o qual muitos desenvolvedores, líderes técnicos e arquitetos estão acostumados. Este guia apresenta padrões comuns e reutilizáveis, além de princípios para o design e a implementação de aplicações nativas de nuvem no Kubernetes. Cada padrão inclui uma descrição do problema e uma solução específica no Kubernetes. Todos os padrões acompanham e são demonstrados por exemplos concretos de código. Este livro é ideal para desenvolvedores e arquitetos que já tenham familiaridade com os conceitos básicos do Kubernetes, e que queiram aprender a solucionar desafios comuns no ambiente nativo de nuvem, usando padrões de projeto de uso comprovado. Você conhecerá as seguintes classes de padrões:

- Padrões básicos, que incluem princípios e práticas essenciais para desenvolver aplicações nativas de nuvem com base em contêineres.
- Padrões comportamentais, que exploram conceitos mais específicos para administrar contêineres e interações com a plataforma.
- Padrões estruturais, que ajudam você a organizar contêineres em um Pod para tratar casos de uso específicos.
- Padrões de

configuração, que oferecem insights sobre como tratar as configurações das aplicações no Kubernetes. • Padrões avançados, que incluem assuntos mais complexos, como operadores e escalabilidade automática (autoscaling).

[Compre agora e leia](#)

CANDLESTICK

Um método para ampliar lucros na Bolsa de Valores



novatec

Carlos Alberto Debastiani

Candlestick

Debastiani, Carlos Alberto

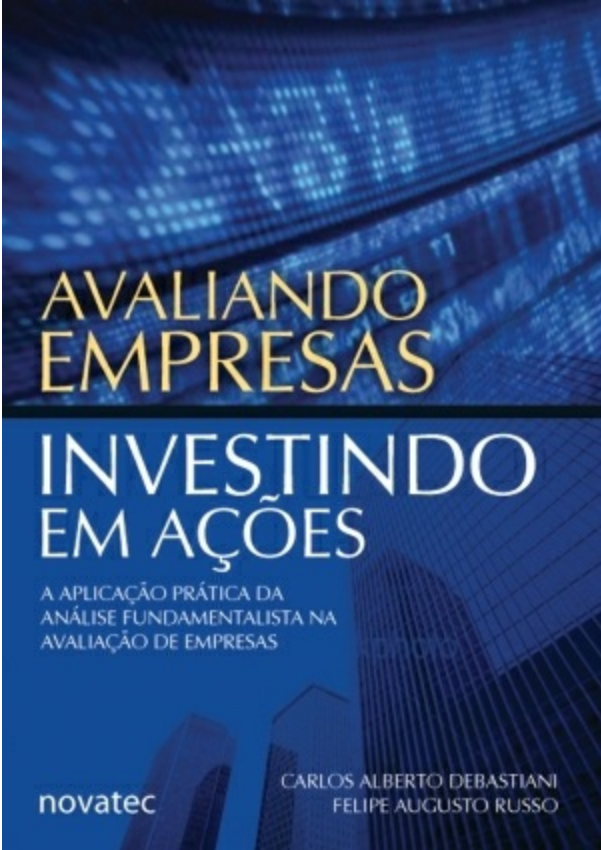
9788575225943

200 páginas

[Compre agora e leia](#)

A análise dos gráficos de Candlestick é uma técnica amplamente utilizada pelos operadores de bolsas de valores no mundo inteiro. De origem japonesa, este refinado método avalia o comportamento do mercado, sendo muito eficaz na previsão de mudanças em tendências, o que permite desvendar fatores psicológicos por trás dos gráficos, incrementando a lucratividade dos investimentos. Candlestick – Um método para ampliar lucros na Bolsa de Valores é uma obra bem estruturada e totalmente ilustrada. A preocupação do autor em utilizar uma linguagem clara e acessível a torna leve e de fácil assimilação, mesmo para leigos. Cada padrão de análise abordado possui um modelo com sua figura clássica, facilitando a identificação. Depois das características, das peculiaridades e dos fatores psicológicos do padrão, é apresentado o gráfico de um caso real aplicado a uma ação negociada na Bovespa. Este livro possui, ainda, um índice resumido dos padrões para pesquisa rápida na utilização cotidiana.

[Compre agora e leia](#)



AVALIANDO
EMPRESAS

INVESTINDO EM AÇÕES

A APLICAÇÃO PRÁTICA DA
ANÁLISE FUNDAMENTALISTA NA
AVALIAÇÃO DE EMPRESAS

novatec

CARLOS ALBERTO DEBASTIANI
FELIPE AUGUSTO RUSSO

Avaliando Empresas, Investindo em Ações

Debastiani, Carlos Alberto

9788575225974

224 páginas

[Compre agora e leia](#)

Avaliando Empresas, Investindo em Ações é um livro destinado a investidores que desejam conhecer, em detalhes, os métodos de análise que integram a linha de trabalho da escola fundamentalista, trazendo ao leitor, em linguagem clara e acessível, o conhecimento profundo dos elementos necessários a uma análise criteriosa da saúde financeira das empresas, envolvendo indicadores de balanço e de mercado, análise de liquidez e dos riscos pertinentes a fatores setoriais e conjunturas econômicas nacional e internacional. Por meio de exemplos práticos e ilustrações, os autores exercitam os conceitos teóricos abordados, desde os fundamentos básicos da economia até a formulação de estratégias para investimentos de longo prazo.

[Compre agora e leia](#)

Marcos Abe

MANUAL DE ANÁLISE TÉCNICA

ESSÊNCIA E ESTRATÉGIAS AVANÇADAS

TUDO O QUE UM INVESTIDOR PRECISA SABER PARA
PROSPERAR NA BOLSA DE VALORES ATÉ EM TEMPOS DE CRISE

novatec

Manual de Análise Técnica

Abe, Marcos

9788575227022

256 páginas

[Compre agora e leia](#)

Este livro aborda o tema Investimento em Ações de maneira inédita e tem o objetivo de ensinar os investidores a lucrarem nas mais diversas condições do mercado, inclusive em tempos de crise. Ensinará ao leitor que, para ganhar dinheiro, não importa se o mercado está em alta ou em baixa, mas sim saber como operar em cada situação. Com o Manual de Análise Técnica o leitor aprenderá: - os conceitos clássicos da Análise Técnica de forma diferenciada, de maneira que assimile não só os princípios, mas que desenvolva o raciocínio necessário para utilizar os gráficos como meio de interpretar os movimentos da massa de investidores do mercado; - identificar oportunidades para lucrar na bolsa de valores, a longo e curto prazo, até mesmo em mercados baixistas; um sistema de investimentos completo com estratégias para abrir, conduzir e fechar operações, de forma que seja possível maximizar lucros e minimizar prejuízos; - estruturar e proteger operações por meio do gerenciamento de capital. Destina-se a iniciantes na bolsa de valores e investidores que ainda não desenvolveram uma metodologia própria para operar lucrativamente.

[Compre agora e leia](#)